THOMSON

游戏编程精粹 6

GAME PROGRAMMING Gems 6

[美] Michael Dickheiser 编
孟宪武 等 译

人民邮电出版社
POSTS & TELECOM PRESS

内容提要

读者将在本书中找到来自 20 多个国家和地区具有不同背景和专长的游戏专家所撰写的 50 多篇文章。本书是游戏编程精粹系列书的最新版本，内容涉及通用编程、数学和物理、人工智能、脚本和数据驱动系统、图形学、音频音效、网络与多人在线游戏，以及游戏测试和手机游戏等内容，具有较强的先进性和实用性。随书附带光盘中提供了全书所有的源程序、演示程序及需要的各种游戏开发的第三方工具。

因此，无论你是一个刚刚起步的游戏开发新手，还是资深业界专家，都能够在本书中找到灵感，增强洞察力及开发的技能。应用书中介绍的开发经验和技巧于实际项目中，将缩短开发时间，提高效率。

Download at www.Pin5i.Com

序

Mark Deloura
madsax@satori.org

欢迎大家来到《游戏编程精粹 6》的精彩世界。游戏编程精粹系列书的第六部正是针对当前面临的挑战而精心设计的。随着游戏编程精粹系列丛书的不断出版，业界对游戏编程人员的需求也不断增长。随着游戏开发团队的规模不断扩张，游戏开发人员也逐步地意识到，在产业推动下，自己正变得越来越专业化。在逐渐专业化的过程中，有时读者也可能涉足一些自己专业领域之外的工作。这个时候，在案头准备一些唾手可得的尖端参考资料，就显得非常重要。我衷心地希望，当读者碰到上述这些情况时，可以选择我们这套游戏编程精粹系列丛书。

目前这一系列书已经出版到了第六部，让我们做一下简要回顾。前几部书为我们提供了一个非常有趣的历史视角。在 2000 年，当我们编撰这个系列书的第一部的时候，当时让业界感觉非常棘手的一些问题，到今天实质上已经不那么难缠了。我们今天所要面对的突出问题则是：实现高级的着色特效，在游戏设计中集成实时的物理特效，多人网络游戏的同步，寻找易于游戏美工和易于游戏策划使用的脚本语言。

但是，现在所面临的最大难题是成本问题。从 PlayStation 3、Xbox 360、Nintendo Revolution，到多核 PC，下一代游戏机产品已经接踵而至。伴随新的性能而来的是更多新的挑战。我们的玩家会期望更高精度的模型和动画，更真实的物理特效和图形图像特效，以及更具智能的游戏 AI。为了能够实现这些“玩家期盼”，我们要不断地扩大开发团队，延长项目时间表，最后还要花费大量的资金。如果我们无法去抬高游戏产品的销售价格，或者不能去扩大我们的目标市场，我们会比几年前更为窘迫，根本无法在预算之内，按时地完成游戏的制作，为玩家提供引人入胜的游戏体验。在这关键时刻，《游戏编程精粹 6》来了！

工具的重要性

作为游戏编程人员要做的一件非常重要的事情，就是让团队中其他成员可以很容易地使用我们开发的技术。不管怎么说，如果我们熬夜赶制出来的精美的着色程序让我们的美工无从下手，不知如何使用，这样的着色程序又有什么用处呢？开发团队规模确实不断扩张，但大多数的人员扩充

Download at www.Pin5i.Com

都源于游戏美工人员的增加。所以，如果我们能够开发出一些简单的工具，让他们的工作更得心应手，这样，游戏产品不但会看上去更赏心悦目，开发制作的进度也会更快速，成本也会更便宜。可谓皆大欢喜！

幸运的是，在这个平台过渡时期，工具的重用要比引擎技术的重用容易得多。工具软件的前端界面可以相对地保持不变，但它的底层和输出结果则可以适当地进行修改，以适应新的目标。这使得工具软件成为一个最容易分摊开发成本的东西，因为它们可以在很长一段时间内，广泛地在多个游戏产品和多个开发团队中进行重用。

在游戏开发过程的开始阶段，为前期制作和早期制作提供可用的工具软件，这对整个项目是非常有帮助的。从实践上看，为了在开发进度上节省时间，这样做也是必需的。但不幸的是，在目前这个阶段看到的情况是：在游戏中间件市场上，可用工具软件的数量呈下降的趋势。所以，如果读者到现在还没有自己的工具软件，该如何是好呢？越来越流行的情况是，若干个小型工作室联合成为一个大型的开发集团，或者干脆被发行商收购。对他们而言，比较合乎逻辑的选择是创建一个中央技术群组。

协作

试图说服很多的工作室一起协作，共享通用的技术，彼此互相依靠，恐怕我们中的大多数人都不会喜欢这种做法。但是，随着开发成本不断地呈螺旋式上升，在多个平台和游戏产品上来分散一项技术的开发成本，这的确是一个聪明的选择。坦白地说，近来，有一些大型项目已经变得过于昂贵，他们不但要在多个平台之间进行成本的分摊，还要把这些成本分摊到未来众多的后续产品系列中！由于开发成本如此之高，这种做法也许是他们唯一能够赚取利润的方法了。但是，如果不去探索如何建立一个强力的品牌，然后在一个特许经销系列的多个游戏中重用这些工具和技术，我们又该如何证明这笔庞大的投入是正确的呢？这看上去有些疯狂，但是，高风险同时也伴随着可能的高回报。

另外一种协作方式也正日渐重要，即全球化的游戏开发。有些公司已经将目光转向那些以前从未考虑的其他地区的工作室，像东欧、中国和印度。通过这种方式，欧洲、北美以及日本的发行商和开发商们又找到了一条新的途径，来控制他们自己的开发成本。生产商们也学会了为一款游戏而将全球多个地区的资源聚拢起来。在这种情况下，拥有可靠的开发工具就变得至关重要。

并行化

随着游戏机开发商们步入下一个系列的游戏机产品，他们也开始面临个新的挑战：如何将他们的游戏引擎并行化。对 PC 游戏开发商而言，他们的时间也所剩无几，必须马上投入到这个方向的研究上。即将到来的多核系统需要认真的研究和创造性的新技术，以便能够充分利用新系统的特性。虽然我们都已经习惯了一些并行架构（比如 CPU 和 GPU 之间的并行架构），但是，新的多核系统迫使我们必须去采用新的思维方式。我们该如何处理我们的游戏引擎，将它合理地分割成很多的更小的部分，以便可以很容易地进行并行化处理？我们是否可以仰仗编译器和其他的工具，使得多核系统的编程工作能变得更容易些？或者，我们不得

Download at www.Pin5i.Com

不自己动手，去寻找新的技巧和技术，以便将并行架构的性能最大化？对老练的程序人员而言，这肯定不是乏味的时期。

在硬件系统处理工作的复杂度不断增加的同时，也给我们带来了一个有趣的副作用，那就是：在一些大型的开发团体中，程序人员的角色出现了分化。现在，很多团队中都有一些专门针对不同系统平台的编程人员，他们负责游戏引擎和硬件系统之间的接口。而大多数的程序人员依然还是在负责编写独立于平台的代码。但是，这些工作之间的界限也不像听起来那么明确。由于各种平台之间的差异，底层编码人员的工作机会绝对是有保障的。这样一来，我们就不用要求高层编程人员也具备丰富的经验，当然也可以因此少一些工资支出。终归，这又是一个可以降低开发成本的方法。

致谢

游戏编程精粹系列丛书为我们提供一个有趣的视角来一览游戏工业近来的技术发展。虽然，业界还有一些其他的资源也在讨论技术问题，例如《游戏开发者》杂志、gamasutra.com网，以及《游戏开发》期刊等，但是没有一种可以像游戏编程精粹系列丛书这样深入、涉及如此众多的素材。在此，我们要感谢所有的作者和编辑人员，他们为这个系列丛书贡献了太多的时间和精力。如果你还没有为这个系列丛书贡献自己独特的观点，那么，在未来的日子里，请你考虑一下，将你的经验整理出来，与我们大家一起分享吧。

最后，我们还要向 Michael Dickheiser 表示衷心的感谢。是他不知疲倦，坚持不懈，为我们雕琢出了如此专业的《游戏编程精粹 6》一书。我想，你一定会发现，Mike 独特的观察角度为我们创造出了一部非常有价值的书籍。我衷心地希望，这本书能成为你的书架上的珍藏！

Download at www.Pin5i.Com

翻译和审校员

孟宪武 mxwbrio@hotmail.com

本书主审并翻译了第 1 章及第 2 章等内容。毕业于北京航空航天大学计算机系，目前供职于班布技术有限公司，之前曾在《中国计算机报》报社和《数字娱乐开发》杂志社工作。

罗岱 edl7878@hotmail.com

负责翻译第 3 章。毕业于北京工商大学计算机及应用专业，目前任北京林业大学数字艺术专业教师。

徐丹 flymemory@sina.com

翻译第 4 章及第 7 章。目前就职于北京百竹数码。长期从事游戏引擎的研究与开发工作。已编写《PC 游戏编程—窥门篇》和《PC 游戏编辑—基础篇》两本图书。曾参与《1937 特种兵》、《傲视三国 2》、《荣耀》等游戏的开发。目前负责开发一款大型网络游戏。

张浩 aladdina@gmail.com

负责翻译第 5 章的 5.1 至 5.5 节。毕业于华中科技大学计算机科学与技术专业，目前在 Autodesk 中国软件研发中心担任软件工程师。

史苏 amble_shisu@hotmail.com

负责翻译第 5 章的 5.6 至 5.10 节。毕业于同济大学计算机科学与技术专业，目前在 Autodesk 中国软件研发中心担任软件工程师。

史晓明 gpg6@cpper.com

负责翻译第 6 章的 6.1、6.4、6.5 和 6.6 节。澳大利亚 BigWorld Pty.Ltd 软件工程师。此前于上海育碧公司工作 7 年，曾参与《雷曼 II》、《细胞分裂》I/II/III 等游戏的开发工作。

吴盛强 Shqiang_Wu@hotmail.com

负责翻译第 6 章的 6.2 和 6.3 两节。毕业于同济大学海洋地质与地球物理专业，目前在盛大网络艺术中心音乐音效部从事游戏音频开发工作。

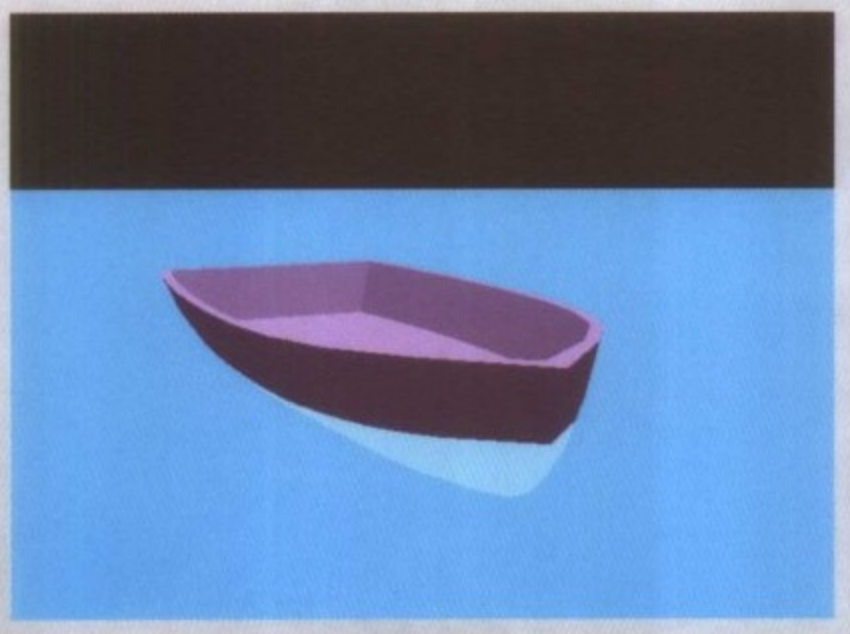
Download at www.Pin5i.Com

彩插1 （上）通过对玩家皮肤进行采样来建立直方图.（中）HSV色彩空间中的面部图像。（下）面部识别（见1.3节）

彩插2 玩家的替身向左倾斜以反映出玩家的运动（见1.3节）

Download at www.Pin5i.Com

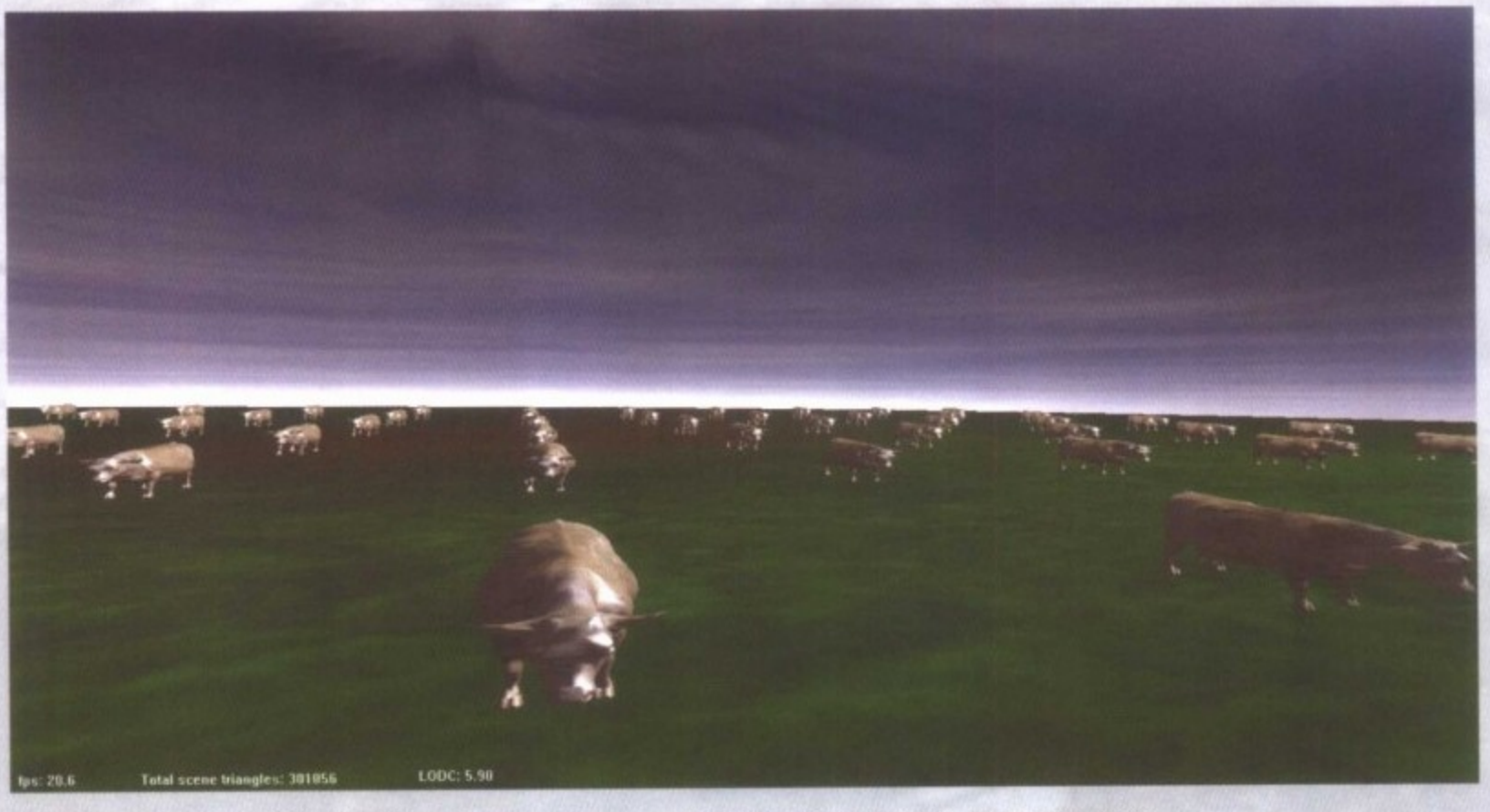
彩插3　玩家的替身回到正中以反映出玩家的运动（见1.3节）



彩插4　一个典型的小船受到的完美浮力

彩插5　任意的、凹体船浮在书中（见2.5节）

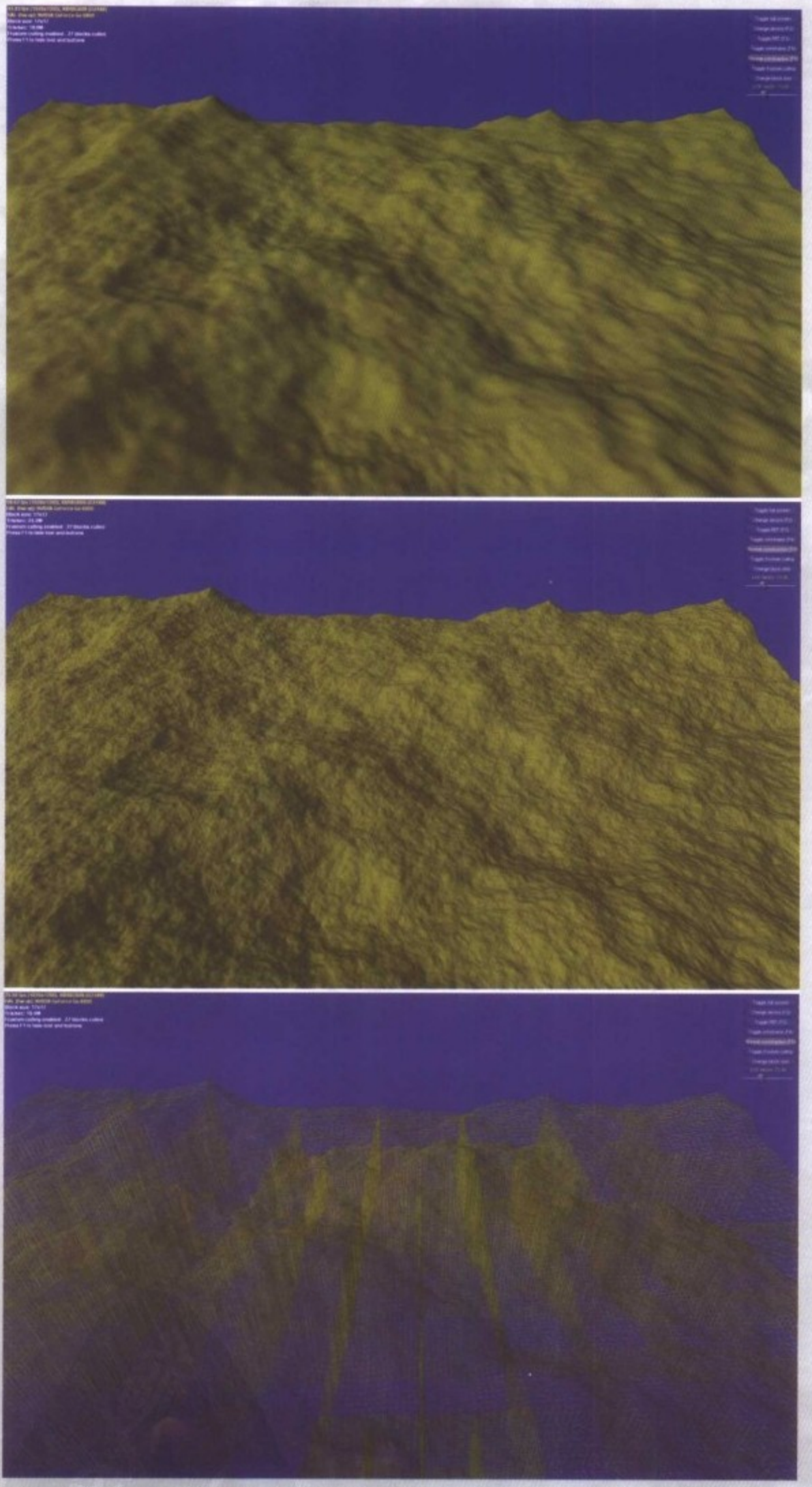
Download at www.Pin5i.Com



彩插6　第3.9节测试程序的一个屏幕截图：一种管理场景复杂性的模糊控制方法

彩插7　在Delta3D中处理角色的属性（见4.5节）

Download at www.Pin5i.Com



彩插8 在最上面的屏幕截图中，法线在vertex shader中计算，在中间的屏幕截图中，使用了normal map。注意线框模式的屏幕截图中垂直方向的"围裙"(见5.5节)

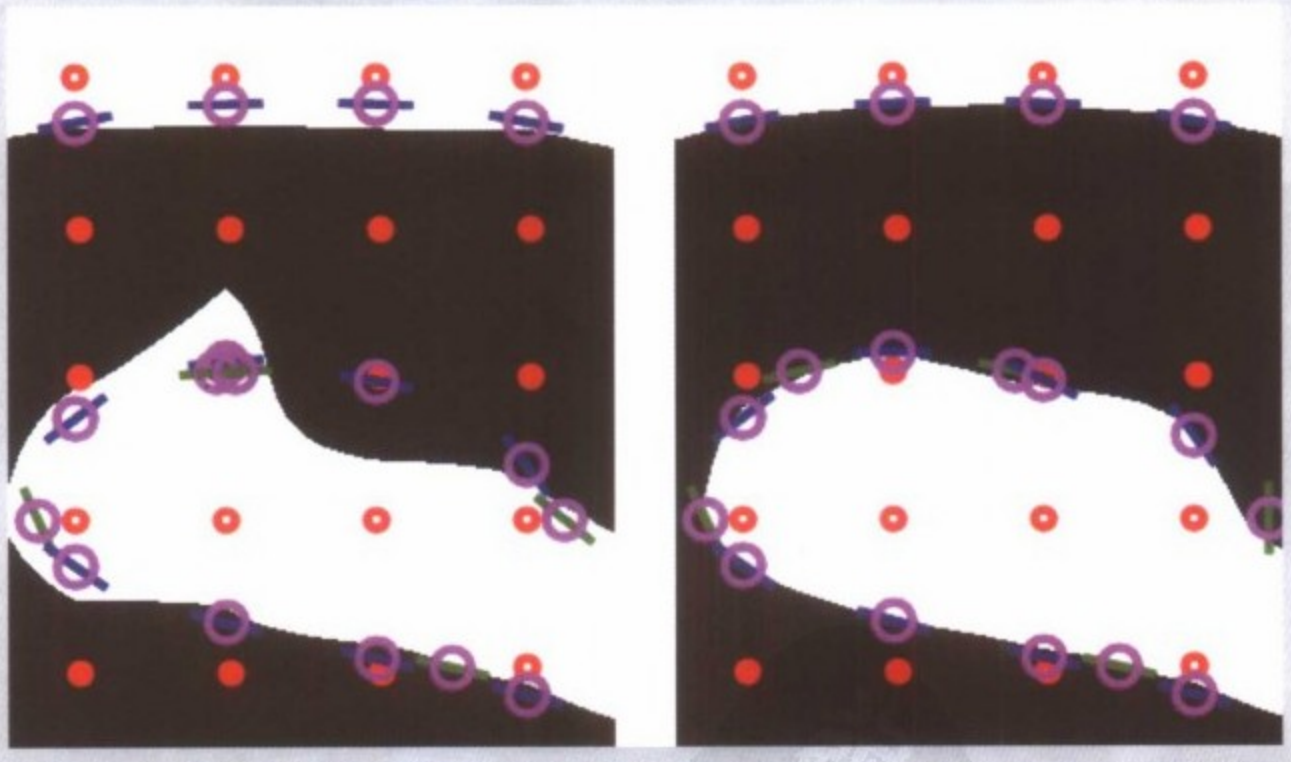
Download at www.Pin5i.Com

彩插9 池塘的水波由用户的交互输入产生，水波通过GPU来模拟（见5.6节）

彩插10 (a)使用不同的深度精度渲染的场景，仅显示了漫反射光照。(b)使用不同的深度精度渲染的场景，组合了漫反射和高光（见5.7节）

Download at www.Pin5i.Com

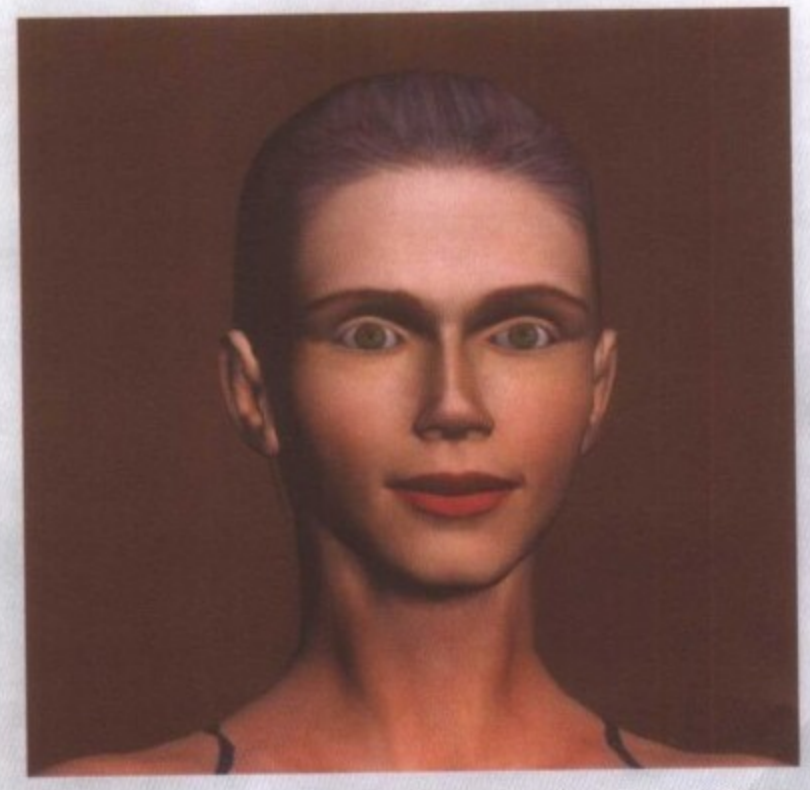
彩插11　使用双现行差值的标准纹理，产生了模糊的字体（上图　64x64像素）。本文提供的技术使用了一个很短的pixel　shader以及一个预处理的纹理来产生很锐利的纹理（下图，纹理尺寸和上图相同，见5.8节）



彩插12　用户可以编辑位置和角度约束来求解临界条件，比如确定切线方向的相交哪一个是最适合的（左边，不足；右边，过度）。约束的显示形式：位置以圆来表示，角度以线来表示

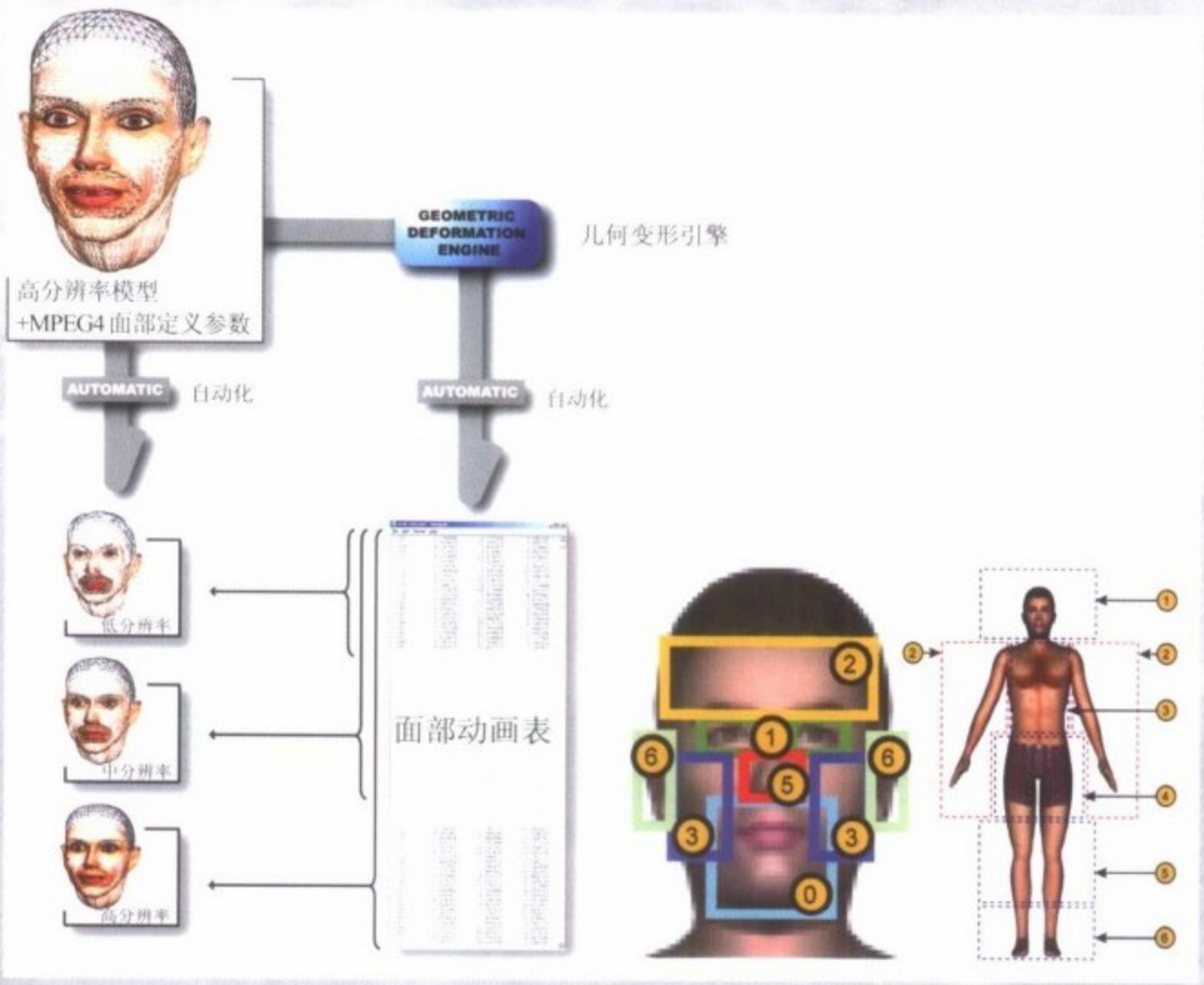
Download at www.Pin5i.Com

彩插13　使用多色图像（上图）会在多个颜色交汇的地方引起令人讨厌的错误，见放大图。把这样一类图像转化成单色图像的逆向工程能解决这个问题（下图，64x64像素，见5.8节）

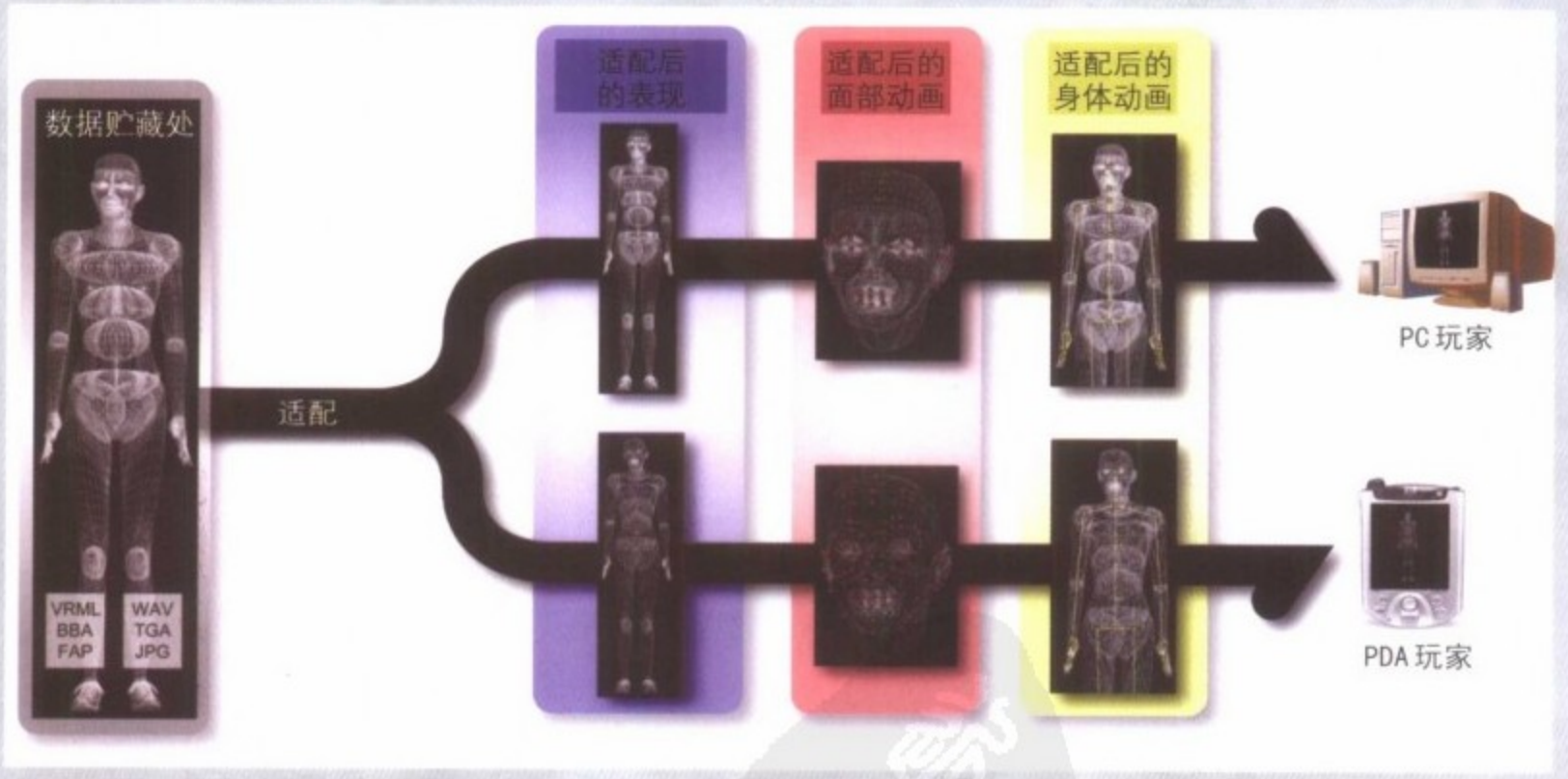


彩插14　一个包含了3D信息的高分辨率面部动画通过网络进行流式传输（见7.1节）

Download at www.Pin5i.Com



彩插15 左，可伸缩性的面部动画系统；右，适配器感兴趣的面部和身体部位（见7.1节）



彩插16 从保存资料的服务库到用户整个过程的数据流图（见7.1节）

Download at www.Pin5i.Com

前　言

Mike Dickheiser Applied Research Associates Inc
mdickheiser@ara.com

“游戏编程精粹”系列丛书已经不再是游戏开发人员的专利了。或者换种说法，它的存在不再只是为了游戏开发人员。在过去的几年中，我们看到了相关产业（包括基于游戏的学习、教育，以及其他的“严肃”游戏）的蓬勃发展。不可否认，这并不是什么新鲜事。而且，围绕着这些相关产业发展的议论也越来越热烈。但是，对于在去年前后发生的一些事情，我们只能用“入侵”一词来描述了。

发现一个新的世界

计算机游戏引擎和相关工具正不断地引起业界的关注。这种关注程度或许可以让大多数的游戏开发人员得以解脱。而在这之前，他们忙得不可开交，根本无暇将目光脱离那满是咖啡渍的键盘。虽然会有人离开这个行业去开拓其他领域，我个人可以百分之百地断定，有无数双的眼睛正在急切地关注着这个产业，看它到底给我们带来了些什么东西。新的公司如雨后春笋般地涌现出来。这些公司将全部精力专注于利用游戏技术来创建商业训练和军事训练环境和模拟器。他们一起出动，开始在这个新世界中圈地划界。

但是，并不是只有产业在迁徙。我们的努力也引起了学术界更多的关注。我这里所说的并不是那些计算机游戏学校和课程（在这些方面，它们也取得了令人兴奋的发展，因此也值得我们去专门进行探讨）。我这里所说的是：如何在那些异常错综复杂、前沿的研究领域中，使用正式授权的游戏技术。只要走马观花地审视一下各个主要大学中正在研究的一些项目，我们就会发现，人们正在使用最好的游戏引擎，把它们作为一个“开箱即用（out-of-the-box）”的工具，用于开发人机交互系统、认知建模、智能主体、交互式叙述、听觉病理学、运动技能疗法、人类信息处理……这个列表还会不断地加长。

最终的认可

有一件事情是很明确的：计算机游戏技术已经最终被认可为真正的技术。对于这一点，最好的证据就是，游戏技术已经不再仅仅局限于去服务最终用户了。它已经成为一个包装精美的起点，全世界的很多研发机构和

Download at www.Pin5i.Com

研究实验室，都用它来进行客户的开发。

这件事情的重要意义在于，游戏产业之外的世界也开始向这个产业反馈他们的信息。诚然，作为一个游戏开发人员，我们过去一直都是从文学和 GPL 工具集中抽茧剥丝，从中得到我们可以得到的东西，来满足我们自己的需求。但是，这个过程一直都是用我们的术语，按照我们的想法来进行的。而现在，我们很快就会看到，我们自己的技术会在我们的眼皮底下被那些“门外汉”们大大改观。我们可以看到，在“游戏编程精粹”系列书的作者和编者中，有许多人已经离开了游戏界，但是他们仍然继续在为塑造这个产业的未来而无私地奉献着。

新兴范式

外界对游戏产业的关注度迅速提升，而与此同时，业界自己也开始发生变化，开始转向追求更好的工具，来支持高级引擎访问和内容创建。这两件事情同时发生，绝对不是偶然的。实际上，对于各种研发机构和大学里的研究人员而言，我们现有产品背后那些复杂的创作工具是最具吸引力的。

从我个人的角度而言，我认为，我们终于开始真正地用正确的方法来做事情了。硬编码和封闭系统的时代即将终结。那种改动一个数字就要重新编译、测试，再次反复的开发周期也将一去不复返。而实际上也确实如此，只是这个过程多少有点过于漫长。我们现在正处于整个产业范围内的“文艺复兴”运动的边缘。在这个运动中，我们会看到高级的、基于组件的内容创作和游戏性创作，这些是现在只在顶尖游戏中才有所展示的技术。

如果不能够认可这个事实，那么，“游戏编程精粹”系列书就算是失职了。所以，我们彻底地接受这个事实，为读者呈现一个全新的章节——“脚本和数据驱动系统”。这个新的章节全面关注业界不断增长的这个趋势——将程序人员从调整数据的循环中解脱出来。但是，这还不是全部：这个新的范式沿袭了工作流的原理，对于我们的创意想法，它可以很容易地转化成非常快速的处理流程。这样，在整个游戏的开发过程中，程序人员、游戏策划，以及美工人员就可以用同一种语言交流、协作。其他的所有事情必然会因此而发生变化。

好了，让我们切入正题吧。

关于《游戏编程精粹 6》

《游戏编程精粹 6》为读者提供了 50 多篇文章。这些文章都是游戏技术领域（及相关领域）的专家们所撰写的。这些专家来自全世界 20 多个国家和地区。他们有着各自不同的背景。这些作者和我们编辑人员密切合作，为大家带来了内容丰富的精粹文章。希望这些精粹内容能够成为读者下一个游戏项目的一部分。书中有些文献颇具创新性。有些文章则对大家熟悉的一些问题提出了新颖的视角。所有这些文章都向我们展示了这些绝顶聪明的人士辛苦工作的成果。他们是这个产业的精英。你将在本书中探索并发现一直为你所寻找的关键性和洞察力。但是首先，请允许我吊吊你的胃口，为你准备好书中的美餐。

Download at www.Pin5i.Com

通用编程

Adam Lake，Intel 软件解决方案集团

一直以来，“通用编程”这一章都是一个展示中心。在这里，我们展示了各种各样的、见解深刻、非常聪明的编程工具。这已经成了一种传统。当然，这一次也不例外。在这一章中，你会看到我们提供了方方面面的内容，从多处理器技术、单元测试技术，到安全指纹技术，以及一个非常酷的计算机视觉技术的应用。你大可不必担心，这仅仅是一个开始：这一章的内容极其丰富，我们统筹编排的内容，里面全都是真正的游戏编程精粹。

数学和物理

Jim Van Verth，红色风暴娱乐公司

死亡和赋税。除此以外，我们还有一个欲望，就是能够从 FPU、CPU 和 GPU 中再获得一个时钟周期。每个人肯定都会有一些至关重要并形影不离的事情，如果有什么东西能被称之为游戏编程的生命线，它当然就是“数学”了。但是请不要担心，这一章的内容不是关于编写无数行的代码，去压榨出你最后那一滴能量。相反，本章的内容是一个很好的混合体，将一些标准话题的新视角和聪明的实际应用相结合，绝对应该在你的下一个游戏项目中找到用武之地。

人工智能

Brian Schwab，索尼计算机娱乐美洲公司

作为一个一直以来都最受读者欢迎的章节，我们非常认真地对待人工智能这一章，绝对不会让你失望。游戏 AI 非常明显地受到了认知科学和机器智能领域尖端研究工作的影响。由于有了学术界的抢眼表现，我们这一章的内容在行为建模这个话题上占了很大的篇幅。由于实现起来非常简单，你肯定可以在下一个及以后的游戏项目中使用这个技术。在此我们也小小地炫耀一下，我们甚至还向大家演示了如何在其他系统的引擎中来使用 AI 技术。考虑一下如何在游戏的图形图像系统中应用人工智能技术。

脚本和数据驱动系统

Graham Rhodes，Applied Research Associates, Inc.

这是我们新创设的章节。本章的这个初创版，让我们有些热泪盈眶。如果不能将“脚本和数据驱动系统”真实的一面展示给读者，我们简直无法再将《游戏编程精粹》系列书继续出版下去。在这一章中，我们首先概括地介绍了一些流行和最新出现的脚本语言。本章会教你如何发动自己的“引擎”，让它运行于一个异常灵活的骨干系统之上。这种灵活性会让你得到竞技场的印象。仔细阅读本章的内容之后，如果你根本没有动心思想去尝试一下其中所讨论的内容，那么我真的会为你感到羞愧。

Download at www.Pin5i.com

图形学

Paul Rowan，Rho 公司

这一次，在本书的“图形学”部分中，我们很好地结合了经典的技术和新兴的技术。另外，有些问题你可能认为自己已经了如指掌了，我们也进行了新的阐述，其中有一些想法真是非常棒，你自己或许从来都没有想到过。如果你正在寻找改进了的空间分割技术，从你的游戏引擎中再挤压出更多的时钟周期，那么你算是来对了。如果你从中得到了一个方法，可以为自己的游戏场景增加想所未想的更多的光照效果，那么你就掌握要领了。想要获得业界中明暗对比最强的路牌和广告牌的渲染效果吗？你可以在这里找到它们。这些只是本章精彩内容的一个小小的开场……

音频音效

Alexander Brandon，Midway Home 娱乐公司

我敢打赌，你从未想过从可变形网格中生成声音，对吧？你也从未考虑过利用空间分析来确定 PAS（Potentially Audible Set，潜在可听集）吧？我会让你搞清楚这些问题的来龙去脉。那么，现在你该知道“音频音效”这一章的内容了：对于音频系统更为先进的使用方法。而且和以前一样，本章的内容也为大家提供了各种音频系统的使用方法，所以，竖起你的耳朵，作好准备去更换你的音箱吧。

网络与多人在线游戏

Scott Jacobs，Virtual Heros 公司

与其他事情相比，近来网络游戏编程人员所面临的难题更让我感到恐惧。分布在世界各地的游戏社区中，数量庞大的玩家一起上线，这就够吓人的了。除此之外，不要忘了这样一个事实：除了程序人员，我们当中的其他人等还在不停地通过制作过程向游戏中增加细节更多、更为复杂的内容，真是让你手忙脚乱。我向这个领域的专家们致敬。他们不但要抽时间去整理归类出相关的问题，还要亲历亲为去撰写相关的讨论文章。这一次，我们为读者朋友们提供了几篇精粹文章来讨论这些问题，让我们这些网络游戏编程人员不用再熬夜加班了，或者至少可以让我们有时间通宵达旦地玩网络游戏了。

结束语

我现在意识到，之前我在文章开始所提出的那个断言是完全错误的。“游戏编程精粹”系列以及这个产业，仍然是为游戏开发人员所准备的。而且实际上，它会一直为游戏开发人员时刻准备着。没有人像我们这样如此辛苦地工作去制造“魔力”。正是因为我们辛苦的工作，才使得这个“魔力”成为我们所独有的。即使其他的行业发现了我们这些技术的好处，并进行交换和买卖，这个行业还是会作为我们的领地，一直保持下去。而且，我也

Download at www.Pin5i.Com

仍然把自己算作其中的一分子。我的心将一直属于游戏。实际上，我并没有把自己看成是一个离开游戏产业的人，我一直把自己看作是一个被委派到“国外”的形象大使。

因此，无论你是一个刚刚上路的游戏编程新手，还是一个满头白发的业界老手，亦或是一个颇受欢迎的过路人，我都真诚地希望，这本书能够为你的世界带来灵感、领悟，或者至少有一两篇对你真正有用的精粹文章。

致谢

在此，我首先要感谢本书的作者朋友们。这些不知疲倦的人们，他们首先要编写程序代码，然后还要抽时间去写文章解释这些代码。与此同时，其他无数的游戏开发人员也在他们身后不停地辛勤工作着。之所以要对他们表示感谢，不仅仅是因为这个系列丛书需要他们，同时也因为如果没有他们，这个产业的声音就会弱化很多，也许其他产业也不会来扣响我们这个产业的大门。我要对这些作者朋友们说：你们的努力和付出现在已经永久地镌刻在产业的时间线上。你绝对可以相信，某年某月某日，会有某地的某个人，利用你的这个慷慨奉献，又完成一件大事，验证并颂扬你所有的这些想法，不断地推动这个产业向前发展。

我不知该如何表达我对 Jenifer Niles 无限的感激之情。感谢她委托我来负责这套系列丛书。能够担当这个角色，我感到万分的荣幸，希望我能够成功地帮助这套系列丛书继续繁荣兴旺，为成千上万的游戏开发人员提供更多有用的信息。Jen，请原谅我曾经给你带来的各种压力！

同样，我还要特别感谢 Mark DeLoura 先生。他不但给与我充分的信任和指导，还在百忙之中为这套丛书贡献颇多。我是作为系列丛书读者的身份来说这些话的。而且，作为一个程序人员，在有些情况下，我正是从这套书中及时找到了完美的代码片段，或者有所感悟。Mark 先生，你能相信吗？你已经出版了 6 本书了！而且这个数字还会继续增加。

那么，对于各章节的编辑们，我是不会忘记他们的，要忘记他们根本是不可能的事情。从外人的角度来看，感谢编辑人员似乎是一个惯例。但是，在完成了本书的编辑工作之后，我现在终于知道，所有对编辑人员所表达的感激之情都是最最诚挚的，而且是远远不够的。我把这个比作是一个疲惫不堪的四分卫，向他的团队所做出的一个软弱无力的点头动作。是这个团队一直用自己疼痛的肩膀扛着这个项目，直到最后艰难地取得“超级碗”橄榄球比赛的胜利。再一次向大家表示感谢，我请大家喝酒。另外，也请代我向你们的家人表示感谢。我知道他们都很想你们。

最后，也是最重要的，我要感谢我的妻子 Sukie。感谢她无尽的耐心和支持。

Download at www.Pin5i.Com

关于封面图片

在这个图片中，未来战士斯科特·米切尔上尉（Scott Mitchell）正在一个都市战场上巡逻。这个图片来自 Tom Clancy 的游戏——《幽灵行动之尖峰战士》（Ghost Recon Advanced Warfighter）。这个图片是由法国育碧公司的工作室 Tiwak 利用 Photoshop、Autodesk 3ds Max 制作出来的。该工作室位于法国的蒙彼利埃。

斯科特·米切尔上尉配备的全能战士系统（IWS）是基于真实的未来军队勇士（FFW）计划。该计划由美国陆军纳提克军人研究中心开发。该系统由 Crye Precision 公司的装甲和迷彩服组成，包括他们的 MULTICAM 战斗服、装甲底盘车、BlackHawk HellStorm Fury 手套，以及 Artisent 公司的头盔。Modulae Rifle-Caseless（MR-C）则是由 Crye 联合公司设计的。

Tom Clancy 的游戏《幽灵行动之尖峰战士》（Ghost Recon Advanced Warfighter）并未收到过美国陆军总部任何形式的赞助和支持。

Download at www.Pin5i.Com

作者简介

以下是本书中所有作者的背景简介。

Luis Otavio Alvares

Luis Otavio Alvares 是巴西阿雷格里港市（也称为愉港市）南大河联邦大学（Universidade Federal do Rio Grande do Sul，简称 UFRGS）应用计算系的教授。1982 年，他在 UFRGS 大学获得了计算机科学硕士学位；1988 年，他在法国格勒诺布尔市的约瑟夫·傅里叶大学获得了信息技术博士学位。他的研究兴趣包括：人工智能在计算机游戏中的应用，多主体系统，以及数据发掘。他多年来一直以组委会成员、评论家和技术委员会成员的身份，服务于几个专业的会议和期刊。

Takashi Amada

Taka.am@gmail.com

Takashi Amada 是索尼计算机娱乐公司的一名软件工程师。2003 年，他获得了日本东京工业大学生物科学学士学位；2005 年，他又获得了奈良工业科技大学信息科学硕士学位。他的研究兴趣主要集中在虚拟现实、实时渲染，以及基于物理的动画。

James Boer

author@boarslair.com

James Boer 的游戏开发职业生涯开始于他对白尾鹿（white-tailed deer）生活习性的潜心观察。这是为了在他的游戏《猎鹿人》中开发出令人惊讶的人工智能。当他终于有所斩获，重返文明世界并接受再教育之后，他参与开发了许多款游戏，包括：《石山猎手》（Rocky Mountain Trophy Hunter）、《猎鹿人 2》、《实况模拟钓鱼》（Pro Bass Fishing）、《微软棒球 2000》、Tex Atomic 公司的《机甲大战》（Big Bot Battles）、《数码宝贝竞技场》（Digimon Rumble Arena），以及《指环王：战略版》等。他的文献作品包括：大量的《游戏开发精粹》文章、合作出版了《DirectX 完全手册》一书。他还在《游戏开发者》杂志上发表了各种类型的文章，并且自己创作出版了《游戏音频编程》一书。最近，ArenaNet 公司将他招至麾下，成为该公司的最新游戏《激战》（Guild Wars）的网络和服务器程序员。

Download at www.Pin5i.Com

Alexander Brandon

abrandon@midway.com

Alexander Brandon 早在 1994 年就涉足游戏音频音效领域。他曾经担任过多种角色，包括：作曲家、音效设计师、配音演员、配音导演、播音员、技术设计专家，以及音效导演等。他参与制作或监制了十几款著名的游戏，包括《虚幻》和《杀出重围》（Deus Ex）。在过去的 5 年中，他一直坚持在游戏开发者大会上发表演讲，并在多所大学，以及其他的游戏和音乐行业的活动上发表演讲。他是《游戏开发者》杂志《Aural Fixation》专栏的专栏作家。他自己创作出版了《游戏音频音效：规划、处理和制作》一书。他目前仍然在加州圣迭哥的 Midway 家庭娱乐公司担任音效经理。

Matthew Campbell

Campbell@bmh.com

在过去的 6 年中，Matthew W. Campbell 一直在专业环境和学术环境中开发软件产品。在两年前完成自己的大学学习后，Matthew 大部分的学术时间都花在了几个与机群并行渲染有关的专题课题上。这些课题的主要目标是，找到一种低成本的解决方案，利用显示墙和 CAVE 自动虚拟环境技术，以驱动大型虚拟环境的仿真。在这段时间中，Matthew 还在科学研究院（ISR）花了几个月的时间，将并行渲染技术集成到现有的显示墙技术中。他最近的一些研究工作主要涉及几个项目。这些项目都是基于开放源代码的游戏引擎——Delta3D（www.delta3d.org），其主要目的是为了让军事仿真和训练能够利用先进的游戏技术，并从中获益。Matthew 在美国弗吉尼亚理工大学（Virginia Tech）获得了计算机科学学士学位，现在是 BMH 联合集团公司（美国维吉尼亚州诺福克）的雇员。

Erin Catto

erincatto@gphysics.com

在过去的 10 年中，Erin Catto 一直都在从事物理引擎的开发工作。他现在是 Crystal Dynamics 公司的物理程序员，正在参与开发《古墓丽影：传奇》。在此之前，他曾经从事过的工作包括：蛋白质的动力学仿真、用于计算机辅助设计的机械仿真软件，以及机器人胳膊的建模和控制。在他年轻的时候，他就在自己的 HP48 计算器上开始编写游戏，包括《蚂蚁》和《斗争》（Joust）。他在康乃尔大学（Cornell University）获得了理论与应用力学博士学位。

Waldemar Celes

celes@inf.puc-rio.br

Waldemar Celes 是巴西里约热内卢天主教大学（PUC-Rio）计算机科学系的助理教授。他是该大学计算机图形学技术小组 Tecgraf 的研究员和高级项目经理。同时，他还是美国康乃尔大学计算

Download at www.Pin5i.Com

机图形学项目的博士后副研究员。他现在的研究方向主要是计算机图形学，包括实时渲染、科学计算可视化、物理仿真，以及分布式图形图像应用。他同时还是 Lua 编程语言的作者之一。

Octavian Marius chincisan

mariuss@rogers.com

1987 年，Octavian 毕业于罗马尼亚 Cluj-Romania 科技大学，获得了电子工程硕士学位。一年之后，他又获得了邮政大学应用电子学的大学文凭。他毕业设计的课题是，建造一台基于 Z80 的，与 Sinclair Spectrum 兼容的个人电脑。从 1988 年到 1994 年，他一直是一家金融机构的 C++ 程序员。1994 年，他来到加拿大，分别在几家公司任职，担任 C++资深软件程序员。在 2000 年，他面临着一个新的挑战：游戏编程。在这个领域中他自学成才，他的学识、热情和辛苦的工作，换来的成果是创作了 Getic 3D 编辑器和 Getic SDK。这两个产品目前都在开发之中。现在，Octavian 是 General Electric 的软件咨询师，并在 Zalsoft 公司担任兼职软件架构师。

Mike Dickheiser

mdickheiser@ara.com

Mike 是一个有 10 年游戏行业编程经验的老手，制作过计算机控制的游戏角色，以及互相追逐的汽车。后来，Mike 转行了。他现在花着政府的钞票，去仿真现实战场上任何可能发生的事情、任何可能出现的事物。凭借他在飞行模拟器开发工作上的经验，以及之前在红色风暴娱乐公司《幽灵行动》系列游戏产品中汽车 AI 的开发经验，Mike 现在每天的工作就是在 Applied Research Associates，Inc 中埋头苦干，寻找一种新的方式，将他对游戏技术的热情注入到严肃仿真世界中。就好像工作、家庭，以及编辑书稿这些工作还不够他忙的，Mike 现在不但要抽出时间去攻读北卡莱罗纳大学计算机科学的硕士学位，还在不懈地致力于世界上第一台真正智能机器的研究。所有这些能力都源于咖啡因、印度食品，以及他不可动摇的信念。他坚信，所有这些努力和付出总有一天会表现出它们的价值。

Thomas Di Giacomo

Thomas@miralab.unige.ch

在各种国际学术期刊、各种行业会议和研讨会上，Thomas Di Giacomo 已经发表了多篇有关计算机图形图像和网络的论文，并为《图形图像编程方法》和《游戏编程精粹 4》等书提供了稿件。他主要的研究方向是动画的细节层次，以及基于物理的动画。

Arjan Egges

egges@mirabla.unige.ch

Arjan Egges 在很多国际学术期刊，以及各种会议和研讨会上，发表了多篇有关计算机图形学的文章。他同时也是《计算机动画与虚拟世界》学报的编辑助理。

Download at www.Pin5i.Com

Luiz Henrique de Figueiredo

lhf@visgraf.impabr

Luiz Henrique de Figueiredo 是巴西里约热内卢纯粹数学与应用数学研究所（IMPA）的副研究员。他也是巴西里约热内卢天主教大学（PUC-Rio）计算机图形学技术小组 Tecgraf 的顾问。他现在的研究方向包括：计算几何、几何建模，以及计算机图形学中的时间间隔方法等。他对编程语言也兴趣十足，是 Lua 语言的设计者之一。

Dominic Filion

dfilion@hotmail.com

Dominic 现在是美国加州橘子郡一家主要的游戏开发工作室的图形工程师。在此之前，他在 DC Studios 公司担任技术主管，并带领技术团队开发成功该公司内部的一个跨平台 3D 引擎。他曾经参与开发了 4 款商业 3D 游戏引擎产品，并担任了其中两款引擎产品开发的总架构师。他还曾经在 Artificial Mind & Movement、Microids 和 Fan Key Studios 等公司工作过。欢迎随时和他探讨书中的文章，或者交个朋友，找他随便聊聊。

Martin Fleisz

Martin 毕业于英国的德比大学（University of Derby），获得了计算机科学学士学位。3 年来，他一直都是一名专业的 C++程序员。在大部分的业余时间里，他也不断地扩充自己在游戏及游戏图形编程技术方面的知识。其余的时间，他喜欢玩视频游戏，打架子鼓。

Chun Che Fung

lanceccfung@ieee.org

Lance C. C. Fung 博士现在是澳大利亚墨尔本大学（University of Melbourne）信息技术学院的副教授。他在西澳大利亚大学获得了博士学位，在英国威尔士大学理工学院获得了硕士学位和学士学位（一级荣誉学士学位）。还在大学学习时，他就开始了在仿真和游戏领域的研究工作。他的课题是：在各种海洋条件下，一艘自动导航控制下的轮船的航行行为仿真。长期以来，他一直在教授人工智能的课程，并且发表了多篇学术期刊论文和会议论文。他主要的研究方向和专长是应用智能技术和智能系统来解决现实生活的难题。

Diego Garces

diegogarces@gmai.com

Diego Garces 现在就职于一家西班牙的新公司——FX Interactive 公司，担任几款游戏的 AI 程序员。在他人生中第一次得到一台 PC 机的时候，他就希望能自己编写游戏。他在学校里就开

Download at www.Pin5i.Com

始编写（编程及图形设计）一些小的游戏。毕业之后，他在其他行业工作了几年，最后还是决定重返他的激情所在——游戏行业。在大学就读的时候，他学的是计算机工程专业，同时也对研究工作产生了浓厚的兴趣。这段时间，在工作之余，他努力抽时间完成他的博士学业。到目前为止，他已令人惊讶地完成了两年的博士学业，而且没有遭到公司的解雇。

Stephane Garchery

stephane@miralab.unige.ch

Stephane 已经发表了多篇计算机图形学的论文，并在《虚拟人手册》一书中撰写了多个章节。他的博士论文涉及 3D 面部动画的实时应用领域。

Julien Hamaide

Julien.hamaide@gmail.com

8 岁的时候，Julien 就开始在他的 Commodore64 机器上编写文本游戏了。不久，他编写了自己的第一个汇编语言程序。他常常喜欢自学一些东西，阅读他父母可以买来的所有书籍。2003 年，21 岁的他从比利时的蒙斯理工学院（Faculte Poly-technique de Mons）毕业，成为一名多媒体电子工程师。他在 TCTS/Multitel 公司工作了两年，负责语音和图像处理工作。他现在就职于 Elsewhere 娱乐公司，负责下一代的游戏机产品。该公司是比利时的一家视频游戏公司。

Anders Hast

aht@hig.se

从 1996 年开始，Anders Hast 就已经成为计算机科学领域的讲师。2004 年，他成为瑞典耶夫勒大学（University of Gavle）的副教授。同年，他获得了瑞典乌普萨拉大学的博士学位。在计算机图形学领域，他已经发表了 20 余篇研究论文和书刊文章。

Roberto Ierusalimschy

Roberto@inf.puc-rio.br

Roberto Ierusalimschy 是巴西里约热内卢天主教大学（PUC-Rio）的一名副教授。他是 Lua 语言的首席架构师。他现在的研究领域是编程语言。

Pete Isensee

pkisensee@msn.com

Pete Isensee 是 Xbox 先进技术小组（XATG）的开发经理。他常年来一直是游戏开发者大会上的演讲嘉宾，也是《游戏编程精粹》系列丛书的撰稿人。他喜欢和别人一起讨论 C++ 程序的性能问题，特别是再配上一杯波尔多葡萄酒的时候。

Download at www.Pin5i.Com

Scott Jacobs

scott@escherichia.net

Scott Jacobs 现在就职于美国北卡罗来纳州凯瑞市的 Virtual Heroes（虚拟英雄）公司，主要工作是利用游戏技术去开发仿真软件和训练软件。他会告诉某些人，他现在主要的研究方向是：框架测试、连续集成，以及多线程游戏引擎设计。如发现自己的谈话令人疑惑不解时，他就会把话题转到山地自行车、烹饪和享用印度餐，以及恐怖电影上。

Toby Jones

thjones@microsoft.com

Toby Jones 是微软公司媒体技术小组（MTG）的软件设计工程师。他现在正从事 Windows Vista 的下一代的多媒体管线的工作。Toby 并不想在自己的职称前面加上“资深”二字，因为他认为自己还没老到 65 岁呢。如果他可以让自己停下来，不去玩 Xbox 的话，他希望自己能够完成他的计算机科学硕士学位。在此之前，他为很多软件和游戏编写了很酷的系统代码，这些软件包括：专业的音频编辑软件 Sound Forge、专业的视频编辑软件 Vegas Video，以及 Xbox 游戏《荒野大镖客》（Dead Man's Hand）和《Prey》。还有，他最终还是成功地迎娶了 Jessica。

Chris Joslin

cjoslin@connect.carleton.ca

Chris Joslin 是加拿大卡尔顿大学的助理教授，研究方向是交互式多媒体、基于上下文关系的动态媒体适应性，以及实时协同虚拟环境（CVE，Collaborative Virtual Environment）。

HyungSeok Kim

kim@miralab.unige.ch

在计算机图形学领域和虚拟现实领域，HyungSeok Kim 已经在各种国际学术期刊和学术会议上，发表了大量的研究论文。在已经发表的论文中，主要的课题集中在：多分辨率建模、3D 内容的适应性，以及实时交互模型。HyungSeok Kim 也是《可视计算机》期刊的编辑助理。

Yongha Kim

ysoyax@gmail.com

由于他在实现动态小精灵（animated sprites）时在快速 VQ 编码算法上的杰出成果，Yongha Kim 获得了韩国浦项科技大学的硕士学位。与传统的方法相比，他的这个算法使得 Phantagram

Download at www.Pin5i.Com

公司的实时战略游戏《炽焰帝国》资源生成的速度提高了 20 倍。在这以后，他分别参与了 Phantagram 公司和 NEXON 公司大型多人在线角色扮演游戏（MMORPG）的开发。他现在是 NEXON 公司一款新 MMORDG 的项目经理。他负责的这款游戏被寄予厚望，希望能够为世界的爱与和平做出更多的贡献。

David L. Koenig

david@touchdownentertainment.com

David Koenig 现在是 Touchdown 娱乐公司（前身是 LithTech 公司）的高级软件工程师。他主要的工作内容是：支持、维护和增强 PC 平台和 Xbox 平台上 Jupiter 3D 引擎技术。他为多款游戏贡献良多，包括：《电子争霸战 2.0》（PC 版）、《电子争霸战 2.0：夺命应用》（Xbox 版）、《杀戮鹫狮》（Gun Griffon）（Xbox 版）、《芝加哥执法者》（Chicago Enforcer）（Xbox 版），以及其他很多游戏产品。他获得了美国休斯敦大学计算机工程技术学士学位。

Adam Lake

Adam.t.lake@intel.com

Adam Lake 是 Intel 公司软件解决方案小组的高级软件工程师。该小组负责着 Intel 公司的“现代游戏技术计划（MGTP）”。在他就职于 Intel 公司这 7 年中，Adam 曾经承担过多项任务，包括：非真实性渲染技术的研究，以及 Shockwave3D 引擎的开发。他设计出了一个流编程架构，包括仿真器、汇编器、编译器以及编程模型的设计与实现。在进入 Intel 公司之前，他先是获得了北卡莱罗纳大学教堂山分校（UNC-Chapel Hill）计算机图形学专业的硕士学位，然后又在美国洛斯阿拉莫斯国家实验室的计算科学方法小组中工作。他发表了一些计算机图形学方面的论文，有些还收录在 SIGGRAPH 和 IEEE 的论文集中。他还撰写了几本计算机图形学书籍中的篇章。在业余时间，他喜欢山地自行车、公路自行车、徒步旅行、露营、阅读、单板滑雪，以及开慢车。

Noel Llopis

llopis@convexhull.com

在 High Moon Studios 公司，Noel Llopis 是下一代游戏技术研究和开发先锋。他是一名机敏开发、自动测试和测试驱动开发模式的坚定支持者。他是《游戏编程人员的 C++》一书的作者，并定期地向“游戏编程精粹”系列丛书和《游戏开发者》杂志投稿。他以前的游戏作品包括：《暗黑标靶》和《机甲先锋》系列。在业余时间，他喜欢在他的个人网站上尝试一些异类的、意想不到的点子。他在马萨诸塞大学艾姆赫斯特分校获得了工学学士学位，然后又在北卡莱罗纳大学教堂山分校（UNC-Chapel Hill）取得了硕士学位。

Download at www.Pin5i.Com

Chris Lomont

chris@lomont.org

Chris Lomont 是一个专业的修理工，曾经当过销售员、研究科学家、视频游戏程序员，研究生助教等。他本人获得过数学、物理，以及计算机专业的学位，并获得了美国普渡大学代数几何的博士学位。除了热衷于计算机图形学及算法，他现在还在为 Ann Arbor 公司进行量子计算的研究。关于他的业余爱好、项目以及研究方向，更详细的信息请浏览他的个人网站 www.lomont.org（他最新的项目是一个用于艺术展示的 LED 立方体，另外，别忘了去看一下 superballs 的视频资料）。如果在数学或算法方面你有什么问题，别忘了发邮件给他。他喜欢这样的挑战。

Jörn Loviscach

jlovisca@informatik.hs-bremen.de

Jörn Loviscach 是德国不莱梅应用技术大学计算机图形学、动画及仿真专业的教授。他的研究兴趣在于：实时渲染、标准 DCC 软件的插件开发、音频处理，以及用户界面等领域。在加入不莱梅应用技术大学之前，Jörn 曾先后在德国国内的几家计算机杂志工作，其中就包括了 c't 杂志。他在那里工作了 3 年，担任副主编。虽然他在十几岁的时候就开始自己设计、制作音乐合成器和韵律电脑，但后来还是选择放弃电子专业，转而去学习物理专业，并获得了博士学位。他现在很少有时间去摆弄那些他收藏的乐器了。

Frank Luchs

gameprogramminggems@visiomedia.com

1983 年，Frank Luchs 为 Atari 计算机编写了第一个音乐程序，并从此开始了音乐/编程的双重职业生涯。他做过的项目涉及范围很广，从为电影和电视作曲、制作，到开发定制的应用程序和多媒体软件，并为这些软件提供音效设计。他自己编曲、制作了几百首歌曲、歌谣、电影配乐。Frank 参与编写了“游戏编程精粹”系列书第 3 章、第 4 章和第 6 章的部分章节。他是 Visionmedia 有限公司的创始人。这是一家专注于虚拟乐器的公司。在他自己的 Visionmedia 公司中，他设计了软件合成器——Saccara。《游戏编程精粹 6》中收录了这个基于音频颗粒的合成方法，并提供了开放源代码。Frank 现在在德国慕尼黑开展电影配乐业务。在编程之余，他喜欢用他自己的合成器装置制造出好多的噪音，或者编写电子交响曲。

Blake Madden

Blake.madden@oleandersolutions.com

Blake Madden 是 Oleander Solutions 公司的软件工程师，主要的工作方向是文本和 2D 图形软件。他现在也致力于一个 wxWidgets 的图形扩展库和一个基于 3D 的尤克牌游戏。在业余时间，他喜欢数字摄影、徒步旅行和云霄飞车。

Download at www.Pin5i.Com

Nadia Magnenat-Thalmann

thalmann@miralab.unige.ch

在过去的 20 年中，Nadia 一直都是三维虚拟人研究领域的先锋。在各个主要的学术会议上，她发表了 200 余篇论文，涉及计算机图形学领域的多项课题。她也是《可视计算机》和《计算机动画与虚拟世界》等学报（前身是《可视化与计算机动画》）的主编。

Enric Marti

enric@cvc.uab.es

1986 年，Enric Marti 加入西班牙巴塞罗那自治大学（Universitat Autonoma de Barcelona，UAB）计算机科学系。1991 年，他在巴塞罗那自治大学获得了博士学位，其论文主题是：将手绘线条图作为三维物体的分析。同年，他升为副教授。他是计算机视觉中心（CVC）的研究员。他现在的研究方向主要是文档分析。说得更具体一点就是 Web 文档分析、图像识别、计算机图形学、混合现实，以及人机交互。他现在正在研究低分辨率 Web 图像（如 GIF、JPEG 文件）的文本信息分割，利用小波（wavelet）从 Web 页面中提取文字信息。另外，他正在开发一个带有三维接口（数据手套和光学眼镜）的混合现实环境。他还是《计算机与图形学》、《计算机视觉与图像分析电子期刊》等学报的审稿人。

Curtiss Murphy

Murphy@bmh.com

在软件项目的开发和管理上，Curtiss Murphy 已经有了 13 年的从业经验。作为一个项目工程师，他曾经为各种政府和军队机构主持了多个严肃游戏项目的设计和开发工作。他的理想是：帮助政府利用低成本的游戏技术，去提高军队的训练水平。他最近负责的 6 个项目都是基于开放源代码的游戏引擎——Delta3D（www.delta3d.org），还有一个项目是美国海军研究总署的公共事务游戏。这款游戏的蓝本就是著名的游戏产品《美国陆军》（America's Army）。Curtiss 获得了美国弗吉尼亚理工学院计算机科学学士学位。他现在弗吉尼亚州诺福克（Norfolk）市的 BMH 联合公司工作。

Viknashvaran Narayanasamy

viknash@hobbiz.com

2004 年，Viknash 获得了新加坡南洋理工大学计算机工程荣誉学士学位。他现在正在澳大利亚默多克大学信息技术分校攻读游戏技术的博士学位。出于自身的兴趣所在，他多年来一直在开发休闲游戏，并在此过程中制作出了大量的游戏演示版本。此前 Viknash 曾经参与开发过 StarLogo TNG 游戏，这是麻省理工学院的一款教育仿真游戏。他现在则在参与开发一款《Murdoch Grand Challenge》游戏，这是默多克大学教育用赛车游戏。他的研发方向包括：

Download at www.Pin5i.Com

通用 GPU 算法、图形学在游戏中的应用，以及面向游戏的复杂系统建模。作为他研究工作的一部分，他现在正在研究 MMP 游戏的复杂系统设计、建模与工程。

Charles Nicholson

Charles.Nicholson@gmail.com

Charles Nicholson 是 High Moon 工作室研发团队的高级程序员。在此之前，他曾在纽约市的财政部门工作，从事网络服务器的编程工作。后来，他在 Disney VR 工作室作实习生，第一次品尝到了游戏编程的滋味，并从此事业兴旺。如果他知道你能指认出他曾经参与开发的游戏，他会非常高兴。他参与过的游戏包括：《暗黑标靶》、Disney 公司的《卡通城 OL》，以及《雪地摩托车》（Polaris SnoCross）。他和他漂亮的女友及宠物鱼一起生活在美国的南加州。

Hugo Pinto

hugo@hugopinto.net

2005 年，Hugo Pinto 在巴西南大河联邦大学获得了人工智能专业的硕士学位。在他的研究实践中，他研究了扩展行为网络（一种机器人系统架构）在《虚幻竞技场》游戏中简单人性化智能代理设计上的应用。从 2000 年至今，他一直在一些国际团队中参与开发商业人工智能应用，并从 2002 年开始，成为 IGDA 的人工智能兴趣小组的成员。他的研究兴趣包括：计算机游戏、人工智能、人机交互，以及人类语言学。

他的研究领域和工作经历跨越了多个学科，从计算机游戏、多代理系统、自然语言处理、认知架构，到机器学习。现在，他把大量的时间都投入到 Vetta 实验室中，去开发顶级的 AI 应用。另外，他还抽空去当独立的咨询顾问和游戏 AI 的讲师，学习德文、希伯来文和阿拉伯文，或者去练练健美。在 Vetta 实验室工作之外，他现在的研究方向集中在实时战略游戏，特别是如何利用多代理协同和学习技术来改进战略 AI 系统。

Frank Puig Placeres

Fpuig@fpuig.cjb.net

Frank Puig 是古巴信息科学大学虚拟现实小组的主管。他设计实现了 CAOSS 引擎，并创建了 CAOSS 工作室。CAOSS 引擎已经成功地应用在两款已经发行的游戏中。他是《游戏编程精粹 5》的作者之一。还设计制作了几个游戏开发工具，改进、简化了动作捕捉数据的处理、高级纹理贴图和动画合成等工作。

Armand Prieditis

prieditis@lookaheaddecisions.com

Prieditis 博士是 LDI 公司和 Locust 游戏公司的创始人和 CEO。LDI 公司是一家专注于

Download at www.Pin5i.Com

实时决策的公司，而 Locust 游戏公司是一个游戏 AI 系统和动画软件的开发商。在 LDI 公司时，他获得了美国国家标准与技术协会高新技术计划（Advanced Technology Program）200 万美元的基金。在成立 LDI 公司之前，Prieditis 是加利福尼亚大学戴维斯分校计算机科学系的教授。他和别人共同主持了第 12 届机器学习国际会议。对于机器学习领域其他的国际会议，以及“人工智能全美及国际联合会议”，他都积极参与，帮助规划、组建会议组织委员会和评审小组。

Steve Rabin

Steve_rabin@hotmail.com

Steve Rabin 已经在游戏行业工作了 15 年之久。他目前是任天堂美国分公司的高级软件工程师。他曾为 3 款已经发行的游戏开发了 AI 系统，也曾为“游戏编程精粹”系列书撰写了多篇稿件，为这个系列丛书的出版做出了杰出的贡献。他还是“Introduction to Game Development and AI Game Programming Wisdom”系列书的创始人和主编。他曾在游戏开发者大会 GDC 上发言。他是 DigiPen 理工学院和华盛顿大学分校游戏开发认证计划的讲师。Steve 获得了华盛顿大学计算机工程学士学位和计算机科学硕士学位。

Arnau Ramisa

aramisa@iiia.csic.es

Arnau Ramisa 毕业于西班牙巴塞罗那自治大学，获得了计算机科学学位。他现在正在该大学人工智能学院攻读计算机视觉与人工智能的博士学位，研究方向是视觉不变量和目标识别。他的兴趣主要在计算机视觉、扩增现实技术（Augmented Reality）、人机接口，当然，还有视频游戏。

Michael Ramsey

Mike Ramsey 是 Blue Fang 公司的主程序员，负责开发 Xbox 360 和 PS3 的相关技术。Mike 分别为“游戏编程精粹”和“AI Wisdom”系列书撰写了稿件。他获得了丹佛的大都会州立学院（MSCD）计算机科学学士学位。在他的业余时间，Mike 喜欢和他的宝贝女儿 Gwynn 一起玩棒垒球（Wiffle Ball）。

Aurelio Reis

AurelioReis@gmail.com

对 Aurelio 来说，视频游戏一直都是一个入口，带他去游览、享受一个新奇、令人激动不已的世界。当他认识到视频游戏是一条创造性出路时，就决定把视频游戏作为自己的职业，并从此倾心奉献，去创造出新的游戏，用溢彩流光的画面唤起人们的敬畏，用迷人

Download at www.Pin5i.Com

的游戏性挑战玩家，引发强烈的情绪共鸣，让玩家沉浸其中。Aurelio 现在是 Raven 软件公司的高级程序员，专门负责游戏性系统和图形技术。他正在为 Raven 公司的下一款产品，开发下一代的多平台技术。在他的闲暇时光（少有），他喜欢作曲、运动，自己制作图形演示 Demo，或者设计素人、独立游戏。

Graham Rhodes

grhodes@gsrhodes.com

和其他很多人一样，由于被自己玩过的街机游戏施了催眠术，Graham Rhodes 开始在高中的 Apple II 计算机上编写自己的街机风格的游戏。那时候，他还是个十几岁的少年。他早期的作品包括 Commodore VIC-20 上开发的街机风格的游戏 Robotron，以及为 Atari 8 位家用电脑开发的 Lunar Craft。20 世纪 90 年代，在他领导下，为 PC CD-ROM 版的《多媒体世界百科全书》（*World Book Multimedia Encyclopedia*）（1997～2000 年）开发了一系列的迷你游戏。这些小游戏是为了向高中生们传授基本的物理知识，其中包括学生控制的、实时物理仿真系统。最近，Graham 一直在领导开发用于工业安全培训的动作游戏。这款游戏采用了最先进的 3D 游戏引擎。Graham 还在为仿真和训练应用开发全新的 3D 图形技术。他曾经发表过的文章包括：为 Charles River Media 的三本书和《游戏编程精粹》系列书撰写的稿件，还有为最近刚刚发行的《游戏开发入门》一书撰写的一个章节。他是游戏开发互联网门户网站 gamedev.net 数学和物理频道的协调员。他为每年一度的游戏开发者大会 GDC 投稿，他是 ACM/SIGGRAPH 和国际游戏开发者联合会（IGDA）的会员。他获得了美国北卡罗莱纳州立大学空间工程硕士学位。

Paul Rowan

Paul@rowandell.com

早在 20 世纪 90 年代，Paul 就在游戏行业摸爬滚打了。他的第一个图像引擎的重点，是为了用最高的 CPU 性能去访问主存中的一个帧缓存。那时只要以 30Hz 的频率在电脑屏幕上显示一些颜色，就可以喝庆功酒了。现在，他的工作重点是为游戏美工和游戏策划人员开发他们最需要的特性。在寒冷的冬夜，他还在那里进行内循环的优化，作为与那些刚入门的旁观者沟通的社交手段。

Sebastien Schertenleib

Sebastien.Schertenleib@epfl.ch

Sebastien Schertenleib 是瑞士联邦理工学院 Vrlab 的研究助理和博士研究生。他在洛桑联邦工业大学获得了计算机科学硕士学位。他的研究方向很大程度上被限定在群体仿真的相关领域，以及 3D 实时应用的平台架构。他现在的研究课题是虚拟遗传仿真。他欢迎各位读者能够通过上面的邮件地址，就他的文章给出一些反馈，或者彼此交换一些看法。

Download at www.Pin5i.Com

Brian Schwab

Brian_schwab@yahoo.com

从古老的 Genesis 到现在的 PS3，Brian Schwab 已经在游戏行业奋战了十几个年头。在这段时间里，他坚持不懈，不断地推进产品的游戏性和游戏 AI，直到把整个系统都做得很出色为止。他是《AI 游戏引擎编程》一书的作者。

Allen Sherrod

programminAce@UltimateGameProgramming.com

Allen Sherrod 从美国德福瑞大学（DeVry University）大学获得了计算机信息系统的学士学位。他曾为《游戏开发者》杂志撰写过稿件，并准备在 2006 年底出版一本关于 DirectX 游戏编程的书。

Larry Shi

Shiw@cc.gatech.edu

Larry Shi 已经与别人合作发表了 15 篇论文，主题涉及：娱乐计算、硅基数字版权管理、安全，以及密码处理。他在娱乐计算领域主要的研究内容包括：在线游戏的数据发行、对玩家生成的数据进行数据挖掘处理、仿真游戏的自动测试，以及游戏性奖励工程。Larry 曾在 nVIDIA 公司工作，他参与了 nForce 平台网络单元的设计，并负责研究如何利用最先进的 GeForce GPU 进行通用计算。

在 2004 年夏天，Larry 还曾在 EA Maxis 任职，参与制作了游戏机游戏 URBZ。他现在的工作重点是为图形应用设计一个数字版权解决方案，为下一代手机网络游戏设计一个新的移动电话服务和基础架构。Larry Shi 获得过两个硕士学位：计算机科学硕士学位和心理学硕士。在 2006 年 Larry 拿到计算机科学博士学位。

Marq Singer

Marq.singer@redstorm.com

Marq 的职业生涯漫长而多变。在 20 世纪 80 年代后期及 90 年代初期，他在影视圈工作，负责过各种项目，担任过多种角色，从商业电视台的普通职员，到恐怖电影的特效制作，包括经典的小制作独立电影《杀手》（Killer）（1989）。他与别人合作创作了《Java Applets 和频道：无需编程》一书，也是几个大学中游戏与技术专业的客座讲师。从 1998 年开始，他开始投身于游戏行业，担任过一些工程职位，包括物理、动画、UI 及游戏 AI。现在，他是红色风暴娱乐公司的物理程序员，从事 Havok 引擎和柔体动力学的工作。红色风暴娱乐公司是法国育碧公司的分公司。他最新的游戏作品包括 PS2 版本和 PC 版本的《彩虹六号：禁闭》（Rainbow Six Lockdown）。

Download at www.Pin5i.com

Vaclav Skala

skala@kiv.zcu.cz

Vaclav Skala 教授是捷克科技大学的博士。他现在主持西波希米亚大学（University of West Bohemia，UWB）“计算机图形图像与可视化中心”的工作，并负责捷克科学院、微软英国研究院和斯柯达汽车公司（德国大众集团）等一些机构和公司的研究、应用项目。CCGV 有很多的国际研究联系机构。由于在“Socrates-Erasmus”项目中，CCGV 在欧共体国家的大学中拥有 30 多个教育联系伙伴，因此计算机图形学专业的硕士研究生通常可以在国外学习整整一个学期。他的研究方向是计算机图形学与可视化算法、算法与数据结构。他是欧洲计算机图形大会执行委员、ACM-SIGGRAPH（国际计算机组织、计算机图形图像专业组织）和计算机图形协会的成员，是《计算机与图形学》*Computer & Graphics*，Pergamon 出版集团、《可视化计算机》（*The Visual Computer*，德国 Springer-Verlag 科技出版集团）编委会成员，他同时还是多个科技会议和研讨会的国际统筹委员会的成员。他独自或与他人合作出版了大量的研究论文，也是 WSCG（中欧国际计算机图形学、可视化和计算机视质）大会的组织者。

Peter Smith

peter@smithpa.com

Peter 是美国海军航空兵武器系统部（NAVAIR）NETC 实验室的承包人。他的工作使他专注于新技术在海军训练中的应用。NETC 实验室的主要重点是利用游戏技术服务于海军训练，包括 Delta3D（一个用于开发训练软件的开放源代码游戏引擎），以及其他的基于游戏的训练系统。Peter 还是美国佛罗里达中央大学的博士生，在建模与仿真专业学习。他已经获得了该大学建模与仿真专业的硕士学位，以及罗斯—豪曼理工学院计算机工程专业的学士学位。

Roger A.Smith

gpg@modelbenders.com

Roger 是 Sparta 公司的总工程师，也是 Modelbenders 有限责任集团公司的总裁。他为军队开发仿真系统，创立了仿真与虚拟世界技术的商业课程，并定期为各种书刊、百科全书、期刊和会议提供技术论文。他曾多次在游戏开发者大会上讲授全天的教程，并在几所大学里开设了仿真与游戏专业课程。

Robert Sparks

Sparks.Robert@gmail.com

Robert Sparks 从 2001 年开始就在 Radical 公司（加拿大温哥华）工作，目前正忙于游戏《疤面煞星：掌握世界》音频部分的编程工作。他过去的游戏作品包括：《辛普森一族：边打

Download at www.Pin5i.Com

边跑》、《辛普森一族：马路狂飙》，以及《绿巨人浩克》。在奥地利 Doppelmayr Seilbahnen GmbH 公司开发滑雪缆车的安全控制系统时，他第一次接触到了嵌入式系统编程。

Ben St. John

Ben.stjohn@siemens.com

1997 年，Ben St. John 加入 EA 公司，参与开发 PS 平台的《NHL 冰球》系列游戏，也由此开始了自己的游戏行业之旅。由于他对德国的喜爱，很快他就搬到了慕尼黑。在慕尼黑，他在西门子公司找到了一个工作，从事图形、图像处理，以及手机游戏（包括开发基于摄像头的手机游戏《打蚊子》）的研发工作。从此，他就转到了西门子公司的中央研发部门，虽然与游戏脱离了干系，但却一直尝试染指这个虚拟世界。

Chris Stoy

cstoy@nc.rr.com

Chris Stoy1997 年就加入游戏行业。他是 Sinister 游戏公司的联合创始人，并帮助开发出了他们的第一款游戏——《影子部队：生死相伴》。后来，他开始主持新技术的开发工作。最近，他一直在红色风暴娱乐公司工作，参与开发了 PS2 版本的《彩虹六号：禁闭》（Rainbow Six Lockdown）。在工作之余，Stoy 还担任了罗利（美国北卡罗来纳州的首府）IGDA 顾问组委员，并热衷于健美运动。

Jim Van Verth

jimvv@redstrom.com

Jim Van Verth 是红色风暴娱乐公司的高级程序员，主要工作内容是 3D 图形学与仿真。他和别人合作出版了《游戏与交互应用的数学精华》一书，也为“游戏编程精粹”系列丛书撰写过稿件。除此以外，他还为游戏开发者大会组织了一个主题为数学和物理的年度教程。Jim 与他的妻子、女儿以及 500 只饥饿的狗，一起生活在风吹草低见牛羊的 Eldritch 平原边缘的一幢鬼屋里。大家都知道，他有时候喜欢夸大其辞（他家里其实只有 1 只饥饿的狗）。

Enric Vergara

evergara@lsi.upc.edu

Enric Vergara 是西班牙巴塞罗那自治大学的计算机工程师，最近他获得了西班牙庞装法布拉大学（Pompeu Fabra University）视频游戏制作专业的硕士学位。他现在正在加泰罗尼亚理工大学的 MoViBio 研究小组攻读博士学位，研究方向是生物医学数据的建模与可视化。

Download at www.Pin5i.Com

Gabriyel Wong

gabriyel@gmail.com

Gabriyel Wong 是 gameLAB 的技术主管。gameLAB 是新加坡南洋理工大学专攻游戏技术的研究和开发机构。在此之前，他在多家公司担任过技术总监和软件开发人员，制作过游戏、仿真和虚拟现实软件。在闲暇时，他喜欢在家照顾他的宝贝儿子，编码、研究实时应用中最尖端的渲染技术和交互技术。

Kok-Wai Wong

k.wong@ieee.org

Kok-Wai Wong 是澳大利亚默多克大学信息技术分校的讲师。他在 1994 年和 2000 年，分别获得了科廷理工大学计算机系统工程学士学位（荣誉）和博士学位。他在 AI 领域已经工作了 10 年的光景。他是默多克大学信息技术分校“游戏与仿真研究小组”的创始人之一，他同时也是 CyberGames 2006 大会主席之一。他现在的研究方向包括游戏 AI、MMP 游戏的复杂系统建模、游戏性的人格化，以及 MMP 游戏的多代理协作的理论。

Download at www.Pin5i.Com

目　录

第1章　通用编程

简介 …… 2

Adam Lake

1.1　Lock-Free 算法 …… 4

Toby Jones

1.1.1　Compare-And-Swap 及其他通用原语 …… 4

1.1.2　Lock-Free 参数化的堆栈 …… 6

1.1.3　Lock-Free 参数化的队列 …… 9

1.1.4　Lock-Free 参数化的 Freelist …… 11

1.1.5　总结 …… 13

1.1.6　参考文献 …… 13

1.1.7　相关资源 …… 14

1.2　通过 OpenMP 来充分利用多核处理器的能力 …… 15

Pete Isensee

1.2.1　OpenMP 应用实例：粒子系统 …… 15

1.2.2　好处 …… 16

1.2.3　性能 …… 16

1.2.4　OpenMP 应用实例：碰撞检测 …… 17

1.2.5　线程组 …… 18

1.2.6　函数的并行化 …… 18

1.2.7　缺陷 …… 19

1.2.8　结论 …… 20

1.2.9　参考文献 …… 21

1.2.10　相关资源 …… 21

1.3　用 OpenCV 库实现游戏中的计算机视觉 …… 22

Arnau Ramisa、*Enric Vergara*、*Enric Marti*

1.3.1　引子 …… 22

1.3.2　游戏中的计算机视觉 …… 22

1.3.3　开放的计算机视觉库 …… 22

1.3.4　计算机视觉在游戏中一个简单的应用 …… 23

1.3.5　未来的工作 …… 31

1.3.6　参考文献 …… 31

1.4　游戏对象的地理网格注册 …… 33

Roger Smith

Download at www.Pin5i.Com

1.4.1 引子 ····· 33
1.4.2 四叉树和八叉树 ····· 34
1.4.3 游戏对象的组织形式 ····· 35
1.4.4 总结 ····· 39
1.4.5 参考文献 ····· 39
1.5 BSP 技术 ····· 40
Octavian Marius Chincisan
1.5.1 什么是 BSP？为什么要使用 BSP？ ····· 40
1.5.2 基于节点的 BSP ····· 41
1.5.3 渲染一个基于节点的 BSP 树 ····· 43
1.5.4 基于节点的 BSP 树（不进行分割） ····· 43
1.5.5 凸状叶子 BSP 树 ····· 44
1.5.6 凸状叶子 BSP 树出入口生成 ····· 47
1.5.7 凸状叶子 BSP 树潜在可视集 ····· 48
1.5.8 PVS 压缩 ····· 51
1.5.9 地形 BSP ····· 53
1.5.10 总结 ····· 53
1.5.11 参考文献 ····· 54
1.6 最相似字串匹配算法 ····· 55
James Boer
1.6.1 基于字符串的 ID 查找难题 ····· 55
1.6.2 问题的定义 ····· 56
1.6.3 现有的一些解决方案 ····· 56
1.6.4 我们自己定制的字串匹配解决方案 ····· 56
1.6.5 解决方案的实际应用 ····· 62
1.6.6 总结 ····· 63
1.6.7 参考文献 ····· 63
1.7 利用 CppUnit 实现单元测试 ····· 64
Blake Madden
1.7.1 单元测试技术概览 ····· 64
1.7.2 CppUnit 概述 ····· 65
1.7.3 运行测试夹具 ····· 67
1.7.4 利用 CppUnit 进行模型类测试 ····· 68
1.7.5 私有函数的单元测试 ····· 74
1.7.6 用 CppUnit 测试底层功能 ····· 75
1.7.7 总结 ····· 79
1.7.8 参考文献 ····· 80
1.8 为游戏的预发布版本添加数字指纹，威慑并侦测盗版行为 ····· 81
Steve Rabin

Download at www.Pin5i.Com

1.8.1 威慑策略 …… 81
1.8.2 利用水印和指纹来进行侦测 …… 82
1.8.3 添加数字指纹的流程 …… 83
1.8.4 数字指纹添加过程的安全性 …… 83
1.8.5 数字指纹的添加策略 …… 83
1.8.6 破解数字指纹 …… 85
1.8.7 总结 …… 86
1.8.8 参考文献 …… 86
1.9 通过基于访问顺序的二次文件排序，实现更快速的文件加载 …… 87
David Koenig
1.9.1 问题的提出 …… 87
1.9.2 解决方案 …… 88
1.9.3 基于访问的二次文件排序的工作流程 …… 89
1.9.4 优化效果 …… 90
1.9.5 影响最终优化结果的因素 …… 90
1.9.6 潜在的问题 …… 91
1.9.7 其他一些通用的最佳实践方法 …… 91
1.9.8 总结 …… 92
1.9.9 参考文献 …… 92
1.10 你不必退出游戏：资产热加载技术可以实现快速的反复调整 …… 93
Charles Nicholson
1.10.1 资产热加载的工作流程 …… 93
1.10.2 资产热加载过程的剖析 …… 94
1.10.3 实际应用中需要考虑的事项 …… 97
1.10.4 示范程序 …… 98
1.10.5 总结 …… 99
1.10.6 进阶参阅 …… 99

第 2 章 数学与物理

简介 …… 102
Jim Van Verth
2.1 浮点编程技巧 …… 104
Chris Lomont
2.1.1 浮点数的格式 …… 105
2.1.2 例程的设计 …… 108
2.1.3 总结 …… 117
2.1.4 参考文献 …… 118
2.2 利用齐次坐标实现投影空间中的 GPU 计算 …… 119
Vaclav Skala

Download at www.Pin5i.Com

2.2.1 相关的数学背景知识 …… 119
2.2.2 利用齐次坐标进行计算 …… 121
2.2.3 直线交叉 …… 124
2.2.4 总结 …… 125
2.2.5 附录 A …… 125
2.2.6 附录 B …… 126
2.2.7 致谢 …… 127
2.2.8 参考文献 …… 127
2.3 利用叉乘积求解线性方程组 …… 128
Anders Hast
2.3.1 简介 …… 128
2.3.2 隐式直线 …… 130
2.3.3 高效的扫描转换的设置运算 …… 132
2.3.4 求解三元一次方程组 …… 134
2.3.5 总结 …… 135
2.3.6 致谢 …… 135
2.3.7 参考文献 …… 135
2.4 适用于游戏开发的序列索引技术 …… 137
Palem GopalaKrishna
2.4.1 相关术语 …… 137
2.4.2 序列 …… 138
2.4.3 范围序列 …… 139
2.4.4 排列序列 …… 142
2.4.5 组合序列 …… 144
2.4.6 总结 …… 147
2.4.7 参考文献 …… 149
2.5 多面体浮力的精确计算 …… 150
Erin Catto
2.5.1 浮力 …… 150
2.5.2 多边形的面积 …… 152
2.5.3 多面体的体积 …… 153
2.5.4 物体部分没入水中的情况 …… 154
2.5.5 算法的鲁棒性 …… 157
2.5.6 阻力 …… 158
2.5.7 关于源代码 …… 159
2.5.8 总结 …… 159
2.5.9 致谢 …… 159
2.5.10 参考文献 …… 159
2.6 带有刚体交互作用的基于粒子的实时流体仿真系统 …… 161

Download at www.Pin5i.Com

Takashi Amada
2.6.1 流体仿真与平滑粒子的流体动力学 …… 161
2.6.2 扩展 SPH 方法，以支持流体和刚体的交互作用 …… 165
2.6.3 与动态刚体的交互作用：仿真更新 …… 169
2.6.4 具体的实现细节 …… 170
2.6.5 相关的优化 …… 173
2.6.6 总结 …… 174
2.6.7 参考文献 …… 174

第3章 人 工 智 能

引言 …… 176
Brain Schwab
3.1 游戏的制作方法——应用基于模型的决策——在雷神之锤 III 中应用蝗虫人工智能引擎 …… 178
Armand Prieditis
Mukesh Dalal
3.1.1 引言 …… 178
3.1.2 目前的游戏人工智能：基于规则 …… 179
3.1.3 规则的问题 …… 180
3.1.4 基于模型的游戏人工智能方法 …… 182
3.1.5 对游戏的接口 …… 182
3.1.6 对游戏人工智能开发者的好处和推论 …… 183
3.1.7 雷神之锤 III 竞技场和蝗虫人工智能引擎 …… 184
3.1.8 相关工作 …… 185
3.1.9 结论和未来的工作 …… 186
3.1.10 参考文献 …… 186
3.2 独立非玩家角色合作行为的实现 …… 187
Diego Garcés
3.2.1 可能的解决方案 …… 187
3.2.2 非玩家角色的结构 …… 189
3.2.3 合作的机制 …… 189
3.2.4 例子：合作寻找玩家 …… 194
3.2.5 结论 …… 195
3.2.6 参考文献 …… 195
3.3 针对游戏的基于行为的机器人架构 …… 196
Hugo Pinto、*Luis Otavio Alvares*
3.3.1 包容体结构 …… 196
3.3.2 扩展的行为网络 …… 199

Download at www.Pin5i.Com

3.3.3 讨论 …… 201
3.3.4 结论 …… 201
3.3.5 参考文献 …… 202
3.4 使用模糊感知器、有限状态自动机和扩展的行为网络为虚幻竞技场游戏构建一个目标驱动的机器人 …… 203
Hugo Pinto、Luis Otavio Alvares
3.4.1 扩展的行为网络设计 …… 203
3.4.2 层次模糊感知器 …… 208
3.4.3 有限状态自动机行为模块 …… 210
3.4.4 结论 …… 211
3.4.5 参考文献 …… 212
3.5 一个目标驱动的虚幻竞技场游戏角色程序：使用扩展的行为网络制作目标驱动的具有个性的代理 …… 213
Hugo Pinto、Luis Otavio Alvares
3.5.1 扩展的行为网络 …… 214
3.5.2 行为选择的质量 …… 216
3.5.3 个性设计 …… 218
3.5.4 结论 …… 220
3.5.5 参考文献 …… 221
3.6 用支持向量机为短期记忆建模 …… 223
Julien Hamaide
3.6.1 支持向量机简介 …… 223
3.6.2 短期记忆模型化 …… 226
3.6.3 CPU 的消耗限制 …… 227
3.6.4 结论 …… 228
3.6.5 参考文献 …… 228
3.7 使用战力值评估模型进行战争役分析 …… 229
Michael Ramsey
3.7.1 基本公式 …… 229
3.7.2 计算兵力 …… 230
3.7.3 计算潜在兵力 …… 230
3.7.4 为武器效力进行建模 …… 231
3.7.5 获得一个理论上的战争结局 …… 232
3.7.6 关于 CEV …… 232
3.7.7 一个 QJM 系统的例子 …… 232
3.7.8 局限性 …… 233
3.7.9 结论 …… 234
3.7.10 参考文献 …… 234
3.8 设计一个多层可插拔的 AI 引擎 …… 235

Download at www.Pin5i.Com

Sebastien Schertenleib

3.8.1 相关工作 ····· 235
3.8.2 AI 引擎架构 ····· 236
3.8.3 数据驱动类和属性 ····· 237
3.8.4 分优先级的任务管理器 ····· 240
3.8.5 性能问题和技术 ····· 241
3.8.6 工具 ····· 243
3.8.7 结论 ····· 244
3.8.8 参考文献 ····· 245

3.9 一个管理场景复杂度的模糊控制方法 ····· 247

Gabriyel Wong、Jialiang Wang

3.9.1 关键思想 ····· 247
3.9.2 为什么使用模糊控制？ ····· 247
3.9.3 工具 ····· 248
3.9.4 系统设计 ····· 249
3.9.5 游戏中的应用 ····· 251
3.9.6 假设 ····· 251
3.9.7 实现考虑 ····· 252
3.9.8 测试和结果 ····· 252
3.9.9 结论 ····· 254
3.9.10 致谢 ····· 254
3.9.11 参考文献 ····· 254

第 4 章 脚本和数据驱动系统

简介 ····· 256

Graham Rhodes

4.1 脚本语言总述 ····· 258

Diego Garcés

4.1.1 为什么要使用脚本语言 ····· 258
4.1.2 简介 ····· 258
4.1.3 语言编码 ····· 259
4.1.4 与 C++的整合 ····· 262
4.1.5 性能特点 ····· 267
4.1.6 开发支持特点 ····· 269
4.1.7 总结 ····· 271
4.1.8 参考文献 ····· 271

4.2 把 C++对象绑定到 Lua ····· 273

Waldemar Celes、Luiz Henrique de Figueiredo

Roberto Ierusalimschy

Download at www.Pin5i.Com

4.2.1 绑定函数 …… 273
4.2.2 绑定宿主对象和 Lua 数值 …… 276
4.2.3 绑定宿主对象和 Lua 对象 …… 278
4.2.4 宿主和 Lua 表绑定 …… 282
4.2.5 总结 …… 283
4.2.6 参考文献 …… 285
4.3 使用 LUA 协同程序实现高级控制机制 …… 286
Luiz Henrique de Figueiredo、Waldemar Celes
Roberto Ierusalimschy
4.3.1 Lua 协同程序 …… 286
4.3.2 过滤器 …… 287
4.3.3 迭代器 …… 288
4.3.4 任务安排 …… 291
4.3.5 协作式多线程 …… 291
4.3.6 总结 …… 296
4.3.7 参考文献 …… 296
4.4 在多线程环境里处理高级脚本执行 …… 297
Sebastien Schertenleib
4.4.1 基于组件的软件和脚本的解释程序 …… 297
4.4.2 协同程序与微线程程序 …… 298
4.4.3 微线程管理器 …… 298
4.4.4 嵌入 Python …… 300
4.4.5 试验和结果 …… 304
4.4.6 总结 …… 305
4.4.7 参考文献 …… 305
4.5 使用非插入型代理导出角色属性 …… 306
Matthew Campbell、Curtiss Murphy
4.5.1 角色、代理和属性 …… 306
4.5.2 非插入型和动态体系结构 …… 308
4.5.3 角色属性 …… 308
4.5.4 角色代理 …… 311
4.5.5 从理论到实践 …… 313
4.5.6 总结 …… 314
4.5.7 参考文献 …… 314
4.6 基于组件的游戏对象系统 …… 315
Chris Stoy
4.6.1 游戏对象 …… 315
4.6.2 基本的游戏对象组件 …… 316
4.6.3 在游戏对象中实现组建管理 …… 317

Download at www.Pin5i.Com

4.6.4 组件间的通信 ····· 319
4.6.5 游戏组件模板 ····· 320
4.6.6 游戏对象模板 ····· 322
4.6.7 数据驱动的游戏对象创建 ····· 323
4.6.8 总结 ····· 324

第5章 图 形 学

简介 ····· 326
Paul Rowan
5.1 交互角色真实的静止动作合成 ····· 328
Arjan Egges, Thomas Di Giacomo 和 Nadia Magnenat-Thalmann
5.1.1 简介 ····· 328
5.1.2 人体动画的主分量 ····· 329
5.1.3 姿势变换 ····· 331
5.1.4 姿势的连续微小变化 ····· 334
5.1.5 总结 ····· 337
5.1.6 参考文献 ····· 337
5.2 用自适应二叉树进行空间剖分 ····· 339
Martin Fleisz
5.2.1 如何建立自适应二叉树 ····· 339
5.2.2 ABT 实现细节 ····· 340
5.2.3 找到合适的分割面 ····· 343
5.2.4 在动态场景中使用 ABT ····· 345
5.2.5 渲染 ABT ····· 346
5.2.6 总结 ····· 347
5.2.7 致谢 ····· 348
5.2.8 参考文献 ····· 348
5.3 用有向包围盒增强对象裁减 ····· 349
Ben St. John
5.3.1 方法概要 ····· 349
5.3.2 传统技术 ····· 350
5.3.3 针对二维的有效解决方案 ····· 350
5.3.4 传统技术上的改进 ····· 352
5.3.5 对包围盒进行筛选 ····· 354
5.3.6 进一步改进 ····· 355
5.3.7 总结 ····· 356
5.3.8 参考文献 ····· 356
5.4 皮肤分离的优化渲染 ····· 357
Dominic Filion

Download at www.Pin5i.Com

5.4.1 简介 ····· 357
5.4.2 分割的概念 ····· 358
5.4.3 权重分割的启发式算法 ····· 359
5.4.4 骨骼调色板的启发式算法 ····· 359
5.4.5 启发式算法的细节 ····· 361
5.4.6 总结 ····· 364
5.5 GPU 地形渲染 ····· 365
Harald Vistnes
5.5.1 算法 ····· 365
5.5.2 细节层次 ····· 366
5.5.3 避免裂缝 ····· 368
5.5.4 视锥体裁减 ····· 369
5.5.5 法线计算 ····· 369
5.5.6 碰撞检测 ····· 370
5.5.7 实现细节 ····· 370
5.5.8 运行结果 ····· 371
5.5.9 总结 ····· 372
5.5.10 参考文献 ····· 372
5.6 基于 GPU 的交互式流体动力学与渲染 ····· 373
Frank Luna
5.6.1 数学背景知识 ····· 374
5.6.2 GPU 实现 ····· 376
5.6.3 流体互动 ····· 382
5.6.4 补充材料 ····· 383
5.6.5 总结 ····· 384
5.6.6 参考文献 ····· 384
5.7 基于多光源的快速逐像素光照渲染 ····· 386
Frank Puig Placeres
5.7.1 延迟解决方案 ····· 386
5.7.2 高端硬件的延迟着色实现 ····· 387
5.7.3 基本存储优化 ····· 389
5.7.4 着色器优化和硬件限制 ····· 391
5.7.5 扩展图像空间（Extending Image-Space）的后处理特效（Post-Processing Effects） ····· 394
5.7.6 总结 ····· 394
5.7.7 参考文献 ····· 394
5.8 路标渲染的清晰化 ····· 395
Jörn Loviscach
5.8.1 反走样阈值划分纹理（Antialiasing Thresholded Textures） ····· 396

Download at www.Pin5i.Com

5.8.2 阈值划分的优化纹理（Optimal Textures for Thresholding） ····· 400
5.8.3 创作程序（The Authoring Application） ····· 403
5.8.4 总结与展望 ····· 405
5.8.5 参考文献 ····· 405
5.9 天空渲染在游戏中的实际运用 ····· 407
Aurelio Reis
5.9.1 所需与所得相悖 ····· 407
5.9.2 天空的组成 ····· 408
5.9.3 瓶颈 ····· 409
5.9.4 立方体天空贴图概要 ····· 411
5.9.5 特殊时间 ····· 412
5.9.6 演示程序的黎明 ····· 413
5.9.7 进入地平线之下 ····· 414
5.9.8 总结 ····· 414
5.9.9 参考文献 ····· 415
5.10 基于 OpenGL 帧缓冲区对象的高动态范围渲染 ····· 416
Allen Sherrod
5.10.1 帧缓冲区对象简介 ····· 416
5.10.2 设置帧缓冲区对象 ····· 417
5.10.3 基于帧缓冲区对象的高动态范围渲染 ····· 419
5.10.4 总结 ····· 421
5.10.5 补充材料 ····· 421
5.10.6 参考文献 ····· 422

第6章 音 频

简介 ····· 424
Alexander Brandon
6.1 由可变形网格（Deformable Meshes）实时生成声音 ····· 426
Marq Singer
6.1.1 对《游戏编程精粹 4》的回顾 ····· 426
6.1.2 概述 ····· 427
6.1.3 对模态分析的简要回顾 ····· 427
6.1.4 声音要求 ····· 429
6.1.5 从变形到声音 ····· 429
6.1.6 总结 ····· 430
6.1.7 进一步阅读 ····· 430
6.1.8 参考文献 ····· 431
6.2 实时音效轻量级生成器 ····· 432

Download at www.Pin5i.Com

Frank Luchs
6.2.1 环境声引擎 ····· 432
6.2.2 声音合成 ····· 432
6.2.3 真实世界范例 ····· 434
6.2.4 总结 ····· 435
6.2.5 范例 ····· 436
6.2.6 参考文献 ····· 436
6.3 实时混音总线 ····· 437
James Boer
6.3.1 并非不重要的任务 ····· 437
6.3.2 实现音量总线链 ····· 438
6.3.3 以比率或分贝的形式来表示音量 ····· 440
6.3.4 执行效率问题 ····· 440
6.3.5 其他增强 ····· 441
6.3.6 总结 ····· 441
6.3.7 参考文献 ····· 441
6.4 可听集（Potentially Audible Sets） ····· 442
Dominic Filion
6.4.1 PVS 入门 ····· 442
6.4.2 PAS 基础 ····· 443
6.4.3 直接声音路径 ····· 443
6.4.4 为门窗创建动态 PAS ····· 447
6.4.5 PAS 扩展：传导（transmission） ····· 448
6.4.6 PAS 扩展：反射 ····· 449
6.4.7 总结 ····· 450
6.4.8 参考文献 ····· 450
6.5 一种开销较低的多普勒效果 ····· 451
Julien Hamaide
6.5.1 多普勒效果 ····· 451
6.5.2 编写多普勒效果程序 ····· 454
6.5.3 线性插值 ····· 454
6.5.4 根据 *R* 来计算下标 ····· 455
6.5.5 非恒定速度 ····· 456
6.5.6 信号对齐（Aliasing） ····· 457
6.5.7 实现 ····· 457
6.5.8 总结 ····· 457
6.5.9 资源 ····· 458
6.6 仿造实时 DSP 效果 ····· 459
Robert Sparks

Download at www.Pin5i.Com

6.6.1 仿造 ····· 459
6.6.2 例子：收音机在房间中播放音乐 ····· 459
6.6.3 声音能量恒定曲线（Constant Power Volume Curve） ····· 461
6.6.4 对声道音量进行进一步控制 ····· 461
6.6.5 在 DirectSound 中播放多声道文件 ····· 462
6.6.6 代价和好处 ····· 462
6.6.7 总结 ····· 462
6.6.8 致谢 ····· 462

第 7 章 网络及多人在线

简介 ····· 464
Scott Jacobs
7.1 3D 动画角色数据的动态自适应流 ····· 465
Thomas Di Giacomo、*HyungSeok Kim*、*Stephane Garchery* 和
Nadia Magnenat-Thalmann
Chris Joslin
7.1.1 简介 ····· 465
7.1.2 背景介绍与相关方法 ····· 465
7.1.3 处理可缩放 3D 数据的准备和实施 ····· 466
7.1.4 自适应数据的传输 ····· 471
7.1.5 总结 ····· 473
7.1.6 参考文献 ····· 473
7.2 大规模多人在线游戏基于复杂系统的高阶架构 ····· 475
Viknashavaran Narayanasamy、
Kok-Wai Wong 和 *Chun Che Fung*
7.2.1 复杂系统和突发性事件 ····· 475
7.2.2 多重架构 ····· 476
7.2.3 基于回馈的决策系统 ····· 484
7.2.4 总结 ····· 485
7.2.5 参考文献 ····· 485
7.3 为游戏物件生成全局唯一标识符 ····· 487
Yongha Kim
7.3.1 游戏物件 GUID 建立的需求 ····· 487
7.3.2 生成 GUID ····· 488
7.3.3 对于特殊情况的处理 ····· 489
7.3.4 总结 ····· 490
7.3.5 参考文献 ····· 490
7.4 利用 Second Life 为大规模多人在线游戏原形设计游戏概念原形 ····· 491

Download at www.Pin5i.Com

Peter A. Smith

7.4.1 简介 ····· 491
7.4.2 为什么要用到 Second Life ····· 491
7.4.3 初试“第二人生（Second Life）” ····· 494
7.4.4 在 Second Life 中的设计要点 ····· 495
7.4.5 原形的开发 ····· 496
7.4.6 一个成功的例子 ····· 498
7.4.7 总结 ····· 499
7.4.8 参考文献 ····· 499

7.5 稳定的 P2P 游戏 TCP 连接及敏感 NAT ····· 501

Larry Shi

7.5.1 问题 ····· 501
7.5.2 技术水平 ····· 502
7.5.3 方法 ····· 503
7.5.4 应用方面 ····· 507
7.5.5 局限性 ····· 507
7.5.6 结论 ····· 508
7.5.7 参考文献 ····· 508

Download at www.Pin5i.Com

第 1 章

通用编程

Download at www.Pin5i.Com

简　介

Adam Lake，英特尔公司软件解决方案小组，3D 图形图像研究所
Adam.t.lake@intel.com

在开发过程的各个方面，游戏开发正在经历着巨大的变革。为了能够从上一代的硬件产品中榨取性能，我们必须专注于汇编指令执行的时间安排，开发使用小矢量指令——例如，SSE（Stream SIMD Extentions，流 SIMD 扩展）指令集——以确保在处理的过程中，数据仍然会保留在寄存器或缓存中。虽然这些问题依然关系重大，但是，与如何开发使用下一代游戏机或 PC 中多线程硬件的这个需求相比，这些问题现在只能处于次要的位置。为了这个目的，我们在这一章节中收录的一些文章，会向大家介绍一些工具和技术，帮助游戏编程人员充分利用这个新的硬件：Pete Isensee 的文章“通过 OpenMP 来充分利用多核处理器的能力”（1.2），以及 Toby Jones 的文章“Lock-Free 算法（锁无关算法）”（1.1）。

我们还收录了两篇关于空间分割的文章：Roger Smith 的文章“游戏对象的地理网格注册”（1.4）；另外一篇文章则详细地实现了流行的空间二叉树分割（BSP）方法，这就是 Octavian Marius Chincisan 的文章——“BSP 技术”（1.5）。根据具体应用环境的不同，我们常常会使用不同的空间分割方案。举个例子，我们可以利用 BSP 方法来解决游戏对象的可见性，但是对于游戏对象的管理，我们则可以使用四叉树，或者是八叉树来实现。这些文章将为读者提供实现自自的 BSP 或八叉树方案所需要的信息。

在这一章中，还有一些作者，根据他们以往项目的经验，撰文描述了他们曾经使用过的一些技术，来改进代码的易维护性和代码质量。Blake Madden 在他的文章“利用 CPPUnit 来实现单元测试”（1.7）中，详细地介绍了一个单元测试套件的实现，还提供了完整的测试用例。Noel Llopis 和 Charles Nicholson 的文章则介绍了游戏资产的热加载（hotloading）技术（1.10），并提供了一种实现的机制，使得我们在游戏开发过程中，当游戏资产被修改后，我们可以进行游戏资产的实时加载。这样一来，就可以大大加快游戏产品推向市场的时间进度。另外有一篇文章可以帮助某些游戏编程人员缩短开发周期。这篇文章介绍了一个算法，来判断最接近的字符串匹配，这就是 James Boer 的文章——“最相似字串匹配算法”（1.6）。在游戏的开发过程中，当游戏美工和程序人员在保存或者调用创建的资源时，如果错误地键入了不正确的文件名，字符串匹配技术就显得非常有用。

对于那些对新一代交互技术感兴趣的读者，我们这里有一篇文章，介绍了如何使用计算机视觉库——OpenCV。在“利用 OpenCV 库来实现游

Download at www.Pin5i.Com

戏中的计算机视觉”（1.3）一文中，Arnau Ramisa 等人向我们展示了如何使用 OpenCV 库来实现：当玩家躲在角落里偷窥的时候，捕捉玩家头部的运动。

对于安全问题，我们收录了 Steve Rabin 的文章：“为游戏的预发布版本添加数字指纹，威慑并侦测盗版行为”（1.8）。作者描述了各种不同的添加数字指纹的方法，开发人员可以用这些方法来对一款游戏添加数字指纹。作者还介绍了各种方法的优缺点，并推荐大家使用一个组合方法，以达到最高的可靠性。

在程序性能方面，我们有一篇文章是关于游戏资产重新排序的内容，并提供了相关的实现代码。这就是 David L. Koenig 的文章——“通过基于访问顺序的二次文件排序，实现更快速的文件加载”（1.9）。考虑到今天的游戏引擎不断增加的加载次数，这篇文章的内容是非常重要的。

本章的这些作者将他们自己独特的看法、个性，以及广博的技术经验，都毫无保留地奉献给广大的读者朋友。我衷心地希望，你能够从这些文章中有所收获。而且这些作者们也鼓励大家将自己的精粹文章拿出来与大家分享。最后祝大家阅读愉快。

Download at www.Pin5i.Com

1.1 Lock-Free 算法

Toby Jones，微软公司
thjones@microsoft.com

虽然游戏编程人员总是将 CPU 逼到其极限状态，但是，他们现在却越来越多地依赖并行处理技术，来获得最大化的程序性能。人们更希望现代的游戏，能够充分利用 PC 机和游戏机（如 Xbox 360）系统中的对称多处理技术（SMP）和芯片多处理技术（Chip Multiprocessing）。

为了获得最高的 CPU 运行效率，我们必须面临的一个挑战是：如何确保所有的 CPU 一直都在进行数据的处理。当多个 CPU 之间需要进行数据共享时，有个难题就出现了。虽然最好是能把 CPU 之间的数据共享最小化，但在大部分情况下，我们根本无法避免多个 CPU 共享数据的情况。传统的技术会引入一些工具，如：Semaphores（信号量）、Mutexes（互斥锁）和 Critical sections（临界段）等，以确保安全地将对数据和资源的访问串行化。

Mutexes（互斥锁）以及其他基于锁（lock-based）的数据结构的使用都是一把双刃剑。它们可以让多个线程安全地存取数据，但是它们的使用却不够灵活，伸缩性差。如果不能获得一个锁定，会造成线程在等待时的自旋，浪费宝贵的 CPU 时钟。最糟糕的情况下，它们会造成昂贵的上下文切换（Context Switch）系统开销。

一个更为现代化的技术是使用 Lock-Free（锁无关）算法。不需要加锁，Lock-Free（锁无关）算法就可以让多个线程访问共享数据。但是，这个算法同时也会将访问操作限制在某些操作之内。从所有的线程会马上执行这个意义上讲，这些算法并不是无等待（Wait-Free）的，但至少有一个线程总是可以执行下去。另外，这个算法不是那么的公平，它不能保证所有的线程可以按顺序访问数据。

Lock-Free（锁无关）算法的一个主要优点是：如果应用在正确的地方，它就可以为我们提供更快的执行速度。这些算法同时也是对死锁免疫的。即使在一个线程执行过程中将其终止，也不会影响其他线程的稳定性。当然，该算法还有很多其他的优点，但是，游戏编程人员通常是因为该算法的速度优势而选择使用。

1.1.1 Compare-And-Swap 及其他通用原语

正确地实现一个 Mutexes（互斥锁）和 Critical sections（临界段），需

Download at www.Pin5i.Com

要我们去使用特定的原子原语，例如“test & set”原语。一个线程可以不断地重复“测试&设置（test & set）”某个内存位，直到最后取得成功。这样就可以确保该线程可以独占式地访问某个资源，直到这个线程将这个内存位清零。但是，对于 Lock-Free 编程，“test & set”原语的功能还是不够强大。

在其开创性的论文[Herlihy9]中，Herlihy 告诉我们，在为 Lock-Free 算法选择原语时，并非所有的原子原语都同样胜任。特别要指出的是，test & set、swap，以及 fetch & add 在计算上并不是十分适用。于是，面对这个挑战，某些原语应运而生，我们称之为“通用原语（universal primitives）”。下面是这些通用原语的列表，以它们近似的效能递增排列：

Compare and Swap（CAS）：用一个已知的值，原子地对内存字（memory word）进行比较。如果比较匹配成功，则用另外一个值来替换内存字中现有的值。

Load Linked/Store Conditional（LL/SC）：LL 加载一个内存字。SC 则在 LL 加载的地址中保存一个整字的内容。前提是这个地址中没有写入其他的值。

Compare and Swap 2（CAS2）：与 CAS 类似，CAS2 是对一个连续的双字地址进行操作。

Double Compare and Swap（DCAS）：与 CAS2 类似，只是比较操作是在两个非连续的整字内存地址间进行的。

Compare and Swap N（CASN）：这是 CAS2 的一个概括式原语，可以操作任意指定数量的连续的整字内存地址。

Multiword Compare and Swap（MCAS）：DCAS 原语的终极概括。该原语可以操作任意数量的非连续的机器整字地址。

CAS 可以很容易地用伪 C（pseudo-C）代码实现，参见程序清单 1.1.1。

程序清单 1.1.1

```
// 这是一个原子原语函数
bool CAS(uint32_t * ptr, uint32_t oldVal, uint32_t newVal) {
    if(*ptr == oldVal) {
        *ptr = newVal;
        return true;
    }
    return false;
}
```

实践中的通用原语

如果所有这些通用原语在所有的硬件上都是可用的，那可就太好了。但是，大部分硬件连这些通用原语的子集都不能支持。一般来说，有的只能支持 LL/SC，或者 CAS，或者是 CAS2。这很不幸，因为很多 Lock-Free 算法都要求高阶的原语，比如 DCAS 原语。

PowerPC 实现了两个指令，用来处理 LL/SC 原语。这两个指令是：Load Word and Reserve Indexed（lwarx）和 Store Word Conditional Index（stwcx）。Lwarx 原语从内存中加载一个整字，并将这个整字的地址保存在一个隐含的寄存器（被称为“预留地址寄存器”）中。对所有的内存存储操作，相应的地址与预留地址进行比较。如果匹配成功，则复位预留地址寄存器。Stwcx 原语则尝试写入这个地址，仅当这个地址与预留寄存器中的地址匹配时，才可以成功写入，然后再将预留寄存器复位。

Download at www.Pin5i.Com

通过 Compare and Exchange（cmpxhg）指令和 Compare and Exchange 8 Bytes（cmpxchg8b）指令，x86 系列可以支持 CAS 和 CAS2 原语。[①]这两个指令将某个内存值与一个寄存器（对于 cmpxchg8b 而言，就是多个寄存器）相比较。如果两个值相等，则将这个内存替换成下一个寄存器的值。

对于现代的 C++编译器，它们最令人满意的一点就是：通常都会包含内联（intrinsics）函数，不需要我们自己去编写汇编代码。这样不但可以提高产品的可移植能力，还可以让编译器进行代码优化。在 Microsoft Visual C++中，我们可以使用_InterlockedCompareExchange（）作为 CAS 的替身来使用。[②]随书光盘中提供了其他 CAS 和 CAS2 的实现代码。

编译优化器可以导致某些多线程代码中常见的问题。Lock-Free 算法在循环中运行，为了正确地进行优化，共享的变量需要用 volatile 来标识。如果可能的话，可以手工地插入一些读写屏障（read-and-write），这是一个更好的方法。

既然现在我们已经有了一些实际的通用原语，我们就可以开始来构建一个 Lock-Free 算法了。

1.1.2 Lock-Free 参数化的堆栈

我们要实现的这个堆栈是一个常见的基于链表的堆栈。每个节点都包含一个值，以及指向下一个节点的指针，参见程序清单 1.1.2。

程序清单 1.1.2

```
template<typename T> struct node {
    T value;
    node<T> * volatile pNext;

    node() : value(), pNext(NULL) {}
    node(T v) : value(v), pNext(NULL) {}
};
```

我们下面定义这个容器类，以及两个原语操作：push（）和 pop（），参见程序清单 1.1.3。

程序清单 1.1.3

```
template<typename T> class LockFreeStack {
    // 注意，这些数据成员的顺序是由 CAS2 默认的
    node<T> * volatile _pHead;
    volatile uint32_t _cPops;

public:
```

[①] 值得注意的是，这个算法是针对 32 位处理器设计的。在 64 位处理器上实现一个 Lock-Free 算法也不会有什么障碍（只要适当的通用原语可用），但是，本书提供的代码取决于实现的细节：所有的指针都是 32 位的。在 x64 系统上，并没有什么原语可以支持 128 位的 cmpxchg，所以，我们必须使用替代的技术来实现 CAS2。有一个选择是使用 32 位的偏移量，作为一个 near pointer。

[②] 作为一种选择，这个函数功能可能会直接内置到平台中。在 Windows 平台上，函数 InterlockedCompareExchange（）是可用的。但是，最好的方法还是使用内联(intrinsics)函数。

Download at www.Pin5i.com

```
    void Push(node<T> * pNode);
    node<T> * Pop();

    LockFreeStack() : _pHead(NULL), _cPops(0) {}
};
```

当我们想要往堆栈中推入一个新的节点时，需要更新这个新节点，使之指向这个堆栈的头节点。然后，我们再将 pHead 指向这个新节点。这样做的目的是为了保持堆栈的完整性。即使其他的线程也试图推入或弹出其他的节点（参见程序清单 1.1.4），也不会出问题。

程序清单 1.1.4

```
template<typename T> void LockFreeStack<T>::Push(node<T> * pNode)
{
    while(true)
    {
        pNode->pNext = _pHead;
        if(CAS(&_pHead, pNode->pNext, pNode))
        {
            break;
        }
    }
}
```

首先，我们将这个节点指向堆栈的头部。接下来，我们原子地检查，看堆栈头部是否发生了变化。如果没有发生变化，我们将_pHead 指向我们的新节点。如果堆栈头部发生了变化，我们就得到新的头部，重新开始。

将一个节点弹出堆栈就是简单地将上述步骤的反过来执行。从堆栈头部第一节点得到后面第二个节点的指针。指针_Phead 指向的就是第二个节点。第一个节点就被提取出来，作为结果返回。从程序清单 1.1.5 中，我们可以看到，这个实现过程是非常简单易懂的，但是其中有一个地方比较费解：

程序清单 1.1.5

```
template<typename T> node<T> * LockFreeStack<T>::Pop()
{
    while(true)
    {
        node<T> * pHead = _pHead;
        uint32_t cPops = _cPops;
        if(NULL == pHead)
        {
            return NULL;
        }

        node<T> * pNext = pHead->pNext;
        if(CAS2(&_pHead, pHead, cPops, pNext, cPops + 1))
        {
            return pHead;
```

Download at www.Pin5i.Com

```
            }
        }
    }
```

从上面我们可以看到，这个实现其实就是 Push()方法的反过程，唯一的例外就是_cPops。之所以需要进行这个额外的检查步骤，其原因就在于 ABA 问题。

1. ABA 问题

基于 CAS 原语的算法所涉及的这个问题与“别名”问题有关。请大家考虑图 1.1.1 中的情况。假设有一个线程正试图从堆栈中弹出节点 A。它得到了一个指向节点 B 的指针，目的是为了将节点 B 更新为新的头节点。如果这个线程正好潜在 CAS 原语之前，先占据了这个点，那么这个堆栈就变得不稳定了。一个新的线程可能会弹出节点 A 和 B，然后再推入节点 A。现在，如果原始线程试图执行其 CAS 原语的话，我们就会看到，头节点指向的仍然是节点 A。所以，它就会尝试让节点 B 成为新的头节点。但问题是，节点 B 根本就不在堆栈中！

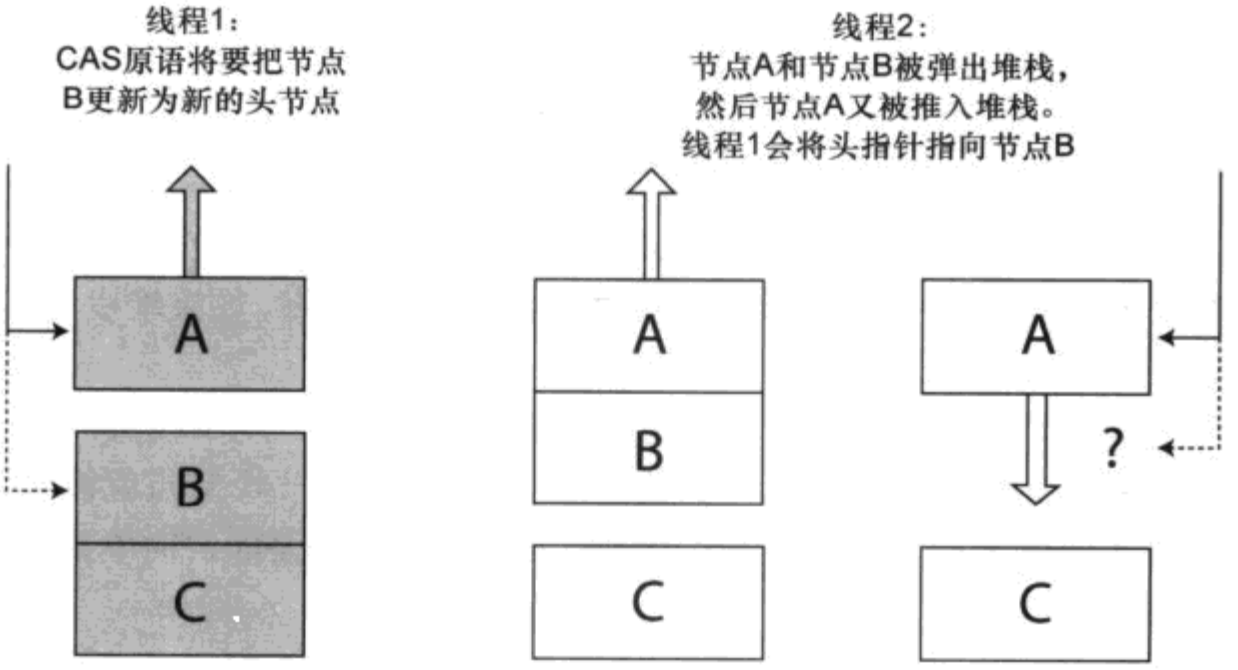


图 1.1.1 CAS 会发现堆栈的头节点并没有变化，以此认定堆栈并没有发生变化。这就是 ABA 问题

解决这个问题的方法是：用一个版本号来标记头节点。一个简单的标记可以计数弹出堆栈的节点个数。如果弹出的节点数目与预期值不匹配，那么就可以判定堆栈的头节点发生了变化，我们必须重新开始。

业界还有一些其他的方法可以解决 ABA 问题。使用 LL/SC 原语就可以完全避免 ABA 问题。因为，在一个竞争的线程中修改头指针会导致保留的指针失效。

2. Lock-Free 技术

到目前为止，一个 Lock-Free 算法所必需的总体概念已经非常明显了。我们必须在类之外的对象中离线地完成我们的操作。完成之后，使用 CAS 原语，将指针指向共享数据。如果 CAS 失败，则重新开始。这个概念虽然很简单，但实践起来却颇有些难度。

Download at www.Pin5i.Com

在上面这个堆栈的例子中，多个线程都可以访问这个堆栈。但是，每个节点中的内容在某一时刻只能由单独一个线程来访问。如果考虑实现对节点内容的并发访问，每个节点就需要额外的 Lock-Free 数据结构，同时还要处理好内存管理的问题。使用 Lock-Free 数据结构来回收被释放的对象，是尤其需要技巧的。但是，这个问题可以通过 hazard 指针来解决[Michael04b]。在随书光盘中的实现代码中，我们对某些难点部分进行了说明。

Lock-Free 的数据存取方法也有一些限制。在基于锁的算法中，并非所有的操作都是可行的。例如，迭代操作的实现就非常困难。这要求每个节点都可以被多个线程访问，还可能要求使用原语，例如 DCAS。而这通常都是不可行的。随着处理器开始在其中内置功能更为强大的通用原语，会有越来越多的操作可供使用。

Lock-Free 算法不能成立在锁定情况下，这一点或许不太明显，但必须解释清楚。特别是所需要的操作不可使用时，人们可能总会很想去试一试。

1.1.3　Lock-Free 参数化的队列

队列是最重要的多线程数据结构之一，应用极为广泛。我们可以很容易地实现一个单链接的 Lock-Free 队列，比前面堆栈的实现复杂不了多少。程序清单 1.1.6 是这个基本的容器类定义。

程序清单 1.1.6

```
template<typename T> class LockFreeQueue {
    // 注意：这些数据成员的顺序是由 CAS2 默认的
    node<T> * volatile _pHead;
    volatile uint32_t _cPops;
    node<T> * volatile _pTail;
    volatile uint32_t _cPushes;
public:
    void Add(node<T> * pNode);
    node<T> * Remove();

    LockFreeQueue(node<T> * pDummy) : _cPops(0), _cPushes(0)
    {
        _pHead = _pTail = pDummy;
    }
};
```

这里我们有了一个简单的队列容器类的典型定义。我们有两个指针，分别指向队列的头部和队列的尾部。为了避免 ABA 问题，我们为每一个队尾提供了一个版本号。概念上最显著的变化是，我们现在要在队列的尾部来增加新节点，而不是在队列的头部。我们还根据[Michael96]的建议，在队列中引入了一个初始节点。

在队列中增加一个新节点比在堆栈中增加一个新节点复杂多了。我们必须同时进行两个操作。我们需要将 pTail 指针指向新节点，同时还要将原来的尾部节点指向新的尾部节点。现在让我们来看一下 Lock-Free 算法带来的复杂度。

如果我们有 DCAS 原语，算法的实现就可以沿用堆栈的例子。但不幸的是，现在还没有哪个硬

Download at www.Pin5i.Com

件可以支持 DCAS。所以，我们需要新的实现方法。在队列这个例子中，可以允许 pTail 出错，但是多个线程会协同更新 pTail 指针（以 Lock-Free 的方式），直到它正确为止，然后我们才能尝试增加新节点。这样一来，我们就只需要去考虑如何自动地更新实际的队尾节点（参见程序清单 1.1.7）。

程序清单 1.1.7

```
template<typename T> void LockFreeQueue<T>::Add(node<T> * pNode) {
    pNode->pNext = NULL;

    uint32_t cPushes;
    node<T> * pTail;

    while(true)
    {
        cPushes = _cPushes;
        pTail = _pTail;

        // 如果_pTail 指向的节点是最后一个节点，
        // 那么，就更新这个节点，使其指向新加入的节点。
        if(CAS(&_pTail->pNext, reinterpret_cast<node<T> *>(NULL),
            pNode))
        {
            break;
        }
        else
        {
            // 由于_pTail 并没有指向最后一个节点，
            // 我们需要一直更新它，直到它指向队列中最后一个节点。
            CAS2(&_pTail, pTail, cPushes, _pTail->pNext,
                                cPushes + 1);
        }
    }

    // 如果_pTail 指向了我们认为的最后一个节点，
    // 那么就更新这个指针，使它指向新增加的节点。
    CAS2(&_pTail, pTail, cPushes, pNode, cPushes + 1);
}
```

这个方法要求 Remove（）知道队尾指针，以防止队列接近空队列的情况。这并不是什么难题，因为 Remove（）自己可以很简单地更新队尾指针。

在程序清单 1.1.8 中，有几个情况需要我们去认真考虑。我们需要确保可以自动地检索所有的局部变量。如果队列看上去快要空了，我们需要去检查 pTail，看是否因为它落在了后面。如果是，那么就重新开始。如果队列已经准备好移除一个节点了，它就拷贝这个值，并尝试调整 pHead。

程序清单 1.1.8

```
template<typename T> node<T> * LockFreeQueue<T>::Remove() {
    T value = T();
    node<T> * pHead;

    while(true)
```

Download at www.Pin5i.Com

```
{
    uint32_t cPops = _cPops;
    uint32_t cPushes = _cPushes;
    pHead = _pHead;
    node<T> * pNext = pHead->pNext;

    // 确定我们不是在其他更新操作的过程之中来得到这些指针
    if(cPops != _cPops)
    {
        continue;
    }
    // 检查队列是否为空
    if(pHead == _pTail)
    {
        if(NULL == pNext)
        {
            pHead = NULL; // 队列为空
            break;
        }
        // 特殊情况，如果队列中确实有节点，但是 pTail 落在后面。
        // 那么就让 pTail 离开队列头部。
        CAS2(&_pTail, pHead, cPushes, pNext, cPushes + 1);
    }
    else if(NULL != pNext)
    {
        value = pNext->value;
        // 移动队头指针，有效地移除节点
        if(CAS2(&_pHead, pHead, cPops, pNext, cPops + 1))
        {
            break;
        }
    }
}
if(NULL != pHead)
{
    pHead->value = value;
}
return pHead;
}
```

在[Fober05]中有一个优化方法，可以不用进行节点的拷贝，但是却损失了免于线程终止的优势。对于高系统开销的拷贝语义的游戏对象，这个优化方法非常重要。

正如你所看到的，特定 Lock-Free 算法的复杂度和可用性，很大程度上取决于特定平台所提供的通用原语的类型。

1.1.4　Lock-Free 参数化的 Freelist

我们现在已经拥有了两个用于共享数据的基本算法。创建正确的 Lock-Free 算法是需要技巧的。通常情况下，在简单 Lock-Free 算法的基础上去实现复杂的 Lock-Free 算法会更有效

Download at www.Pin5i.Com

率。

通过这些工具，我们会创建一个 Lock-Free 的 Freelist（空闲块列表）。内存管理程序使用 Freelist 来跟踪记录那些游戏可以使用的内存块。在这个例子中，我们会构建一个固定大小的 Freelist，但它的特性却非常吸引人，比如减少了内存碎片[Glinker04]。

程序清单 1.1.9 是其完整的实现代码。我们设计的这个 Freelist 可以用 Lock-Free 的方式来创建、销毁对象。与[Glinker04]的方法相比，用 Lock-Free 方式来实现一个 Freelist 的技术不尽相同。因为我们并没有在空闲内存中保存一个指针列表。取而代之的是，我们有自己的 Lock-Free 堆栈管理节点，该节点包含了实际的对象。对象的分配或销毁与进栈、出栈一样简单！

程序清单 1.1.9

```
template<typename T> class LockFreeFreeList {
    LockFreeStack<T> _Freelist;
    node<T> * _pObjects;
    const uint32_t _cObjects;

public:
    LockFreeFreeList(uint32_t cObjects) : _cObjects(cObjects)
    {
        _pObjects = new node<T>[cObjects];
        FreeAll();
    }
    ~LockFreeFreeList()
    {
        delete[] _pObjects;
    }
    void FreeAll()
    {
        for(uint32_t ix = 0; ix < _cObjects; ++ix)
        {
            _Freelist.Push(&_pObjects[ix]);
        }
    }
    T * NewInstance()
    {
        node<T> * pInstance = _Freelist.Pop();
        return new(&pInstance->value) T;
    }
    void FreeInstance(T * pInstance)
    {
        pInstance->~T();
        _Freelist.Push(reinterpret_cast<node<T> *>(pInstance));
    }
};
```

在分配一个新对象时，我们就从堆栈中弹出一个节点。然后使用 placement new（）操作符创建 v-table，并调用对象构造器。FreeInstance（）则简单地调用对象析构器，将节点重新推入堆栈中。

Download at www.Pin5i.Com

如果要考虑对齐的因素，我们可以让节点指针在对象自身上重叠。这样做不会在本质上改变这个算法，但可能会丧失类型安全。

其他技术

在某些情况下，一个 Lock-Free 空闲块列表可以和一个 STL 分配器一样有用。虽然线程安全的分配器不一定会给你一个线程安全的 STL 容器类。但是，如果多个游戏的子系统可以共享一个分配器，我们就可以节省一些系统开销。

这个 Freelist 特别适合分配固定大小的对象，但问题是，我们是否可以用 Lock-Free 的方式来进行常规的内存管理？我们可以采用[Michael04b]的技术来管理对象的生命周期。[Michael04b]介绍了一个非常复杂，但却是 Lock-Free 方式的内存分配器。

1.1.5　总结

Lock-Free 算法的基本概念非常容易理解，但实践起来却是非常的困难。Lock-Free 算法的功能十分强大，但是它们同时也带来了复杂度的开销和操作上的限制。如果使用得当，这些技术可以帮助游戏开发人员从他们的硬件平台上榨取最后的一点性能。

在构建 Lock-Free 算法时，在较简单的 Lock-Free 数据结构（诸如堆栈、队列等）之上进行开发是最简单的。但是，在使用底层的 Lock-Free 技术时，最好先在一个外部对象上来实现大部分的操作，然后再 CAS 进来。CAS 原语是通用原语的基础，这些通用原语是我们创建 Lock-Free 数据结构所必需的。至于这些原语是如何实现的，在不同的处理器上，其实现的方法也各不相同。

Lock-Free 算法囊括了一个新的，发展快速的领域。几乎每个月都会有新的论文发表出来，改进现有的技术，改变以前的概念。

未来是光明的。业界正在研究如何为处理器增加事务型内存（transactional memory）和功能更强大的通用原语。这会为我们打开方便之门，让我们可以开发出更为灵活的算法。C++ 标准协会也正在考察多线程扩展集的提案[Alexandrescu04]。毋庸质疑，这些提案一定会包括 Lock-Free 语义。

1.1.6　参考文献

[Alexandrescu04] Alexandrescu, Andrei, Hans Boehm, Kevlin Henney, Doug Lea, and Bill Pugh, “Memory Model for Multithreaded C++.” C++ Standard Committee, WG21/N1680=J16/04-0120, September 2004.

[Fober05] Fober, Dominique, Yann Orlarey, and Stéphane Letz, “Optimized Lock-Free FIFO Queue Continued.” Laboratoire de Recherche en Informatique Musicale, Technical Report TR-050523, 2005.

[Glinker04] Glinker, Paul, “Fight Memory Fragmentation with Templated Freelists.” *Game Programming Gems 4,* Charles River Media, 2004.

[Herlihy91] Herlihy, Maurice, “Wait-Free Synchronization.” *ACM Transactions on*

Download at www.Pin5i.Com

Programming Languages and Systems, 13(1): pp. 124–149, January 1991.

[Michael96] Michael, Maged and Michael Scott, "Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms." 15th ACM Symp. on Principles of Distributed Computing, May 1996.

[Michael04a] Michael, Maged, "Scalable Lock-Free Dynamic Memory Allocation." The 2004 ACM SIGPLAN Conference on Programming Language Design (June 2004): pp. 25–46.

[Michael04b] Michael, Maged, "Hazard Pointers: Safe Memory Reclaimation for Lock-Free Objects." *IEEE Transactions on Parallel and Distributed Systems,* 15(6): pp. 491–504, June 2004.

1.1.7 相关资源

[Grove04] Groves, Lindsay, Simon Doherty, Mark Moir, and Victor Luchangco. "A Practical Lock-Free Queue Implementation and its Verification." *FM in NZ,* 2004.

[Pugh90] Pugh, William. "Skip Lists: A Probabilistic Alternative to Balanced Trees." *Communications of the ACM,* 33(6):668–676, June 1990.

[Sutter05] Sutter, Herb. "The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software." *Dr. Dobbs Journal.* March 2005.

Download at www.Pin5i.Com

1.2　通过 OpenMP 来充分利用多核处理器的能力

Pete Isensee，微软公司
pkisensee@msn.com

在过去的十年中，CPU 的时钟速度并没有按照我们已司空见惯的发展级数在增长。其原因十分复杂，最好还是留给电子工程师们考虑吧。我们面对的现实情况是。游戏玩家马上就会玩到多处理器的机器上。要想充分利用这些新架构的先进特性，需要我们在软件开发的过程中去使用多线程的方法。游戏开发商们现在正面临着一个复杂的任务如何将他们的游戏引擎“多线程化”。

这个高性能计算的行业开始着手处理这个问题已经有一段时间了。游戏行业常常需要去调度几十个，有时甚至是几百个独立的处理器，协调它们对共享内存的使用。业界现在有很多的解决方案，其中之一就是 OpenMP[OpenMP05]。OpenMP 是一个可移植的、工业标准化的 API，也是一个 C/C++语言的编程协议，可以支持并行程序开发。游戏开发人员们现在所使用的大部分编译器都集成了 OpenMP 技术。本文向大家简要地介绍了 OpenMP 的概况，探讨了如何更好地利用这个技术来提高游戏的性能。

1.2.1　OpenMP 应用实例：粒子系统

假设你的游戏有一个粒子系统，需要在每一个帧中重新计算所有粒子的位置，就像下面的这个循环：

```
for( int i = 0; i < numParticles; ++i )
    UpdateParticles( particle[i] );
```

在多处理器架构下，我们最好将上面这个循环分割成几个部分，这样一来，每个部分就可以在一个独立的硬件线程上完成。举个例子，如果有两个线程，你就可以按照下面这个方式来实现程序功能：

```
// 在线程 0 上运行半个循环
for( int i = 0; i < numParticles/2; ++i )
    UpdateParticles( particle[i] );
// 在线程 1 上运行另外半个循环
for( int i = numParticles/2; i < numParticles; ++i )
    UpdateParticles ( particle[i] );
```

OpenMP 可以帮助你准确地完成上述功能，只需要使用简单的编译器

Download at www.Pin5i.Com

预处理指令#pragma：

```
#pragma omp parallel for
for( int i = 0; i < numParticles; ++i )
    UpdateParticles ( particle[i] );
```

打开了 OpenMP 功能开关的编译器会自动生成相应的代码，将这个循环分割成多个并行的程序段，每个程序段都可以独立地执行。具体分割成几个并行程序段，这个数量取决于所运行的硬件平台、OpenMP 的运行时间，以及编程人员已经设置好的选项设置。

1.2.2 好处

关于这个应用实例，确实有几个突出的特色值得我们大书特书的。首先，这个技术完全是独立于平台的。如果使用的编译器不支持 OpenMP 选项，那么，它就会自动忽略预处理指令#pragma。这段代码仍然可以顺利编译，正常运行。而如果编译器支持 OpenMP 选项，编译器就会自动向目标架构中引入适当的并行结构。如果目标平台并没有多个处理器或硬件线程，OpenMP 的系统开销也可能会非常的小。

第二点，与典型的多线程代码不同，这个代码是可读的。随便谁来看这段代码，他们都可以清晰、准确地看到这个循环将要完成什么样的工作。最后一点，将这个任务并行化，我们只需要一行代码。而使用其他的方法，最好的情况下也需要几十行与平台相关的代码，才能将这个循环多线程化。

很明显，OpenMP 确实好处颇多。我们甚至可以在项目的最后关头来使用 OpenMP 进行优化，因为它可以很容易地添加到现有的代码中。

1.2.3 性能

就你来说，你希望能够从 OpenMP 中得到什么样的性能表现呢？这个答案取决于具体使用的编译器、平台，以及被并行化的代码。但是，如果我们可以考察几个具体的应用实例，就可以对潜在的性能提升有个大致的了解。首先来看看前面的这个例子。假设变量 numParticles 的值等于 100 000，而 GetNewParticlePos 函数只占用很少固定数量的时间（参见随书光盘中的代码实例）。

表 1.2.1 中罗列的是前面那个粒子系统分别在两个不同的系统平台上，使用 Visual Studio 2005 编译器进行编译后的性能指标对比。第一个系统是一台台式电脑，配置了两个 Xeon 处理器，每个处理器有两个超线程。第二个系统是一台 Xbox 360 游戏机，配备了 3 个处理器，每个处理器有两个硬件线程。

表 1.2.1 粒子系统使用 OpenMP 获得的性能提升

硬 件 平 台	OpenMP 的线程数量	OpenMP 性能提升	Windows 线程性能提升
双核 2.3GHz Pentium 处理器	4	2.9 倍	2.9 倍
3 核 3.2GHz Xbox 360	6	4.6 倍	4.5 倍

Download at www.Pin5i.Com

上表中的第二列表示的是 OpenMP 线程组所使用的硬件线程的数量。第三列表示的是用 OpenMP 将这个循环并行化后所获得的性能提升。我们可以看到，这个提升是非常明显的：3～5 倍（取决于不同的目标平台）。由于我们只是改动了一行代码，这个结果是相当不错了！最后一列表示的是我们使用标准线程技术所获得的性能提升。使用 Windows 线程调用和同步原语来编写这个粒子应用，大概需要 60 多行的代码（随书光盘上有提供），而且还要付出相当多的精力去跟踪调试、优化。

如此费事不说，最后的性能提升与我们直接使用 OpenMP 的效果一样。实际上，在 Xbox 360 上，调用 Windows 同步原语的系统开销要高于直接使用 OpenMP。这是因为，OpenMP runtime 可以直接调用内核导出函数（kernel exports）。从这个简单的实验中，我们可以清楚地看到，使用 OpenMP，投入少，收益多。

1.2.4　OpenMP 应用实例：碰撞检测

这里有一个更为复杂的应用实例，可以向我们展示如何在一个典型的碰撞检测循环中来使用 OpenMP。由于这个循环中的每一次迭代都需要消耗一定量的时间。根据物体是否相交，这个时间消耗量也是不同的。因此，我们会使用动态调度技术，来告诉编译器要在运行时调度线程组，而不是简单地在线程之间平均地分割这些迭代。

```
bool anyCollisions = false;

#pragma omp parallel for schedule( dynamic )
for( int i = 0; i < numObjects; ++i )
{
    for( int j = 0; j < i; ++j )
    {
        if( SpheresIntersect( obj[i], obj[j] ) &&
            ObjectsIntersect( obj[i], obj[j] ) )
        {
            anyCollisions = true;
        }
    }
}
```

要想评估这个例子的性能，是一件更为艰难的事情。因为最终的性能取决于多个变量，包括彼此相交的物体数量，以及计算每个相交所需要的时间。为了这个例子描述的方便，我们假设变量 numObjects 等于 1 000；SphereIntersect 函数只需要很少，固定数量的时间；而 ObjectsIntersect 需要的时间是 SphereIntersect 函数的 100 倍。我们还假设球面相交率为 10%，物体相交率为 1%。在这些条件下，表 1.2.2 列出了相应的性能指标。

表 1.2.2　　碰撞检测使用 OpenMP 获得的性能提升

硬 件 平 台	OpenMP 的线程数量	OpenMP 性能提升
双核 2.3GHz Pentium 处理器	4	3.3 倍
3 核 3.2GHz Xbox 360	6	5.4 倍

Download at www.Pin5i.Com

通常来讲，无论从文件大小，还是从速度上来说，OpenMP 只增加很少的运行时系统开销。Visual Studio OpenMP DLL 的大小只有 60KB。在这个特定的例子中，性能已经提升到了理论上的最大值。举个例子，在一个有 6 个硬件线程的 Xbox 360 平台上，并行化版本的执行速度已经达到了串行版本的 5.4 倍。

1.2.5 线程组

OpenMP 是基于一个非常简单的概念：线程组。当程序进入一个并行代码区段时，线程组就开始工作了。当线程完成它们各自的工作后，线程组又开始休息，等待下一个并行代码区段。从某种意义上讲，线程的创建工作还是需要相当的系统开销。因此，大部分的 OpenMP 实现（包括 Visual Studio）都在第一次使用线程的时候将其创建，并在整个程序中重复使用这个线程组。

OpenMP 允许嵌套的并行程序区段。单独的线程可以生成自己的线程组。每个线程组中线程的数量也是可以通过编程来配置。在前面两个表中，“OpenMP 的线程数量”这个表单项表示的就是：在给定的平台上，这个线程组中最大的线程数量。我们可以通过 API：omp_get_max_threads（），来获得这个数字。

1.2.6 函数的并行化

一般来讲，OpenMP 都是用于数据并行化的。也就是说，像前面那两个例子一样，将循环代码并行化。但是，OpenMP 也可以用于函数级的并行化。我们来看看下面这个 QuickSort 的实现代码：

```
template< typename T >
void qsort( T* arr, int lo, int hi )
{
    if( hi <= lo )
        return;
    int n = partition( arr, lo, hi );
    qsort( arr, lo, n-1 );
    qsort( arr, n+1, hi );
}
```

所有对 qsort 函数的递归调用都是彼此独立的。这是因为它们分别运行于数组中彼此孤立的区段。这绝对是个完美的机会，可以将这些调用并行化。一种方法是将独立的递归调用分到它们各自独立的 OpenMP 并行代码区段中：

```
#pragma omp parallel sections
{
    #pragma omp section
    qsort( arr, lo, n-1 );
    #pragma omp section
    qsort( arr, n+1, hi );
}
```

Download at www.Pin5i.Com

这样，对 qsort 的所有递归调用都被安排在它们各自的 OpenMP 代码段中。为了并行地执行这些调用，我们将它们包在大括号中。这样，OpenMP 就知道哪些代码段需要并行执行。但是，在大多数平台上，递归并行代码的系统开销很可能会抵消掉 OpenMP 所能带来的性能提升。这里还有一个更完善的解决方案：计算出一系列的代码分区，然后针对这些代码分区进行高层次的并行化处理。如果有一个得当的代码分区函数，每个分区的大小会大致相同。由此获得的性能提升完全值得我们去花这些工夫。

```
template< typename T >
void qsortParallel( T* arr, int lo, int hi )
{
    if( hi <= lo )
        return;
    int n = partition( arr, lo, hi );  // 一级分区
    int m = partition( arr, lo, n-1 ); // 二级分区
    int q = partition( arr, n+1, hi );

    #pragma omp parallel sections
    {

        #pragma omp section
        qsort( arr, lo, m-1 );      // 串行 qsort

        #pragma omp section
        qsort( arr, m+1, n-1 );

        #pragma omp section
        qsort( arr, n+1, q-1 );

        #pragma omp section
        qsort( arr, q+1, hi );
    }
}
```

至于这个函数的性能评估，我们可以假设随机整数的数组大小是 1 000 000，那么相关的性能指标如表 1.2.3 所示。

表 1.2.3　　qsort 函数使用 OpenMP 获得的性能提升

硬 件 平 台	OpenMP 的线程数量	OpenMP 的性能提升
双核 2.3GHz Pentium 处理器	4	1.5 倍
3 核 3.2GHz Xbox 360	6	1.4 倍

1.2.7　缺陷

与其他所有技术一样，OpenMP 也有它自己的缺陷。首先，OpenMP 的设计并不能解决所有的多线程问题。复杂的多线程场景和复杂的同步化操作，最好还是使用本地线程技术（native thread）。如果你想编写一个新的游戏引擎，OpenMP 可能并非上佳之选。

Download at www.Pin5i.Com

其次，支持 OpenMP 的编译器一般不会去进行检测，以确保可以正确地将你的代码并行化。举个例子，假设我们很天真地想去并行化上面那个 qsort 例子中的代码分区：

```
// 有害的 OpenMP 代码，千万别这样做
int n, m, q;

#pragma omp parallel sections
{

    #pragma omp section
    n = partition( arr, lo, hi );

    #pragma omp section
    m = partition( arr, lo, n-1 );      // 噢，不行，变量 n 可能没有被定义

    #pragma omp section
    q = partition( arr, n+1, hi );      // 噢，不行，我们可能没有定义变量 n
}
```

大部分的编译器在编译这段代码的时候都不会报错。但这里的问题是：第二个代码段与第三个代码段，都依赖于第一个代码要完全执行完毕，才能获得变量 *n* 的一个合法的值。但是，OpenMP 并不能保证并行代码段按照这个顺序去执行。于是，这个代码段的最后结果是无法确定的，很可能的结果是应用程序崩溃。确保只将 OpenMP 应用于顺序无关的代码上，这是编程人员的责任。这一原则不仅仅适用于那些并行代码段，也适用于处理有顺序依赖关系的循环代码。记住，一定要将并行环境中的代码运行结果，与单线程环境下该程序的运行结果进行校验。

OpenMP 另外一个缺陷就是调试。当编译器碰到一个 OpenMP 程序段时，它就会生成定制的代码，以调用 OpenMP 运行时间库。但不幸的是，OpenMP 的内核就像一个黑匣子。对于有些编译器和调试程序，这会使我们的调试工作变得异常困难。

最后一点，使用 OpenMP 并不能保证你一定能在多处理器系统中获得性能的提升。根据你具体的用法，OpenMP 运行时的系统开销可能会抵消掉它所带来的好处。例如，在第一个例子中，当变量 numParticles 的值达到 100 的数量级时，OpenMP 所带来的性能提升就可以忽略不计了。对于任何一种优化技术，我们一定要将其剖析清楚，核实其带来的性能提升。

1.2.8 结论

OpenMP 是一个快速、有效的优化技术，可以帮助我们充分利用多核处理器和超线程处理器。它的使用非常简单，可以应用在最后关头的优化工作中。OpenMP 也非常的灵活，可以应用在跨平台的代码中。但是，人无完人，OpenMP 也有其自身的缺点。但是，如果使用得当，OpenMP 会是多线程竞技场上一个非常优秀的工具。

OpenMP 在游戏领域更多潜在的应用包括：粒子系统、蒙皮、碰撞检测、仿真、寻路、顶点变换、信号处理、程序化合成（Procedural Synthesis），以及分形算法等。

虽然 OpenMP 的出现已经有一段时间了，但对大多数游戏编程人员来说，它仍然是一个

Download at www.Pin5i.Com

新技术。最好的学习参考资料是 OpenMP 的使用规范说明，你可以从这个网站找到相关资料：http://www.openmp.org。这个规范说明非常简洁，可读性极好。还有一个好的参考资料就在你手边，就是你所使用的编译器的说明文档。

1.2.9　参考文献

[OpenMP05] OpenMP C/C++ specification. Available online at *http://www.Openmp.org*.

1.2.10　相关资源

[Chandra00] Chandra, Rohit, *Parallel Programming in OpenMP,* Morgan Kaufmann, 2000.

[Gaitlin05] Gaitlin, Kang Su and Pete Isensee, "Writing Multithreaded Applications with OpenMP in Visual Studio 2005," *MSDN Magazine* (Oct 2005): pp. 78–91.

[Isensee05] Isensee, Pete, "Effective Use of OpenMP in Games," Game Developer Conference 2005, available online at *http://www.c,pevents.com/Sessions/GD/EffectiveUse.ppt.*

[Sutter04] Sutter, Herb, "The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software," *Dr. Dobb's Journal,* March 2005: pp 16–22, available online at *http://www.gotw.ca/publications/concurrency-ddj.htm*.

Download at www.Pin5i.Com

1.3 用 OpenCV 库实现游戏中的计算机视觉

Arnau Ramisa，西班牙科学研究院人工智能研究所（CSIC）
aramisa@iiia.csic.es
Enric Vergara，西班牙加泰罗尼亚理工大学图形图像信息处理系
evergara@lsi.upc.edu
Enric Marti，西班牙巴塞罗那自治大学计算机科学系计算机视觉研究中心
enrich@cvc.uab.es

1.3.1 引子

数字摄像头的价格越来越便宜，很多人都买得起了。于是，在过去 5 年中，摄像头已经成为很多普通家庭中一个常见的计算机外设产品。但是，计算机的这些“眼睛”还只是被当作视频会议的工具。在深入挖掘摄像头作为输入设备的潜力这一点上，我们做的远远不够。幸运的是，已经有一些厂商开始行动起来，尝试去改变这种状况，像索尼公司的 EyeToy 以及 Camgoo。

1.3.2 游戏中的计算机视觉

在新兴的计算机视觉游戏市场中，人们将所有的注意力都集中在那些只能通过网络摄像头才能玩的游戏上。而这些游戏却忽视了传统的计算机输入设备。这也是合理的，这个新的人机交互接口可以容纳（或者可以处理）玩家的整个身体信息。但是，对于那些通过键盘、鼠标和游戏杆操作的传统游戏，它们也可以从数字摄像头所提供的丰富的信息源中有所受益。面部识别、跟踪，以及姿态识别等领域发展迅猛，期待着有人能找到恰当的方法，将它们应用在计算机游戏中。这些新兴的技术，以及我们在这些年所经历的数字摄像头的发展，为我们打开了无限可能的空间，拓宽了我们人类与机器，特别是与计算机游戏，进行交互的方式。

为了说明计算机视觉技术在传统游戏中的应用，也为了向读者介绍 OpenCV 库，我们在本文中提供了一个简单的例子。

1.3.3 开放的计算机视觉库

英特尔公司的 Open Computer Vision Library（OpenCV，开放的计算机视觉库）是一个功能强大、运行速度快、且易于使用的算法和数据类型的

Download at www.Pin5i.Com

集合。这些算法和数据类型都是用 C/C++编写的，目的是为了帮助我们开发实时的计算机视觉应用[OpenCV04]。

这个库所覆盖的领域包括：影像统计、数学形态学、轮廓检测和处理、直方图、光流（optical flow），以及姿态识别等很多很多的领域。本文介绍的这个项目可以在 Sourceforge.net 上找到，网站里面有全部的源代码、说明文档和应用演示。另外，在 Yahoo Groups 中还有一个非常活跃的 Web 群组，大家在里面就 OpenCV 这个库，彼此交换信息、bugs、经验、问题及相关的答案。

在游戏中应用计算机视觉有一个主要的缺陷，这就是由此产生的计算开销。OpenCV 库是针对 Intel IPP（Intergrated Performance Primitives）[InteLIPP]的使用而设计的，并面向 Intel 公司 Pentium 系列器进行了优化。因此，简单、稳定的计算机视觉应用应该不会降低游戏的帧速。

1.3.4　计算机视觉在游戏中一个简单的应用

在很多游戏中都提供了这种功能选择：玩家可以小心谨慎地靠在拐角处，探出头去，看是否对面有敌人。但不幸的是，这些类型的游戏中需要使用的按键数量非常大，使得玩家很难充分利用这个特性。如果能够利用头部动作去映射游戏中要执行的相同动作，就可以增加玩家的沉浸感。实际上，当我们沉浸在游戏中时，我们会经常地移动我们的头部和身体。在本文中，我们提出了一个简单的计算机视觉算法，可以利用玩家头部的运动去触发前面所说的这个动作。

我们提出的这个算法可以检测到图像中那些与皮肤颜色一样的像素，然后分割出玩家的头部，最后返回一个与玩家头部在摄像头图像中的位置相关的返回值。这个算法是基于 OpenCV 的示范程序“camshiftdemo.c”，并根据我们的要求进行了简化。

我们可以对这个算法进行如下的概括：首先，我们可以捕捉到一个校准帧的图像，然后根据这个图像创建一个直方图，以便确定玩家皮肤的颜色值。为了提高精确度，我们选择了三个不同的区域，作为考察的对象。然后，找到下一帧中所有像素能成为玩家皮肤像素的概率，作为与像素颜色相对应的直方图中柱体的值。利用这些信息，我们就可以过滤掉那些概率最小的像素。最后，针对捕捉到的图像中我们上面所选择考虑的三个区域，计算出这三个区域中过滤过的像素的数量。我们选择其中数量最大的那个区域。

要知道，我们是要将这个算法应用在游戏中，所以，将所有的代码和变量封装在一个类（class）中，这绝对是个好主意。这样可以使这个算法更便于移植，易于使用。另外，根据这个应用的特性，单例类（singleton class）是非常理想的选择：我们只希望一个对象可以使用这个摄像头。而且，这样也可以让我们在这个应用中的任何地方使用摄像头。虽然这里提供的这个例子是一个独立的程序，但是，读者朋友们可以在随书光盘中找到一个面向对象的实现程序。

在我们开始编写程序之前，我们应该首先安装 OpenCV 库，然后新建一个项目（project），其中包含所有必需的库文件和头文件。随书光盘中有一个 Visual C++ 7 配置好的 workspace 文件。

Download at www.Pin5i.Com

为了测试这里提供的代码，在终端模式下的一个简单的C++项目就足够了。这个项目文件的配置如下：所需要的OpenCV头文件是cv.h、cxcore.h，以及highgui.h。这些文件所在的目录也应该添加到这个项目的附加头文件目录中。另外，我们必须将cv.lib、cxcore.lib，以及highgui.lib，作为附加库包含进来。而且它的目录也必须作为附加库目录添加进来。最后，我们必须将cv.dll、cxcore.dll，以及highgui.dll拷贝到项目目录中。

在程序的第一行中，我们必须说明所需要的全局变量、函数原型，并启动main函数。

```
#include "cv.h"
#include "cxcore.h"
#include "highgui.h"
#include <iostream>

#define ESC_KEY 27

CvCapture* capture = NULL;
IplImage* image, *frame, *mask, *HSVimage, *h_plane, *s_plane,
    *v_plane, *backProj;
CvHistogram* hist = NULL;
int userPos;
bool flag=true;
void paintEllipse();
void calcHistogram();
bool detectionLoop();
void showImage();
```

```
int main()
{
```

接下来，我们需要初始化摄像头，开始捕捉图像帧。OpenCV中的HighGUI库提供了几个不错的函数，可以完成这个工作。从摄像头中捕捉图像，这个工作非常简单：

```
int numCam=-1;
if( !(capture = cvCaptureFromCAM( numCam )) )
{
    std::cout<<"ERROR: Camera can't be initialized."<<std::endl;
    return -1; //error
}
```

上面这段代码中的变量numCam指的是我们将要使用的摄像头的标识代码。缺省的值为0，但是，如果只有一个摄像头，或者使用哪个都可以，我们就可以将这个值设置为-1。一旦摄像头的初始化工作结束，我们就可以开始捕捉图像帧了：

```
// 说明部分
cvNamedWindow("WEBCAM",1);

// 第一次捕捉代码段（完成变量的初始化）
frame = cvQueryFrame( capture );
if( !frame ) return false;
image = cvCreateImage( cvGetSize(frame), IPL_DEPTH_8U, 3 );
```

Download at www.Pin5i.Com

```
//CreateImage( Image sizes, pixel depth, channels)
image->origin = frame->origin;

// 主循环代码段
for(;;)
{

    frame = cvQueryFrame( capture );
    if( !frame ) return false;

    cvCopy( frame, image, 0 );

    paintEllipse();

    cvShowImage( "WEBCAM", image );

    char c;
    c = cvWaitKey(10);          // 等待 10 毫秒，看是否有键按下
    if( c == ESC_KEY )
        break;
}

calcHistogram();

detectionLoop();

// 初始化成功
cvReleaseCapture(&capture);
cvDestroyWindow("WEBCAM");
cvReleaseImage(&image);
cvReleaseHist(&hist);
cvReleaseImage(&HSVimage);
cvReleaseImage(&h_plane);
cvReleaseImage(&v_plane);
cvReleaseImage(&s_plane);
cvReleaseImage(&backProj);

return 0; //no error
}
```

在上面的代码中，主循环会从摄像头中获得一帧图像，并显示在一个我们所创建的名为 WEBCAM 的窗口中。当我们按下 ESC 键时，循环终止。函数 cvQueryFram 返回一个指针。该指针指向的是从摄像头中捕获的最后一个图像帧。我们将其拷贝到变量 image 中，因为返回的 frame 指针无法修改或删除。

下一步我们要做的事情是，获得玩家皮肤的一个采样，以创建一个直方图。后面各帧中所有的像素都要和这个直方图进行比对。为了简单起见，避免使用鼠标事件和回调函数，我们提出了一个比较基础的方法，就是在图像上画一个椭圆，将玩家面部放到这个椭圆的中央位置。还有一个更为复杂的方法，大家可以参考 OpenCV 中提供的“camshiftdemo.c”

Download at www.Pin5i.Com

示范程序。

下面这个函数就是用来实现我们提出的“椭圆法”：

```
void paintEllipse()
{
    cvEllipse(image,cvPoint(image->width/2,image->height/2),
        cvSize(image->width/6,image->height/3),0,0,
        360,CV_RGB(255,0,0),2);
    cvFlip(image,NULL,1);          // 为了最简单的可视化操作，将图像进行翻转
}
```

这样，图像的中央位置上就会出现一个红色的椭圆。然后，玩家应该将他们的面部图像调整到这个椭圆中。这里要注意的是：背景中的像素一定不能出现在这个椭圆内部。如果把头发也排除在这个椭圆之外，效果会更好。当玩家准备妥当，就可以按下 ESC 键。图 1.3.1 和彩插 1（上）是这个过程的一个例子。

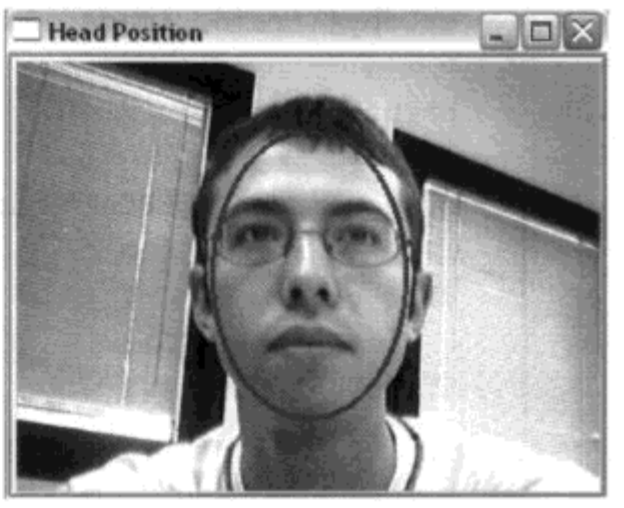


图 1.3.1 通过对玩家皮肤的采样，来创建直方图

现在我们已经有了一个采样图像，我们就可以用下面这个函数来创建直方图：

```
void calcHistogram()
{
    //  直方图计算代码

    cvFlip(image,NULL,1); // 将图像翻转回正常状态
mask=cvCreateImage(cvGetSize(image),IPL_DEPTH_8U,1);
cvZero(mask);
mask->origin = image->origin;
cvEllipse(mask,cvPoint(image->width/2,image->height/2),
    cvSize((image->width/6)-5,(image->height/3)-5),0,0,
        360,cvScalar(255),CV_FILLED);

HSVimage=cvCreateImage(cvGetSize(image),IPL_DEPTH_8U,3);
HSVimage->origin=image->origin;

cvCvtColor( image, HSVimage, CV_RGB2HSV );

h_plane=cvCreateImage(cvGetSize(image),IPL_DEPTH_8U,1);
s_plane=cvCreateImage(cvGetSize(image),IPL_DEPTH_8U,1);
v_plane=cvCreateImage(cvGetSize(image),IPL_DEPTH_8U,1);
h_plane->origin = image->origin;
s_plane->origin = image->origin;
v_plane->origin = image->origin;

cvCvtPixToPlane( HSVimage, h_plane, s_plane, v_plane, 0 );
IplImage* planes[] = { h_plane };
int h_bins = 32;
int hist_size[] = {h_bins};
```

Download at www.Pin5i.Com

```
    float h_ranges[] = {0, 180};
    float* ranges[] = {h_ranges};

    hist=cvCreateHist(1,hist_size,CV_HIST_ARRAY,
        ranges,1);
    cvCalcHist( planes, hist, 0, mask );
}
```

我们在代码开始部分创建的图像 mask 主要是用来过滤掉图像中那些我们不想考虑的像素，也就是红色椭圆之外的那些像素。

接下来，我们将图像的颜色表示从 RGB 模式切换到 HSV 模式。HSV 的颜色空间[HSVspace]可以被考虑成一个圆柱体，每种颜色由三个值来定义：Hue（色度）、Saturation（饱和度）和 Value（亮度）。Hue（色度）是颜色的类型，以 0 度到 360 度的角度值来表示主波长（例如，红、绿、黄）。（在 OpenCV 库中，这个值被分割成两部分，以适应 8 个字节的字长）。Saturation（饱和度）是图像的纯净度，或者说颜色中灰色的数量。在颜色空间中，这个值是用距离圆柱体中心的距离来表示的。最后一个，Value（亮度）表示的是颜色的亮度，或者说是颜色在圆柱体中的高度。

我们之所以要改变颜色空间，是因为在 RGB 的表示模式中，光照的变化会影响 R（红色）、G（绿色）、B（蓝色）三个值。这样我们就无法将采集的图像与参考直方图进行比较了。但是，在 HSV 颜色空间中，Hue 的值是不会随着光照的变化而变化的[Sandeep02][Sedlacek04]。

在下面这个代码段中，HSV 图像的三个色道被分割成一个平面的三个图像。每个图像代表一个颜色成分。然后我们就可以创建直方图了。直方图中的柱体数量（h_bins）对应的是这个直方图在某个维度上可能的值。这里，我们使用了 32 个柱体。这就意味着像素的 H 值被采样分割成 4 组。对像素值的采样过程是很有趣的。由于我们考虑的是一个范围内的值，而不是一个具体的值，这样我们就能去处理 Hue 中微小的变化，也因此可以带来更为稳定可靠的表现。变量 h_ranges 对应的就是一个从图像中采集的像素的范围值。在这例子中，这个范围是[0，180]（回忆一下前面我们所说的，Hue 的值是一个角度，被分割成两部分，以适应 8 个字节的字长）。最后，直方图被创建出来，并填充了 H 平面的值，并用 mask 进行比较过滤。

现在我们可以开始检测部分的工作了。

```
bool detectionLoop()
{
    // 检测循环代码
    backProj=cvCreateImage( cvGetSize(image), IPL_DEPTH_8U, 1 );
    backProj->origin = frame->origin;
    char c;

    for(;;)
    {
        frame = cvQueryFrame( capture );
        if( !frame ) return false;

        cvCopy( frame, image, 0 );
```

Download at www.Pin5i.Com

```
cvCvtColor( image, HSVimage, CV_RGB2HSV );
cvCvtPixToPlane( HSVimage, h_plane, s_plane, v_plane, 0 );
IplImage* planes[]={h_plane };

cvZero(backProj);
cvCalcBackProject( planes , backProj, hist );

CvScalar mean=cvAvg( backProj);
cvThreshold( backProj,backProj,floor(mean.val[0]),
    255, CV_THRESH_BINARY );

int vol[3]={0,0,0};
cvSetImageROI( backProj,cvRect(0,image->height/4,
    image->width/3,image->height*3/4) );
vol[0]=cvCountNonZero( backProj );

cvSetImageROI( backProj,cvRect(image->width/3,
    image->height/4,image->width/3,image->height*3/4) );
vol[1]=cvCountNonZero( backProj );

cvSetImageROI( backProj,cvRect(image->width*2/3,
    image->height/4,image->width/3,image->height*3/4) );
vol[2]=cvCountNonZero( backProj );
cvResetImageROI( backProj );
```

```
            if(vol[0]>vol[1] && vol[0]>vol[2]) userPos=-1;
            else if(vol[1]>vol[0] && vol[1]>vol[2]) userPos=0;
            else userPos=1;

            // 可视化代码
            showImage();
            c = cvWaitKey(10);
            if( c == ESC_KEY ) break;
            if( c == 'b' ) flag=flag?0:1;
        }
    }
```

在这里，我们又一次用到了函数 cvQueryFrame 来捕捉图像帧，并拷贝到变量 image 中。接下来，我们将这个图像转换为 HSV 颜色空间，并分割成几个平面。函数 cvCalcBackProject 给 backProj 中的所有像素进行赋值操作。相应的赋值就是：在图像 h_plane 中同一个像素给定的一个值，在直方图中相应这个值的柱体的值。简单地说，在参考图像中，像素的 Hue 值出现的频率越高，那么指定给图像 backProj 的值也就越大。为了避免噪声和无关像素，我们对反向投影图像使用了极限值限定，去掉了所有在 backProj 平均值之下的像素。然后，我们计算得到三个定义窗口（左视窗、中央视窗和右视窗）中的有效像素。图 1.3.2 和图 1.3.3 向我们演示了这个过程的几个步骤。

为了避免去计算玩家的肩部，我们不去考虑图像最下面四分之一部分的像素。接下来，我们将计算出来的结果进行排序。值最大的那个对应的就是玩家头部的位置。

Download at www.Pin5i.Com

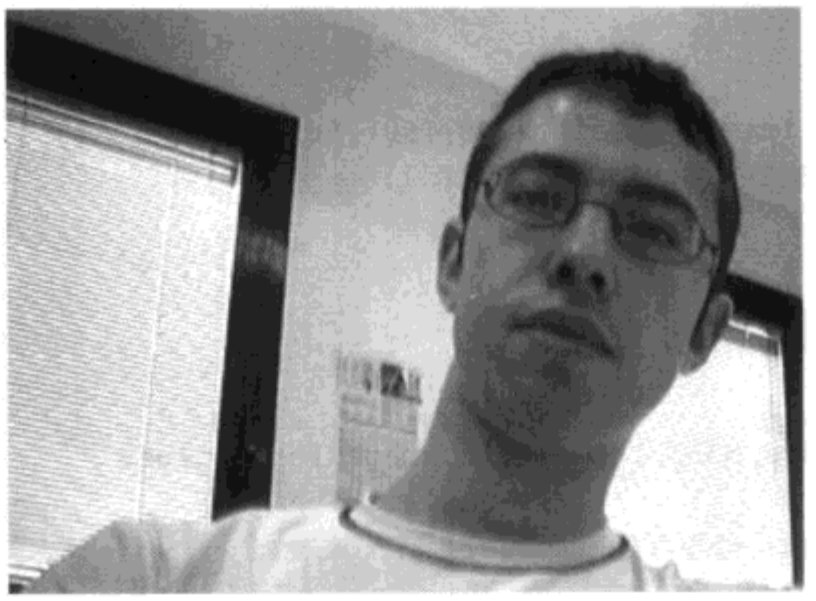


图 1.3.2　作者面部的 HSV 颜色空间图像

最后，为了看一看这个算法的输出结果，我们调用下列函数。这个函数产生的结果类似于图 1.3.4，以及彩插 1 中下面那个图的效果。

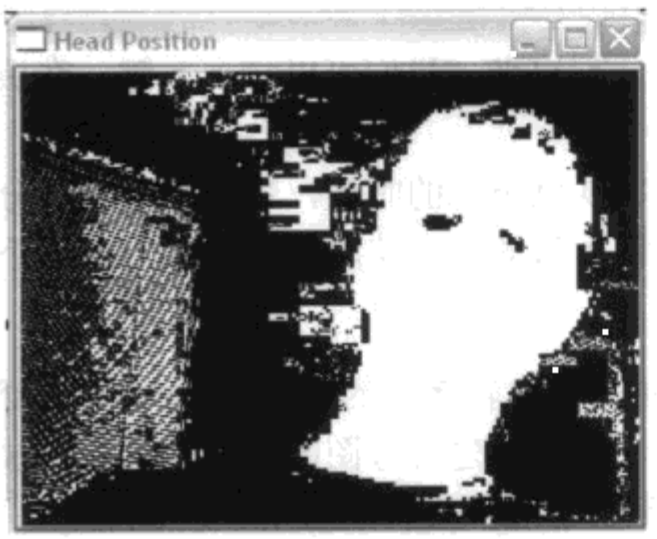


图 1.3.3　作者面部图像的反转图像

图 1.3.4　检测到玩家的面部

```
void showImage()
{
    cvRectangle( image, cvPoint(0,image->height/4),
        cvPoint(image->width/3,image->height), CV_RGB(0,0,255) );
    cvRectangle( image,cvPoint(image->width/3,image->height/4),
        cvPoint(image->width*2/3,image->height), CV_RGB(0,255,0) );
    cvRectangle( image, cvPoint(image-> width*2/3,image>height/4),
        cvPoint(image->width,image->height), CV_RGB(255,0,0) );

    switch(userPos)
    {
    case -1:
        cvCircle( image, cvPoint(50,190),40,
            CV_RGB(0,0,255),CV_FILLED );
    break;
    case 0:
        cvCircle( image, cvPoint(159,190),40,
```

Download at www.Pin5i.Com

```
                CV_RGB(0,255,0),CV_FILLED );
        break;
        case 1:
            cvCircle( image, cvPoint(270,190),40,
                CV_RGB(255,0,0),CV_FILLED );
        break;
        }

        if(flag)
        {
            cvFlip(image,NULL,1); //flip image for easiest
                                  //visualization
            cvShowImage("WEBCAM", image);
        }
        else
        {
            cvFlip(backProj,NULL,1); //flip the image
            cvShowImage("WEBCAM", backProj);
        }
    }
```

为了检验这个算法是否可以将玩家的皮肤正确地分割出来，我们可以按下 b 键，在摄像头图像和反转图像之间切换。

由于这个方法的本质所限，当场景中的物体和玩家肤色类似的时候，系统就会出现分类错误，并影响到最后的结果。所以，最好是把这样的物体从场景中移除。另外，对于这个算法，非常突然的光照变化也是非常危险的。

我们实际的应用表明，如果硬件中使用的是比较老式的、低分辨率的摄像头，或者光照的条件不充分，在利用 H 平面和 S 平面创建直方图的时候，这个算法会更为稳定、可靠。你可以在随书光盘中看到这个算法变种的实现代码。

最后的算法实现在游戏中的实际效果可以参考图 1.3.5 和图 1.3.6，以及彩插 2 和彩插 3。从中我们可以看到玩家在游戏中的替身形象会随着实际玩家的动作而做出相应的反应。

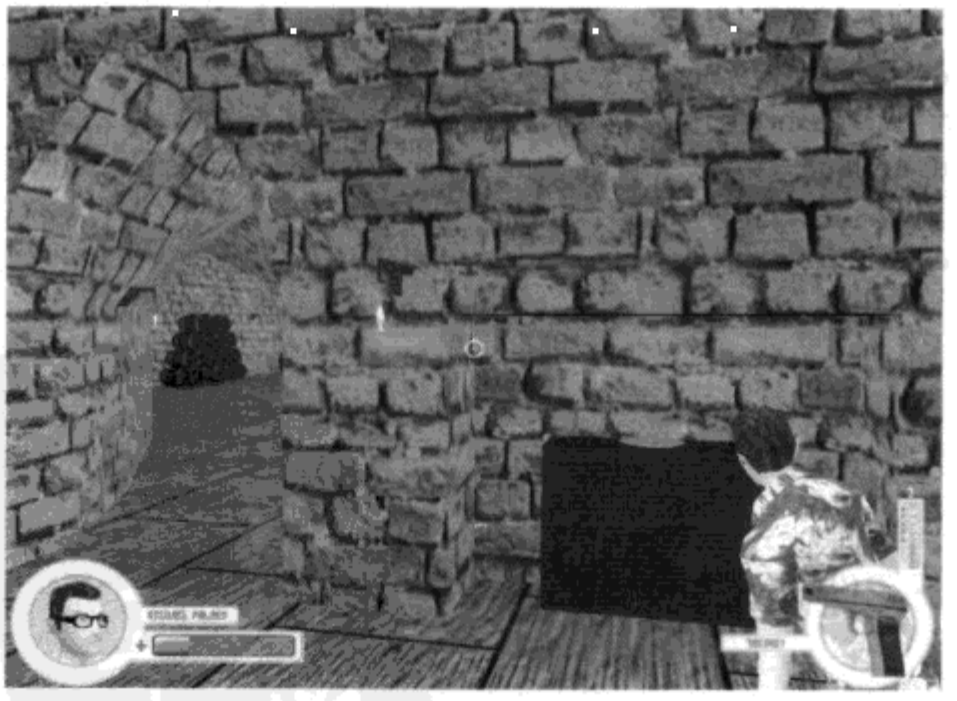


图 1.3.5 玩家向左倾斜

Download at www.Pin5i.Com

图 1.3.6 玩家回到中央位置

这个算法有一个面向对象的版本，已经应用在一款第三人称的动作射击游戏中，实际效果也非常不错。

1.3.5 未来的工作

虽然在特定的环境下，这个算法的实际运行结果还比较满意，但还有很多地方需要加以改进。例如，光流利用，检测相连的成分；利用背景扣除法，完善肤色检测方法[Vezhnevets03]，或者利用计算机视觉领域其他的技术。另外，我们也鼓励读者朋友们去探索计算机视觉世界，找到新的方法去拓展玩计算机游戏的概念。

本文的这个课题得到了西班牙科技部的支持，项目号 TIC2003-09291。另外，我们还要感谢西班牙庞裴法布拉大学视听研究所的大力支持。

1.3.6 参考文献

[HSVspace] Wikipedia article on the HSV color space. Available online at *http://www.en.wikipedia.org/wiki/HSB_color_space*.

[IntelIPP] More information about the Intel Integrated Performance Primitives is available online at *http://www.intel.com/cd/software/products/asmo-na/eng/perflib/ipp/index.htm* and *http://www.support.intel.com/support/performancetools/libraries/ipp/sb/cs-010656.htm*.

[OpenCV04] Intel's Open Computer Vision Library beta 4 (August 13, 2004). Available online at *http://www.sourceforge.net/projects/opencvlibrary*.

[Sandeep02] Sandeep, K. and A. N. Rajagopalan, "Human Face Detection in Cluttered Color Images Using Skin Color and Edge Information." 2002. Available online at *http://www.ee.iitb.ac.in/*

Download at www.Pin5i.Com

~*icvgip/PAPERS/166.pdf*.

[Sedláček04] Marián Sedláček, "Evaluation of RGB and HSV models in Human Faces Detection." 2004. Available online at *http://www.cg.tuwien.ac.at/student work/CESCG/CESCG-2004/web/Sedlacek-Marian*.

[Vezhnevets03] Vezhnevets, V., V. Sazonov, and A. Andreeva, "A survey on pixel-based skin color detection techniques." Graphicon-2003, Moscow, 2003.

Download at www.Pin5i.Com

1.4　游戏对象的地理网格注册

Roger Smith，Modelbenders 有限责任公司
gpg@modelbenders.com

1.4.1　引子

大部分的游戏对象只对两个问题感兴趣："我能看到谁？"和"我能射到谁？"如果这两个问题是由一个计算机控制的对象（AI 主体，或者是一个 NPC）所提出的，它们就提出了一个技术难题：如何判断哪个游戏对象处于自己的射程之内？很不幸的是，在有些时候，这个难题就意味着我们要搜遍整个游戏对象的列表，针对每一个游戏对象都去进行瞄准线测试，或者是射程范围测试。但是，这样的工作效率非常低，而且也完全没有这个必要。

在虚拟的游戏世界中，游戏对象之间进行交互作用最主要的组织特征就是地理位置。游戏对象趋向于和那些离自己非常临近的游戏对象进行交互。这种交互作用包括：探测器检测、武力交战、沟通交流，以及交换补给品等。

游戏编程人员必须要管理动态游戏对象（也就是那些活着的、喘气的、移动着的游戏对象）列表，以便维系它们之间的地理关系。我们常用的链表和树型结构并不能用来维护这些信息。虽然链表通常都管理着所有的游戏对象，但是管理的方法是一种近乎通用的方式，因此列表的顺序并不包含任何有用的信息。树型结构可能在数据结构中嵌入一些有用的信息，包括地理位置。但是，树型结构的实现，比如四叉树，通常都只能适用于静态地形，而无法适用于动态对象。本文介绍了一个简单的游戏对象列表的管理方法，使用地理网格，有效低降低了射程决策和瞄准线决策工作的计算开销。对于一个大型的游戏战场场景，通过地理信息来管理游戏对象，可以将瞄准线决策及其他相关的地理信息检测工作的数量，从 100 多万次减少到几十次。

有些游戏喜欢采用天然的网格系统来管理游戏对象。如果游戏关卡中表现的是一个地牢、建筑物，或者是一个空间站的内部场景，那么相应的游戏世界就自然地被分割成很多的房间。这种情况下，门户引擎（Portal Engine）就可以有效地利用这些房间信息，根据游戏对象所在的房间，识别出临近的游戏对象。可见的对象和可交互的对象只能存在于该对象所在的房间中，或者相邻的，有入口可达的房间中。虽然一款游戏中可能会包含有几百个游戏对象，但是，可能只有两三个会与当前正在寻找目

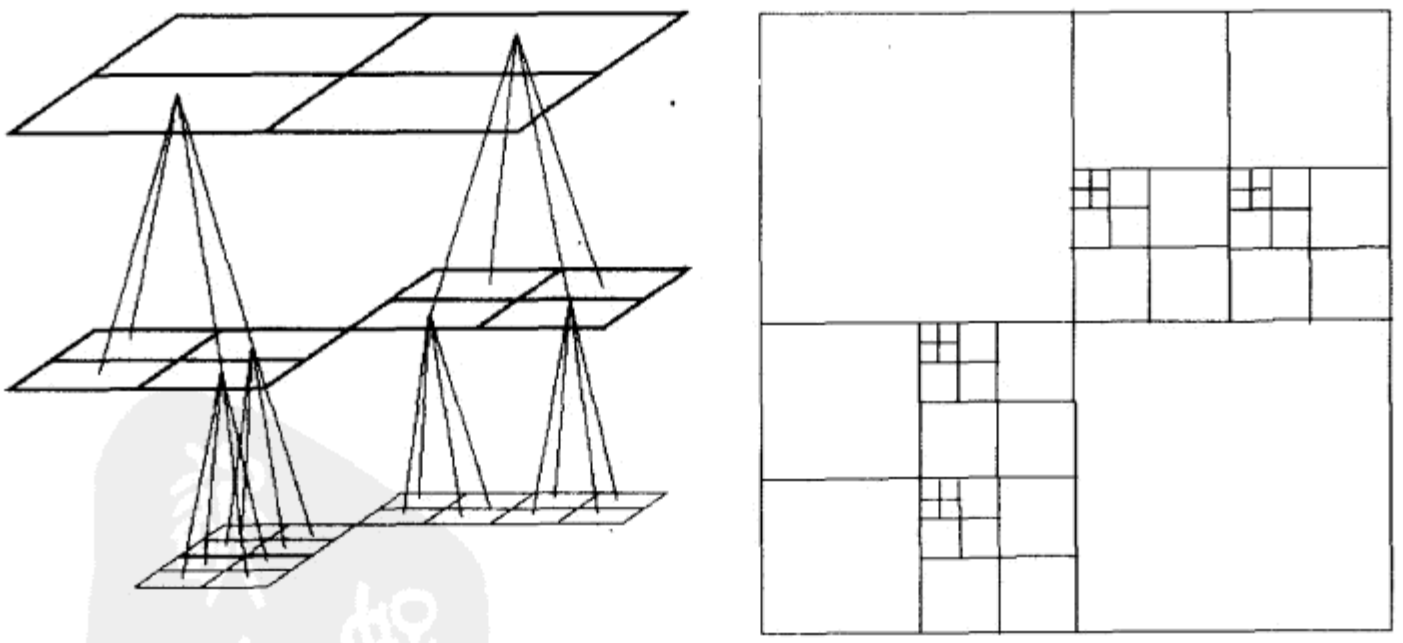
Download at www.Pin5i.Com

标对象的游戏对象处于同一个房间中。这样就可以显著地减少了识别可交互对象的所需要的工作量。

但不幸的是，并非所有的游戏都处于如此完美构建的虚拟世界中。对于大型的开放式战场，或者空间战场，这样的虚拟世界中可能会包含几百个甚至几千个游戏对象。如果场景中没有墙垛、山脉，或者森林之类的东西来分离这些游戏对象，那就相当于所有的游戏对象都处于同一个房间中。如果 AI 代理（或者其他的游戏系统）想要识别出最接近的，最值得进行交互的目标对象，就会面临很棘手的计算问题。在《游戏编程精粹》系列丛书的第一部中，Svarovsky 很简单地介绍过这个问题[Svarovsky00]。在本文中，我们将更为深入地探讨这个问题，详细地解释为什么这些开放空间的场景如此难以处理，并提出一个相对简单的解决方案。这个方案已经在几个模拟器中得以实现。在《游戏编程精粹》系列书的第二部中，Pritchard 也开发了一种基于地砖式地图的瞄准线（LOS）系统[Pritchard01]。但是他的讨论仅局限于从人类玩家的视角出发可以检测到的游戏对象，并没有考虑来自图像渲染引擎和人眼的检测结果。本文则囊括了为 AI 代理提供的完整的检测决策方案。这些 AI 代理在游戏引擎中完全依赖于射程计算和 LOS 计算。当然，我们也对这个方法进行了扩展，可以适用于游戏中的任何一个需要做出类似查询的系统。

1.4.2 四叉树和八叉树

图形编程人员对四叉树肯定非常熟悉。这些数据结构可以将一个二维空间巧妙地分割成 4 个稍小一些的子空间。这个过程可以一代一代地继续下去，直到整个二维空间被分割成包含地形信息的小方块（见图 1.4.1）。一个传感视锥（sensor-viewing frustum）可以覆盖这些小方块的某个特定的子集，并识别出那些需要被画出来的格子。因此，计算机硬件就不必把整个地形数据库全都渲染出来，只需要去渲染探测器可以看到的一小部分的地形[Ferraris01]。



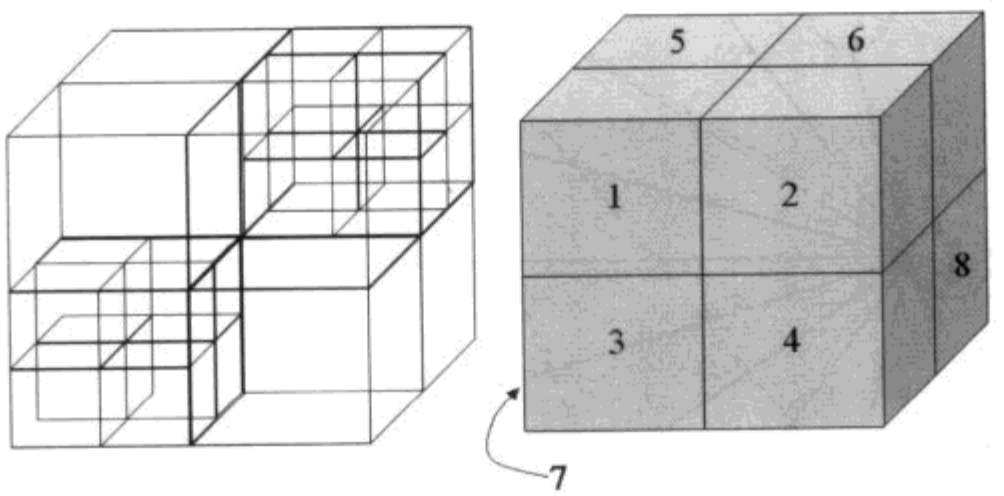
（a）侧视图　　（b）顶视图

图 1.4.1　四叉树将一个二维空间不断地分割成更小的区域

八叉树则可以在三维空间中实现同样的想法[Kelleghan97]。如果某个游戏世界不能被简

Download at www.Pin5i.Com

化为一个二维表面，那么，一个三维世界就可以被分割成 8 个相邻接的空间。我们用这 8 个子空间来生成数据树，而不是像二维空间的 4 个子空间（见图 1.4.2）。



（a）开放的　　（b）着色的

图 1.4.2　八叉树将一个三维空间分割成较小的 8 个立方体

由于地形、建筑物、河流以及森林等通常都位于固定的地理位置上，在一个地理网格中来管理它们历来是一个很自然的方法。四叉树和八叉树可以快速地识别出某个特定场景中需要渲染出来的地形块。

对于飞船、巨魔和士兵，这些在游戏世界里不停游荡的对象，我们还有一种类似的方法，可以优化这些游戏对象之间的检测工作。

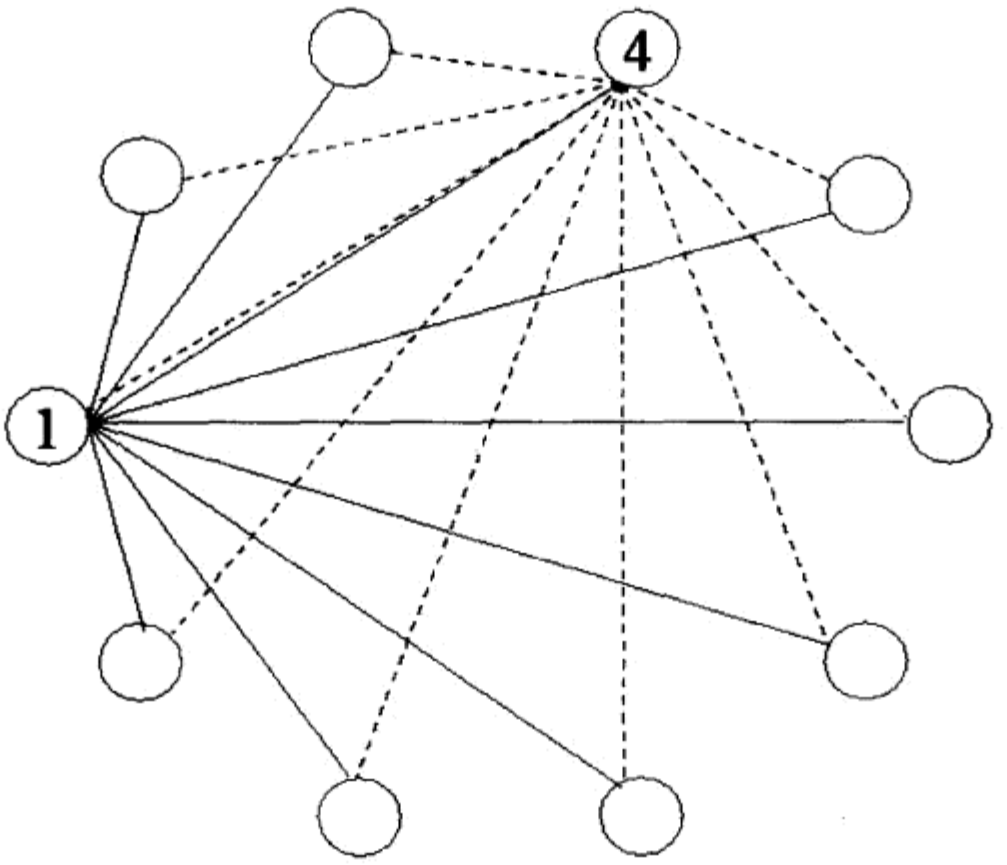
1.4.3　游戏对象的组织形式

在本文中，“游戏对象地理网格的注册”指的是：在地形数据所使用的同类型的网格中，动态地管理那些移动的游戏对象。在有些情况下，这些对象实际可能会使用一模一样的网格系统。

动态对象总是不断地改变它们的地理位置和其他的状态变量。单个士兵，或者是成组的士兵，从一个位置移动到另一个位置，向敌方逼近，或者逃离敌方。因此，从一个时间段到下一个时间段，可见的对象，或者是交战的游戏对象列表就会发生相应的变化。如果用穷举法来解决这个问题，游戏引擎必须不断地计算游戏每一对游戏对象之间的 LOS 检测和射程检测。只要游戏对象的数量相对不太大，这个方法是可以承受的。举个例子，如果游戏世界只包含 10 个互相厮杀的对象，那么 LOS 计算的次数就只有 90，也就是说每个对象要计算它和其他 9 个对象之间的 LOS 检测（见图 1.4.3）。这通常被称为平方阶问题，或者 O（n^2）。虽然从性能和伸缩性上看，这个解决方案是比较完整的，但是，它却是最糟糕的一个方法。如果游戏对象的数量增加到 100，那么射程计算的次数就会激增到 10 000。如果游戏中有 1 000 个游戏对象，相应的计算次数就会增加到接近 100 万次。

对于穷举法，我们有一个简单的改进算法，就是实现一个配对方案。也就是说，从对象 1 到对象 4 计算出来的射程，马上用来判断从对象 1 到对象 4，以及从对象 4 到对象 1 的可见性或探测器检测。这就意味着，一个对象只需要针对那些在对象列表中比它晚添加进来的对象进行射程的计算。这样就把射程计算的次数减少了一半，如图 1.4.3 所示。

Download at www.Pin5i.Com



穷举法= 10个对象*9次检测=90次计算
配对记忆法=9+8+7+6+5+4+3+2+1+0=45次计算

图 1.4.3 采用穷举法进行对象和对象之间的检测是一个平方阶问题

如果没有其他的控制机制，我们在每个时间步长中都要进行这些计算。业界通常会加入一些过滤机制，来缓解这个问题，例如，跟踪记录某个对象上一次位置变化发生的时间，以便避免在相同的两个位置之间进行不必要的重复的射程计算。但是，如此一来，就要求我们必须保留必要的信息，记录下特定的游戏对象配对之间上一次射程计算的情况。因此，这个做法无法达到降低算法时间复杂度的目的。

瞄准线 LOS 则是一个地理问题。要想得到一个高效的解决方案，要求我们必须用地理的方法来解决这个问题，就像我们使用四叉树来解决静态地形信息一样。

1. 网格注册

随着它们位置的变化，移动的对象必须注册到一个地理网格中。这就是说，我们不但要计算一个地理网格的位置，还要计算游戏对象更为通用的位置。随着一个对象从一个方格子移动到另外一个方格子（就像棋子在棋盘上改变其位置），我们必须把这个对象注册到新格子中，并从旧格子中将其注销，如图 1.4.4 所示。这个图中显示了 2 个对象列表，分别对应的是格子（1，5）和格子（1，8）。在时间 1234 的时候，这两个格子中各有 2 个对象。但是，到了时间 1235 的时候，对象 1（O1）已经移动到了一个新的位置。

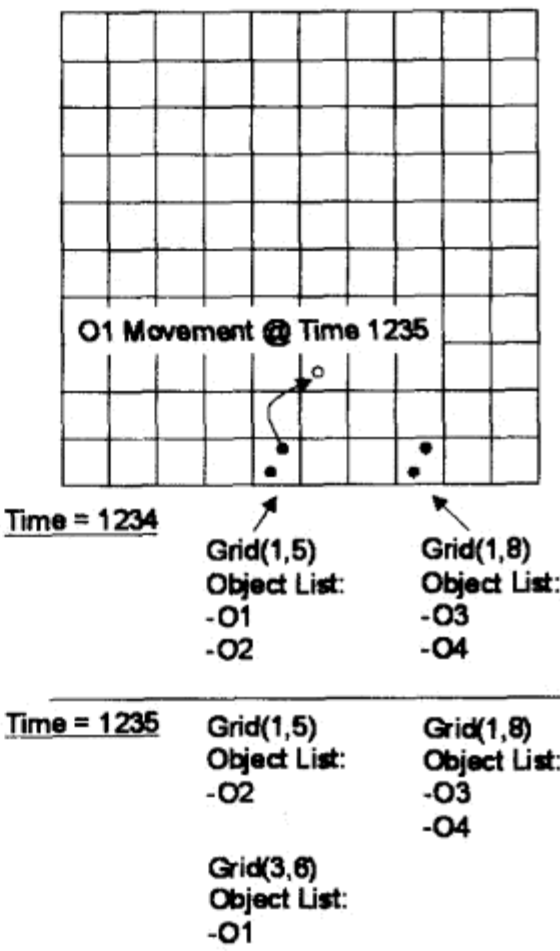


图 1.4.4 当某个对象最新的位置位于一个新的网格块中时，我们就执行一次网格注册操作

Download at www.Pin5i.Com

因此，格子注册过程必须改变对象 O1 的注册信息，将它注册到格子（3，6）上。与我们前面所介绍的执行所有的射程计算所需要的计算开销相比，管理格子位置所带来的计算开销要小得多。这种计算开销上的减少，一部分原因是因为，与更为复杂的射程计算和 LOS 计算相比，判断一个格子的位置是一个更为简单的问题。但是，最主要的原因是因为我们大大地减少了必须要执行的操作的数量。

对象的网格注册是一个线性阶 O（n）问题，而 LOS 则是一个平方阶 O（n^2）问题。因此，对于一个拥有 100 个对象的游戏，在每个时间步长中，我们最多需要 100 次网格注册操作。如果不使用地理网格（并放弃其他的检测过滤机制），在每个时间步长中，这 100 个游戏对象需要将近 10 000 次的射程计算。这样看来，网格注册法将 10 000 次的射程计算变成了 100 次更简单的注册操作。

2. 火力覆盖区或探测器覆盖区

网格注册法并不能完全免除所有的射程计算或者 LOS 计算。它只是简单地将计算限定于更少量的游戏对象。如图 1.4.5 所示，搜索对象的探测器覆盖了少量的几个格子。这些格子的识别方法有很多，在其他关于四叉树的文章和地形数据库管理[Frisken03]的文章中有详细的介绍。一旦这个列表的格子被识别完毕，我们只需要考虑那些注册在这些格子中的对象，针对它们进行侦察或与之交火。但是，只是因为某个对象注册在其中的某个格子中，我们根本无法保证这个对象也位于探测器的覆盖区中。从图 1.4.5 中我们可以很清楚地看到，在很多情况下，探测器并不会覆盖一个网格方块的全部。对于那些位于被部分覆盖的格子中的游戏对象，它们实际上也许并不在探测器的范围之内，需要将它们从潜在可侦察或可与之交火的对象列表中排除出去。因此，针对这些格子中的非常小的对象群组，我们仍然必须分别执行一次射程计算和一次 LOS 计算。

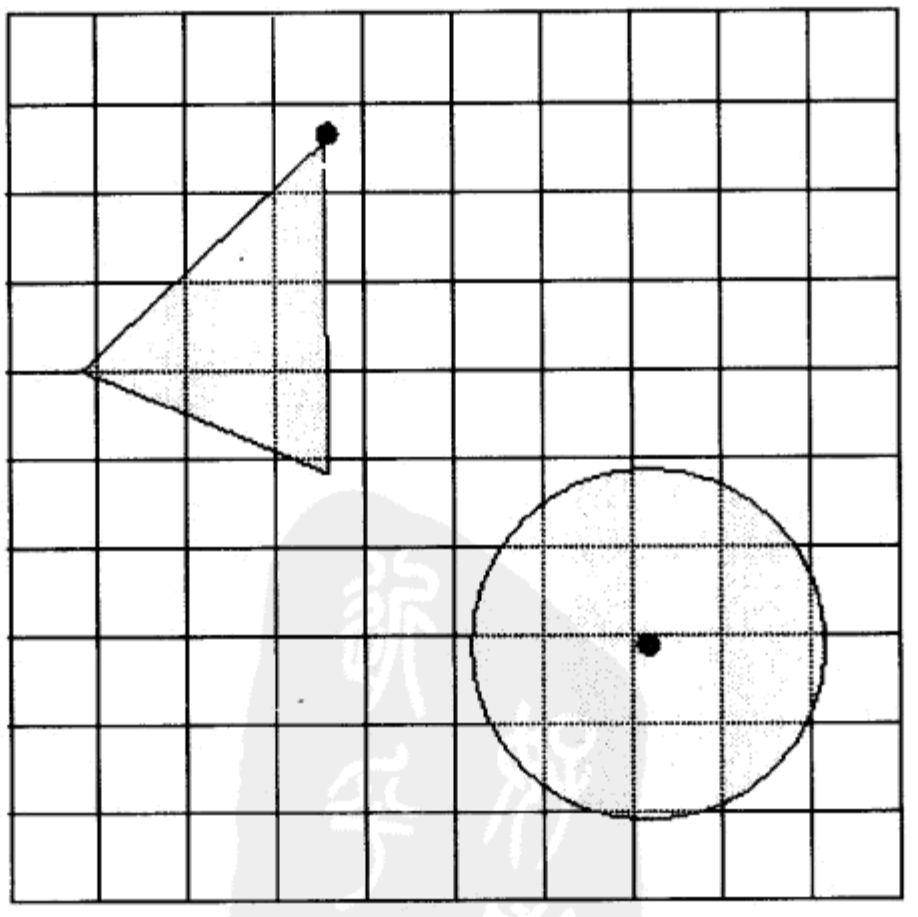


图 1.4.5

Download at www.Pin5i.Com

程序人员可以实现特殊的探测器覆盖区算法，以便将那些部分位于探测器覆盖区之内的网格方块，与那些完全位于探测器覆盖区之内的网格方块区分开来。对某些游戏而言，这样一来，对于那些完全在探测器覆盖区的格子中所包含的游戏对象，我们就不用再去执行射程计算或 LOS 计算了[Pritchard01]。然而对于另外一些游戏，三维地形是 AI 或 NPC 做出可见性决策的一个关键要素。这个时候，对于覆盖区内格子中的所有对象，我们还是有必要执行一次单独的 LOS 计算。虽然网格系统可以识别出哪些对象是位于探测器范围之内的，但是，它无法判断，穿过地形和障碍物，从探测器到被选中的对象之间是否存在一个完整的 LOS 向量。

3. 放大覆盖区

我们的目标是：将查询系统必须要进行可视性判断的游戏对象的数量最小化。但是，在这个过程中，我们也要小心谨慎，以确保我们没有不恰当地排除了那些在探测器覆盖区边缘移动的对象。在大部分的游戏中，在一个时间步长中，游戏对象会沿着一个常数向量，从 A 点移动到 B 点。对象的这个移动是有可能与探测器覆盖区的边缘交叉的。这样一来，在计算工作执行的两个不连续的时间段内，这个对象就不会存在于覆盖区之内（见图 1.4.6a）。

为了能够正确处理这些情况，我们通常会把覆盖区的大小稍稍放大一些。这样，探测器就可以捕获那些存在于与覆盖区直接相邻的格子中的对象，以及那些在移动中的，与覆盖区有可能相交叉的对象。至于覆盖区到底应该放大多少，我们可以根据网格方块的尺寸、游戏对象最大的移动速度，以及时间步长的长度，综合这些因素试探性地做出决策。在进行放大的时候，我们必须把一个时间步长中，某个游戏对象到达覆盖区边缘所经过的所有格子都包含进来。如果在一款游戏中，移动操作先于检测操作，在进行覆盖区的放大操作之后，我们会发现游戏对象已经移动到了终点位置。这时候，系统就会采用反向位置推算法来计算这个对象到达其终点位置所经过的路径，并判断其路径是否与探测器覆盖区交叉。

覆盖区的放大最常见的有两三种情况。第一种情况是：覆盖区尺寸的放大，就像前面刚刚提到过的。第二种情况是：网格系统是由很多的方块组成的，而覆盖区的形状则可能像是一个楔型，或是一个圆。将圆形区域方格化的过程会产生一个位于覆盖区之外的区域。最后，这个被方格化的圆可能不会精确地落在格子的边界上。如果是这样，我们就将覆盖区再进行放大，使之可以包括被方格化进来的网格方块的全部区域（见图 1.4.6b）。

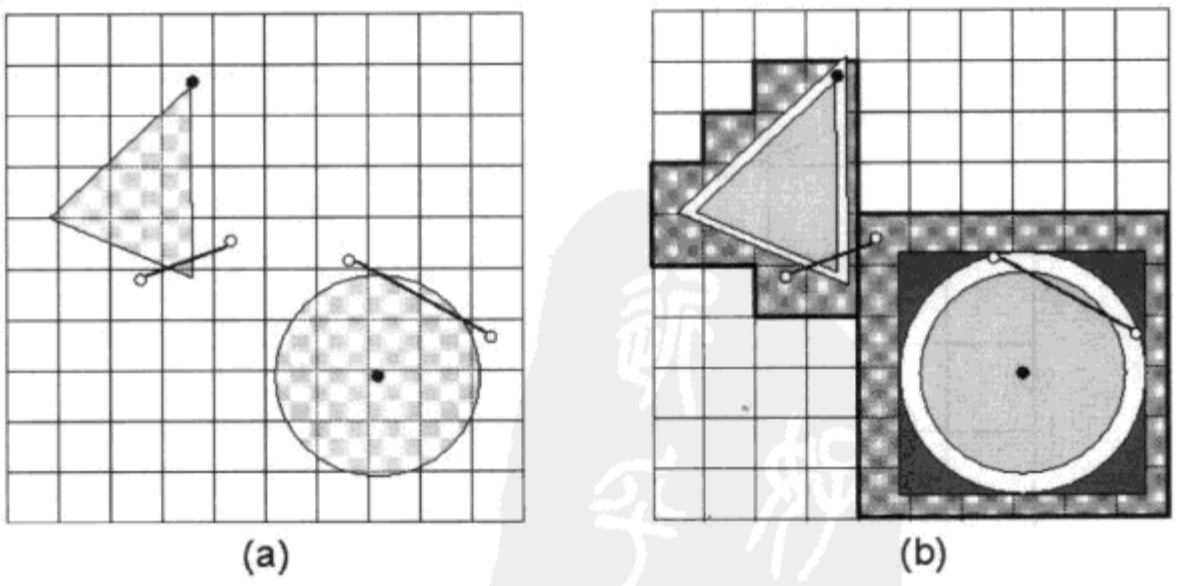


图 1.4.6 覆盖区放大之后，就可以捕获那些在一个时间步长种，其移动向量与探测器覆盖区边缘相交叉的游戏对象

Download at www.Pin5i.com

1.4.4　总结

本文所介绍的这个方法并不复杂。经过我们的介绍，它看上去就像是一个显而易见的方法，可以用来创建一个更具伸缩性的检测算法。但是，有些项目还是在继续对这个方法及其变种，进行重新的发现。这些项目的数量不在少数。我们在这里提出这个算法，就是希望以后能够节省程序人员的时间，让他们不用再去重复我们过去做过的这些工作。

我们的目标是：用一种与处理静态地形中的游戏对象相同的方式，来管理动态对象的位置。这些对象一会儿进入游戏，一会儿又离开游戏，经常地变化自己的位置，而且有时候还要跨据两个或更多的网格方块。所有这些行为都对相应的对象管理工作带来了更多的挑战。但是，我们也因此得到了回报。我们因此而节省出来的处理时间，远远超过了实现一个地理网格系统来减少 LOS 计算和射程计算次数（这些计算是一款游戏中重复次数最多的计算）所带来的成本支出。

1.4.5　参考文献

[Ferraris01] Ferraris, Jonathan, “Quadtrees.” GameDev.net. January 2001. Available online at *http://www.gamedev.net/reference/articles/article1303.asp*.

[Frisken03] Frisken, S. and R. Perry, “Simple and Efficient Traversal Methods for Quadtrees and Octrees.” *Journal of Graphics Tools,* Vol. 7, Issue 3. May 2003.

[Kelleghan97] Kelleghan, M., “Octree Partitioning Techniques.” *Game Developer,* July 1997.

[Pritchard01] Pritchard, M., “A High-Performance Tile-Based Line-of-Sight and Search System.” *Game Programming Gems 2,* Charles River Media, 2001.

[Svarovsky00] Svarovsky, Jan, “Multi-Resolution Maps for Interaction Detection.” *Game Programming Gems,* Charles River Media, 2000.

Download at www.Pin5i.Com

1.5 BSP 技术

Octavian Marius Chincisan，自由职业者
mariuss@rogers.com

本文介绍了一款功能齐全的 BSP（Binary Space Partition 二进制空间分割）编译器。这个编译器集成了一些新的算法，可以自动生成出入口（Portal，也有译为“孔洞”）和潜在的可视集计算（Visible Set Computation）。这些特性都是当前在 Getic 3D Editor 和 BSP 编译器(www.zalsoft.com)中广泛使用的。本文介绍的这个 BSP 编译器，其全部的实现代码都收录在随书光盘中。

1.5.1 什么是 BSP? 为什么要使用 BSP?

BSP 技术可以层次化地将一个三维空间分割成若干个凸状的子空间。分割工作是由一个指定的平面来完成的。这个平面将一个给定的空间分割成两个大小近乎相当的子空间。然后，再用一个给定的平面，将每个子空间分别分割成两个更小的子空间。在每个子空间中，这个分割过程一直持续下去，直到满足一个事先设置的条件。

根据分割工作的执行方法，以及几何体在 BSP 树中的保存方式，BSP 树的类型也不尽相同。本文中所涉及的算法可以用于开发不同类型的 BSP 树，诸如 K-D 树（多维二叉树）、AABB 树（沿坐标轴的包围盒），以及 beam 树等。在最初的时候，人们使用 BSP 树来对那些采用 back-to-front 渲染方法（后端至前端的渲染，是指优先渲染 Z 值最低的多边形）进行渲染的多边形进行分类排序。到了今天，甚至是在当代的视频显示卡技术中，这种分类排序法仍然是必不可少的。首先，由于在游戏场景中使用了透明的面（face）和混合的对象，游戏引擎不得不对多边形进行从后端到前端的排序，以便能够在帧缓存中执行正确的混合操作。其次，相对于整个场景而言，针对一个 BSP 树进行的碰撞检测执行速度要更快些。这一点可以从我们光盘中那个碰撞检测的实现中得到印证。这个碰撞检测系统的实现包括：对象与场景的交叉、动态光照、动态阴影、雾化、光环特效、网络流量可视区等很多的特性。

在本文的第二部分中，我们会探讨实体叶子（Solid-leaf）型 BSP 中潜在可视集的计算问题。潜在可视集可以通过简单的布尔特征位测试，放弃处理那些不可见的几何体，以此提高程序性能和帧率。在接下来的内容中，我们会介绍几种常用的 BSP 技术。

Download at www.Pin5i.Com

1.5.2　基于节点的 BSP

BSP 是一棵二叉树。在这样定义的一个结构中，每个节点中保存着两个指向同类型子节点的链接。除此之外，每个节点中还保存着来自同一个场景的若干个多边形。因此，我们称之为“基于节点的 BSP”。下列代码片段说明了一个节点的数据结构。每个节点实例保存着前节点和后节点的指针，以及场景中的几何体，也就是若干个多边形和分割平面。

```
NODE{ NODE front, back, parent
    List polygons;
    Plane plane;
};
```

我们将后节点和前节点统称为两个子节点。通过前节点和后节点，我们将所有的节点实例链接在一起，直到最后我们将整个被分割的空间几何体表示出来。下列代码段就是基于节点的 BSP 算法的实现。整个实现例程可以在随书光盘中的源代码目录中找到。

```
Node BuildBSP(List polygons, Node& node)
{
    if(polygons.IsEmpty())
        return null;
    List front_list,back_list;
    Polygon polygon = Pick_Splitter_AndRemove(polygons);
    node.plane = polygon.GetPlane();
    node.polygons.Add(polygon);
    for_each (polygon in polygons)
    {
        if(polygon is on front of node.plane)
            front_list.Add(polygon);
        else if(polygon is on back of node.plane)
            back_list.Add(polygon);
        // 用这个分割平面来分割这个多边形
        // 并相应地添加多边形分割产生的多边形片段
        else if (polygon is not coplanar with node.plane)
        {
            Polygon front_polygon, back_polygon;
            SplitPolygon(polygon, &front_polygon,
            &back_polygon);
            back_list.Add(back_polygon);
            front_list.Add(front_polygon);
        }

        // 将所有共面的多边形添加到这个节点
        else
            node.polygons.Add(polygon);

}
```

Download at www.Pin5i.Com

```
        BuildBSP(front_list, node.front);
        BuildBSP(back_list, node.back);
    }
```

为了向大家更详细地解释，我们将这个算法分解成几个独立的不同的步骤，并配以图示说明。下面让我们从图 1.5.1 和图 1.5.2 开始。

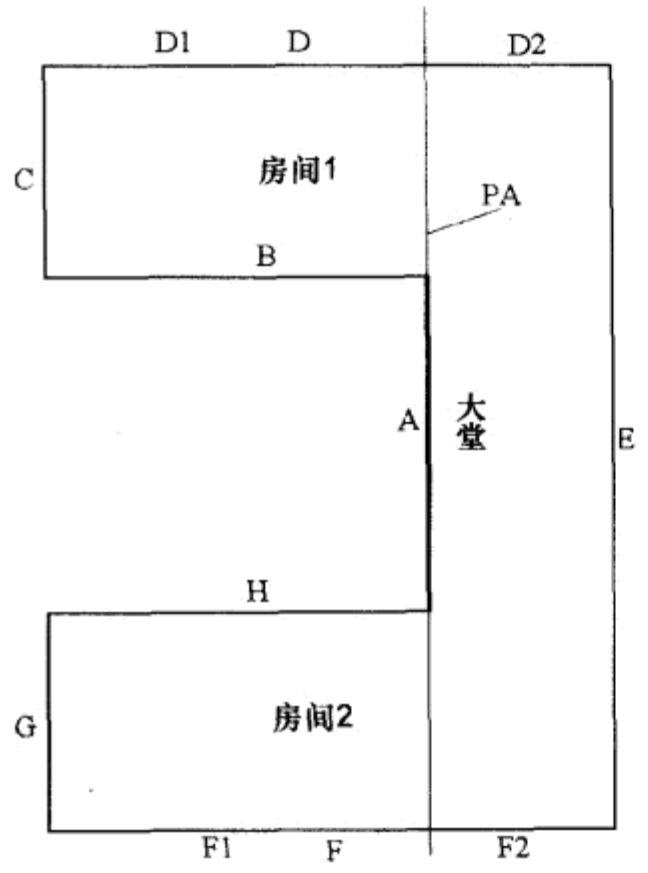


图 1.5.1 一个需要处理的场景范例

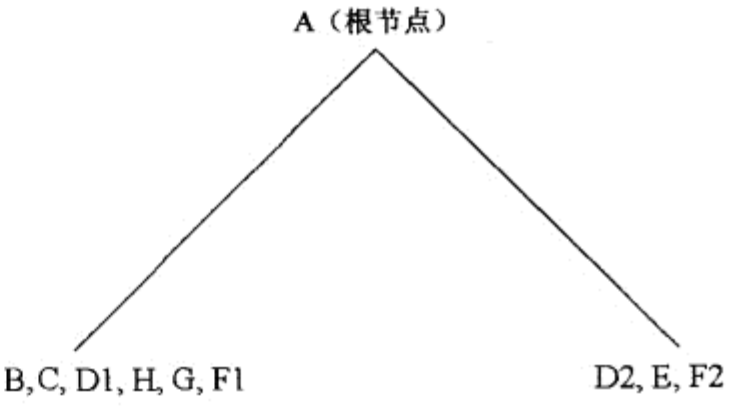


图 1.5.2 BSP 处理过程的第一步

由多边形 A 到多边形 H，我们定义了一个简单的场景，其中包括两个房间和一个大堂。在我们开始之前；必须实现一个函数，来挑选分割平面。这个函数必须要完成两件事情：一是保持树的平衡；二是将多边形的数量尽量保持在与原场景中的多边形数量（在本例中，就是 8 个多边形）接近的水平上。前端/后端多边形的数量与分割后多边形的数量之间的平衡比率可以预先设定。如果达到这个比率，该函数就返回一个可能的分割平面。在一个典型的 BSP 编译器中，平衡比率的计算公式通常是：

```
abs(front_count-back_count)+(both*balance_vs_cuts);
```

第一个分割平面我们选择的是多边形 A 的一个平面。这也就产生了我们对 BSP 算法的第一眼印象，参见图 1.5.2。在后端多边形列表中（多边形 B、C、D1、H、G 和 F1），我们选择多边形 B 的平面作为下一个分割平面。而在前端多边形列表中（多边形 D2、E 和 F2），多边形 E 的平面被选为分割平面。

接下来，我们用多边形 B 的平面将所有的后端多边形进行分割，用多边形 E 的平面将所有的前端多边形进行分割，就得到了如图 1.5.3 所示的一棵 BSP 树。下一步是分割所有的多边形（见图 1.5.4）。直到两个子空间中不存在任何多边形，分割过程才告结束。到此为止，所有的多边形都被保存在 BSP 节点中。构建 BSP 树的中间步骤如图 1.5.3 到图 1.5.5 所示。

Download at www.Pin5i.Com

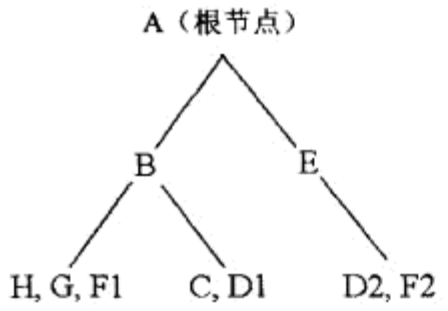


图 1.5.3　BSP 树构建的第二步

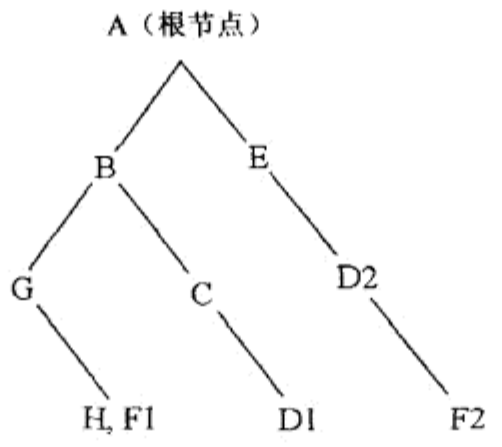


图 1.5.4　BSP 树构建的第三步

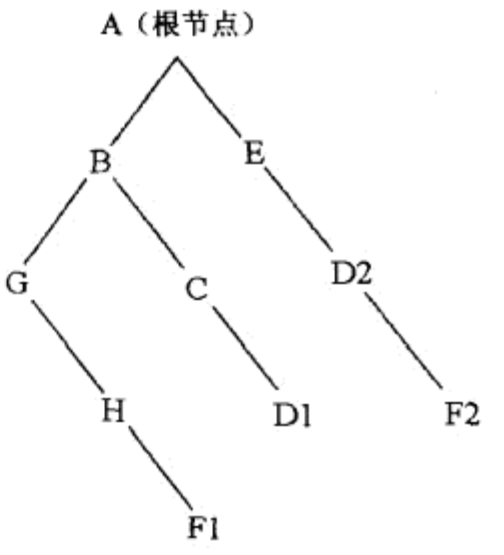


图 1.5.5　BSP 树构建完成

1.5.3　渲染一个基于节点的 BSP 树

在空间分割完成之后，渲染就成了一个非常简单的过程。从根节点开始，一直持续到所有的单个节点，我们将摄像机（视点）与分割节点的平面进行比对。如果摄像机位于某个分割节点的一侧，那么我们就渲染那些位于这个分割节点另外一侧的节点。然后，我们再渲染当前节点及当前所在这一侧的节点。简化的渲染算法可以用下列伪代码来描述：

```
void Render(Point camera_position,
    NODE& node=ROOT_NODE)
{
    // 摄像机位于该节点平面的前端
    if(Distance(camera_ position, node.plane)<0)
    {
        Render(camera_position, node.front)      // 渲染前端节点
        Render(node.polys);                      // 渲染当前节点
        Render(camera_position, node.back)       // 渲染后端节点
    }

    //摄像机位于该节点平面的后端
    else
    {
        Render(camera_position, node.back)       // 渲染后端节点
        Render(node.polys);                      // 渲染当前节点
        Render(camera_position, node.front)      // 渲染前端节点
    }
}
```

1.5.4　基于节点的 BSP 树（不进行分割）

一个基于节点的 BSP 树还有另外一个构建方法，就是在分割的过程中避免去分割多边形。在我们这个例子中，我们没有对多边形 D 和 F 进行分割。取而代之的是，我们将这两个

Download at www.Pin5i.com

多边形全部添加到前端多边形列表和后端多边形列表中。图 1.5.6 向我们展示了这个操作过程的结果（图 1.5.7 显示的是我们这个场景中可以走动的区域）。

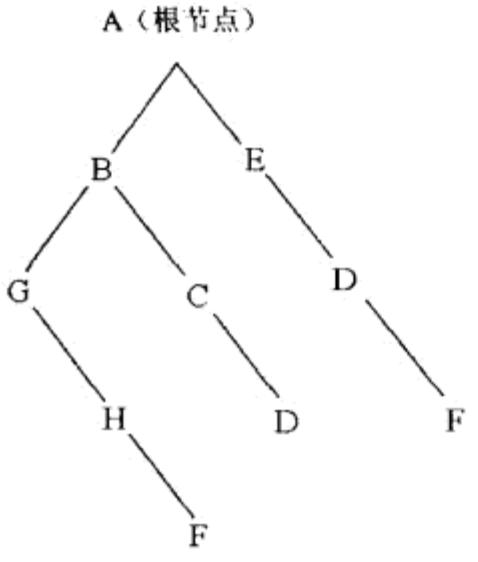


图 1.5.6 步进行分割的 BSP 树

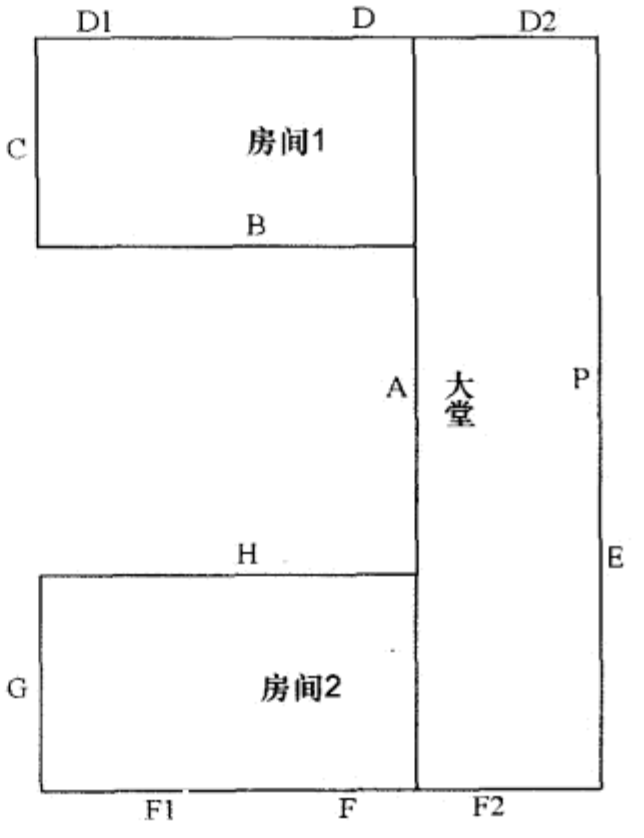


图 1.5.7 场景中可以走动的区域

根据图 1.5.6 中的 BSP 结构图，多边形 D 和 F 出现在多个节点中。相关的解决办法是使用一个计数器，每次在调用渲染函数时，计数器就增加一个步进值。我们用这个计数器来标识所有被渲染过的多边形。如果某个多边形的计数值与当前帧的计数相同，那么我们就忽略这个多边形，不进行渲染。

1.5.5 凸状叶子 BSP 树

正如它的名字所暗示的，一个叶子 BSP 树将多边形保存在叶子节点中。每个叶子会定义一个凸状区域。这就意味着，它们可以向一条由两个多边形组成的边弯曲，而不会下落。这样，我们就可以把 BSP 树表示为一个图形，其中的所有叶子都用出入口（portal）连接起来。BSP 树的这个图形化表示使得潜在可视集（Potentially Visible Set，简称 PVS）的计算成为可能。在我们开始用示例场景来说明之前，我们先通过下列这段伪代码，简要地回顾一下叶子 BSP 树的构造过程：

```
Node BuildSolidleafBSP(List polygons, Node& node)
{
    List front_list, back_list;

    // 从多边形中挑选一个作为分割平面（splitter）
    // 避免选择那些已经被标识为分割平面的多边形
    Polygon polygon = Pick_Splitter(polygons);

    // 将这个多边形标识为分割平面
```

Download at www.Pin5i.com

```
FlagAsSplitter(polygon);
front_list.Add(polygon);              // 将其添加到前端多边形列表中
node.plane = polygon.GetPlane();      // 将其保存在节点中

for_each (polygon in polygons)
{
    // 参考这个分割平面，分割所有的多边形
    if(polygon is on front of node.plane)
        front_list.Add(polygon);
    else if(polygon is on back of node.plane)
        back_list.Add(polygon);

    // 用分割平面来分割多边形
    else if (polygon is not coplanar with node.plane)
    {
        Polygon front_polygon, back_polygon;
        SplitPolygon(polygon, &front_polygon,
            &back_polygon);
        back_list.Add(back_polygon);
        front_list.Add(front_polygon);
    }
    else if(polygon is coplanar with node.plane)
    {
        if(polygon is facing as node.plane)
            front_list.Add(front_polygon);
        else
            back_list.Add(back_polygon);
        }
    }
    if(IsConvex(front_list) or
        AllPolygonsAreSPlitters(front_list))
    {
        // 前端节点是一个叶子，保存着所有的多边形
        // 它们形成一个由叶子多边形包围的凸状空间
        node.front.polygons = front_list;
    }
    else
        // 继续执行分割过程
        BuildBSP(front_list, node.front);

    If(IsEmpty(back_list))
    {
        // 后端叶子为 null，表示这是一个实体空间
        node.back = 0;
    }
    else
    {
        // 继续执行分割过程
        BuildBSP(back_list, node.back);
    }
}
```

Download at www.Pin5i.Com

我们会在后面的图 1.5.10 中看到最后生成的实体的叶子 BSP 树。这棵树的构建所使用的分割平面，与我们前面所讲的基于节点的 BSP 树所使用的分割平面是同一个多边形。对于图 1.5.1 中这个场景，我们下面就一步一步地解释它的叶子 BSP 树的构造过程。分割平面选择的是 PA，它所在的多边形就被标识为一个分割平面。多边形 D 和多边形 F 分别被分割为 D1、D2，以及 F1 和 F2。根据它们相对于分割平面的位置，这些分割产生的多边形被分别添加到相应的列表中。多边形 D2 和 F2 被添加到前端多边形列表中，而多边形 D1 和 F1 则被添加到后端多边形列表中。

关于构造一棵 BSP 树的第一步，请参见图 1.5.8。接下来，我们对前端多边形列表进行凸状性测试。这个过程就是对于前端多边形列表中的所有多边形进行一对一的分割。如果分割迭代产生的多边形位于所有多边形的最前端，那么这些多边形就圈定了一个凸状区域。在我们这个例子中，前端多边形 D2、E、A 和 F2，它们彼此都位于对方的前端。因此，他们定义了一个凸状区域。记住，所有的面都是向内的。在这种情况下，被处理过的节点被标识为叶子，所有的多边形都添加到该节点上。后端多边形列表中包含多边形 D1、C、B、H、G 和 F1。多边形 B 的平面 PB 被选定为下一个分割平面。所有的多边形都要比对这个分割平面进行分割。图 1.5.9 中示意的是这棵 BSP 树构建的第二步。

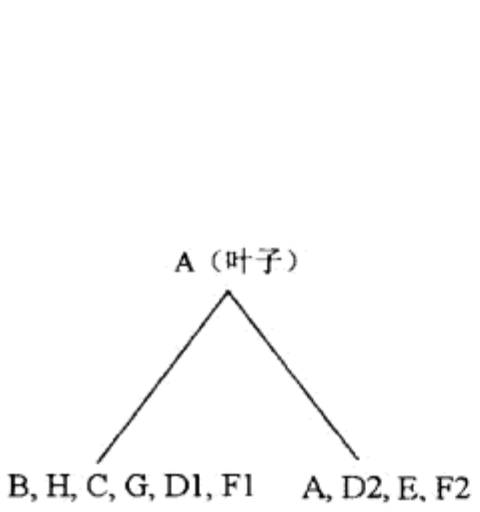


图 1.5.8 实体 BSP 树构造的第一步

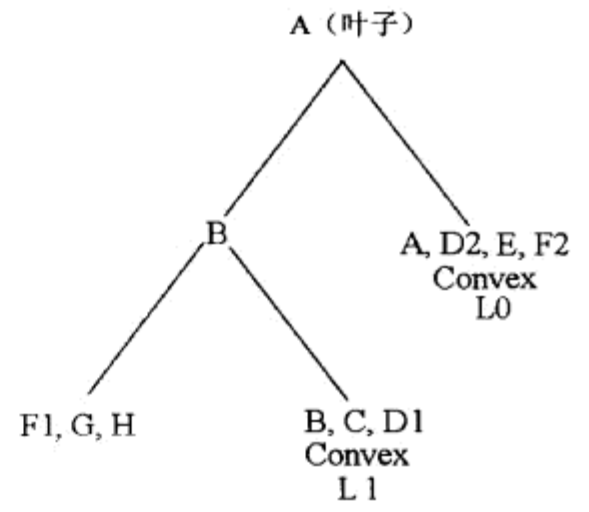


图 1.5.9 实体 BSP 树构造的第二步

我们对前端多边形列表进行凸状性测试，而利用后端多边形列表继续进行 BSP 分割过程。在前端多边形列表中，多边形 B、C 和 D1 彼此都位于对方的前端。下一步我们就要构建一个叶子型的节点，把所有的多边形添加进去。这个阶段的 BSP 树如图 1.5.9 所示。在后端多边形列表中，多边形 G 的平面被选定为下一个分割平面。如此这般继续下去就生成了最后的 BSP 树图，如图 1.5.10 所示。

场景中所有的多边形都作为凸状区域，位于 BSP 的前端节点中。这些区域被称为空叶子，或凸叶子。而后端节点中只有实体空间（图 1.5.10）。正如我们在图 1.5.7 中所看到的，可走动的区域总是存在于 BSP 树中，在多边形 F、G、H（叶子 L2）的前端，多边

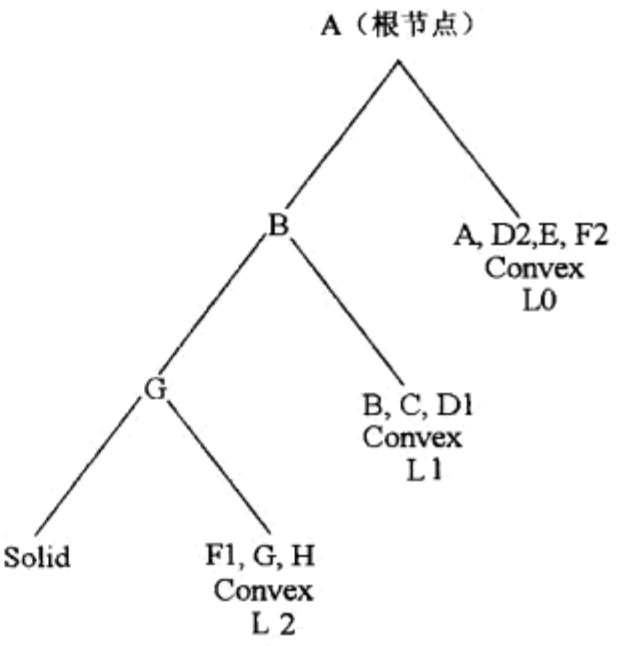


图 1.5.10 实体 BSP 树最终的结构示意

Download at www.Pin5i.Com

形 B、C、D 的前端（叶子 L1），以及多边形 A、D2、E 和 F2（叶子 L0）的前端。

1.5.6　凸状叶子 BSP 树出入口生成

出入口生成（Portal Generation）是自动 PVS（潜在可视集）计算的中间步骤。为了向基于 Portal（出入口）的游戏引擎提供出入口信息，我们同样需要出入口生成这个步骤。出入口（Portal）是两个临近的空叶子之间的虚拟多边形。它们也可以被看作是两个凸状区域之间的通道。一个自动出入口生成算法一般包括两个简单的步骤。（参见[Getic]，三维编辑器开发）相对于其他的任何一个自动出入口生成算法，这个算法要更快一些，而且也更为简单。这个算法大致如下：

```
// 找到所有两两接触的叶子配对
void FindPairTouchingLeaves()
{
    for_each(leaf1 in leaves)
    {
        for_each(leaf2 in leaves)
        {
            if(leaf1==leaf2) continue;
            if(leaf1.box.Touches(leaf2.box))
              {
            // 找到两个叶子共同的父节点，并在那里
            // 创建一个巨大的多边形。这就是初始出入口
            Node node = FindCommonParent((Node)leaf1,
            (Node)leaf2);
            Portal portal = CalculateInitialPortal(node);

            // 用两个叶子的边来分割初始出入口多边形
            ClipWithLeavesides(leaf1, portal);
            ClipWithleavesides(leaf2, portal);

            // 保留下来的多边形就是合法的出入口
            listBSP_tree.Portals.Add(portal);
                }
            }
        }
    }
```

在随书光盘中，你可以找到这个出入口生成程序的详细实现代码。

在用这个算法来处理图 1.5.7 的 BSP 树后，我们知道了两对相邻的叶子：叶子 L0 和叶子 L1，以及叶子 L0 和 L2。对于每一对叶子，这棵 BSP 树恰好只有一个共同的节点（根节点），参见图 1.5.11。而在图 1.5.12 中，我们可以看到，初始出入口 PrA 是在根节点的平面上创建的。

为了找到叶子 L0 和 L1（参见图 1.5.13）之间的出入口 P1，我们参考叶子 L1 和 L0 中的多边形，对初始出入口 PrA 进行分割。为了找到叶子 L0 和 L2（参见图 1.5.13）之间的出入口 P2，我们用叶子 L0 和 L2 上的多边形对初始出入口 PrA 进行分割。这样就完成了出入口

Download at www.Pin5i.Com

的判断工作。出入口 P1 将叶子 L1（房间 1）与叶子 L0（大堂）分割开来；而出入口 P2 将叶子 L2（房间 2）与叶子 L0（大堂）分割开来。

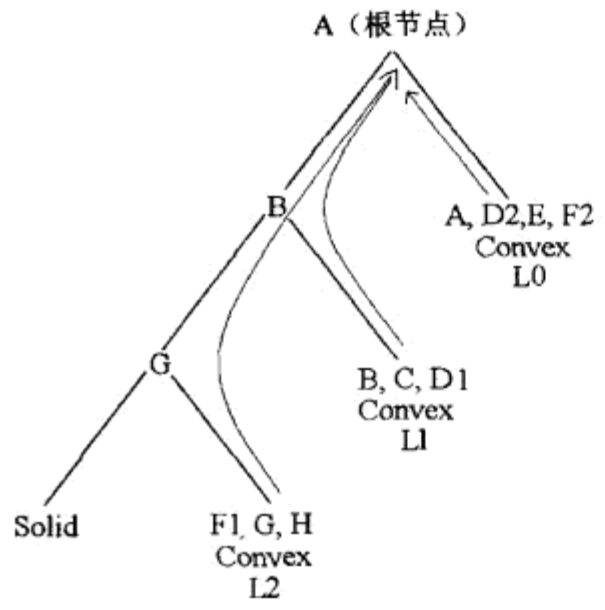


图 1.5.11 根节点 A 就是公共节点

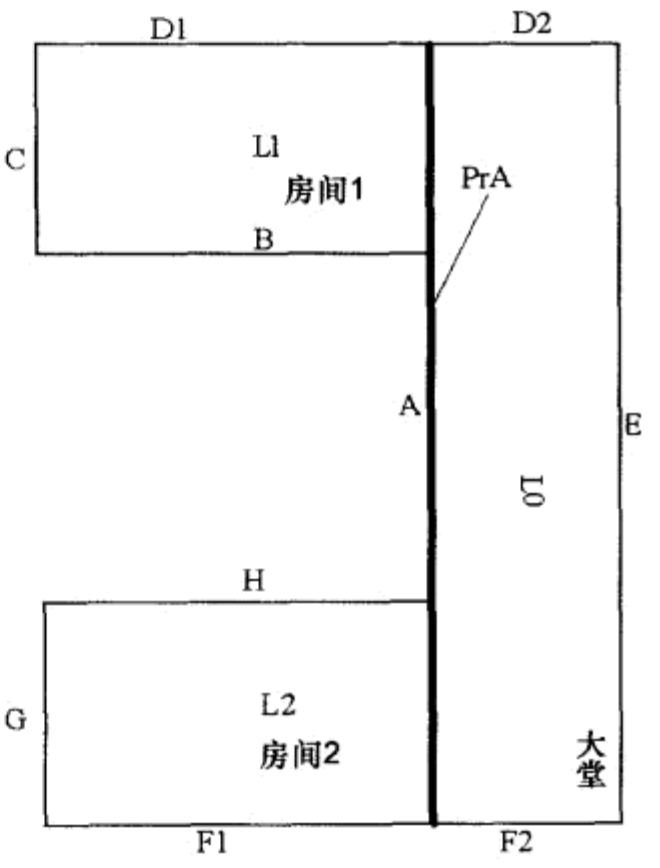


图 1.5.12 公共节点上的初始出入口

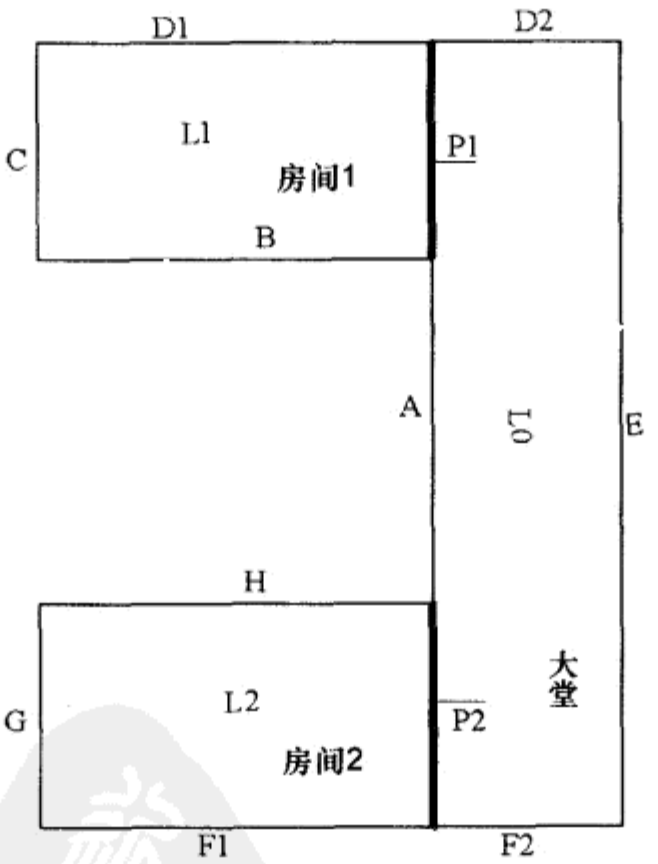


图 1.5.13 最终产生的有效的出入口

1.5.7 凸状叶子 BSP 树潜在可视集

对于凸状叶子 BSP 树 PVS（潜在可视集）的算法，为了描述的方便，我们可以考虑在上

Download at www.Pin5i.Com

面的场景中再增加两个额外的房间（见图 1.5.14）。增加两个额外的房间就创建出一个更为复杂的场景，由此也就可以创建出更为真实的 PVS（潜在可视集）。新增加的房间形成了两个新的叶子（叶子 L3 和 L4）。当然，我们又得到了两个新的出入口（出入口 P3 和出入口 P4）。PVS（潜在可视集）计算的第一步就是复制 BSP 树中所有的出入口。为了完成这个工作，我们首先进行直接的拷贝，然后再将其顶点进行反转，这样就复制了所有的出入口。出入口及其复制品共享同一个空间，但是朝向则彼此相反（见图 1.5.14）。

第二步是将所有的出入口与其后端的叶子联系起来。这样一来，叶子自动就变成了出入口的所有者。现在，每个出入口都隶属于某个叶子。叶子 L0 拥有的是出入口 P1、P4 和 P2；叶子 L1 拥有出入口 P1'；叶子 L2 拥有的是出入口 P2'和 P3；叶子 L3 拥有的是出入口 P3'；而叶子 L4 拥有的是出入口 P4'。现在，PVS（潜在可视集）计算的准备工作已经完成。每个叶子都可以看到其拥有的出入口所能看到的场景。

叶子 L4 可以看到出入口 P4 所能看到的场景。例如，假设叶子 L4 通过其出入口向各个方向上放射出光线（见图 1.5.15）。如果光线照射到 BSP 中其他的出入口，这就意味着叶子 L4 可以看到拥有这个被照射到的出入口的叶子。在我们这个例子中就是叶子 L2 和 L1。叶子 L4 完全看不到出入口 P3，因为 P3 处于阴影中，因此，出入口 P3 的所有者就是叶子 L3。

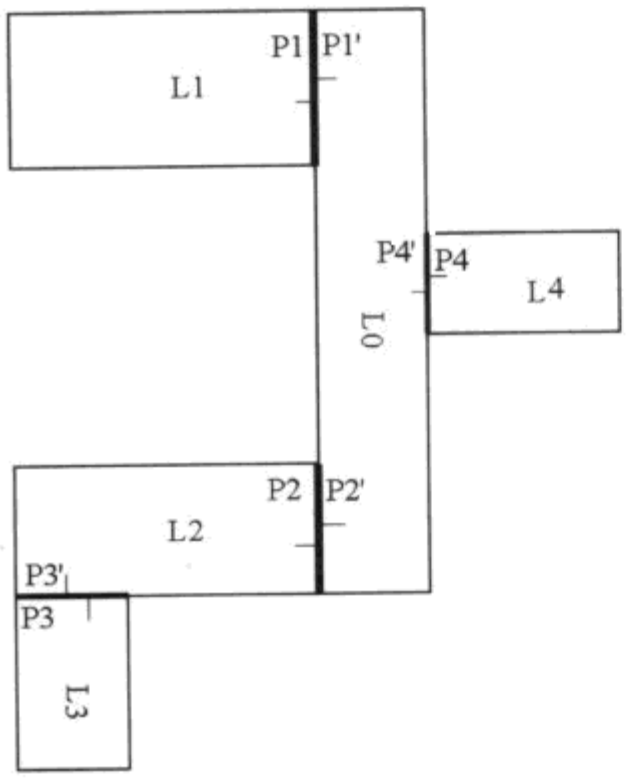


图 1.5.14　用于 PVS（潜在可视集）计算的新场景

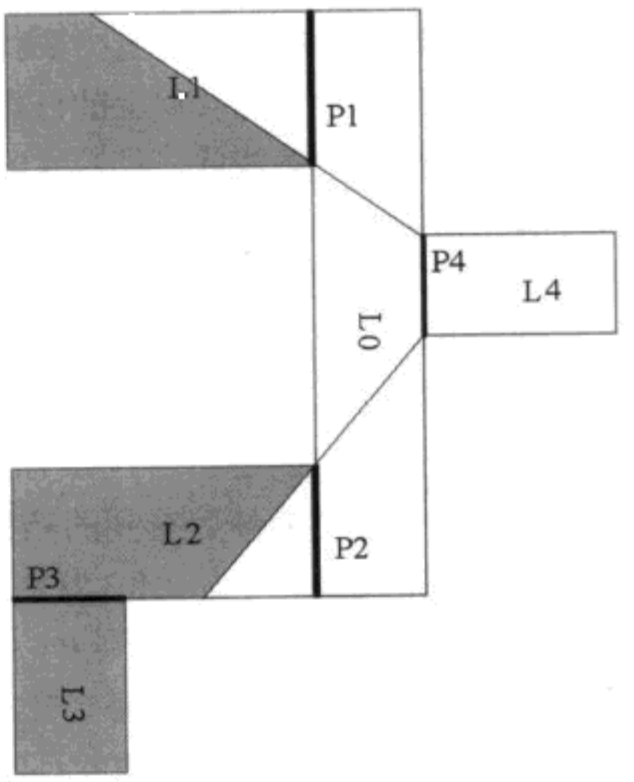


图 1.5.15　叶子 L4 的全景视图

我们本节内容的目的是：利用一个简单的算法，生成 80%～95%PVS 精确的数据。算法的基础就是出入口之间的相对位置，以及朝向可见性（见图 1.5.16），并结合了出入口之间的 LOS（瞄准线）可见性（见图 1.5.17）。在图 1.5.16 中，出入口 P1 永远也看不到出入口 P2。这是因为，出入口 P2 完全位于出入口 P1 的前面，而且彼此并不朝向对方。反过来也是这样：出入口 P3 无法看到出入口 P1，因为出入口 P1 完全位于出入口 P3 的后面，并且彼此并不相对。另外，无论其朝向如何，出入口 P3 也永远无法看到出入口 P4。因为它们位于同一个平面中。但是，出入口 P4 可以看到出入口 P1，因为它们彼此都位于对方的前面，而且彼此都朝向对方。

Download at www.Pin5i.Com

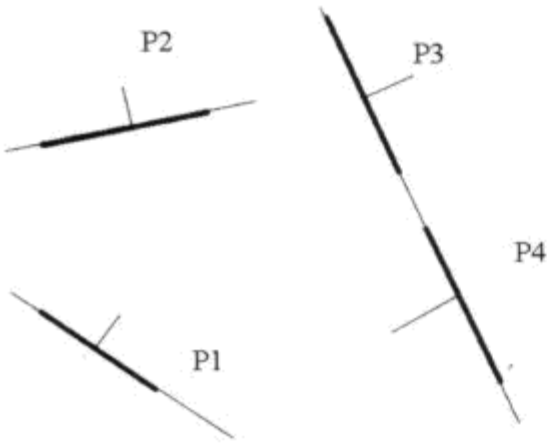


图 1.5.16 出入口之间的朝向

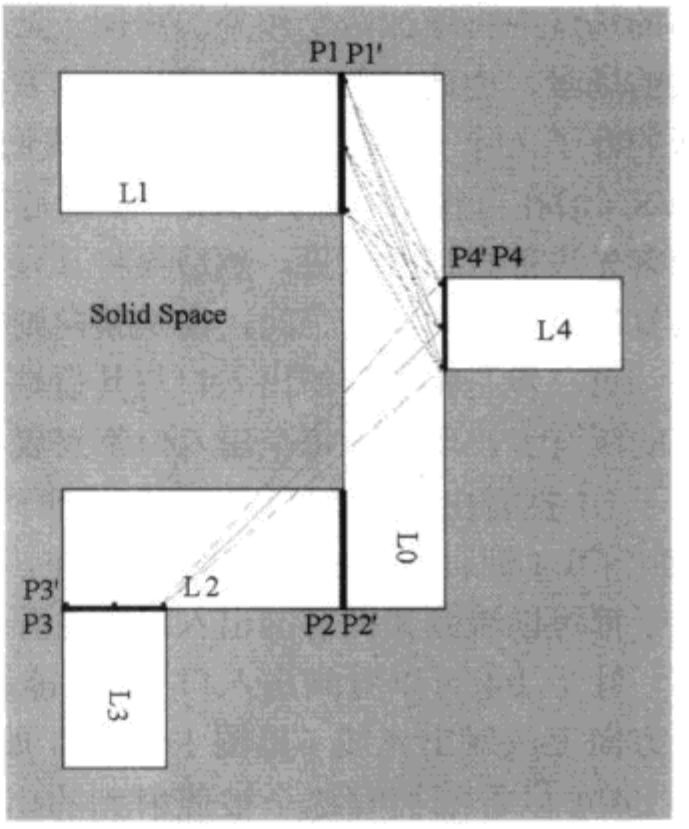


图 1.5.17

既然我们已经完成了基本的步骤，现在我们就可以进行 LOS（瞄准线）可见性测试了。在图 1.5.17 中，我们的任务就是至少要找到这样一条线段，从某个出入口的任意一点上发射出一条线段，连到另外一个出入口上的任意一点，中途不与 BSP（实体空间）发生碰撞。如果我们至少找到了这样的一条线段，我们就可以判定这两个出入口可以互相看到对方。如果有两个出入口可以互相看到对方，那么，它们的所有者（叶子）也可以互相看到对方。在实际的算法实现中，并不是所有的线段都可以从一个出入口区域连向另外一个出入口区域，如果这样我们的算法会变得出奇的慢。因此，我们只会选择少数的几个线段进行反射，并且，它们会均匀地分布在一个出入口的区域内（见图 1.5.17）。

图 1.5.17 所展示的是，出入口 P1'和 P4'之间，以及出入口 P3'和 P4'之间的可见性检测。出入口之间的交叉检测最后得到的结果是：出入口 P1 可以看到出入口 P4。因为在它们之间，至少存在着一条线段，不与 BSP 发生碰撞。而出入口 P3 则看不到出入口 P4，因为从出入口 P3 发出的线段没有一条能够连到 P4 上（在不与 BSP 发生碰撞的前提下）。

PVS 数据分配情况如下：每个叶子分配 5 个可见性标识位。这些标识位表示着每个叶子的 PVS。如果叶子 L1 可以看到叶子 L2，那么，叶子 L1 中代表 L2 可见性的特征位就被设置为 1。在我们这个例子中，叶子 L4 可以看到叶子 L1、L2 以及 L0，因为叶子 L4 的出入口 L4'可以看到叶子 L1 的出入口、叶子 L2 的出入口 P2'，以及叶子 L0 的出入口 P4。对于这些反复过程的最终结果，参见表 1.5.1。

表 1.5.1 叶子之间的 PVS

叶　子	L0 的可见性	L1 的可见性	L2 的可见性	L3 的可见性	L4 的可见性
0	1	1	1	1	1
1	1	1	0	0	1
2	1	0	1	1	1
3	1	0	1	1	0
4	1	1	1	0	1

Download at www.Pin5i.Com

我们将所有的数据放到一大块内存中，然后将所有的叶子链接到它们各自的 PVS 集上。这样，最终的 PVS 集就创建完毕。根据 PVS 进行渲染，通常的代码可以参看下列伪代码：

```
void Render(Point camera_pos)
{
    leaf current_leaf = Get_Camera_Curent_leaf(camera_pos,
        ROOT_NODE);
    for_each(leaf in leaves)
    {
        BYTE* pPvs = leaf.pPVS;
        If(pPvs[current_leaf.leaf_index]
            Render_leaf(leaf);
    }
}
```

其中，函数 Get_Camera_Curent_Leaf（）根据 BSP 对摄像机进行分类，直到它达到当前的叶子。从当前的叶子开始，我们对所有其他的叶子进行 PVS 值的检测，以此判断是否应该渲染它们的多边形。如果你能够对 PVS 渲染算法进行仔细的研读，你就会发现，它完全忽略了 back-to-front 渲染方法，却与节点 BSP 技术非常的类似。即便实体叶子 BSP 的渲染例程可以执行经典的 back-to-front 渲染方法，这种渲染方法也并不是总会令人满意。从给定的叶子开始，我们可以遍历整个二叉树，用可见性标识位来标识所有的节点。这个技术将 PVS 从单一叶子的层面扩展到了若干个大型的叶子集群，可以帮助我们根据 PVS 来选择剔除 BSP 的某些整个的分支。

每当摄像机从一个叶子移动到另外一个叶子，它需要执行下列操作：为可见性计数器增加一个增量，然后，将当前叶子中所有的 PVS 叶子的值设置为这个可见性计数器的值。从 PVS 中每一个叶子开始，在 BSP 树中一直向上遍历，直到最后到达根节点。在向上遍历的过程中，还要将所有遇到的节点标识为当前可见性计数器的值。在向一个新的 PVS 叶子遍历的过程中，如果发现了一个以前被标识过的节点，那么就停止向上遍历。如此这般，我们就将 PVS 从叶子层面拓展到了整个 BSP 分支。只要摄像机的位置仍然在一个叶子上徘徊，我们就没必要去重新标识所有的可见叶子，或者重新去标识父节点。这样我们就可以大大提高 BSP 渲染的性能了。

1.5.8　PVS 压缩

我们在这里给出了在 Quake 引擎中所使用的非常流行的以 bit 为单位逐一进行 PVS 测试的公式[Abrash97]：

```
Pvs[i_>>3] & (1<<(i_&7)) or Pvs[i/8] & (1<<(i&7))
    // 用于处理 unsigned char 类型排列的 PVS 缓冲区

Pvs[i_>>5] & (1<<(i_&31)) or Pvs[i/32] & (1<<(i&31))
    // 用于处理 unsigned long 类型排列的 PVS 缓冲区
```

上面是对叶子 i 进行的一个简单的 PVS 位（bit）测试。这个节点 i 类似于以字节为单位逐一进行测试的方法中的 Pvs[i]。这个 PVS 已经被压缩了 8 倍。我们不需要使用 25 个字节，

Download at www.Pin5i.Com

为了安全起见，我们推荐使用 25/8，四舍五入为最接近的字节数：

```
PvsSz = ( (sz + 7) / 8 );
```

这个场景是由 5 个叶子组成的，每个节点产生一个字节。在计算得到 PVS 数据的大小后，我们就分配一个连续的缓冲区来保存这些 PVS 数据，并且让每个叶子记住在这个连续的 PVS 缓冲区中它的 PVS 索引值。

接下来，我们来看看如何在 PVS 缓冲区中一个给定的偏移量上存储一个 bit（位）。在上面那个以 bit 为单位的 PVS 公式中，第一部分使用的是字节偏移量（i/8），而第二部分则使用的是字节中的 bit（位）。这样，对于左侧一个介于 0 和 8 之间的一个 I 的值，上述公式会产生 Pvs[0]；对于在 8 和 16 之间的一个 I 值，就会产生 Pvs[2]，依此类推。

在上述 PVS 公式的第二部分，（1<<i&7）在这个字节中，将我们的 bit 从左到右移动到索引位。“i&7”是 i 以 7 为模，将范围限定在 0～7 之间。5 个叶子的 Pvs[leaf/8]&（1<<leaf&7）总是指向第一个字节（每个叶子一个字节）。我们并不是使用字节索引来测试叶子 i 的 Pvs，而是通过检测索引位 i 上的 bit 值，并应用面向 bit 的 PVS 公式，来测试叶子 i 的 Pvs。最后，我们就可以在一个渲染显示例程中，结合使用 Back-to-front 渲染方法、潜在可视集（PVS），以及视锥剔除法等。

```
//  PVS 的 Back-to-front 渲染和视锥
void RenderBack2Front(Vertex camera_pos, NODE node)
{
    leaf current_leaf = Get_Curent_leaf(camera_pos,
        ROOT_NODE);
    if(current_leaf != _cached_cam_leaf)
    {
        frame_counter++;
        _cached_cam_leaf = cam_leaf;
        BYTE* pPvs = current_leaf.pPVS;
        for_each(leaf in leaves)
        {
            index = leaf.index;
            if(pPvs[index >> 3] &
                (index<<( index & 7)))
            {
                leaf.frame = frame_counter;
                while(node = leaf.parent)
                {
                if(node.frm_counter ==
                    frame_counter)
                break;
            }
            node.frm_counter = frame_counter;
            }
      }
    }
    Recurse_B2F_Render(camera_pos, node, current_leaf);
}
```

Download at www.Pin5i.Com

```
void Recurse_B2F_Render(Point camera_pos, NODE node)
{

    // 剔除 PVS 集群
    if(node.frm_counter != frame_counter)
        return;
    if(Out_Of_Frustrum(camera_pos,::BoundingBox(node)))
        return;
    if(::Isleaf(node))
    {
        Render_leaf(node);
        return;
    }
    side = node.plane.GetSide(camera_pos) >=0;
    Recurse_B2F_Render (camera_pos, side ?
        node-> node->pNodeFront:
        node-> node->pNodeBack);
    Recurse_B2F_Render (camera_pos, side ?
        node-> node->pNodeBack):
        node-> node->pNodeFront);
}
```

1.5.9　地形 BSP

在本文的结束部分，我们来探讨一下如何根据地形环境来创建一棵地形 BSP 树。虽然这并不是一个常见的应用，但是，通过快速视锥剔除，一个地形 BSP 树可以大大提高系统剔除不可见几何体的速度，参见图 1.5.19（图像选自 Getic BSP Editor）。要完成这个视锥剔除操作，我们可以在 Getic BSP 编辑器中，去观察一个地形 BSP 环境中被剔除的叶子。对于地形而言，通过在 X 轴和 Z 轴之间的变换，BSP 编译器会生成虚拟分割平面。为了将一个地形的最大范围分割开来，我们通常都会引入虚拟分割平面。

分割过程所使用的算法可以沿用前面节点 BSP 一节中的同一个算法。对于前端节点列表和后端节点列表，我们不断地重复这个过程，直到多边形的数目达到一个我们预先设定的数量，也就是每个叶子/节点的多边形数量限制。在达到这个外部限制时，我们就可以选定一个分割平面了，因为剩下的几何体都位于这个平面的前端。在一个叶子包围盒的 6 个面中，我们选择其中一个面向叶子中心的面，在这个面上来创建上一个分割平面。所有的多边形都存储在一个叶子节点中。图 1.5.18 显示的是一个视锥中可见的叶子。图 1.5.19 显示的是在一个地形网格上的一个单一分支上进行的分割过程。随书光盘中附带的 BSP 编译器包括了本文所讲到的全部内容。

1.5.10　总结

即使现在业界已经有了视频卡上的剪贴技术，BSP 技术仍然是一个大部分游戏引擎中所采用的最主要的分割算法。读者可浏览、试用随书光盘中完整的 BSP 编译器。

Download at www.Pin5i.Com

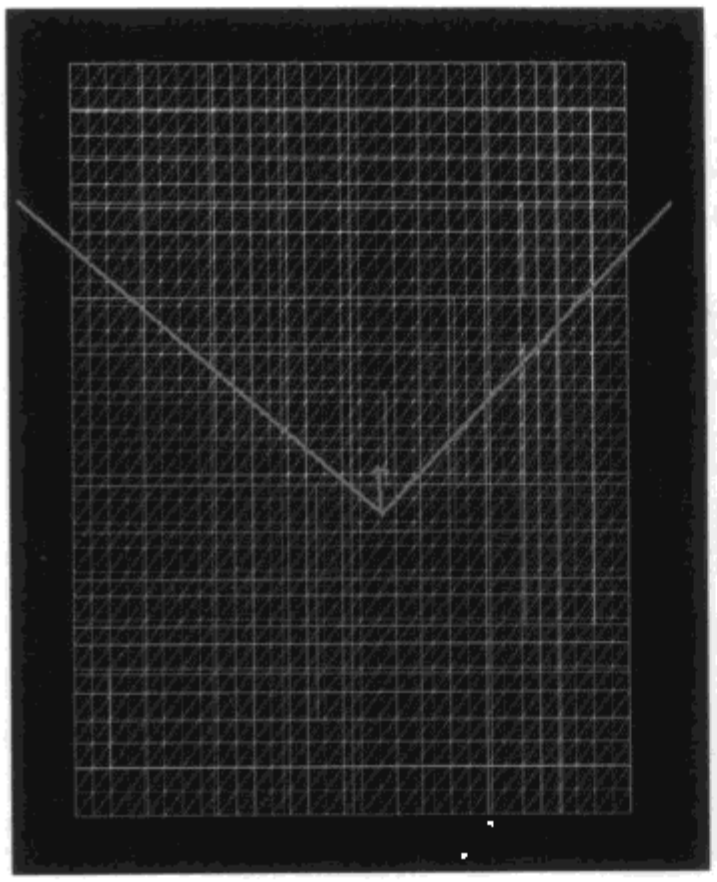


图 1.5.18 通过视锥进行叶子剔除

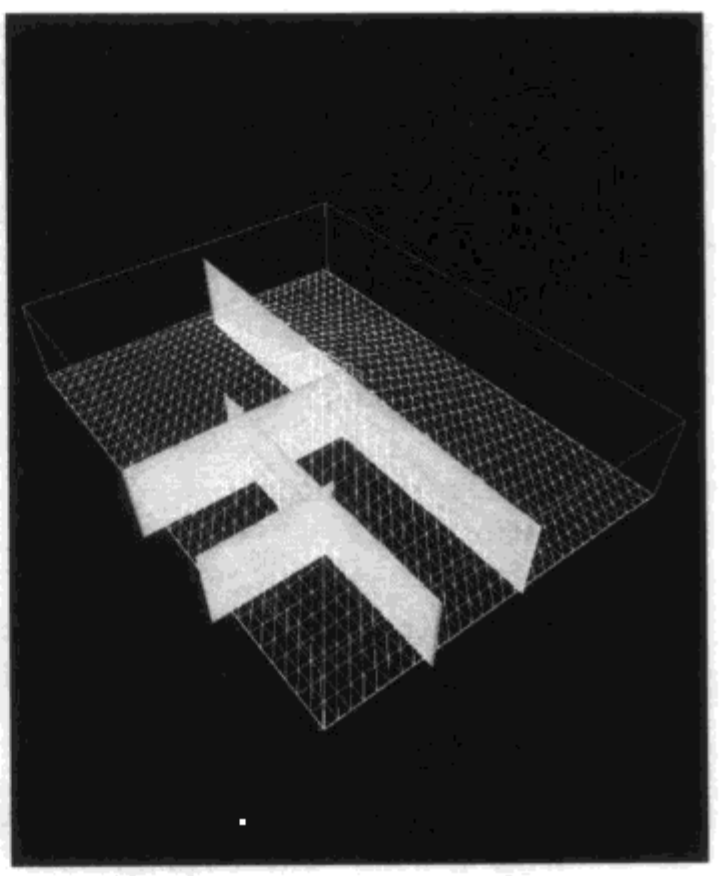


图 1.5.19 对一个地形实施分割过程

1.5.11 参考文献

[Abrash97] Abrash, Michael, *Graphics Programming Black Book.* Coriolis Group Books 1997.

Download at www.Pin5i.Com

1.6　最相似字串匹配算法

ArenaNet 公司　James Boer
author@boarslair.com

在本文中，我们将向大家介绍字符串匹配算法。我们可以用这个算法来得到任意两个字符串之间匹配的程度。更为重要的是，我们还会向大家演示一下，如何在一个便捷的错误报告机制的核心部分来使用这个算法。在这样的错误报告机制中，所有的数据都是通过基于字符串的 ID 来索引，排序的。如果没有找到完全匹配的字符串 ID，这个错误报告机制就会为用户给出一个最佳的猜测，告诉用户应该使用哪个最相似的字符串来代替使用。关于这个技术的应用，最好的一个例子就是 Google 搜索引擎。在本文中，我们还会带领大家讨论一种新的方法，可以将这个技术功能的一个非常简单的版本应用到游戏数据库中。

1.6.1　基于字符串的 ID 查找难题

随着游戏开发人员越来越多地使用数据驱动的开发方法，对于代码和数据之间的接口，其管理难度也与日俱增。在创建易于理解、可读性很强的数据元素时，我们经常会使用字符串来作为一种自我描述形式的辨别方法，来区分所有类型的数据元素。在游戏开发的过程中，当我们在一个脚本或一个数据表中键入这些字符串标识符时，难免会出现输入错误的情况。为了提高效率，对于这些类型的数据查询，我们一般都会要求精确的字符串匹配。因此，即使是一个特别微小的输入错误，也会导致字符串查询的彻底失败。这样的话，开发人员就不得不去搜索，并用正确的字符串去替换掉那些错误的字符串。

但不幸的是，人类会以无数种方式去制造出千奇百怪的错误。我们明明看着输入的是“SallyExample”，但搜索结果显示，最合理的匹配字串居然是“SillyExample”。但是，这仅仅是在处理非常小规模的数据集时，这样的错误是比较明显的，也还可以及时发现。人类并不擅长去处理大规模的数据集。举个例子，假设你本来要输入的字符串是“LightRedMagicFlash”，但是却错误地输入成了“BrightRedMagicFlash”，这样的错误就很难去处理了。即使我们有一个按字母顺序排列的键值列表，它的作用也是杯水车薪。因为，上面两个字符串的第一个字母根本就不匹配。

Download at www.Pin5i.Com

1.6.2 问题的定义

对于这种字符串 ID 的查询问题，我们已经找到了一些简单的解决方法。如果没有找到某个特定的字符串，我们就使用最相似字串匹配算法，去检索所有可用的 ID 字符串，从中找到、并向用户显示最相似的搜索结果。虽然从应用的角度上看，这是一个非常简单的概念，但事实证明，模糊字串匹配本身也是一个不太容易解决的问题，这主要是因为这个算法的效率问题。因此，业界人士也进行了大量的探索工作，试图可以找到一种高效率的方式，来解决这个问题。

大部分的解决方案都有一些特殊的要求，要么是要求为核心算法分配一定的空间，要么是需要搜索空间按照一个特定的方式来组织。但是，对于我们这个特殊的问题，最适合的解决方案是那些无需在运行时效率最优化，但却需要在资源分配和代码大小这两个方面最优化的解决方案。

设计需求所要求的是一个简单的函数：给定两个普通的 C 风格的 null-terminated 的字符串（以 null 结尾的字符串），该函数就会返回一个介于 0 和 1 之间的浮点数，表示这两个字符串之间的匹配度（两个字符串之间的相似程度）。理想状态下，这个函数也应该有很高的运行时效率，不需要任何的存储空间（只有几个变量需要一小块的堆栈空间）和运行时的空间分配。这是因为，我们希望这个算法去执行的只是很多次微型的搜索操作，而不是在一个大型数据集上进行传统的搜索操作。而空间分配的系统开销肯定会超过搜索算法本身的系统开销。

1.6.3 现有的一些解决方案

实际上，我们真正要寻找的是 Levenshtein 距离算法（或称为莱文斯特公式）的一个更为实用的实现[Wikipedia05]。这个公式是由俄国科学家 Vladimir Levenshtein 在 1965 年发明的。根据从一个字符串转换成另外一个字符串所需要的编辑次数，这个公式可以描述出两个字符串的相似度。

对于我们上面所描述的实现需求，单有这个公式是不够的，因为我们需要一个相似度指数，以便可以建立一个有意义的入口。但是，我们可以很容易地将这个公式改造成一个指数，因为我们可以定义最长字符串的长度上限。

对于我们上述的这些考虑，业界有一个潜在的竞争者，那就是 Bitap 算法，也被称为 shift-or 算法或者是 Baeza-Yates-Gonnet 算法。Bitap 算法会生成位掩码（bit mask），来表示匹配字串的值，以此作为一种依附于莱文斯特公式的手段。虽然 Bitap 算法非常之快，但是，这个算法需要分配一个数组空间，其大小等于比较字符串的长度。我们前面已经做出了选择，尝试去创造一个新的解决方案，一个不需要额外存储开销的解决方案。即使是在算法层次上，这个方案也不会带来任何运行时效率的损失。另外，我们所要求的这个算法，可以将不正确的大小写匹配，与常见的一对字符不匹配的情况区别开来。

1.6.4 我们自己定制的字串匹配解决方案

那么，我们又该如何来定义一个“相似匹配”呢？还是让我们来看一下前面提到的假设

Download at www.Pin5i.Com

的关键字比较：BrightRedMagicFlash 与 LightRedMagicFlash 之间的比较。从这两个字符串的比较中，我们就可以从下面了解到，我们这个算法在进行匹配判断时所使用的基本规则。

1. 匹配规则

首先，我们要判断出两个字符串中比较长的那一个，并以它的长度作为长度计算的基准值。在我们这个例子中，比较长的字符串是“BrightRedMagicFlash”，其长度为 19。每个参与匹配的字母表示着一个用 1 除以这个长度基准值所得到的值。在我们这个例子中，这样操作之后，每个字母就可以得到一个近似值 0.0526。每个匹配的字母会将这个值添加到一个总分值上（匹配值）。对于我们这个特定的例子，我们最后可以得到一个匹配值 0.947，表示这是两个非常相似的字符串。

对于大小写错误，我们可以向匹配值中增加正常值的一个百分比值。在我们的实现中，如果碰到大小写错误，我们选择的百分比是 90%。对于位置不正确的文本，我们只会计算它一半的值。这是因为，为了有利于前一个子集匹配，我们会放弃上一个最新发现的匹配。这看上去是一个公平的估值计算，可以允许合理数量的位置错误字符。

最后，缺少的、或者多出来的字符会使精确度的值成比例下降。这个比例就是实际匹配字符集的长度与正确键值长度的比。换句话说，如果正确的键值有 10 个字符，而一个潜在的匹配只有其中的 9 个字符，那匹配结果就是 90%。

你也许会注意到，我们这个规则集与莱文斯特距离公式非常相似。根据我们的估算，只要我们依旧忠实于我们最初的设计意图，就没有必要对字母严格地使用上述规则。

2. 具体算法

下面让我们来仔细研读一下函数 stringMatch（），看看它是怎么工作的。

程序清单 1.6.1　函数 stringMatch（ ）

```
const float CAP_MISMATCH_VAL = 0.9f;

// 这就是我们这个字符串匹配算法。它
// 会返回一个介于 0 到 1 之间的浮点值，表示两个字符串
// 之间匹配度的一个近似百分比。
float stringMatch(char const *left, char const *right)
{
    // 获得左右两个字符串，及最长字符串的长度值
    // （以此作为赋值计算中进行比较的基础）
    size_t leftSize = strlen(left);
    size_t rightSize = strlen(right);
    size_t largerSize = (leftSize > rightSize) ?
        leftSize : rightSize;
    char const *leftPtr = left;
    char const *rightPtr = right;
    float matchVal = 0.0f;
    // 对左侧的字符串进行迭代操作，直到字符串的最后一个字符
    while(leftPtr != (left + leftSize) &&
        rightPtr != (right + rightSize))
    {
```

Download at www.Pin5i.Com

```
        // 首先，我们进行一个简单的左/右匹配检测
        if(*leftPtr == *rightPtr)
    {
        // 如果匹配，就向匹配总值（matchVal）加上这个字符的百分比值
        matchVal += 1.0f / largerSize;
        // 如果还没有到字符串的结尾，那么就将指针前进一步
        if(leftPtr != (left + leftSize))
            ++leftPtr;
        if(rightPtr != (right + rightSize))
            ++rightPtr;
    }
    // 如果简单匹配失败，
    // 那么就尝试一个忽略大小写的匹配检测
    else if(::tolower(*leftPtr) == ::tolower(*rightPtr))
    {
        // 我们把一个因为大小写不同而匹配失败的值考虑为
        // 正常值的 90%
        matchVal += CAP_MISMATCH_VAL / largerSize;
        if(leftPtr != (left + leftSize))
            ++leftPtr;
        if(rightPtr != (right + rightSize))
            ++rightPtr;
    }
    else
    {
        char const *lpbest = left + leftSize;
        char const *rpbest = right + rightSize;
        int totalCount = 0;
        int bestCount = INT_MAX;
        int leftCount = 0;
        int rightCount = 0;
        // 这里我们在外层循环中遍历整个左字串，
        // 但是为了确保不会越过我们当前的最佳数量（bestCount），
        // 我们也会进行提前推出循环的条件检测
        for(char const *lp = leftPtr; (lp != (left + leftSize)
            && ((leftCount + rightCount) < bestCount)); ++lp)
        {
            // 内部循环计数
            for(char const *rp = rightPtr; (rp != (right +
                rightSize) && ((leftCount + rightCount) <
                bestCount)); ++rp)
            {
                // 在这里，我们不考虑大小写的分别
                if(::tolower(*lp) == ::tolower(*rp))
                {
                    // 这是“健康度”检查
                    totalCount = leftCount + rightCount;
                    if(totalCount < bestCount)
                    {
                        bestCount = totalCount;
```

Download at www.Pin5i.Com

```
                        lpbest = lp;
                        rpbest = rp;
                    }
                }
                ++rightCount;
            }
                ++leftCount;
                rightCount = 0;
            }
            leftPtr = lpbest;
            rightPtr = rpbest;
        }
    }
    // 为了防止浮点错误，将 matchVal 的值进行范围限定
    if(matchVal > 0.99f)
        matchVal = 1.0f;
    else if(matchVal < 0.01f)
        matchVal = 0.0f;
    return matchVal;
}
```

这个函数多少有点长，大家很难马上理解。所以，下面我们将其进行分解，逐段地进行分析、讲解，看看它到底是怎么运作的。（你也可以在随书光盘中看到所有的这些代码）

一上来你就会注意到，我们首先计算的是两个字符串之间最长的那个字符串的长度。我们利用这个长度值来确定，每个潜在的匹配字符应该赋予什么值。每个字符的值应该是：1/（最长字符串的长度）。

```
float stringMatch(char const *left, char const *right)
{
    // 获得左、右两个字符串，及最长字符串的长度值
    // （以此作为赋值计算中进行比较的基础）
    size_t leftSize = strlen(left);
    size_t rightSize = strlen(right);
    size_t largerSize = (leftSize > rightSize) ?
        leftSize : rightSize;
    char const *leftPtr = left;
    char const *rightPtr = right;
    float matchVal = 0.0f;
```

现在这个作业完成了，我们就要开始主循环了。其中的 while 循环将会一直持续下去，直到左循环指针（leftPtr）或者是右循环指针（rightPtr）指到了字符串的尾部。

```
// 对左侧的字符串进行迭代操作，直到字符串的最后一个字符
while(leftPtr != (left + leftSize) &&
    rightPtr != (right + rightSize))
{
    ...
}
```

Download at www.Pin5i.Com

我们首先测试的是最简单的情况——完全匹配。在这种情况下，如果有一个字符匹配成功，就将它的值增加到匹配总值中（matchVal），然后将左循环指针（leftPtr）和右循环指针（rightPtr）分别前进一步。我们不必担心这个循环会跑出这个数组的范围之外，因为我们在每个循环的尾部都检查是否会越界。

```
// 首先，我们进行一个简单的左/右匹配检测
if(*leftPtr == *rightPtr)
{
    // 如果匹配，就向匹配总值（matchVal）加上这个字符的百分比值
    matchVal += 1.0f / largerSize;
    // 如果还没有到字符串的结尾，那么就将指针前进一步
    if(leftPtr != (left + leftSize))
        ++leftPtr;
    if(rightPtr != (right + rightSize))
        ++rightPtr;
}
```

如果完全匹配失败，我们接下来就进行一个大小写无关的比较。在这次比较中，如果两个字符匹配成功，我们只向匹配总值（matchVal）中增加单个匹配字符值的 90%，然后还要将左循环指针（leftPtr）和右循环指针（rightPtr）分别前进一步。

```
// 如果简单匹配失败，
// 那么就尝试一个忽略大小写的匹配检测
else if(::tolower(*leftPtr) == ::tolower(*rightPtr))
{
    // 我们把一个因为大小写不同而匹配失败的值考虑为
    // 正常值的 90%
    matchVal += CAP_MISMATCH_VAL / largerSize;
    if(leftPtr != (left + leftSize))
        ++leftPtr;
    if(rightPtr != (right + rightSize))
        ++rightPtr;
}
```

到此为止，前两个匹配检测工作就完成了。我们下面要进入这个算法的真正核心所在。下面的代码会尝试将两个字符串指针向前推进，推进到下一对最理想的匹配字符上。（请大家注意，在这个匹配检测中，我们并不考虑字符的大小写，因为我们没必要将我们的算法复杂化）

下面的代码利用一个外部循环和一个内部循环，将左实验指针（lp）和右实验指针（rp）向前推进，在所有新的匹配组合中进行循环遍历，来完成上述工作。在左/右两个字符串中，我们应该推进几步才能得到一对匹配的字符？这两个值记录在 leftCount 和 rightCount 中。根据这两个值，我们就可以计算出一个匹配度值。将这两个值相加，我们就得到最后的分值（totalScore）。这是为了防止出现最坏匹配的情况。在这种情形下，一个无关的字母与另外一个字串中接近尾部的某个字符相匹配。我们将这样的匹配标识为低匹配度的匹配，并选择其他更为合理的，在两个字符串中位置更靠前的匹配。

Download at www.Pin5i.Com

```
char const *lpbest = left + leftSize;
char const *rpbest = right + rightSize;
int totalCount = 0;
int bestCount = INT_MAX;
int leftCount = 0;
int rightCount = 0;
// 这里我们在外层循环中遍历整个左字串，
// 但是为了确保不会越过我们当前的最佳数量（bestCount），
// 我们也会进行提前推出循环的条件检测
for(char const *lp = leftPtr; (lp != (left + leftSize)
    && ((leftCount + rightCount) < bestCount)); ++lp)
{
    // 内部循环计数
    for(char const *rp = rightPtr; (rp != (right +
        rightSize) && ((leftCount + rightCount) <
        bestCount)); ++rp)
    {
        // 在这里，我们不考虑大小写的分别
        if(::tolower(*lp) == ::tolower(*rp))
        {
            // 这是“匹配度”检查
            totalCount = leftCount + rightCount;
            if(totalCount < bestCount)
            {
                bestCount = totalCount;
                lpbest = lp;
                rpbest = rp;
            }
        }
        ++rightCount;
    }
    ++leftCount;
    rightCount = 0;
}
```

另外，我们也充分利用了当前最佳匹配，以避免对某个字符串进行不必要的深层搜索。如果两个字符串并不匹配，这样做并不会对这个算法的最坏情况有所帮助，但是，通过对搜索深度的限制，我们确实可以提高匹配检测过程的平均速度。

```
for(char const *lp = leftPtr; (lp != (left + leftSize)
    && ((leftCount + rightCount) < bestCount)); ++lp)
{
    for(char const *rp = rightPtr; (rp != (right +
        rightSize) && ((leftCount + rightCount) <
        bestCount)); ++rp)
    {
```

当我们得到了具有最佳“匹配度”得分的匹配时，我们就将这些指针复制到我们的主叠代指针中，然后再重复这个循环。

Download at www.Pin5i.Com

```
leftPtr = lpbest;
rightPtr = rpbest;
```

最后，我们一定要将最后的结果限定在 0.0 和 1.0 之间，避免出现任何潜在的浮点错误。

```
if(matchVal > 0.99f)
    matchVal = 1.0f;
else if(matchVal < 0.01f)
    matchVal = 0.0f;
return matchVal;
```

3. 一些可能的优化措施

毋庸质疑，这并不是一个最优化的算法。虽然我们已经进行了一些最基本的优化工作，但还是有很多的方法，可以将我们这个解决方案改造成一个更高效的解决方案。但是，采用某些经典的字符串匹配算法和技术，有两个基本的障碍。

首先在开始的时候，我们希望利用一个没有额外存储要求，也不需要动态分配的算法。除此之外，我们提出的这个算法不允许使用先进的技术（诸如：创建索引值的数据哈希表等）。这些苛刻的要求严重地限制了我们可用的选择，算法的声明必须被限制在一个固定数量的代码变量上。

坦白地讲，这个算法的最佳性能根本就不是一个首要考虑的方面，因为这个代码只在异常情况下才会运行。而在游戏正常的执行过程中，我们根本不会去调用这个函数。如果你认为有必要，那么我们就将这个算法的优化作为练习题，留给读者朋友们去完成。

1.6.5 解决方案的实际应用

我们最初的意图并不是作为学术练习来创造出一个模糊字串匹配算法。我们的目的是在一个基于字符串的标识符键值匹配不成功时，能够通过我们的这个算法，给出更有价值的错误信息。程序清单 1.6.2 向我们展示了一个简单的基于模板的函数。任何将 std::string 用作键值的 STL map 都可以使用这个函数。

程序清单 1.6.2 closeMatch 函数

```
template<class _InIt> inline
_InIt closestMatch(_InIt _First, _InIt _Last, char const *val,
    float limit)
{
    float maxVal = limit;
    InIt _max = _Last;
    for (; _First != _Last; ++_First)
    {
        float newVal = stringMatch(_First->first.c_str(), val);
        if (newVal > maxVal)
        {
            maxVal = newVal;
            _max = _First;
        }
```

Download at www.Pin5i.Com

```
        }
        return (_max);
    }
```

这个特殊的函数对一个 map 的所有键值执行一个线性搜索，并返回一个最相似匹配的叠代器（iterator）。这个匹配至少指向一个 limit 的值。然后，系统就可以显示出一个更有意义的错误信息，例如：

没有找到名为“Key Word”的对象 ID。你是否要查找的是“keyword”？

如果在 limit 当前值的范围内没有找到匹配字串，那么这个代码就会显示一个更为普通的错误信息。

1.6.6　总结

通过使用一个相当简单的技术，对于那些采用基于字串 ID 查询的游戏引擎，我们现在可以提供更好的调试反馈信息。如果 ID 拼写错误，我们这个算法还可以提供一个合理的建议，而置换工作也非常容易实施。对于这些问题，虽然业界已经有了很多的解决方案，但是，它们无法满足我们这个项目的特殊需求。

只需要很少的工作量，我们就可以将这个基本的错误处理机制进行集成和扩展。这样，当我们在查找一个字符串 ID 时，任何简单的拼写错误会很快地得到修正。这样的工作变得越来越轻松。

1.6.7　参考文献

[Wikipedia05] “Levenshtein distance.” September 20, 2005. Available online at *http://en.wikipedia.org/wiki/Levenshtein_distance*.

Download at www.Pin5i.Com

1.7 利用 CppUnit 实现单元测试

Oleander Solutions 公司 Blake Madden
Blake.madden@oleandersolutions.com

在产品质量保证王国中，我们有两个最基本的测试领域：黑盒测试和白盒测试。黑盒测试指的是由公司的质保（QA）团队，对一个完整集成系统所进行的测试。这个测试包括了基于脚本的功能测试，以及手工测试（可以是松散的，非系统性的测试，也可以按照既定的测试计划进行）等相关的技术。与黑盒测试不同的是，白盒测试则是由开发团队所进行的质量确认工作。这个工作是要在 QA 团队的测试工作开始之前开展的。从传统角度上讲，白盒测试通常包括：同级评审（peer review）和单元测试。关于单元测试，我们会在本文后面的部分进行专门的探讨。从本质上讲，单元测试就是利用自动测试，针对某个软件系统中各个小型组件，在开发人员的层面上，去验证这些组件的运行结果和运行行为。在这个过程中，我们不用去考虑系统通常是如何使用这些组件的。

虽然听上去，单元测试与自动功能测试非常类似，但实际上，它们是完全不同的两种测试方法，也是彼此互补的两种测试方法。功能测试是一种严格的、高层次的全系统测试方法；而单元测试则是一种低层次的、模块化的测试方法，让我们可以在一个更为孤立的层面上，去进行特定的类（Class）和函数的测试。也就是说，在我们对一个函数进行测试时，不用去考虑系统的其他部分是如何使用这个函数的，也不必去考虑其他的类和函数是如何与之进行交互的。

1.7.1 单元测试技术概览

和高层次的自动测试一样，在单元测试中，我们最好也创建一些小的应用程序，与实际游戏的代码库分离开来。这些小的应用程序有时候也被称为测试装具模块（test harness）。这些测试装具模块（test harness）应该包含你所有的测试用例，以及这些测试用例的运行及运行结果的审核。通过这种方式，你就可以创建一些小型的、本地化的测试程序。这些测试程序可以只包含代码库中那些你所需要的文件。这一点也是非常重要的。这样我们就不会将单元测试代码与实际游戏代码，特别是游戏的发行版本混在一起。

单元测试就是一些小型函数的集合。对于一个给定的功能，我们利用这些函数来测试所有可能的情况，确认这个功能在正常情况下只会产生我们预期的行为，并且足够的稳定可靠，可以去处理潜在的灾难性的出错情况。

Download at www.Pin5i.Com

单元测试最主要的好处在于：因为单元测试是与系统的其他部分隔离开来的，你可以很容易地创建单元测试，并向被测试的函数传递任何类型的数据，而不必去利用真实的系统数据，一步一步地执行整个游戏。单元测试的另一个好处是：你的测试装具模块（test harness）不仅会去运行你自己的测试用例，还可以进行测试结果的比较。对于所发生的所有系统冲突或意外的系统错误，它还可以生成一个详细的报告文件。从这个角度来看，我们几乎可以把单元测试看作是一个智能的调试程序，它可以帮助你完成大部分的测试工作（参见图 1.7.1）。

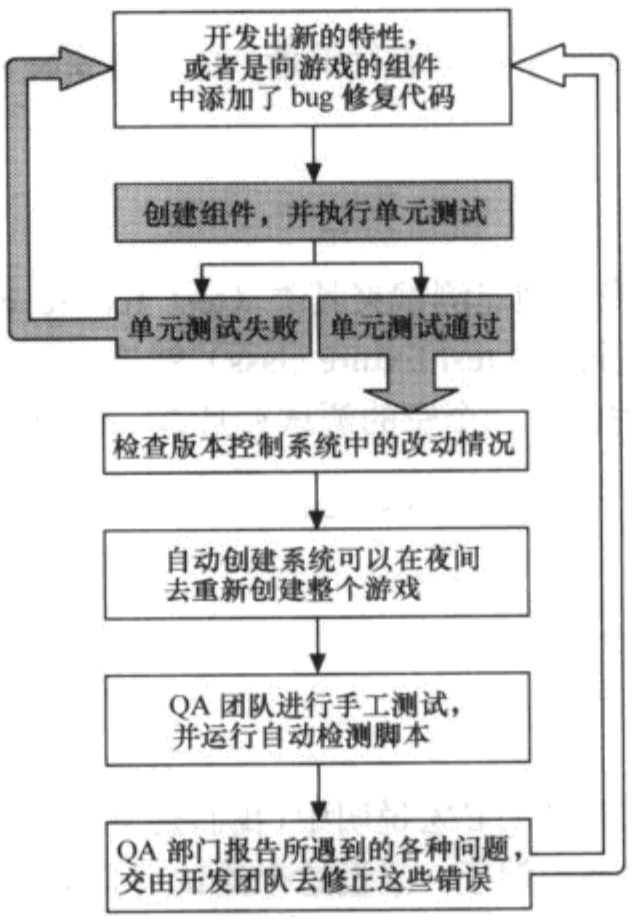


图 1.7.1　单元测试在整个测试过程中的适用环节

我们可以为现有的函数编写单元测试代码，也可以在其他代码编写之前进行单元测试代码的编制。在测试驱动的环境中，后面这个技术非常流行。在系统仍然处于设计阶段时，我们就可以开始准备单元测试了。为了便于描述，本文假设你将为一个现有的代码库创建一个单元测试。

1.7.2　CppUnit 概述

CppUnit 是一个 C++单元测试库，我们可以从因特网上免费得到[CppUnit05]。CppUnit 为我们提供了一个框架，便于我们针对具体的被测试函数去编制、组织，并运行单元测试。它还提供了很多测试结果报告方法，使你可以很容易地评估测试结果。CppUnit 有着丰富的特色功能，比如易于使用的条件检测宏。它可以处理所有的错误日志工作，并向你报告这些信息。这样我们就不用从头开始编制测试用例，使得测试用例的编制工作更加高效。

1. 包含 CppUnit

为了在测试程序中包含 CppUnit，你首先需要创建 CppUnit 库，然后作为依赖库文件将 cppunitd.lib（用于游戏的调试版本），或者 cppunit.lib（用于游戏的发布版本）包含在头文件

Download at www.Pin5i.Com

中。在你的 main 头文件中，请参考设置下列这些头文件：

```
#include <cppunit/TestCase.h>
#include <cppunit/extensions/HelperMacros.h>
#include <cppunit/ui/text/TestRunner.h>
#include <cppunit/XmlOutputter.h>
#include <cppunit/TestResultCollector.h>
#include <cppunit/CompilerOutputter.h>
```

最后，如果你是在 Windows 平台上进行编译，你还需要将你的运行时间库设置为“多线程 DLL”。

2. 创建测试用例

在 CppUnit 中，我们首先要创建测试夹具类（test fixture class）。这可以从 CppUnit::TestFixture 派生得到。测试夹具类（test fixture class）就是一个容器类，用来管理那些用于测试特定函数或类的所有测试用例。一个空的测试夹具类（test fixture class）大致如下：

```
class MathTest : public CppUnit::TestFixture
{
public:
    CPPUNIT_TEST_SUITE(MathTest);
    CPPUNIT_TEST_SUITE_END();
};
```

在其中的CPPUNIT_TEST_SUITE宏说明中，我们要说明需要这个测试夹具类(test fixture class）执行的所有测试用例。举个例子，假设这个测试夹具类（test fixture class）要测试一个 math 类（math class），而你又需要一个特殊的测试用例，来检测这个 math 类的 ADD 函数。你可以像下面这段代码一样，创建一个测试用例函数，来完成上述测试工作：

```
void TestAddFunction()
{
    math my_math;
    CPPUNIT_ASSERT(my_math.add(2,2) == 4);
}
```

在这个测试用例中，我们用（2，2）来调用这个 math 类的 ADD 函数。然后利用 CppUnit 提供的 CPPUNIT_ASSERT 宏来验证 ADD 函数调用的结果是否等于 4。如果 ADD 函数没有正确地返回结果 4，CppUnit 会在日志文件中将这个错误记录下来，并根据你所设置的报告输出方式，为你显示这个错误信息。在完成这个测试用例的编制工作后，你只需要将这个测试用例添加到测试夹具类（test fixture class）TestFixture 中，并添加到 CPPUNIT_TEST_SUITE 宏语句中，就像下面这样：

```
class MathTest : public CppUnit::TestFixture
{
public:
    CPPUNIT_TEST_SUITE(MathTest);
    CPPUNIT_TEST(TestAddFunction);
```

Download at www.Pin5i.com

```
    CPPUNIT_TEST_SUITE_END();

    void TestAddFunction()
    {
        math my_math;
        CPPUNIT_ASSERT(my_math.add(2,2) == 4);
    }
};
```

1.7.3　运行测试夹具

既然你已经完成了测试用例的编制工作，并将它们添加到一个测试夹具类（test fixture class）中进行管理，现在你就需要创建一个测试装具模块（test harness），让它来实际执行这个测试夹具类，并输出最后的运行结果。我们可以这样来做：创建一个 CppUnit::TextTestRunner 对象，然后将你的测试夹具类添加进来，设置结果输出方式，然后再运行这个对象。

对于这个例子，你可以将输出结果重新定向到一个 XML 文件中。这个 XML 文件使用了一个样式表来更好地显示输出结果，使其更便于浏览。第一步工作是创建测试的执行者（runner），一个 CppUnit::TextTestRunner 对象，就像这样：

```
CppUnit::TextTestRunner runner;
```

接下来，创建输出流，指向我们保存输出结果的 XML 文件。你可以简单地使用 std::ofstream 来实现：

```
std::ofstream ofs("tests.xml");
```

然后，我们要创建这个 XML 文件的一个句柄（是一个 CppUnit::XmlOutputter 对象），设置它的样式表，然后对测试执行对象 runner 进行设置，将其运行结果定向到这个 XML 句柄：

```
CppUnit::XmlOutputter* xml =
    new CppUnit::XmlOutputter(&runner.result(), ofs);
xml->setStyleSheet("report.xsl");
runner.setOutputter(xml);
```

最后，将所有你要执行的测试夹具类（test fixture class）添加到这个测试执行对象（runner）中，然后运行这个执行对象 runner：

```
runner.addTest( LoadModelTest::suite() );
runner.run();
```

在这个测试程序运行结束后，我们可以打开“tests.xml”文件，从中获得相应的报告信息：运行了多少个测试，多少个测试通过，多少个测试失败，以及关于那些运行失败的测试用例的详细信息。我们在随书光盘中提供了一个测试报告和样式表的例子，供大家参考。

Download at www.Pin5i.com

我们建议大家，首先运行测试程序，然后再向版本控制系统提交变更信息。为了方便起见，通过使用简单的 shell 文件或批处理文件，或者是功能更丰富的系统（比如 CruiseControl，你甚至可以将测试程序集成到版本创建过程中。

1.7.4 利用 CppUnit 进行模型类测试

下面我们举一个模型加载类的例子，向大家介绍一下如何对这样一个简单的模型加载类进行单元测试。假设你在游戏引擎中使用了一个定制的模型文件格式，并且编写了一个专用的类，来加载并渲染这些模型。下面就是这个类的原型：

```
class model
{
public:
    void load_file(const char* file_path);
    void render();
    void animate(float speed, bool loop = true);
    // 我们利用一些实用函数来加载模型文件
    void parse_header_section(char* file_text);
    void parse_triangle_section(char* file_text);
    void parse_mesh_section(char* file_text);
    void parse_material_section(char* file_text);
    void parse_animation_section(char* file_text);
    void prepare_joints();
    // 从文件中加载纹理（支持 BMP 格式）
    void load_texture(const char* file_path);
    // 获取文件信息的函数
    double get_version() const
        { return m_version; }
    model_type get_type() const
        { return m_model_type; }
    const char* get_name() const
        { return m_name; }
    const char* get_author() const
        { return m_author; }
    // 获取数据信息的函数
    size_t get_number_of_vertices() const
        { return m_number_of_vertices; }
    size_t get_number_of_triangles() const
        { return m_number_of_triangles; }
    size_t get_number_of_meshes() const
        { return m_number_of_materials; }
    size_t get_number_of_materials() const
        { return m_number_of_materials; }
    size_t get_number_of_joints() const
        { return m_number_of_joints; }
private:
    // private 数据
```

Download at www.Pin5i.Com

```
    double m_version;
    model_type m_model_type;
    char* m_name;
    char* m_author;
    unsigned short m_number_of_vertices;
    unsigned short m_number_of_triangles;
    unsigned short m_number_of_meshes;
    unsigned short m_number_of_materials;
    unsigned short m_number_of_joints;
    vertex_t* m_vertices;
    triangle_t* m_triangles;
    mesh_t* m_meshes;
    material_t* m_materials;
    joint_t* m_joints;
};
```

通过自动化(automation)操作，这个类最可能公开的是：load_file、render，以及 animate 这几个函数。因为无论是 QA 团队，还是创建游戏模式的玩家，他们对那些实用函数根本没什么兴趣。在你开发这个类的过程中，会不断地对这实实用函数进行多次的改动、修复一些 bug 等。在你进行这些改动时，如何去处理对这些改动工作的测试，一般会面临两种选择：

- 将这些变动集成到游戏的本地版本中，用各种各样的模型文件来测试这个版本，然后再核实所有这些变动的地方，并在源代码控制系统中登记注册这些变动。
- 只是简单地在源代码控制系统中登记注册这些变动，让 QA 部门在第二天来测试这些变化的地方。测试的方式可以通过手工游戏试玩测试，或者是进行脚本测试。

对于第一种选择，首先，你至少要重新创建这个系统中的一个部件（比如 DLL 文件，或者是 SO 文件）；然后，你要在调试程序中一步一步地运行。与此同时，你还要启动运行整个游戏，以便可以到达模型加载任务发生的地方。考虑到这些因素，第一种选择可以说是相当费时的。而第二种选择则带有某些风险性。因为 QA 部门可能并不知道你已经做出了修改，他们还会用大量的时间去跟踪这个问题，而这时一个核心的操作（比如加载一个关卡，或者是角色模型）就会立即造成整个游戏系统的崩溃？另外，一旦 QA 部门向你通报了这个问题，那么，你花在调试工作上的时间，以及你修复 bug 后花在新版本重建工作上的时间，对 QA 来说，就是在浪费时间。因为在这一天剩下的时间里，QA 部门只能面对一个不可用的版本。

面对这种两难的情形，单元测试就成了救兵。单元测试可以让你去检测验证那些微小的核心代码变化所带来的运行行为，而不需要去跟踪调试整个系统。单元测试还可以让你进行某些深入、彻底的版本健全性测试（Sanity Testing），这样你就可以在登记这些改动之前，来查验这些改动的运行结果，也就不用去担心这些改动会打乱游戏系统的其他部分。

下面，我们就举几个例子，看看如何对实用函数 parse_header_section 进行单元测试。我们要做的第一件事情就是，对一个函数的预期行为进行步进跟踪，并详细地记录下来那些可能会出错的地方，包括向这个函数传递一个非法的参数。下面，让我们从模型文件的标题节（header section）开始。模型文件的标题节（header section）内容大致如下：

Download at www.Pin5i.Com

```
<HEADER>
    <VERSION>1.1</VERSION>
    <TYPE>WorldLevel</TYPE>
    <NAME>Character Select Gallery</NAME>
    <AUTHOR>Blake Madden</AUTHOR>
</HEADER>
```

函数 parse_header_section 应该加载这样的一些信息，比如：模型的类型（type）、名称（name）、作者（author），以及所基于的文件格式等。文件格式版本信息和模型的类型信息是必需的信息。如果缺少其中任意一个，这个函数就应该相应地发出 model_invalid_version 或 model_invalid_type 异常信息。模型的名称和作者都是可选值。即使没有找到这些信息，这个函数应该仍然可以加载模型的版本信息和类型信息，也不会报错。最后一点，如果这个标题节（header section）信息的结构不正确，那么，这个函数就应该发出 model_invalid_header 异常信息。掌握了这些基本知识，我们接下来就可以为这个函数创建测试用例了。

第一个测试用例应该是一个比较简单的测试：向这个函数传递一个非法的参数，我们就假定去传递一个空指针。我们的预期是，如果传递给函数的标题节（header section）文字信息是非法的，那么该函数就会发出一个 model_invalid_header 异常信息。因此，你的测试用例应该向函数 parse_header_section 传递一个空指针，并验证这个函数会使用 CppUnit 的 CPPUNIT_ASSERT_THROW 宏，发出 model_invalid_header 异常信息。下面就是这个测试用例的大致模样：

```
void TestInvalidHeaderNullValue()
{
    model my_model;
    CPPUNIT_ASSERT_THROW(my_model.parse_header_section(NULL),
        model_invalid_header);
}
```

从上面的代码中你可以看到，我们创建了一个模型对象，然后调用函数 parse_header_section，并向它传递一个空指针，以强制它产生 model_invalid_header 异常信息。我们将这个调用打包在 CPPUNIT_ASSERT_THROW 宏中。其中，第一参数是你的函数调用，第二个参数是你希望它发出的异常信息的类型。如果你的这个函数调用并没有产生预期的异常信息，那么 CppUnit 就会将这个错误作为一个失败的测试用例，记录在日志中，让你以后去查阅。

另外一个测试用例是向函数传递一个格式上有问题的标题节（header section）。在这个测试用例中，我们还是期望这个函数可以发出 model_invalid_header 异常信息。这个测试用例的内容大致如下：

```
void TestInvalidHeaderIllFormatted()
{
    model my_model;
    CPPUNIT_ASSERT_THROW(my_model.parse_header_section(
        "<HEADER"
        "<VERSION>1.1</VERSION>"
```

Download at www.Pin5i.Com

```
            "<TYPE>WorldLevel</TYPE>"
            "<NAME>Character Select Gallery</NAME>"
            "<AUTHOR>Blake Madden</AUTHOR>"),
            model_invalid_header);
}
```

注意，在这个标题节测试用例中，在第一个“HEADER”的后面缺少了一个“>”，并且还缺少了一个结束符</HEADER>。

第三个测试用例应该去测试这个情形：如果标题节中缺少了版本节“<VERSION>”，或者缺少了类型节“<TYPE>”，这个函数会有怎样的运行行为呢？正如我们前面讲过的，这两个信息是必需的。如果没有找到这些信息，这个函数就应该相应地发出 model_invalid_version 异常和 model_invalid_type 异常。对于缺少版本号，或者缺少整个版本节的测试用例，其内容大致如下：

```
void TestInvalidHeaderInvalidVersionSection()
{
    model my_model;
    // 版本节 (version section) 中缺少版本号
    CPPUNIT_ASSERT_THROW(my_model.parse_header_section(
        "<HEADER>"
        "<VERSION></VERSION>"
        "<TYPE>WorldLevel</TYPE>"
        "<NAME>Character Select Gallery</NAME>"
        "<AUTHOR>Blake Madden</AUTHOR>"
        "</HEADER>"),
        model_invalid_version);
    // 缺少版本节
    CPPUNIT_ASSERT_THROW(my_model.parse_header_section(
        "<HEADER>"
        "<TYPE>WorldLevel</TYPE>"
        "<NAME>Character Select Gallery</NAME>"
        "<AUTHOR>Blake Madden</AUTHOR>"
        "</HEADER>"),
        model_invalid_version);
    // 格式上有问题的版本节 (version section)
    CPPUNIT_ASSERT_THROW(my_model.parse_header_section(
        "<HEADER>"
        "<VERSION>1.2"
        "<TYPE>WorldLevel</TYPE>"
        "<NAME>Character Select Gallery</NAME>"
        "<AUTHOR>Blake Madden</AUTHOR>"
        "</HEADER>"),
        model_invalid_version);
}
```

我们还应该创建另外一个类似的测试用例，去验证在类型（TYPE）节内容非法的情况下，该函数会发出 model_invalid_type 异常信息。由于篇幅所限，我们在此就省略了这个测试用例。

Download at www.Pin5i.com

用执行失败的条件去测试一个函数固然是很重要的，但是，用可以正常工作的数据去检测函数的运行结果是否正确，也是同样重要的。对于条件测试，我们可以使用CPPUNIT_ASSERT 宏。这个宏与标准的 ASSERT 宏几乎一模一样。在这个例子中，我们要创建一个测试用例，用合法的文本信息来调用函数 parse_header_section，然后再验证其版本信息和类型信息是正确的。这个测试用例可参考下列代码：

```
void TestInvalidValidHeader()
{
    model my_model;
    // 绝对合法的标题节
    my_model.parse_header_section(
        "<HEADER>"
        "<VERSION>1.2</VERSION>"
        "<TYPE>WorldLevel</TYPE>"
        "<NAME>Character Select Gallery</NAME>"
        "<AUTHOR>Blake Madden</AUTHOR>"
        "</HEADER>");
    CPPUNIT_ASSERT(my_model.get_version() == 1.2);
    CPPUNIT_ASSERT(my_model.get_type() == world_level);
    CPPUNIT_ASSERT(strcmp(my_model.get_name(),
        "Character Select Gallery") == 0);
    CPPUNIT_ASSERT(strcmp(my_model.get_author(),
        "Blake Madden") == 0);
    // 没有包含模型的作者（AUTHOR）和模型的名称（NAME）
    my_model.parse_header_section(
        "<HEADER>"
        "<VERSION>1.2</VERSION>"
        "<TYPE>WorldLevel</TYPE>"
        "</HEADER>");
    CPPUNIT_ASSERT(my_model.get_version() == 1.2);
    CPPUNIT_ASSERT(my_model.get_type() == world_level);
    CPPUNIT_ASSERT(strcmp(my_model.get_name(), "") == 0);
    CPPUNIT_ASSERT(strcmp(my_model.get_author(), "") == 0);
}
```

在这个例子中，我们向这个函数传递一个完全合法的标题节，然后再去确认相应的版本（VERSION）、类型（TYPE）、名称（NAME），以及作者（AUTHOR）的值全都可以正确地加载进来。如果其中有哪个（或哪些）项加载失败，CppUnit 就会把相应的错误信息记录到日志文件中。

请大家回忆一下我们前面所讲的，在标题节的文件规格说明中，模型的名称（NAME）和作者（AUTHOR）都是可选项，所以，你还应该用没有这些信息项的标题节来测试这个函数，并确认这两个项的值被设置为空值。最后这个部分的内容是非常重要的，因为你需要确认这个事实：如果这个函数在标题节中找不到 NAME 项，或者是 AUTHOR 项，那么它就会将这个类中那些从上一个模型加载调用中得到的值全部清除，并将 NAME 项和 AUTHOR 项的值设置为空字符串。如果要进行这个测试，请将下列这个测试用例添加到你的 TestInvalidValidHeader 测试用例中：

Download at www.Pin5i.Com

```
// 标题节中不包含作者（AUTHOR）项和模型名称（NAME）项
my_model.parse_header_section(
    "<HEADER>"
    "<VERSION>1.2</VERSION>"
    "<TYPE>WorldLevel</TYPE>"
    "</HEADER>");
CPPUNIT_ASSERT(my_model.get_version() == 1.2);
CPPUNIT_ASSERT(my_model.get_type () == world_level);
CPPUNIT_ASSERT(strcmp(my_model.get_name(), "") == 0);
CPPUNIT_ASSERT(strcmp(my_model.get_author(), "") == 0);
```

在这之后，你可以采用同样的方式，继续为这个类中的所有其他函数编制更多的测试用例。例如，我们可以创建额外的测试用例，去测试动画语法分析函数 parse_animation_section，确认它可以：正确地加载正确的动画编号，相应动画中所有的帧及帧速正确地加载，稳定可靠地处理错误的三角形数据。

在编制完所有的模型加载测试用例之后，我们现在就可以将它们全部组织到一个测试夹具（test fixture）类中。在 CppUnit 中，我们可以创建一个从 CppUnit::TestFixture 派生得到的测试夹具（test fixture）类。下面就是一个空的测试夹具（test fixture）类：

```
// 对加载一个模型文件的功能进行的单元测试
class LoadModelTest : public CppUnit::TestFixture
{
public:
    CPPUNIT_TEST_SUITE(LoadModelTest);
    CPPUNIT_TEST_SUITE_END();
private:
    Model m_model;
};
```

对于所有需要让这个测试夹具（test fixture）类运行的测试用例，我们都要进行说明。说明的方法是将这些测试用例添加到这个测试夹具（test fixture）类的 CPPUNIT_TEST_SUITE 节中。下面就是一个可供参考的例子：

```
CPPUNIT_TEST_SUITE(LoadModelTest);
    CPPUNIT_TEST(TestInvalidHeaderNullValue);
    CPPUNIT_TEST(TestInvalidHeaderIllFormatted);
    CPPUNIT_TEST(TestInvalidHeaderInvalidVersionSection);
    CPPUNIT_TEST(TestInvalidValidHeader);
CPPUNIT_TEST_SUITE_END();
```

现在，我们将这些测试用例作为成员函数添加到这个 LoadModelTest 类中。现在我们的 LoadModelTest 类应该会变成这个样子（限于篇幅，我们抽掉了其中的测试用例部分）：

```
//对加载一个模型文件的功能进行的单元测试
class LoadModelTest : public CppUnit::TestFixture
{
public:
    CPPUNIT_TEST_SUITE(LoadModelTest);
```

Download at www.Pin5i.Com

```
        CPPUNIT_TEST(TestInvalidHeaderNullValue);
        CPPUNIT_TEST(TestInvalidHeaderIllFormatted);
        CPPUNIT_TEST(TestInvalidHeaderInvalidVersionSection);
        CPPUNIT_TEST(TestInvalidValidHeader);
    CPPUNIT_TEST_SUITE_END();

    void TestInvalidHeaderNullValue();
    void TestInvalidHeaderIllFormatted();
    void TestInvalidHeaderInvalidVersionSection();
    void TestInvalidValidHeader();
};
```

1.7.5 私有函数的单元测试

在前面的例子中，你也许会注意到，那个 model（模型）类看上去封装性很差，因为它所有的实用函数（比如函数 parse_header_section）都是公共函数（public function）。但是，如果你真的将这些函数说明为私有函数（private function），或者是被保护的函数（protected function），那么测试夹具（test fixture）类就无法访问到这些函数了。这就是为什么我们要将这些实用函数说明为 public 函数。这是单元测试中一个常见的、需要注意的地方。因为在大多数面向对象的程序语言中，外部类是无法访问其他类的私有函数的。最常见的解决方案是打破封装。但是，如果你必须将这些实用函数保有为私有函数，那么，C++也提供了另外一个工作区（workaround）：friend（友元）关键字。我们只需要在这个类被测试之前，对测试夹具（test fixture）类进行前置说明（forward-declare），然后再将测试夹具（test fixture）类说明为一个 friend（友元），就像这样：

```
class LoadModelTest;        // 前置说明

class model
{
public:
    friend LoadModelTest; // 将测试夹具（test fixture）类说明为 friend
    void load_file(const char* file_path);
    void render();
    void animate(float speed, bool loop = true);
private:
    // 用于加载模型文件的实用函数
    void parse_header_section(char* file_text);
    void parse_triangle_section(char* file_text);
    void parse_mesh_section(char* file_text);
    void parse_material_section(char* file_text);
    void parse_animation_section(char* file_text);
    void prepare_joints();
    // 从文件中加载纹理（支持 BMP 格式）
    void load_texture(const char* file_path);
    ...
};
```

Download at www.Pin5i.Com

现在，测试夹具（test fixture）类 LoadModelTest 就可以完全访问 model 类了。

1.7.6　用 CppUnit 测试底层功能

在这里，我们要强调一下单元测试的深度，这一点很重要。我们现在举一个例子，假设我们要在游戏中实现一个定制的内存分配器，以便最优化地对内存进行管理。对 QA 部门来说，他们没有任何直接的方法来进行手工测试，或者是自动测试来检验这个内存分配器。虽然 QA 部门可以漫不经心地通过试玩游戏，来简单地测试这个内存管理程序，但是，他们绝对没有办法去测试这个内存管理程序应付潜在问题的稳定可靠性。而这些潜在的问题必然会在系统运行中的某个地方出现。这些问题涉及的情况比较复杂，比如：系统没有正确地释放内存，在某些情况下内存分配失败，如果没有经过正确的测试，甚至是内存分配重叠的问题也必然会出现。单元测试就是一个合手的工具，可以在系统的某个小组件上（例如内存管理程序）运行上述这些情况的测试用例，在这些担心变成问题之前，去验证这些子系统的稳定可靠性。

正如我们前面所提到的，进行单元测试的第一步就是要全面了解被测试组件的所有运行行为，并详细地记录下来那些可能出错的地方。在我们这个例子中，内存管理程序控制着一个独立的全局堆（global heap），并从中分配小片的内存。当某个分配请求出现时，如果这个堆中碎片太多，它就会从操作系统的自由存储区（free store）中进行分配（通过 new 或 malloc）。当你请求释放一个指针时，这个内存管理程序会正确地从自己管理的堆中（或者从操作系统的自由存储区中），释放相应的内存块。最后，这个内存管理程序应该具有足够的智能，如果某个指针并不是它之前分配的指针，它也不会将这个指针删除。要实现这个功能，我们可以让系统记录下所有当前分配的指针，形成一个列表，当接到指针释放请求时，就与这个列表进行比对。只有在这个列表中的指针才会被释放。

现在，我们已经了解了这个类预期的运行行为，我们就做出一个简短的列表，罗列一下可能发生的问题：

- 如果从一个已经非常破碎的堆中申请一大块内存，就会造成内存分配程序崩溃，或者无法正确地从操作系统中得到相应的内存。请求失败，并返回 NULL。
- 对于从操作系统那里获得的内存，在进行释放时，不会像预期的那样被释放。
- 编制内存分配请求，使之可能会意外地返回重叠指针。
- 大小为零的内存分配请求会导致程序崩溃，或者其他无法预见的行为。在这个例子中，我们希望内存管理程序不去分配大小为零的内存块，而是发出一个 std::bad_alloc 异常信息。
- 向内存管理程序的 deallocate 函数（内存释放函数）传递一个它从未返回过的指针，可能会造成错误的释放操作。

了解了这些要点，我们现在就可以创建一些简单的函数，来测试这些情况，以验证预期的行为确实会发生。在我们开始编制测试用例之前，首先用一些新的常量来创建一个测试夹具（test fixture）类，并创建测试用例需要用到的一个内存管理对象。测试夹具（test fixture）类的大致框架如下：

```
// 内存分配管理程序的单元测试
class HeapAllocatorTest : public CppUnit::TestFixture
{
```

Download at www.Pin5i.Com

```
public:
    CPPUNIT_TEST_SUITE(HeapAllocatorTest);
    CPPUNIT_TEST_SUITE_END();
private:
    /*所有测试用例都要用到的内存管理对象
    （最大的堆容量未1000字节）*/
    heap_allocator<char, 1000> m_memory_manager;
    // 不同的分配大小
    static const size_t FIRST_ALLOCATION_SIZE = 500;
    static const size_t SECOND_ALLOCATION_SIZE = 400;
    static const size_t THIRD_ALLOCATION_SIZE = 300;
    static const size_t TOO_BIG_ALLOCATION_SIZE = 1100;
};
```

在CppUnit中，当你在多个不同的测试用例之间使用同一个成员对象时，那么在调用每个测试用例之前，我们有必要首先运行一些常见的清除程序。在这个例子中，内存管理类有一个清零函数（clear），我们需要调用它来将堆内存清零。CppUnit提供了两个虚函数，setupUp和tearDown，需要我们分别在运行测试用例之前和之后来分别调用它们。我们可以用下列方法：在我们的测试夹具（test fixture）类HeapAllocatorTest中添加tearDown函数的一个公共过载（public overload）：

```
void tearDown()
{
    m_memory_manager.clear();
}
```

接下来，我们开始为内存分配例程和内存释放例程编写测试用例。虽然在前面的例子中，我们只为每个测试创建了一个测试用例函数，但是，可以在一个函数中包含一个以上的测试。除了CppUnit提供的CPPUNIT_ASSERT宏之外，另外还有一个函数CPPUNIT_ASSERT_MESSAGE，可以让你在它失败的时候，通过一个断言（assertion）嵌入一个诊断信息。通过这种方法，你就可以创建出更为通用的测试函数，其中还可以包含大量的测试用例，同时仍然可以保留相当数量的详细的错误信息。这种做法有一个唯一可能的缺点，在某个特定的测试用例函数中，第一个失败的断言（assertion）会导致函数的终止运行，然后测试夹具（test fixture）类就会跳过这个函数，转向下一个测试用例。这是因为，如果一个特定的测试断言失败，CppUnit就会认为，造成这个断言失败的原因，也会不经意地造成下一个断言的误报。对我们的内存分配测试来说，这是一个安全的假设。因为，如果某个内存分配为NULL，或者包含有被破坏的数据，那么所有后面的测试都会产生未定义的行为。

了解了这个情况，我们应该为那些涉及内存分配的情况增加一个测试用例。我们首先发出一些内存分配请求（包括一个大小为0的分配请求），将内存管理程序的堆搞得零碎些，然后，我们再发出一个分配请求，其大小要超出内存管理程序可以处理的范围。为了验证该系统一切正常，我们要测试那些由内存管理程序返回的指针，以确保它们都不是空指针，然后我们还要确认，在所有分配的内存中没有彼此重叠的情况。我们的测试用例可能会是这个样子：

```
void TestAllocation()
    {
```

Download at www.Pin5i.Com

```
// 这是我们要复制到被分配的数组中的数据
char data1[FIRST_ALLOCATION_SIZE];
std::memset(data1, 'a', FIRST_ALLOCATION_SIZE);
char data2[SECOND_ALLOCATION_SIZE];
std::memset(data2, 'b', SECOND_ALLOCATION_SIZE);
char data3[THIRD_ALLOCATION_SIZE];
std::memset(data3, 'c', THIRD_ALLOCATION_SIZE);

/* 分配数组，验证分配功能正常，并填充数据*/
and fill with data*/
char* array1 =
    m_memory_manager.allocate(FIRST_ALLOCATION_SIZE);
CPPUNIT_ASSERT_MESSAGE("First allocation returned NULL",
    array1);
std::memcpy(array1, data1, FIRST_ALLOCATION_SIZE);
char* array2 =
    m_memory_manager.allocate(SECOND_ALLOCATION_SIZE);
CPPUNIT_ASSERT_MESSAGE("Second allocation returned NULL",
    array2);
std::memcpy(array2, data2, SECOND_ALLOCATION_SIZE);
//内存分配程序的堆碎片太多，这个请求对它来说要求的内存空间太大了，
char* array3 =
    m_memory_manager.allocate(THIRD_ALLOCATION_SIZE);
CPPUNIT_ASSERT_MESSAGE(
    "Requesting too much memory returned NULL", array3);
std::memcpy(array3, data3, THIRD_ALLOCATION_SIZE);

/* 尝试去分配一个大小为 0 的数组，并
确认它会抛出一个 bad_alloc 异常信息 */
CPPUNIT_ASSERT_THROW(m_memory_manager.allocate(0),
    std::bad_alloc);

/* 测试被分配内存的完整性，确保返回的指针不会重叠*/
returned pointers don't overlap*/
CPPUNIT_ASSERT_MESSAGE(
    "First allocation's data is corrupt."
    "Allocations may overlap.",
    std::memcmp(array1, data1, FIRST_ALLOCATION_SIZE) == 0);
CPPUNIT_ASSERT_MESSAGE(
    "Second allocation's data is corrupt."
    "Allocations may overlap.",
    std::memcmp(array2, data2, SECOND_ALLOCATION_SIZE) == 0);
CPPUNIT_ASSERT_MESSAGE(
    "Third allocation's data is corrupt."
    "Allocations may overlap.",
    std::memcmp(array3, data3, THIRD_ALLOCATION_SIZE) == 0);

// 释放所有的内存
CPPUNIT_ASSERT(m_memory_manager.deallocate(array1,
    FIRST_ALLOCATION_SIZE));
CPPUNIT_ASSERT(m_memory_manager.deallocate(array2,
```

Download at www.Pin5i.Com

```
            SECOND_ALLOCATION_SIZE));
        CPPUNIT_ASSERT(m_memory_manager.deallocate(array3,
            THIRD_ALLOCATION_SIZE));
    }
```

最后，我们要为内存释放功能创建测试用例。因为对内存释放功能来说，它并不关心被破坏的数据，我们需要为所有不同的内存释放测试用例创建独立的函数。第一个测试用例会执行一个简单的内存分配操作，并确保相应的内存释放功能是正常的。我们同时还要进行另外一个简单的测试，确保内存管理程序不会去释放空指针：

```
void TestDellocationSimple()
{
    char* array1 =
        m_memory_manager.allocate(FIRST_ALLOCATION_SIZE);

    // 进行释放
    CPPUNIT_ASSERT(m_memory_manager.deallocate(array1,
        FIRST_ALLOCATION_SIZE));
    CPPUNIT_ASSERT_MESSAGE(
        "Allocation deallocated a NULL pointer.",
        m_memory_manager.deallocate(NULL, 1) == false);
}
```

接下来的这个测试用例会确保内存管理程序不会去释放它原先没有分配过的内存块：

```
void TestDellocationFromFreeStorage()
{
    // 在内存管理程序之外分配下面这个数组
    char* array1 = new char[FIRST_ALLOCATION_SIZE];

    /* 内存分配程序不应该去释放这个数组，因为它并没有为这个数组分配过内存 */
    CPPUNIT_ASSERT_MESSAGE(
        "Allocator deallocated a pointer that it did not"
        "originally allocate.",
        m_memory_manager.deallocate(array1, FIRST_ALLOCATION_SIZE)
            == false);
    delete [] array1;
}
```

最后，我们可以增加一个测试用例，以确认当碰到一个内存块大于内存管理程序的内部堆时，该内存块也可以正确地被释放：

```
void TestDellocationFromUnmanagedPointer()
{
    // 这个内存分配请求要大于内存分配程序的堆
    char* array1 =
        m_memory_manager.allocate(TOO_BIG_ALLOCATION_SIZE);

    /* 虽然内存分配程序时从它自己堆之外的地方创建了这个数组，
    它应该也可以将其释放 */
    CPPUNIT_ASSERT_MESSAGE(
```

Download at www.Pin5i.Com

```
            "Allocation larger than managed heap wasn't deallocated.",
            m_memory_manager.deallocate(array1,
            TOO_BIG_ALLOCATION_SIZE));
}
```

在我们完成所有的测试用例之后，我们要把它们添加到测试夹具（test fixture）类的定义部分中，就像这样：

```
CPPUNIT_TEST_SUITE(HeapAllocatorTest);
    CPPUNIT_TEST(TestAllocation);
    CPPUNIT_TEST(TestDellocationSimple);
    CPPUNIT_TEST(TestDellocationFromFreeStorage);
    CPPUNIT_TEST(TestDellocationFromUnmanagedPointer);
CPPUNIT_TEST_SUITE_END();
```

我们最终的测试夹具（test fixture）类大概应该是这样的（限于篇幅，我们拿掉了函数部分）：

```
// 内存分配管理程序的单元测试
class HeapAllocatorTest : public CppUnit::TestFixture
{
public:
    CPPUNIT_TEST_SUITE(HeapAllocatorTest);
        CPPUNIT_TEST(TestAllocation);
        CPPUNIT_TEST(TestDellocationSimple);
        CPPUNIT_TEST(TestDellocationFromFreeStorage);
        CPPUNIT_TEST(TestDellocationFromUnmanagedPointer);
    CPPUNIT_TEST_SUITE_END();
public:
    void tearDown();
    void TestAllocation();
    void TestDellocationSimple();
    void TestDellocationFromFreeStorage();
    void TestDellocationFromUnmanagedPointer();
    private:
    //Allocation sizes
    static const size_t FIRST_ALLOCATION_SIZE = 500;
    static const size_t SECOND_ALLOCATION_SIZE = 400;
    static const size_t THIRD_ALLOCATION_SIZE = 300;
    static const size_t TOO_BIG_ALLOCATION_SIZE = 1100;
};
```

现在，我们可以将这个测试夹具（test fixture）类添加到测试的执行程序（runner）（我们前面提到过），然后就可以运行我们的测试程序了。所有内存管理程序发生的失败情况都会被记录在一个 XML 日志文件中，并交由你来审核。

1.7.7　总结

正如你在本文中所看到的，单元测试是一个预防性方法，可以在系统整合之前去测试那些比较小的组件。它不但可以让你对底层代码的变动信心十足，还可以让你用一种自动化的

Download at www.Pin5i.com

方式来测试程序中某些孤立的部分，由此可以把你从劳动密集型的调试工作中解救出来，让你只需要去测试方便的小型的函数。

CppUnit 提供了一个易于使用的框架，大大地简化了测试用例的创建工作和运行工作，由此也进一步提高了单元测试的可用性。另外，CppUnit 的报告管理能力可以很好地组织和呈现测试结果，让你可以快速、方便地进行测试结果的审核。

1.7.8 参考文献

[CppUnit05] CppUnit. Available online at *http://cppunit.sourceforge.net/cgi-bin/moin.cgi*.

Download at www.Pin5i.Com

1.8　为游戏的预发布版本添加数字指纹，威慑并侦测盗版行为

Steve Rabin，任天堂美国公司
steve_rabin@hotmail.com

没有人愿意看到自己的游戏产品被盗版。但是，如果你的游戏产品在正式发布之前，就已经被盗版了，这可真是一件让人心碎的事情。如果你的游戏已经售出了一百万套，你可以公开地去打击盗版行为。但是，如果公司内部的人将预发布版本泄露出去，这可就另当别论了。所以，最大的问题应该是：我们如何才能防止内部的泄露呢？

相应的解决方案要解决两个方面的问题。首先，对于内部那些可以接触预发布版本的人员，我们必须能够防止他们将预发布版本泄露出去。这是首要必须解决的问题。第二，如果某个版本已经被泄露出去了，我们总得找出到底是谁干的，以便让他（们）负必要的法律责任。不管怎样，如果你的产品已经被泄露出去，可以说你就算失败了。话虽这么说，但是，我们必须能够侦测出相关的犯罪团伙，这样才能有效地实施你的威慑策略。这一点也非常重要。

本文首先简要地介绍了威慑策略，然后深入地探讨了如何通过嵌入式数字指纹技术，来实现盗版侦测的方法。

1.8.1　威慑策略

威慑策略的技巧是：让相关人员都能够意识到，对他们个人而言，如果不小心造成了版本的泄露，对他们自己是绝对有害。对于一个项目中的主要开发人员，这通常都不会是个问题。因为他们都把自己的赌注压在了游戏上，个人利益与产品的成功息息相关。但是，对于周边人群，例如测试人员，或者是游戏评论人员，游戏的成功对他们来说并不是一个强有力的激励因素。

对于这些周边人群，唯一有效的威慑方法就是要让他们明白，自己所接触到的游戏版本已经烙上了他们的指纹。他们必须意识到，如果自己将游戏泄露出去，就要承担责任。这种威慑力可以是真实存在的，也可以是想象中的，只要能够让大家深信不疑，就算达到目的了。这样说来，威慑策略最重要的部分就是一个郑重的警告：如果某人所负责的游戏版本被泄露出去，那么他一定会被发现，也一定是要负责任的。

Download at www.Pin5i.Com

就像你在自己家的外面竖一块牌子，警告那些想要来入室盗窃的家伙们，我可安装了警报系统啊。对项目的周边人员的警告，要确保让所有的人都知道，产品中装配有特殊的装置，对他们是有所提防的。但是，你也许并不会真的需要一个警报系统，因为，如果你把自己的“告示牌”做得足够显眼，绝对会吓到很多人，让他们不敢冒险把产品泄露出去。当然，如果你真的没有一个警报系统，上面的这些举措就要非常小心，千万别走漏了风声！

除了明确地告诉相关人员，他们的指纹已经被嵌入到了某个游戏版本中，比较明智的作法是：还要在游戏中设立一个签名区，来显示这个指纹（或许可以放在游戏开始的介绍画面中）。这就成了一个提醒器，不断地提醒相关人员，千万别乱来。但是，这绝对不应是唯一一个嵌入指纹的地方。因为，这个地方的指纹可以很容易地被拿掉。真正的指纹必须深深地隐藏在游戏当中，而且应该几乎是不可能被侦测到，或者被拿掉。

至于那些要显示在游戏的介绍画面中的信息，下面有一个例子可供参考：

> *这个预发布版本已经签发给 Steve Rabin。*
> *严禁与他人分享，或者散布这个版本。为了便于跟踪检测一个被泄露的版本，*
> *Steve Rabin 的数字签名已经直接嵌入到这个版本的资产和可执行文件中。*

这个简单的警告信息应该可以让任何有意将产品外泄的人感到无比的恐惧。实际上，即使我们不在产品的预发布版本中再融入任何其他的防范措施，这个简单的信息也可以为你带来 99%的安全效果。

1.8.2 利用水印和指纹来进行侦测

在纸质的流通货币中，人们已经设计使用了水印技术。这是为了向检查纸币的人证明，这个纸币是合法的。而且，这个水印的设计也极难伪造和仿制。例如，在一张 5 美元的纸币上，如果你将这个纸币举到光亮处，你就可以在纸币的正面右侧看到林肯总统淡淡的头像。这个水印就是一个保证，证明这张纸币是真的，不是假币。

对于音乐、图片，以及电影，我们可以在其中嵌入可见的，或者是不可见的水印。举个例子，博物馆可以在他们所有绘画陈列品的复制品或数字图片中，安放一个不可见的水印，以证明他们的所有权。在音乐唱片行业，他们也采用了一些同样的方案，为了证明其所有权，在音乐中嵌入了数字水印。

数字指纹采用了与数字水印相同的技术，但二者的区别在于：数字指纹是将最终用户的信息编码到作品中，数字水印则是将作品所有人的信息嵌入其中。对于游戏预发布版本的数字指纹应用，其目的是获得一个无法被人检测到的指纹信息。另外，如果需要的话，开发商可以很清楚地标识出这个版本的产品最初是签发给谁的。数字指纹并不是一个类似传统水印的图片，它实际上只是几个数据位（bit）的一个重新排序，或相加的和。但是，如果要想从软件中将数字指纹清除掉，则应该是非常困难的，或者说是不可能的，除非你把游戏搞得无法运行了。

我们可以把数字指纹安放在纹理、3D 模型、动画、音乐、音效，以及可执行代码中。实施的技巧是改变相应的原始文件，这样别人就不会注意到数字指纹的存在，也就无法检测到数字指纹，更无法改变数字指纹。数字指纹不会影响到产品的游戏性，或者游戏体验。

Download at www.Pin5i.Com

1.8.3　添加数字指纹的流程

作为一个事务性的工作，如果每个要签发的游戏版本，在添加数字指纹的时候，都需要程序人员或游戏美工的参与，那就显得太浪费时间了。因此，我们最终要采用的数字指纹添加方案必须快捷而简单。

最佳的解决方案应该是：对于一个完成后的游戏版本，作为刻盘前的最后一项工作，再来添加数字指纹。但是，由于我们选择的方法不尽相同，这种方案几乎没有什么可行性。因为，我们或许需要在游戏版本被编译和打包之前来添加数字指纹。无论如何，我们应该让那些非技术人员通过一个简单的过程，就可以为游戏版本添加数字指纹。

1.8.4　数字指纹添加过程的安全性

在理想状态下，添加数字指纹的过程应该遵循柯克霍夫原则（Kerckhoffs's Law）。按照柯克霍夫原则，即使一个系统的所有信息都变成了公共知识（Public knowledge），无人不晓，但只要密钥没有泄露，那么这个系统应该仍然是安全的。但不幸的是，到目前为止，我们还找不到什么有效的方法，来实现这种安全级别的数字指纹系统。攻击者根本不用去破解数字指纹，他只需要把数字指纹搞跨，就可以击溃整个安全体系。如果攻击者知道数字指纹的确切位置，他就可以改动数字指纹，或者干脆把它搞得不可读。

因此，我们可以实现的最高的安全级别就是“Security through obscurity”（不公开就是安全）。也就是说，我们这个数字指纹的添加过程是高度机密的。为了确保这个秘密不会被泄露，我们建议，最多只能有两个人可以知道某个游戏版本数字指纹添加过程的全部细节。

有一种策略是，创建一个数字指纹添加程序，来对已经完成的游戏版本添加数字指纹。这个程序应该设有使用密码，以确保只有几个相关人员可以使用这个程序，为游戏版本添加数字指纹。为了提高安全性，对于负责添加这个数字指纹的人，我们应该把他/她的名字加密到数字指纹中去。

很明显，那些知道如何去为游戏版本添加数字指纹的人，以及那些被授权去为游戏版本添加数字指纹的人，他们有能力去仿制一个数字指纹。这是一个不安全因素。为此，我们应该只让那些我们绝对信得过的人可以参与到这个过程中，比如主程序和制作人，千万别让实习人员去为游戏版本添加数字指纹！

1.8.5　数字指纹的添加策略

在下面的内容中，我们为大家介绍了一些数字指纹的添加策略。对于下面的这些方法，我们首先应该注意的是：方法实现的难度与用这个方法添加的数字指纹被查找到的难度之间的关系。一旦攻击者查找到了数字指纹，就可以将其破解，或者将数字指纹破坏。所以，我们首要的目标是如何让数字指纹变成几乎无法被发现的数字指纹。

Download at www.Pin5i.Com

1. 介绍画面中的人名

这个策略主要是起到昭示和威慑的作用。所以很明显，这样的数字指纹会很容易地被发现，并且也可以很轻易地将其消除。但是，这个方法实现起来非常简单，因为我们只需要改动一些文字或者是图片就可以了。

2. 附加文件

这个方法需要我们在游戏版本中增加一个或多个垃圾文件。之所以被称为垃圾文件，是因为游戏的正常运行根本不需要这些文件。这些文件存在的唯一价值就是为了数字指纹。这个技术堪称绝妙，因为它不但实现起来非常容易，而且添加的数字指纹也极难被发现。特别是我们知道，大多数的游戏都有着成千上万个文件。这个技术就像是在一个大干草堆上插了一根有标记的缝衣针。

3. Adobe Photoshop CS 的 Digimarc 滤镜

如果要在游戏中的纹理文件中添加数字指纹，我们可以使用 Photoshop 的 Digimarc（数字水印）滤镜。这个特性让你可以用四种不同级别的耐久性（Durability），将 1 到 16 777 215 之间的任意一个数字嵌入到图片中。数字水印的耐久性代表着其被破坏的难易程度。如果有人用简单的滤镜功能来处理这个图片，以破坏其中的数字水印。那么，这个图片的数字水印的耐久性越高，就越难以被破坏。如果不考虑数字水印的耐久性，当有人知道数字水印添加在哪个纹理文件中，他就可以用 Photoshop 读出这个数字水印。另一方面，这种形式的数字指纹，是一种非常不牢靠的数字指纹。因为所有可以访问这个软件的人都可以读出水印或写入新的水印。但是，由于一款游戏怎么也得有成百上千的纹理图片，在其中的两三个文件中添加数字指纹也就足够了。真要把它们都找出来，可不是一件容易的事情。

4. 利用定制算法，为图片添加数字指纹

由于 Photoshop 的 Digimarc（数字水印）滤镜是一个广泛使用的工具，一种变通的方法是开发一个自己的算法，来为图片添加数字指纹。结合我们上面讲过的“干草堆中的缝衣针”的方法，我们只需要为几个图片文件加上数字指纹就足够了。大海捞针似地想找到这些文件，几乎是不可能的。

5. 利用定制算法，为 3D 模型、动画或音效文件添加数字指纹

对于 3D 模型、动画和音效文件，我们可以微妙地改变这些文件的数据，就可以为它们添加数字指纹了。如果你知道某个文件最初应该是什么样子的，对这个文件做些微小的改动，我们就可以将数字指纹嵌入其中。虽然这样会不可避免地引入数据噪音，但如果操作正确，应该不会改变这些文件的原貌和效果。不幸的是，这个方法实现起来颇为困难，但攻击者也几乎无法找到其中的数字指纹。

6. 为可执行程序添加数字指纹

前面讲到的那些技术都是集中在如何修改游戏的资产（图片、动画、3D 模型等等）。但

Download at www.Pin5i.Com

我们要知道，游戏的可执行代码也是一个非常适合隐藏数字指纹的地方。如果在编译之前，在代码死区（dead area）中添加数字指纹，这多少有些不值得，同时也会增加数字指纹嵌入工作的流程复杂度。最理想的做法是：将数字水印添加工作作为一个后期处理过程，对一个已经编译完毕的可执行文件添加数字指纹。

有一个方法是在代码中说明几个静态数据，其中包括一个唯一的密钥和一些死区，比如一个 24 个字节的标记（marker），后面跟一个 32 个字节的垃圾区（garbage）。作为一个后期处理过程，我们可以利用工具软件读入游戏的可执行文件，找到那个 24 个字节的标记区，然后用一个加密的数字指纹来覆写那个 32 个字节的垃圾区。而读出这个数字指纹的过程也很简单，同样是先找到那个 24 个字节的标记区，然后就可以对下面的 32 个字节的垃圾区进行解密操作了。为了让这个数字指纹尽量的隐蔽，我们应该注意：让我们定义的这个静态数据看上去与静态数据区中其他的数据字节没有什么分别。

还有一个更迂回的方法可以帮助我们嵌入无法找到的数字指纹，就是重新编排关键指令的顺序。我们都知道，在游戏代码中，有些位置上的代码，其执行顺序是无关紧要的（毕竟，这是乱序处理器提高执行速度的方法）。如果你能够找到这样的 5 个连续的指令，那么你就可以拥有 120 种不同的排列方式，来重新排列这 5 条指令。如此这般，通过对现有指令代码的重新排序，你就可以在其中隐式地嵌入一个 1～119 的数字作为数字指纹（这里请注意，你不能嵌入数字 0，这样就相当于保持原来的指令顺序）。虽然这个方法技术上实现起来要更为复杂，但要找到这样嵌入的数字指纹也几乎是不可能的。

7. 多种策略的组合使用

虽然上面的每个技术都是各有功效的，但如果能综合使用，则威力会更强大，更难以破解。多种技术的结合使用，使得至少会有一种技术能够幸存下来，未被破解的概率大大提高，你可以根据未被破解的数字指纹，来准确地辨别出被泄露的版本。由于上面讲到的大部分技术都比较容易实现，所以，如果能够再结合一些你自己设计的方法，是非常明智的做法。表 1.8.1 中罗列了各种方法的简要信息，以及各自实现的容易程度和被破解的困难程度。

表 1.8.1　　数字指纹添加技术一览

技 术 名 称	实现的容易程度	破解的难度
介绍画面中的人名	极容易	不是很困难
附加文件	极容易	非常困难
Adobe Photoshop CS 的 Digimarc 滤镜	极容易	有些难
利用定制算法，为图片添加数字指纹	非常容易	极困难
利用定制算法，为 3D 模型、动画或音效文件添加数字指纹	不是非常容易	极困难
为可执行程序添加数字指纹	有些容易	极困难
多种策略的组合使用	不确定	极困难

1.8.6　破解数字指纹

对攻击者来说，如果没有关于实现数字指纹的内部情报，要想破解它是极其困难的。但是，如果有两个攻击者，他们各自手里有一套加了数字指纹的游戏版本。他们就可以串谋起

Download at www.Pin5i.Com

来，比较两个光盘中的文件，很快就可以找到二者之间的区别[Wu04]。由于这些区别就是我们添加的数字指纹，他们就可以破解或者去掉其中的数字指纹。举个例子，他们可以计算两个数字指纹的平均值，基本上就可以毁掉他们的身份证据。

现在我们还没有一种十分可靠的方法，来对付这种串谋攻击。这是一个非常活跃的研究领域，有大量的未知空间等待我们去发现。关于这个领域更详细的介绍，请大家参阅[Wu04]。

最后要提醒大家注意的是：同一个数字指纹应该总是应用到同一个人身上。这是比较可行的方法。否则的话，在开发过程中，这个人会在不同的阶段拿到两个版本的游戏，经过比较，他自己也可以进行串谋攻击。

1.8.7 总结

也许会有人觉得，如此这般费劲地去保护一个游戏的预发布版本，这多少有点偏执。但是，预发布版本的泄露会带来很大的风险，所以，防止这种灾难发生所付出的成本还是非常值得的。很多大的发行商都已经实现采用了某种数字指纹处理技术。如果很多小型的开发商也能够这样做的话，他们也会从中受益。正如我们前面所探讨的，从复杂到简单，我们有很多的方法可以为自己的游戏加上数字指纹。

请大家记住，数字指纹方法的最重要的目的在于威慑。所有相关人员都应该明白，他们自己要对任何版本的泄露事件负责。不管怎样，你还是需要义正词严，发挥威慑的力量，同时使用某类超级秘密的方法，适当地为自己的游戏版本添加数字指纹。

实施添加数字指纹的过程，最理想的实现是：开发一个单独的程序，可以向游戏的最终版本中添加数字指纹，并能够从中读出已添加的数字指纹。而且，只能有1个，或2个关键人物知道这个程序是如何运做的，只有他们有权为游戏版本添加数字指纹。如果每个人都知道，他们所拿到的版本都已经添加了数字指纹，那么你就有理由确信，在游戏正式发布之前，不会有人冒险去尝试对外泄露你的游戏。

1.8.8 参考文献

[Wu04] Wu, Min, Wade Trappe, Z. Jane Wang, and K. J. Ray Liu, "Collusion-Resistant Fingerprinting for Multimedia." *IEEE Signal Processing Magazine,* March 2004. Available online at *http://www.ece.umd.edu/~minwu/public_paper/Jnl/0403FPcollusion_IEEEfinal_SPM.pdf.*

Download at www.Pin5i.Com

1.9　通过基于访问顺序的二次文件排序，实现更快速的文件加载

David Koenig，Touchdown 娱乐公司
david@touchdownentertainment.com

很多游戏面临着从存储媒介中加载资源的需求。当操作系统中存在大量文件的文件句柄时，操作系统的运行速度就会变得非常慢。为了进行优化，很多游戏都只能从打包的资源文件中加载必需的资源。这些打包的资源文件就是一个个大型的文件数据库，以单个文件或一组文件的形式存在着。在这些数据库中，都保存着一个完整的目录层次信息。资源文件有效地解决了文件句柄开销过大的问题。但是，另外一个问题又出现了。这些资源文件的顺序通常就是硬盘上的目录结构的一个镜像。而游戏很少会按照资源文件在目录结构中的顺序来访问它们。相反，多数情况下，它们都会在资源文件中跳跃式地访问资源文件。这就成了一个主要的瓶颈。如果能够顺序地进行访问，那么就可以更快速地加载那些必需的资源。如果游戏使用的数据集十分大，那么其资源文件的布局中存在的这个弱点就会暴露出来。对于所有的游戏来说，它们对资源的使用呈现出了某些固定的模式。为了优化资源文件的加载时间，下一步我们要做的就是根据游戏对资源的使用模式来整合数据。然后我们可以研究这些使用模式，在资源文件中提供一个优化后的文件排序。对于这个普遍性的问题，本文介绍了一个颇有潜力的解决方案。

1.9.1　问题的提出

对于一根链条来说，它的最大强度只能等于其所有链环中强度最小的那个链环的强度。而说到系统的运行速度，当前最薄弱的一环就是大容量存储器，比如硬盘、CD-ROM 驱动器以及 DVD-ROM 驱动器。在 PC 或游戏机的硬件配置中，大容量存储器是速度最慢的硬件之一，也是发展速度最慢的硬件产品。今天的硬盘驱动器，其运行速度几乎与 4 年前的速度没有什么差别。而视频显示卡的速度则已经提高了很多倍。但是，如果我们不能把自己的内容在一个合理的时间量内传递到显卡上，那么视频显示卡所获得这些速度上的提升也不会产生太多的价值。大家可以参阅[History03]，去看看硬盘存储技术发展年鉴。

Download at www.Pin5i.Com

1.9.2 解决方案

那么，我们现在还有什么其他的选择吗？这个问题的答案取决于具体项目的需求。不同的游戏项目有着各自不同的资源需求。在整个开发周期的初期阶段，开发人员应该明确这些需求。为了提高加载速度，我们在下面介绍了一些可能的方法。当然，这绝对不是一个最终的清单，可能的优化方法还有很多。

1. 解决方案 1：定制文件格式

定制内容文件的格式，把每个文件及其附属文件都包含在一个文件中。基本上讲，你可以把一个完整的场景画面保存为一个文件。另外一种方法是，选择场景中的单个游戏对象，将它们导出，然后再连同其所有的附属资源文件一起保存成一个文件。这样就可以按照顺序来加载资源。这个方法的一个主要问题就是冗余数据过多。如果几个游戏对象需要交叉共享某些资源，那么，它们最终的文件中就都会包含这些共享的附属资源。这样一来，最终开发出来的游戏在文件容量上会高出很多。

2. 解决方案 2：每个关卡使用一个资源文件

对于每一个给定的关卡，我们将它所有的资源打包成一个资源文件。不同的关卡有各自的资源文件。这样做的好处是缩短了磁盘磁头移动的距离，就多少可以节省一些读盘的时间。否则，如果一个关卡使用多个资源文件，当文件顺序和加载顺序不同时，磁盘磁头就不得不跳来跳去。这个方法的主要优点是：在项目接近尾声的时候，实现起来相当容易。但鉴于游戏项目的不同，这个方法可能并不会带来明显的效果。这是该方法的一个弱势。

3. 解决方案 3：重新排序文件列表

根据各个资源被打包到资源文件中的方式，重新排列各个资源被加载的顺序。很多时候，在游戏中的某个固定的时间，我们需要加载一系列已知的文件。这个方法就特别适合于这种情况。比如，在玩家进入游戏世界之前，我们就加载一套标准配置的武器模型。这在第一人称的射击游戏中非常常见。而这种情形就是一个非常好的例子。通常来说，一个普通的武器系统要包括 10～20 个带有纹理或材质的模型。在这种情况下，开发人员所需要做的是在加载这些资源时，记录下来每个文件指针的位置。然后，我们就可以按照这些资源在资源文件中出现的顺序，对加载顺序重新排序。从理论上讲，采用这个方法，应该可以加快资源的加载速度。具体效果则由于项目的不同而有所区别。

4. 解决方案 4：对资源文件的二次排序

为了获得最优化的加载性能，我们在加载资源的时候，资源的存储区域需要尽可能是连续的。理想的状态是完全无碎片的文件加载。这种情况并不是总能遇到的。我们能做的是重新对一组经常同时需要被加载的资源进行排序，获得一个更为优化的排列。如果我们用这个方式来编排相关的资源，我们很可能就会得到最优化的状态。当游戏切换到一个新的状态时，

Download at www.Pin5i.Com

例如需要加载前端用户界面，这些文件就可以按顺序连续地加载进来。这就意味着，加载过程只需要一次大的文件指针的跳转。文件系统来完成这个跳转工作，并将这一组资源读取进来。然后，它就可以跳转到另外一个数据组。这是一个比较大的改进，省去了每个文件加载时的指针跳转。

1.9.3　基于访问的二次文件排序的工作流程

1. 第一步：运行游戏，采集数据

使用一个标准的打包资源文件，来运行我们的游戏。将文件访问顺序及加载时间记录到一个文本日志文件中。这个日志文件中应当包含游戏所需要访问的文件列表。如果能够根据游戏所能切换到的不同状态，将这个日志文件分割成不同的记录区，这会是个很不错的想法。这个日志文件可以有效地为文件顺序优化程序提供必要的信息，告诉它哪些文件可以组成一组。这种组合的变化是无穷无尽的。但却完全取决于游戏项目使用资源的方式。日志文件中也应该将加载操作的用时记录下来，这样我们就可以用这个数据作为基准，去评估优化后的效果。

2. 第二步：分析采集到的数据，优化打包顺序

运行打包顺序优化程序，来处理上面采集到的日志文件。这一步的工作与具体的项目密切相关。我们在随书光盘中提供了相关的示例代码。这段代码采用了一个面向关卡的方法。

然后我们读入所有的通用数据，以及某个关卡专用的资源。再针对这个关卡，进行打包顺序的优化。还有一个更可靠的方法是对若干组顺序加载的文件进行优化。如果是这样，那么，我们在上一步的日志中，就应该记录下来每一个子系统所访问的资源及访问的时间。优化工作最佳的起点是日志文件中那些所有关卡都要用到的通用数据。举个例子，这些通用数据包括：游戏中的用户界面、一个标准的武器系统，或者是一个通用的着色器库。

3. 第三步：重新打包资源文件

根据优化后的顺序，打包资源文件。我们所使用的资源打包工具软件应该具备这样的能力：就是可以按照一个输入文件中说明的既定的顺序，去打包资源文件。

4. 第四步：再次运行游戏并采集数据

我们现在需要去验证一下，看经过优化后的资源文件是否缩短了加载时间。用新打包的资源文件，来再次运行我们的游戏，并记录相应的加载用时。将这个新得到的数据与第一步中得到的原始数据进行比较。游戏加载的速度提高了吗？如果答案是否定的，那就请你继续阅读本文下面的内容，看看有哪些可能的问题会影响到最后的优化效果。

Download at www.Pin5i.com

1.9.4 优化效果

表 1.9.1 中是我们利用随书光盘中的样本数据获得的优化结果。

表 1.9.1 样本数据的加载用时

文件系统	加载用时（单位：毫秒）
Windows 文件系统（NTFS）	6 077.858 499
资源文件	1 443.543 244
优化后的资源文件	795.332 088

我们从上表可以看到，与标准的资源文件相比，优化后的资源文件，其加载速度提高了近两倍。表中实际的加载用时数量都非常小，而且优化后的提升也只有两倍，这是因为我们的样本数据本身就是非常小的。在对一个大型文件（大于 1GB）进行的测试中，与一个目录交叉的资源文件相比，利用优化后的资源文件，我们的加载速度提高了 20～50 倍。具体来说就是从 10 秒提高到 200 毫秒，这是一个非常显著的速度提升。在某些游戏加载模式中，最初访问的那些文件都是比较零散的，磁盘磁头需要多次的跳转。但通过对资源文件的二次排序，就可以解决这个问题。在这种情况，利用优化后的资源文件可以获得最佳的提升效果。

1.9.5 影响最终优化结果的因素

我们最后得到的结果只是一些数字而已。有些时候，这些数字并不能告诉我们所有发生的事情。有些因素确实可以影响到最终结果的有效性。

1. 硬盘的高速缓存

这是产生畸变结果的罪魁祸首，也是我们要去与之对抗的一个大难题。如果你想看到这个问题对优化结果的影响，只需要去运行我们的示例代码就可以。立刻去再次运行这个示例代码，你就会看到两次运行结果的巨大差异。这就是那些被缓存处理的硬盘扇区所造成的。解决这个问题的方法取决于具体的运行平台。我们一般的选择是重新启动机器。还有一个选择是从另外一个不同的存储介质中加载资源文件。

2. 文件碎片

假设操作系统实际上是将我们优化后的资源文件保存在几个文件片段中，而不是保存在一个连续的文件中。最后的运行结果可能会让你大吃一惊，优化后文件的加载速度居然比未经优化的资源文件还要慢！笔者在撰写本文时就实实在在地碰到了这个问题。这个问题同样也会影响最终用户。一般来说，这个问题不会出现在 CD-ROM 或 DVD-ROM 上。因为对于这类存储介质，你可以完全控制文件的物理映射。

Download at www.Pin5i.Com

1.9.6　潜在的问题

根据访问顺序去打包资源文件的做法会影响到某些特殊的领域。而你在开发过程中可能根本不会去考虑这些领域。这个方法可能会给 MOD 游戏再选者社团带来性能上的打击。如果你是根据游戏中故事情节对资源的需求来打包资源文件，那么你就将文件系统的性能峰值锁定在你的内容中。如果有一个玩家利用你游戏中的内容创建了一个新的关卡，他们就会发现，这个优化后的系统，其性能还不如那个未优化的来得好。到目前为止，这个问题更多地出现在 PC 游戏中，游戏机游戏的情况相对要好一些。但这无疑是一个我们需要认真考虑的问题。

1.9.7　其他一些通用的最佳实践方法

除了我们这个方法，另外还有一些标准的优化方法，开发人员可以利用它们来提高资源文件的加载速度。这些方法并不是什么新鲜的点子，我们在此只是带领大家回顾一下现有的一些技术。

1. 顺序存取

从文件开始处顺序地读取文件中所有的字节。这是提高文件加载性能最关键的一个环节。顺序地读取文件可以充分利用系统中大容量存储设备的硬件缓存。通常情况下，大容量存储设备会稍微地超前读取被请求扇区之外的扇区。这其实是系统在预估应用程序会请求哪些扇区的内容。要想充分利用这个特性，唯一的方法就是顺序地加载数据。

2. 数据的预处理

一般来讲，如果不需要进行转换，系统就可以将数据直接加载到内存中，这样数据加载的速度就会更快。从表面上来看，一次数据类型的转换工作似乎是微不足道的，但随着数据集的不断扩张，数据转换的次数也会急剧增长。举个例子，假设最终用户使用的是一台低端机器，系统的内存及 Video RAM（视频随机存取存储器）都很有限。与其在加载时进行数据的向下采样（Down-sampling），不如我们去考虑发行一个预先创建好的“低端”数据集，减少对内存的需求。关于这个内容的详细信息，可参考[Olsen00]一文。

3. 在内存中缓存文件

如果可以避免的话，尽量不要去二次加载一个文件。如果游戏世界中很多多边形都使用同一个纹理，那么，资源管理系统不应该在游戏文件频繁引用它时，多次地加载这个纹理文件。我们可以把这类文件保存在一个公共资源库中，以便进行快速的查找，避免不必要的多次加载。

4. 数据压缩

加载数据所用的时间越短，游戏就可以有更多的时间去做一些更有趣的事情。对于那些

Download at www.Pin5i.Com

易于压缩的大型文件，比如超大的文本文件，我们可以考虑使用某种形式的压缩算法。在因特网上有一个优秀的免费压缩库，名为 zlib（请参见[Zlib05]）。

5. 了解你的硬件系统

大容量存储设备通常都有一个最优的数据块（Data Chunk）加载大小。对于 Xbox DVD 播放机，最优的数据块大小是 128k 字节（参见[Mansur05]）。对于不同的系统，这个最优的数据块大小也不尽相同。如果你使用的是一个知名的平台，一定要确保自己是根据该平台的使用规范来加载数据的。如果你手头上是一个 PC 游戏，你可能无法得到这些信息。因为目前还没有一个标准的 PC 硬件集。作为惯例，每次从存储介质种读取一个字节最糟糕的方法，请一定避免这样的操作。

1.9.8 总结

优化工作的细节与手头上具体的项目息息相关。不同的项目，应该采用不同的优化方法。我们首先要考虑清楚，自己的项目是如何使用资源的，这样才能判断出到底什么样的优化最适合自己项目的需求。与其他所有的优化工作一样，不要在项目临近结束时才去考虑这些问题。在产品开始制作之前，你掌握着最灵活的开发时间，应该在这个时候规划好加载速度优化的工作。对于基于访问顺序的二次文件排序这个方法，你也要考虑清楚，看自己的游戏是否可以充分利用这个方法。这个方法可能会使你的加载时间缩短一半，甚至更多。请记住，为了能够得到一个最佳的运行性能，你的游戏会准确无误地告诉你它所需要的是什么。去听听它的声音吧。我们在随书光盘中提供了源代码，这是本文中这个方法的一个简单的示范程序。

1.9.9 参考文献

[History03] "The History of Computer Data Storage: The timeline :)" available online at *http://www.usbyte.com/common/history_of_storage.htm*, 2003.

[Mansur05] Mansur, Mark, "DVD Layout and Load-Time Tips and Tricks," available online at *https://xds.xbox.com*, March, 18, 2005.

[Olsen00] Olsen, John, "Fast Data Load Trick." *Game Programming Gems 3,* Charles River Media, 2002.

[Zlib05] zlib, 2005. Available online at *http://www.zlib.net*.

Download at www.Pin5i.Com

1.10　你不必退出游戏：资产热加载技术可以实现快速的反复调整

High Moon Studios Noel Llopis　Charles Nicholson
llopis@convexhull.com
charles.Nicholson@gmail.com

说到如何去制作一款精雕细琢的游戏产品，不断地反复，不断地实践试玩，确实可以让我们的产品更加完美。虽然内容制作人员的技术水平很大程度上左右着最终游戏的质量，但是他们的努力经常会受制于令人痛苦的工作流。只是为了看一下 AI 脚本的一行代码的改变会产生怎样的效果，游戏策划就不得不先退出游戏，重新创建一个几百兆的资源文件，然后再重新运行这个游戏。如果能够让美工和策划人员快速、轻松地对游戏进行反复的调整，就可以为我们带来更好的内容，更多的实验数据，并最终为我们带来一个更完美的游戏产品。

资产的热加载过程就是这样一个过程：系统在游戏运行中自动地重新加载资产，而不需要停止或重新启动某个关卡。很多类型的资产都可以进行热加载，包括：纹理、模型、关卡几何体、脚本、游戏对象数据、声音和动画等。这样一来，只需要几秒钟的时间，我们就可以完成一次反复调整的过程。即使我们是在游戏机上开发游戏，也是如此。资产的热加载技术不但可以对游戏的质量产生巨大的影响，而且你也可以得到内容制作人员的满心感激。

1.10.1　资产热加载的工作流程

下面是一个典型情形的示意图，在这个图中，我们可以看到资产热加载是如何集成到工作流中，以及系统的各个不同的部分是如何彼此交互的。相关的步骤，如图 1.10.1 所示。

1．游戏美工在 Photoshop 中修改了一个纹理。

2．然后，他单击定制的“Export（导出）”按钮，保存纹理的源文件，并触发资产转换程序（Asset Convter），将 TGA 文件转换为更优化的二进制格式。资源转换程序还可以执行其他更为复杂的操作，诸如：改变色深（也称为位深度，bit depth）和压缩格式，或者使用恰当的过滤器来生成纹理映射。

3．这时候，文件监视器（file monitor，它一直在后台运行着，并监视

Download at www.Pin5i.Com

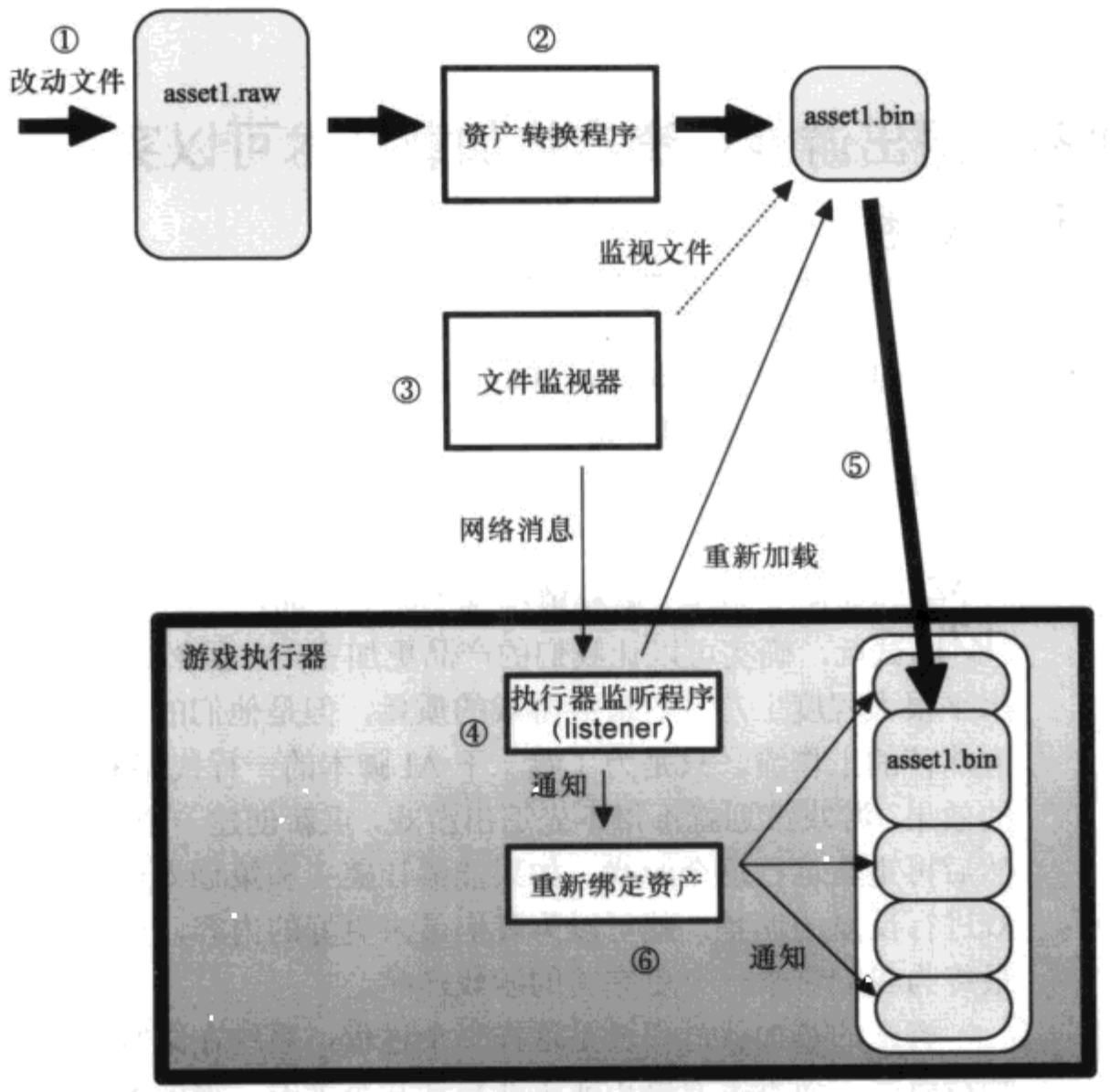


图 1.10.1 热加载工作流集成示意图

着相关的文件）就能检测到它所监视的文件中有一个被改动了（也就是那个由资产转换程序新创建的纹理文件）。然后，通过网络通道，文件监视器可以联络到游戏执行器（Game Runtime），并通知它，这个纹理文件需要重新加载。

4．游戏执行器（Game Runtime）通过网络接收到这个消息。然后，它就从这个消息中检索到这个资产的文件名，随后就启动真正的重新加载的过程。

5．这个新改动的纹理就被加载到内容中。这个操作中并不需要进行转换。因为在前面的步骤中，我们在导出这个纹理文件的同时，已经将其转换为二进制格式了。

6．原先那个旧的纹理文件已经过时了，被系统移除，并由这个新的纹理文件来代替它的位置。

7．在接下来的一帧中，系统在进行渲染时，就开始使用这个新的纹理文件了。

从那个游戏美工单击“Export”按钮，到这个新的纹理在游戏中显示出来，所用的时间是非常短的，也就是不到 1 秒钟的时间（转换操作的用时+网络通信的用时+加载时间）。对于大型的游戏资产，或者转换操作需要的系统开销过大，这个时间相对会长一些，但也很少会超过几秒钟。

1.10.2 资产热加载过程的剖析

在资产的热加载过程钟，主要涉及 4 个主要的元素：资产转换程序、文件监视器、执行器监听程序，以及资产的重新绑定。

Download at www.Pin5i.Com

1. 资产转换程序

资产转换程序读取 raw 格式或其他中间格式的资产文件，并创建一个二进制的、对应于系统平台的文件格式，这样游戏的执行器可以直接加载这种格式的文件。这一步骤并不是必须要执行的（执行器也可以直接加载未进行格式转换的文件），但却有几个实实在在的好处：

- 它可以尽早地在制作过程中检测出存在的错误或不一致的数据，这样内容制作人员可以马上得到系统的反馈。有了这个机制，我们就可以认定数据是正确的。
- 由于它的存在，我们就不需要再向执行器中添加大量复杂的加载代码了。取而代之的是，执行器直接加载一个内存块，将资产加载在这个内存块中，随时待命，以供使用。如果我们想要使用多种类型的资产（比如，几种不同的纹理格式：TGA、TIFF、PGN 等），这个程序就更为重要。执行器可以简单地先加载一种类型，然后将转换工作留给资产转换程序来完成。由此产生的是非常快速的加载时间，因为执行器只需要取加载数据，使用数据，不需要进行任何语法分析、内存分配等等其他的工作。

资产转换程序与我们一直用来进行游戏资产创建的工具是一模一样的，不是为了支持资产的热加载而专门开发的。唯一有所区别的地方是：在这里，只要美工或策划人员修改了某一个资产文件，就会立即触发资产转换程序；而不是将它作为一个批处理过程，一次性地来处理大量的资产。

资产的转换并不只是简单地改变它们的格式。复杂的资源会依赖于其他的资源，所以，资产转换工具同时还要进行一些依赖性检查，并创建所有需要重新创建的内容。

2. 文件监视器

文件监视器的工作是监视已创建资产的目录，并在任何文件发生变化时，通知执行器的监听程序。当资产转换程序以最终的平台专用形式，创建一个资源资产时，最后的结果文件就被保存在文件监视器所监视的这个目录中。文件监视器一直在不停地监视着这个目录，这个时候，它就会注意到有一个写文件的操作发生了，并将这个文件通过网络发送给执行器的监听程序。

实现一个文件监视器最简明、最有效的方法是通过 OS 专有的文件事件钩子函数（file event hooks）。文件监视器不必不停地去监视某些文件的状态，相反，文件监视器可自行注册，这样，当某个目录下的任何文件发生变化时，它就可以收到一个相应的事件（event）。

文件监视器在 Win32 系统上的实现可以使用 ReadDirectoryChangesW 函数。其他的平台有它们各自的文件事件通知函数，比如在 Linux 下，我们可以使用 dnotify/inotify。

ReadDirectoryChangesW 函数的文档看上去有些复杂，但是它却简化了我们的问题，将一个目录与一个事件关联起来。当这个目录中的文件发生变化时（执行任何文件操作时），就会发出相应的信号。文件监视器通过 GetOverlappedResult 函数，每 200 毫秒就轮询一次这个事件，看它是否结束。如果返回值为 True，这就意味着目标目录中有一个文件发生了变化。多个发生变化的文件集合用一个大型的 char buffer 返回。这个 char buffer 的结构必须与 FILE_NOTIFY_INFORMATION 的结构相符。只有那些打开了 FILE_ACTION_MODIFIED 属性的表项才会被处理，表明它们刚刚被写入磁盘。

在很短的时间段中，修改很多的文件，这是很常见的情况，因此，系统会将那些正在改变的文件的文件名列表在本地缓存 1 秒钟的时间，然后再将这些文件通过网络发送给监听器。这样可以减少批处理的次数，但同时也带来了一个副作用，就是要对付 ReadDirectoryChangesW

Download at www.Pin5i.Com

函数的怪异行为：它经常会针对同一个文件发回重复的通知信息。而每个文件的通知次量与发生变化的文件的总数之间似乎有所关联！

处理完所有改动过的文件，并将它们在网络上发送完毕，文件监视器必须要再次调用ReadDirectoryChangesW 函数来重新启动监听过程。比较方便的是，我们可以重用 Event 对象。所以，在文件监视器这个程序的运行周期中，它只需要一个 Event 对象。

如果目标平台使用的文件系统与开发平台使用的文件系统不一样（就像开发 Xbox 游戏那样），那么，在向游戏执行器报告这些变化之前，文件监视器还应该将那些变化了的文件复制到目标平台上，或者，它至少应该触发另外一个程序来完成复制工作。

3. 执行器监听程序

执行器监听程序不停地监听一个特定的网络端口，来准备接收来自文件监视器的通知。一旦有通知进来，它就启动一个特定资产重载的过程。

为了能够正确地完成自己的工作，监听程序必须知道那个需要被重载的资产，它的文件名是什么，以及这个文件将要替换掉执行器中的哪个资产。在最简单的情况下，执行器可以知道所有的文件名，而这就是它加载一个新文件所需要的全部信息。而在其他情况下，如果执行器并没有一个文件名与具体资产之间的映射表，我们就要将文件名及对应资产的 ID 一起传递给监听程序。在这种情况下，文件监听器也必须要知道资产的 ID。至于具体的形式，可以是一个在资产创建时生成的一个映射文件，也可以是游戏资产中某个它可以访问的文件。

执行器监听程序是资产热加载系统中唯一一个需要添加到游戏执行器中的部分。这样的结构用起来非常方便。由于我们只需要在开发过程中来使用资产的热加载，因此，使用资产热加载技术的目的之一就是尽可能少地去改动游戏的执行代码。对于需要添加到执行器中的新增代码，我们应该将它们的数量保持在一个最低的水平上。而且，在游戏正式发行之前，我们也可以用#ifdefs 语句，很容易地将这些添加进去的代码移除。

4. 资产的重新绑定

一款游戏需要很多不同类型的资产。一个典型的游戏应该包括：纹理、模型、动画、声音、音乐、对象定义及实例、用于控制和 AI 的脚本、可见的或物理静态几何体，以及一大堆游戏专用类型的资产。这些类型的资产中，有一些可以很好地适用于上面的热加载形式，而其他类型的资产就需要我们更加小心地对待了。

一旦执行器监听程序被通知某个文件发生了变化，游戏就开始去加载新的资产。这个过程可以是同步方式的，也可以是异步方式的。以异步方式实现的好处是：在加载的过程中，帧率不会波动太大，但新的资产可能无法在最前面的几帧中显示出来。另一方面，虽然同步加载会造成游戏微小的停顿，但它的好处是实现起来相对比较简单，而且，最重要的一点是这种同步加载方式不需要额外的内存。因为我们并不需要在内存中同时保存旧版本和新版本的资产。由于资产热加载这个功能只是用于开发过程中，因此除非你手头上已经有一个非常稳定可靠的异步资产加载系统，否则采用同步加载方式就足够了。

通常情况下，我们无法确保新的资产文件所需要的内存大小，正好与它要替换的那个资产的内存大小一模一样。所以，更多的时候，我们会为新的资产分配不同的内存地址。由于我们希望游戏在以后的运行中来使用这个新的资产，而不是去使用那个旧的资产，因此，我

Download at www.Pin5i.com

们需要使用某种形式的间接处理方式（比如，一个句柄，或者一个弱引用），或者需要去跟踪记录哪些部分的代码正指向那个旧的资产，并将它们重新指向新的资产。

将这些最新改动过的资产加载到目标平台上只是这场战役的序曲。当我们把新的资产加载到内存中后，它们还得去替换掉它们原来的实例，并绑定到某种资源管理系统上。这个过程被称为“重新绑定”。这个过程最终的结果就是：不用重新启动游戏，新的资产就变成了可见的、可用的。

重新绑定过程的主要问题是：系统总是很难判断出哪些对象还在引用那些将要被替换掉的旧资产。在 C++语言中，native pointer（本地指针）类型不足以帮我们解决全部的问题。因为我们不能使用它来判断出目标何时将被销毁。这样就会产生空悬指针（dangling pointer）的问题，继而又会带来运行的失常。

为了解决这个问题，有一个解决方案就是禁止使用直接资源指针，转而去使用一个资源句柄表（resource handle table）。句柄表就是一个全局表，其中包含了游戏当前所加载的所有资源的句柄。这个句柄表的拥有者可能会是一个主资源管理器对象。如果游戏中的某个对象需要使用某个资源，它必须使用一个资源 ID，通过这个句柄表来访问这个资源。然后，这个对象就会得到一个返回的临时指针，该指针指向的就是它需要使用的那个资源。但这个对象并不缓存这个指针。如果在以后的某个时间，该对象还想使用这个资源，那么，它必须再次从句柄表中申请这个资源的指针。如果某个资源被另外一个热加载的资源替换掉了，资源管理器只是简单地用新的资源来更新相应的表项，但仍然保持资源 ID 不变。以后在进行查询时，句柄表就会返回这个新加载的资源，而客户端代码却对此一无所知。为了方便起见，我们还可以将句柄机制封装到一个智能指针中。这个智能指针的运行行为与 C++的 native 指针大致相同。

在所有需要重新绑定的资源类型中，纹理可能是最简单的一种。因为它们没有外部附属文件，也无须进行状态维护。一般只需将纹理加载到内存，更新其指针。模型几何体和粒子数据也与纹理一样易于处理。即使动画也是偏于易于处理的资产的范围。由于这些资产主要都是由自然状态的轨迹数据组成的。为目前为止，我们讨论过的资产都是是比较简单的。根据游戏的定义，如果被重载的资产越是基本的资产，那么在运行时替换这类资产所需要的步骤就越具破坏力。举个例子，假设我们有一个关卡定义，其中包括了起始位置，以及这个关卡中所有角色的状态。那么这个关卡定义的热加载就会表现出更为复杂的问题。为了能够让这个新数据变为可见的，最简单（而且可能是唯一的）方法就是先销毁这个关卡，然后再重建这个关卡。同样地，内存容量的限制也会使得我们完全无法实现大型静态关卡几何体和大型纹理的重载。如果关卡几何体足够庞大，对它进行热加载所需要的时间，与简单地重新启动整个游戏的时间相差无几。所以，我们不值当去进行关卡几何体的热加载。

虽然资产的热加载技术可以大大降低对游戏进行反复调整所用的时间，但我们没必要对所有的资源都使用这个技术。如果存在技术或时间上的限制，我们可以对大部分最通用的资产实现热加载功能，以最小的工作量获得最大的回报。

1.10.3　实际应用中需要考虑的事项

1. 网络连接

在开发用的计算机与目标平台之间保持一个活跃的网络连接是非常有用的。各种资产热

Download at www.Pin5i.Com

加载命令只是多种可能的开始。这样的一个网络连接可以为我们带来一个全功能的调试控制台。在这个控制台上，我们可以在各个状态之间切换，执行各种命令，或者查询并显示各种变量。无论是在各种不同的平台上，甚至在远程的机器上，这个控制台的工作模式都是完全统一的。

2. 内存碎片

不断地销毁和重载资产文件，会导致内存碎片的产生，最终，会使系统可能无法为那些新创建的资源分配连续的内存。有一种方法可以解决这个问题，如果新旧两个资产的大小一致，就可以重用现有资产的内存。但是，如果新旧两个资产的大小不一致，那么内存碎片还是无法避免的。

我们采用的方法是简单地让内存碎片的发生顺其自然。这些问题其实只会在开发过程中才会出现。而开发工作所使用的系统的内存通常都比目标平台的内存要大。所以，我们应该可以空出一部分的内存。如果在经过 8 个小时的热加载操作之后，游戏最终无内存可用了，那么我们就可以简单地重新启动游戏，然后继续工作。

还有一种变通的方法，在开发系统中隔离出一大块内存，并使用一些著名的堆管理算法来管理这块内存。这样就可以将碎片保持在一个最低的水平上，而我们付出就是一些空间的浪费。

最后，如果我们还是会发生内存不足的情况，我们只好退而求其次，勉为其难，将资源文件进行压缩。同样地，因为资源的压缩也只是在开发过程中使用的，所以，多花上几秒钟的时间也不是什么问题（而且这种情况每几个小时才会出现一次）。既然我们已经掌握了这个技术，可以对各种资产运筹帷幄，那么这个微小的时间损失不是什么太大的问题。

3. 资源的打包文件

在加载资产的时候，你可以使用一个大型的资源打包文件。这个打包文件中包含着某个特定关卡所需要的所有资产。如果是这样的话，你还需要确保你仍然可以直接地加载单个资产文件。这同时还意味着资产转换程序不应该将修改后的资产文件打包成一个打包文件，而是应该将这些被修改的资产保留为单个独立的文件。

1.10.4 示范程序

在随书光盘中，我们提供了一个示范程序，实现了纹理的热加载。它囊括了我们在本文中所介绍的三个主要的系统部件：

资产转换程序：将 TGA 文件转换成二进制纹理。这样，通过一个简单的头文件格式，OpenGL 就可以直接加载这个纹理。

文件监视器：监听某个特定目录中的文件是否发生变化。

显示程序（执行器）：一个简单的 GLUT 应用程序，显示 3 个加上了纹理的茶杯。它还监听某个端口，看是否有从文件监视器传来的通知。

为了测试资产热加载功能，我们可以首先来运行这个显示程序。然后再启动文件监视器，

Download at www.Pin5i.Com

并指示它去监视哪个目录，并告之执行器的网络地址。修改一个“textureX.tga”文件，或者干脆直接拷贝过去，覆盖它。然后点击运行“Make”，以触发资产转换程序。这也会触发文件监视器，发出一个通知，然后显示程序中马上就会进行重载。

1.10.5　总结

资产的热加载是一个非常有用的技术，实现起来也不太费事。本文介绍了一个可能的实现，尽可能地减少了离线工具的工作量，并将对游戏执行器的改动工作最小化。通过对那些经常被改动的资产使用热加载策略，或者对游戏中所有的资产来实现热加载，在他们进行反复的修改时，内容制作人员就可以不用再退出游戏了，加快工作速度，为他们的项目创作出最好的内容。

1.10.6　进阶参阅

Bloom, Charles, “The making of *Oddworld: Stranger's Wrath.*” GameTech 2005. Available online at *http://www.cbloom.com/3d/game_tech_04.zip.*

Hawkins, Brian, “Handle-Based Smart Pointers.” *Game Programming Gems 3,* Charles River Media, 2002.

Johnstone, Mark S., “The Memory Fragmentation Problem: Solved?” ISMM98.

Llopis, Noel, “The Beauty of Weak References and Null Objects.” *Game Programming Gems 4,* Charles River Media, 2003.

Download at www.Pin5i.Com

Download at www.Pin5i.Com

第 2 章

数学与物理

Download at www.Pin5i.Com

简　介

Jim Van Verth，红色风暴娱乐有限公司
jimvv@redstorm.com

自从艾萨克·牛顿时代开始，数学和物理之间就建立了它们自己的联盟。但是，这种合作伙伴关系也并不是一直都很稳定的。比如，美国的最伟大的物理学家理查德·费曼（Richard Feynman）就对数学一点都不感兴趣。但无论如何，如果没有数学的存在，那么也就不会有物理学的存在。反过来，如果没有物理学，那么，在数学领域中那些在物理学启迪下取得的成就可能就要推迟很长的时间才能面世。当然，如果不是若干年前牛顿帮助缔造了微积分学，那么他自己也会发现，他很难去证明自己的重力理论。

数学和物理学之间的这种相互依赖的关系也延续到了游戏开发中。用数学来描述的物理学的定律依然成立。但如果离开了数学，我们就不能去编制一个程序来演示这些定律。在计算机领域中，用来解决物理仿真问题的大部分算法都是来自于数值分析。而数值分析本身就是数学的一个分支。归根结底，我们可以把这些都归结为公式，以及这些公式的代码实现。正是基于这一点，我们将数学和物理编排在同一章中。

我们从基本原理入手。在数学的理想化世界中，实数有着无限的精度。即使我们用 0 去除某个实数，我们得到的也仅仅是一些新的探索途径。在生硬、冰冷的数字计算机的世界中，我们可没有如此的幸运。但是，我们有 Chirs Lomont，这也是我们的幸运所在。他为我们提供了一篇文章，带领我们走进这个冰冷的世界。他在文章中向我们展示了如何利用些许的创意，我们就可以利用浮点数表示方法的种种限制，编写出更富效率的代码。我们可以把一个浮点字的各个表示比特（bit）看成是一个整数，这样，大量的浮点运算就可以更快速地得到处理，或者至少是得到与整数处理通道平行的处理速度。

我们当中很少有人会把齐次坐标、求解线性方程组，以及叉乘积（cross-product）看作是彼此相关联的课题（除了我们知道它们都是计算机图形系统的组成部分）。在本章中，我们收录了两篇文章，它们分别从各自不同的角度，介绍了同一个技术。第一篇是 Vaclav Skala 的文章。文章重点介绍了齐次坐标和线性系统之间的关系，以及如何巧妙地利用叉乘积，找到直线与平面的交叉点，以提高裁剪代码的效率。第二篇文章是 Anders Hast 的作品，他对如何利用叉乘积来求解线性系统这个主题，作了进一步的扩展。说得具体一点，这篇文章向我们介绍了如何利用这一相同的技术，

Download at www.Pin5i.Com

来加速对三角形属性进行扫描转换的初始过程，提高小矩阵求逆的计算速度。

大部分游戏都包含有自己的对象集，诸如：玩家数据、动画对象、或者是技术图表等。在谈及这些对象集的时候，很多的开发人员都会从数据管理的角度来看待它们，而不是从数学的角度来看问题。Palem GopalaKrishna 的文章为我们提供了一个新的视角，向我们展示顺序索引（sequence indexing）如何可以助你一臂之力。顺序索引（sequence indexing）法一直都在尝试解决符号集和符号子集的枚举问题和排序问题。我们可以利用这个方法对我们的数据进行归类、压缩，为我们数据生成行为和规格。

在今天，对于刚体之间的碰撞检测和处理，很多人对其相关的基本技术都已经了然于胸。如果你的游戏世界中只有砖墙和通道，这些基本的技术也就足够用了。但是，地球上 70%的面积都被水覆盖着。所以，我们的游戏也应该对此有所反映。因此，刚体和流体之间交互作用的实时管理技术也逐渐变得炙手可热。在《游戏编程精粹 6》的这一章内容中，我们很幸运地可以享有两篇精彩大作。在其中的一篇文章中，Erin Catto 介绍一个简单的几何技术，来计算一个多边形刚体在流体中的浮力。作者在这篇文章中还和大家探讨了一个阻力的近似模型。本章的最后一篇文章来自于 Takashi Amada，其内容也对接到流体与刚体的交互作用上。作者在文章介绍了一个名为“平滑粒子的流体动力学”的方法。这是一个基于粒子的方法（相对于标准的基于单元的方法），可以用于流体及流体与刚体之间交互作用的仿真。

Download at www.Pin5i.Com

2.1 浮点编程技巧

Chris Lomont，Cybernet 系统公司
chris@lomont.org

在从前，视频游戏中的浮点代码是非常少见的。在那个时候，与基于整数的代码相比，浮点代码实在是太慢了。所以，在进行软件光栅化的时候，大家只能借助于定点数的手段。在今天，这样的日子已经一去不复返了。浮点运算现在已经变得很快了，很多的处理器也都配备了独立的浮点处理通道和整数处理通道。所以，如果能将浮点代码与整数代码恰当地分割开来，程序的性能会得到大幅的提升。现代的游戏编程大量地倚重于浮点代码。从 GPU 到物理引擎，从声音处理到碰撞例程，再到交叉测试，浮点代码无处不在。我们可以这样预测：在未来的行业中，游戏会从使用实时的射线跟踪引擎启动。到了那个时候，浮点编程技巧会变得越来越有用。不幸得是，很多编程人员不但对浮点编程（诸如：数值分析和误差传递）的错综复杂一无所知，而且还经常犯下一些低级错误，比如用“==”来进行浮点值的比较。本文会讲授一些浮点编程的基础知识，同时也会介绍一些极好的编程技巧，来提高浮点代码的运行速度。

大部分的编程人员，包括那些非常出色的人士，他们甚至搞不清楚，在他们所选择使用的平台上，浮点数是如何存储的。你可以做个有趣的实验，首先问问你的同事，整数值是如何存储的，以及如何进行整数的算术运算操作。然后，针对浮点数，再问问他们同样的问题。真是够尴尬的，能够深入、完整地解释游戏编程中生僻的 BCD（二进制编码的十进制数）的程序人员不在少数。但能够把浮点编程的细节内容说清楚的人，却寥寥无几。你也许会经常地在工作中用到浮点代码，那请回忆一下，最后一次使用 BCD 编程是什么时候呢？

和整数编程一样，要想熟练掌握浮点代码，我们首先要理解浮点数值是如何以比特（bit）的表示形式存储的。本文会详细地介绍几个主流的系统所采用的浮点数的存储格式。而对于那些没有实现这个存储方法的冷门系统，它们大体上总是会采用一个几乎完全一样的方法，只是稍稍做了些修改而已。所以，通过研读本篇内容，你所能获得的知识，会让你在算法开发的工作中获益匪浅。

在我们开始之前，最后还有一点提请大家注意：在《游戏编程精粹 2》中，我们曾经收录了一篇内容类似的文章[King01]。本文对那篇文章进行了几个方面的扩展，提出了很多新的技巧，也至少对一处内容进行了改进。

Download at www.Pin5i.Com

现在，让我们首先了解一下浮点数的位（bit）表示形式。

2.1.1　浮点数的格式

几乎所有你碰到的浮点数都遵守 IEEE 754 标准。IEEE 754 标准是一个浮点数标准格式，精确地定义了浮点值的存储方式，以及在基本操作中误差传递的方式；如何处理上溢出和下溢出、“非数值”（Not-a-Number，NaN）和无穷大。在 IEEE 754 标准成为各个平台的执行标准之前，浮点代码几乎谈不上什么可移植性，在不同的平台上，总是会产生完全不同的、错误的计算结果。关于这些历史故事，大家可以参阅 William Kahan 的文章“浮点数之父”一文[Kahan98]。

最有可能的情况是：你所能见到的所有的浮点数表示格式，甚至是那些并未严格遵守 IEEE 754 标准的格式，都是 IEEE 754 标准的变种。IEEE 754 标准范例包括了存储于 PC 和当前最流行及(或许是)未来游戏机中的浮点数。虽然 3DNow!和 SSE/SSE2/SSE3 都采用了 IEEE 754 的设计标准，但在其他细节方面的支持上（比如 IEEE 754 标准要求的舍入模式），它们与 IEEE 754 标准之间多少有些差异。NVIDIA 和 ATI 公司的 GPU 产品，以及 PlayStation 2 游戏机也采用了位兼容（bit-compatible）的浮点数格式。如此看来，在很多领域中，深入了解浮点数的位表示方法是非常有用的。

1. IEEE 754 浮点数格式基础

IEEE 754 定义的单位大小是一个 32 位的浮点数格式，这也是很多 C/C++语言实现中使用的标准的浮点数据类型。该格式 32 位设计布局，如表 2.1.1 所示。

表 2.1.1　　32 位浮点数存储格式

符号位 S	偏置指数 E	压缩的尾数 M
第 31 位的 1 个比特位	第 30～23 位中的 8 个比特位	第 22～0 位的 23 个比特位

这个格式表示的浮点数就等于 $(-1)^{s} \cdot 2^{E-127} \cdot (1+M/2^{23})$。其中 M 是一个 23 位的无符号整数（unsigned integer）。下面我们将这个公式分解成一个一个的比特位，详细介绍一下。

在一个 32 位的浮点数表示中，最高一位的就是符号位 S。如果 S=0，则表示的是正数；如果 S=1，则表示的是负数。接下来位于第 23～30 位上的 8 个比特位是偏置指数 E。“偏置”的意思是，将这个 8 个位上的值看作是一个无符号的 8 位整数 E，然后再减去一个偏移量 127，得到一个真正的指数（E–127），并以此作为 2 的幂。这样，上面的公式就可以表示非常大、或非常小的数值了。

最后一个部分（这也是大多人最容易出错的地方），第 0～22 位上的 23 个比特位表示的是压缩的尾数。你可以把它看成是一个 23 位的无符号整数 M。这 23 个比特位表示的是一个[0，1]之间的小数，也就是 $M/2^{23}$。

“压缩的”这个词意味着，在尾数前面有一个隐含的 1（也就是没有在格式中表示出来的意思），因此我们就得到了 $1+M/2^{23}$。为了叙述方便，我们用 1.M 来表示这个值。存储在这个压缩格式中的数值也被称为“规格化的数”（normalized number），因为在这个数值中，开头第一个比特位被移位到隐含的位置上，不需要把它写下来。指数也会相应地减小，以便维持

Download at www.Pin5i.Com

一个相同的量级。这样一来，与“非规范化”的数相比，我们就可以多存储一个比特位。在本文中，我们用 M 和 E 来表示一个浮点数格式中的无符号比特位的值。

我们可以举一个例子，看看表 2.1.2 中这个比特位的样式，它表示的就是一个浮点数：

表 2.1.2　　剖析一个浮点数

1	1000 0010	101 0000 0000 0000 0000 0000

那么这个浮点数的值是多少呢？符号位是 1，所以这个数一定是个负数。指数位表示的是 E=130，所以，指数就是 130–127=3。尾数位表示的是 $M=2^{22}+2^{20}$，所以尾数的值 1.M 就是 $1+(2^{22}+2^{20})/2^{23}=1.625$。这样，表 2.1.2 表示的浮点数就是 $(-1)^1 \cdot 2^3 \cdot 1.625=-13$。

在 PC 和几乎其他所有的平台上，其他单位大小的浮点数也遵循相同的设计布局，但也有各自微小的差异，参见表 2.1.3。

表 2.1.3　　浮点数的单位大小

类　型	float	double	long double	Half-Float[①]	SPARC long double
位数	32	64	80	16	128
符号位	1	1	1	1	1
指数位	8	11	15	5	15
偏置指数	127	1023	16383	15	16383
尾数位	23	52	64	10	112
是否压缩	是	是	否	是	是
小数精度	7	16	19	3	34

注意，其中 80 位的长双精度型（long double）不是压缩形式的。所以，那个原来被隐藏的那一个比特位就显式地成为了尾数中最重要的一个比特位。另外一个重要的注意事项是：x86 处理器内部的浮点运算使用的是 80 位浮点表示，所以，很多表示格式在进行计算时都使用 80 位的格式，而在存储的时候再进行压缩。让人沮丧的是，现在微软的 C/C++编译器无法存取 80 位的浮点值。在微软的 C/C++编译器中，long double 和 double 类型都被看作是 64 位数据类型。如果你确实需要这种精度的浮点值，唯一的选择只能是汇编语言。

2. IEEE 754 的特例

就这么简单啊，是吗？那好，根据我们上面给出的规则，我们来试试表示几个重要的数值，比如 0。你会发现，使用上面的规则，我们根本无法表示 0 这个数。为什么呢？因为在尾数部分总有一个隐含的 1。所以，在上面的实数表示方案中，我们需要加入一些特例，来表示 E 的极值情况。除了数字 0，IEEE 754 标准中还定义了其他几个特殊的数值：

当 $1 \leqslant E \leqslant 254$ 时，就用上面的规则来表示这个数。

有符号的 0，±0：当 E=0，且 M=0 时，那么表示的就是数字 0。但是，符号可能是 0，或 1。对于这两种情况，我们认为它们表示的都是浮点数 0。

非规范化数（也被称为次规范化数）：当 E=0，且 $M \neq 0$ 时，这时候，那个隐含的 1 就不存在了。这时候表示的就是那些非常非常小的数（接近于 0），同时也允许适当的下溢出。其

① 用于 DirectX 9.0 和 OpenGL1.2 中。ATI 和 nVIDIA 芯片组，及其他一些地方都使用这种类型的浮点数。

Download at www.Pin5i.Com

代价是这些模式所表示的数都没有什么意义。

无穷大：当 E=255，且 M=0 时，根据不同的符号位，这时候表示的是±∞。

“非数值”：当 E=255，且 M≠0 时，表示的就是一个“非数值”NaN。如果 M 的第一个比特位是 0，则表示的是一个有信号的“非数值”（Signaling NaN，SNaN）。如果 M 的第一个比特位是 1，则表示的是一个无信号的“非数值”（Quiet NaN，QNaN）。Intel 只实现了一个 NaN，并称之为 QNaN。有信号的 NaN（SNaN）在代码中触发一个异常（exception），而 QnaN 则不会（所以说它是“无信号的”）。

上面的这些规则完整地描述了 IEEE 754 标准中 32 位浮点数的各种可能的情况。这些定义使得一些特殊的数也可以进行正确的运算，比如：∞+∞=∞、∞−∞=*NaN*，以及 1/0=∞（这个并不是严格正确的）。更古怪的还有：$\sqrt{-0}=-0$。很明显，我们对上述规则进行归纳，就可以为其他数据大小的浮点数设置同样的特例数值的位模式。更详细的内容，请大家参阅[754Ref05a]和[754Ref05b]。

到这里，我们已经了解了全部的浮点数的存储规则。这对我们后面要探讨的内容大有帮助。在后面的内容中，我们将探讨：为了易于检查，如何将各种各样的位模式映射到一个实数排列上。在表 2.1.4 中，我们罗列了各种特殊的浮点数（从负的最大值，到正的最大值）及其表示方式。生成这个表的代码在随书光盘中 MakeNumberLine 函数中。

表 2.1.4　特殊的浮点数和它们的表示方式

浮　点　数	十六进制表示	有符号的 32 位整数	名　　称
–1.#QNAN	FFFFFFFF	–1	负的 NaN
–1.#QNAN	FFFFFFFE	–2	
…	…	…	
–1.#QNAN	FF800002	–8388606	
–1.#QNAN	FF800001	–8388607	
–1.#INF	FF800000	–8388608	–∞
–3.40282e_038	FF7FFFFF	–8388609	负的规范化数
–3.40282e_038	FF7FFFFE	–8388610	
…	…	…	
–1.17549e–038	80800001	–2139095039	
–1.17549e–038	80800000	–2139095040	
–1.17549e–038	807FFFFF	–2139095041	负的非规范化数
–1.17549e–038	807FFFFE	–2139095042	
…	…	…	
–2.8026e–045	80000002	–2147483646	
–1.4013e–045	80000001	–2147483647	
0	80000000	–2147483648	–0
0	00000000	0	+0
1.4013e–045	00000001	1	正的非规范化数
2.8026e–045	00000002	2	
…	…	…	
1.17549e–038	007FFFFE	8388606	
1.17549e–038	007FFFFF	8388607	
1.17549e–038	00800000	8388608	正的规范化数

Download at www.Pin5i.Com

续表

浮 点 数	十六进制表示	有符号的 32 位整数	名 称
1.17549e–038	00800001	8388609	
...	...	...	
3.40282e_038	7F7FFFFE	2139095038	
3.40282e_038	7F7FFFFF	2139095039	
	7F800000	2139095040	+∞
1.#QNAN	7F800001	2139095041	正的 NaN
1.#QNAN	7F800002	2139095042	
...	...	...	
1.#QNAN	7FFFFFFE	2147483646	
1.#QNAN	7FFFFFFF	2147483647	

这里要注意的是，如果我们取得非负递增浮点数，将它们的位模式看成是有符号的整数，那么结果也是递增的。同样地，把它们看成是有符号的整数时，负的浮点数要比正的浮点数小。不幸的是，在被看作是有符号的整数时，负的浮点数是递减的。

还有一点需要提醒大家注意，在处理“非规范化”数、无穷大和 NaN 时，Intel 公司的处理器产品要比 AMD 公司的产品慢很多，几乎慢了近 900 倍。关于这个有趣的内容，大家可以去参考[Dawson05b]一文。所以，当我们设计浮点代码来处理特殊情况，或者非常小的数时，你要根据不同的系统，仔细地分析自己的代码。

我们接下来简短地探讨一下浮点数的比较问题，将这部分内容的告一段落。我们需要一个数字 0 的表示方法，但不幸的是，我们得到了两个不同的位表示模式：+0 和–0。如果把它们看成是浮点数（float），那么它们就是相等的。但很明确的是，当我们把它们看成整数（int）时，我们就得小心处理了。即使是进行浮点数的比较，我们有时候也会碰到一些问题。举个例子，在 C/C++中，对于任意一个浮点数 v，v= =v 的结果是 true 吗？或者给定两个浮点数 v1 和 v2，下列这些判断式中至少有一个是 true 吗：v1＞v2，或者 v1= =v2，再或者 v1＜v2？到本文结束的时候，大家就可以看到令人惊奇的结果了。

2.1.2 例程的设计

在本文中，我们没有足够的篇幅来讲解浮点例程的数值分析。相反，我们只能向大家介绍一些实用的技巧，利用浮点数的位表示形式，来操作浮点值。虽然不像我们直接使用 CPU 的浮点运算单元那样稳定、精确，但这些技巧通常可以为我们带来更快速的代码。当你需要在一个核心例程中提高额外的 40%的运行速度，但尝试了所有最新的算法技巧，也无法进一步提高代码的运行速度了，这个时候，本文介绍的这些技巧就是救命稻草，帮助你和你所钟爱的项目解决这些难题。对于其他的优化，你需要对最终代码的执行进行用时统计，确保它可以在你的目标平台上正常运行，并且可以达到你所要求的精度级别。

如果我们可以把一个浮点值（float）看成一个整数（int），就可以访问它的各个比特位，因此，利用 C++中的布尔运算和移位操作，我们就可以来操作一个浮点值的每一个位。在很多情况下，这样的操作比浮点例程要快得多。但这样做，通常会微微地损失一些精度，或者无法正确地处理非规范化数和异常值。可还有一些情况是，如果把浮点值当作整数值来处理，

Download at www.Pin5i.com

反而会让我们的例程更精确。

有些时候，我们介绍的这些技巧并不能兼容所有的平台/系统架构。我们会尽量告诉大家其中的问题和需要注意的地方。最后一点，除非你真的需要，否则千万不要过分倚重这些技巧。它们通常会给我们带来无法移植的、难以维护的代码。但是，如果不使用这些技巧，我们也无法获得可能的性能提升。

1. 准备工作

为了能够访问一个浮点数表示中的各个位（bit），我们要求 float（浮点类型）和 int（整型）的长度是一样的，都是 32 位的。如果你的编译器不能满足这个要求，那么我们也可以使用其他相当的整数长度。如果你想随意操作 Visual Studio.NET 中的 double（双精度浮点）类型浮点数的各个表示位，你就可以使用_int64 类型，因为它们两个都是 64 位的。在整个代码中，我们定义了 float（浮点类型）类型的变量 fval，并定义了变量 ival，作为这个浮点数的整数表示。

我们要创建一个例程，来检查运行平台的位数，以确保我们这个库可以在上面正常地运行。随书光盘中的提供的例程 TestArchitecture 就是来完成这个工作的。我们希望目标平台可以支持如下特性：float 类型和 int 类型的长度一致，可以进行我们想要的无符号的移位操作，而且能够按照我们的需要呈现出相应的行为。如果运行平台支持这些特性，例程 TestArchitecture 就返回 true；否则它就会返回 false。如果它返回 false，你就得仔细地检查自己的例程，改变所定义的支持特性，创建出一个可以在现有平台上正常运行的版本。

接下来，我们首先会向大家介绍几种方法，如何使用 C/C++来直接访问一个浮点数的各个表示位。最常见的方法就是一个直接的转型操作：

```
int ival= (* (int*) &fval);
```

对于那些不主动进行优化的 C++编译器，我们应该使用下列操作，这样要快一些：

```
int&  ival= (* (int*) &fval);
```

但是，Falk Huffner[Anderson05]指出，新的 C99 ISO 标准为这个操作指定了未定义的行为，所以，我们或许应该使用这个操作：

```
memcpy (&ival, &fval, sizeof (int));
```

第三种方法是使用 float 和 int 类型的一个 union（详见[King01]），例如：

```
typedef  union{float f; int i; }IntOrFloat;
```

第四种方法是使用内联汇编代码（inline assembly），简单地将一个浮点值的内存读入到一个 32 位的寄存器中，然后再去对这个寄存器的内容进行相应的操作。

我们在随书光盘中提供的代码可以支持前两种方法，是通过两个#define 语句来控制的。我们的代码中也有一些地方使用了 union 方法。

Download at www.Pin5i.Com

在我们的代码中，第一种方法是默认的方法。在通篇的代码中，我们使用了两个宏：_fval_to_ival（）和_ ival _to_ fval（），它们来决定该使用哪种方法。你要对你的平台非常了解，才能知道哪种方法最适合。有些时候，使用 union 的方法总是会产生一个内存拷贝操作。而如果用一个很小的值来调用 memcpy（），有时候会被彻底地优化掉，并不会产生一个函数调用，而是让编译器直接使用 float 变量的内存位置。如果不去尝试使用各种方法，并仔细地查看生成的汇编代码，那么我们根本无法提前预知哪种方法最适合自己。

其他常见的一些操作，根据一个 float 类型的正负，需要一个全是 0，或者全是 1 的位掩码（bitmask）。[King01]本质上使用的是：

```
int mask=(inval>>31);
```

但这种用法可能会遇到一些问题，因为在 C/C++标准中，无论符号是否被保留，对一个有符号整数（signed int）的右移位操作，都是执行期定义的（implementation-defined）。详见[Lomont05]。在 Intel 架构下使用微软的编译器，这个语句可以正常工作。对符号整数（signed int）的右移位操作也要嵌套在例程 TestArchitecture 中。所以，我们定义了一个宏，来处理这件事情：

```
#ifdef _SIGNED_SHIFT
#define SIGNMASK(i) ((i)>>31)
#else
#define SIGNMASK(i) (~(((unsigned int)(i))>>31)-1))
#endif
```

为了测试，我们需要构造和解构一个浮点值。给定一个符号、指数和 unsigned int 类型的尾数，例程 MakeFloast（）和 SplitFloat（）就可以创建一个浮点值。而且，这个过程也是可逆的。另外，还有一个例程 DumpFloat（），它可以向 ostream 类输出各种格式的浮点值。

最后要说明的是，我们的例程不能正常地处理无穷大和 NaN（非数值）。对于“非规范化”数，我们通常也需要进行特殊的处理。但是，由于大部分的游戏代码并不需要去处理这些特殊情况（“非规范化”数除外），所以，我们的例程用起来相当不错。

2. 对符号位的操作

在这里，我们首先来看看有哪些不同的技巧，可以让我们来随意操作浮点数表示中的符号位。举个例子，下面是一个快速求取绝对值的一个例程：

```
float FastAbs(float fval)
{
    int ival;
    _fval_to_ival(fval,ival);
    ival &= 0x7FFFFFFF; // 去掉符号位
    _ival_to_fval(ival,fval);
    return fval;
}   // FastAbs
```

另外一个技巧是，在对一个值进行求反操作时，我们可以将这个值的整数表示与 0x8000000 进行异或（XOR）操作，这样就可以将其符号位反转过来。我们还可以利用下列

Download at www.Pin5i.Com

语句，快速地查询一个浮点值各个方面的信息：

```
#define FI(f) (*((int *) &(f)))   //将浮点值（float）转换为 int（整数）

// 将浮点值（float）转换为 unsigned int（无符号整数）

#define LessThanZero(f)
    (FU(f) > 0x80000000U)
#define LessThanOrEqualsZero(f)
    (FI(f) <= 0)
#define IsZero(f)
    ((FI(f)<<1)==0)
#define GreaterThanOrEqualsZero(f)
    (FU(f) <= 0x80000000U)
#define GreaterThanZero(f)
    (FI(f) > 0)
```

对于上述这些例程，我们也可以改成函数版本的，但是调用函数的系统开销会抵消掉这些技巧所带来的性能提升。大家也许注意到了，因为我们定义和使用了若干个宏，你只能以变量的形式来调用它们。你无法去编译这些宏，例如：IsZero（0.0f）。如果你使用的平台架构不能保证“类型双关（type pun）”[①]的话，或者你不喜欢去干扰全局名字空间（global namespace）的话，那么就使用下列的函数版本吧：

```
bool LessThanZero(float fval);
bool IsZero(float fvai);
bool LessThanZero(float fval);
bool LessThanOrEqualsZero(float fval);
bool GreaterThanZero(float fval);
bool GreaterThanOrEqualsZero(float fval);
```

3. 限值操作

还有一系列的非常有用的例程，功能是将浮点值限定在一个指定的范围之内。我们的库中支持下列这些例程：

```
// 将一个浮点值限定在[0，1]范围内
float Clamp01(float fval);
// 将一个浮点值限定在[A，B]范围内（A 必须小于 B）
float ClampAB(float fval, float A, float B);
// 将一个浮点值限定在[0，无穷大]范围内
float ClampNonnegative(float fval);
```

我们下面举一个例子，看看这些例程是怎么用的。我们假设要将一个浮点值限定在[0，1]之间。通常情况下，大家会这么来实现：

```
if (fval < 0.0f) fval = 0.0f;
```

---

① 关于“类型双关”（type pun）的详细内容，可浏览网址 http://msdn.microsoft.com/library/default.asp。或者，你也可以去 http://www.google.com，搜索相关的信息。“类型双关”就是一个内存位置上有两个不同的含义，就像语言中的双关语。

Download at www.Pin5i.Com

```
if (fval > 1.0f) fval = 1.0f;
```

在一个通道化的架构下，分支和比较都会消耗系统性能。使用我们这些基于比特位的技巧，我们可以去掉分支操作，也就可以更快地完成限值操作。我们的办法是利用符号位，制作一个掩码，将被处理的值归零，或者原封不动地保留这个值。下面的代码是将一个浮点值限定为 0：

```
int s;
IntOrFloat val;
val.f = fval;          // 赋值为一个浮点值
s = SIGNMASK(val.i);   // 如果其整数值的符号为是 1，则符号掩码全都是 1
val.i &= ~s;           // 如果为负值，则归为 0
```

接下来我们再将浮点值限定为 1。大家要注意的是，[King01]中有一个类似的例程，但是把符号位弄颠倒了。

```
// 将浮点值限定为 1
val.f = 1.0f - val.f;      // 如果 val>1，则结果为负
s = SIGNMASK(val.i);       // 如果符号位为 1，那么符号掩码全都是 1
val.i &= ~s;               // 如果是负值，则归为 0
val.f = 1.0f - val.f;      // 再转换回去
return val.f;
```

这个方法是否比前面那个比较的方法来得更快些，这取决于很多因素，因此我们也不能说哪个会更快些。另外，在其他浮点计算执行的同时，上面的代码可以利用 x86 系列的整数处理通道。其他的 Clmap*函数都大同小异，在进行基于整数的无分支的限值操作之前，都必须使用浮点值的移位和缩放操作。

4. 类型转换

还有一组非常有用的窍门，可以实现快速的 int 类型到 float 类型，以及 float 类型到 int 类型的转换。大家可以参阅[King01]，其中详细讨论了这些转换操作是如何实现的。其大致的想法就是为一个浮点数加上一个足够大的值，将其整数部分强行挤压到尾数部分；然后利用掩码操作，屏蔽掉无用的指数部分和符号位；最后，对于负数值的处理，要减去一个常量。他在文中提供了下面这两个函数，用于处理在$|fval|<2^{22}=4194304$ 范围之内的整数值。

```
float IntToFloat(int ival)
{
    IntOrFloat val,bias;
    val.i = ival;
    bias.i = ((23+127)<<23)+(1<<22);
    val.i += bias.i;
    val.f -= bias.f;
    return val.f;
}   // IntToFloat

int FloatToInt(float fval)
```

Download at www.Pin5i.Com

```
{
    int ival;
    fval += (1<<23) + (1<<22);      // 将整数部分压入尾数部分
    _fval_to_ival(fval,ival);       // 类型转换
    ival &= (1<<23)-1;              // 屏蔽掉无用的信息
    ival -= (1<<22);                // 处理负数值
    return ival;
// FloatToInt
```

我们这里提供了一个新的版本，将上述函数的处理范围扩展到$|fval|<2^{31}=2147483648$以内的浮点值。它的工作原理与上面那个限制稍多的版本是一样的，但在进行类型转换时[①]，我们使用了 64 位的双精度浮点类型（double），而不是 32 位的浮点类型（float）。

```
// 将 float（浮点）类型转换为一个 long（长整）型，及反向转换
// 适用于|fval|<=2^31-1 的浮点数
long FloatToLong(float fval)
{
    // 向上转型，获得足够的位数
    // 值为 0x59C00000=2^51+2^52
    double temp = fval +
        (((65536.0*65536.0*16.0)+32768.0)*65536.0);
    return (*(long *) &temp) - 0x80000000;
}  // FloatToLong
```

5. 数学函数

我们还能找到其他的什么技巧呢？不知道大家注意到没有，浮点数这种以位（bit）为单位的表示方法，让我们可以很容易地访问尾数值的线性部分，以及指数位中的对数/指数部分。我们可以利用上述的便利，将浮点数的各个位作为索引，来查寻各种数学函数的值，比如 sin 和 cos 函数。[King01]一文中对这方面内容有详尽的介绍，此处不再赘述。

另外一个可能性，就是利用指数位来快速地得到以 2 为底的对数[Anderson05]：

```
int IntLog2(float fval)
{
    // 提取指数部分，去掉符号位，没有偏移量
    assert(fval > 0);
    unsigned int ival;
    _fval_to_ival(fval,ival);
    return (ival>>23) - 127;
} // IntLog2
```

请大家注意，上面这个函数对“非规范化”数是无效的。因为当指数 E=0 时，我们需要扫描尾数，才能得到正确的值。在随书光盘中的示范代码中，我们提供了速度稍慢些的函数 IntLog2Exact。这个函数功能稍多些，可以得到当 E=0 时以 2 为底的对数。

---

① 顺提醒大家，大多数情况下，在代码中使用双精度浮点（double）类型并不会比使用 float（浮点）类型慢太多（PS2 除外，因为 PS2 上的双精度浮点是用软件来实现的）。自己测试一下，你就会相信了。

Download at www.Pin5i.com

在因特网上，大家谈论得最多的（而关于浮点类型转换为整数类型的话题则紧随其后，排在第二位）可能就是计算平方根的快速算法。我们也不甘为人后，在此推出一个通用的版本：

```
float FastSqrt2(float fval)
{
    assert(fval >= 0);
    float retval;
    int ival;
    _fval_to_ival(fval,ival);
    ival -= 0x3f800000;    // 从偏置指数中减去 127
    ival >>= 1;            // 需要进行有符号的移位操作，以保留符号
    ival += 0x3f800000;    // 对新的指数重新进行偏置
    _ival_to_fval(ival,retval);
    return retval;
}  // FastSqrt2
```

正如我们前面所提到的，很多人推荐这个例程。但是，这个例程中有一个讨厌的 bug。关于这个 bug，我没见过有谁提及过。这个 bug 到底是什么呢？你可以尝试去找这么一个浮点值，将其作为参数，这个例程就会返回无用的垃圾数据。你可以在本文结束部分看到这个答案。

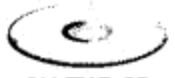
那么，这个例程的工作流程是怎样的呢？其中的关键就是右移位操作，也就是相当于将指数位除以 2，也就有效地获得了这个数量的平方根。我们对压缩的尾数也进行右移位，因为 $\sqrt{1.M} \approx 1.(M/2)$。减去或加上 0x3f800000，就可以从指数中减去或向指数中加上一个量。这样看来，使用移位操作的效果非常不错。而对于 $\sqrt{0}$，这个例程会返回一个值——8.13152e-020，非常小的一个数，但不是 0。而且，如果你想要真正理解浮点值，那就请你去证明一下，利用这个例程所计算出来的结果的最大相对误差，与正确的平方根之间的比不会超过 $\frac{3}{2\sqrt{2}} - 1 \approx 0.06$，也就是约为 6%。下面我们还提供了另外一个例程 InvSqrt。这个例程中采用了牛顿-拉夫逊迭代法（Newton-Raphson），可以提高计算结果的精度，但会损失一些运行速度。

让我们首先来尝试下如何计算平方根的倒数 $1/\sqrt{x}$，然后再去看看我们提供的另外一个计算平方根的实现：

```
float InvSqrt(float x)
{
    assert(x > 0);
    // 详细的工作原理介绍，请参阅[Lomont05]
    float xhalf = 0.5f*x;
    int i;
    _fval_to_ival(x,i);          // 获得浮点值的各个位
    i = 0x5f375a86 - (i>>1);     // 初始猜测 y0，0x5f375a86 就是我们所说的魔法数字
    _ival_to_fval(i,x);          // 再将这些位转换为浮点值
    x = x*(1.5f-xhalf*x*x);      // 牛顿迭代，提高计算结果的精度
    return x;
}  // InvSqrt
```

Download at www.Pin5i.Com

如果要详细地解释这个例程的工作原理，需要占用本文大量的篇幅。简而言之，我们需要的是某个浮点值的-1/2 次方。所以，我们采用移位、减法和一个常数来解决这个问题。这个常数余下的部分，再结合尾数部分，就可以为我们提供一个最好的初始猜测值。关于这个例程的详细介绍，以及我们是如何推导出 0x5f375a86 这个常数的内容，读者可以在线阅读[Lomont05]一文。这篇文章中详细地介绍了这个例程。



随书光盘中提供了另外一个计算平方根的函数。这个函数采用的方法是利用下列公式来计算 x（$x>0$）的平方根：$x \cdot 1/\sqrt{x}$。这个方法不但快，而且比前面那个平方根函数还要精确。另外，这个函数也没有前面那个函数所带有的（却可以修复的）缺陷。对于 $\sqrt{0}$，这个函数甚至可以返回 0。这是因为，对于 0，函数 InvSqrt 会返回一个非常大的有限值（术语应该是“无穷大”），然后再乘以 x（x=0），最后的结果就是 0。

6. 比较

对于一个浮点数各个表示位访问的技巧还可以用在比较操作上。对于那些正在学习浮点计算的程序员来说，他们首先要学会的是，避免用下列代码来进行浮点值的比较：

```
if (fval1 == fval2) // ...
```

由于舍入误差、内部表示误差，或其他的问题，这样的代码通常不会带给我们所预期的结果。于是，很多程序员都换用下列的代码：

```
if (fabs(fval1 - fval2) < tolerance) // ...
```

其中 tolerance 是某个非常小的值。但这个代码也会导致很多难以发现的 bug。特别麻烦的是一旦我们选定了 tolerance 的一个值，它并不会随着参与比较的两个值的大小而自动缩放。（相关内容的探讨请参阅[Knuth97v2]，这是很重要的一个问题）我们真正应该使用的代码应该是这样的：

```
if (fabs(fval1 - fval2) < max(fval1, fval2)*tolerance) ...
```

这个代码比前面那个好一些，tolerance 可以自动缩放。但是，这个版本虽然更精确了，但却比上面那个慢了近 5 倍。我在开发一个光线追踪器（Ray Tracer）时碰到了这个问题，在经过几个小时的思考后，我最终找到了一个解决办法。我的这个方法在本质上与 Bruce Dawson 在[Dawson05]一文中提出的方法是相同的：

```
bool DawsonCompare(float af, float bf, int maxDiff)
{
    int ai,bi;
    _fval_to_ival(af,ai);       // 获得各个表示位
    _fval_to_ival(bf,bi)
    if (ai < 0)
        ai = 0x80000000 - ai;   // 正序
    if (bi < 0)
        bi = 0x80000000 - bi;   // 正序
```

Download at www.Pin5i.com

```
    int diff = ai - bi;          // 计算二者之间的差
    if (abs(diff) < maxDiff)     // 是否足够接近?
        return true;
        return false;
}
```

现在，两个数之间的比较就可以这样来完成：

```
if (true == DawsonCompare(fval, aval, 1000)) // ...
```

其中 1000 代替了 tolerance（公差），在整个浮点范围内可以很好地进行缩放。我们这个时候其实是在比较任意两个浮点值是否“相等”。我们将这两个浮点值的表示位看成是 int（整数）类型，它们之间的差在 1000 以内，在整个数值范围内可以缩放自如。[①]这个方法比现有的其他方法都要快！但是，这并不是我们优化工作的结束。

这个方法的工作原理是什么呢？这个点子是我在观察了浮点数值轴后，从我们所讨论的顺序问题中获得的。如果两个值都是正值，只要看它们作为整数是否很接近。如果其中一个，或者两个值都是负数，则需要进行一些位的操作，来校正顺序。更详细的内容请大家参阅[Dawson05]。

为了去掉分支，我们只要采用一些巧妙的位操作，就可以获得下列这个更快的代码：

```
bool LomontCompare(float af, float bf, int maxDiff)
{
    // 更快速的例程，可在各个编译器之间进行移植
    // 运行时间常量，与具体参数无关
    int ai;
    int bi;
    _fval_to_ival(af,ai);
    _fval_to_ival(bf,bi);
    int test = SIGNMASK(ai^bi);
    assert((0 == test) || (0xFFFFFFFF == test));
    int diff =
        (((0x80000000 - ai)&(test)) | (ai & (~test))) - bi;
    int v1 = maxDiff + diff;
    int v2 = maxDiff - diff;
    return (v1|v2) >= 0;
}
    // LomontCompare
```

[Lomont05]一文中详细地介绍了这个例程（以及其他很多变种）的工作原理。最终结果是这个例程的功能与 DawsonCompare 一样，但在所有测试的机器上[②]，我们这个例程要更快一些。它比 KnuthCompare 要快很多，而且在一个很大范围数值空间中，这个算法都很稳定。现在，这个算法已经应用在一个实时的 CSG（Constructive Solid Geometry，构造性实体几何）光线追踪引擎中。该引擎目前仍在开发之中。

[①] 1000 这个值只是一个大致的估计。它取决于你要进行比较的两个值是如何生成的。你应该针对你自己的例程去做些实验，找到最适合你要求的值。

[②] 我们在现有的，及老版本的几个 AMD 和 Intel PC 上进行了测试，但并没有在游戏机上进行测试。如果你搜集到了相关的数据，请发邮件给我。

Download at www.Pin5i.com

7. 未来的方向

上面介绍的这些函数只是一些非常浅显的应用。还有一些其他方面的应用技巧，我们作为练习题留给大家，包括如下内容：

快速内联交换：既然我们可以把一个浮点数看成是一个整数。你可以利用两个浮点数（af 和 bf）的整数表示（ai 和 ab）来进行交换。不需要临时变量，我们使用的是 XOR（异或）操作：

ai^=bi；bi^=ai；ai^=bi；

将表示位用作一个表的索引，实现快速的表查找函数：这个方法可以很好地适用于 sin、cos，以及其他很多函数。[King01]也涉及了这方面的内容。

通过巧妙地修改指数部分，实现快速立方根和其他方根的求解：随后也可以实现快速的尾数近似求解。

通过对指数部分的操作，实现快速的乘以或除以 2 的 n 次幂的操作：就像可以通过移位操作来实现整数乘 2 的运算一样，你也可以通过对指数位的加减，实现对浮点值进行移位操作，完成除法或乘法的运算。这和我们在求解平方根时的方法类似。

利用表示位来选择 bin（桶），快速基数排序：这样我们就可以得到一个线性时间排序算法。但是请大家注意，对于负的浮点数，它们是逆序排列的。所以你需要使用一个类似于我们在函数 LomontCompare 中所使用的技巧。你不能对一个浮点值进行模板的实例化，但是你可以对一个整数值进行这个操作。这样一来，你就可以创建一个模板，让它在编译时，为某些常量或其他工作来执行浮点的计算。参见[Rosten05]，它提供了一个库，就是实现这个功能的。

利用浮点寄存器每次移动 64 或更多的位，实现快速内存拷贝（memcpy）。

我们这个技巧列表只是一个开始。还有很多的应用技巧，需要我们去发掘，并不断地添加进来。

2.1.3　总结

在本文中，你已经看到了一个浮点值是如何存储的。而且，现在你也意识到，这种存储表示方式的无限应用潜力，利用这些知识，我们可以将其作为最后的王牌，来创建快速（但却愚钝的）代码。为了达到下一个境界，获得快速而不蠢钝的代码，我们需要有能力去分析相关的例程，去验证这些例程所执行的数学运算的结果，进行最坏情况和一般情况下的误差分析，并在目标平台上进行详细的性能测试。这些任务颇为艰巨，而且单调乏味。作为一个例子，可参阅[Lomont05]一文中对例程 InvSqrt 进行的详细的误差分析。

最后，为了奖励那些苦读到此的读者朋友，对于在文章中所提出的问题，我们将它们令人惊奇的答案罗列于此：

对于一个浮点值 v，你能假设 v==v 一定能返回 ture 吗？不，不能！对于 NaN（非数值），这个代码是无效的。一般情况下，我们不应该在自己的例程中引入 NaN（非数值）。但如果无法避免，我们就不应该再依赖这个假设了。

对于两个浮点值 v1 和 v2，下列语句中，至少有一个是真的（true）：v1>v2；v1==v2；和 v1<v2？答案还是：否！如果有一个操作数是 NaN，那么所有的假设都是不成立的。

再来看看例程 FastSqrt2 中那个神秘的 bug。利用你在本文中所学到的知识，利用 FastSqrt2

Download at www.Pin5i.Com

来计算—0 的平方根。其结果居然是 2.76701e+019，这是一个非常大的值！为了修改这个 bug，紧接在_fval_to_ival 语句后面，插入下列语句，以便在移位操作之前，屏蔽掉符号位：

```
ival &= 0x7FFFFFFF
```

如果还想了解更多的有关浮点数的知识，还可以参阅其他相关的文章，包括[Hecker96]、[Hsieh04]，以及这本很棒的书——[Warren02]。

最后提醒大家，本文这些应用技巧的代码实现都包含在随书光盘中。本文相关的网站[①]也会提供这些代码，以及后续的改动和最新的版本。我们也在不断地搜集各种函数运行时间的信息，随时会将这些信息发布在网站上。

2.1.4 参考文献

[754Ref05a] Vickery, Christopher, "IEEE-754 References." Available online at *http://babbage. cs. qc .edu/ courses/cs341/ IEEE-754references.html.*

[754Ref05b] "IEEE Arithmetic." Available online at *http://cch.loria.fr/documentation/IEEE754/numerical_comp_guide/ncg_math.doc.html.*

[Anderson05] Anderson, Sean, "Bit Twiddling Hacks." Available online at *http:// graphics .stanford.edu/~seander/bithacks.html.*

[Dawson05] Dawson, Bruce, "Comparing Floating-Point Numbers." Available online at *http://www.cygnus-software.com/papers/comparingfloats/comparingfloats.htm.*

[Dawson05b] Dawson, Bruce, "x86 Processors and Infinity." Available online at *http://www.cygnus-software.com/papers/x86andinfinity.html.*

[Hecker96] Hecker, Chris, "Let's Get to the (Floating) Point." *Game Developer Magazine,* February/March 1996. Available online at *http://www.d6.com/users/checker/pdfs/gdmfp.pdf.*

[Hsieh04] Hsieh, Paul, "Programming Optimization." Available online at *http:// www. azillionmonkeys.com/qed/optimize.html.*

[Kahan98] Kahan, William, "An Interview with the Old Man of Floating-Point," 1998. Available online at *http://www.cs.berkeley.edu/~wkahan/ieee754status/754story.html.*

[King01] King, Yossarian, "Improving Performance with IEEE Floating-Point." *Game Programming Gems 2,* Charles River Media, 2001.

[Knuth97v2] Knuth, Donald, *The Art of Computer Programming: Volume 2, Seminumerical Algorithms,* 3rd Ed. Reading, Massachusetts, Addison-Wesley, 1997.

[Lomont05] Lomont, Chris, "Taming the Floating-Point Beast," 2005. Available online at *http://www.lomont.org/Math/Papers/2005/CompareFloat.pdf.*

[Rosten05] Rosten, Edward, "Floating point arithmetic in C++ templates," 2005. Available online at *http://mi.eng.cam.ac.uk/~er258/code/fp_template.html.*

[Warren02] Warren, Henry S., Jr., *Hacker's Delight.* Addison-Wesley, July 2002.

[①] 可浏览 http://www.lomont.org/software，或者在 Google 上搜索"Chris Lomont"，查找作者和相关的代码。

Download at www.Pin5i.Com

2.2　利用齐次坐标实现投影空间中的 GPU 计算

Vaclav Skala，捷克西波希米亚大学
skala@kiv.zcu.cz

在计算机图形学领域中，人们经常利用线性代数来求解给定的问题，这种例子比比皆是。他们求解的问题涉及很多方面，包括：计算两个点之间的直线，两条直线的交点，或者是直线的修剪。在某些情况下，直接使用现成的数学公式是比较困难的。特别值得一提的是，从计算稳定性（computational stability）的角度来说，类似于 $a==b$ 这样的检测，其稳定性是非常值得怀疑的。由于计算的精度有限，即使是非常简单的两条直线之间的共线性（collinearity）测试，也不是非常的可靠。

除了这些问题，还有一个因素需要我们去认真考虑一下。在计算机图形学应用领域中，我们通常会在齐次坐标中，也就是在真正的投影空间中（参见[Ferguson01]、[Hill01]和[Shirley02]）来表示一个点（point）。用户和程序人员常常使用除法操作，将齐次坐标中的点转换为欧几里德（Euclidean）坐标。相对于其他的浮点运算而言，这种除法操作不但非常耗时，而且还可能会造成严重的数值不稳定性（numerical instability），或者是致命的除以 0 的异常。

在这篇文章中，我们会向大家介绍一种新的方法，直接使用齐次坐标来进行常见的交叉点的计算。这个方法可以为我们带来更高的数值稳定性。而且在某些情况下，还可以明显地提高运行效率。如果主处理器支持向量和矩阵运算，或者是在 GPU（Graphics Processing Unit，图形处理器）上的应用程序中，这个方法更是十分的方便。

2.2.1　相关的数学背景知识

为了让你能够更好地理解本文后面的内容，我们首先来回顾一些必要的数学知识。

1. 齐次坐标

假设我们在 E^2 空间（或者说是二维空间）中有一个点，用欧几里德（Euclidean）坐标表示为 $X=(X, Y)$。大家都知道，对于这同一个点，我们可以在三维齐次坐标中将其表示为 $x=[x, y, w]^T$。其中：

$$x=wX \quad y=wY, w\neq 0 \tag{2.2.1}$$

Download at www.Pin5i.Com

w 的一个常见的值就是 1，所以点 X 又常常被表示为 $x=[x, y, 1]^T$。请大家注意，由于 w 的值有无限种可能性，我们可以用齐次坐标中无限数量的点来表示点 X。另外，如果 $w=0$，那么上述公式所表示的实体就不是一个点了，而是一个向量，也就是一个无限远的点。

在齐次空间中，给定一个点 $x=[wX, wY, w]^T$，其对应的欧几里德（Euclidean）坐标（X, Y）可以计算如下：

$$X=x/w \quad Y=y/w,\ w\neq 0 \tag{2.2.2}$$

对于 E^3 空间（或者说是三维空间）中的点，它们的齐次坐标表示也可以采用类似的方法计算出来。

2. 行列式运算

矩阵 **A** 的行列式可以被递归地定义如下：

$$\det(\mathbf{A})=|\mathbf{A}|=\sum_{j=1}^{n} a_{ij}(-1)^{(i+j)}\det(\tilde{A}_{ij}) \tag{2.2.3}$$

其中，$\tilde{A}_{ij}$ 是从矩阵 **A** 中拿掉第 i 行和第 j 列后所形成的子矩阵。对于一个 3×3 的矩阵，我们可以计算如下：

$$\begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = a(ei-fh)-b(di-fg)+c(dh-eg) \tag{2.2.4}$$

我们还会用到行列式另外两个特性。第一个特性是：用一个系数 q 乘以行列式的某一个行，等于用这个系数乘以该行列式，其中 $q\neq 0$。这个特性可以表示为：

$$\begin{vmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & & \vdots \\ qa_{k1} & qa_{k2} & \cdots & qa_{kn} \\ \vdots & & \vdots \\ a_{n1} & \cdots & a_{nn} \end{vmatrix} = q \cdot \begin{vmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & & \vdots \\ a_{k1} & a_{k2} & \cdots & a_{kn} \\ \vdots & & \vdots \\ a_{n1} & \cdots & a_{nn} \end{vmatrix} \tag{2.2.5}$$

第二个特性则是：如果行列式为 0，那么这个行列式矩阵的这些行或列是线性相关的。也就是说，把某一行（或某一列）的倍数（也就是乘以一个系数）加到另外一行（或一列）上，行列式不变。这样的矩阵是不可逆的。同样的，如果这个矩阵表示的是一个线性方程组，这样的方程组是无解的。

3. 向量运算

向量 $\boldsymbol{a}=[a_1, a_2, a_3]^T$ 和向量 $\boldsymbol{b}=[b_1, b_2, b_3]^T$ 的向量积（点乘积）可以定义如下：

$$\mathbf{a}^T\mathbf{b}=\mathbf{b}^T\mathbf{a}=\mathbf{a}\cdot\mathbf{b}=\|\mathbf{a}\|\|\mathbf{b}\|\cos\varphi \tag{2.2.6}$$

其中，φ 是向量 **a** 和向量 **b** 之间的夹角。

向量 $\boldsymbol{a}=[a_1, a_2, a_3]^T$ 和向量 $\boldsymbol{b}=[b_1, b_2, b_3]^T$ 的叉乘积（cross product）可以定义如下：

$$\mathbf{a}\times\mathbf{b}=\begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix} \tag{2.2.7}$$

Download at www.Pin5i.Com

其中，$\mathbf{i}=[1,\ 0,\ 0]^T$，$\mathbf{j}=[0,\ 1,\ 0]^T$，$\mathbf{k}=[0,\ 0,\ 1]^T$。

根据前面的公式 2.2.4，这个结果就等于：

$$\mathbf{a}\times\mathbf{b}=(a_2b_3-a_3b_2)\mathbf{i}-(a_1b_3-a_3b_1)\mathbf{j}+(a_1b_2-a_2b_1)\mathbf{k} \tag{2.2.8}$$

这样我们就得到了一个新的向量，它垂直于向量 a 和向量 b。请大家注意，这只是为三维空间中的向量定义的公式，而且 $\mathbf{a}\times\mathbf{b}=-\mathbf{b}\times\mathbf{a}$。

对于四维空间中的向量，我们可以引入一个叉乘积，作为另外一个行列式：

$$\mathbf{a}\times\mathbf{b}\times\mathbf{c}=\begin{vmatrix}\mathbf{i} & \mathbf{j} & \mathbf{k} & \mathbf{1}\\ a_1 & a_2 & a_3 & a_4\\ b_1 & b_2 & b_3 & b_4\\ c_1 & c_2 & c_3 & c_4\end{vmatrix} \tag{2.2.9}$$

其中，$\mathbf{i}=[1,\ 0,\ 0,\ 0]^T$，$\mathbf{j}=[0,\ 1,\ 0,\ 0]^T$，$\mathbf{k}=[0,\ 0,\ 1,\ 0]^T$，$\mathbf{l}=[0,\ 0,\ 0,\ 1]^T$。

与三维空间的情况类似，这个公式也生成一个垂直于向量 a、b、c 的新向量。我们会在后面用到这个定义。

4. 对偶原理

对偶原理（Principle of Duality）指出，如果我们调换任意一个定理中的一对字词，这个定理依然成立。举个例子，在 E^2 空间中，“点”和“线”就是对偶元素，还有“通过”和“位于”，以及“相交”和“相连”等都是对偶元素。同样地，对于三维空间，一个“点”和“平面”是对偶元素。

那么，这个原理在计算机图形学领域意味着什么呢？我们都知道，在 E^2 空间中，一条直线 p 可以被定义为：

$$aX+bY+c=0 \tag{2.2.10}$$

根据这个公式，我们可以在 E^3 空间中规定一个向量 $\mathbf{a}=[a,\ b,\ c]^T$，来定义直线 p。另外，公式 2.2.10 可以乘以任意一个非 0 的 w 值，在几何学上，得到的仍然是同一条直线。所以，直线 p 实际上和齐次空间中的一个点是对偶元素(这正是我们所希望的)，而且在对偶空间中，它们都由一个向量 **a** 来表示。能够理解这个概念非常重要，因为我们后面要利用对偶原理来推导出一些有用的公式。

2.2.2　利用齐次坐标进行计算

现在，我们会向大家介绍如何利用齐次坐标，来求解一些简单的线性系统。我们首先从 E^2 空间的情况入手，然后，在将这些方法扩展到 E^3 空间。

1. E^2 空间的情况

让我们来考虑 E^2 空间中的两条直线 p_1 和 p_2：

$$p_1:\ a_1X+b_1Y+c_1=0 \text{ 和 } p_2:\ a_2X+b_2Y+c_2=0 \tag{2.2.11}$$

如果想得到这两条直线的交点，我们就可以求解下面这个非齐次方程的一个线性系统，也就是 X、Y 的值：

Download at www.Pin5i.com

$$\begin{bmatrix} a_1 & b_1 \\ a_2 & b_2 \end{bmatrix}\begin{bmatrix} X \\ Y \end{bmatrix}=\begin{bmatrix} -c_1 \\ -c_2 \end{bmatrix} \tag{2.2.12}$$

上面这个方程，大家都知道，其相应的求解如下：

$$X=\frac{D_x}{D} \qquad Y=\frac{D_y}{D} \tag{2.2.13}$$

其中，$D_x=\det\begin{vmatrix} -c_1 & b_1 \\ -c_2 & b_2 \end{vmatrix}$ $D_y=-\det\begin{vmatrix} a_1 & -c_1 \\ a_2 & -c_2 \end{vmatrix}$ $D=\det\begin{vmatrix} -a_1 & b_1 \\ -a_2 & b_2 \end{vmatrix}$

但是，上面的求解公式中都需要除以一个“D”值。这样的话，当 D 趋近于 0 时，求解的结果就会在数值上变得不稳定。所以，我们换用齐次坐标，将公式 2.2.12 改写成如下形式：

$$\begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \end{bmatrix}\begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}=\begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{2.2.14}$$

所以呢，我们也可以将公式 2.2.11 也用齐次坐标来表示：

$$\mathrm{x}=[x, y, w]^T=[D_x, D_y, D]^T \tag{2.2.15}$$

然后，再根据公式 2.2.2，我们最后就可以计算出来 X 和 Y 的值：

$$X=\frac{x}{w}=\frac{D_x}{D} \qquad Y=\frac{y}{w}=\frac{D_y}{D} \qquad D\neq 0, \tag{2.2.16}$$

这个结果与我们公式 2.2.13 是匹配的。如果 D=0，那么这个两条直线就是平行的，或者就是同一条直线。

我们再来审视一下 D_x、D_y 和 D 的计算公式，对于齐次交叉点 x，它的计算可以看成是两个向量 $\mathbf{a}_1$ 和 $\mathbf{a}_2$ 的叉乘积：

$$\mathbf{x}=\mathbf{a}_1\times\mathbf{a}_2=\det\begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \end{vmatrix} \tag{2.2.17}$$

其中，$\mathbf{i}=[1,0,0]^T$，$\mathbf{j}=[0,1,0]^T$，$\mathbf{k}=[0,0,1]^T$

现在，让我们来看看对偶问题。正如我们前面提到的，根据 E^2 状态中对偶原理，一条直线对偶于一个点，反之亦然。而对于我们现在这种情况，对偶问题就是，如何根据两个给定的点，来确定一条直线。有一个方法可以解决这个问题，就是利用下面这个线性方程组：

$$\begin{bmatrix} X_1 & Y_1 & 1 \\ X_2 & Y_2 & 1 \end{bmatrix}\begin{bmatrix} a \\ b \\ c \end{bmatrix}=\begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{2.2.18}$$

其中，我们给定的两个点是 $\boldsymbol{x}_i=[X_i, Y_i, 1]^T$，而直线 p 则由向量 $\mathbf{a}=[a, b, c]^T$。作为另外一种替代方案，我们也可以使用下列公式：

$$\mathbf{a}=\mathbf{x}_1\times\mathbf{x}_2=\begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ X_1 & Y_1 & 1 \\ X_2 & Y_2 & 1 \end{vmatrix} \tag{2.2.19}$$

和求解两条直线的相交一样，这就意味着，由给定的两个点所确定的那条直线可以通过叉乘积来确定。

Download at www.Pin5i.Com

但是，假设给定的两个点是用齐次坐标表示的，而且 w 的值并不等于 1，也就是 $\boldsymbol{x}_i=[x_i, y_i, w_i]^T$，这又该如何呢？正常情况下，我们可以利用除法操作，将齐次坐标转换为欧几里德（Euclidean）坐标，然后再代入公式 2.2.18 中。这里，我们采用另外一种方法。请大家注意，如果使用齐次坐标，我们将公式 2.2.19 中行列式的第一行乘以一个 $w_1\neq 0$，再将其第二行乘以一个 $w_2\neq 0$。根据公式 2.2.5 所定义的恒等式，我们最后可以得到公式 2.2.20：

$$\mathbf{x}_1\times\mathbf{x}_2=\begin{vmatrix}\mathbf{i} & \mathbf{j} & \mathbf{k}\\ x_1 & y_1 & w_1\\ x_2 & y_2 & w_2\end{vmatrix}=\begin{vmatrix}\mathbf{i} & \mathbf{j} & \mathbf{k}\\ w_1X_1 & w_1Y_1 & w_1\\ w_2X_2 & w_2Y_2 & w_2\end{vmatrix}=w_1w_2\begin{vmatrix}\mathbf{i} & \mathbf{j} & \mathbf{k}\\ X_1 & Y_1 & 1\\ X_2 & Y_2 & 1\end{vmatrix}=w_1w_2\mathbf{a} \qquad (2.2.20)$$

最后的结果就是，我们将原来的向量 $\boldsymbol{a}$ 缩放了 w_1w_2 倍。但是，正如我们前面所说的，如果 $w_1w_2\neq 0$，那么由 $w_1w_2\boldsymbol{a}$ 所表示的直线与向量 $\boldsymbol{a}$ 所表示的向量，在几何学上是同一条直线。所以，通过使用公式 2.2.20，我们利用两个给定点的齐次坐标，就可以直接确定直线 p，而不需要进行除法运算。

2. E^3 空间的情况

将上述方法扩展到 E^3 空间的情况也非常简单。在 E^3 空间中，一个点对偶于一个平面，反之亦然。所以我们得到如下结论：

- 三个彼此不共面的平面，它们的交叉点 $\mathbf{x}$ 可以用下列公式计算：

$$\mathbf{x}=\mathbf{a}_1\times\mathbf{a}_2\times\mathbf{a}_3=\begin{vmatrix}\mathbf{i} & \mathbf{j} & \mathbf{k} & \mathbf{1}\\ a_1 & b_1 & c_1 & d_1\\ a_2 & b_2 & c_2 & d_2\\ a_3 & b_3 & c_3 & d_3\end{vmatrix}, \qquad (2.2.21)$$

其中，$\mathbf{a}_i=[a_i, b_i, c_i, d_i]^T$ 表示的就是一个平面 $\rho_i=a_iX+b_iY+c_iZ+d_i$，$i=1, \cdots, 3$。而 $\mathbf{x}=[x, y, z, w]^T$ 就是交叉点的齐次坐标。$X=x/w$、$Y=y/w$ 和 $Z=z/w$ 则是欧几里德坐标。

- 和 E^2 空间中的情况一样，在齐次除法运算之前，我们可以首先求得向量 $\mathbf{x}$ 的 w 坐标，然后再分析奇异（表示三个平面共面）和接近奇异（表示三个平面接近共面）的情况。
- 对于由三个不同的点（或者说是不重合的点）所确定的平面，可以用下列公式来计算：

$$\mathbf{a}=\mathbf{x}_1\times\mathbf{x}_2\times\mathbf{x}_3=\begin{vmatrix}\mathbf{i} & \mathbf{j} & \mathbf{k} & \mathbf{1}\\ x_1 & y_1 & z_1 & w_1\\ x_2 & y_2 & z_2 & w_2\\ x_3 & y_3 & z_3 & w_3\end{vmatrix}, \qquad (2.2.22)$$

其中，$\mathbf{x}_i=[x_i, y_i, z_i, w_i]^T$ 表示的是给定的点 $\mathbf{x}_i$，且 i=1，…，3。$\mathbf{a}=[a, b, c, d]^T$ 表示的是由这三个点所确定的平面 ρ，也就是：$\rho=aX+bY+cZ+d$。

- 我们可以求得向量 $\mathbf{a}$ 的齐次坐标 w，然后再分析奇异和接近奇异的情况。如果 $\mathbf{a}=[0, 0, 0, 0]^T$，则这三个给定的点并不是彼此不同的。

我们这里考虑所有的 4 种情况，这就是说，我们得到了一个非常可靠，而且简单的方法，可以来确定：

Download at www.Pin5i.com

- 在 E^2 空间中，以齐次坐标给定的两个点所确定的直线；
- 在 E^2 空间中两条直线的交点；
- 在 E^3 空间中，以齐次坐标给定的三个点所确定的平面；
- 在 E^3 空间中三个平面的交点。

如果主硬件系统支持向量/矩阵运算，或者使用 GPU，那么，我们这里提出的这个方法将会更富效率。在本文的附录 A 中，我们提供了一个在高级着色器编程语言中常用的 4D 叉乘积的实现。

2.2.3 直线交叉

业界已经开发出了很多的算法，来确定一条直线，或者是一条射线与一个物体的交叉点。在某些应用中，直线是以参数的形式出现的，而物体则是一个被隐式描述的几何体（例如：一条直线，或一个平面）。对于这种情况，有一个典型的例子就是 CyrusBeck（CB）算法[Cyrus78]。该算法主要是解决 E^2 空间中凸多边形的直线修剪问题，以及 E^3 空间中凸多面体的直线修剪问题。E^2 空间的情况比较简单，但是 E^3 空间的情况就需要一些小技巧。有一种方法就是用普吕克（Plucker）坐标来定义 E^3 空间中的直线（参见[Blinn77]）。但是，我们下面提供了一种更为简单的方法。

线面相交

由于 E^3 空间中的情况比较复杂，我们下面将向大家详细地进行介绍。而在此基础上，再修改出适合 E^2 空间的公式就相对容易，留给读者去完成。

我们以参数形式定义 E^3 空间中的一条直线：$\mathbf{X}(t)=\mathbf{X}_A+(\mathbf{X}_B-\mathbf{X}_A)t$。其中，$\mathbf{X}_A=[X_A, Y_A, Z_A]^T$ 和 $\mathbf{X}_B=[X_B, Y_B, Z_B]^T$，是以欧几里德坐标给出的点。和前面一样，我们再定义一个平面ρ：$aX+bY+cZ+d=0$。其中，向量 $\mathbf{a}=[a, b, c, d]^T$ 表示的是一个给定的平面。我们可以将平面ρ用另外一个稍有不同的形式表示出来：$\alpha^T X+d=0$。其中，$\alpha=[a, b, c]^T$，是这个给定平面的法线向量。

大家都知道，这样一个线面相交公式的参数 t 可以这样计算得到：

$$t=-\frac{\alpha^T \mathbf{X}_A+d}{\alpha^T(\mathbf{X}_B-\mathbf{X}_A)} \tag{2.2.23}$$

和我们前面求解交叉点的推导方法一样，我们也可以用齐次坐标将参数 t 表示为：$[\tau, \tau_w]$。其中：

$$\tau=-(\alpha^T \mathbf{X}_A+d) \tag{2.2.24}$$

而 $$\tau_w=\alpha^T(\mathbf{X}_B-\mathbf{X}_A)$$

通过上述步骤，我们并不会立即就需要去使用除法运算。在修改的算法中，除法运算被推后了。

这个表示方法直接带来了一个新的问题：如果定义直线的点是以齐次坐标形式给出的，那会发生什么情况呢？通常情况下，程序人员会使用除法运算，将给定的点转换为欧几里德坐标。但是，如果我们仍然将这些点保持为齐次坐标形式，我们可以将公式 2.2.23 改写成公式 2.2.25：

Download at www.Pin5i.com

$$t=-\frac{\alpha^T X_A+d}{\alpha^T(X_B-X_A)}=-\frac{\alpha^T\frac{\zeta_A}{w_A}+d}{\alpha^T(\frac{\zeta_B}{w_B}-\frac{\zeta_A}{w_A})}, \tag{2.2.25}$$

其中，X_A、X_B、α_T，以及 d 和前文一样。而 $\mathbf{X}_A=[x_A, y_A, z_A, w_A]^T$ 和 $\mathbf{X}_B=[x_B, y_B, z_B, w_B]^T$ 是以齐次坐标给定的点。$\xi_A=[x_A, y_A, z_A]^T$ 和 $\xi_B=[x_B, y_B, z_B]^T$。

将公式 2.2.25 等号右侧分式的分子分母同乘以一个 $w_A\neq0$ 和 $w_B\neq0$，我们就得到了公式 2.2.26：

$$t=-\frac{w_B(\alpha^T\zeta_A+w_Ad)}{\alpha^T(w_A\zeta_B-w_B\zeta_A)} \tag{2.2.26}$$

这样，对于直线上的点是用齐次坐标给出的情况，我们可以将公式 2.2.24 改写成公式 2.2.27：

$$\tau=-w_B(\alpha^T\zeta_A+w_Ad)=-w_B\alpha^T\mathbf{x}_A \tag{2.2.27}$$

和　　$\tau_w=\alpha^T(w_A\xi_B-w_B\xi_A)$。

如果使用我们提出的齐次坐标表示，并且对最基本的算术运算（包括比较运算）进行了优化，我们就可以获得一个更为可靠的算法。由于这个算法不需要执行除法运算，我们还可以获得 CPU 速度的提升。

在最近的研究中，本文提出的这个方法被应用到 CB 算法中。在 E^2 空间中，对于 $N\geqslant6$ 的情况，这个改进后的 CB 算法要更快一些。其中 N 是给定凸多边形的边数。对于 $N\geqslant500$ 的情况，改进的 CB 算法要比原来优化过的 CB 算法快了近 20%。这就意味着，对于 E^3 空间，算法性能还会提高，因此我们可以去检测更多的多面体的面。

2.2.4　总结

在本文中，我们向大家介绍了一个新的方法，使用齐次坐标来完成一些常见的计算工作。对于那些特别容易在 GPU 上实现的算法，这个方法对它们有着非常直接的影响。我们这个方法主要的创新在于：它提高了算法的可靠性，将除法运算延迟到最后时刻再执行，如果系统支持，我们还可以直接利用齐次坐标来进行相关的计算。本文介绍的这个方法已经应用在一些直线和线段的修剪算法中，以更高的可靠性，为我们带来了更简单、更快速的解决方案[Skala04]、[Skala05]。

我们介绍的这个方法，其主要的先进性在于：

- 图形几何体通常都可自然地以齐次坐标来表示；
- 如果计算结果能够保持齐次坐标的形式，那么我们就有可能完全摒弃除法运算；
- 如果硬件系统支持向量/矩阵运算，我们的方法可以获得明显的速度上的提升；
- 为算法实现了简单、紧凑的代码；
- 在现在的 GPU 产品上实现起来很简单（参见附录 A 和附录 B）；

我们希望这个方法可以为我们带来新的算法设计和新的计算模型。

2.2.5　附录 A

4D 空间中的叉乘积定义如下。

Download at www.Pin5i.Com

$$\mathbf{x}_1 \times \mathbf{x}_2 \times \mathbf{x}_3 = \det \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} & \mathbf{1} \\ x_1 & y_1 & z_1 & w_1 \\ x_2 & y_2 & z_2 & w_2 \\ x_3 & y_3 & z_3 & w_3 \end{vmatrix} \tag{2.2.A1}$$

我们可以在 GPU 上用 Cg/HLSL 语言实现这个计算，代码如下：

```
float4 cross_4D(float4 x1, float4 x2, float4 x3)
{
    float4 a;

    a.x=dot(x1.yzw, cross(x2.yzw, x3.yzw));
    a.y=-dot(x1.xzw, cross(x2.xzw, x3.xzw));
    // or a.y=dot(x1.xzw, cross(x3.xzw, x2.xzw));
    a.z=dot(x1.xyw, cross(x2.xyw, x3.xyw));
    a.w=-dot(x1.xyz, cross(x2.xyz, x3.xyz));
    // or a.w=dot(x1.xyz, cross(x3.xyz, x2.xyz));

    return a;
}
```

或者，我们可以采用这个更为紧凑的代码实现：

```
float4 cross_4D(float4 x1, float4 x2, float4 x3)
{
    return ( dot(x1.yzw, cross(x2.yzw, x3.yzw)),
            -dot(x1.xzw, cross(x2.xzw, x3.xzw)),
             dot(x1.xyw, cross(x2.xyw, x3.xyw)),
            -dot(x1.xyz, cross(x2.xyz, x3.xyz)) );
}
```

这个代码是比较简单的，采用了现在 GPU 产品上提供的向量操作。而 Cg/HLSL 语言直接支持 E^3 空间的叉乘积计算。

2.2.6 附录 B

直线与平面交叉点的参数形式的计算公式如下：

$$t = -\frac{w_B(\alpha^T \zeta_A + w_A d)}{\alpha^T (w_A \zeta_B - w_B \zeta_A)} \tag{2.2.B1}$$

我们可以将它用齐次坐标的形式表示出来：

$$\tau = -w_B(\alpha^T \zeta_A + w_A d) = -w_B \alpha^T x_A \qquad \tau_w = \alpha^T (w_A \xi_B - w_B \xi_A) \tag{2.2.B2}$$

我们可以利用 Cg/HLSL 语言，在 GPU 上很容易地实现公式 2.2.B2，代码如下：

```
float4 intersection_3D(float4 a, float4 xa, float4 xb)
{
    float4 t;
    t.x=-xb.w*(dot(a.xyz,xa.xyz)+xa.w*a.w);
```

Download at www.Pin5i.Com

```
    t.w=dot(a.xyz,xa.w*xb.xyz-xb.w*xa.xyz);
    return t;
}
```

我们可以看到，本文介绍的这个方法很大程度上得益于向量/向量运算。

2.2.7　致谢

在此，笔者向捷克西波希米亚大学的学生和同行们表示衷心的感谢。感谢他们热情的推荐、建设性的谈论和积极的建议，帮助我来完成这篇文章。我特别要感谢 Ivo Hanak，他指点我去如何评估硬件特性。还要感谢 Martin Janda，他帮助我完成了对 Cyrus Beck 算法实验性的验证工作。还有就是 Libor Vasa 和 Petr Lobaz，他们对本文提出了很关键的意见。

这项研究工作得到了捷克国家教育部、MSM 项目 235200005、微软研究院（英国）项目号 No.2004-360，以及 ATI 公司的赞助支持。

2.2.8　参考文献

[Blinn77] Blinn, J. F., "Homogeneous Formulation for Lines in 3 Space." *Computer Graphics,* Vol. 11, No. 2, SIGGRAPH 77: pp. 237–241.

[Cyrus78] Cyrus, M. and J. Beck, "Generalized Two and Three-Dimensional Clipping." *Computers & Graphics*, Vol. 2, No. 1, pp. 23–28, 1978.

[Ferguson01] Ferguson, Stuart R., *Practical Algorithms for 3D Computer Graphics.* A. K. Peters, 2001.

[Hill01] Hill, Francis S., *Computer Graphics using OpenGL.* Prentice Hall, 2001.

[Shirley02] Shirley, Petr, *Fundamentals of Computer Graphics.* A. K. Peters, 2002.

[Skala04] Skala, Vaclav, "A New Line Clipping Algorithm with Hardware Acceleration." *cgi*, Computer Graphics International 2004: pp. 270–273.

[Skala05] Skala, Vaclav, "A new approach to line and line segment clipping in homogeneous coordinates." *The Visual Computer*, Vol. 21, No. 11, pp. 906–914, Springer Verlag, 2005.

Download at www.Pin5i.Com

2.3 利用叉乘积求解线性方程组

Anders Hast，瑞典耶夫勒大学创意媒体实验室
aht@hig.se

在计算机图形学领域中，叉乘积[Nicholson95]最常见的用途就是用来计算一个多边形的法线[Hearn04]，或者就是用来计算一个多边形的面积[O'Rourke98]。本文将会向大家介绍叉乘积另外一些鲜为人知的应用。对于那些对修剪领域不是很精通的人来说，这些应用会让他们大吃一惊。说得具体一点，我们可以用叉乘积来求解线性方程组。举个例子，可以利用叉乘积来计算两条隐式直线的交叉点；也可以用它来计算给定的两个点所确定的一条直线的系数[Springall90]。我们还会向大家展示，如何利用叉乘积高效地计算出对一个多边形所进行的双线插值的设置（setup）运算。最后，还会向大家介绍，如何利用叉乘积来计算一个 3×3 矩阵的逆矩阵。由于 CPU 和 GPU 上都实现了叉乘积运算，只要我们小心地使用这些特性，它就可以为我们带来高效率的代码。

2.3.1 简介

首先来看一下，如何利用叉乘积来求解线性方程组。我们将叉乘积[Nicholson95]定义如下：

$$\begin{aligned}\mathbf{v}_1 \times \mathbf{v}_2 &= \det\begin{bmatrix}\mathbf{i} & \mathbf{j} & \mathbf{k}\\ x_1 & y_1 & z_1\\ x_2 & y_2 & z_2\end{bmatrix}\\ &= \begin{vmatrix}y_1 & z_1\\ y_2 & z_2\end{vmatrix}\mathbf{i} + \begin{vmatrix}z_1 & x_1\\ z_2 & x_2\end{vmatrix}\mathbf{j} + \begin{vmatrix}x_1 & y_1\\ x_2 & y_2\end{vmatrix}\mathbf{k} \qquad (2.3.1)\\ &= C_X\mathbf{i} + C_y\mathbf{j} + C_z\mathbf{k}\end{aligned}$$

在此，我们可以清楚地看到，叉乘积并不是什么神秘的东西，只是一系列的行列式。我们将利用这个事实，来求解二元一次方程组。这样的方程组可以定义如下：

$$\begin{aligned}x_1 a + y_1 b &= z_1\\ x_2 a + y_2 b &= z_2\end{aligned} \qquad (2.3.2)$$

我们可以将这个方程组改写成矩阵形式的，得到公式 2.3.3：

$$\mathbf{Mv}=\mathbf{k} \qquad (2.3.3)$$

Download at www.Pin5i.Com

其中的系数矩阵 M 就是：

$$\mathbf{M}=\begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \end{bmatrix} \tag{2.3.4}$$

而常量列就是：

$$\mathbf{k}=\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \tag{2.3.5}$$

最后，包含需要被求解的变量的向量 v 定义如下：

$$\mathbf{v}=\begin{bmatrix} a \\ b \end{bmatrix} \tag{2.3.6}$$

求解这样的一个方程组，我们至少有两种方法。一种方法是将这个方程组改写为公式 2.3.7：

$$\mathbf{v}=\mathbf{M}^{-1}\mathbf{k} \tag{2.3.7}$$

这样的话，我们必须要计算得到矩阵 M 的逆矩阵。另外一种可行的方法是，使用克莱姆法则（Cramer's Rule）[Nicholson95]。这样，上述方程组的两个解分别由公式 2.3.8 和公式 2.3.9 计算得出：

$$a=\frac{\begin{vmatrix} z_1 & y_1 \\ z_2 & y_2 \end{vmatrix}}{\begin{vmatrix} x_1 & y_1 \\ x_2 & y_2 \end{vmatrix}} \tag{2.3.8}$$

和

$$b=\frac{\begin{vmatrix} x_1 & z_1 \\ x_2 & z_2 \end{vmatrix}}{\begin{vmatrix} x_1 & y_1 \\ x_2 & y_2 \end{vmatrix}} \tag{2.3.9}$$

求解的过程是这样的：首先计算系数矩阵的行列式，这样就得到了上述公式中的分母。接下来，我们用常量列来替换系数矩阵中的第一列，这样就得到了未知数 a 的公式中的分子。同样地，我们用常量列来替换系数矩阵中的第二列，这样就得到了未知数 b 的公式中的分子。

如果我们将公式 2.3.8 中分子矩阵的列调换一下顺序，那么公式 2.3.8 的正负关系就变了。

$$a=\frac{\begin{vmatrix} y_1 & z_1 \\ y_2 & z_2 \end{vmatrix}}{\begin{vmatrix} x_1 & y_1 \\ x_2 & y_2 \end{vmatrix}} \tag{2.3.10}$$

我们可以对公式 2.3.9 进行同样的动作，得到公式 2.3.11：

$$b=\frac{\begin{vmatrix} z_1 & x_1 \\ z_2 & x_2 \end{vmatrix}}{\begin{vmatrix} x_1 & y_1 \\ x_2 & y_2 \end{vmatrix}} \tag{2.3.11}$$

Download at www.Pin5i.Com

我们这样做有我们的目的。现在，我们可以像公式 2.3.1 所定义的那样，利用叉乘积来计算未知数 a 和 b 的分母和分子了，这是因为：

$$a = -\frac{C_x}{C_z}, \tag{2.3.12}$$

和

$$a = -\frac{C_y}{C_z}, \tag{2.3.13}$$

如果不使用叉乘积，求解一个线性方程，我们依次需要 6 次乘法运算、3 次减法运算、1 次除法运算以及 2 次乘法运算。而使用叉乘积运算后，我们只需要 1 次叉乘积运算、1 次除法运算和 2 次乘法运算。如果我们使用的系统可以提供硬件支持的叉乘积运算，那么使用叉乘积来求解线性方程可以节省一些时间。

在某些情况下，我们会使用隐式线性方程，或者采用这种形式：

$$\begin{aligned} x_1 a + y_1 b + Z_1 &= 0 \\ x_2 a + y_2 b + Z_2 &= 0 \end{aligned} \tag{2.3.14}$$

注意，因为 Z_i=$-z_i$，所以，行列式的结果已经被取反了。因此，在这种情况下，我们就不需要再对 a 和 b 取反了。我们需要做的是计算叉乘积，用 Z_i 的值来替换 z_i 的值，然后再除以这个最新的 z 值。

2.3.2 隐式直线

众所周知，叉乘积技术不但可以用来计算两个给定的点所确定的一条隐式直线的系数，还可以用来计算两条隐式直线的交点[Skala04]，[Skala05]。首先介绍一下，如果根据两个给定的点，获得一个隐式直线方程，如图 2.3.1 所示：

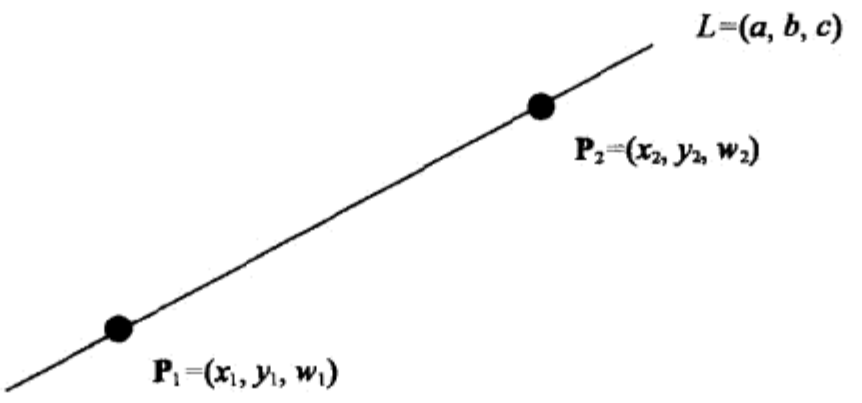


图 2.3.1 由定义该直线的两个点的叉乘积，我们就可以获得这条隐式直线

在 $\mathbf{R}^2$ 中的一条隐式直线可以定义如下：

$$ax + by + cw = 0 \tag{2.3.15}$$

对于这个方程所表示的直线上的点 P，它与法线向量 n 的点乘积等于 0：

$$\mathbf{n} \cdot \mathbf{p} = 0 \tag{2.3.16}$$

其中，

$$\mathbf{n} = (a, b, c)$$

Download at www.Pin5i.Com

$$\mathbf{p} = (x, y, w) \tag{2.3.17}$$

有些读者可能已经看出来了，由于使用了 3 个坐标值来表示 $\mathbf{R}^2$ 中的一个点，实际上就是用齐次坐标来表示这些点[Foley97]。另外，要注意，对于同一条隐式直线，我们可以有无数个不同的方程式来定义这条直线。这些方程式彼此互为纯量倍数。也就是说，如果公式 2.3.15 乘以任意一个不为 0 的数 s，新得到的这条直线在几何学上仍然是同一条直线。也就是说，公式 2.3.18 所表示的直线，与公式 2.3.15 表示的直线是同一条直线。

$$s\,(ax + by + cw) = 0 \tag{2.3.18}$$

假设空间中的两个点定义如下：

$$\begin{aligned} \mathrm{P}_1 &= (x_1, y_1, w_1) \\ \mathrm{P}_2 &= (x_2, y_2, w_2) \end{aligned} \tag{2.3.19}$$

如果只考虑常见的情况，将上述公式中的 w_1 和 w_2 都设为 1，那么要想找到穿过这两个点的那条直线，我们必须要求解下列方程组：

$$\begin{aligned} ax_1 + by_1 + c &= 0 \\ ax_2 + by_2 + c &= 0 \end{aligned} \tag{2.3.20}$$

既然这个线性方程的任意一个纯量倍数都是合理的，因此可以选择 c=1 的纯量倍数，将问题进一步简化，参见公式 2.3.21：

$$\begin{aligned} ax_1 + by_1 + 1 &= 0 \\ ax_2 + by_2 + 1 &= 0 \end{aligned} \tag{2.3.21}$$

这样就得到了两个二元一次方程。通过利用前面提到的叉乘积的方法，来求解这个线性方程组，图 2.3.1 中那条直线的系数就可以计算如下：

$$L = \mathrm{P}_1 \times \mathrm{P}_2 \tag{2.3.22}$$

然后，再将其除以 z 的值。

正如所希望的，实际上我们并不是必须要执行这个除法操作。和前面简化方程时的原因一样：除以任意一个非零数并不会影响结果直线的几何特性。因此，我们可以直接从叉乘积中选择一个。

那么，我们就把 a、b、c 的值分别设定为叉乘积中 x、y 和 z 的值。但是，如果叉乘积中 z 的值是 0（这样就会造成非法的除法运算），那么我们就可以判定，满足条件的直线不存在。这两个点是重合的。

同样地，对于两条隐式直线 L_1=（a_1，b_1，c_1）和 L_2=（a_2，b_2，c_2），如图 2.3.2 所示，它们的交叉点也可以利用叉乘积计算出来。要想得到这个交叉点，我们需要找到一个点（x，y，w），它同时位于这两条直线上。为此，需要求解下列方程组：

$$\begin{aligned} a_1x + b_1y + c_1w &= 0 \\ a_2x + b_2y + c_2w &= 0 \end{aligned} \tag{2.3.23}$$

再一次使用叉乘积来求解这个方程组，这个交叉点的计算公式如下：

$$\mathrm{P} = (a_1, b_1, c_1) \times (a_2, b_2, c_2) \tag{2.3.24}$$

这样就会得到一个以齐次坐标表示的点。然后，用 P 除以其第三个分量，就得到了另一种表示形式 P_E=（x，y，1）。同样，如果第三个分量的值等于 0，那么这个方程组就是无解的，也就是说两条直线没有交叉点。

Download at www.Pin5i.Com

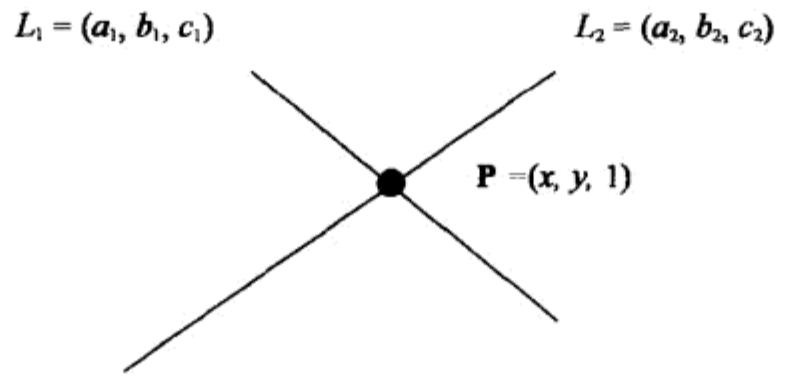


图 2.3.2 计算两条隐式直线的叉乘积就可以得到它们的交叉点

2.3.3 高效的扫描转换的设置运算

当 CPU 要在软件中渲染多边形的时候，最关键的问题是，如何让针对多边形的增量计算尽快地完成。这些计算工作主要包括在每个顶点上的属性集插值运算，例如高氏着色的强度[Gouraud71]；z-buffering 的 z 值，以及纹理贴图的纹理坐标[Hecker95]。下面，以高氏着色作为讲解实例。对于高氏着色，我们需要一个方程，来定义多边形表面上的着色强度。这个着色强度取决于当前的位置坐标。首先要计算出这个方程的各个系数。着色强度会形成一个平面，会在后面解释给大家。由于红色、绿色和蓝色分量的处理过程是一样的，为了简单起见，只考虑通用的灰阶强度。

着色强度可以定义如下：

$$\Phi(x,y) = ax + by + d \tag{2.3.25}$$

对于一个特定的三角形，只要能够求解下列方程组[Hast02][Hast03][Hast04]，就可以得到相应的系数：

$$\begin{aligned} x_0a + y_0b + d &= \phi_0 \\ x_1a + y_1b + d &= \phi_1 \\ x_2a + y_2b + d &= \phi_2 \end{aligned} \tag{2.3.26}$$

其中，ϕ_0 是（x_0，y_0）处的强度，ϕ_1 是（x_1，y_1）处的强度，而 ϕ_2 是（x_2，y_2）处的强度。上述方程组的求解结果是：

$$a = \frac{(y_2 - y_0)(\phi_1 - \phi_0) - (y_1 - y_0)(\phi_2 - \phi_0)}{(x_1 - x_0)(y_2 - y_0) - (x_2 - x_0)(y_1 - y_0)} \tag{2.3.27}$$

$$b = \frac{(x_1 - x_0)(\phi_2 - \phi_0) - (x_2 - x_0)(\phi_1 - \phi_0)}{(x_1 - x_0)(y_2 - y_0) - (x_2 - x_0)(y_1 - y_0)} \tag{2.3.28}$$

$$d = \phi_0 \tag{2.3.29}$$

很明显，上面这些求解公式的计算工作需要大量的减法运算和乘法运算。但是，可以利用前面提到过的技术，把它们简化为两次向量减法运算和 1 次叉乘积运算。不管怎样，上面公式中的除法运算仍然是不能少的。

利用相对坐标，可以用叉乘积技术来求解公式 2.3.26 中的方程组。只是要注意，须要从（x_0，y_0）和强度 ϕ_0 开始，因此，d=ϕ_0。相对坐标及强度的计算公式如下：

$$X_1 = x_1 - x_0$$

Download at www.Pin5i.Com

$$X_2 = x_2 - x_0$$
$$Y_1 = y_1 - y_0$$
$$Y_2 = y_2 - y_0$$
$$\Phi_1 = \phi_1 - \phi_0$$
$$\Phi_2 = \phi_2 - \phi_0 \tag{2.3.30}$$

只需要两个向量的减法运算，我们就可以完成所需的计算工作。如果各个向量的分量设置如下：

$$\mathbf{v}_0 = \begin{bmatrix} x_0 \\ y_0 \\ \phi_0 \end{bmatrix}, \mathbf{v}_1 = \begin{bmatrix} x_1 \\ y_1 \\ \phi_1 \end{bmatrix}, \mathbf{v}_2 = \begin{bmatrix} x_2 \\ y_2 \\ \phi_2 \end{bmatrix} \tag{2.3.31}$$

接下来，我们可以定义出 $\mathbf{V}_1$=（X_1，Y_1，$\boldsymbol{\Phi}_1$）和 $\mathbf{V}_2$=（X_2，Y_2，$\boldsymbol{\Phi}_2$），则有：

$$\mathbf{V}_1 = \mathbf{v}_1 - \mathbf{v}_0$$
$$\mathbf{V}_2 = \mathbf{v}_2 - \mathbf{v}_0 \tag{2.3.32}$$

然后，需要我们求解的方程组就变成了下列形式：

$$X_1 a + Y_1 b = \Phi_1$$
$$X_2 a + Y_2 b = \Phi_2 \tag{2.3.33}$$

利用叉乘积技术，我们求得 $\mathbf{V}_1$ 和 $\mathbf{V}_2$ 的叉乘积：

$$\mathbf{N} = \mathbf{V}_1 \times \mathbf{V}_2 \tag{2.3.34}$$

这样，就可以计算出公式 2.3.25 中的方程组的系数：

$$a = -\mathbf{N}_x / \mathbf{N}_z$$
$$b = -\mathbf{N}_y / \mathbf{N}_z$$
$$d = \phi_0 \tag{2.3.35}$$

其中，**N** 的下标表示的是向量中相应元素的位置，从 x 开始，到 z 结束。

下面来看一下，目前为止的进展程度。对于两个彼此不平行的向量 $\mathbf{V}_1$ 和向量 $\mathbf{V}_2$，它们的叉乘积应该是一个与它们正交的向量，如图 2.3.3 所示。那么，对于我们这个例子，这又意味着什么呢？通过计算相对坐标，我们得到了两个位于（x，y，Φ）空间的向量 $\mathbf{V}_1$ 和 $\mathbf{V}_2$。而多边形本身是位于屏幕空间中的，并没有相应的 z 值。但是，它需要把着色强度的值作为第三个坐标的值。因此，我们可以说，这个强度平面就位于（x，y，Φ）空间中。和我们在（x，y，z）空间中的做法一样，我们可以利用公式 2.3.26 来计算这个平面的法线向量。

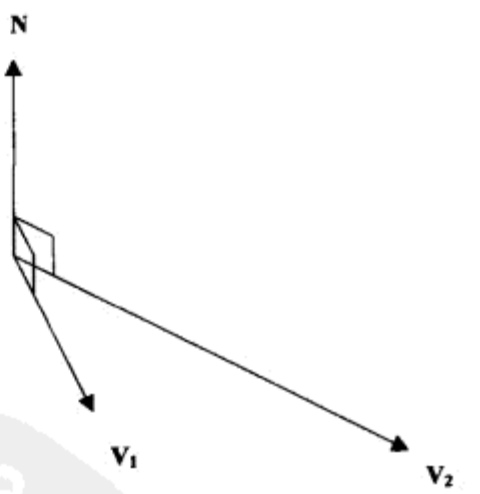


图 2.3.3　法线与位于（x，y，ϕ）空间的两个向量正交

我们现在来证明：**n**=（a，b，-1）也是公式 2.3.25 所定义的那个平面的法线。这其实是非常明显的。因为在公式 2.3.35 中是这样来计算 a 和 b 的：用 **N** 除以它的第三个分量 $\mathbf{N}_z$ 的反，也就是$-\mathbf{N}_z$。如果将 **N** 的三个分量都除以$-\mathbf{N}_z$，那么就可以得到值为-1 的第三个分量。但是，下面要利用著名的平面向量方程[Nicholson95]（参见公式 2.3.36），来推导出公式 2.3.25，以便比较正式地向大家证明上述命题：

Download at www.Pin5i.Com

$$\mathbf{n} \cdot (\mathbf{P}-\mathbf{P}_0) = 0 \tag{2.3.36}$$

其中，P 和 $\mathbf{P}_0$ 是位置向量。将该平面的法线向量、起点以及平面上任意指定的一个点定义如下。

$$\begin{aligned} \mathbf{n} &= (a, b, -1) \\ \mathbf{P}_0 &= (0, 0, d) \\ \mathbf{P} &= (x, y, \Phi) \end{aligned} \tag{2.3.37}$$

将上述公式代入公式 2.3.36，我们就得到了：

$$(a,b,-1) \cdot ((x,y,\Phi) - (0,0,d)) = 0 \tag{2.3.38}$$

最后，将这个点乘积展开，就可以得到：

$$ax + by - \Phi + d = 0 \tag{2.3.39}$$

这下就很清楚了，这就是公式 2.3.25 中那个方程的隐式形式。而且也满足前面已经知道的定义一个平面的方程的各个系数。当所有的变量都为 0 时，常数 d 自然就是该方程的结果。而各个变量的系数就组成了该方程所定义的这个平面的法线。

2.3.4 求解三元一次方程组

叉乘积还有一个妙用，就是可以用来求解三元一次方程组。显而易见，这里可以再一次使用克莱姆法则（Cramer’s Rule），生成总共 4 个行列式，来计算三个分子（因为此时有三个未知数）和一个分母。一个 3×3 矩阵的行列式可以计算如下：

$$\det(\mathbf{M}) = \mathbf{v}_1 \cdot (\mathbf{v}_2 \times \mathbf{v}_3) \tag{2.3.40}$$

其中：

$$\mathbf{M} = \begin{bmatrix} \mathbf{v}_1^T \\ \mathbf{v}_2^T \\ \mathbf{v}_3^T \end{bmatrix} = \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \end{bmatrix} \tag{2.3.41}$$

总的计算开销是 4 次点乘积运算、4 次叉乘积运算、1 次除法运算，以及 3 次纯量乘法运算（scalar multipliction）。除此之外，在计算每个行列式时，我们还需要调整矩阵中相应元素的位置。

求解三元一次方程组的另外一种方法，是利用克莱姆法则（Cramer’s Rule）的一个变种——伴随公式（adjoint formula）[Nicholson95]，来计算公式 2.3.7 中的逆矩阵：

$$\mathbf{M}^{-1} = \frac{1}{\det(\mathbf{M})}\mathrm{adj}(\mathbf{M}) \tag{2.3.42}$$

将矩阵所有位置上的元素都替换为元素的代数余子式，就得到了该矩阵的伴随矩阵。一个 3×3 矩阵 M 的伴随矩阵可以定义如下：

$$\mathrm{adj}(\mathbf{M}) = \begin{bmatrix} y_2z_3 - y_3z_2 & y_3z_1 - y_1z_3 & y_1z_2 - y_2z_1 \\ x_3z_2 - x_2z_3 & x_1z_3 - x_3z_1 & x_2z_1 - x_1z_2 \\ x_2y_3 - x_3y_2 & x_3y_1 - x_1y_3 & x_1y_2 - x_2y_1 \end{bmatrix} \tag{2.3.43}$$

如果仔细观察一下这个矩阵的每一个列，就会发现，它们其实就是叉乘积！显而易见，我们可以得到公式 2.3.44：

Download at www.Pin5i.Com

$$\mathrm{adj}(\mathbf{M}) = [\mathbf{v}_2 \times \mathbf{v}_3 \quad \mathbf{v}_3 \times \mathbf{v}_1 \quad \mathbf{v}_1 \times \mathbf{v}_2] \tag{2.3.44}$$

因此，通过 4 次叉乘积、1 次点乘积、1 次除法，以及 1 次纯量矩阵乘法，我们就可以计算出逆矩阵。

注意，最终的目标是求解方程组，上面这些并不能给我们带来什么收获。为了求解方程组，还需要计算一个向量矩阵乘法，然后纯量矩阵乘法就可以转换为纯量向量乘法。这样的计算过程需要的运算次数和直接使用克莱姆法则是一样的。而且，我们可能还要小心数值问题。但不管怎么说，这两种方法都是很有用的，因为它们都使用了硬件中实现的叉乘积运算。如果我们真的需要一个逆矩阵，第二种方法会更有效，因为它只涉及少量的向量运算。

2.3.5　总结

很多人，至少是那些了解修剪算法的人都知道，可以用叉乘积来计算两条隐式直线的交叉点，也可以用叉乘积来计算隐式直线方程的系数。而实际上，叉乘积也是求解线性方程组的重要手段之一。在本文中，我们利用克莱姆法则向大家证明了叉乘积可以用来求解二元一次方程组。在扫描转换的高氏着色和其他的顶点属性中，我们会广泛地使用多边形的双线插值。对于这种双线插值的设置计算，我们也可以利用叉乘积来完成。另外，我们还向大家介绍了，如何利用叉乘积，来高效率地计算出一个 3×3 矩阵的逆矩阵。由于 CPU 和 GPU 都在硬件中集成了特殊的指令来计算叉乘积，本文的内容对某些应用来说会是非常有帮助的。

2.3.6　致谢

在此非常感谢西波希米亚大学的 Vaclav Skala 教授，感谢他指导我如何利用叉乘积来计算两条隐式直线的交叉点，也因此激发了我的灵感，去探索叉乘积更多的用途。

2.3.7　参考文献

[Foley97] Foley, J. D., A. van Dam, S. K. Feiner, and J. F Hughes, *Computer Graphics: Principles and Practice*, 2nd Ed. Addison-Wesley, 1997.

[Gouraud71] Gouraud, H., "Continuous Shading of Curved Surfaces." June 1971. *IEEE Transactions on Computers*, Vol., C-20, No. 6.

[Hast02] Hast, A., T. Barrera, and E. Bengtsson, "Improved Bump Mapping by using Quadratic Vector Interpolation." Eurographics '02 short/poster, 2002.

[Hast03] Hast, A., T. Barrera, and E. Bengtsson, "Fast Setup for Bilinear and Biquadratic Interpolation over Triangles." *Graphics Programming Methods*, Jeff Lander, Ed. Charles River Media, 2003: pp. 299–314.

[Hast04] Hast, A., "Improved Algorithms for Fast Shading and Lighting." Ph.D. Thesis, 2004.

[Hearn04] Hearn, D. and M. P. Baker, *Computer Graphics with OpenGL*. Pearson Education Incorporated, 2004.

[Hecker95] Hecker, C., "Perspective Texture Mapping, Foundations." *Game Developer*

Download at www.Pin5i.Com

Magazine, April/May 1995: pp. 16–25.

[Kugler96] Kugler, A., "The Setup for Triangle Rasterization." August 1996. *Eleventh Eurographics Hardware Workshop '96,* Poitiers, France.

Linear Interpolation over Triangles." 1995. *Computer Graphics Forum,* Vol. 14, No. 1: pp. 17–24.

[Nicholson95] Nicholson, W. K., *Linear Algebra With Applications,* 3rd Ed. PWS Publishing Company, 1995.

[O'Rourke98] O'Rourke, J., *Computational Geometry in C,* 2nd Ed. Cambridge University Press, 1998.

[Skala04] Skala, V., "A New Line Clipping Algorithm with Hardware Acceleration." Computer Graphics International 2004: pp. 270-273.

[Skala05] Skala, V., "A new approach to line and line segment clipping in homogeneous coordinates." *The Visual Computer,* Vol. 21, No. 11, pp. 906–914, Springer Verlag 2005.

[Springall90] Springall, T. L. and G. Tollet, "A shading approach to non-convex clipping." July 1990. *APL 90: For the Future conference proceedings,* Vol. 20, Issue 4: pp. 369–372.

Download at www.Pin5i.Com

2.4　适用于游戏开发的序列索引技术

Palem GopalaKrishna，印度理工学院计算机科学与工程系
krishnapg@yahoo.com

序列索引技术是操作对象的一门艺术。它尝试去回答这样一个简单的问题：对于给定的一个对象的集合，该如何创建、访问，并识别从这个对象集合中提取出来的各种不同的对象组群。大部分的游戏都包含有多个大型的对象集，所以，如果能够很好地理解序列索引技术的概念，这会对游戏开发工作大有帮助。

从数学的角度出发，我们将这些游戏对象的集合用“序列”来表示。本文将介绍一些数据公式，用来对下列几个著名的序列进行索引（indexing）和解索（deindexing）：

- 范围序列；
- 排列序列；
- 组合序列。

本文中提供了几个小窍门，讲授如何能够在游戏环境中更好地利用这些序列，完成各种各样的任务：从确定性随机（deterministic randomness）的仿真到游戏对象的连续化。

2.4.1　相关术语

在开始讨论序列之前，为了避免术语上的混淆，有几个简单的符号和定义，需要在这里明确一下。

整个文章中将用符号“#”来表示一个集合的大小。例如，#{a，b，c}=3，#{0}=1，而#{}=0。

还有一个相关的运算符是符号“⊥”，用来表示一个集合（数组）中某个符号的位置（索引）。例如，（$b\perp\{a, b, c\}$）=1，而（$a\perp\{a\}$）=0。在这里，假设的前提条件是：相应的符号总是（惟一地）存在于数组中。这样，$x\perp V$ 总是会满足这个条件：$0\leqslant(x\perp V)<\#V$。

用符号“n！”来表示阶乘积的概念。一般而言，$n!=n\cdot(n-1)\cdot(n-2)\cdot\cdots\cdot3\cdot2\cdot1$。举个例子，$4!=4\cdot3\cdot2\cdot1$，也就是等于 24，这个符号对下面的计算工作大有帮助。运算符 C_n^r，由 $\binom{n}{r}$ 来表示，

Download at www.Pin5i.Com

其计算公式是 $C_n^r=\left(\frac{n!}{r!(n-r)!}\right)$。我们会在后面的内容中介绍这个运算符和阶乘积的用法。

2.4.2 序列

一个序列就是一系列数据项，每个数据项代表一组有序的符号。而这些符号又满足这个序列所独有的某些特征。我们把一个序列表示为一个有序集 **S**，其中第 i 个数据项用 **S**[i]来表示，而该数据项中第 j 个符号就被表示为 **S**[i] [j]。所以，如果：

$$\mathbf{S}=\begin{Bmatrix}1234\\3467\\4789\\5789\\\vdots\end{Bmatrix}$$

表示的是一个序列，那么 **S**[0]=*1234*，**S**[1]=*3467*，**S**[2]=*4789*，**S**[3]=*5789*，依此类推，这些就是这个序列的数据项。而 **S**[0][0]=*1*，**S**[0][1]=*2*，…，**S**[3][2]=*8*，**S**[3][3]=*9*，…这些则是它的符号。我们这里用斜体字来表示序列中的元素，把它们看作是有序的一组符号，它们并不一定表示的就是数字。一个序列的长度，我们表示为|**S**|。序列的长度就是序列中数据项的个数。所以，如果序列 **S** 中包含数据项 **S**[0]，**S**[1]，…，**S**[m−1]，那么||**S**||=m。

注意，在这个例子中，序列 **S** 中的符号都满足这个特征：对于所有的 $0\leqslant j\leqslant 3$，$i\geqslant 0$，则 **S**[i] [j]<**S**[i] [j+1]。当被描述为数学或逻辑表达式时，序列中所有的数据项都满足这个特征。这个特征也为我们提供了这个序列的特征表达式。通过各种不同的特征表达式，我们就可以生成任意多个序列。但是，大家应该注意的是，一个序列是由特征表示式来定义的，而不是由单个的符号来定义的。基于这一点，对于下面这两个序列：

$$\mathbf{S}_1=\begin{Bmatrix}1234\\3467\\4789\\5789\end{Bmatrix}\text{and}\qquad \mathbf{S}_2=\begin{Bmatrix}abcd\\cdfg\\dghi\\eghi\end{Bmatrix}$$

我们认为就是同一个序列（这是因为，虽然它们的符号是不同的，但它们的特征表达式是相同的）。在提取单个符号的同时，还可以提炼它们的逻辑关系。在很多领域的计算工作背后，这种机制是一个至关重要的因素。比如：模式识别、数据压缩、信息理论，以及人工智能等众多的领域，都需要这种机制。

虽然一个特征表达式就可以完整地、惟一地定义一个序列，但是，并非所有给定的序列，我们都可以提出相应的逻辑表达式或数学表达式（DNA 序列就是这样一个著名的例子）。在这种情况下，我们就无法非常简洁地引用这样的序列，更不可能显式地表述其所有的数据项。因此，如果某个序列存在一个可用的特征表达式，那么这个特征表达式一定就是这个序列的

Download at www.Pin5i.Com

压缩版本。举个例子，所有的程序都是它们各自指令序列的特征表示式。[①]如果某个序列不存在任何已知的特征表示式，那么就被称之为“不可压缩的”序列。不可压缩的序列代表着随机性，我们很少会用到它们。

最后一点，有个很好玩的地方提醒大家注意一下，单个的符号，比如 **S**[0][0]，或者是 **S**[0][1]，它们不一定必须是整数或字母。它们也可以是其他任何一种复杂的数据类型，比如：字符串、鼠标的坐标位置、事件描述符，或者是类对象等。那么，这些复杂的符号又会产生什么类型的序列呢？而且更为重要的是，游戏开发工作如何从中收益呢？这些问题也正是我们在下面的内容中试图详细解答的问题。

2.4.3　范围序列

如果序列的数据项中，每个符号的取值都在一个固定的范围内变化（各个符号之间彼此独立），那么由这些符号所组成的数据项就构成了一个“范围序列”。举个例子，像下面这个序列：

$$\mathbf{S}_{\mathbf{R}} = \begin{Bmatrix} a20 \\ a25 \\ a26 \\ b20 \\ b25 \\ b26 \end{Bmatrix}$$

这就是一个范围序列。它的每个数据项由三个符号组成。每个符号的值都分别取自这三个集合：{a, b}、{2}、{0, 5, 6}。它们各自的范围大小可以表示为：**R**[0]=#{*a*, *b*}=2, **R**[1]=#{*2*}=1, **R**[2]=#{*0*, *5*, *6*}=3。

一般来讲，对于一个给定的范围序列，如果它的每个数据项有 *n* 个符号组成。这 *n* 个符号分别从集合 **V**[0]，…，**V**[*n*−1]中取值。这些集合各自的大小是 **R**[0]=# **V**[0]，…，及 **R**[*n*−1]=# **V**[*n*−1]，则这个序列的长度可以用下列公式计算出来：

$$\|\mathbf{S}_{\mathbf{R}}\| = \mathbf{R}[0], \cdots, \mathbf{R}[n-1] = \prod_{i=0}^{n-1} \mathbf{R}[i]$$

也就是说，序列 $\mathbf{S}_{\mathbf{R}}$ 的长度就等于所有集合大小的乘积。对于我们这个例子，$\|\mathbf{S}_{\mathbf{R}}\| = \mathbf{R}[0] \cdot \mathbf{R}[1] \cdot \mathbf{R}[2] = 2 \cdot 1 \cdot 3 = 6$。因此，我们这个例子序列中有 6 个数据项。我们可以生成的任意一个其他的数据项（满足该序列的范围特征），就是这 6 个中某个数据项的副本。

给定任意一个数据项的索引，这个数据项的各个符号就可以计算如下。序列 $\mathbf{S}_{\mathbf{R}}$ 的第 *i* 个数据项（$0 \leqslant i < \|\mathbf{S}_{\mathbf{R}}\|$）的第 *j* 个符号（$0 \leqslant j < n$），由下面公式给出：

[①] 一个程序就是一组详尽指令的简明表示形式。这些指令交由机器取执行，以完成某个特定的功能。对于任何一个典型的程序而言，机器所执行的指令数量要远远大于程序的大小。举个例子，一个简单的程序语句，如：for（i=0；i<10 000；++i），这表示 CPU 要执行 10 000 个运行时指令。因此我们可以这样说，程序就是硬件在运行这个程序时实际要执行的指令集的压缩版本。对于编程语言来说，如果能在它们的语句中打包进去更多的功能，那么在源代码中，我们表达一个事实所需要的语句就越少。但在目标平台上执行时，就需要更多的处理能力来理解这段代码所表达的事实。对于我们这个 C 语言的 for 循环语句，其压缩比已经达到了惊人的 10 000：1。但较之一个普通的 10 岁小孩日常语句的压缩比来说，基本上是微不足道的。一个 10 岁的小孩，每天要表达诸如，“到这里来”、“看那个”等意思，他的语句压缩比要比我们这个 for 语句高得多。如何降低这种上下文相关的指令的复杂度，这是 AI 程序员必须要克服的主要障碍。然而，游戏环境的交互性直接依赖于这种技术。

Download at www.Pin5i.Com

$$S_R[i][j]=V[j]\left[\frac{i\%\Pi_{k=j}^{n-1}R[k]}{\Pi_{k=j+1}^{n-1}R[k]}\right]$$

注意，$\Pi_{k=n}^{n-1}R[k]=1$。

反过来，如果给定一个数据项的各个符号 $t[0]$ $t[1]\cdots t[n-2]$ $t[n-1]$，这个数据项所对应的索引 i 可以用下列公式计算得到：

$$i=\sum_{j=0}^{n-1}((t[j]\perp V[j])\bullet\Pi_{k=j+1}^{n-1}R[k])$$

在编码实现上述这些公式时，为了避免重复计算乘积，我们可以维护一个一维的查找数组。该数组的第 j 个元素的值为 $\Pi_{k=n}^{n-1}R[k]$。下面是提供的伪代码：

```
void PreComputeProducts(int Product[], int Range[])
{
     Product[NoOfSymbols] = 1;
     for(int k=NoOfSymbols-1; k>=0; --k)
         Product[k] = Product[k+1]*Range[k];
}
```

下面给大家举个例子，利用这个数组，根据一个索引值，来计算出相应的数据项：

```
TermType GetTerm(SymbolType Values[][], int TermIndex)
{
    TermType Term;
    for(int j=0; j < NoOfSymbols; ++j)
    {
        int i = (TermIndex % Product[j])/Product[j+1];
        Term[j] = Values[j][i];
    }
    return Term;
}
```

反之，根据给定的某个数据项的各个符号，也可以计算出这个数据项的索引值，这个函数实现的源代码在随书光盘中提供给大家。

几乎每一个任意给定的数据项的随机集合，我们都可以把它们聚合起来，并断言它是某个现有范围序列（也可能是一个新的范围序列）的一部分。从这个意义上讲，范围序列本质上是通用的。因此，对所有的编程用途而言，范围序列颇为受用。虽然其他各种类型的序列对编程工作也很有用，但范围序列最为突出的特征是，范围序列中的符号以各自不同的方式来生成自己的值。这个特性使得范围序列成为一个显然比其他序列更有优势的选择。各个符号值之间的独立性为程序人员提供了更多的灵活性和通用性，这是其他种类的序列所不能提供的。下面举个例子，让我们来考虑下面这个假设的类（Class）：

```
class WildWestPerson
{
    enum Sex { Female, Male };
```

Download at www.Pin5i.Com

```
        enum Nature { Good, Bad, Ugly };
        enum Living { Dead, Alive };

        Sex m_SexVal;
        Nature m_NatureVal;
        Living m_LivingVal;
        BYTE m_RatingVal; //Valid values between [1,5];
    };
```

上面这个类有 4 个成员变量。每个成员变量可以分别从 2，3，2 和 5 这 4 个数字中取值。对于所有可能从这个类创建的独特的对象，我们都可以用一个包含 4 个符号的范围序列来表示。这 4 个符号各自的范围是 **R**[0]=#Sex=2，**R**[1]=#Nature=3，**R**[2]=#Living=2，以及 **R**[3]=#{1…5}=5。

对这类范围序列的特征进行仔细的分析，就可以为游戏策划工作提供更为透彻的思路。举个例子，在上面的例子中，最大的序列长度是∏**R**=2 • 3 • 2 • 5=60。因此，上面这个类可以提供最多 60 个独特的对象（也就是人物）。我们可以利用这些信息，结合游戏中的其他元素进行分析、理解，并/或者为游戏行为创建正确的衡量标准。这些游戏行为可以是活泼度、可重复性，以及交互能力等。同样地，还是使用我们这个例子，如果只有前面提到的类 WildWestPerson（再没有其他的什么）可以控制游戏主角的行为能力，那么玩家就可以获得 60 种不同风格的游戏。[①]

总的来说，如果变量的取值范围比较明确，例如类对象，或统计图表，对这种变量的集合进行索引，范围序列就大有用武之地。首先检查一个给定类的实例中的成员变量，并利用上面的技巧，我们就可以指定一个唯一的索引值，来表示这个实例相对于其他实例的位置。一旦某个类的所有实例都这样进行了索引，无限可能就尽在掌握之中。

- 程序人员可以将这个索引作为一种哈希算法，来找到或删除重复的对象，只存储惟一的、无重复的对象。
- 我们可以把和一个对象相关联的索引作为通信令牌来使用，在网络上传递该对象的状态（不需要任何其他的位打包例程）。
- 同样地，我们也可以利用这些索引，将对象连续化，以便可以快速地保存游戏的状态。
- 如果我们不访问对象的成员变量，而是直接在对象之上执行一些复杂的功能，索引的作用也是非常明显。举个例子，如果要比较 WildWestPerson 类的两个实例，正常情况下，我们都会编写一段代码，来比较这两个对象的所有成员变量。还有一种方法是推导出这个对象序列的特征表达式，并针对这个序列定义一个比较运算符，比如，我们可以这样定义：序列中靠前的对象要“小于”其后面的对象。这样，就可以在离线状态下提前计算出比较的结果，然后在运行时加载这个结果，或者是在运行时来改变比较的结果。这使得我们可以将游戏数据（策略决策）与游戏概念孤立开来。对于每一款精心打造的游戏来说，游戏概念都是一笔无价的资产。

[①] 这里描述的这个例子都是假设的情况。但是在现实生活中，游戏世界要处理成百上千个不同的范围序列。正是这些序列为游戏提供了成千上万种，甚至是几百万种不同的变化。在大部分时间中，对游戏的变化贡献最大的是来自于玩家对游戏事件所做出的行为和反应，而不是来自于游戏本身内部的设置。如果能够将玩家的行为/反应链用序列来表示，并研究这些序列的特征，会极大地有助于我们去分析玩家的行为，形成一套标准的行为准则，来改进我们的游戏策划和游戏风格。

Download at www.Pin5i.Com

2.4.4 排列序列

在数学的概念中，“排列”就是用各种不同的方式来编排一组对象。对于给定的一个符号的集合，对其中所有符号，用所有可能的方法进行穷举式的重新排列，得到一个“排列序列”。这个序列中的每一个数据项都表示着这些符号的一个不重复的排列方式。举例来说，对于这个有序的符号集{a，b，c}，可以形成一个排列序列：

$$\mathbf{S}_P = \begin{Bmatrix} abc \\ acb \\ bac \\ bca \\ cab \\ cba \end{Bmatrix}$$

范围序列和排列序列之间的主要区别是，在范围序列中，一个数据项中的每个符号，它的取值是独立于其他符号的。而在排列序列中，一个符号的位置很大程度上要依赖于（并完全取决于）其相邻的符号。

一般而言，对于一个给定的符号集 **A**，它所确定的一个排列序列的长度由（#**A**）！来表示。这和我们上面看到的情况是相符合的：在这个例子中，$\mathbf{S}_p$ 的长度就等于$\|\mathbf{S}_p\|$=（#{*a*，*b*，*c*}）！=3！=6。序列 $\mathbf{S}_p$ 有 6 个数据项，每个数据项表示一个独一无二的排列。我们所能想到的其他的排列肯定是这个序列中某个数据项的副本。

和范围序列的情况一样，对于一个给定的排列序列，如果知道了任意一个数据项的索引值，就可以计算出这个数据项中每一个符号。序列 $\mathbf{S}_p$ 中第 *i* 个数据项（$0 \leqslant i < \|\mathbf{S}_p\|$）的第 *j* 个符号（$0 \leqslant j < n$）可以用下列公式计算得到：

$$\mathbf{S}_p[i][j] = A_j\left[\frac{i\%(n-j)!}{(n-j-1)!}\right], \text{其中 } A_j = A - \bigcup_{k=0}^{j-1}\{\mathbf{S}_p[i][k]\}$$

在这里，A_j 表明了这样一个事实：随着我们在一个数据项中，从一个符号前进到下一个符号，我们同时也在削减我们的选择。每个数据项的具体情况也不尽相同。例如，对于 *i*=2 的情况：A_0=[*a*，*b*，*c*]，A_1=[*a*，*c*]，而 A_2=[*c*]。在实际应用中，可以通过交换运算来处理这个事情（要想了解更为详细的信息，可参考随书光盘中的示范代码）。

反过来，如果给定一个数据项的各个符号 *t*[0] *t*[1]…*t*[*n*−2] *t*[*n*−1]，那么该数据项的索引值 *i* 就可以用下列公式计算得到：

$$i = \sum_{j=0}^{n-1}[(t[j] \perp A_j) \bullet (n-j-1)!], \text{其中 } A_j = A - \bigcup_{k=0}^{j-1}\{t[k]\}$$

给定若干个符号的集合，如果需要对这些集合进行穷举排列，此时，排列序列就可以发挥自己的作用了。举例来说，如果有一组对象必须要执行一个重复序列的动作，比如，格斗序列，或者是动画序列。排列序列会帮助我们去处理这种任务。假设有一组军舰停泊在港湾

Download at www.Pin5i.Com

之中。下面考虑它们的一个动画序列。虽然事实上，这些军舰基本上都是静止不动的，但是，为了让它们看上去更逼真，惯用的方法是时不时地让军舰的桅杆或甲板动一动。最直接的方法是，每过几秒钟就随机选择一艘军舰，然后播放桅杆的动画。但是，随机地选择军舰，这种做法有个潜在的、而且是不可预期的问题：个别军舰可能永远都不会被选中，因此也永远都不会播放桅杆的动画。

另外还有一种做法，就是不采用随机选择的方式，而是采用完全的循环方式。虽然采用这个方法可以让所有的军舰都会选择到，并执行桅杆的动画，但实际效果看上去却不够真实。其效果就好像风是固定地围绕着军舰在那里转悠，并以一种精确的设计好的方式来吹动军舰的桅杆。更好的方法应该是一种确定性的、但却不可测定的方法。而排列序列可以非常容易地以非常直接的方式为我们提供这样的一种方法。

还是举个具体的例子来考虑 4 艘军舰的情况。军舰 *a*、*b*、*c* 和 *d* 需要每隔几秒钟就摇动它们的桅杆。在第一个时间段中，我们选择军舰 *a*，播放它的桅杆摇动的动画。几秒钟之后，我们选择下一艘军舰 *b*，播放它的桅杆摇动的动画。然后再选择军舰 *c*，最后再选择军舰 *d*。这样就完成了一个回合的操作。在下一个回合中，仍然从军舰 *a* 开始，然后再选择军舰 *b*。但是，这次将军舰 *c* 和军舰 *d* 的顺序调换一下，得到了新的顺序是 *abdc*。然后在接下来的回合中，再次调整选择的顺序，这次的顺序是 *acbd*。再下一个回合的顺序应该是 *acdb*，然后再是 *adcb*，……依此类推。用这种方法，我们每个回合都可以保持变化了的顺序，让最终的效果看上去就好像桅杆的动画是随机的，但实际上却是非常确定的。

除此之外，通过分析这个排列序列的特征，可以更深入地体会“确定性混沌（deterministic chaos）”的本质，从而极大地提高游戏产品的真实感。例如，假设游戏中玩家要面对三个敌对的弓箭手/枪手 *a*、*b*、*c*。它们分别按照下面这个序列朝玩家射击：

表 2.4.1　　**射手排列序列**

a,b,c,	*a,c,b,*	*b,a,c,*	*b,c,a,*	*c,b,a,*	*c,a,b,*	*a,b,c,*	*b,a,c,*	*b,c…*	…

每个回合只有一个射手射击，然后再藏身于柱子后面，这样玩家就不会因为三个敌人同时向他射击，被压制得抬不起头来。对于上面的这个序列（很明显是一个排列序列），射手 *a* 第一个射击，然后是射手 *b*，接着是射手 *c*。然后又轮到射手 *a*，依此类推。仔细观察，就能从这个序列中发现一个有趣的现象。如图 2.4.1 所示，它向我们揭示了每个单独的射手两次相邻射击之间的延时。

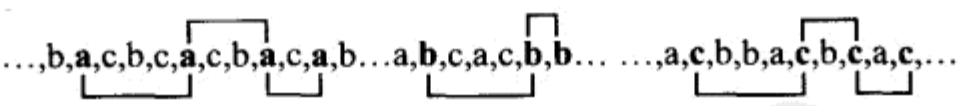


图 2.4.1　排列序列的各个数据项为我们展示了“确定性混沌”

对于每个单个的射手，其相邻的两次射击之间的间隙不会总是一样的。例如，我们可以来看一下射手 *a* 的情况。如图 2.4.1 所示，他的前两次射击之间的间隔要比接下来两次射击之间的间隔要大一些。而其他射手两次射击之间的射击间隔也是不一样的：射手 b 有两次连续的射击，其间没有其他的射手插进来。在这个序列中，这是其他射手享受不到的特权。但是，这个射击序列也有自己的规律——没有哪个射手的射击次数太多，或者太少。在每个射击回合中，所有三个射手都有一次机会去射击。

Download at www.Pin5i.Com

从玩家的角度来看，这种方法可以产生一种很自然的效果。一方面，射击的规律性向玩家传达了其对手的本性：受过训练，且坚决果断。另一方面，由于同一个射手两次连续射击之间的间隔时间不同，这又向玩家表明，这些射手不是系统来控制的，也使得这些射手看上去不那么机械。玩家都希望游戏里的对手更人性化，他们对第二个方面的特征会有很多种不同的解读。他们会认为，敌方可能遇到了什么偶然的事故，比如，在关键时刻却需要重新装弹。这在一般的战斗中是很常见的事情。或者他们会认为，敌方不知道该采取怎样的动作，因为玩家的某个移动打乱了敌方将要实施的策略。

如果能用可视化的行为来解释同一个射手不同的射击间隔，那么，由此会进一步增强玩家在这个方面的游戏体验。我们来举例说明。假设（在每一个射击序列中）在某个回合的时候，其中一个射手不得不将他的射击动作推后，那么我们可以让他去做下面的一个事情。

- 重新装弹。
- 看自己是否被击中了，是否在流血。
- 本来想扔一枚手榴弹出去，结果发现自己的武器库中没有手榴弹。骂了一句脏话，然后只好用原来的武器去射击。
- 如果对他有额外的帮助，则移动到房间的另一个角落。可能是他发现局势越来越糟糕，所以打算撤回去。
- 本来打算要撤退了，但看到自己的战友还在战斗，只好又返回来，以快速、突发式的射击来恢复自己的射击动作。这样的行为偶尔可以解释得通，为什么会出现连续两次不间断射击的情况，就像我们前面说到了射手 *b* 的情况。

以上只是罗列了很少的一些选择，各种可能的行为动作是无穷无尽的，只要肯于思考。至于应该采用哪种选择，这要取决于两次射击之间的时间间隔。我们可以提前计算出射击序列的各个数据项，以便确定两次射击之间的时间长短，然后，我们就可以利用这些信息做出最后的决策，让角色采取哪种行为来填补相应的时间段。

我们还可以创建更为复杂的行为，来进一步改进这个方法。一种可行的方法是在多个不同组群的对象之间来共享使用同一个排列序列，而不是每个组的对象使用其自己的序列。每个对象组应该使用同一个序列中不同的数据项，这样，每个对象组的行为完全不同于其他组的行为。举个例子，如果我们有两组对象都使用表 2.4.1 中的那个序列，那么一组对象可以使用其中深灰色的数据项，而另外一组则可以使用那些浅灰色的数据项。

如果游戏开发人员和游戏策划可以对这些特征进行深入、量化的研究，就能够给玩家带来更好的游戏体验。

2.4.5 组合序列

组合序列就是从很多的符号中选择少数几个符号所得到的结果。给定一个有 n 个符号的集合，从中选择 r 个符号（其中 $r \leqslant n$），所得到的结果就组成了一个组合序列。组合序列的每一个数据项表示着独一无二的符号的选择。例如，从有序集 $\{a, b, c, d\}$ 中的 4 个符号中选取 2 个符号，就会得到下面这个序列：

Download at www.Pin5i.Com

$$\mathbf{S}_C = \begin{Bmatrix} ab \\ ac \\ ad \\ bc \\ bd \\ cd \end{Bmatrix}$$

一般来说，给定一个有 n 个符号的集 **A**，从这 n 个符号中选取 r 个符号所形成的组合序列 $\mathbf{S}_c$ 长度，可以由下列公式计算得到：

$$\| \mathbf{S}_C \| = \binom{n}{r} = \frac{n!}{r!(n-r)!}$$

那么，上面那个例子的长度就是：$|\mathbf{S}_C| = \binom{4}{2} = 6$。其中的 6 个数据项，每一个都表示着一个无重复的选择。其他我们所能想到的数据项都是这些选择中某个数据项的复制品。

和前面的做法一样，给定任意一个数据项的索引值，就可以计算出组成这个数据项的所有符号。组合序列 $\mathbf{S}_C$ 的第 i 项（$0 \leqslant i < |\mathbf{S}_C|$ ）的第 j 个字符（$0 \leqslant j < r$ ）可以由下列公式计算得到：

$$S_C[i][j] = A_j[q_j]$$

其中：

$$q_j = \min\left\{ d \,|\, \text{index}_j < \sum_{k=0}^{d} \binom{n_j - k - 1}{r_j - 1}, 0 \leqslant d \leqslant n_j - r_j \right\}$$

$$\text{index}_j = i - \sum_{l=0}^{j-1} \sum_{k=0}^{q_j - 1} \binom{n_j - k - 1}{r_j - 1},$$

$$A_j = A - \bigcup_{k=0}^{j-1} \{ A_k[m] \,|\, 0 \leqslant m \leqslant q_k \},$$

$$n_j = \#A_j, \text{ and}$$

$$r_j = r - j$$

反过来，如果给定一个数据项的各个组成符号 $t[0]$ $t[1]\cdots t[n-2]$ $t[n-1]$，则该数据项的索引值可以用下列公式来计算：

$$\text{index} = \sum_{j=0}^{r-1} \sum_{k=0}^{p_j - 1} \binom{n_j - k - 1}{r_j - 1},$$

在上述公式中，$p_j = (t[j] \perp A_j)$。而 A_j、n_j 和 r_j 和我们前面定义的一样。

虽然这些运算看上去非常复杂，特别是和前面那两种序列的运算相比，就显得更为复杂。但是，这些运算的基本原理却和排列序列是相同的。在为一个数据项生成符号的时候，我们会根据 r 的大小和这个数据项的索引值，逐渐地削减可用的基础符号。无论是排列序列，还是组合序列，我们都可以用嵌套循环很容易地完成这个工作。要想了解更细节的内容，可参考随书光盘中的示范代码。

Download at www.Pin5i.Com

这里应该指出的是，在排列序列中，符号的位置是非常重要的；而在组合序列中，我们只考虑某个数据项中有哪些符号的组合，并不考虑各个符号的位置。举个例子，在组合序列中，*abc* 和 *bac* 是相同的，会被看作是同一个数据项。因此，排列序列和组合序列之间最大的差别就是，组成排列序列中的所有数据项的符号都是一样的，只是符号的位置不同；但是在组合序列中，绝对不会有两个数据项的组成符号是相同的。也就是说，在组合序列中，组成某个数据项的字符绝对不会再出现在同一序列中其他的数据项中。

在我们需要做出选择的情况下，组合序列就派上了用场。举个即时战略游戏的例子。假设在一个即时战略游戏中，要从 8 个部落中，选出 5 个敌对的部落。要在游戏中完成这种选择工作，最常见的实现方法是使用一个有种子的随机数发生器。虽然有种子的随机数发生器可以保证相同的种子就会生成相同的序列。但对于反过来的情况，这种方法就无法提供任何的保证。也就是说无法保证，当我们不想让随机序列重复的时候，这些随机序列就一定不会重复。这个问题的产生主要是因为缺少一个纯随机数发生器。所以，当想要做出随机化的选择时，并没有什么有效的方法，可以保证所得到的选择在游戏所有过去的活动中是独一无二的。

另一方面，组合序列可以提供一个有界的保证：在一个序列实例的上限数值范围内，可以保证它所生成的序列不会有任何的重复。这主要归功于组合序列的特性所在：同一个符号子集只能创建唯一的一个数据项。也就是说，肯定不会有两个数据项是从一个符号子集创建而来的。在这个序列的长度之内，所有的数据项都是独一无二的，不存在重复的数据项。因此，在一款游戏中，对于玩家在游戏过程中所有的动作和选择，可以用组合序列索引把它们不重复地“索引”起来，用来表示游戏活动。

组合序列还有一个非常有用的应用，可以用来评估一个图形中相关路径的复杂度。这个图形可以是玩家在游戏中需要去探索的一个抽象的科技发展流程图，也可以是一个需要开拓的地形图。我们可以用组合序列来表示玩家在遍历这些图形的过程中所做出的各种选择，并利用这些序列来调谐相关的游戏性。

例如，考虑图 2.4.2 中这个假设的科技发展流程图。在每个节点上，玩家都面临一系列的选择（选择集），他需要从中选择出一个，以便能够前进到结束节点。每个选择都会产生一个不同的路径，导致不同的结果。如果把这个图形中所有可能的路径都罗列出来，那么最终产生的这个序列就具备这样的特征：不会有两个途径会共享同一个选择集。这其实就是组合序列的基本特征。图 2.4.2 中已经标出了这个序列所有可能的路径，也就是{*ad*，*aeh*，*bfh*，*cg*}。在这里，要注意的是，这个序列中，有些数据项包含 3 个符号，而另外一些则只有 2 个符号。但是，这并不会妨碍这个序列的实际使用，因为我们总是可以在后面附加一个无用的符号。

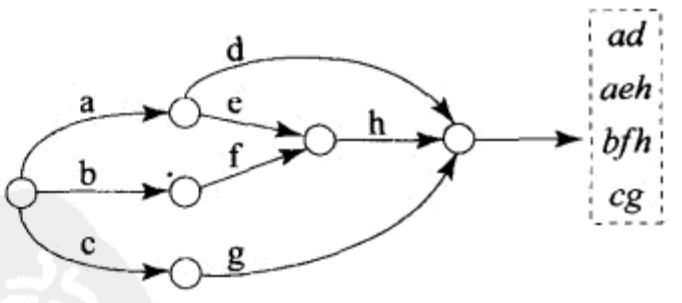


图 2.4.2

还有要注意的地方是，上面这个序列并没有包含一个组合序列中所有可能的数据项。比如，数据项 abc 就不在其中。因为对我们这个图形来说，abc 这个路径是无效的。这样，这个图形所提供的有限的选择方案，也就限制了哪些数据项可以出现在最后的结果序列中。如果将这样的限制条件最小化，那么最后的序列中就会有数量更多的有效数据项，也因此会让

Download at www.Pin5i.com

游戏变得更加的非线性化。这种枚举过程可以帮助游戏策划为游戏创建一个衡量指标，用来评估游戏提供给玩家的非线性化程度。

这种表示抽象选择的应用也向我们说明了范围序列与组合序列之间的差异。范围序列是按照程序人员定义的对象（比如：类对象）来对游戏进行编码。而组合序列则是通过玩家的选择和动作来控制游戏。例如，组合序列更适合用来表示一个完整的有效任务选择的一个子集，或者是宝藏的子集。这个子集中的财宝都是从一个可用的财宝全集中搜集来的。在玩家看来，这些选择、动作和事件更接近于游戏概念；而在程序员看来，这些选择、动作和事件并不是与固有的程序逻辑十分的吻合。

同样地，组合序列的适用范围可以被确定在游戏的活动层。例如，假设在游戏的进程中的某个时刻，不同玩家之间引发了成百上千个游戏活动。每个活动都代表着一个大型组合序列中的一个独一无二的数据项（该数据项是由一个指定玩家所做出的选择和事件组成的）。不同的游戏，以及不同的活动之间，它们相关的选择也不尽相同，因此最后生成的数据项也各不相同。但是，每一个游戏活动中都要涉及几百万个游戏帧。在这些游戏帧中，每一帧中游戏的状态可以用一个范围序列的数据项来表示（这些数据项是由程序人员所定义的类对象所组成）。因此，在活动层中，对于组合序列中的每一个数据项，相应地在游戏帧这一层，会有一个由几百万个数据项组成的范围序列。换句话说，组合序列特别适合用来编码玩家的选择（在宏观层面上），而范围序列则更适合用来编码规律性的东西（在微观层面上）。

这就将我们带到了组合序列最后一个应用上：将玩家的选择路径编码成一个组合序列，我们就可以分析这个序列，以便更好地了解玩家的策略，调谐相关的游戏性，使之可以更好地与玩家的水平相吻合。

例如，如果玩家总是沿用一个固定的选择路径，我们可以动态地调整这些选择所对应的奖励和好处，强制玩家去采用一个新的策略。这对那些带有诸如配对等特性的多人游戏来说颇为有用。如果我们想用科技树（Technology Tree，即时战略游戏中用来了解科技、技术、技能的发展走向的分支图）来调谐前面提到的即时战略游戏，组合序列的这个应用也会对我们的工作大有帮助。首先，将玩家的动作或选择进行编码，变成一个组合序列的数据项。然后，对比不同的路径，看看如果采用某个路径到底有多少个玩家可以成功或失败，根据这个信息对序列中每个数据项赋予一个成功或失败的评估值。经常发生的情况是某些选择会比其他的选择更容易成功。对于这种情况，要么确实是对玩家战术战略的反映，要么是游戏的平衡性上有了漏洞。如果能够找到这些问题，并妥善解决，就会让所有玩家的游戏体验更上一层楼。

2.4.6　总结

本文介绍了序列索引的理论。这个理论可以用来处理结构化的枚举数据，或者是根据对象的特征，对它们进行排序或索引。本文重点解释了三类重要的序列及其数学特性。对游戏开发人员和游戏策划来说，这三类序列非常有用。范围序列可以用来为那些取值范围已知的变量进行索引，比如类对象和图形统计数据。排列序列则非常适合为穷举排列来增加索引，比如有关的动画序列和格斗序列。如果要从很多的符号中选择一个符号的子集，比如一个非

Download at www.Pin5i.Com

线性游戏中玩家可以使用的任务选择，我们就可以使用组合序列。

这些序列还有另外一些更为有趣的应用：我们可以用范围序列来进行位图字体的编码，也可以用排列序列来枚举脚本代码的所有分支。通常用这些脚本代码来进行程序的调试和变换字母顺序的操作。我们可以用组合序列来实现一些特殊的任务，一些“选择性忽略式”的任务，比如人工智能代理的“概念学习”（通过有选择性地忽略一些所观察事实的某些属性，为一个观察集合做出相应的假设），或者是仿真随机式的选择行为。

说到随机选择行为的仿真，需要提醒注意的是，经常使用的有种子的随机数发生机制，就是通用的序列索引机制的一个变种。在有种子的随机数发生机制中，这个种子就是随机数序列的索引。但是，这种随机数序列与前面所探讨的那些通用序列之间，存在着一些微小的差别：随机数序列是不可能具备“解索”的功能的。也就是说，我们可以利用种子很容易地计算出一个随机数序列，但无法根据一个给定的随机数序列，来计算出所对应的种子。因此，有种子的随机数生成器可以看作是一个单向的部分索引器。

对于那些有兴趣去探索序列索引技术更深层次应用的读者朋友，我们有必要向大家介绍一下相关程序实现的注意事项。虽然本文中所提供的数学公式简单易懂，实现起来也比较容易，但是，这种原始形式的数学公式，它们的计算效率相对较低。这是因为，对于序列中某个特定的符号 **S**[*i*][*j*]，其基本形式的数学公式并不能直接引用其他的任意一个 **S**[*x*][*y*]的值。而在实际应用中，我们可以从上一个计算（也就是 **S**[*i*][*j*−1]的计算）中获得一些中间值，而这些中间值对 **S**[*i*][*j*]的计算工作大有帮助。从这一点我们可以看出，相关的计算工作还需要进行更细致的优化。本文相关的源代码为大家提供了一个中间层次的实现，为入门读者和资深读者都留出了足够的实验空间。

另外一个重点就是阶乘函数的计算。对于那些需要用到阶乘函数的公式，我们应该明白：最基本的数据类型（比如 4 个字节的整数类型）并不能容纳超过 11! 或 12!。因此，任何超过这个范围的值都需要多倍精度算术运算。因为多倍精度算术运算可以消除算术运算中所有的精度限制。为此，本书提供的源代码使用了 Gnu Multiple Precision（GMP）库。GMP 库是一个开放源代码库，可以帮助我们很容易地实现有符号整数、有理数和浮点数的多倍精度运算。具体实现的细节，请大家参见随书光盘中的源代码。

最后要说的是，本文只探讨了三种类型的序列。但是，序列索引理论博大精深，这里只能算是蜻蜓点水，还有无数的变种有待去发现并加以利用。序列索引理论横跨了信息科学的所有分支（例如：编码技术、压缩技术，以及数列分析，等等），也是“信息源程序”概念的主要贡献者[Kolmogorow65][Li97]。除此之外，序列索引理论还与确定性混沌的概念密切相关，而后者则是所有动力学系统和建模方法论都共有的一个自然特征。关于这些内容更详细的信息，可以从[Atmanspacher91]和[Atmanspacher02]中获取。这些领域中的研究探索工作可以为游戏开发工作带来一个全新的视角和工具集，把游戏开发工作和玩游戏同样都变成一个令人享受的过程。

Download at www.Pin5i.Com

2.4.7　参考文献

[Atmanspacher91] Atmanspacher, H. and H. Scheingraber, H., Eds., *Information Dynamics.* NATO ASI Series, Plenum Press, 1991.

[Atmanspacher02] Atmanspacher, A. and R. Bishop, Eds., *Between Chance and Choice.* Academic, 2002.

[Kolmogorov65] Kolmogorov, A. N., "Three Approaches to the Quantitative Definition of Information." *Problems of Information Transmission*, 1965, No. 1: pp. 1–7.

[Li97] Li, M. and P. Vitanyi, *An Introduction to Kolmogorov Complexity and its Applications.* Springer, 1997.

Download at www.Pin5i.Com

2.5 多面体浮力的精确计算

Erin Catto，Crystal Dynamics 公司
erincatto@gphysics.com

刚体的仿真技术带来了很多新的能力和新的挑战。比如说，我们可以这样假设：在一个水池边，有一座倒塌的房屋，玩家可以用房屋的残骸做成一个临时的筏子。玩家可能会选一块大小合适的墙板当筏子，并指望它能够以一种令人可信的方式漂流下去。为了实现这个场景，相应的游戏必须要逼真地仿真物体的浮力。很明显，现实主义风潮在动力学仿真领域的不断增长，使得人们对动力学仿真的期望值也必然会水涨船高。这样一来，对于游戏中一个完美的刚体仿真系统来说，如果要为玩家提供一套可玩的流体仿真机制，那么浮力的计算和仿真就是一个很重要的因素。

本文将向大家介绍一种有效的方法，来计算作用在刚体上的浮力和阻力。这个算法可以精确地计算出一个在水体中的多面体的浮力。我们将核心的公式都以向量的形式提供给大家，这样就可以针对 Intel 的 SIMD（Single Instruction, Multiple Data streams 单指令，多数据流）计算指令集进行必要的优化。

[Fagerlund]和[Gomez00]这两篇文章也对浮力的实时计算进行了类似的研究。Fagerlund 利用了一个嵌入的球体，来大致估算一个物体浸没在水下的部分。但这个方法需要额外的球体创建步骤，而且可能会需要太多的球体。Gomez 的方法是在物体的表面分布若干个点，并将一定比例的物体表面面积归结到每一个点上。然后，他再计算每个表面点上排水量的垂直柱体。他这个方法同样需要额外的创建工作，而且也需要在物体表面上放置大量的点（比如，一个立方体可能需要放置 20 或 30 个点）。

本文介绍的方法与上述方法不同。提出的这一算法不需要额外的创建工作。从几何学角度上讲，这个算法只需要知道组成多面体的顶点和三角形。但是，这个方法仅适用于平坦水面的情况。

从流体静力学的角度上看，该算法是相当精确的。因为我们忽略了水的惯性。虽然这样会让物体在水面上忽沉忽浮的效果多少显得有点不够真实，但毕竟我们这个算法比全动态水体仿真要简单得多。

2.5.1 浮力

根据阿基米德定律，一个漂浮在水面上的物体，它所受到的浮力等于该物体总排水量的重量：

Download at www.Pin5i.Com

$$\mathbf{F}_b=\rho V g\mathbf{n} \tag{2.5.1}$$

其中ρ是水的密度，V 是物体浸在水下部分的体积，g 是重力加速度，而 **n** 是向上的矢量。

如图 2.5.1 所示，浮力抵消了重力。其中，物体的重力见公式 2.5.2：

$$\mathbf{F}_g = -mg\mathbf{n} \tag{2.5.2}$$

在这里，m 是物体的质量。**x** 是物体的质心，浮力的作用中心（浮心）是 **c**。如果物体的平均密度比水的密度大，那么物体就会下沉。反过来，如果物体的平均密度小于水的密度，那么物体就会浮在水面上。

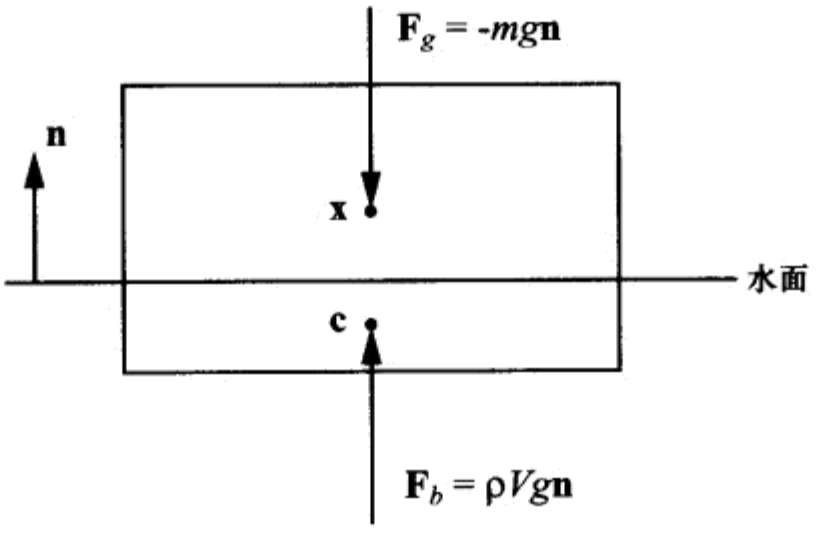


图 2.5.1　作用于同一个物体上的浮力和重力

浮力还会造成物体的摆动或上下浮动。如果一个物体坠入水中，那么它的重力上还要加上一个惯性力。这样，相应的排水量就会超过物体的重量。在浸入水中一定的深度后，物体就会加速上浮。这个过程会不断重复下去，直到物体的动能耗尽为止。

如图 2.5.2 所示，浮力会围绕物体的质心产生一个力矩。之所以会这样，是因为浮心 c 正好就是物体在水面下面那部分体积的中心点。而这个中心点并不一定会与物体的质心重合。浮力围绕物体的质心所产生的力矩可以这样计算得到：

$$\mathbf{T}_b = \mathbf{r}_b \times \rho V g\mathbf{n} \tag{2.5.3}$$

其中的“×”是叉乘积，而 $\mathbf{r}_b$ 是从质心 x 指向浮心 c 的半径矢量。

$$\mathbf{r}_b = \mathbf{c} - \mathbf{x} \tag{2.5.4}$$

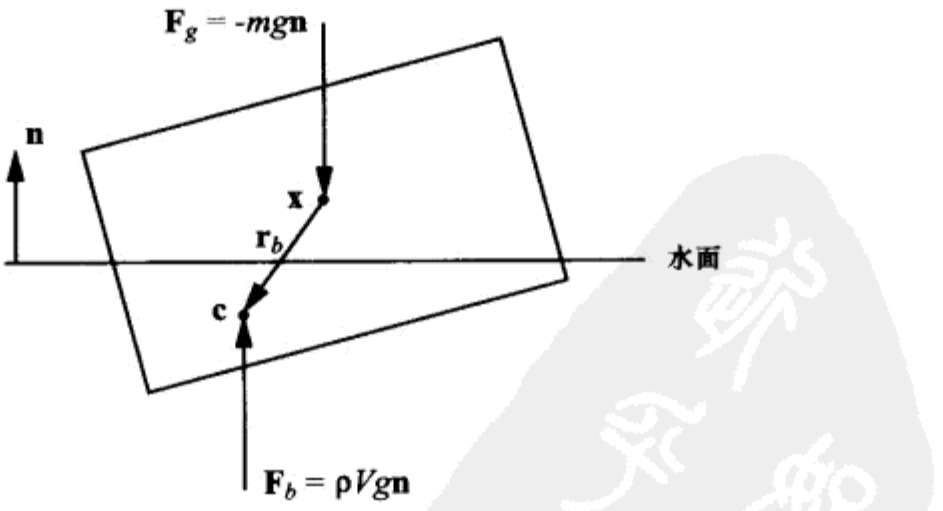


图 2.5.2　浮力力矩

Download at www.Pin5i.Com

浮力力矩使得物体在某些方向上比其他的方向更为稳定。假设物体的重量与及排水量相等，物体漂浮在水面上，我们来考虑其所有的位置和方向的一个集合。对于这样的一个集合，一个稳定的平衡结构，也就是物体质心的高度位于（局部的）最低点，才是该集合的要素。这就是为什么，一个薄木片平躺在水面上，比它立在水面上更为稳定。

2.5.2 多边形的面积

要想计算多面体的浮力，我们可以从一个更简单的问题入手，即计算一个多边形的面积。而多边形面积的计算又建立在“有符号的面积”这个概念之上。因此，我们会首先讨论这些稍微简单些的问题。

假设我们现在手头上有一个三角形，参见图2.5.3。一个三角形的有符号面积在数量级上等于一个正常三角形的面积。三角形顶点的顺序决定了相应的符号。如果三角形顶点的顺序是逆时针的，那么就会产生一个正号；而顺时针的顺序则会产生一个负号。我们将这个三角形的边向量定义如下：$\mathbf{a}=\mathbf{v}_2-\mathbf{v}_1$ 和 $\mathbf{b}=\mathbf{v}_3-\mathbf{v}_1$。大家回想一下，叉乘积 $\mathbf{a}\times\mathbf{b}$ 的长度就是图中相关联的平行四边形的面积。而这个三角形的面积就等于这个平行四边形面积的一半。如果这个多边形的平面有一个单位法线 $\mathbf{k}$，则相应的有符号的面积就是：

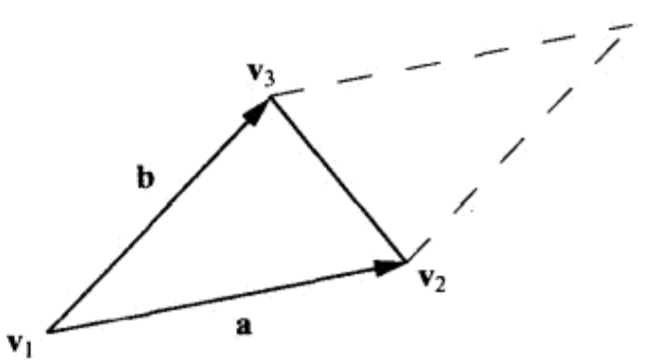


图 2.5.3 一个假想的三角形

$$A = \frac{1}{2}(\mathbf{a}\times\mathbf{b})\cdot\mathbf{k} \qquad (2.5.5)$$

这样一来，如果三角形的顶点顺序是逆时针的，则 A 就是正的；如果三角形的顶点顺序是顺时针的，则 A 就是负的。

根据[O'Rourke98]一文的理论，一个多边形的面积就等于所有相关三角形的有符号面积的总和。由任意的一个点 $\mathbf{p}$ 和这个多边形的每一个边，就构成了这些相关的三角形。因此，图 2.5.4 中这个四边形的面积就可以这样求和得到：

$$\begin{aligned} A &= A_1 + A_2 + A_3 + A_4 \\ &= A(\mathbf{v}_4,\mathbf{v}_1,\mathbf{p}) + A(\mathbf{v}_3,\mathbf{v}_4,\mathbf{p}) + A(\mathbf{v}_1,\mathbf{v}_2,\mathbf{p}) + A(\mathbf{v}_2,\mathbf{v}_3,\mathbf{p}) \end{aligned} \qquad (2.5.6)$$

注意，前两个三角形的面积是正的（顶点顺序是逆时针方向），后面两个三角形的面积是负的（顶点顺序是顺时针方向），而最后的总和是正的。还要注意的是相关的三角形是如何形成的：将 $\mathbf{p}$ 作为最后一个顶点，以逆时针方向连接多边形的每一条边。

每个三角形的质心就是其所有 3 个顶点的平均值，比如：

$$\mathbf{c}_1 = \frac{1}{3}(\mathbf{v}_1 + \mathbf{v}_2 + \mathbf{p}) \qquad (2.5.7)$$

而多边形的质心就是所有相关三角形质心的面积加权平均值。因此，图 2.5.4 中那个多边形的质心就是：

$$\mathbf{c} = \frac{1}{A}(A_1\mathbf{c}_1 + A_2\mathbf{c}_2 + A_3\mathbf{c}_3 + A_4\mathbf{c}_4) \qquad (2.5.8)$$

Download at www.Pin5i.com

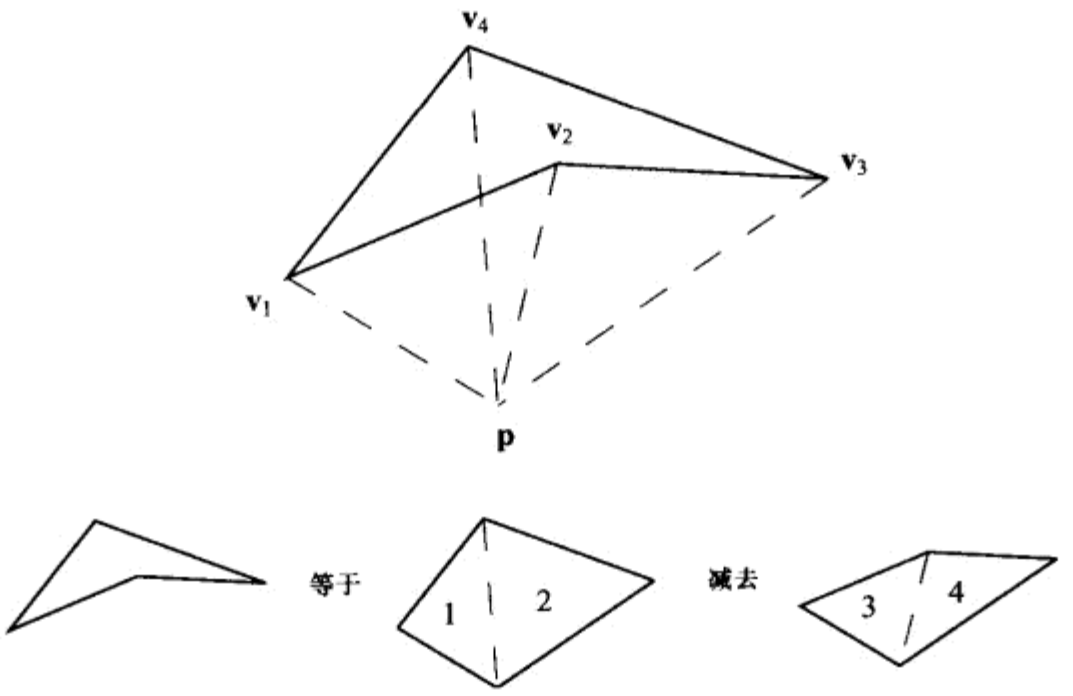


图 2.5.4　一个多边形的面积就等于所有相关三角形有符号面积的总和。原多边形的面积等于三角形 1 和三角形 2 的无符号面积，减去三角形 3 和三角形 4 的无符号面积。

由于我们使用的是有符号的面积，所以，某些质心可能会是一个负的加权值。

2.5.3　多面体的体积

在这里，先假设多面体是完全没入水中的。在后面的内容中，我们再探讨最常见的情况，就是多面体部分地没入水中的情况。计算多面体的体积，我们使用的方法与计算多边形的面积所使用的方法很类似。多边形的面积就是相关三角形的有符号面积的总和；而多面体的体积就是其相关的四面体的有符号体积的总和。每个相关的四面体都是由该多面体的一个三角面和任意指定的一个点 **p** 所组成的。如果点 **p** 位于这个三角面的后面，则体积的符号是正号；如果点 **p** 位于这个三角面的前面，则体积的符号是负号。参见图 2.5.5。

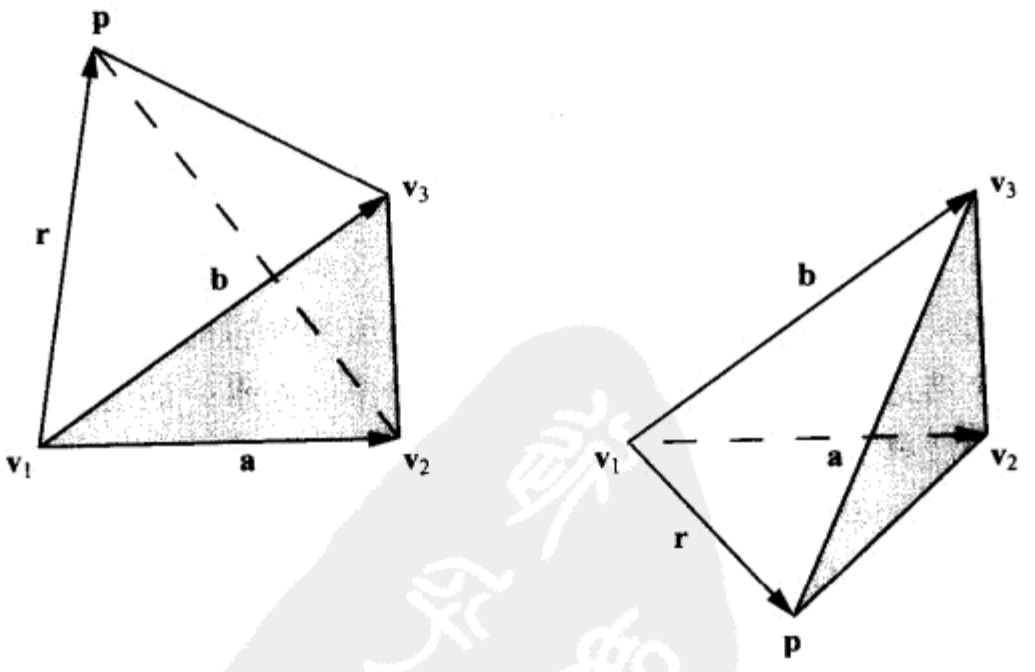


图 2.5.5　两个多面体。左面的多面体，其体积是正的（点 **p** 位于后部）；而右面的多面体，它的体积是负的（点 **p** 位于前部）

Download at www.Pin5i.com

我们将[Weisstein]一文中的公式进行了扩展，就可以计算出一个四面体的有符号的体积：

$$V=\frac{1}{6}(\mathbf{b}\times\mathbf{a})\cdot\mathbf{r} \tag{2.5.9}$$

其中，$\mathbf{a}=\mathbf{v}_2-\mathbf{v}_1$、$\mathbf{b}=\mathbf{v}_3-\mathbf{v}_1$，而 $\mathbf{r}=\mathbf{p}-\mathbf{v}_1$。很简单，和三角形的情况一样，一个四面体的质心就是其所有顶点的平均值：

$$\mathbf{c}=\frac{1}{4}(\mathbf{v}_1+\mathbf{v}_2+\mathbf{v}_3+\mathbf{p}) \tag{2.5.10}$$

更详细的信息请大家参阅[Weisstein]一文。

与二维的情况类似，多面体的体积就是其所有相关四面体有符号体积的总和。而多面体的质心就是其所有相关四面体的质心的加权平均值。

$$V=\frac{1}{6}\sum_{i=1}^{n}(\mathbf{b}_i\times\mathbf{a}_i)\cdot\mathbf{r}_i \tag{2.5.11}$$

$$\mathbf{c}=\frac{1}{V}\sum_{i=1}^{n}V_i\mathbf{c}_i \tag{2.5.12}$$

由于这些公式都是以向量的形式给出的，所以我们可以很容易地在 SIMD 硬件上对它们进行优化。参见程序清单 2.5.1。

程序清单 2.5.1　下列代码用来计算一个四面体的体积和质心

```
float TetrahedronVolume(Vec3& c, Vec3 p, Vec3 v1, Vec3 v2, Vec3 v3)
{
   Vec3 a = v2 - v1;
   Vec3 b = v3 - v1;
   Vec3 r = p - v1;

   float volume = (1.0f/6.0f)*dot(cross(b, a), r);
   c += 0.25f*volume*(v1 + v2 + v3 + p);
   return volume;
}
```

2.5.4　物体部分没入水中的情况

1. 简介

对于物体部分没入水中的情况，首先让我们考虑二维的情形，如图 2.5.6 所示。顶点 $\mathbf{v}_1$、$\mathbf{v}_2$ 和 $\mathbf{v}_3$ 没入水中，而顶点 $\mathbf{v}_4$ 则位于吃水线之上。为了计算物体没入水下部分的面积，我们必须要用吃水线将多边形进行分割。分割之后就会生成新的多边形，如图 2.5.7 所示。

图 2.5.7 中这个多边形的面积就是原来那个多边形没入水中的面积。为了计算这个没入水中的面积，我们要选择一个点 $\mathbf{p}$，它要分别与这个多边形所有的边形成若干个三角形。我们将点 $\mathbf{p}$ 置于吃水线上，那么三角形（$\mathbf{v}_4$、$\mathbf{v}_5$ 和点 $\mathbf{p}$）的面积就变为 0。这样一来，我们就从面积总和的公式中去掉了一个项，使算法更加高效。

由于那条位于吃水线上的边对水下部分的面积没有任何影响，我们就可以将修剪算法进行简化。我们可以独立地修剪每一条边。对于任意一条边，只有三种情况：位于吃水线之上、

Download at www.Pin5i.Com

位于吃水线之下，或者是与吃水线交叉。位于吃水线上面的边对水下面积的计算没任何效用。位于吃水线下面的边直接影响水下面积的计算。最后一种情况，那些与吃水线交叉的边，只有吃水线下面的部分才对水下面积的计算有作用。

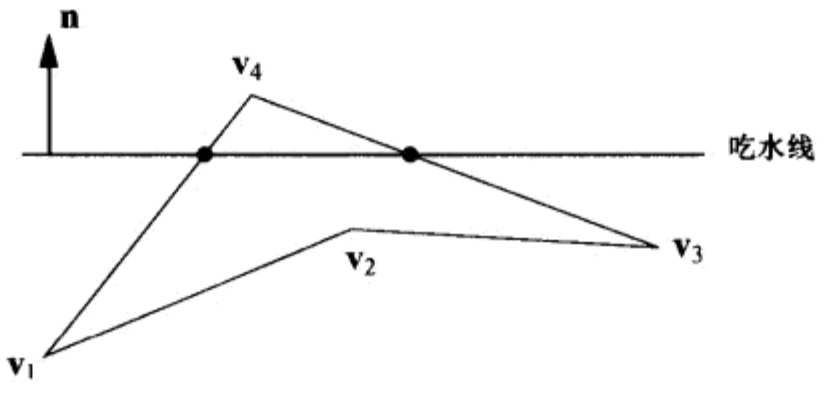


图 2.5.6　一个部分没入水中的多边形

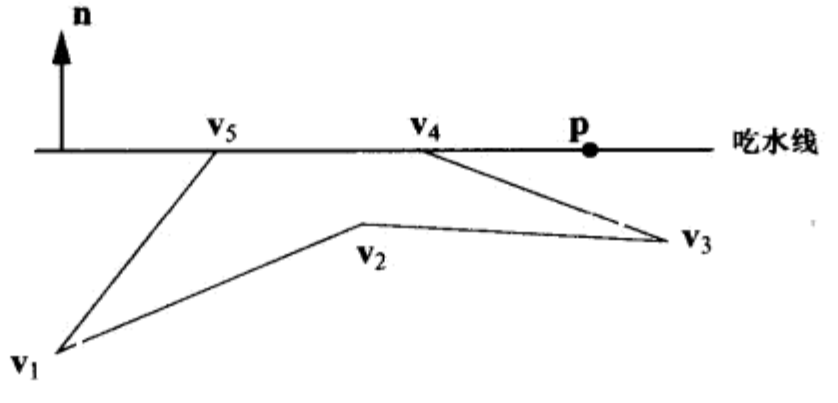


图 2.5.7　沿着吃水线将多边形进行裁剪分割

而对于三维的情况，我们就要用水平面对多面体的每一个三角面进行裁剪。裁剪的过程会在水平面上留下一个复杂的形状。它可能是若干个多边形，而这些多边形中间也可能会有一些孔洞。如果我们将点 **p** 置于水平面上，我们就不用再去考虑那些位于水平面之上的面了。这样就可以大大地简化我们的这个三维算法。

2. 裁剪过程

所有的三角形都属于下列三种类型之一：

1. 位于水平面上方；
2. 位于水平面下方；
3. 与水平面相交。

第一种类型中的三角形对于水下体积没有任何贡献。第二种类型中的三角形对于体积的计算有着直接的影响。对于类型 3 中的三角形，必须用水平面进行裁剪，产生一个或两个属于第二种类型的三角形。

[Eberly01]一文中提出了一个算法，用平面来裁剪三角形。虽然他的这个算法主要是应用于图像渲染中的视锥裁剪过程，但也可以很好地应用于浮力的计算。我们这里使用了该算法的一个修改版本，针对我们的浮力计算进行了优化。

我们来考虑一个三角形与水平面交叉的情况。这可能会出现两种情况，如图 2.5.8 所示。

情况 A：只有一个顶点位于水平面下方。这样会产生一个属于第二种类型的三角形。

Download at www.Pin5i.com

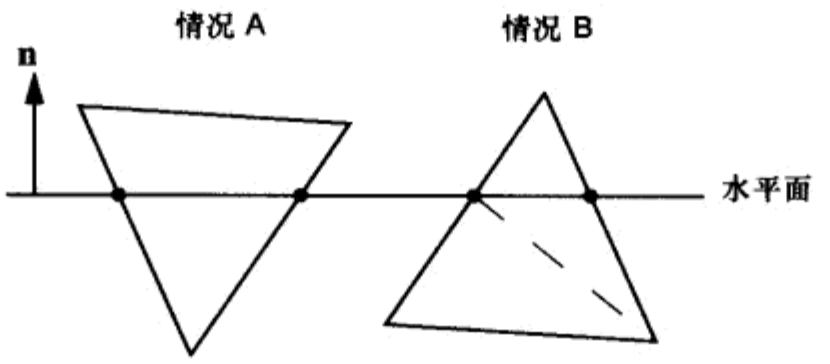


图 2.5.8 两种不同的裁剪情况：情况 A 和情况 B

情况 B：两个顶点位于水平面下方。这样会产生一个四边形。这个四边形由两个属于类型 2 的三角形构成。

对于情况 A，我们要确定两个裁剪点，然后再处理裁剪生成的那个属于类型 2 的三角形。对于情况 B，我们同样要确定两个裁剪点，并把生成的四边形当作 2 个类型 2 的三角形来处理。

一个典型的图形处理流程会根据视锥对三角形进行裁剪，然后将生成的三角形添加到已渲染三角形列表中。我们这里则有些不同，在我们的浮力算法中，我们是根据水平面对三角形进行裁剪，然后直接在体积的计算中使用生成的三角形。算法本身并不会去更新三角形列表。这样既简化了代码，也节省了很多的内存。

在体积计算的开始部分，相关的代码要计算出每个顶点的深度 d，并将结果保存在一个数组中。主循环要处理每一个三角形，并检查三角形三个顶点的深度。如果发现某个三角形属于类型 2，就将该三角形的顶点及其深度传递给三角形裁剪器。请大家参见程序清单 2.5.2。裁剪器首先判断这个三角形的配置（是属于情况 A，还是属于情况 B），然后再通过线性插值完成裁剪工作，并将新产生的三角形传递给四面体体积计算函数。

程序清单 2.5.2 用于三角形裁剪的代码

```
float ClipTriangle(Vec3& c, Vec3 p,
    Vec3 v1, Vec3 v2, Vec3 v3,
    float d1, float d2, float d3)
{
    Vec3 vc1 = v1 + (d1/(d1 - d2))*(v2 - v1);
    float volume = 0;

    if (d1 < 0)
    {
        if (d3 < 0)
        {
            Vec3 vc2 = v2 + (d2/(d2 - d3))*(v3 - v2);
            volume += TetrahedronVolume(c, p, vc1, vc2, v1);
            volume += TetrahedronVolume(c, p, vc2, v3, v1);
        }
        else
        {
            Vec3 vc2 = v1 + (d1/(d1 - d3))*(v3 - v1);
```

Download at www.Pin5i.Com

```
                volume += TetrahedronVolume(c, p, vc1, vc2, v1);
            }
        }
        else
        {
            if (d3 < 0)
            {
                Vec3 vc2 = v1 + (d1/(d1 - d3))*(v3 - v1);
                volume += TetrahedronVolume(c, p, vc1, v2, v3);
                volume += TetrahedronVolume(c, p, vc1, v3, vc2);
            }
            else
            {
                Vec3 vc2 = v2 + (d2/(d2 - d3))*(v3 - v2);
                volume += TetrahedronVolume(c, p, vc1, v2, vc2);
            }
        }
        return volume;
    }
```

2.5.5　算法的鲁棒性

下面考虑一种特殊情况，假设有一个多面体只是微微没入水中。那么，用前面的裁剪方法，只能得到一个小细条。由于存在四舍五入的误差，对这个多面体的体积进行求和时，就会产生一个负的体积。而且，质心可能不一定会位于这个多面体水下的那部分。虽然这些误差相对是比较微小的，但我们认为有必要处理好这些问题，以避免算法出现伪结果。

对于四面体的体积计算公式，其计算的精度取决于四面体的品质。高品质的四面体拥有平衡的内角。而对于低品质的四面体，其内角之间在数值上差异很大。

先来考虑二维的情况，参见图 2.5.9。可以注意到这是个低品质的三角形，因为它的三个内角之间相差太多。其面积的计算公式如下：

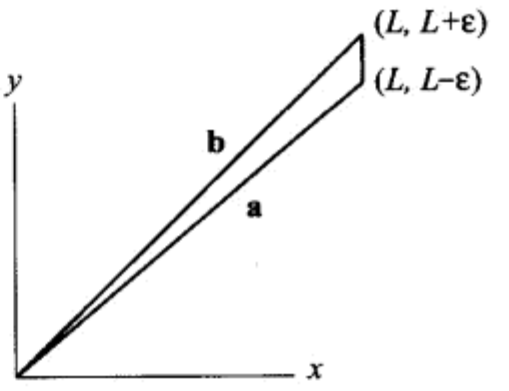


图 2.5.9　一个细长的三角形

$$
\begin{aligned}
A &= \frac{1}{2}(\mathbf{a}\times\mathbf{b})\cdot\mathbf{k} \\
&= \frac{1}{2}(a_x b_y - a_y b_x) \\
&= \frac{1}{2}[(L^2 + L\varepsilon) - (L^2 - L\varepsilon)]
\end{aligned}
\tag{2.5.13}
$$

这个三角形的准确面积是 $L\varepsilon$。现在假设 L 的值比较大，而ε的值很小。那么公式中的叉乘积就取决于两个大数之间的差，最后会产生一个相对很小的值。这个计算结果容易产生舍入误差，因为 L^2 这一项占主导地位。

很明显，四面体的品质取决于点 **p** 的具体放置位置。虽然前面已经决定将点 **p** 置于水平面上，但并没有指定它在水平面上具体的位置。我们可以将点 **p** 直接放置到多面体水下部分

Download at www.Pin5i.com

的上方，这样就有可能会提高四面体的品质。为了获得这种品质上的提升，我们需要将一个在水下面的顶点投影到水平面上。在我们的实现中，被投影的顶点是从水下顶点集中任意选择的。

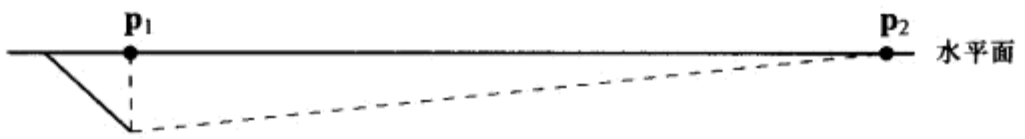


图 2.5.10 这个二维例子展示了点 **p** 位置的选择如何来影响三角形的品质。如果选择的是 $\mathbf{p}_1$，那么就会生成一个高品质的三角形，因为其三个内角是充分平衡的。如果选择的是 $\mathbf{p}_2$，则我们得到的就是一个低品质的三角形，因为它的内角是不平衡的。而对于三维的情况，点 **p** 位置的选择对四面体的质量有着类似的影响

即使 **p** 位置是最优的，四面体体积的计算公式也仍然会出现不精确的情况。在代码中增加一些防护机制可以帮助我们避免出现不正确的结果。首先，如果多面体在水下的部分非常小，那么我们就放弃浮力的计算。其次，如果计算出来的总体积是负的，我们也要放弃浮力的计算。

2.5.6 阻力

当浮力和重力一起作用于物体，会使物体产生沉沉浮浮的上下振动。在实际情况中，由于阻力的存在，这个振动才会逐渐消失。阻力也使得水流可以移动物体。

由水施加在一个刚体上的阻力是非常复杂的。它们取决于刚体的表面特性和刚体的形状。如果想获得一个精确的阻力模型，就需要一个全动态水体仿真系统。考虑到相关的计算开销，这种精度的阻力计算是否会明显地提高最终效果的视觉真实感，这是值得商榷的。因此，这里会使用一个近似的阻力模型。这个模型使用我们前面的浮力计算结果，而且计算开销也相对较小。

下面这个就是阻力的计算公式：

$$\mathrm{F}_d = \beta_l m \frac{V}{V_T}(\mathrm{v}_w - \mathrm{v}_c) \tag{2.5.14}$$

其中，β_l 是线性阻力系数，以 1 为单位，随着时间而变化；m 是多面体的质量；V 是多面体水下部分的体积；V_T 是多面体的总体积；$\mathbf{v}_w$ 是水流的速度，而 $\mathbf{v}_c$ 是浮力中心（浮心）的速度。为了简单起见，我们假设阻力 F_d 作用于浮力中心（浮心）。

阻力 F_d 的方向与浮心相对于水流的运动方向相反，阻碍着物体的运动。阻力的计算公式有很多，但公式 2.5.14 的实用效果却是非常好的。

单独一个阻力 F_d 并不足以消散角速度，特别是当浮力中心（浮心）直接位于质心下面的时候。所以，加入一个阻力力矩是颇有帮助的：

$$\mathrm{T}_d = -\beta_a m \frac{V}{V_T} L^2 \omega \tag{2.5.15}$$

在这里，β_a 是有角阻力系数，以 1 为单位，随着时间而变化。L 是多面体的平均宽度。这些参数可以确保产生力矩的单位。

阻力系数 β_l 和 β_a 的值可以通过数值实验来选定。首先，将阻力系数设为 0，然后用一个有代表性的多面体（例如一个盒体）来运行浮力仿真程序。这个盒体应该以一种稳定的

Download at www.Pin5i.Com

方式左右摆动。如果它不稳定，那么就逐渐减小数字积分器中的时间步长，直到这个盒体稳定下来。接下来逐渐增加β_l的值，直到经过几个时间周期后，盒体的上下振动消失为止。然后，用一个初始的角速度（围绕垂直轴）使这个盒体下落。在上下振动消失之后，这个盒体应该会继续旋转。最后，我们逐渐增加β_a的值，直到经过几次旋转后，旋转最终消失为止。

2.5.7　关于源代码

随书光盘中提供了相关的源代码和一个演示程序。代码关注的重点更多的是在代码的清晰性上，而不是代码的效率，所以你可以对它进行若干的优化。图 2.5.11、彩插 4 和彩插 5 是这个演示程序的截屏图。

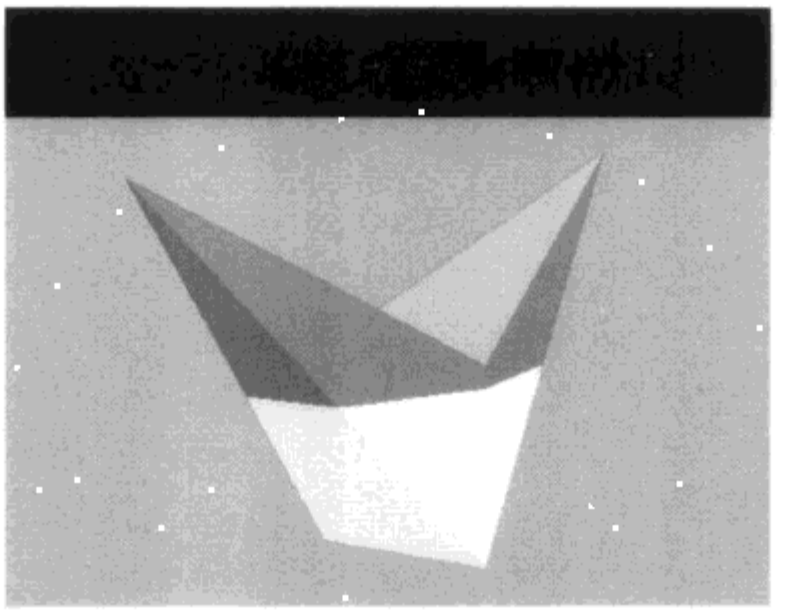


图 2.5.11　一个凹多面体的浮力仿真

2.5.8　总结

本文介绍了，对于平滑表面的水体，如何精确地计算出一个浸入其中的多面体的流体静力学浮力。除此之外，还提出了阻力和阻力力矩的近似模型，用以仿真能量损耗和水流耦合。我们这个算法效率很高，不需要额外的创建步骤，而且易于实现，也可以很容易地集成到物理引擎中。

2.5.9　致谢

在此，笔者要感谢 Crystal Dynamics 公司的团队为本文的创作所提供的支持。

2.5.10　参考文献

[Eberly01] Eberly, David H., *3D Game Engine Design.* Morgan Kaufmann, 2001.

Download at www.Pin5i.com

[Fagerlund] Fagerlund, Mattias, "Buoyancy Particles or Bobbies." Available online at *http://www.cambrianlabs.com/Mattias/DelphiODE/BuoyancyParticles.asp*.

[Gomez00] Gomez, Miguel, "Interactive Simulation of Water Surfaces." *Game Programming Gems,* Charles River Media, 2000.

[O'Rourke98] O'Rourke, Joseph, *Computational Geometry in C,* 2nd Ed. Cambridge University Press, 1998.

[Weisstein] Weisstein, Eric W., "Tetrahedron." Available online at *http://mathworld.wolfram.com/ Tetrahedron.html*.

Download at www.Pin5i.Com

2.6　带有刚体交互作用的基于粒子的实时流体仿真系统

Takashi Amada，索尼计算机娱乐公司
Taka.am@gmail.com

对流体的运动行为进行逼真的、实时的渲染，这是让玩家沉浸于交互式应用（例如计算机游戏）的方法之一。而流体与刚体之间的交互作用也是非常重要的，因为在现实世界中，流体和刚体之间互相影响，而这种相互作用也影响着它们各自的运动行为。基于计算流体动力学（Computational Fluid Dynamics，简称 CFD）的流体仿真是一种非常有效的方法，可以渲染表现出流体那些在视觉上似是而非的运动行为。但是，对于那些需要快速仿真的实时流体渲染来说，很多的 CFD 技术，其相应的计算开销通常都过于庞大。而且，很多传统的技术并不能非常容易地实现流体与刚体之间交互作用的仿真。

本文将介绍一种新的方法，利用平滑粒子的流体动力学技术，来仿真流体、流体与刚体之间的交互作用等。同时还提供了一个快速的实现代码，这个方法可以实时仿真水体与刚体之间的交互作用。

2.6.1　流体仿真与平滑粒子的流体动力学

1. 流体仿真的基本方法

大家或许已经听说过用来描述广义流体流运动的纳维叶-斯托克司方程。对于不可压缩的流体，例如水体，此处给出纳维叶-斯托克司方程的一个有效版本，参见公式 2.6.1。这个偏微分方程描述了不可压缩流体的动量守恒，等价于牛顿运动第二定律。

$$\rho\frac{D_{\mathbf{v}}}{D_t} = -\nabla p + \mu\nabla^2\mathbf{v} + \rho\mathbf{f} \tag{2.6.1}$$

在这个公式中，ρ是流体的密度（这是一个标量），$\mathbf{v}$ 是流体的速度（这是一个向量），p 是流体的压力（是一个标量），μ是流体的粘性系数（也是一个标量，这里我们假设它是一个常数），$\mathbf{f}$ 是作用在粒子每个单位质量上的外力（例如重力），∇ 是公式 2.6.2 所定义的向量梯度算子，而 ∇^2 则是公式 2.6.3 所给出的拉普拉斯算子。

Download at www.Pin5i.Com

$$\nabla = \mathrm{i}\frac{\partial}{\partial x} + \mathrm{j}\frac{\partial}{\partial y} + \mathrm{k}\frac{\partial}{\partial z} \tag{2.6.2}$$

$$\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2} \tag{2.6.3}$$

公式 2.6.1 还有很多的简化版本。为了能够进行流体的仿真，必须要仿真公式 2.6.1，或者是仿真它的任意一个简化的版本，并结合其他一些关键的控制方程，包括连续性（质量守恒）和能量守恒。对于刚体而言，只要用 6 个自由度（也就是围绕着三个轴，在每个轴上进行的平移和旋转变换）就可以表示出刚体的运动。与刚体不同的是，无论流体的形状如何复杂，它都有无穷数量的自由度，无法用刚体来简单地表示。为了有效地仿真流体运动的有关控制方程，我们必须要仔细地考虑流体的天然特性。

流体（比如水）是由大量的分子所组成的。即使是体积非常小的一滴流体，其中也包含着数量巨大的分子。所以，在大多数情况下，如果为了仿真流体，而将流体建模为独立分子的一个集合，这种做法有些不切实际。从宏观的角度上看，我们可以将水或其他的流体看成是一个连续流。对于这个连续流，其物体特性的变化在其整体内部是连续的。而实际上，纳维叶-斯托克司方程就是基于这个观点才推导出来的——流体是连续的。如果我们接受这个观点，那么我们就可以在现代的计算机系统上，在合理的运行时间之内，创建出逼真的流体仿真。

所有的流体仿真都可以归结为以下两类：欧拉仿真和拉格朗日仿真。在欧拉仿真中，流体的物理特性是用参考帧中某些固定的点来表示的。这些点并不会随着流体一起运动（可是如果参考帧发生了运动或变形，这些点也会在空间中运动）。对于一个给定的点，当流体流过这个点后，它的物理特性也随之发生了变化。创建一个欧拉式的流体仿真，最典型方法是：首先创建一个网格，比如图 2.6.1 中的那个网格。这个网格不一定必须是均衡的，也不用对网格的列或行进行特别的安排。流体的属性就保存在网格的这些顶点中。我们可以用各种不同

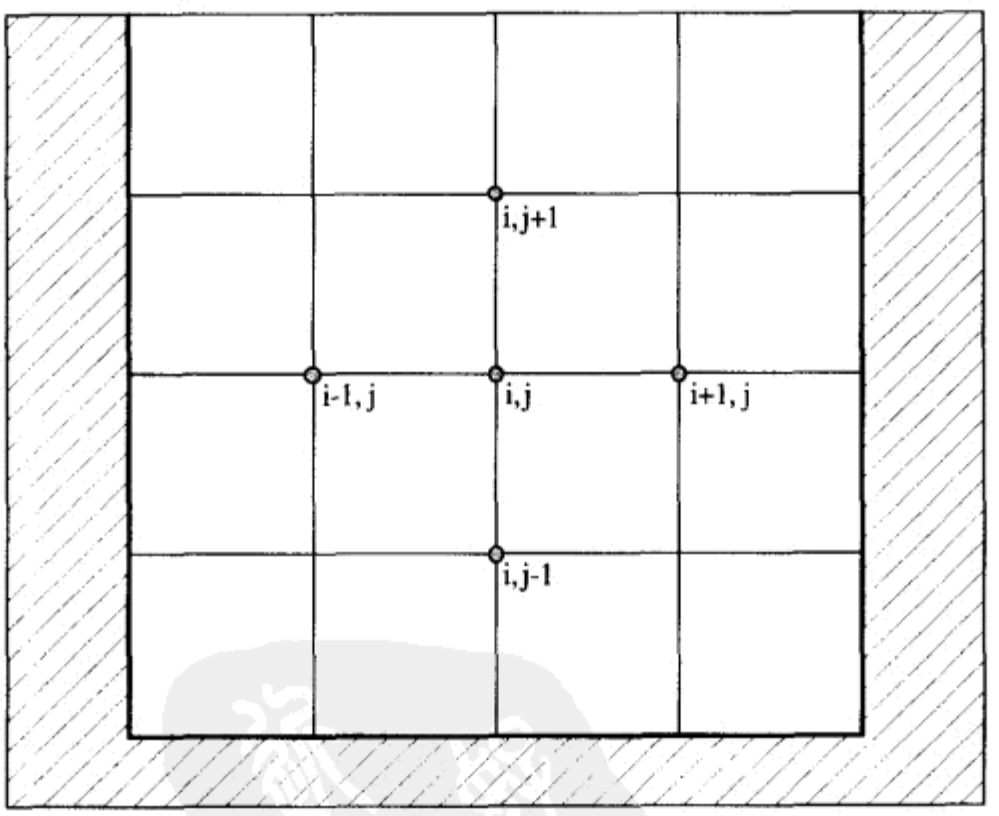


图 2.6.1 二维欧拉流体仿真所使用的一个简单的网格。被仿真的对象是蓄水池中的水体

Download at www.Pin5i.com

的方法，将网格有关的控制方程离散化，比如：有限差，或者是有限体积（finite-volume）方法。仿真程序就使用这些离散方程，一帧接一帧地更新顶点的属性。

已经有些开发人员创建了欧拉仿真，用来在游戏中对流体进行建模。例如，Jos Stam 提出了欧拉仿真中著名的稳定流体技术，使用粒子跟踪法，来获得必需的稳定性[Stam99]。甚至在 GPU 产品中，业界也已经实现了欧拉仿真[Noe04]。

欧拉仿真法确实有一些优势。欧拉仿真不但实现起来非常容易，而且还可以产生比较平滑的效果。但是，在某些情况下，欧拉仿真法就略显不足。虽然实现一个基本的仿真系统是非常容易的，但在实际应用中，却很难得到完美的效果，特别是当仿真系统需要任意的边界，或者是有什么其他的力作用在流体上的时候。如果想要在流体仿真中获得最完美的细节，比如漩涡或涡流，相关的内存开销和计算开销会让人望而却步。另外，欧拉仿真法也不是特别适合用来建模飞溅的流体，或者是积蓄在池子中的流体；也不适合建模两种不同类型流体的混合体，或者是那些与任意刚体交互作用的流体。

在拉格朗日仿真中，我们要将流体的某个特定部分的物理特性表示出来，并且在这个部分随着时间进行运动时，还要跟踪这个部分的物理特性。与欧拉仿真法类似，我们也可以用一个网格来实现拉格朗日仿真。这个时候，网格单元（对于二维网格而言，网格单元指的就是组成网格的三角形或四方格；而对于三维网格，网格单元指的是四面体或其他类似的东西）实际上是随着流体而运动的，而且在仿真的过程中，每个网格单元的质量保持恒定。与基于网格的欧拉仿真一样，基于网格的拉格朗日仿真程序也面临着同样的难题。例如，如果想让网格单元变形，同时还要使网格单元中的流体质量保持恒定，并且要满足纳维叶-斯托克司方程和能量守恒定律，这是非常困难的。在实际应用中，这样通常会让整个流体的运动行为变得特别的不真实。

除了网格表示方法，还可以利用流体的粒子表示方法，来实现拉格朗日仿真。从概念上讲，基于粒子的拉格朗日仿真与单个分子的粒子仿真非常类似。每个拉格朗日粒子表示的就是一组特定的分子，比如流体的某个特定的部分，参见图 2.6.2。

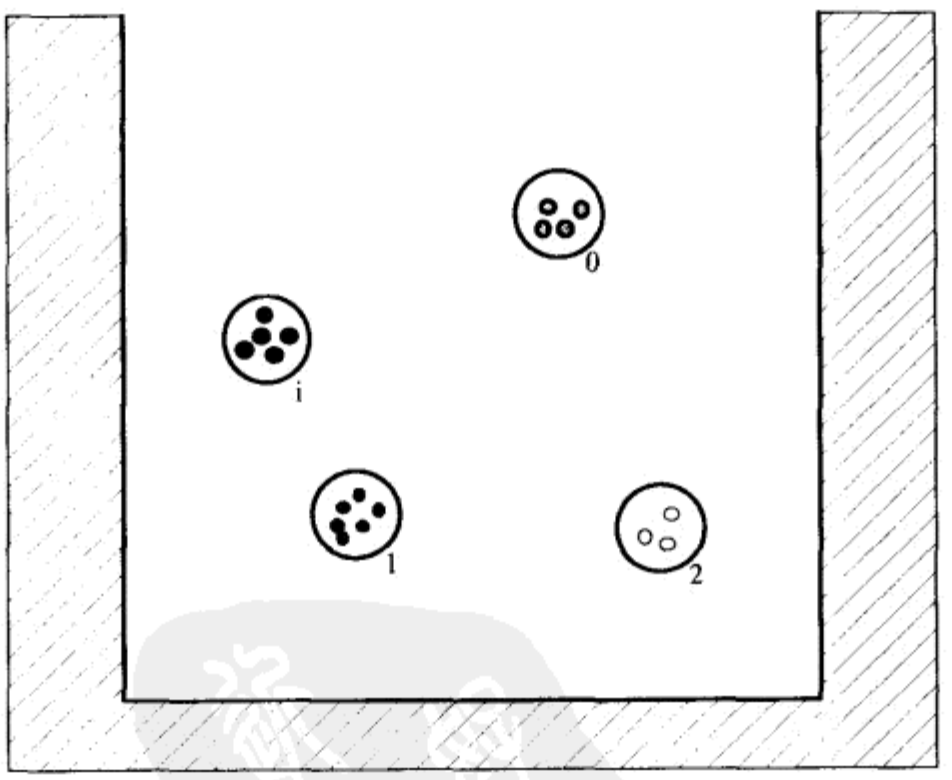


图 2.6.2　基于粒子的拉格朗日流体表示法

Download at www.Pin5i.Com

当某个粒子运动的时候，它的运动方式就代表着它在概念上所包含着的那些分子的运动方式。从统计学的角度来看，由一个给定的粒子所表示的那些分子，在仿真过程中的所有帧中，它们都仍然会逗留在这个粒子的附近，如图 2.6.3 所示。注意，对基于粒子的拉格朗日仿真方法，我们永远无法去直接判断出每个粒子中有多少个分子。我们会为每个粒子指定一个初始质量，并且在所有的帧中，这个质量会一直保持恒定。这样一来，连续性很自然地得到了满足，也可以很安全地忽略这个控制方程了。而且，要想表现出不同类型的流体混合在一起的情形，这个控制方程也是无足轻重的。

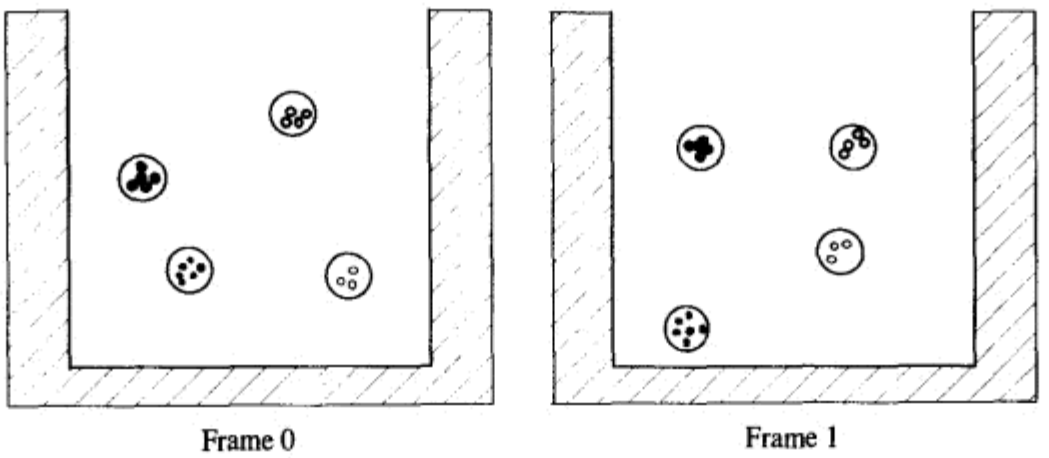


图 2.6.3 拉格朗日观点下的流体随时间的推化过程

有关的控制方程都是基于连续流的假设而推导出来的，但是基于粒子的拉格朗日仿真并不是完全将流体看作是一个连续流。你也许会想，我们可以使用一个传统的刚体－粒子碰撞反应。当然，你确实可以使用刚体—离子碰撞反应。但是，由于存在着连续压力和黏性力，不同的流体其运动行为也是不同的。因此，使用这样的方法（刚体—粒子碰撞反应）并不现实，也不会产生逼真的效果。为了获得真实的仿真效果，必须将流体的某些表示看作是连续流来使用，以便处理拉格朗日粒子之间的交互作用。一个连续流表示可以生成公式 2.6.1 中所体现的那些物理特性的导函数。这些导函数负责驱动仿真程序向前发展。平滑粒子流体动力学方法为我们提供了一个简单的方法，不需要使用网格，就可以计算出仿真所必需的流体的物理特性以及它们的导函数。

2. 利用平滑粒子流体动力学实现流体的仿真

正如前面所提到的，对于基于粒子的拉格朗日仿真法，其使用的粒子是有质量的，而且这个质量在整个仿真过程中一直保持恒定，因此我们才能安全地忽略连续性方程。连续性是自动满足的。此外，由于假设的是一个粘性恒定的不可压缩的流体，我们同样可以安全地忽略能量方程。这是从纯流体运动中分离出来的一种情况。

现在还剩下一个方程需要我们去仿真，那就是纳维叶-斯托克司方程（公式 2.6.1）。从拉格朗日仿真法的角度来看，纳维叶-斯托克司方程左边的项 Dv/Dt，就是一个给定流体粒子速度的时间变化率，也就是流体粒子的加速度。如果对于一个给定的粒子，我们可以计算出密度和纳维叶-斯托克司方程右侧的各个项，就可以计算出该粒子的加速 Dv/Dt，然后再计算出作用在粒子上的合力。接下来，就可以利用传统的粒子动力学技术来更新粒子的位置。这就是平滑粒子流体动力学（SPH）方法的工作原理。

平滑粒子流体动力学（SPH）方法是一个简单的 CFD（计算流体动力学）技术。通过对

Download at www.Pin5i.Com

相邻粒子的粒重物理值求和，我们可以插值计算出流体的物理值。这与光子贴图的工作原理非常类似。

利用公式 2.6.4，平滑粒子流体动力学（SPH）方法可以表示出在任意一个位置 **r** 上的插补值 $A(\mathbf{r})$。积分域是整个流体体积。

$$A(\mathbf{r}) = \int A(\mathbf{r}')W(\mathbf{r}-\mathbf{r}',h)d\mathbf{r}' \tag{2.6.4}$$

特性 $A(\mathbf{r})$ 可以是任何一个连续流体的物理特性，比如：密度，或者是压力。函数 $W(\mathbf{r}, h)$ 是一个核函数，具有下列两个特性：

$$\begin{aligned} &\int W(\mathbf{r}-\mathbf{r}',h)d\mathbf{r}' = 1 \\ &\lim_{h\to 0} W(\mathbf{r}-\mathbf{r}',h) = \delta(\mathbf{r}-\mathbf{r}') \end{aligned} \tag{2.6.5}$$

其中，h 是这个核函数的有效范围；而 δ 是Δ（delta）函数，当（$\mathbf{r}-\mathbf{r}'$）等于 0 时，这个函数的值等于 1，否则就等于 0。在进行仿真的时候，利用公式 2.6.4 进行插值的物理特性，根据各自独有的不同特征，对于不同的流体特性，我们应该使用不同的核函数。为了简单起见，在对公式 2.6.4 进行求值时，我们可以简单地使用离散数量的粒子的总和，而不是计算连续的积分。

Muller 等人在[Muller03]一文中详细地介绍了如何利用 SPH 方法来实现稳定的、快速的流体仿真。他们的文章精确地提出了若干个核函数，以及前面推荐使用的离散求和的方法。我们会在后面的文章中介绍支持刚体交互作用的扩展方法，而[Muller03]一文就是这个方法的基础所在。

应该注意到，虽然使用的是不可压缩形式的纳维叶-斯托克司方程，但是在仿真的过程中，粒子的密度却是变化的。这是插值操作造成的后果。而且，虽然这在技术上与控制方程不一致，但却无法完全避免这个现象。所有的数值方法都会存在数值误差，而这也就是 SPH 方法的缺陷所在。但是在某些情况下，即使是仿真不可压缩的流体，密度的变化也是真实存在的。举个例子，当将两种不同的，不可压缩的流体混合在一起时，则混合流体的密度就会发生变化。所以，在整个仿真过程中，密度的变化是符合实际情况的。下面的章节中介绍，如何去仿真流体与刚体的交互作用，而上面这个特殊的情形，事实证明是非常有用的。

2.6.2　扩展 SPH 方法，以支持流体和刚体的交互作用

[Muller03]一文中所介绍的流体仿真技术是一个比较基础的技术，并没有告诉我们如何去仿真一个运动的刚体与一个用 SPH 方法仿真的流体之间真实的交互作用。本文后半部分将介绍 SPH 技术的一个扩展，这一技术可以帮助我们很容易地实现流体和刚体之间的交互作用。

在游戏世界中，需要与流体发生交互作用的实体大致包括两种类型。第一种是静态实体，比如游戏世界中组成游戏关卡的几何体，它们是静止不动的。第二种是游戏世界中的动态实体。这些实体是可以运动的，诸如木箱、游戏角色，或者是一些交通工具。对于静态实体和动态实体，它们的处理方式也是不同的，在下面的内容中将做详细解释。

Download at www.Pin5i.Com

1. 与静态世界几何体的交互作用

对于一个 SPH 流体粒子渗透到一个静态实体时的情况，有一个简单却比较实际的方法，就是计算作用在这个粒子上的反作用力。这个作用力迫使该粒子重新回到游戏世界的流体域中。这就是 Penalty Force（阻力）方法，它非常适合处理流体与静态几何体的交互作用。Penalty Force（阻力）使得流体粒子在经过几个仿真帧之后，可以运动到实体之外的区域。在这里，我们没有必要去计算粒子对几何体施加的作用力，因为这些几何体都是静止不动的。

当一个流体的质点与一个静态物体发生渗透时，就会产生一个阻力（而且，这个力要加到公式 2.6.1 中的那个 **f** 项上）。Moore 和 Wilhelms 在[Moore88]一文中对 Penalty Force（阻力）方法作了详细的介绍。为了本文叙述的方便，如果一个流体粒子与一个静态实体发生渗透，那么作用在该粒子上的 penalty force（阻力）可以用公式 2.6.6 计算得到。

$$\mathbf{F}^{\text{col}} = k^{s} d\mathbf{n} + k^{\text{d}}(\mathbf{v} \cdot \mathbf{n})\,\mathbf{n} \tag{2.6.6}$$

其中，d 是该粒子渗透到静态实体中的距离；k^s 是弹性常数；k^{d} 是阻尼常数；**n** 是碰撞点上的法线向量；**v** 是相对速度；而 $\mathbf{F}^{\text{col}}$ 是作用在该流体粒子上，沿着法线向量方向的碰撞反应阻力。我们要将这个碰撞力除以粒子的质量（以便获得单位质量上的作用力），然后将计算结果加到该粒子的纳维叶-斯托克司方程中的 **f** 项中。至于上面的两个常数，我们可以通过实验来找到最佳的值。对于每个常数，我们可以从一个较小的值开始，逐渐缓慢地增加它的值，直到达到我们预期的效果。

2. 如何处理流体与动态刚体的交互作用

动态刚体（在后文中简称为“刚体”）指的是那些在游戏过程中运动的物体，它们可能会与流体（比如水）发生交互作用。

在这个扩展的 SPH 方法中，我们把刚体看作是流体的一部分，对它们进行相应的处理。为了这个扩展的 SPH 方法可以正常工作，这里要求刚体的初始密度要大于流体的质量密度。我们将刚体表示为一组粒子。这些粒子的更新方式与流体粒子的更新方式是相同的。只是在计算流体与刚体之间的压力作用时，才需要进行一些特殊的处理。刚体粒子会对流体粒子施加一个压力，将流体粒子“推离”刚体。同样地，流体粒子也会施加一个压力，使刚体产生净平移或旋转。

公式 2.6.7 给出了作用于一个刚体粒子上的压力计算公式；而作用在一个流体粒子上的压力可以用公式 2.6.8 计算得到：

$$p = \begin{cases} k^{\text{rigid}}(\rho - \rho_0^{\text{rigid}}) & \rho \geqslant \rho_0^{\text{rigid}} \\ 0 & \text{否则} \end{cases} \tag{2.6.7}$$

$$p = \begin{cases} k^{\text{fluid}}(\rho - \rho_0^{\text{fluid}}) & \rho \geqslant \rho_0^{\text{fluid}} \\ 0 & \text{否则} \end{cases} \tag{2.6.8}$$

其中，ρ_0^{rigid} 是刚体的初始密度，或者说是静止密度。注意，由于插值操作和混合操作的影响，单个刚体粒子的初始密度会上下波动。这和流体粒子的情况一样，单个流体粒子的密度也会上下波动。公式中的变量 ρ_0^{fluid} 是流体的初始密度（静止密度）。公式 2.6.8 会阻止流体粒子渗入到刚体中，因为压力总是正的，形成一个排斥力，使流体粒子向远离刚体粒子的方

Download at www.Pin5i.Com

向运动。

在我们的定义中，刚体的运动被限定为刚体围绕着质心的平移和旋转。由于刚体不会产生变形，在一个时间步长的结尾，刚体粒子的运动行为就像是一组简单的质量粒子。这些质量粒子刚性地链接在一起，组成刚体的外部形状。在传统的刚体仿真中，会有一个净力和一个净力矩作用在刚体上，使刚体作为一个整体来运动。在我们的方法中，压力是分别独立地作用于每一个刚体粒子，使它们分别独立地运动。由于这种运动的独立性，以及不受约束的刚体粒子更新，所以在一次更新操作之后，原来组成刚体的那些刚体粒子并不一定会仍然保持刚体原来的形状。为了在每个时间步长结束的时候，让那些组成刚体的刚体粒子仍然会保持刚体原来的形状，我们要采取一个修正的步骤，来强化刚体粒子的刚性。第一步，我们独立地更新每一个刚体粒子；接下来，再强化粒子的刚性，如图 2.6.4 所示。

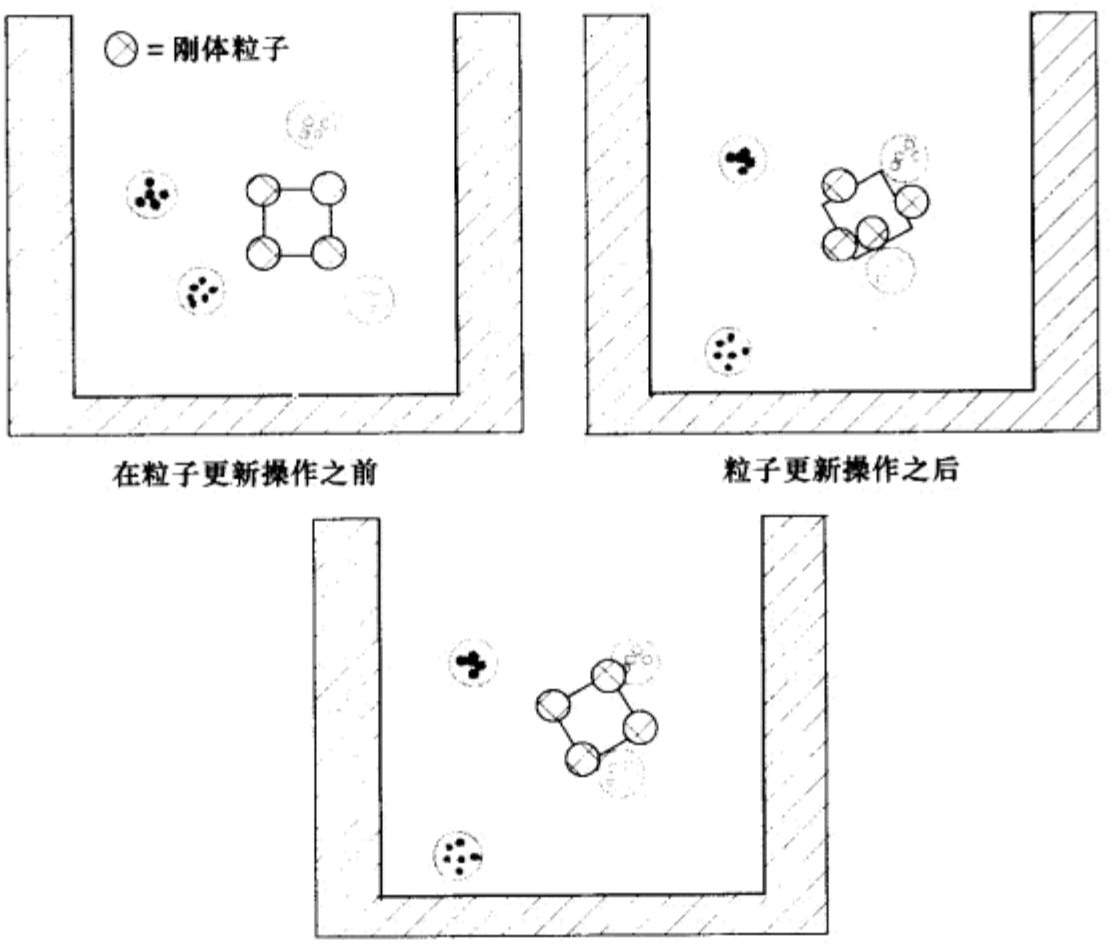


图 2.6.4　粒子初始更新操作之后，以及应用刚性约束之后，刚体粒子的位置对比示意

为了强化粒子的刚性，在粒子更新操作之后，我们会利用粒子的位置和运动，来计算出这个刚体的新的位置和方向，以及其他的一些刚体动力学特性的值。然后，再去更新粒子的位置，使它们可以位于相对于这个新刚体的一个位置上。在计算新刚体的位置和方向的时候，我们用一种近似的方法来处理刚体的运动。这个方法要比全刚体仿真要简单一些，但对于刚体与流体之间交互作用，同样也可以获得稳定的仿真效果。

一个刚体的总质量 M 可以用公式 2.6.9 来计算得到。

$$M = \sum_j m_j \tag{2.6.9}$$

其中，m_j 是单个刚体粒子 j 的质量，质量的总和就是计算出一个刚体中所有粒子质量的总

Download at www.Pin5i.Com

和。假设所有粒子的质量都是相同的，那么对于一个刚体，其质心速度就可以用公式 2.6.10 计算得到。

$$\mathbf{v}_g = \frac{1}{N}\sum_j \mathbf{v}_j \tag{2.6.10}$$

其中，N 是粒子的数量，而 $\mathbf{v}_j$ 是单个粒子 j 的速度。刚体 ω 的角速度可以用公式 2.6.11 近似地计算出来。

$$\omega = \frac{1}{I}\sum_j \mathbf{q}_j \times \mathbf{v}_j \tag{2.6.11}$$

在这个公式中，$\mathbf{q}_j$ 是单个刚体粒子 j 的位置。这个位置是相对于该粒子相应刚体的质心 $\mathbf{r}_g$ 的。这两个量是由公式 2.6.12 和公式 2.6.13 来定义的。刚体粒子位置向量 $\mathbf{q}_j$ 是在仿真开始的时候以局部刚体坐标系统来计算的，而且以后就不用重新计算了。它们表示的是相对位置，也就是为了保持刚体的形状，刚体粒子必须落定的位置。

$$\mathbf{r}_g = \frac{1}{N}\sum_j \mathbf{r}_j \tag{2.6.12}$$

$$\mathbf{q}_j = \mathbf{r}_j - \mathbf{r}_g \tag{2.6.13}$$

标量变量 I 与刚体的转动惯量有关，也就是转动惯量除以刚体的质量。标量变量 I 的定义参见公式 2.6.14。

$$I = \sum_j |\mathbf{q}_j|^2 \tag{2.6.14}$$

利用公式 2.6.9 到公式 2.6.14 中可以计算的这些刚体的特性值，来获得整个刚体的一个更新位置和方向。举个例子，在仿真中经常遇到的情况是，刚体的位置是由它的质心来定义的，所以公式 2.6.12 直接就为我们提供了更新后的位置。我们可以像下面这样来计算粒子的更新位置和方向。

我们要为每一个刚体保存相应的旋转矩阵 $\mathbf{R}(t)$，以及质心 $\mathbf{r}_g(t)$。在仿真的过程中，可利用下列公式来更新它们的值。

$$\begin{aligned}\mathbf{r}_g(t+\Delta t) &= \mathbf{r}_g(t) + \Delta t\, \mathbf{v}_g\left(t+\frac{1}{2}\Delta t\right)\\ \mathbf{R}(t+\Delta t) &= \mathbf{R}(t) + \Delta t\, \mathbf{Q}\left(t+\frac{1}{2}\Delta t\right)\end{aligned} \tag{2.6.15}$$

在这里，$\mathbf{Q}(t)$是在时间 t 上旋转矩阵的变化时间比率，由公式 2.6.16 给出。在时间 $t+1/2\,\Delta t$ 上，利用时间 $t+1/2\,\Delta t$ 上的粒子速度，计算得到刚体 ω 的新值，并利用 $\mathbf{R}(t)$（因为 $\mathbf{R}(t+1/2\,\Delta t)$ 还没有计算出来），就可以计算出 $\mathbf{Q}$ 的值。

$$\mathrm{Q}(t) = \begin{bmatrix} 0 & -\omega_z & -\omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & \end{bmatrix}\mathrm{R}(t) \tag{2.6.16}$$

一旦计算得到了刚体的角速度和质心速度，那么利用公式 2.6.17，就可以计算出受刚体运动所约束的每个刚体粒子速度的运动学速度。

$$\mathbf{v}_j = \mathbf{v}_g + \omega \times \mathbf{q}_j \tag{2.6.17}$$

Download at www.Pin5i.Com

2.6.3　与动态刚体的交互作用：仿真更新

1. 更新流体粒子的位置和速度

流体粒子的加速度 a_j 其实就是纳维叶-斯托克司方程中 Dv/Dt 这一项。因此，要计算这个加速度，需要首先去求解[Muller03]中介绍的那些 SPH 方程。如果需要的话，每个单位质量上的重力，或者是与一个静态网格物体发生碰撞后产生的罚力，这些力都可以添加到 **f** 项上。一旦计算得到了这个加速度，就可以利用传统的粒子仿真技术，来更新其他的两个运动学特性：位置和速度。对于流体粒子的位置和速度这两个特性的更新，有一种方法是使用 Leap-Frog（跳蛙）Verlet 数值积分法，如公式 2.6.18 所示。

$$\begin{aligned}\mathbf{r}_j(t+\Delta t) &= \mathbf{r}_j(t) + \Delta t\, \mathbf{v}_j\left(t+\frac{1}{2}\Delta t\right)\\ \mathbf{v}_j\left(t+\frac{1}{2}\Delta t\right) &= \mathbf{v}_j\left(t-\frac{1}{2}\Delta t\right) + \Delta t\, \mathbf{a}_j(t)\end{aligned} \qquad (2.6.18)$$

这个方法需要我们在程序实现中，保存特定时间步长上的粒子的速度 $\mathbf{v}_j$。这些特定的时间步长是相对于位置 $\mathbf{r}_j(t)$和加速度 $\mathbf{a}_j(t)$偏移半个时间步长（$1/2\,\Delta t$）的值。启用跳蛙法是比较容易的。我们只要简单地忽略这样的事实：在我们对粒子各个特性的值进行初始化的时候，我们要在偏移时间步长上保存相应的速度。例如，我们可以将时间等于$=-0.5\Delta t$ 的速度设置为粒子的启动速度，并从这里开始进行处理。

虽然 Leap-Frog（跳蛙）Verlet 数值积分法只计算半个时间步长上的速度，但是，为了计算 SPH 更新操作中涉及的纳维叶-斯托克司方程的各个项，我们还需要知道时间 t 上的速度 $\mathbf{v}_j\,(t)$。我们可以利用公式 2.6.19 来计算这个速度。这个值不需要保存。我们可以在 SPH 更新的过程中快速地计算出这个值，然后就不用管它了。

$$\mathbf{v}_j(t) = \frac{1}{2}\left\{\mathbf{v}_j\left(t-\frac{1}{2}\Delta t\right) + \mathbf{v}_j\left(t+\frac{1}{2}\Delta t\right)\right\} \qquad (2.6.19)$$

当然，跳蛙技术并不是惟一可以用来更新粒子位置和速度的方法。大家可以放心大胆地试用其他的方法，比如不使用速度的 Verlet 方法。

2. 更新刚体粒子的位置和速度

在这一方法中，刚体粒子的更新方法与流体粒子的更新方法完全相同。这使得这一方法的程序实现更为漂亮！

为了强化粒子的刚性，在刚体粒子被更新之后，首先要计算刚体更新后的位置和方向，这两个特性在公式 2.6.15 中有描述。一旦得到了整个刚体的新位置和新方向，就可以去更新那些组成这个刚体的单个粒子。在下一个时间步长 $t+\Delta t$ 上，每个刚体粒子的位置可以用公式 2.6.20 计算得到。这个新得到的位置就可以强化刚体粒子的刚性。

$$\mathrm{r}_j(t+\Delta t) = \mathrm{R}\,(t+\Delta t)\,(\mathrm{r}_j(t) - \mathrm{r}_g(t)) + \mathrm{r}_g(t+\Delta t) \qquad (2.6.20)$$

Download at www.Pin5i.Com

2.6.4 具体的实现细节

1. 邻居搜索

如果利用 SPH 技术来更新粒子，我们要对点 **r** 位置上的流体的各个特性物理值进行插值操作，也就是要在平滑长度 h 的范围内，对所有粒子的物理特性值与核函数的乘积进行求和。要完成这个工作，就需要去搜索周围那些有直接影响的粒子。如果能够尽可能快地搜索到邻居粒子，这对仿真算法是非常关键的。

目前有很多的空间分割方法，都可以实现非常快速的空间搜索。一个比较简单的方法是：将空间分割成统一的，大小为 h 的体素（体素，来源于 volumetric pixel，组成三维图像的基本六面体称为体素）。如果某个体素中包含了某个粒子的中心点，那么我们就将这个粒子分配到这个体素中。假设一个粒子 A，我们可以查询一个 3×3×3 的体素空间（包含粒子 A 的体素，以及 26 个相邻的体素）中的粒子，这样就可以获得粒子 A 的邻居粒子。如图 2.6.5 所示为一个二维的体素网格。对于这个二维的例子，相邻的体素数组的维数是 3×3。程序清单 2.6.1 中的伪代码就是将粒子分配到各个体素中，并定位一个给定粒子 i 的邻居粒子。

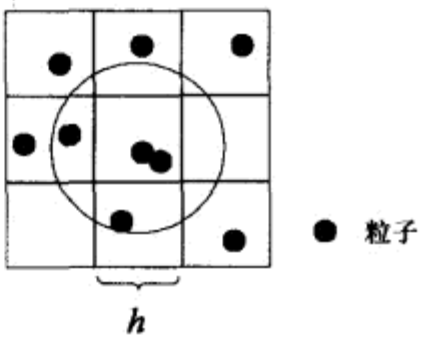


图 2.6.5 在二维空间中相邻的粒子

程序清单 2.6.1 分割粒子，并定位给定粒子的邻居粒子

```
// 首先将所有的刚体粒子分配到相应的体素中
for 每个刚体粒子 i {
    allocate_to_voxel(i)
}

// 为流体粒子 i 定位邻居粒子
// 并将这个流体粒子分配到相应的体素格子中
for 每个流体粒子 i {
    for 每个邻居体素 g {
        for g 中的每个粒子 j {
            if h>distance(i,j)
            add_pair_to_list(i, j)
            // 搜集粒子 i 的邻居粒子
        }
    }
    allocate_to_voxel(i)
}
```

在下一次搜索中可以重复使用这次邻居搜索步骤中计算得到的距离，以便节省下一次搜索所需要的计算时间。根据场景的具体情况，如果相邻的粒子群差别不大，那么就可以跳过邻居粒子搜索的过程，这样可以进一步地缩短仿真的时间。

2. 运行仿真程序

下面这几段伪代码表述了这一仿真程序的部分实现细节。程序清单 2.6.2 中展示了这个

Download at www.Pin5i.Com

仿真程序的主入口点，以及整个仿真循环。程序清单 2.6.3 则是定位邻居粒子的函数。它是程序清单 2.6.1 的一个变种。程序清单 2.6.4 的功能则是被更新粒子的密度计算。程序清单 2.6.5 是计算作用在粒子上的力的打包代码。程序清单 2.6.6 的伪代码是用来更新整个刚体的运动属性。最后一个，程序清单 2.6.7 是流体粒子的更新代码。

程序清单 2.6.2　仿真程序的主入口点和仿真循环

```
procedure simulation_main {
    initialize

    while simulating {
        find_neighbors_of_fluid_particles

        compute_particle_densities
        compute_pressures       // 包括刚体/流体的交互作用

        compute_all_forces      // 考虑静态碰撞和重力

        //依照跳蛙法，
        // 更新每个流体粒子的位置和速度
        for each fluid particle i {
            update_position_and_velocity(i)
        }

        for 每个刚体 r {
        // 执行刚体粒子的初始化更新，然后再
        // 计算整个刚体更新后的位置和方向
        compute_rigid_body_motion(r)
        // 现在我们来更新每个刚体粒子的位置，
        // 以便强化粒子的刚性
        for  刚体 r 中每个刚体粒子 i {

            // 为了根据公式 2.6.20 来强化粒子的刚性，
            // 更新每个刚体粒子的位置和速度
            update_rigid_position_and_velocity(i)
        }
    }
}
```

程序清单 2.6.3　定位流体粒子的邻居粒子

```
procedure find_neighbors_of_fluid_particles {
    clear(grids)
    for 每个流体粒子 i {
    // 计算粒子 i 所对应的体素的索引值

    for  与 voxel[g] 相邻的每个粒子 j {
        dist = compute_distance(i, j)
        if (dist < h) {
            // 保存邻居粒子和它的距离
        }
```

Download at www.Pin5i.Com

```
        }
        // 将粒子分配到相应的体素中
        add(voxels[g], i)
    }
}
```

程序清单 2.6.4 利用 SPH 方法来更新粒子的密度

```
procedure compute_particle_densities {
    for 每个粒子 i {
        for 每个粒子 j，neighbor_list[i]中的 dist {
            d = compute_density(dist, pos[i], pos[j])
            // 由于密度贡献值的对称性，
            // 我们将计算得到的密度贡献值分别加到粒子 i 和粒子 j 的密度上
            density[i] += d
            density[j] += d
    }
}
```

程序清单 2.6.5 计算作用在粒子上的力

```
procedure compute_all_forces {
    for 每个粒子 i {
        for 每个粒子 j，neighbor_list[i]中的 dist {
            // 计算两个粒子之间的密度贡献值
            f = compute_force(dist, i, j)
            force[i] += f
            force[j] -= f
    }
}

function compute_force(float r, int i, int j) {
    // grad_spiky_coef 和 lap_vis_coef 是我们
    // 预先计算出来的 SPH 方程的系数
    // grad_spiky_coef=-45.0f/(PI*h^6)
    // lap_vis_coef=45.0f/(PI*h^6)
    if (is_rigid[i] or is_rigid[j])
    {
        force = mass[j]*(h - r)/density[i]/density[j]*(-
        0.5*(max(pressure[i], 0) + max(pressure[j],
        0))*grad_spiky_coef*(h - r)/r + (vel[j] -
        vel[i])*mu*lap_vis_coef)
    }
    else
    {
        force = mass[j]*(h - r)/density[i]/density[j]*(-
        0.5*(pressure[i] + pressure[j])*grad_spiky_coef*(h - r)/r +
        (vel[j] - vel[i])*mu*lap_vis_coef)

    }
```

Download at www.Pin5i.Com

```
        return force
    }
```

程序清单 2.6.6　计算整个刚体的运动属性

```
procedure compute_rigid_body_motion(r) {
    for  rigid[r]中的每个粒子 i {
        // 根据作用在粒子上的力，来更新粒子的速度
        update_velocity(i)
    }

    // 根据公式 2.6.10 和公式 2.6.11
    // 计算出重心和 q[i]
    // 根据公式 2.6.12 计算出 I 的值
    for    rigid[r]中的每个粒子 i  {
        q[i] = pos[i] - r_g[i]
        I += length(q[i])^2
        omega = cross(q[i], vel[i])
    }
    omega /= I

    // 根据公式 2.6.18 和公式 2.6.19，
    // 计算出更新后的旋转矩阵
    compute_rotation_matrix
}
```

程序清单 2.6.7　更新流体粒子

```
procedure update_position_and_velocity(int i) {
    // 仿真程序要保存流体粒子的位置、速度，以及半个时间步长上的速度
    for  每个流体粒子 i  {
        // 计算 v (t+1/2dt)
        vel_half_next = vel_half[i] + t*acc[i]

        // 计算 r (t+dt)
        pos[i] = pos[i] + t*v_half_next;
        // 计算 v (t)
        vel[i] = 0.5*(vel_half_next + vel_half[i])
        vel_half[i] = vel_half_next
    }
}
```

2.6.5　相关的优化

纳维叶-斯托克司方程右侧部分的计算是用 3 个核函数来完成的。由于核函数的有效半径 h 是一个常数，因此，我们可以预先计算出这些核函数的系数，以便获得更快的计算速度。

最初级的算法实现是这样做的：在每次计算包含粒子交互作用的方程时，对所有的粒子进行迭代。但是，一个粒子 A 对另外一个粒子 B 的影响，以及反过来，粒子 B 对粒子 A 的影响，其实是对称的。所以，为了算法实现的效率，应该首先计算各种物理特性的影响作用

Download at www.Pin5i.Com

值，然后再去插值计算各种物理特性的值。

2.6.6 总结

平滑粒子流体动力学技术是常见的流体仿真技术。本文介绍了如何来扩展这项技术，使我们可以对流体与刚体之间交互作用进行仿真。每个实实在在的游戏程序都需要进行流体的仿真。如果将流体仿真系统当作粒子系统来渲染显示，那么这些粒子的运动行为看上去就像是一个流体。粒子的数量越多，这种幻象的可信度就越高，但同时也增加了仿真的系统开销。为此我们可以选择一些折衷的方法，比如，可以生成一个连续的流体表面，并利用各种水体渲染技术来渲染这个流体表面。具体的渲染步骤可以参考下列步骤来实现。

1．根据粒子的分布的情况，生成隐式的流体表面。

2．利用诸如 Marching Cubes（行进立方体）[Bloomenthal94]之类的方法，将这个流体表面分割成若干个三角形。

3．最后再利用任意一种你喜欢的着色技术对这个表面进行着色处理。

如果你能够使用硬件着色器来仿真光线的反射和折射，这样渲染出来的流体看上去会非常的逼真。对于本文所讨论的这些方法，随书光盘提供了简单的实现代码，以供参考。

2.6.7 参考文献

[Bloomenthal94] Bloomenthal, Jules, "An implicit surface polygonizer." *Graphics Gems IV,* Academic Press, pp. 324–349, 1994.

[Moore88] Moore, Matthew, and Jane Wilhelms, "Collision Detection and Response for Computer Animation." *Proceedings of the 15th Annual Conference on Computer Graphics and Interactive Techniques,* ACM Press: pp. 289–298, 1988.

[Müller03] Müller, Matthias, David Charypar, and Markus Gross, "Particle-based Fluid Simulation for Interactive Applications." *Proceedings of the 2003 ACM SIGGRAPH Eurographics Symposium on Computer Animation,* Eurographics Association: pp. 154–159, 2003. Available online at *http://graphics.ethz.ch/Downloads/ Publications/Papers/2003/mue03b/p_Mue03b.pdf.*

[Noe04] Noe, Karsten, "Implementing Rapid, Stable Fluid Dynamics on the GPU." 2004. Available online at *http://projects.n-o-e.dk/GPU_water_simulation/gpuwater. pdf.*

[Stam99] Stam, Jos, "Stable Fluids." *Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques,* ACM Press: pp. 121–128, 1999.

Download at www.Pin5i.Com

第 3 章

人工智能

Download at www.Pin5i.Com

引　言

Brain Schwab，索尼计算机娱乐美国公司
brain_schwab@yahoo.com

人工智能，在提起它时我们真的知道它意味着什么吗？对于一些人来说，人工智能意味着机器能够像人一样地思维，对于另一些人来说，它只意味着游戏中角色执行一段事先写好的脚本。人们对人工智能的理解是如此不同，可见这个领域的范围是非常宽广的，同时也是非常混乱的。

很多研究者，包括很多工程师和爱好者都从事着人工智能的研究和开发——但是他们却是如此的不同。他们的方向不同、使用的工具不同、观念不同、目标不同。然而有时却十分鼓舞人心，那些纯粹的高级游戏人工智能产品的出现使我们这些在这个领域工作的人激动不已。在过去，人工智能程序员可能会找到一两篇关于路径寻找优化或者常规状态机扩展方面的文章。但现在，在网上随意的一次搜索就可以查到数百篇参考文献（很多还附带专业化设计的代码）。这些文献涉及到不同的 AI 技术：战斗和地形分析，规划算法，人工智能学习。而且，这个参考文献的列表还在不断增长。

然而，我们不能忘记在这么多可资利用的资源的情况下，我们所用于游戏开发的人工智能工具也仅仅是一个工具。你应该根据自己正在做的工作的特点选择合适的工具，而不要幻想有这么一种人工智能技术可以解决你所有的问题。你要考虑你正在开发的游戏的特点，还应该评估各种因素，比如时间进度、实际需要达到的智能水平、人工智能执行者的特定行为。如前所述，人工智能不是一个简单的问题，也不是一个单一的问题。

《游戏编程精粹 6》在本部分列出的文章实际上是一组不同的工具包。Armand Prieditis 讨论了一种基于模型的系统，该系统用来实现人工智能规划。Diego Garces 提供了更多的实现多单元合作的技巧和窍门。Hugo Pinto 和 Luis Otavio Alvares 为我们提供了很多在游戏中使用传统的机器人智能系统的宝贵经验。Julien Hamaide 探讨了使用支持矢量机来实现短期记忆。Michael Ramsey 的文章讲述了定量的战斗结果评估模型。Sebastien Schertenleib 讨论了一个灵活的人工智能引擎的创建过程，给出了本书第 4 章（脚本和数据驱动系统）所探讨问题的一个典型应用。希望这些文章中有一篇或几篇会帮助你更深入地认识你工作中所遇到的人工智能问题。

另一个值得在游戏开发团队中进行人工智能开发的程序员（目前很多

Download at www.Pin5i.Com

工作室已日渐意识到对于更先进系统的需求，并正在雇用主专业之外的人工智能工程师）注意的是其他领域的开发也可能用到你所使用的人工智能工具，并且使用你所制作的作品。人工智能刚刚开始被用到游戏开发的其他方面。随着人工智能的功交日益为业界所了解，并获得广泛的认同，人工智能编程人员的工作机会还会持续增长。很多公司正在使用人工智能做自动的产品检测，组织开发过程，将大量复杂的离线物理参数用到游戏中，以及其他诸多应用中。Gabriyel Wong 实际上讨论了使用模糊逻辑来控制游戏图形系统的细节等级决策。

总之，人工智能程序员的工作是一把双刃剑。我们有更多更好的人工智能工具可资利用，但是，我们也在不断寻找在每一个游戏中使用这些工具的新的方法。我们的一个始终不变的目标是开辟游戏人工智能的新的领域并创造出更自然的行为。

我们必须不断地在游戏开发的各个层面创新地使用我们的技术，但是仍然要站在我们的主要目标上：创造有趣的游戏体验。通过使玩家感觉到自己好像在和真人玩游戏，我们可以为游戏带来玩家急切盼望的有趣元素，同时使我们继续成为游戏的创作者。

Download at www.Pin5i.Com

3.1 游戏的制作方法——应用基于模型的决策——在雷神之锤 III 中应用蝗虫人工智能引擎

Armand Prieditis, Lookahead Decisions Inc.
Prieditis@LookaheadDecisions.com
Mukesh Dalal, Lookahead Decisions Inc.
Mukesh@LookaheadDecisions.com

开发一个好的游戏人工智能引擎是困难的。通常，通过基于规则的方法来开发游戏的人工智能。但是，规则在开发、修改和调试的过程中是不稳定、昂贵和费时的，并经常导致游戏出现非智能的行为。这篇文章讨论一种新的游戏智能开发方法：基于游戏世界模型的方法——行动、效果和观察。基于模型的方法更稳定，更省时也更经济，并且非常易于修改，这也使得游戏所表现的行为更加智能化。这篇文章将讨论在游戏“雷神之锤 III（Quake）”中使用基于模型的人工智能开发方法开发蝗虫的智能引擎中所获得的一些初步成果。我们将从特点和优点两个方面来讨论这种开发方法对游戏智能设计者的影响，并提出一个未来游戏智能开发的构想。

3.1.1 引言

见图 3.1.1，现代的计算机游戏通过综合图形的、物理的和人工智能的方法来达到游戏的真实感。真实感游戏体验是很难确切定义其内涵的，但一般地说，这通常指游戏的沉浸感以及游戏中出现的非玩家角色的智能性。在真实感游戏中，游戏中对象的外观和行为与现实中对象的外观和行为非常相似，并且非玩家角色的行为也合乎逻辑。游戏人工智能的目标不是创造一个不可打败的对手，而是通过非玩家角色的智能行为来创造更富有沉浸性更有趣的游戏体验。

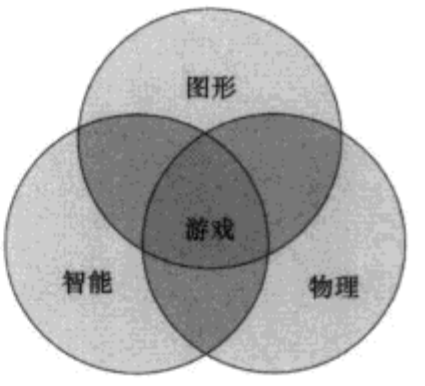


图 3.1.1 游戏综合使用了多种技术

比如，没有人工智能，创造一个完全不会被打败的对手仍然是可能的。我们以欺骗玩家为例，开发人员可以使非玩家角色直接进入玩家正在躲藏的房间，而一个具有智能的非玩家角色却会采取一个系统化且合乎逻辑的方法来搜索每一个房间。相比之下，后一种方法更可信，从而使游戏更有沉浸感，也更有趣。实际上，一个真正智能的非玩家角色应该以最大化游戏玩家的兴趣和游戏的沉浸感为目标，而不是以不被打败为目标。

Download at www.Pin5i.Com

在过去的几年中，游戏中的图形技术和物理特性模拟技术取得了很大的进展。实际上，图形开发包，如 Maya 和 3Ds Max 带来的令人瞠目的图形已经是一件普通的事了。物理特性模拟开发包，如 Havok 可以使开发人员创造出完全真实的世界。但是，对图形技术和物理特性模拟技术的应用已经不足以使一个游戏具有独特性了。所以，游戏开发人员现在正在寻找使自己游戏进一步异化的创新。因为游戏人工智能没有像图形技术和物理特性模拟技术那样取得巨大的发展，所以它提供了一个游戏创新和异化的空间。不仅如此，在一个游戏中，程序员允许花费在人工智能计算上的 CPU 时间正在稳步地增加，因为高速的图形卡正越来越多地取代原来由 CPU 执行的功能。这篇文章介绍一种新的游戏人工智能的开发方法，这种方法利用了上述的计算上的可能性。

3.1.2 目前的游戏人工智能：基于规则

目前的游戏人工智能主要是基于规则的。规则的形式是条件→行动。当条件被满足时，行动就被执行，这个行动可能是单个的移动、一个移动脚本的执行或者一个动画片段的播放。

比如说，规则可以是这样的：如果健康值低并且敌人可见，或者敌人的武器比玩家的好，那么玩家应该撤退。撤退的过程可以是仅仅播放几秒钟的视频，也可以是播放视频直到条件变化导致另一条规则被选择。规则的条件可能包含若干布尔值，这些布尔值的关系是简单的“或”关系。

另一个流行的规则是路径寻找。非玩家角色需要找到一条抵达目的地的最短路径。这种规则在没有动态的障碍物或者其他角色的环境中能工作得很好。

判断规则条件是否符合的方法可以有很大的不同。它可以是如图 3.1.2 所示的决策树形式。这棵决策树包含了条件和与之对应的一组操作。树的非叶子节点对应着当前情况下的一个判断，而叶子节点表示行为。例如，根节点判断非玩家角色是否有武器，如果有，那么接着判断非玩家角色是否靠近敌人，如果靠近，那么非玩家角色就发起攻击。而另一条发起攻击的路径是在非玩家角色没有武器的情况下，判断敌人是不是比非玩家角色小，如果是，就发起攻击。

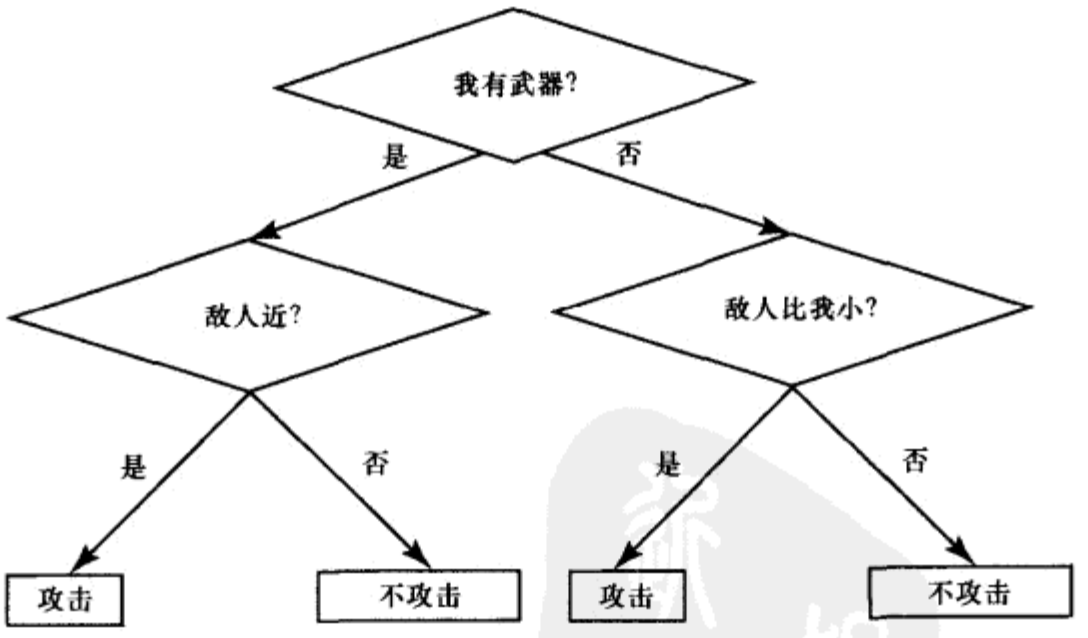


图 3.1.2 一个简单的决策树

一句话，每一条从树根到树叶的路径都代表着一个符合条件→行动形式的规则，路径中的非叶子节点代表一组条件，而路径终点的叶子节点代表行动。尽管决策树能够更加紧凑地

Download at www.Pin5i.com

归类规则，同时也比一组规则要好维护，但从决策能力来说，他们和标准的规则是等价的。

另一个编码一组规则的方法是有限状态机。一个有限状态机包含一组开发人员定义的状态，这些状态对应着特定的行动。状态间的连线表示情况判断，并且使得状态从一个过渡到另一个[Rabin02]。例如：在如图 3.1.3 的有限状态自动机中，最初的状态可能是空闲状态，非玩家角色没有行动直到发现了敌人。当发现了敌人时，行动是攻击（比如：播放一段子弹发射的视频片断）。当达到攻击的状态，如果非玩家角色又找不到敌人了，那么就会激发一个搜索的行为，这也可能表现为一个相关视频片断播放。

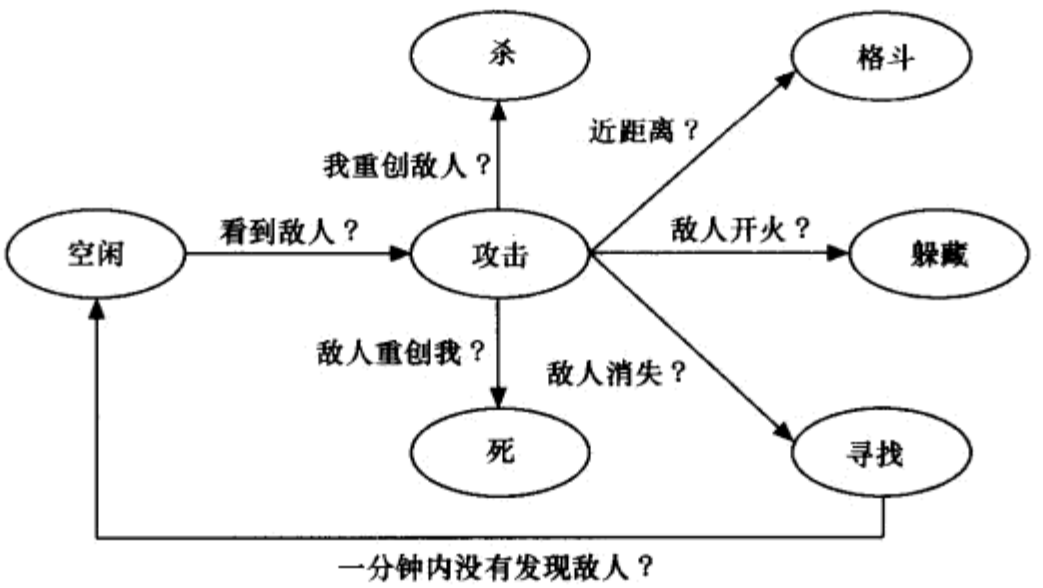


图 3.1.3 一个典型的有限状态机

与决策树相比，有限状态机相当于一组带标记的规则。标记和状态相对应，标记设置在一个规则的行动部分，也执行相关的行动。尽管有限状态机决策能力并不比标准规则高，然而它在一定程度上更容易维护，因为它并不需要显式地维护标记和其对应的相关状态。因此，这种方法在很多商业游戏中被应用，如帝国时代（Age of Empires）、敌对国家（Enemy Nations）、半条命（Half Life）、毁灭战士（Doom）和雷神之锤（Quake）。

还有一些其他方法也可以编码规则。模糊逻辑可以用来判断条件满足的概率。类似地，神经网络判断条件满足的方法就更加复杂，但是它们的确可以用来编码规则。和标准规则相比，这些方法使得我们可以以更紧凑的方式来表达一个特定的条件。它们也使得通过案例来了解规则成为可能。专家系统包含一系列的规则和一个内置的规则解释器。这个解释器可以添加它自己的内部符号（如一些不直接出现在条件中的符号），这样就可以进行更复杂规则的匹配判断。和其他的方法相比，专家系统虽然能够有效地提高运行时的灵活性，但却是相当慢的，因此在游戏中很少采用。需要注意的是，专家系统等价于一组规则，因为解释器可以对其执行的规则进行反向推导：它可以找出对一种情况应用某种规则的最小先决条件，也可以找出一个行动的后续行动。

总而言之，规则可以表现为不同的形式，其中的一些形式比另一些形式要容易修改、维护和理解，但是所有的实现都等价于条件→行动。

3.1.3 规则的问题

尽管规则经常成为实现游戏人工智能的第一选择，但是，它仍然有一些缺点，总结如下：

规则经常导致非智能的行为。规则在度量一个行动的长期后果时存在困难。这是因为有

Download at www.Pin5i.Com

可能需要多个非玩家角色进行前后连续的多个行动后才能发现一开始的行动是错误的。比如：如果所有非玩家角色都按照最短路径规则靠近目的地，那么最终会导致路径堵塞。原因是所有的非玩家角色都走同一条路径，结果造成交通堵塞。

规则的开发费时、成本高。这是因为游戏开发人员需要透彻地了解游戏，这样才能找出合适的规则。

基于规则的行为受到游戏开发人员智力的限制。不通过游戏开发人员的艰苦努力，就不会有很好的游戏智能。在紧张的开发时间表下，希望游戏开发人员在有限的时间内开发出足够的专门技术来使所有的非玩家角色具有智能经常是过分的要求。而且，好的程序员不一定总能够制作出好的游戏角色。

基于规则的行为会使玩家产生怀疑，因而破坏游戏的沉浸性。一个游戏是否成功的重要影响因素是玩家在玩游戏时不产生怀疑。非玩家角色的非理性行为会使玩家产生不信任感。非玩家角色欺骗玩家就更糟糕了，非玩家角色越精明反倒越会弄巧成拙，这样玩家会时时感受到有一只开发者的手在背后操纵游戏，这也就破坏了沉浸性。

规则是脆弱的，当情况稍稍超出规则定义的范围，规则就失效了。比如：一个非玩家角色正沿着最短路径前进时路径暂时堵塞了，这时最短路径规则就失效。其他的一些原因也能带来规则的失效，这是因为规则难于处理以下的一些情况：不确切或并行处理的行动；与时间直接相关的行动；位置的不确定；漏掉的、不精确的或不确定的观察资料。

规则难于修改和调试，这导致一种低效的编码-补丁的开发过程。通常修改一个脆弱不严谨的规则的办法是为它增加一个例外处理。因此，程序员不得不不断地添加例外处理代码来处理新出现的情况。例如：一条规则是如果非玩家角色的生命值低于 50%，那么去寻找一个急救包。规则失效发生在角色停止攻击而开始寻找急救包时，此时敌人可能只剩下一点点生命值，他只需再攻击几秒就可以消灭敌人。因此，这时突然停止攻击看起来是不明智的。那么对这条规则的修正可以是：当非玩家角色的生命值小于 50%而又没有攻击一个生命值小于 5%的敌人时，就去寻找急救包。这条规则解决了目前的问题，角色会在消灭了快死的敌人后才开始寻找急救包。但是，如果非角色玩家的弹药很少怎么办呢？尽管敌人已经很虚弱，但是弹药不够用了，无法继续攻击敌人，这就需要添加另外一个例外处理，可以是这样的：如果非玩家角色生命值低于 50%，而且当前没有攻击生命值小于 5%的敌人，就去寻找救包；或者非玩家角色生命值低于 50%，在攻击敌人生命值小于 5%的敌人时非玩家角色的弹药小于 5%，那么也要开始寻找急救包。这样不断的编码-补丁的开发方法，不但没有效率，而且会产生意大利面条式的代码，这种代码难于维护，并且当游戏引入新特性时容易崩溃。这样发现问题-对症解决的过程也是非常痛苦的，因为复杂的代码内容和程序运行状态意味着错误往往要在程序运行一段时间后才最后出现。

规则永远也没完。在编码-补丁的过程中添加规则并在不同的条件下测试规则直到它失效，然后再对规则打补丁，以处理使规则失效的特定的情况，这样的开发模式在产业里广泛地存在。这就不可避免地导致了没有完成的人工智能产品。这是因为编码-补丁的过程直到开发进度的最后期限还没有完成。产品伴随着侥幸心理发布了，厂商希望游戏的最终玩家不会发现游戏中存在的没有修补好的部分，而这些部分甚至 AI 开发人员都没有察觉。当然游戏玩家在发现规则失效时通常是兴奋的，他们对自己第一个在新闻组或博客上发表关于游戏的错误感到非常自豪。这常使所有认为游戏是个好游戏的评论和宣传统统作废。

规则是不可预知的、不稳定的。反复补丁的规则系统会很迅速地变得不可维护，而且也

Download at www.Pin5i.Com

会变得不可预知。一个没有考虑到的情况可能使系统崩溃，也可能使系统进入死循环或者出现不可预知的奇怪行为。

规则不能适应多非玩家角色的情况。用规则来控制单个的非玩家角色已经是非常困难了。随着非玩家角色的增加，特别是当非玩家角色之间有共谋或敌对的关系时，规则的复杂性和规则失效的可能性就会呈指数级地增加。

在了解了主流的游戏人工智能规则实现方法的这些缺点后，可以知道，游戏中的人工智能技术并不像游戏的图形技术和物理特性模拟技术那样成熟。很久以来，游戏智能开发人员就已经意识到了基于规则的游戏人工智能方法的局限性。

3.1.4 基于模型的游戏人工智能方法

按照理性决策模型理论，最好的决策能够最大化预期的结果，这个预期的结果通过计算决策的后续决策序列得到。例如，图 3.1.4 显示了我们可能考虑的两个选择。每一个选择都有两个后续的选择，后续的选择带来结果。图中的结点代表状态或情况；箭头代表将结果移入新的状态。根节点发出的两个箭头表示当前面临的两个选择，下面的箭头表示未来可能的选择。最优的选择是右侧的，因为它最大化了预期结果。

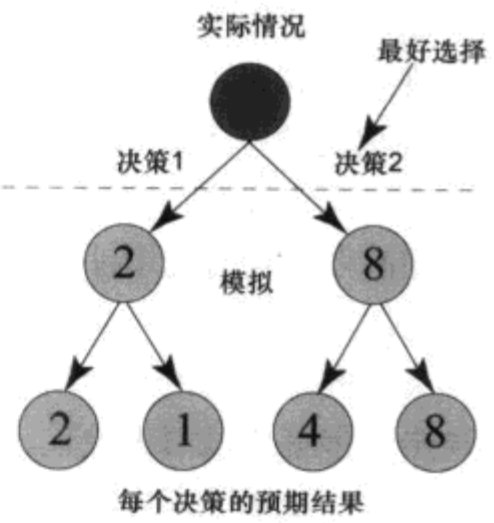


图 3.1.4

我们把这种方法叫作基于模型的游戏人工智能。因为它包含着系统动态性的高层次的描述：可以做的行为和行为的结果。基于模型的决策可以总结成如下三步。

1. 通过构建一棵前瞻树，**模拟**每一个决策所带来的后果。
2. **评估**决策树上每一个决策序列的结果。
3. **选择**预期结果最好的决策。

在经典的棋盘游戏中，模型描述了每一次合法的移动，游戏怎么结束，谁赢了。在视频游戏中，模型描述为每一个允许的行动，它的效果，及可以发生的其他事件。注意不要把游戏规则和游戏智能实现的规则方法混淆。游戏智能实现的规则方法用来规定在一个特定情况下应该执行的行为。而基于模型的实现方法则把在特定情况下可以执行的行为描述为一个模型。我们也可以把基于模型的方法看作是非玩家角色行为的高级特性。

3.1.5 对游戏的接口

典型的基于模型的决策产生系统提供给游戏的接口通常是不同的，但是基本的过程是一样的。例如，在蝗虫游戏引擎中，接口采用了 4 个步骤：观察、更新、决策和行动（见图 3.1.5）。观察涉及到感觉信息的获取。更新是这样一个过程，它利用观察得到的信息来更新关于发生了什么的判断。在一个行动过程中，决策指的是利用当前获得的判断来选择一个行动过程，按照某种标准，这个行动过程有可能在一定程度上改善角色的处境。行动指的是对环境的有效的改变。这个接口模型也假设世界的当前状态是为游戏决策生成系统所知道的。例如，它假设决策系统已经知道了所有非玩家角色的当前位置；更新过程考虑了所有的当前状态的变化。

Download at www.Pin5i.com

如图 3.1.5 所示，观察→更新→决策→行动的循环是无休止的。虚线指出了接口，它把蝗虫引擎和游戏世界实际发生的观察和行动分割开来。也就是说，蝗虫引擎的内部判断结构和决策制定的过程是和游戏世界分割开来的。实际上，它的内部判断结构一般只是游戏世界的一个近似，同时，它依赖于游戏世界表示的复杂程度。

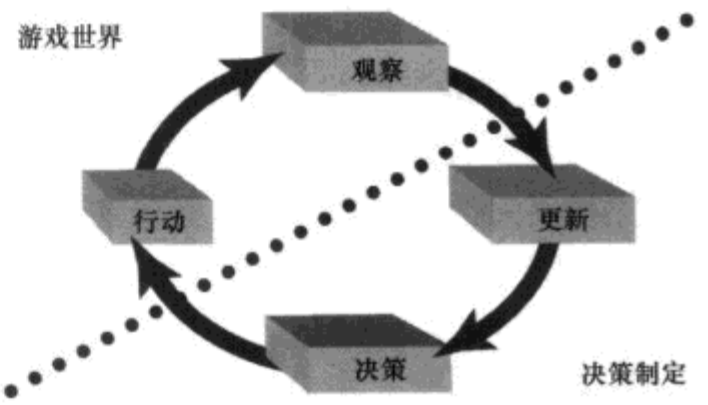


图 3.1.5　对游戏世界的接口

实时的决策制定是这个模型的一个重要方面。在任何复杂世界中，预先制定一个行动过程来全面地对世界做出反应是不大可能的，因为世界是不可预知的。比如，产生了一条路经，基本上指出了一个非玩家角色如何抵达目的地，但是，这条路径很难完整地应用，因为其他的非玩家角色可能拦住了去路，或者忽然出现某个障碍。

蝗虫引擎考虑了这些情况，决策和观察的交替进行使得决策所产生的效果可以得到监控，这就使得改变一个行动的过程成为可能。当然，这个模型也允许决策制定模块制定一个详细的计划，但是在大多数应用中，这样的计划因为不可预测性的原因不能完全地应用。每一个决策制定模块都会使用一个内部的有关游戏世界的判断模型，这个游戏世界是以特定的参数的形式给出，然后这些参数会根据观察到的信息进行更新。

3.1.6　对游戏人工智能开发者的好处和推论

与常规的基于规则的开发方法相比较，基于模型的游戏人工智能开发方法，如在蝗虫游戏引擎所显示出的那样，至少可以提供游戏人工智能开发者如下好处：

使行动变得更智能化。这是因为它考虑了每一个决策对未来所产生的影响。一个为蝗虫引擎所支持的非玩家角色具有符合逻辑始终如一的一组分值，这些分值刻画了该角色要完成什么。非玩家角色的一次次的行动最终导致这些分值的完成。蝗虫人工智能引擎无缝地把短期的战术性的决策和长期的战略性的计划结合了起来，并且引擎还会根据环境和状态的变化来平稳地调整目标的优先级。最终的结果是非玩家角色行为更智能化了。

减少了开发的时间和成本。使一款游戏按时发布并且不超出预算是非常重要的。游戏开发人员知道人工智能开发有可能不能按照计划完成。通常，开发小组在还未能使非玩家角色的行为完善之前，时间就用光了，结果发布的只是一个刚刚够用的人工智能设计。用蝗虫游戏引擎的方法，模型可以迅速地并且相对廉价地建立起来，这个建立的过程也不需要非常专门的技术和经验。所有需要的仅是行为的描述、行为的效果以及观察是如何与状态相联系的描述。

带来自然的智能行为。这指的是非玩家角色能够显示出不是游戏开发人员在一开始就设计好的智能行为。这一点我们可以用一个很好的类比来说明：深蓝国际象棋开发小组用一个类似的基于模型的方法来开发他们的象棋程序，这个程序与人博弈，最终打败了象棋世界冠军卡斯帕罗夫。深蓝国际象棋开发小组成员并不是世界级的棋手，然而，深蓝计算机所表现出来的智能并没有受到这些开发人员智力的限制。这就使得游戏开发人员不必整天为开发规则操心，而是可以开始思考非玩家角色的创造性，这些创造性包括非玩家角色的个性、武器、

Download at www.Pin5i.Com

感觉、目标。

创造更好的沉浸性。当非玩家角色象玩家一样地行动时，真实感就得到了延续。更智能化的行为戏剧性地提高了玩家在游戏中得到的愉悦感。

更稳定可靠。前瞻比规则更稳定可靠，因为它总是针对情况的，同时也是针对一般目标的。这就使得它一点也不脆弱。当作一个决策的时候，蝗虫引擎考虑所有的因素，比如非玩家角色的健康值、敌人的健康值、急救包的接近程度、弹药值。给出了这些输入，蝗虫引擎总是在给定时间范围内返回一个最好决策。时间越多，返回的决策越好，这是因为更多的关于决策对未来的影响都将被考虑到。蝗虫引擎也能处理复杂地形，它能够自动创建分级的路径点。

可以应用到多个非玩家角色上。蝗虫引擎的前瞻算法基于分布式决策技术，这减少了在传统的多个非玩家角色情况下遇到的问题：分支因素的指数性增长。引擎也可以安装到一个网络的不同机器上。

使得修改和除错更容易。如果其他的一些因素需要在非玩家角色的行为过程中被考虑进来，它们可以相对简单地添加。比如，如果不同的非玩家角色需要不同的个性，改变特定的分值折扣就可以反映非玩家角色的个性。当一个胆小的非玩家角色尽早地去取急救包时他的得分值会大打折扣，反之，一个大胆的非玩家角色在接近敌人而不考虑自己的健康时可能也会打折扣。一个懒惰的非玩家角色可能当他发生任何移动时有一个附加的扣分，这就鼓励他尽可能呆在开始的位置。蝗虫引擎更简单并且几乎是上下文无关的，这就使得维护更容易且查错更迅速。添加和改变行为仅是一个简单的修改非玩家角色的模拟和目标的问题。通过基于模拟的前瞻技术，蝗虫引擎避开了基于规则的实现方法导致的像意大利面条一样的代码。简言之，当模型改变时，由模型导致的决策也就改变了，没有更多的工作要做。

使得特性的移植成为可能。游戏玩家希望看到不同的游戏中角色有相似的个性。当一个模型在一个非玩家角色上建立起来，它就能够容易地被移植到一个完全不同的游戏中。就蝗虫引擎而言，我们可以基于角色的行为特点而不是角色的外观实现来建立一个角色库。

可以带来更大的产品区别。我们提到游戏智能是游戏产品特色的最后一个前沿。然而，基于规则的实现方法带来的是几乎一样的“香草味”。基于模型的实现方法，特别是蝗虫人工智能引擎，给设计者带来了如下几个优点：游戏开发者可以在较长时间赋予一小群完全独立行为的角色高级行动，设计者也可以控制一大群简单角色在较短时间进行一个低级的联合行动。蝗虫引擎基于模型的控制也可以用来控制低级的移动。群聚和拥挤行为在蝗虫引擎中是最简单的特性，并且这个特性完全是嵌入在目标描述中，而不是在控制规则中。因此，群聚行为是自然发生的而不是预先设计的。

和基于规则的实现方法相比，基于模型的实现方法有两点不足。第一，它需要建立模型。然而，建立模型比建立一组规则要容易，并且也有可能通过交互来建立一个模型。第二，创建前瞻树可能比处理规则要慢。但是，在大多数复杂的应用中，花更多的时间来得到一个更好的决策是值得的。我们已经发现蝗虫引擎在数毫秒内就可以为多个非玩家角色做出决策。这为游戏的实时响应和变化提供了足够的时间，同时也展示了游戏的智能。

3.1.7 雷神之锤 III 竞技场和蝗虫人工智能引擎

这个部分描述雷神之锤 III 竞技场（Q3A），一个由 id Software 公司开发的多玩家、第一

Download at www.Pin5i.Com

人称射击游戏。正如我们看到的，实时决策制定过程在这个游戏中是复杂的：你只有几分之一秒来对凶恶的非玩家角色做出反应。非玩家角色持续地企图消灭你；你必须考虑如何获得健康值和武器，并且躲开某些非玩家角色；你必须从自己获得的可以使用的若干不同武器中进行选择，每一种武器都有一个最佳用法。

玩家在地图中，或者说在竞技场中移动，同时杀伤或杀死敌人。分值取决于游戏的模式。如果玩家的生命值为 0，那么玩家角色就死亡。很快地，玩家角色会在全图的某一个位置获得重生，但是玩家将失去以前获得的装备。游戏在玩家或者团队达到某一个分值时结束，或者在达到指定的时间时结束。

武器系统被设计为在每一种情况下都有一种最佳武器。一个主要的决策任务就是选择合适的武器。武器也周期性地在特定的地点出现。你可以把武器从地下捡起来从而简单地获得一个武器。所有的武器都有一定量的弹药。弹药箱有时候是隐藏着的，它们散布在整个游戏世界中。

当你的生命值变低的时候，你可以选择 4 种不同的急救包：黄急救包恢复 25 点生命值，橘黄色急救包恢复 50 点生命值，绿色的急救包恢复 5 点生命值（但是可以超过 100 点），蓝急救包可以恢复 100 点生命值（并且可以超过 100 点）。你最多可以获得 200 点的生命值。

在 Q3A 中的移动是相对简单的：你可以跑、走、蹲伏或者跳。在默认情况下，角色总是在跑，然而不看场合到处跑并不明智。走是很有用的，当你走的时候，你不会发出噪音，这使得你可以暗中接近敌人。尽管蹲伏也可以在移动时减小噪音，但是移动速度慢。蹲伏的好处是使你更接近地面，这就使你变成了一个更小的目标，这样你就可以躲在一个低矮的物体后面，也可以进行偷窥而不致成为一个大的目标。

蝗虫人工智能引擎在 Q3A 中被用来控制所有的非玩家角色，没有使用基于规则的控制。角色执行的动作如下：朝某一个方向前进一段距离、朝某一个方向后退一段距离、继续上一次的动作、用当前武器开火、空闲。我们测试了不同的前瞻深度并发现 3 到 8 级的前瞻深度效果最好。超过 5 级，游戏的性能会受到影响。蝗虫引擎每秒钟更新屏幕 20 到 30 次（大多数游戏每秒更新屏幕 20 多次）。在每次更新期间，蝗虫引擎让 2 到 3 个非玩家角色做一次完全的前瞻。蝗虫引擎的决策很快，大约一个非玩家角色需要 13ms。

原来的 Q3A 规则花费了 6 个月时间开发了 50 000 行代码。蝗虫版本删除了所有的规则，花了两星期时间，仅用了 2 000 行代码就完成了——极大地节约了成本和时间。在质量上，蝗虫引擎使非玩家角色做出更多的理性决策，是一个更具有挑战性并且更狡猾的对手，因此增强了玩家的游戏体验感，同时也提高了非玩家角色的智能。

3.1.8　相关工作

Soar（[Laird00]和[LairdDuchi00]）与我们所做工作思路最为接近。Soar 的独特性在于它使用规则来进行决策，并且也使用了后向链和子记分来解决前进过程中的僵局以及在有竞争关系的规则中做出选择。比如，为了控制真人玩家的特征，它使用了 715 条规则、100 个操作符（用来改变内部状态），20 个子状态。因为 Soar 是一个认知框架，它也包含了人类的记忆模型，比如有限的工作记忆、符号处理、归类。相对而言，我们的游戏智能的实现方法更多地关注了性能而不是人类的认知模型。

Download at www.Pin5i.com

3.1.9 结论和未来的工作

未来的前景是清楚的，那就是制作出一个像现在的图形系统一样好的游戏人工智能系统。我们的研究表明，尽管基于规则的人工智能实现方法已经被广泛使用，但是它不能帮助我们实现最终目标。早在 20 世纪 80 年代，象棋程序的设计者就已经认识到这一点，并完全抛弃了基于规则的设计方法而采用了基于模型的设计方法，这导致了在 20 世纪 90 年代，象棋程序击败了当时的象棋世界冠军卡斯帕罗夫。此后，游戏人工智能的设计者们才开始认识这一点。这篇文章详细介绍了在蝗虫人工智能引擎中所使用的基于模型来制定决策的方法。我们发现蝗虫引擎与传统的基于规则的实现方法相比带来了几个明显的好处，也提高了决策制定的效率。

3.1.10 参考文献

[Laird00] Laird, J., "It Knows What You're Going to Do: Adding Anticipation to a Quakebot." AAAI 2000 Spring Symposium Series: Artificial Intelligence and Interactive Entertainment. AAAI Technical Report SS-00-02.

[LairdDuchi00] Laird, J. and J. Duchi, "Creating Human-like Synthetic Characters with Multiple Skill Levels: A Case Study using the Soar Quakebot." AAAI 2000 Fall Symposium Series: Simulating Human Agents.

[Rabin02] Rabin, Steve, "Implementing a State Machine Language." *AI Programming Wisdom,* Charles River Media, 2002: pp. 314–320.

Download at www.Pin5i.Com

3.2 独立非玩家角色合作行为的实现

Diego Garcés, FX Interactive
diegogarces@gmail.com

在大多数现代动作游戏中，玩家通常都要面对多个人工智能控制的敌人。因为玩家对有挑战性的游戏的要求越来越迫切，所以我们需要制作更复杂的行为。但是仅有单个的复杂行为也还不够的，通常来说，更重要的是在合适的时间和地点出现适当的行为。如果行为间的合作问题没有很好地解决，那么尽管非玩家角色都有着共同的目标：消灭玩家并提供玩家有趣的游戏体验，然而非玩家角色却表现得好像忽视了其他非玩家角色的存在一样。

要为玩家提供一种真实智能的感觉，一群非玩家角色必须在一起工作并且相互配合地行动。

当一个游戏智能代理试图决定在什么时间、什么地点发生一个什么行为的时候，它不得不考虑很多和它的内部状态相关的诸多因素。个性、健康值、装甲值、弹药值或者当前的武器都可能成为决定因素。

然而，一个更为重要的因素是对其他非玩家角色最可能发生的行为进行判断。当把这一点考虑在内的时候，非玩家角色就好像在相互合作完成一个共同的目标，而不是相互阻碍。阻挡另一个非玩家角色的射击路线、企图从同一个地点攻击或者对同伴的行为毫无察觉，这些都是常见的人工智能错误。这些错误使得玩家忘记了个别的行为（尽管这些行为很完美）而专注于探察敌人的愚蠢，最终破坏了有趣的游戏体验。

这篇文章讨论了一些可以容易地添加的机制，这些机制可以使独立的非玩家角色具有更多的合作性。尽管每一个非玩家角色独立地决定它自己的行为，增加一些附加的信息并简单地进行通信将使得游戏看起来好像真的发生了合作行为一样。

3.2.1 可能的解决方案

和很多人工智能问题一样，合作，能够用很多方法来解决。一种解决方案是什么也不做。一些游戏类型避免了添加复杂的合作行为，因为敌人被假定为只会混乱地行动。然而，通常的情况并不是如此，有两种传统的方法来解决合作问题：通过集中控制的游戏实体，或者通过给每一个游戏实体更多独立的控制。

1. 集中控制的方法

传统地，人们在游戏中使用集中控制的解决方案。在这样的解决方案

Download at www.Pin5i.Com

中，一个并不一定要代表某个真实的游戏实体的智能代理分析玩家和所有非玩家角色的状况。当得到所有的信息后，它进行计算得到一种合作策略。为了执行这一策略，它给每一个人工智能代理发出指令。如果一切顺利，所有的非玩家角色将以一种完美的合作方式进行行动，尽管他们只是在盲目地执行指令。

一些游戏的设计者需要严格地控制非玩家角色的行为。在这样的情况下，固定的且缺少独立性的组织方法将会是最好的选择，因为这样设计者可以更好地实现控制的层次。

尽管一些游戏获益于智能代理的自治性，然而，对于合作行为采取集中式的解决方案能够提供一个更容易控制的系统。如果设计者需要一个在特定情况下发生的具体的合作行为，只需要在集中式的智能代理中直接添加一些优化代码就可以了，这比修改一批独立的机制以使它们在特定的情况下有特定的行为要来得容易。而且，集中式的方法也容易修改排错，容易将行为和不同的非玩家角色对应起来。

集中式的方法的最大的缺点在于扩展性：当系统逐步变大时，这样的解决方案也会变得越来越复杂。系统是怎么变大的呢？当条件变多、智能代理的行为变复杂、更多的非玩家角色需要控制时，系统就会变大。这些问题是未来游戏设计中有待解决的方面。不仅如此，因为集中控制方法是统一的结构，因此在其计算策略时或者在其执行结束之前，它很难根据变化做出改变。

总之，集中式的控制方法没有很好的扩展性，而且不能很好地适应变化的环境，并且可能耗费很多的 CPU 资源而影响非玩家角色的响应能力。此外，它很难处理在设计阶段没有预见到的情况。随着多核处理器在下一代游戏机和当代 PC 中的出现，给集中式结构的实施带来了更多的障碍，因为将一个进程分开使其能够充分利用多处理器的优势并不是一件容易的工作，如果有多个处理器，那么就更困难了。

2. 独立控制的方法

另一种解决方案[Orkin03]源于对得到公认的分布式计算策略的探入探究。在分布式的系统中，前面提到的集中式的很多问题都得到了解决。因此，值得推荐的解决方案是将合作任务分配到所有的智能代理中。这样的组织方法提供了一个可扩展的、灵活的结构，通过它，每一个智能代理都可以被分配一个独立的线程来执行。非玩家角色们通过自行决定要做什么而变得独立起来。他们不再仅仅是执行指令，而是主动地参与了决策的过程。

合作，通过在独立的智能代理间通信和共享信息而自然地显现出来，而不是显式地通过编码实现。当解决简单的局部问题的时候，很容易找到一个解决方案为每一个智能代理选择一个最好的行为，接着，才把合作当作一个单独的问题考虑。任何时候都没有全局的策略。这种方法允许当大的游戏状况发生变化时出现相对应的一些行为的调整，而不必重新计算一个全新的全局策略。

因为非玩家角色是完全独立的，当我们需要的非玩家角色增加时，这种解决方案可以自如地扩展系统规模。当添加新的非玩家角色，并且要让他们使用相同的合作机制时，独立控制方法显然比创建一个集中控制实体来考虑处理这些新非玩家能要容易得多。分布式的解决方案也解决了多核硬件处理的问题，因为，每一个非玩家角色都可以被指定到一个不同的线程上执行。多线程的编程问题仍然需要面对，但是在这样的计算框架下，任务的分解已经不是问题了。

任何人工智能控制的实体的响应速度都是不需要担心的，因为每一个人工智能代理独立地决定该做什么，它的决策过程的计算时间要比集中式解决方案中全局策略的计算时间要短。

尽管有这些优点，但是没有系统是完美的，此法也不例外。让一组独立的实体合作起来

Download at www.Pin5i.Com

以使他们做应该做的事是一个挑战。而且，在没有专用机制的情况下，对群组行为进行细微的控制会非常困难。

3.2.2　非玩家角色的结构

我们致力于完全独立的非玩家角色，所以系统必须提供他们这么做的所有能力。有了这个目标，我们就可以选择一个灵活的结构来表达游戏智能代理。它包含一些控制单元，这些控制单元用来完成智能代理在游戏中必须执行的任务。

一个典型的非玩家角色应该能够动作，他能够在关卡中移动，感觉周围的世界，并在感觉的基础上采取行动。为了完成这些任务，系统定义了动作、移动、感觉、行为控制器（见图 3.2.1）。

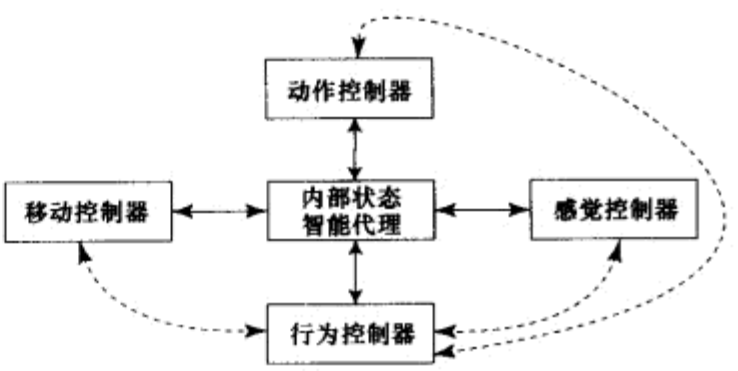


图 3.2.1　非玩家角色的结构和连接

这篇文章将特别介绍行为控制器部分。行为控制器用来控制行动的执行，也就是控制在游戏中的每一个时间片代理做什么。这些行动有的很简单，比如跳开来以免被击中，有些复杂，比如关卡首领的攻击例程。他们也可以组成树状结构或层次结构来将简单的行动构建成更为复杂的行动。控制器包含一组正在活动的行动，这些行动在每一帧画面更新时执行，并且也有一组等待的行动，这组行动持续地监听执行条件是否符合，在符合的时候就开始执行行动。

这样的组织方法非常有用，它给系统带来了很大的灵活性。在同一个时间可能有多个行动被激活，比如，如果可能，在射击的时候从一个位置移动到另一个位置。等待的行动允许执行那些需要一段时间以后才能开始执行的行动（比如：还没有完全确定），因为当新的行动在搜集所需的所在数据时，旧的行动将依然保持活动的状态。路径计算有时候是费时间的，而该行为需要一条路径来启动执行（比如：去掩护）。这不应该让角色静止地站着等待路径信息；相反，角色应该接着执行先前的行为，直到得到路径信息。

想象一下，我们要设计一个进行巡逻并且当发现玩家就会发起攻击的角色。这可以简单地通过上述的设置等待行动而完成。我们设置一个等待行动，行动执行的条件是看到玩家。我们只要把巡逻行动设置为活动行动，攻击行动设置为等待行动，那么非玩家角色就会处理剩下的工作。

这些特性使合作系统的要求不再那么高了。我们可以花更多的时间去调节各种行为使其更好地合作而不用担心角色被冻结。

3.2.3　合作的机制

到目前为止，每一个非玩家角色都是执行当前的行为，并评估由非玩家角色决策定出的等待行动能否执行，比如攻击或者逃跑。然而这不足以造成一个完全智能的印象。如果非玩家角色都允许独立地自己决定做什么，他们将产生完全混乱的行为。现在，我们如何让非玩家角色产生相互关联的行为以使玩家感觉到游戏的智能性、角色的合作和挑战性呢？

因为非玩家角色是自己做出决定，最好的办法是提供一个机制让他们来交换信息。通过这种方法，他们中的每一个都可以得到所有需要的信息以便做出一个深谋远虑的决定，确定

Download at www.Pin5i.Com

执行什么样的行动，在哪里执行。

这篇文章提出了在非玩家角色之间交换信息的若干机制：

- 攻击口；
- 独占区域；
- 触发系统；
- 敌人事件；
- 玩家互斥器；
- 允许区域；
- 黑板。

这些机制不是企图建立超级智能的代理。它们只是提供简单的解决方案，当这些方案组合起来工作的时候，就能够模拟出多个智能代理协同工作的效果。

在下面的部分，我们将叙述上述的每一种机制，从概念上加以解释，并探讨使其行之有效的原因，同时考察它们如何给游戏角色带来合作的感觉。

1. 攻击口

今天的大多数游戏中，在同一时间里很多个敌人会对玩家发起攻击。如果他们都单独地做出决定，他们可能都觉得近距离攻击玩家最好。在这样的情况下，所有的敌人将包围这个玩家，并可能相互阻挡。显然，这不是一个期望的结果。这种现象的发生是因为非玩家角色在作决定时缺乏其他非玩家角色的必要信息。

作为一个普遍的规则，最好将敌人分散在有效空间中而不是挤在玩家周围。最好的办法是建立这样的一个非玩家角色，他能够进行情况分析，考虑地形拓扑结构、其他非玩家角色的位置、游戏需要的难度，最后他做出决定从哪儿攻击。然而，因为其复杂性和可用于此类计算的 CPU 时间有限，拥有这样的智能代理并非总是可行的。

攻击口以一种方便的方法解决了这个问题。攻击口就像一个令牌，标记了一个最小攻击距离，非玩家角色有什么样的令牌，就表示他对玩家的攻击不能小于什么样的最小距离（见图 3.2.2）。

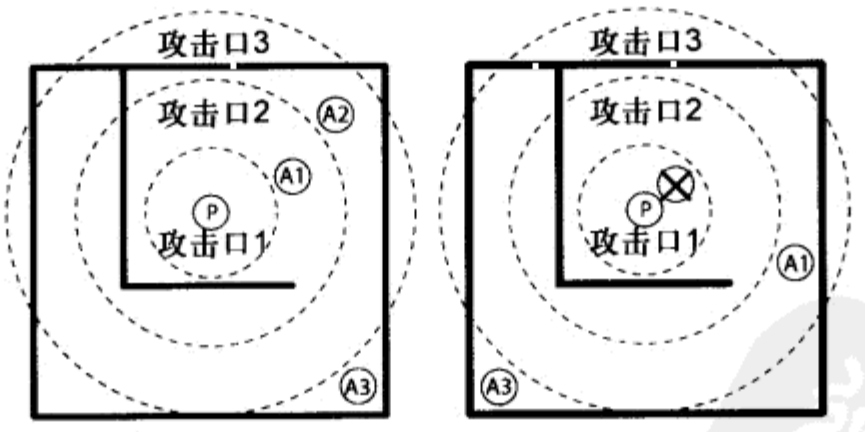


图 3.2.2 在左侧的图中，3 个代理（A1-3）每个有一个攻击口，他们正在攻击玩家。在右侧的图中，代理已经移动了。A1 移动到了和其攻击口相符合的位置，A2 移动到了比其攻击口要求更近的位置，A3 已经移动到了一个正确的位置，尽管他经过了一段离玩家更近的位置

这些攻击口可以调整分散非玩家角色，它们可以被指定到每一个关卡和每一组敌人中。每一个进行攻击的非玩家角色必须占有一个攻击口，并且他离玩家的距离不能比攻击口规定的距离近。

每当非玩家角色寻找一个位置进行攻击的时候，他就请求一个最好的攻击口（距离最近

Download at www.Pin5i.com

的攻击口）。当得到这个攻击口后，他就开始进行计算以找出符合攻击口要求的攻击点。当攻击结束后，非玩家角色就尝试换一个最不严格的攻击口——也就是说，找一个离玩家至少要比当前的攻击口要远的攻击口。如果新选择的攻击位置比原先的最好攻击口所允许的远，那么非玩家角色就放弃当前的攻击口而选择那个较不严格的——所有的非玩家角色都允许另一个非玩家角色使用原来的攻击口以使后者可以在需要的情况下更接近玩家。

2. 独占区域

尽管攻击口解决了非玩家角色拥挤的问题，但是并不能解决所有的问题。因为它只不过设置了一个最小距离要求罢了。这一点可以用相互重叠的攻击口充分说明。在仅有一个最小距离要求的情况下，两个非玩家角色可能选择从同一个位置来进行攻击，如果这个位置对两个攻击口来说都符合距离要求的话，在这样的情况下，非玩家角色没有办法知道该位置已经被另一个非玩家角色占据了。

另一个多敌人攻击的问题是阻挡射击线路。一个敌人正在一个地点攻击，这时忽然另一个敌人闯过来挡住了火力的线路，并停下来开始射击，从而迫使第一个敌人停止射击（因为他被第二个家伙挡住了）。这极大地降低了游戏的智能感并且也产生了其他问题，比如敌人被友军的火力击中。

独占区域正是解决这些问题的一种方法。它将对场景中的区域和点的可用性增加另一个限定条件。独占区域表明这些区域已经或即将被某些非玩家角色所占有。因此独占区中的点就可以没有限制地做为攻击点。

每一个非玩家角色都登记他们正在使用的或者将要用于攻击的区域，包括相对应的火力线路。这个区域可能是围绕攻击位置的一个圆或者是非玩家角色独自保护的一个区域。

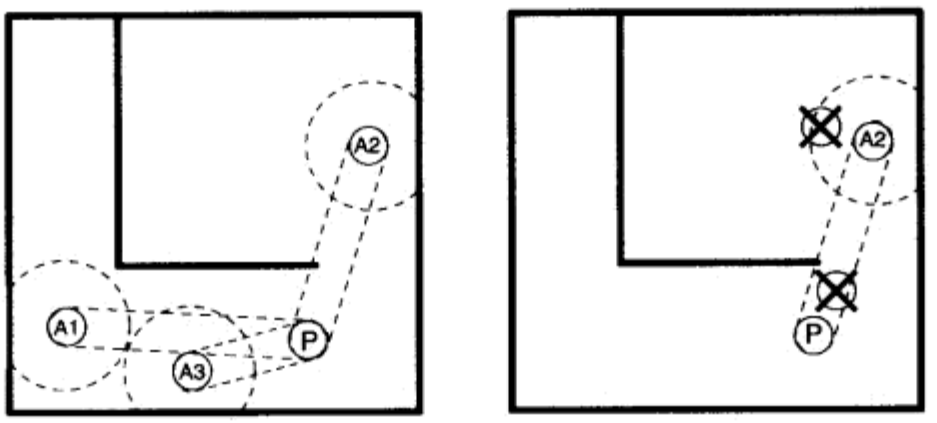


图 3.2.3　代理 3 和 1（A3 和 A1）不能移动到那个位置，因为火力线和 A2 周围的区域已经分别定义了

这些区域在一个公共的地方进行登记以使其他的非玩家角色能够获取这些信息。当寻找一个新的攻击点的时候（比如，找一个好的掩护地点），那些在独占区域内的位置在搜索时会被排除在外。

这样的一个机制确保了两个非玩家角色不会选择同一个地点进行攻击。不仅如此，独占区域的拓扑结构、大小、形状都可以根据敌人的类型、行为或者关卡要求进行自定义。

3. 触发器系统

触发器系统在游戏人工智能中被广泛地运用。它们有多种用途，其中的一个是通信。关于怎样为游戏引擎建立一个触发器系统，细节可参看[Orkin02]。

Download at www.Pin5i.Com

有时通信是基于位置的。在这种通讯类型中，触发器系统在游戏实体间的通讯中非常有用。在非玩家角色发出话语声明或者发出声音时，触发器就变得特别方便。比如，如果一个非玩家角色发现了玩家并发出叫声让对方停止，这个叫声应该为邻近的非玩家角色听到，因而他们可以赶过来帮助。发出警告的行为可以用在发出叫声的地方放置一个触发器来简单地实现（见图3.2.4）。在非玩家角色之间的这种类型的通信可以使用户感觉到非玩家角色的智能，因为一些游戏事件（这里是声音）在游戏中产生了一个真实的反应（在这里，另一个非玩家角色赶过来察看或者帮助）。发出声音后产生的反应将比一个事先编码的事件来得真实，因为事先编码的事件不能完全地和玩家的移动相配合。

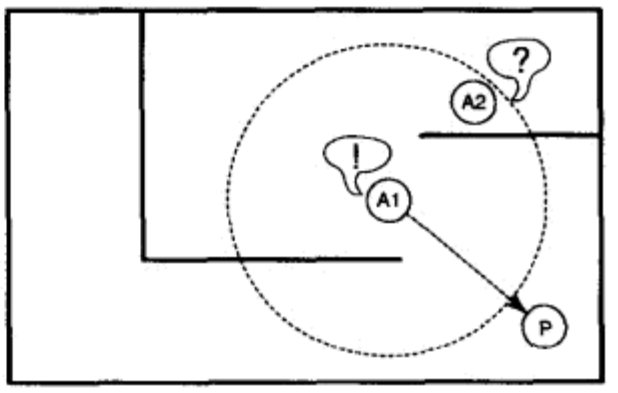


图 3.2.4 代理 1 看到了玩家并设置了一个触发器（A1）。代理 2（A2）听到了并警告自己

触发器大多数是用在声音效果中（比如爆炸、射击、开门或者非玩家角色的对话）。但是触发器并不仅仅限制在显式的通讯中。比如一个死亡的非玩家角色，他可能想和任何在附近能看到他的尸体的人“通信”。收到这个通讯的敌人可以发警告并且开始查找入侵者。这种类型的通讯方式，使用触发器比人工地检查附近的非玩家角色是否还活着要容易得多。

4. 敌人事件

前述的机制提供了在非玩家角色间的一些不太直接的通讯方式。他们通过获得攻击口、登记独占空间或者设置触发器广播事件来达到表明自己意图的目的。通过这些机制提供的信息，其他的非玩家角色就可以相应地调整自己的行为。

然而，在一些情况下，更直接的通讯是需要的。比如，如果一个非玩家角色挡住了另一个非玩家角色，被挡住的角色应该能够直接给阻挡的角色发送消息，通知他让开。

一个典型的消息传递框架可以用在这种情况下。消息可以被送到一个特定的非玩家角色、一组非玩家角色或者所有角色。消息的处理和游戏的每一帧同步，不过这一点可以根据实际的情况改变。在收到消息后，如果这个消息被接受的话，非玩家角色可以改变他的行为。（代理能根据消息做些什么？发来的消息比当前的行动有更高的优先级么？）更多有关消息传递机制的细节请参看[Rabin02]。

敌人事件可以和触发器组合应用。在一些游戏类型中，动态组队非常有用。在某些情况下，一个非玩家角色决定编组攻击。他产生一个触发器来向附近的单元声明已发布了一个组队要求。收到触发器的非玩家角色用一个敌人事件作为回应（给触发器源，队长），表示他们准备加入这个组。队长就可以分析这些回应，以决定哪些非玩家角色将加入这个组或者完全撤销组队召唤（比如，因为一个新的游戏情况发生了，或者仅仅因为没有合格的队员），并切换到另一个战术。

5. 玩家互斥器

攻击口可以通过限制离玩家很近的敌人数目来达到将非玩家角色分散在一定的距离范围内的目的。有时这还不够，我们还需要控制同时对玩家发起攻击的非玩家角色的数目。这在格斗战斗类游戏中特别明显，此时只有一个敌人允许和玩家进行战斗，其他的敌人只能等待或者使用远距离攻击。事实上，对于一些行为，把它看成是对玩家的独占使用是有助于处

Download at www.Pin5i.Com

理的。把玩家看成一个在多个非玩家角色间共享的资源，就可以使用在多线程环境下的传统的资源存取机制，特别是互斥器。

互斥器是一个标记谁可以获取共享资源的令牌。在这样的情况下，共享的资源是玩家。当一个非玩家角色想进行格斗攻击时，他就试图获得互斥器。如果可以得到（比如其他的非玩家角色没有拿着互斥器），那么非玩家角色就取得互斥器，其他的非玩家角色如果想要进行格斗，就不得不等待，直到当前的互斥器拥有者完成他的攻击。

6. 允许区域

在一些特定的关卡或任务中，将多个非玩家角色分布在不同的区域是重要的。想象一下见图 3.2.5 那样的相互连通的房子。设计者可能希望一个特定的敌人从一个房间发起攻击，而其他的敌人控制其他屋子。如果没有其他的限制，第一个屋子里的敌人就可能接近玩家或者走出指定给他的屋子以寻找一个好的掩护点。

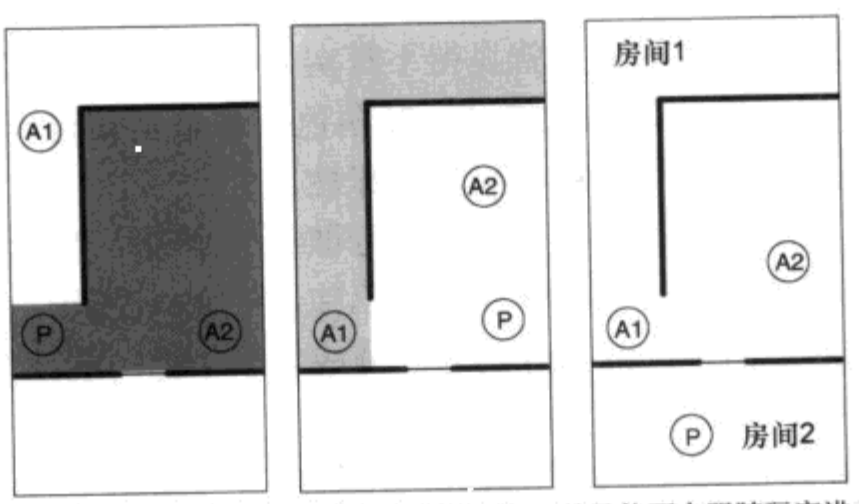


图 3.2.5 允许区域。（左）代理 2（A2）保护它的区域和门，并且他不会跟随玩家进入代理 1（A1）的区域。（中）A1 不能进入；他必须从那里攻击。（右）玩家能躲到房间 2 里；敌人不会跟随

为了使这一切成为可能，我们应该定义敌人允许移动到的区域、允许攻击的区域或者通常执行任何行为的区域。非玩家角色可以穿过不允许的区域进入允许区域的一个位置，但是任何目标地点都不能在允许区域之外。

使用允许区域，游戏设计者可以设计特定的情况。这样的机制允许设计者将一些设计态的知识直接加到非玩家角色的合作行为中。允许区域也能够由非玩家角色来设立——比如，一个方队的队长能够指定每个方队成员的区域，这样来分配可获得的空间并以可能的最佳方式保护关卡。

7. 黑板

黑板系统[Isla02]是一个在非玩家角色之间交换和发布信息的重要的工具，也可以用来达成合作。它是一个集中式的信息共享机制——一个公共的地方，每一个非玩家角色都可以在这里发布信息以使其他使用黑板的非玩家角色能够根据新的信息来改变自己的行为。这样就实现了一种信息发布者和阅读者之间的隐含的合作。可以用黑板发布各种不同的信息——从非玩家角色的企图、非玩家角色的理解到行动建议。

因为黑板要保存各种不同的信息，所以为此目的而特设的黑板不会非常灵活，因为每添加一个新的系统信息类型就需要修该黑板的定义。黑板系统在游戏中被作为一种强有力的简单的通讯机制广泛使用。因此，值得花一些时间建立一个灵活的系统并使之能够用到别的地

Download at www.Pin5i.Com

方，或者扩展智能代理而无须对黑板的内部工作做任何改动。

一个通用的资产系统提供了一个好的解决方案。资产可以是任何类型的并能通过名字或者身份号码快速存取的信息。采用这样类型的系统，就能够非常简单地添加新的信息到黑板并且将其状态容易地连续存到硬盘。这样的持续性可用来实现在不同的游戏关卡之间利用已存盘的游戏维持信息，或保持数据的延续性。

对传统黑板数据的优化可以使黑板功能变得更加强大，比如，设定一个过期时间就能够自动处理过期的信息，并且避免黑板容量过度增长占用大量的内存（比如存满了大量过时的信息）。

3.2.4 例子：合作寻找玩家

在这一个部分，我们将考察在某一关卡中多个敌人寻找玩家的典型实例。它将会用到一些前面已经解释过的机制。

在游戏引擎中，玩家的精确位置是知道的。然而，如果没有非玩家角色曾经看到躲藏的玩家，直接把敌人送到玩家躲藏的位置就显得不真实。如果我们在开发渗透类游戏（玩家可以偷偷地移来移去而不被发觉），欺骗性的非玩家角色或者总是知道玩家在哪里，或者总是找不到玩家，这些情况都会完全破坏游戏体验的趣味性。

组织搜索的一种可能的办法是将非玩家角色分散在一个关卡里，这样他们能用更少的时间检查更多的地方。每一个非玩家角色都登记自己独占的允许空间，并在其内进行搜索。每个关卡都可以由设计者通过简单的网格分割方法分成若干个简单的区域。

因为非玩家角色登记了一个搜索区域，所以他会移动到这个区域最近的点并继续开始检查这个区域。非玩家角色在黑板上发布哪些点已经被找过了，并附带一个过期时间或者其他设计态指定的信息。这样非玩家角色就知道在哪些点的寻找是不成功的，并且其他的非玩家角色也知道这些信息。这一点是重要的，因为区域可能有一点相互覆盖，如果一个非玩家角色完成了检查他的区域就可以移动到下一个区域，非玩家角色们能够共享同一块区域（参看图 3.2.6）。

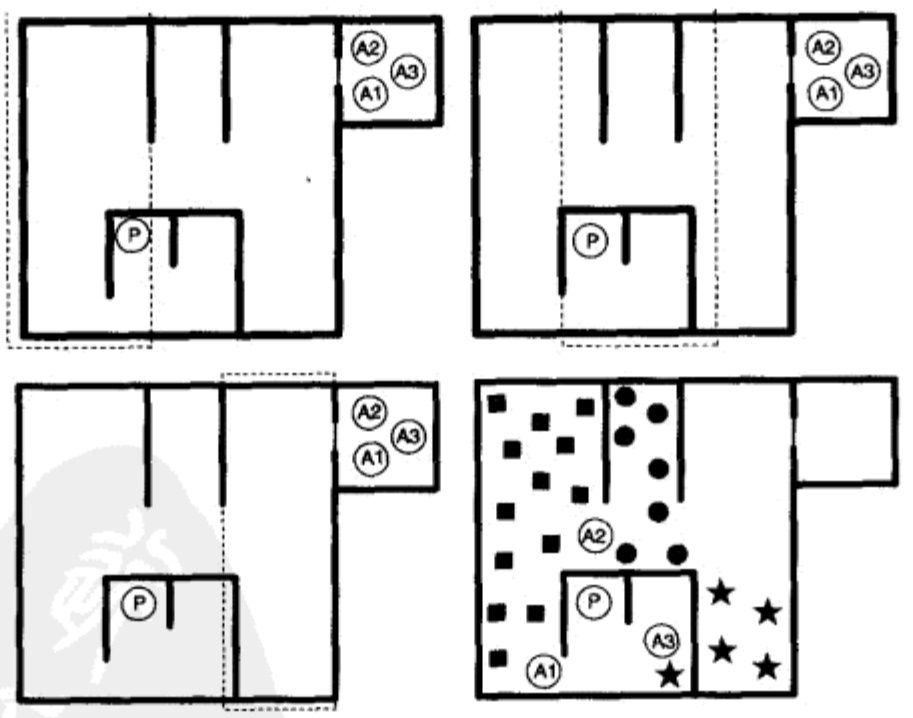


图 3.2.6 上左，下左：搜索区域。下右搜索过的点

如果一个非玩家角色找到了躲藏的玩家，他可以通过在自己所在的位置设置一个触发器

Download at www.Pin5i.Com

来激发警告并让其他非玩家角色过来帮忙。不仅如此，如果需要用特定的行为来面对玩家，非玩家角色也可以与特定单元通讯，比如激战或者狙击。这样的通讯可以通过非玩家角色的请求增援事件来完成。

3.2.5　结论

在这篇文章中，我们介绍了几种使用独立的智能代理来很好地表达非玩家角色方法。没有集中控制的命令发给每一个非玩家角色。他们基于自己掌握的信息选择合适的行动。这些信息不一定全部从环境中获得，也可以从其他的非玩家角色那里获得，并且后者使得非玩家角色之间可以相互合作。如果一个非玩家角色知道关于其他非玩家角色正在做什么的细节信息，他就能对自己的行为做出一个正确的选择而不影响其他智能代理的兴趣。

随着下一代多线程游戏要求的增长，游戏独立智能代理变得更重要了。由于多个线程会被分配到不同的处理器，你可以将 CPV 密集型的计算（比如路径寻找或者覆盖式搜索）放在不同的线程上，并且让行为控制器去处理那些不能确定的等待时间。

因为独立智能代理是彼此相互独立的，所以可以将他们分配到所有可供使用的处理器。独立代理的工作不会发生什么变化，只是通讯机制必须根据多处理器间信息交换的要求作相应的改变。

3.2.6　参考文献

[Isla02] Isla, Damian, and Bruce Blumberg, "Blackboard Architectures." *AI Game Programming Wisdom,* Charles River Media, 2002.

[Orkin02] Orkin, Jeff, "A General-Purpose Trigger System." *AI Game Programming Wisdom,* Charles River Media, 2002.

[Orkin03] Orkin, Jeff, "Simple Techniques for Coordinated Behavior." *AI Game Programming Wisdom 2,* Charles River Media, 2003.

[Rabin02] Rabin, Steve, "Enhancing a State Machine Language Through Messaging." *AI Game Programming Wisdom,* Charles River Media, 2002.

Download at www.Pin5i.Com

3.3 针对游戏的基于行为的机器人架构

Hugo Pinto、Luis Otavio Alvares
hugo@hugopinto.net

最近，因为三维动作游戏领域和移动机器人领域的相似性，在游戏开发中对于机器人架构的兴趣日渐增长起来。

机器人必须决定每一步都做什么，这仅仅取决于他对环境的感知和他自己的知识。机器人必须很快地做出决定，在很多个小时以内都不需要人工干预并处理很多有可能相互冲突的目标。机器人需要在有限的处理能力的情况下完成感知和决策的过程。在真实的世界里行动，机器人面对的是一个由各种不同实体构成的快速变化的环境。

现代游戏的动作环境也显示出了相似的性质。在三维游戏里的智能代理也面对着一个类似的连续的情况。这样的环境有很多实时的可以选择的行动和与其他实体进行的交互。计算能力是有限的，代理有着相互冲突的目标，比如杀死敌人并保证自己的安全。最主要的区别是：在一个游戏中，如果不是明确需要，游戏机器人感知器中不会有噪音，并且感知是不昂贵的。

由于能够带来更多的可信度，不欺骗玩家的智能代理逐渐变得流行起来。不进行欺骗，我们指的是一个角色程序有着和玩家相同的信息，不多也不少。这个角色程序感知的只是他能够看到的，在一定距离内能够听到的，通过询问、注意或者被告知而推理获知的信息。这就给他的行动带来了很多的真实感，从而导致了玩家更好的沉浸性。我们看到不欺骗的角色程序可以被做成一个机器人：我们给他提供感知器和行动器，编制出他自己的决策机制。有利之处是机器人领域有许多经过很好测试、被证明可行的解决方案。我们有大量唾手可得的技术可直接用来在三维游戏中修改移植，并进行测试。

专业的开发人员和学术人员已经考察了机器人架构在游戏中的应用，并得出了令人鼓舞的结果。[Yiskis03]和[Champandard03]已经使用了包容体结构[Brooks86]，并用后者开发了雷神之锤 II 的角色程序。[Pinto05-a]已经应用了扩展的行为网络[Dorer04]来设计虚幻竞技场游戏的智能代理。在这篇文章中，我们将介绍这些架构和它们的应用，探讨其他一些有前景的技术，并指出所讨论的系统的扩展问题。

3.3.1 包容体结构

包容体结构[Brook86]是被作为一种自治机器人的控制结构而被提出

Download at www.Pin5i.Com

来的。机器人有着多个目标和几种感知能力。并且在面对噪音和小的系统失败时需要具有稳定性。包容体结构已经被用来建造了几个机器人，包括一个在真实的办公室里面搜集空可乐瓶的机器人。在游戏中，它被用来控制雷神之锤 II 的角色程序（bot）[Champandard03]。

包容体结构程序有着一个非常突出的优点，那就是我们不是按照信息流来划分智能代理的层次，而是基于我们需要的智能代理的行为类别来划分——按照它的能力来划分。

比如我们想要建立一个简单的进行攻击的智能代理。图 3.3.1 所示为一个按照程序流的分割方法。我们有一个模块用来把感知到的信息翻译成为一种合适的形式给计划模块使用。计划模块把预先决定的计划发送给执行模块，执行模块将顺序地执行该计划。这样的方法有一个问题，那就是新的感知和行动被添加进来时，计划模块可能面临组合爆炸。另一个问题是我们经常需要重新计划，使用复杂的试探性机制，因为计划有可能在执行的过程当中已经过时了。

一个按照能力来划分的方法如图 3.3.2。在这里我们添加更多的感知器也不会带来问题，因为每一层仅使用几个感知器并且直接与一个输出模块相连。重新计划也不是问题，因为系统是快速的，并且所有的层都并行地执行。

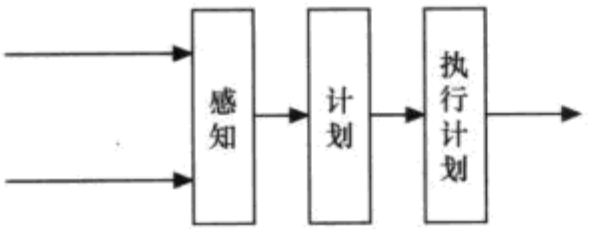


图 3.3.1　基于程序流的射击代理划分方法

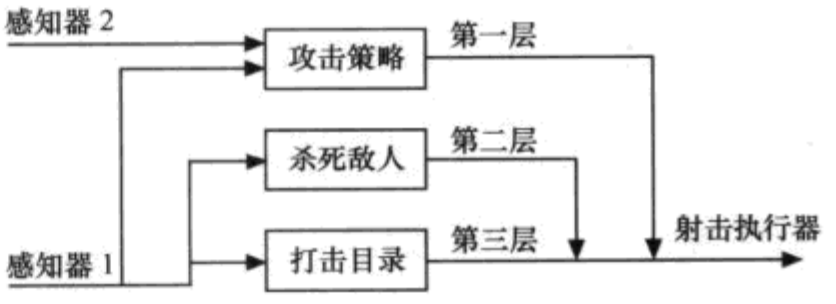


图 3.3.2　基于能力的射击代理划分方法

我们看到较高的层为智能代理指定了一个更特殊的行为类别，并在其下包含更常规的行为层。杀死敌人的能力（第 1 层）包含击中目标能力（第 0 层）。类似地，攻击策略包含了杀死敌人的能力。因此，包容体结构正如名字指出的那样：上面的层包含了下面的层。

每一层都可能包含若干个行为模块，它们的执行序列送到一个行动器就构成能力。在图 3.3.3 中，我们可以看到击中目标能力由三个行为模块构成：发现实体、选择目标和射击。

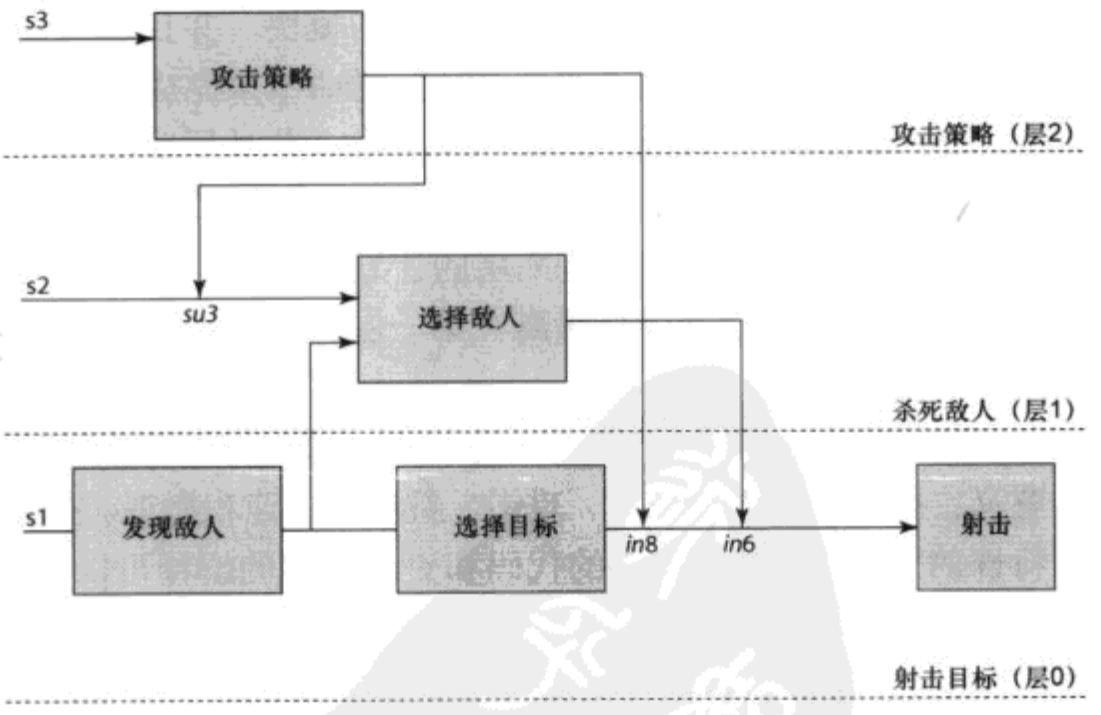


图 3.3.3　包容体结构的射击代理细节

Download at www.Pin5i.com

高层的模块可以取代低层模块的输入或者取代它们的输出。高层模块可能收到来自于低层的任何模块的输入。图 3.3.3 所示为杀死敌人这一层和射击目标这一层的连接关系，也显示了攻击策略层和它下面两个层的连接关系。注意，我们有三个感知器——s1，s2 和 s3，还有一个单一的行动器——射击。杀死敌人这一层观察所有被发现的敌人并最终输出一个敌人的坐标，以取代下层的选择目标模块的输出。攻击策略层有一个输入（s3），它的输出可以替换选择敌人模块的输入（s2），也可以取代选择敌人模块的输出。整个结构的行为可以这样来描述：射击目标层随机地选择一个目标进行射击。杀死敌人层检测在视线范围内是否有敌人，并且在有敌人的情况下，结合相关信息（从 s2 获得）来确定一个敌人作为攻击目标。攻击策略层能够影响选择敌人模块的行为，这是通过替换这个模块的一个输入来完成的。当然，也可以直接地替换下层的输出。

有三个设计问题值得讨论。第一，当行为模块设计实现之后模块本身就不再改变——也就是说，我们只操作它的输入和输出。第二，上面的每一层都可能依赖于其下层的某一个模块，这就导致了自底向上的设计。第三，每一个行为都是异步执行的，这自然地促成了对并行处理的应用。

我们看到这样的机制的稳定可靠性自然地来自于它的组织结构：如果上层的模块停止正常运行，下层的模块仍然会继续它们的工作。如果我们的智能代理因为某种原因不能够产生一个攻击计划，它仍然会攻击它看到的敌人。

现在我们已经清楚包容体结构是什么，它怎么工作，我们可能要怎么设计它。那么，把它应用到游戏设计时，有没有一些需要遵循的特别的原则？[Yiskis03]建议用下层的模块来完成直接的工作，并用上层的模块来完成长远的目标。他建议加强限制条件，不允许高层的行为能力绕过其下层—也就是说，高层的能力只能够通过发送消息给它下面的层来完成它们的目标。[Champandard03]提出了一个原则，那就是低层能力应该是在默认情况下被应用的能力，而高层能力是对默认规则例外的补充。

在[Yiskis03]中给出了一个在游戏角色动画控制中使用包容体结构的例子。这个结构自底向上共有 4 层：移动、行动、行为和策略。下层的移动主要负责实际的动作动画的播放并且实现一些必需的物理限制。比如，如果智能代理在跑动的过程中被障碍绊倒，那么一个“停止”和一个“返回”动画就被播放。行动层有所有行动的模块，并且这些行动仅在一段有限的时间内发生。行动层的所有行动都是通过给移动层发送命令来完成。在这里我们应用了前面讨论过的原则：所有的动画都通过对低层的调用来完成，并且行动层的任何行为都不直接与动画系统打交道。行为层包含一些需要用不确定的时间来完成的行为，在通常情况下是行动层的一些行为的循环。而策略层则制定一些高层的计划。

这样的结构的一个好处是：动画集成被隔离在第一层。当新的层添加进来的时候，我们无需担心要再修改它。

[Champandard03]将包容体结构应用到了雷神之锤的 II 游戏中。这个结构总共有 7 层，每一层都有着明确的意义：探索、检查、聚集、攻击、狩猎、逃跑和撤退。探索层是最底层的。我们看到这是一个特殊的“懦夫”机器人。他的代码和更进一步的解释可以在[Champandard03]中找到。

稳定性，速度，模块化，使得包容体结构成为实时动作领域的一个非常有趣的候选方法。与其他的基于行为的结构比较起来，这种方法的主要限制是：需要明确地指定模块之间的每

Download at www.Pin5i.Com

一个连接，并且在各种行为能力之间要有一个固定的框架。下一个框架将解决这些问题。

3.3.2　扩展的行为网络

在机器人世界杯比赛中，[Dorer04]用扩展的行为网络来为机器人选择合适的动作。那个首次使用这种扩展行为网络的队就获得了比赛的亚军[Dorer99]，并且在接下来的一年又再次在比赛中获胜[Dorer00]。除了在机器人足球比赛中获胜之外，扩展行为网络（EBNs）在游戏虚幻竞技场中的应用也颇有成效[Pinto05-b]。

扩展的行为网络是行为网络的最新进展[Maces89]，是另一种也在不断发展的基于行为的结构，详细信息在文章[Rhodes96]，[Goetz97]，和[Nebel03]中进行讨论。

一个扩展的行为网络可以看作一组相互连接的目标和行为模块，他们之间相互传递激活能量，从目标开始，跟着是模块。在每一步都选择那些不使用相同的资源且拥有更高的执行能力和更高能量的模块来执行。

扩展的行为模块的结构由一组行为模块、一组目标、一组连接目标和模块的链接、一组资源和一组控制参数构成。图 3.3.4 显示了一个虚幻竞技场角色程序的行为网络。

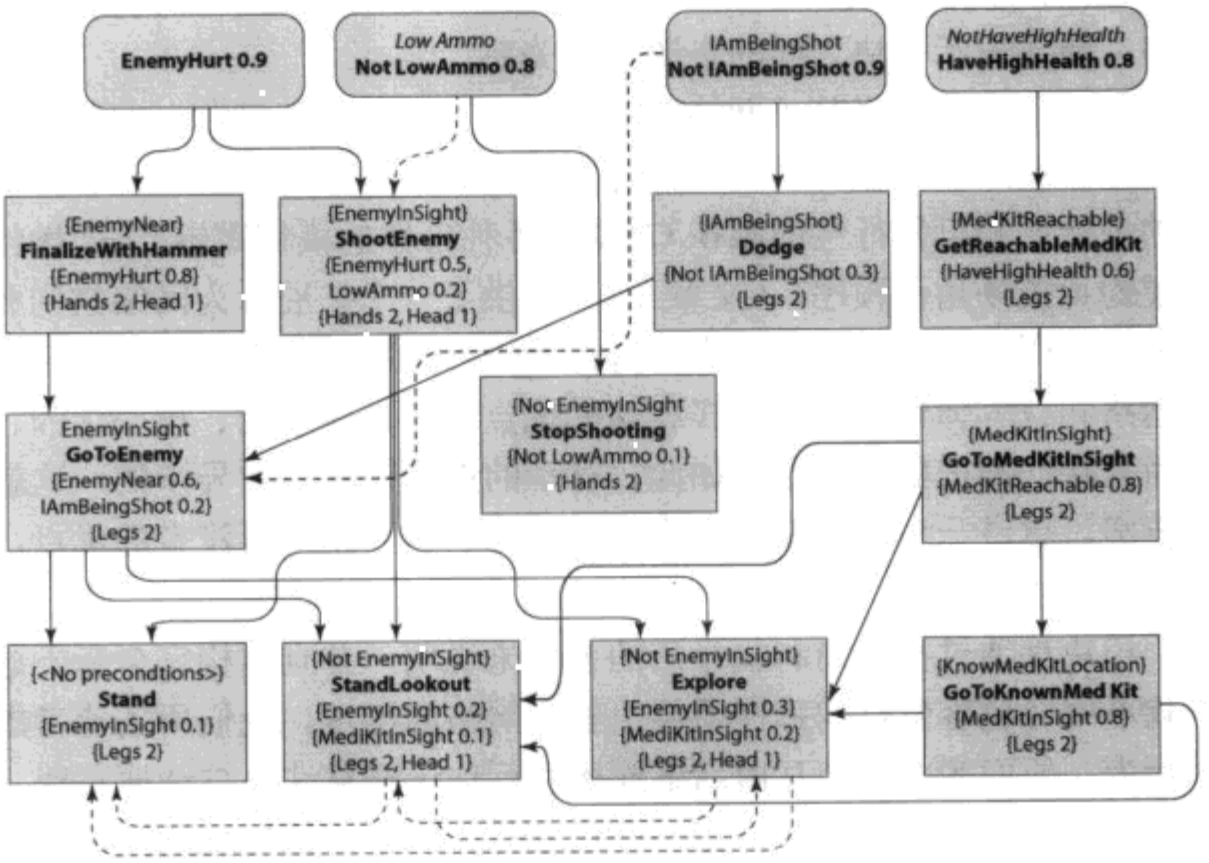


图 3.3.4　一个简单的虚幻竞技场游戏角色程序行为网络

目标的定义包含如下几个部分：一个必须被满足的命题（条件）、一个强度值和一个析取命题。其中析取命题描述了目标发生的环境，被称为相关条件。强度值是一个静态的、环境无关的表示目标重要性的值，相关条件提供动态的、环境相关的表示目标重要性的值。使用两种条件来定义目标使我们可以表示出代理所在环境不同引起的目标的重要性的变化。在图 3.3.4 中，目标“有高的健康值”的强度是 0.8，并且环境条件是“没有高的健康值”。当机器人的健康值降低时，该目标的重要性将增加。在某一个时间，一个目标的总强度值由当时的动态的和静态的重要性的值相乘来得到。网络中所有的命题（条件）都是实值的，这样就

Download at www.Pin5i.Com

允许使用模糊逻辑。

每一个行为模块被定义为一组条件、一个行动、一个效果列表和一组资源。第一个列表是一组实值的命题用来表达模块执行所需要的条件。效果列表也是一组命题（每一个都可能被否定），用来描述行为发生以后，命题值的变化。资源列表是一个两元组的列表（例如资源，数量），用来表示当代理执行行为的时候各种资源需要达到一个什么值。

连接是按照模块的效果、模块的条件和目标建立起来的。从一个目标或者模块 B 出发到达一个模块 A 的继承性连接绑定一个命题，这个命题在 A 的效果列表中，但是在 B 中，这个命题在条件列表中，命题在每一个列表中都有相同的符号，比如真（+）或者假（–）。从“敌人受伤”（EnemyHurt）到“射击敌人”（ShootEnemy）模块的连接就是一个例子。从目标或者模块 B 到达模块 A 的冲突性连接也绑定一个命题，这个命题在 B 的条件列表中，在 A 的效果列表中，两个模块的列表的对应命题将具有不同的符号。如图 3.3.4 所示中，从“非低弹药”（NotLowArmmo）到“射击敌人”（ShootEnemy）的连接就是一个冲突性连接。用这样的方法，一个模块或者目标就会企图抑制那些执行之后减弱其条件的模块，并且试图执行那些在执行之后增强其条件的模块。

在图中，目标对应的是圆角的矩形，而行为对应的是直角的矩形。实线表示继承性连接，虚线表示冲突性连接，曲线是资源性连接。箭头方向指示了继承性和冲突性连接的激活方向。在资源性连接中，箭头表示模块之间的资源依赖关系。

通过激活的传播，行动选择基于网络节点之间的相互激发和抑制。在每一步，被选择执行的模块是那些具有最高执行值的模块，并且它们都拥有执行需要的所有资源。一个模块的执行值是它们的激活性和可执行性的乘积（比如：各种条件的模糊逻辑“与”操作）。文章《一个目标驱动的虚幻竞技场角色程序》（文章 3.5）提供了一个扩展行为网络结构和它的行为选择算法的详细描述，所以在这里我们将集中讨论增强行为网络的属性和它的设计问题。

扩展行为网络和包容体结构具有很多相似的优点：都是快速的、模块化的、可以渐进的设计的方法，而且它们都很稳定。不仅如此，行动的选择是并发的异步的，这就允许采取快速的并行执行方案。更进一步，扩展行为网络有一些包容体结构所没有的优点，并且突破了包容体结构的一些限制。

因为每一个模块都通过执行前和执行后的条件来说明，并且结构完全基于这些信息来连接，这样我们就不用像包容体结构那样手绘出整个网络的连接。我们所需要考虑的是设计实现目标和各种行为。他们的内部交互由网络连接和行动选择算法来自动地实现。

扩展行为网络有着更强的健壮性。在一个包容体结构中，健壮性的根基在于高层的失败不会影响到低层的功能。然而，低层的个别行为失败却可能影响到所有其他的层次。作为一个设计师，我们不得不编写代码并预见到所有的交互活动以防止这样的情况发生。在扩展行为网络中，如果一个模块失败了，结构会自动地使用那些能够使目标得以实现的任何其他的模块。

其他的一些优点是环境相关的目标，使用实值的命题，结构如何处理目标的交互。环境相关的目标允许更容易地把环境室间设计与更自然的代理设计区分开来。我们能够明确说明一个目标并非总是重要的，而是仅在某些给定的情况下才变得重要。比如，如图 3.3.4 所示，我们说明了代理只有在他的健康值不高的情况下，才应该考虑获得一个高的健康值（目标“获得高健康值”，HaveHighHealth）。同样的情况也适用于弹药（目标“非低弹药”，NotLowArmmo），

Download at www.Pin5i.Com

代理在弹药充足的情况下不需要关心他的弹药储备。

我们的目标和行为之间的框架不是脆弱的，它具有更强的适应性。哪个模块执行是积累的激活性和可执行性的一个函数。激活性则是代理的目标、存在的其他模块以及代理所处的近期状况的函数（相关）。模块的层次随着代理的当前行为而改变。实值的命题使得我们可以处理那些易变的概念，比如近、强和健康。

为代理构建一个行为网络，我们首先定义它的目标。接着我们组装那些完成目标的模块。最后我们调整激活传播的参数来完成所期望的全局的行为。网络的定义和组装的细节在文章《使用模糊感知器、有限状态自动机和扩展的行为网络为虚幻竞技场游戏构建一个目标驱动的机器人》（文章 3.4）中论述，参数的设定在文章《一个目标驱动的虚幻竞技场角色程序》中讨论（文章 3.5）。

扩展行为网络的一个有趣的特性是：改变一些用来控制激活传播的全局参数——主要是一个模块从前一步保留了多少激活值，以及符合执行条件的最小执行值——我们就可以戏剧性地全面影响代理的行为。这就使得设计者能很容易地创作几个不同的角色。这方面的情况在文章《一个目标驱动的虚幻竞技场角色程序》（文章 3.5）和[Pinto05-b]中详细地进行了讨论。

在把扩展行为网络应用到虚幻竞技场游戏的过程中，我们做了一些实验来评估行为选择的质量和代理的性能。对于后者，我们比较了一个用扩展行为网络建立的代理和一个完全用层次反应结构建立的代理。扩展行为网络代理有着更好的性能，因为它有更好的行为选择机制。这样的行为选择机制显示了健壮性、反应性、行为的连续性以及并发行为的合理组合性。这些在文章《一个目标驱动的虚幻竞技场角色程序》（文章 3.5）中进行了详细说明。

3.3.3　讨论

因为我们已经指出了被包容体结构和扩展行为网络的优点和他们各自的一些特殊缺陷，现在我们将要讨论基于行为的系统的一些共同的局限性：缺乏变量。

缺乏变量使得模块的通用性很低；对于每一种新的武器类型，我们都需要创建一个全新的模块。然而，正是这样的一个局限性才使得它们能够使用更少的资源并有更高的速度。通用的推断结论和变量使用的能力是以更低的速度和更大的处理要求为代价的。

幸运的是，我们已经作了一些尝试，融入了变量使用而同时又保持基于行为的系统的有趣的属性。[Horswill99]提出了一个结构，允许有限地使用变量。我们对这种结构在游戏中的应用不太了解，尽管它在未来的虚幻竞技场游戏中的应用曾被提及。同样地，[Rhodes96]提出了一个行为网络模型允许以一种有限的方式使用变量。把这样的模型与扩展行为网络模型相结合将会产生一个更具灵活性的模型。

3.3.4　结论

我们已经看到包容体结构和扩展行为网络在机器人领域和动作游戏领域都起到了积极的作用。这些特性就是健壮性、反应性、高速度、和模块化。这二种方法都可以实现渐进设计，模块化开发和并行处理。

在包容体结构中，我们必须指定在模块和层次之间的每一个连接；然而在行为网络中，结构形式本身就能处理行为的交互。

Download at www.Pin5i.Com

在包容体结构中，行为的层次是固定的，然而在扩展行为网络中它是具有适应性的。尽管扩展行为网络是快速的，特别是当他们与计划系统比较的时候，然而这样的适应性的代价是需要为行为选择花更多的时间。

扩展的行为网络的一个显著的特点是：允许通过简单地调节常量，把一些简单的个性特点创建到代理中去。基于行为的系统使用有限的资源，这使得他们在实时策略和动作领域占有举足轻重的地位。

3.3.5 参考文献

[Brooks86] Brooks, R., "A Robust Layered Control System for a Mobile Robot." *IEEE Journal of Robotics and Automation.* Vol. RA-2, No. 1., 1986.

[Champandard03] Champandard, A., *AI Game Development*. New Riders Publishing, 2003.

[Dorer99] Dorer, K., "Extended Behavior Networks for the Magma Freiburg Team." RoboCup-99 Team Descriptions for the Simulation League. Linkoping University Press: pp. 79–83, 1999.

[Dorer00] Dorer, K., "Concurrent Behavior Selection in Extended Behavior Networks." Team Description for RoboCup2000, Melbourne, 2000.

[Dorer04] Dorer, K., "Extended Behavior Networks for Behavior Selection in Dynamic and Continuous Domains." *Proceedings of the ECAI Workshop on Agents in Dynamic and Real-time Environments,* 22/23, August 2004, Valencia, Spain.

[Goetz97] Goetz, P., "Attractors in Recurrent Behavior Networks." Ph.D. Thesis, University of New York, Buffalo, 1997.

[Horswill99] Horswill, I. D. and R. Zubek, "Robot architectures for believable game agents." AAAI Spring Symposium on Artificial Intelligence and Computer Games, AAAI Technical Report SS-99-02, 1999.

[Maes89] Maes, P., "How to do The Right Thing." *Connection Science Journal,* Vol. 1, No. 3., 1989.

[Nebel03] Nebel, B. and Y. Babovich, "Goal-Converging Behavior Networks and Self-Solving Planning Domains, or: How to Become a Successful Soccer Player." s.l. *IJCAI03,* 2003.

[Pinto05-a] Pinto, Hugo, "Designing Autonomous Agents for Computer Games with Extended Behavior Networks: An Investigation of Agent Performance, Character Modeling and Action Selection in Unreal Tournament." Masters of science thesis, Universidade Federal do Rio Grande do Sul, Brazil, 2005.

[Pinto05-b] Pinto, H. and L. O. Alvares, "Extended Behavior Networks and Agent Personality: Investigating the Design of Character Stereotypes in the Game Unreal Tournament." *Proceedings of the Fifth International Working Conference on Intelligent Virtual Agents,* Kos-, Greece, 2005.

[Rhodes96] Rhodes, Bradley, "PHISH-Nets: Planning Heuristically in Situated Hybrid Networks." Masters of science thesis, Massachusetts Institute of Technology, Boston, 1996.

[Yiskis03] Yiskis, E., "A Subsumption Architecture For Character-Based Games." *AI Game Programming Wisdom II,* Charles River Media, 2003.

Download at www.Pin5i.Com

3.4 使用模糊感知器、有限状态自动机和扩展的行为网络为虚幻竞技场游戏构建一个目标驱动的机器人

Hugo Pinto、Luis Otavio Alvares
hugo@hugopinto.net

当为一个游戏设计代理时，常需要考虑至少三个问题：它如何决定每一步做什么，它怎样感知它周围的环境，以及如何设计它的基本行为。

这些因素相互影响和限制：根据它拥有的知识和对环境的感知，一个代理能够做出决策，并且代理的行为必须要服从于决策机制的指导，这样才能很好地协调。同样，代理的感知不仅影响什么行为将被执行，而且也影响行为如何去执行；比如，“攻击最弱的敌人”将受到“谁是最弱的敌人”的感知的影响。

扩展的行为网络[Dorer04]是一类行为选择框架，它们被设计成要为一个在动态的复杂的环境中的代理选择一组十分恰当的行为以实现多个目标。它们使用实值的条件和模块化的行为，特别适宜于和模糊感受器相集成。它们被成功地应用在机器人世界杯[Dorer99]和虚幻竞技场游戏[Pinto05]中。

模糊感知器在很多领域中都被成功地应用，使用它我们可以有连续的属性和任意的范畴，并且有令人满意的结果。对于游戏来说，它们特别适合于感知那些像“敌人在附近”和“敌人强壮”之类的概念，因为附近和强壮是使用连续的度量才能更好地领悟的概念。

在研究领域和游戏开发领域，有限状态自动机都是广为人知的、非常成熟的技术，并且已经被广泛地应用到游戏中。它是一种非常好的模型化方法，可以用来对基本的行为能力和行为模块进行设计，比如“回家”和“射击敌人”。

在这篇文章中，描述了我们如何设计和集成了模糊感知器、有限状态自动机和扩展的行为网络，来为虚幻竞技场游戏创建一个机器人。

3.4.1 扩展的行为网络设计

扩展的行为网络（EBNs）结合了传统的计划系统（比如：基于先决条件和效果的行为之间的连接）和连接系统（比如激活传播）的特点。它们通过一个静态的结构和一个行动选择算法来指定。

扩展的行为网络的结构由一定数量预先定义的集合构成。这些集合是

Download at www.Pin5i.Com

一组行为模块、一组目标、一组连接目标和其他模块的连接、一组资源和一组控制参数。一个行为模块类似于一个 STRIPS 操作符[Nilsson71]（译者注：列出了输入和输出），它有一个模块执行之后所获得的期望效果的列表，和一个模块变为活动状态所需要条件的列表。连接是根据模块的效果、条件和目标建立起来的。

通过激活传播，网络节点相互激励和抑制，行动选择正是基于这种机制。在每一步，被选择执行的模块是那些具有最高执行值的模块，并且它们都具有执行需要的所有资源。一个模块的执行值是它们的激活性和可执行性的乘积（比如用模糊逻辑的“与”操作它的各个条件）。行为网络结构和行为选择算法的细节在文章《一个目标驱动的虚幻竞技场角色程序》（文章 3.5）讨论。

扩展的行为网络被作为一种控制机制而设计，这种机制用于控制在环境中的代理。从工程师的角度看，这意味着被代理的认知系统用到系统的抽象，比如实体的命题和表述，需要根据代理执行的任务、代理所处的环境和它的目标来建立。

代理的环境不仅由游戏的特点和游戏的规则来定义，也通过在代理和游戏环境之间发送的、由代理接收的初始的感知消息来描述。这样，当设计感知器、目标和行为模块时，我们需要考虑游戏角色程序包的消息和命令。

图 3.4.1 所示为在代理、游戏角色和虚拟竞技场游戏服务器之间的通信关系。

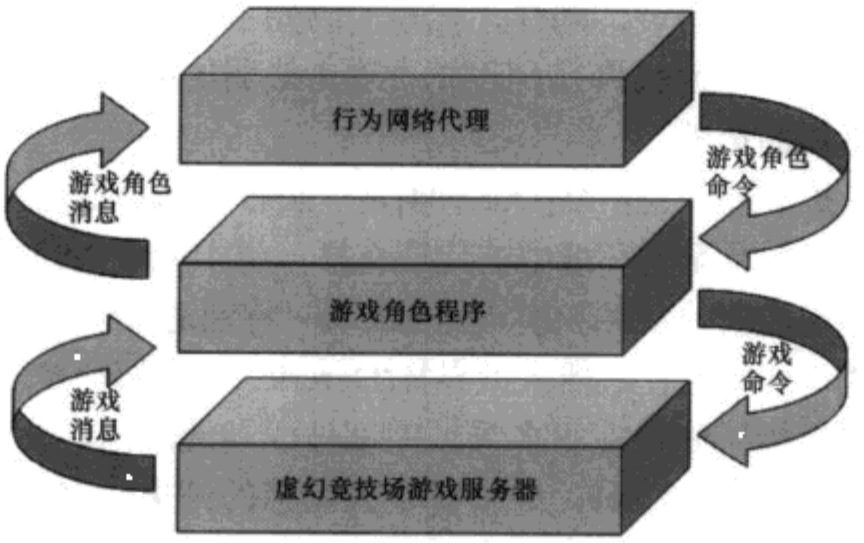


图 3.4.1 集成行为网络代理到游戏角色包和虚幻竞技场游戏服务器

虚幻竞技场服务器发送游戏消息到游戏角色程序。接着游戏角色程序发送它自己的消息到我们的代理。代理感知器处理这些消息，并更新行为网络中条件的值和代理的内部状态。此时进行行为选择，并且通过向游戏角色程序发送命令来执行相应的行为，游戏角色程序接着发送低层的虚幻意技场命令到服务器。

那么，让我们看一看如何为虚幻竞技场代理设计一个简单的行为网络，这个网络用来在地图 DM-Stalwart 中进行死亡竞赛。我们将一步一步地展现我们设计这个网络的过程，以说明在每一个选择背后的逻辑依据。

在死亡竞赛游戏中，一个代理主要关心的是如何杀死尽可能多的敌人并避免自己死亡。它必须能够发现、追踪和射击敌人，也应该能够避免被射中并能补充自己的健康值。

我们从定义代理的目标开始。按照游戏的要求，我们知道它必须杀死敌人并且维持自己的生命。为了杀死敌人，代理就必须伤害它的敌人直到敌人死亡；为了维持生命，代理就必须保持自己的健康值，并且在健康值快耗尽的情况下恢复健康值。为了完成杀死敌人的需要，我们给代理一个“伤害敌人”（EnemyHurt）的目标。为了它自己的生存，我们添加了一个“避

Download at www.Pin5i.Com

免被射中”（Not IAmBeingShot）的目标和一个“保持高健康值”（HaveHighHealth）的目标。

现在我们必须设计完成这些目标的行为。为了伤害一个敌人，我们的代理必须攻击他。一个明智的方法是有一个攻击敌人（AttackEnemy）的行为，它的先决条件是“敌人在视线内”（EnemyInSight），它的效果是“伤害敌人”（EnemyHurt）。这样一个通用行为设计的问题是，如果我们使用射击武器去发起攻击的话，我们就不得不指定一个附加的效果来说明弹药减少的情况。如果我们攻击采用的是一种接触性武器（如攻击锤或链锯），我们就需要接近敌人，并且行为的执行不会影响弹药的储备量。因此我们创建两个攻击行为而不是一个：“用锤子攻击”（FinalizeWithHammer）和“射击敌人”（ShootEnemy）。图 3.4.2 显示了这样的行为模块与目标“伤害敌人”（EnemyHurt）之间的连接。注意，他们使用了完全相同的资源，这就使得它们相互之间是互斥关系，（除非我们用这样的一个网络来控制一个四只手、两个头的代理）。

现在我们需要两个模块以使“敌人在附近”（EnemyNear）和“敌人在视线内”（EnemyInSight）成为事实。因为代理仅有的感知能力是视觉，因此它告诉我们，如果一个敌人在附近的话，这个敌人首先必须在视线内。我们有三种方法来使“敌人在视线内”（EnemyInSight）模块成为现实：站着不动（Stand），并等待一个敌人在我们面前出现；原地不动并观察周围是否有敌人（StandLookout）；或者激发一个关卡搜索（Explore）行为来找到敌人。这三个可能的行为分别用“站着”（Stand）、“站着观察”（StandLookout）、“搜索”（Explore）三个模块来表示。站着不动，不需要先决条件；不过我们希望只在视线里没有敌人的时候进行原地观察或者搜索。如果我们被射击，我们就必须停止搜索并查明是什么东西在向我们射击，所以我们添加一个先决条件“没有被射中”（Not IAmBeingShot）到“搜索”（Explore）行为模块。在左面的目标完成路径中有一个模块将“敌人在附近”（EnemyNear）作为执行效果。一个细节处理模块“靠近敌人”（GoToEnemy）有先决条件“敌人在视线内”（EnemyInSight）和效果“敌人在附近”（EnemyNear）、“被射中”（IAmBeingShot）。我们添加“被射中”（IAmBeingShot）作为效果是因为靠近敌人就容易被敌人射中。图 3.4.3 显示了目前创建的网络，只有一个目标“伤害敌人”（EnemyHurt）被包含其中。

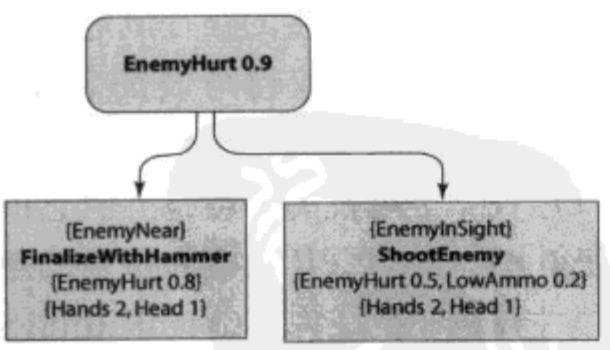


图 3.4.2　目标“伤害敌人”和它的支持模块

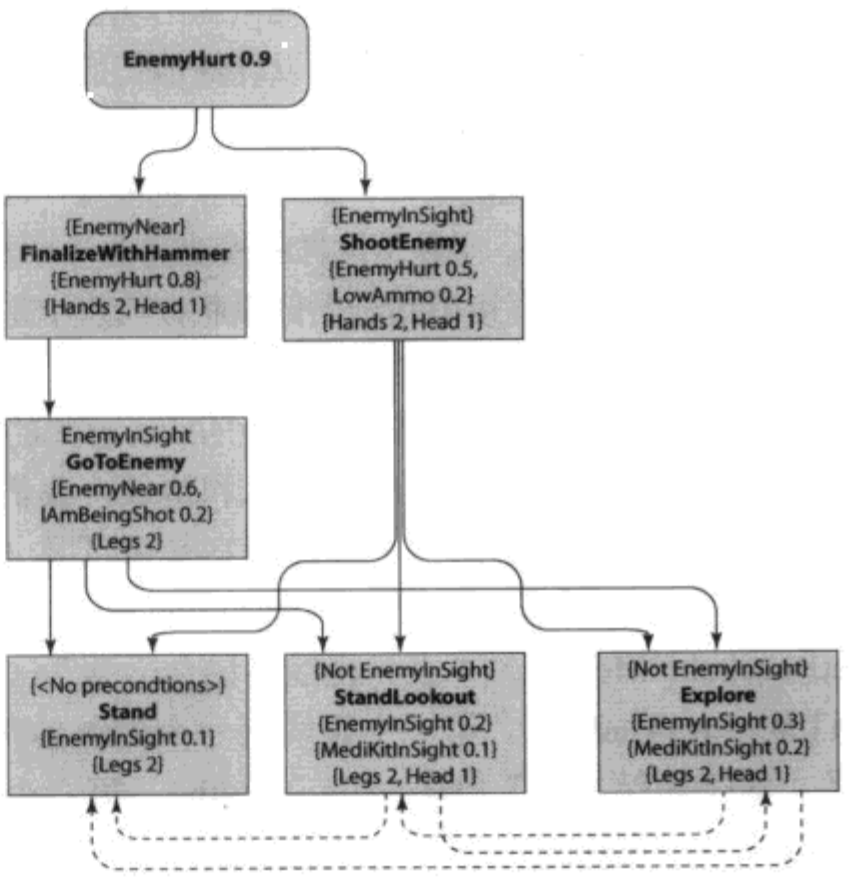


图 3.4.3　目标“伤害敌人”（EnemyHurt）行为网络

Download at www.Pin5i.Com

现在让我们设计模块来支持目标“不被击中”（Not IAmBeingShot）。避免被击中最基本的办法，除了完全不进行战斗之外，就是“躲避子弹”。我们创建一个行为“躲避”（Dodge），它具有“被击中”（IAmBeingShot）的先决条件和“不被击中”（Not IAmBeingShot）的效果。代理只需要用他的腿来躲避就可以了。这样的网络如图 3.4.4 所示。

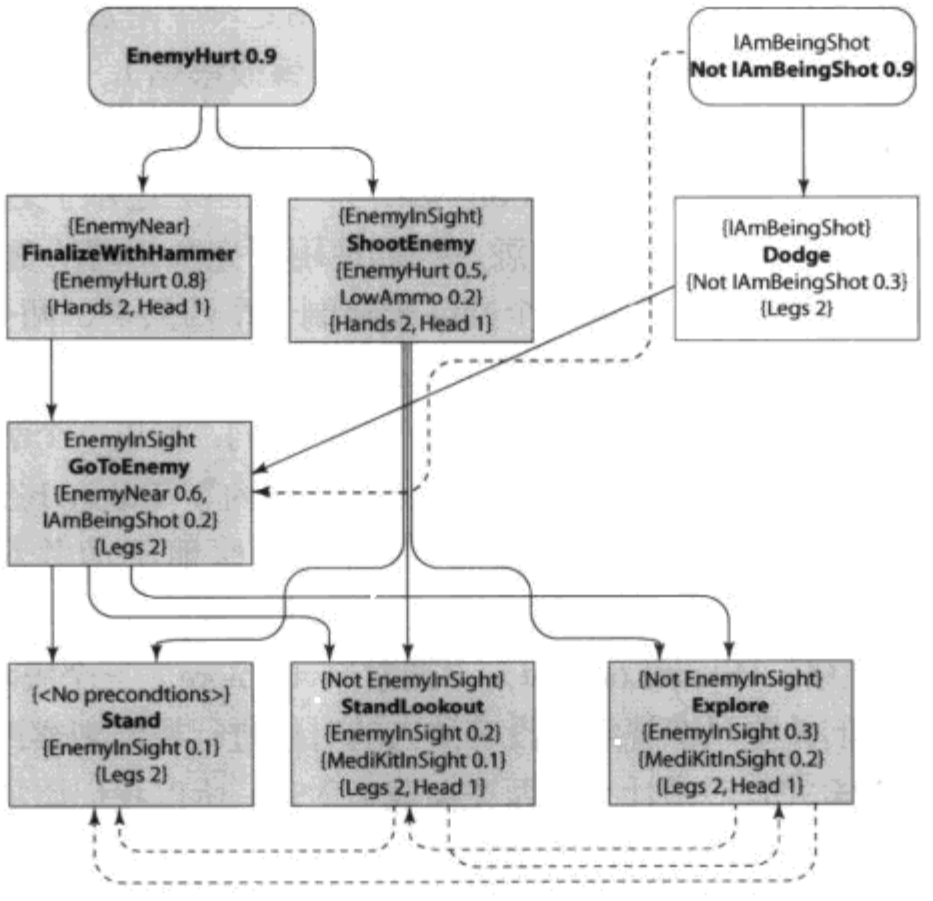


图 3.4.4 目标“伤害敌人”（EnemyHurt）和“不被击中”（Not IAmBeingShot）行为网络

现在让我们来考虑如何达到目标“保持高健康值”（HaveHighHealth）。健康瓶和急救包在虚幻竞技场游戏中有固定的位置。按照我们在测试中所使用的游戏选项，这些东西如果被取走，在消失了一段时间之后，他们会魔术般地重新出现在同样的地点。因此，在一个关卡里，这些健康物品总是有固定的位置，这使我们可以存储他们的位置，并且在我们需要的任何时候，只要到我们知道的位置去取这些健康物品就可以了。我们第一个解决方案是“取得已知的急救包”（Get Known Medical Kit），它具有先决条件“知道急救包位置”（Know Medical Location）和效果“保持高健康值”（HaveHighHealth）。问题是这样的模块需要一个非常高的抽象层次：在几百步之外得到一个已知的急救包和在附近直接得到一个健康瓶在模块中是同样处理的。而且，这样的模块必须包含一些检测，来查看是否有一条到达急救包的路径（看急救包是否可及），并且还要决定哪一个已知的急救包将被取得。能不能通过考察环境的或者感知的信息来把这个模块设计得更简单呢？

通过检查本游戏角色程序的消息和协议，我们看到机器人收到一些原始的关于在视线内物品的消息：它的级别，它的位置和它是否可及（比如：是否有一条直接的路径到达物品）。如果我们考察了这样的信息，我们就能够制作三个不同的模块：“取得可及的急救包”（GetReachableMedKit）、“去到视线内的急救包”（GoToMedKitInSight）、“去到已知的急救包”（GoToKnownMedKit）。第一个模块有一个先决条件，那就是有一个可及的急救包并且它的效果是“保持高健康值”（HaveHighHealth）。第二个模块使得视线内的急救包成为可及的急救包。最后一个模块通过去到已知的急救包的位置，使得“急救包在视线内”（MedKitInSight）为真。我们知道“站着观察”（StandLookout）、“搜索”（Explore）两个模块都可能满足“去

Download at www.Pin5i.Com

到已知的急救包”（GoToKnownMedKit）的先决条件，因为这两个模块都可能发现一个已知的急救包位置。图 3.4.5 显示了这些模块在网络中的组合方式。

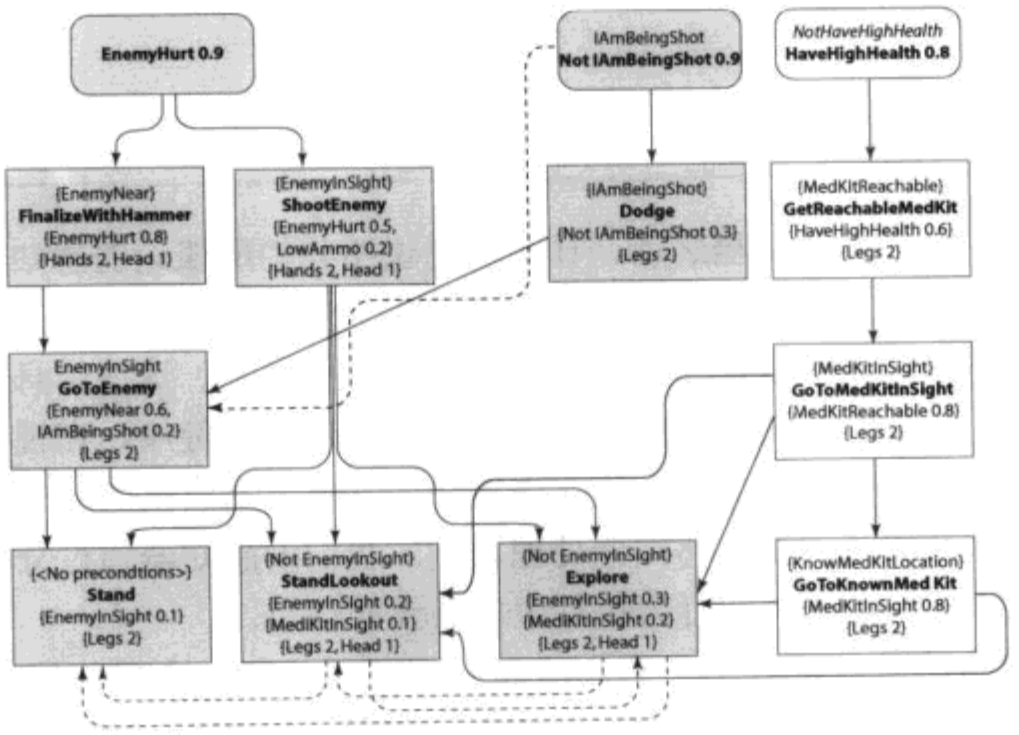


图 3.4.5　行为网络和健康相关模块（白色背景）的添加

我们的网络看起来已经就序了，但是我们还没有提到在虚幻竞技场游戏中用来做一些特殊任务的附加模块。每当我们告诉游戏向一个特定的代理或位置射击的时候（命令 SHOOT），它就会不断地射击，直到我们告诉它停止射击（命令 STOPSHOOT）。在一个模块内部我们可以很好地处理这一点，但是基于不浪费弹药的既定目标的事实，我们需要停止射击，我们为它创建了一个目标，“非低弹药”（Not LowAmmo）。停止射击应该是一个独立的有完全行为能力的模块，“停止射击”（StopShoot），它有先决条件“Not EnemyInSight”（视线内无敌人）和效果“非低弹药”（Not LowAmmo）。图 3.4.6 显示了我们完成的网络。我们添加了一个环境条件“LowAmmo”（低弹药）到目标非低弹药中，因为我们想要让代理在其弹药减少时“担心”它可能军火值过低。

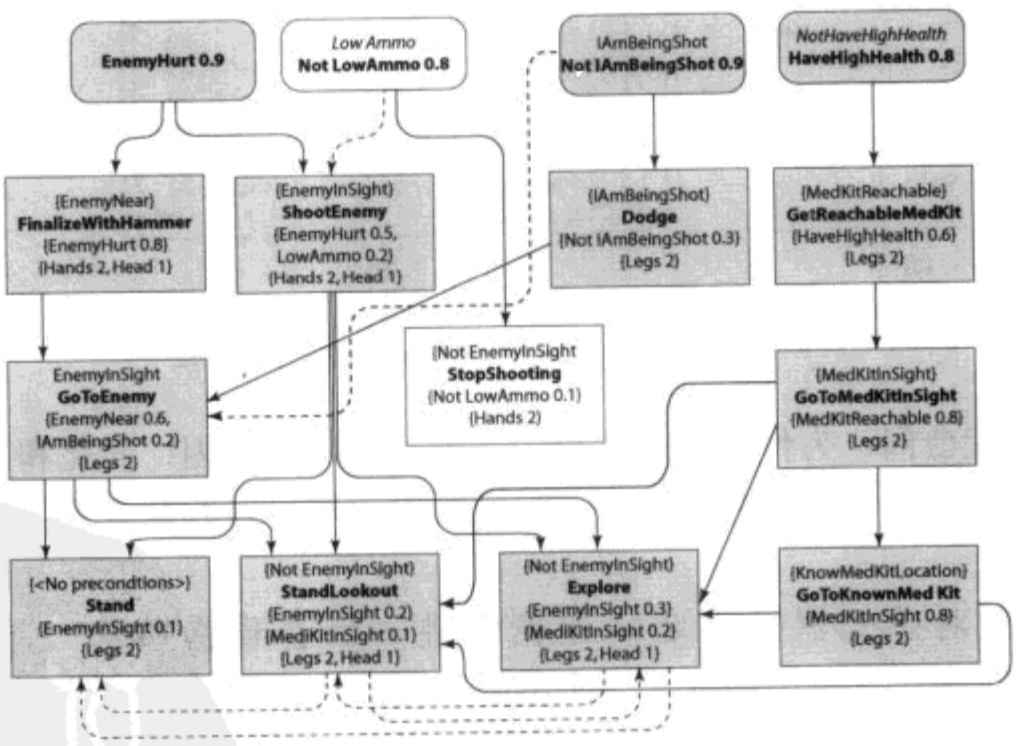


图 3.4.6　完成的行为网络

Download at www.Pin5i.Com

现在网络组建完成了，但也给我们提出三个问题：如何验证网络的命题？每一个的行为的动作是如何执行的？所有的功能是如何一起工作的？在下一个部分，我们探讨第一个问题，并给出一个代理感知机制的概览。

3.4.2 层次模糊感知器

在开始讨论我们的网络感知器之前，让我们先说明我们所谈论的感知器是一种什么样的感知器。在这篇文章的一开始，我们指出了代理的感觉受到它的内部状态、它的认知和它当前活动的影响。初始的感觉由游戏角色包通过消息来进行定义。这些信息包括：机器人的位置，机器人的速度，机器人的旋转，机器人的弹药，机器人的健康，在视线内的敌人，和看到的东西。行为网络工作在命题之上，这些命题的属性值由高层次的感知信息组成。这些高层次的感知信息由这个部分讨论的感知器处理得到。我们的感知器通过分析游戏角色程序的消息和代理的知识来为行动选择和行为执行生成必要的信息。

每一个行为网络的条件都有一个感知器和它相联系。每一个感知器都有一个归属函数（membership function）P 和一个内部状态函数（internal state function）I。函数 P 根据当前感知到的代理的状态来返回一个对应于条件的模糊命题，取值的范围是[0，1]。函数 I 更新代理的内部状态。注意，这个功能暗含感知器的活动顺序，在每一次活动中都可能会改变内部的状态。图 3.4.7 和图 3.4.8 说明了一个高层次的感知器“感知视线中的敌人”（SensorEnemyInSight）的操作。

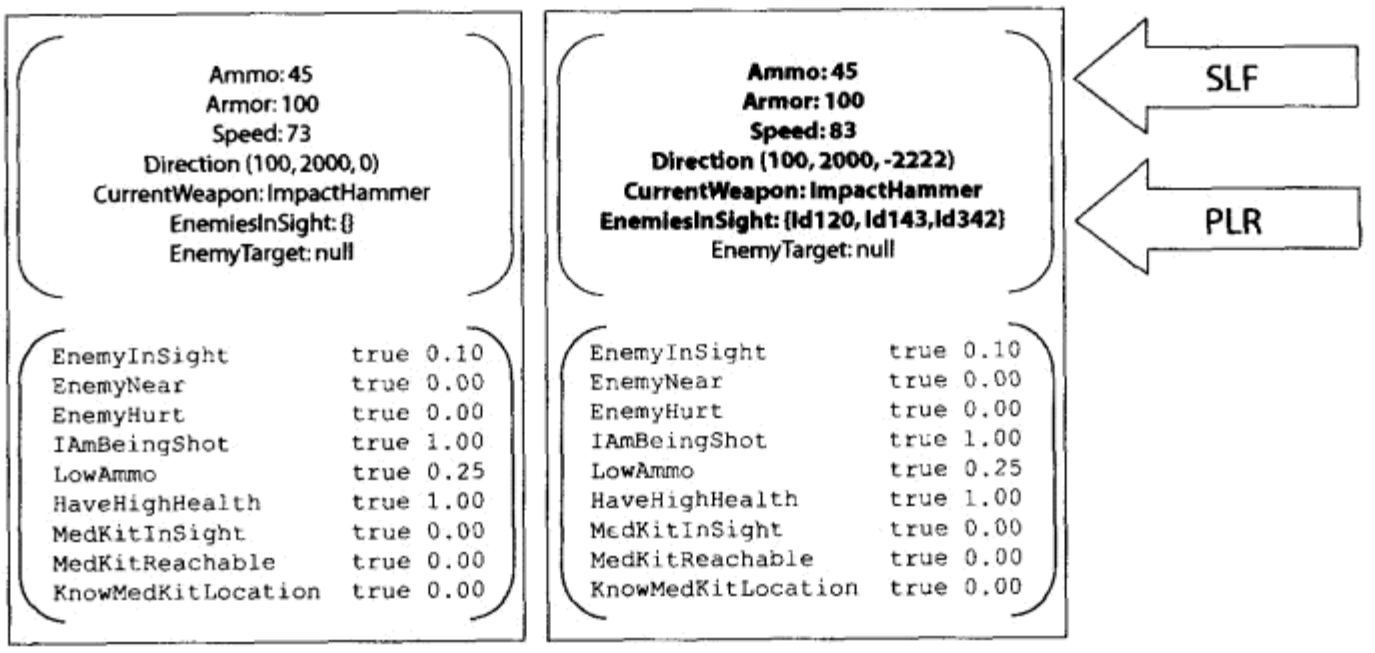


图 3.4.7 （a）在收到游戏角色程序消息之前代理的内部状态和命题 （b）在收到游戏角色程序消息 SFL 和 PLR 后的内部状态和命题，黑体部分是被改变的字段

细心的读者可能会问，为什么只有真值的命题呢？两个原因：简化和节省存储。操作符求命题 p 的否命题的时候使用公式：$N(p)=1-p$，所以我们可以非常方便地在命题和它的否命题之间进行转换。

为了完成我们对感知器的说明，让我们进一步看看感知器“感知近距离敌人”（SensorEnemyNear）的操作，因为这是一个很好的说明感知器操作顺序的重要性的例子。如图 3.4.9 所示，感知器“感知视线中的敌人”（SensorEnemyInSight）设置目标，接着感知器“感知近距离敌人”（SensorEnemyNear）得到“目标敌人”（EnemyTarget）、机器人自己的位置和

Download at www.Pin5i.Com

目标位置，然后，设置命题“敌人在附近”（EnemyNear）的实际值。这个感知器改变了命题，但是不改变内部状态。注意，感知器“感知视线中的敌人”（SensorEnemyInSight）首先操作是非常重要的，因为对远近的评估需要取决于这个感知器对目标的设置。图 3.4.10 显示了 P 操作如何设置命题“敌人在附近”（EnemyNear）实际的值。

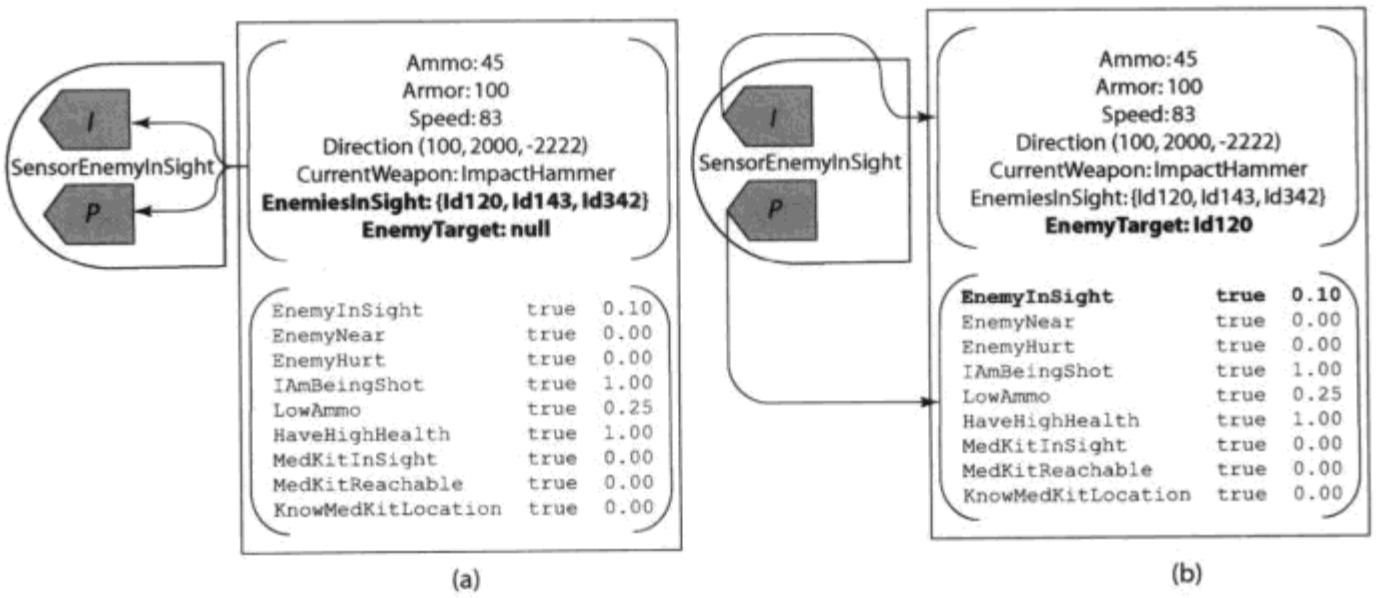


图 3.4.8　（a）感知器“感知视线中的敌人”（SensorEnemyInSight）读取代理的内部状态信息，函数 I 和 P 收到同样的输入。黑体部分是感知器实际用到的信息　（b）感知器“感知视线中的敌人”（SensorEnemyInSight）更新内部状态和行为网络的命题。函数 I 设置了“目标敌人”（EnemyTarget）为 Id120，并且 P 设置了命题“敌人在视线中”。设置的部分用黑体表示

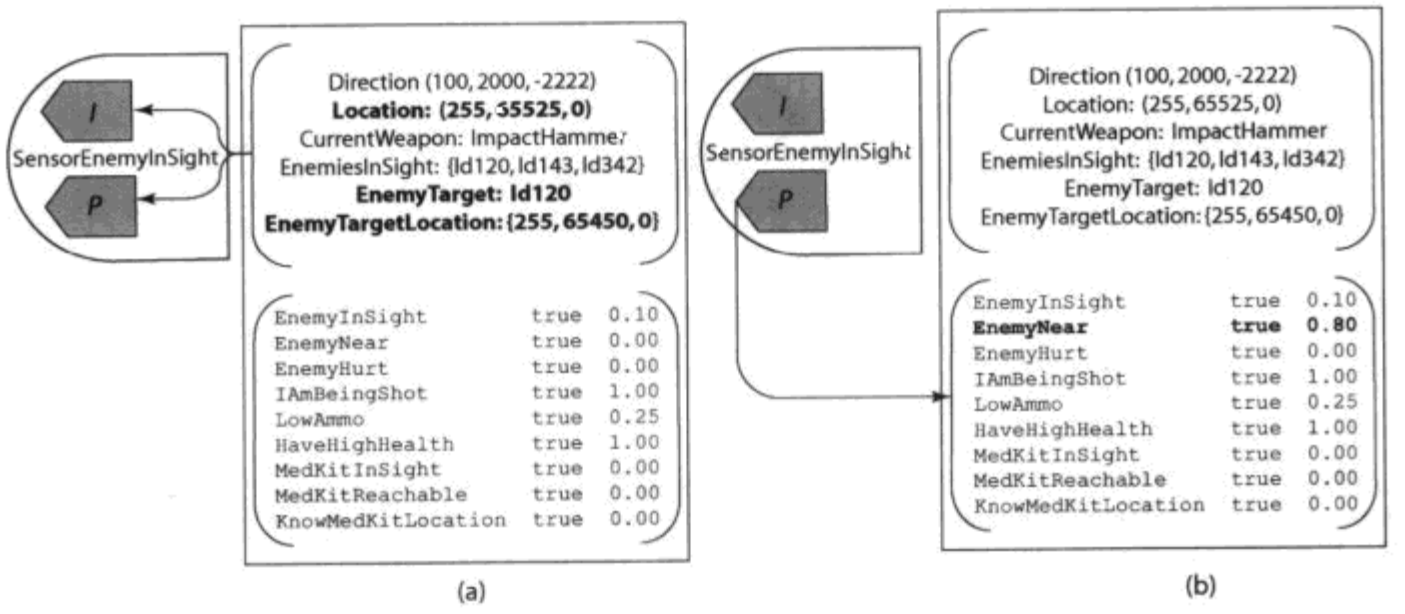


图 3.4.9　（a）感知器“感知近距离敌人”（SensorEnemyNear）读取内部状态数据　（b）感知器“感知近距离敌人”（SensorEnemyNear）更新命题“敌人在附近”（EnemyNear）的实际值

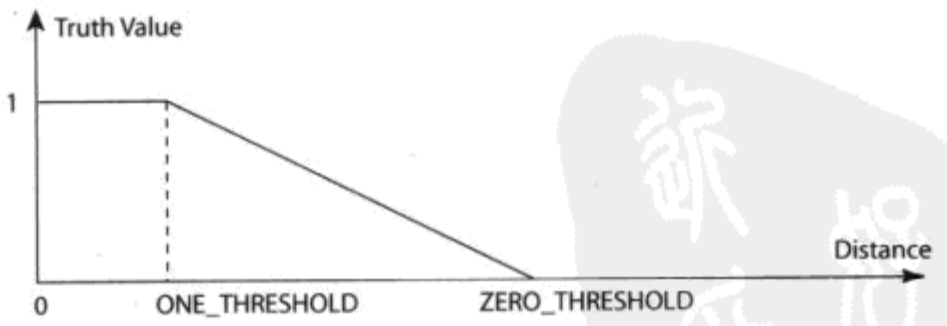


图 3.4.10　“敌人在附近”（EnemyNear）的真值公式。ONE_THRESHOLD（1 阈值）表示当距离小于某一个值的时候，敌人真的很近。ZERO_THRESHOLD（0 阈值）表示当距离大于某一个值的时候敌人真的很远

Download at www.Pin5i.com

在探讨了代理的感知和网络的命题如何得到值之后，我们来考察行为模块。

3.4.3 有限状态自动机行为模块

行为模块的动作用增强的有限状态自动机（FSMs）来建立。当被选择执行时，每一个行为动作都有一个确定的执行时长。每一个行为模块都负责监视自己动作的执行，并且在需要的时候开始、继续或者中止每一个状态。

一个行为动作在执行时可能发送一个或者更多的初级命令给游戏。所有的行为都可以访问代理的内部状态。图 3.4.11 所示为“靠近敌人”（GoToEnemy）行为的序列图。

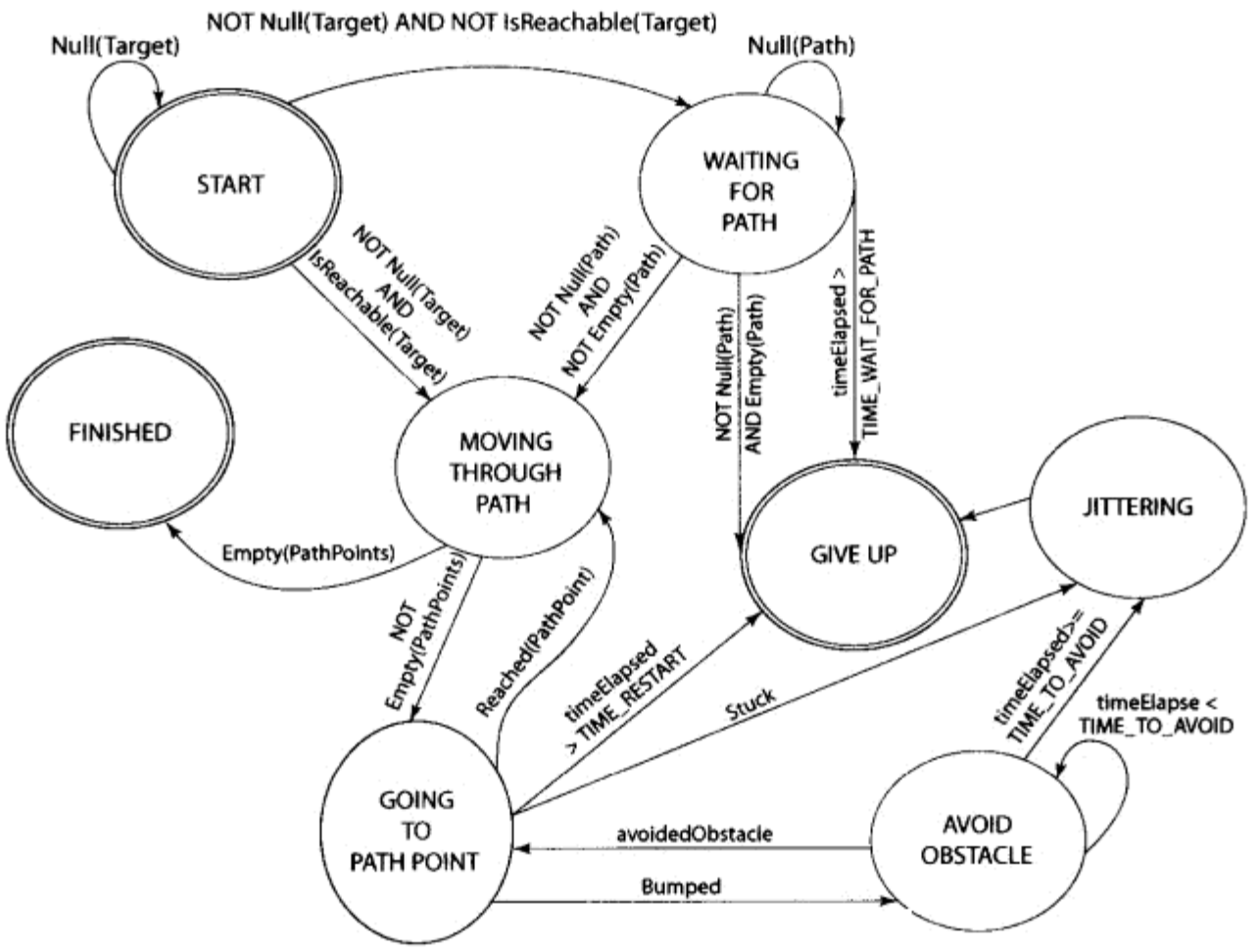


图 3.4.11 “靠近敌人”（GoToEnemy）行为的序列图。Null(X)、IsReachable(X)、Reached(X)和 Empty(X)是谓词。Stuck 和 Bumped 是布尔标志。Target、TimeElapsed、PathPoints 和 PathPoint 是变量。TIME_TO_AVOID、TIME_WAIT_FOR_PATH 和 TIME_RESTART 行为用来自我监视的常量。状态 START、FINISHED 和 GIVE UP 是端状态。FINISHED 是表示成功达到目标；GIVE UP 表示中止执行

我们看到行为是非常复杂的，尽管高层的动作是简单的。代理必须找到一条到达敌人的路径，处理在路途中可能遇到的障碍，并且当断定目标敌人不可及时，就要放弃。

让我们按照图中显示的一些路径来说明行为如何与感知和代理的内部状态相联系；行为动作如何与游戏角色程序包的初级动作相关联。

让我们从 Start 状态开始。可变的目标是感知器“感知视线中的敌人”（SensorEnemyInSight）设置的，这在前面的部分已经描述。如果没有目标被设置（Null(Target)），代理就不做任何事情。如果目标不是直接可及的，模块就请求一个到达它的路径，并且进入等待路径 WAIT FOR PATH 状态。在收到到达目标的路径之后，代理开始移动（MOVING THROUGH PATH）。

在状态 WAIT FOR PATH 中，模块发送一个 GetPath 命令到游戏并等待对应的路径消息 PTH。如果等待的时间（time Elapsed）超过了最长的允许时间（TIME_WAIT_FOR_PATH），

Download at www.Pin5i.com

代理就放弃靠近目标（状态 GIVE UP）。

但是，这些路径到底如何表示呢？在虚幻竞技场游戏中，关卡被图形所覆盖，图形扩展到各关卡的大部分区域。图形的每一个节点都是一个可能到达的目标位置，叫做导航点（NavPoint：navigation point）。一个导航点可以放置一个物品、一个武器或者仅仅是代理可能移动到的一个位置。我们可以给出代理移动的三维坐标，来让代理进行移动，但是，我们通常是只提供一个导航点的序列号（NavPoint ID）。在存在一条到达目标的路径的情况下，对 GetPath 命令的响应就是提供一个导航点（NavPoint）的列表，这个列表导向目标；当不存在路径的时候，响应为空（Null）。

接着从图中的 MOVING THROUGH PATH 状态往下。当代理走完了所有的导航点之后，它就到达了状态 FINISH，并且成功地结束了这个行为。如果还有导航点要走，它就进入状态 GOING TO PATH POINT。如果它在走向某一个导航点的时候遇到一个障碍，它就进入状态 AVOID OBSTACLE。如果在过了一段时间之后代理还没有避开障碍物，它就进入状态 JITTERING。在这个状态中，代理尝试一些随机的动作，以使它自己离开所呆的地方。这是一种在代理被卡住的情况下所使用的一种原始的试探式的方法。一种更好的实现方法是制作一个关卡的内部地图，并且为可能被卡住的情况进行一些推理（比如：拦住代理通过起来太窄的地方、洞、能够跳过的障碍，等等）。在随机移动（Jittering）之后，代理总是放弃（GIVE UP 状态）。我们之所以这样处理动作的过程，是因为我们考虑到每次在代理不能够避开一个障碍时，都可能离我们开始向目标移动的那一刻起已有很长时间了，此时，最好是重新从 START 状态开始。

细心的读者可能会疑惑：行为到底是如何执行的呢？它是不是从一个状态转换到另一个状态直到抵达终端节点？在中间状态时能够中止么？

这个方法把代理的内部状态作为参数来接受。通过检查代理的内部状态，包括检查它的命题，每一个模块决定下一步该做什么。因此，我们看到，代理从来不中断一个行为；只有模块自己可能那样做。

对所有的行为进行中断操作通常都是在行为处于非终止状态，但从它启动执行开始已经过了很长一段时间的情况下。此时，行为重置并重新从状态 START 开始。每一次对执行方法的调用最多产生一次状态转换。通过这样的系统构建方法就使得行为的执行和进度安排有了更大的灵活性。

3.4.4 结论

这篇文章介绍了如何通过集成模糊感知器、行为网络和有限状态自动机来创建一个虚幻竞技场游戏代理。利用行为网络可以将代理的目标和行为分开，并形成相互独立的模块，并且它们都有简单、灵活和稳定可靠的结构。人们能够一次处理一个目标，而由网络的结构来处理它们的交互。可以发现模糊感知器是一种计算网络命题的简单易行的方法。有限状态自动机提供了一个网络行为模块的有趣的解决方案。

在前文所述中，行为网络选择在每一时间片哪一个模块将被执行；一旦被选择执行，一个行为模块就需要监视和控制它自己的执行流。这是一个明智的分开高层决策和低层执行的方法。

本文没有提及设计的一个重要方面的内容：设置网络的全局参数和目标强度值的属性。这些问题在文章《一个目标驱动的虚幻竞技场游戏角色程序》（文章 3.5）中详细讨论。

Download at www.Pin5i.com

3.4.5 参考文献

[Dorer99] Dorer, K., "Extended Behavior Networks for the Magma Freiburg Team." RoboCup-99 Team Descriptions for the Simulation League, Linkoping University Press, 1999: pp. 79–83.

[Dorer04] Dorer, K., "Extended Behavior Networks for Behavior Selection in Dynamic and Continuous Domains." *Proceedings of the ECAI workshop Agents in dynamic domains,* U. Visser, et al. (Hrsg.) Valencia, Spain, 2004.

[Kaminka02] Kaminka, G., et al., "GameBots: a flexible test bed for multiagent team research." Communications of the ACM, Vol. 45, Issue 1: pp. 43–45, 2002.

[Nilsson71] Nilsson, N. J., "STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving." *Artificial Intelligence,* 1971: pp. 189–208.

[Pinto05] Pinto, H. and L. O. Alvares, "An Extended Behavior Network for a Game Agent: Investigating Action Selection Quality and Agent Performance in Unreal Tournament." To appear in MICAI-2005, *Advances in Artificial Intelligence,* 2005. Alexander Gelbukh, Ed.

Download at www.Pin5i.Com

3.5 一个目标驱动的虚幻竞技场游戏角色程序：使用扩展的行为网络制作目标驱动的具有个性的代理

Hugo Pinto、Luis Otavio Alvares
hugo@hugopinto.net

在设计一个游戏机器人（一个游戏角色程序）时，一个关键的考虑是如何选择它的动作以展现出目标驱动的行为。当机器人有着许多相互冲突的可能的目标时，这样的任务就变得更复杂了。如果机器人也处在一个快速变化的环境，并且每时每刻不得不考虑很多因素时，我们就需要解决一个非常困难的问题了。基于搜索的方法变得不再可行，因为搜索空间很大，并且传统的计划方法也变得更为困难，因为在代理完成计划的时候，环境可能会改变。

行为网络是作为一个在复杂的动态环境中选择充分有效的行为的机制被提出来的。自从行为网络出现之后一直在不断地发展，正如在[Tyrrell93]、[Rhodes96]、[Goetz97]、[Dorer99]和[Nebel03]表明的一样。它们已经被应用到动物模拟[Tyrrell93]、交互的故事讲述[Rholdes96]、机器人世界杯[Muller01]以及虚幻竞技场游戏[Pinto05-a]中。

扩展的行为网络展现了一些现代的行为网络模型。它们使用实值的命题来表达效果和条件，能够详细说明与环境相关的目标，并且能够并行地选择行为。

有效的行为选择是重要的，但是代理如何选择它的行为，并且这些行为如何影响它的个性也是一个计算机游戏关心的问题。如果在建造一个合适的目标驱动行为的代理时，我们也能够考虑到代理个性方面的设计，这难道不是一件很有趣的事情么？行为网络正可以这样做。[Rhodes96]已经应用了行为网络模型 PHISH-NETs 来设计角色的个性，并且我们也已经使用了扩展行为网络来设计了一些典型角色。

第一人称射击游戏，比如虚幻竞技场游戏，构成了这样的一个领域，在这里，扩展行为网络的应用是有趣的。我们有置身于三维延续的虚拟环境中的代理，他们用很多的方法和一些敌人进行交互。一个代理需要面对的环境是复杂多变的。代理有很多可以利用的武器，每一种都有特定的属性，并且有一些物品可用。代理在不同的地形里移动，并且与其他的代理进行交互。动作的范围也是广泛的（比如跑、走、转身、爬行、射击、改变武器、跳、扫射、拿起物品以及使用物品等），并且一个代理还可能同时执行多个动作，比如边跳边射击。不仅如此，代理也有许多可能相互冲突的目标，比如保护本身安全和进行攻击。

对于大多数的游戏模式，简单、刻板的个性已经足以描述我们目前的

Download at www.Pin5i.Com

大都很肤浅的角色。在本文中，我们介绍了扩展行为网络模型以及对它的行为选择质量的试验结果。文中还说明了如何通过调节和修改行为网络来实现不同的角色个性。

3.5.1 扩展的行为网络

一个扩展的行为网络可以被看作一个相互连接的模块和目标的集合，从目标开始接着流向模块，通过激活的传播，模块之前相互抑制或者激发。在每一步，那些具有高激活性和可执行性且不使用相同资源的模块被选择来执行。在下一部分，我们将详细地考察网络的结构和行为选择算法。

1. Structure

一个扩展的行为网络被定义为一组预定义的集合。它们是一个行为模块集合、一个目标集合、一个资源集合和一个控制参数集合。图 3.5.1 显示了在实验中所使用的行为网络的部分规范。图 3.5.2 显示了从这个规范得到的网络，目标用圆角矩形来代表，行为用直角矩形来代表，资源节点用八边形来代表。直线表示继承性连接，虚线表示冲突性的连接，点划线表示资源性连接。

```
Module
name ShootEnemy
precondtion EnemyInSight
action ShootEnemy
effects EnemyHurt 0.6
        LowAmmo 0.1
using Hands 2 Head 1
endModule

Resource
name Legs
amount 2 endResourse


Resource
name Head
amount 1 endResource

Resource
name Hands
amount 2 endResource
```

```
Goal
name EnemyHurt
Context <none>
condition EnemyHurt
strength 1.0
endGoal

Goal
name Not LowAmmo
context LowAmmo
condition Not LowAmmo
strength 0.7
endGoal


Parameters
name ActivationInfluence value 1.0
name InhibitionInfluence value 0.9
name Inertia value 0.5
name GlobalThreshold value 0.6
name ThresholdDecay value 0.1
endParameters
```

图 3.5.1 一个简单行为网络的设置

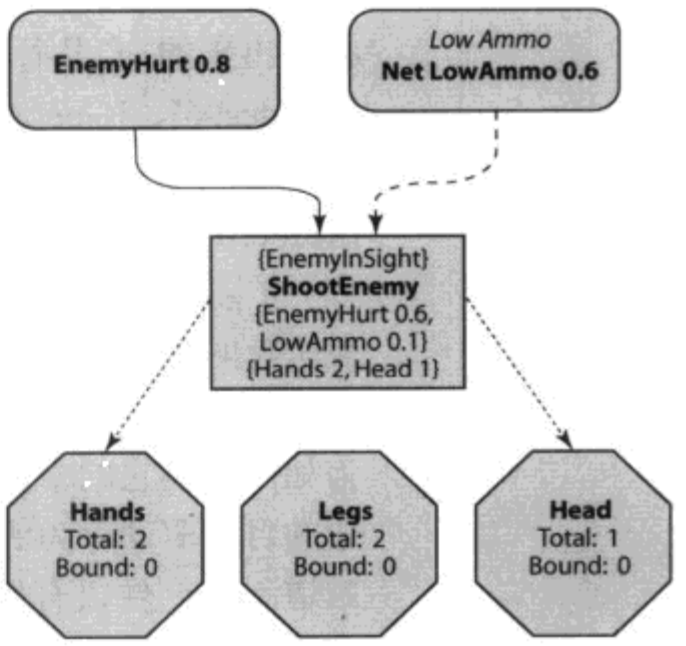


图 3.5.2 一个简单的行为网络图

一个目标 Gi 被定义为一个必须满足的命题、一个强度值和一些相关的析取命题。相关的析取命题用来提供目标的环境，叫做相关条件。强度值提供了静态的、与环境无关的目标重要性度量，而相关条件提供了动态的环境相关的目标重要性度量。

在目标中使用两种条件使我们能够表达当代理处于不同情况的时候，目标重要性的变化。一个与环境无关的目标没有相关条件。在图 3.5.1 所示中，目标“伤害敌人”（EnemyHurt）就是这样的一个例子。注意没有相关条件的目标等价于一个总是相关的目标，也就是说，它的相关性是最大的。

每一个行为模块被定义为一组条件、一个行动、一个效果列表和一组资源。第一个列表是一组实值的命题，用来表达模块执行所需要的条件。效果列表也是一组命题（每一个都可能被否定），用来描述行为发生以后，命题值的变化。资源列表是一个两元组的列表（例如资

Download at www.Pin5i.com

源，数量），用来表示当代理执行行为的时候各种资源需要达到一个什么值。

目标和模块用两种连接来结合。对于从模块或目标 B 到 A 的继承性连接，每一个在 B 的条件列表中的命题在 A 的效果列表中，这样在连接的两端的命题都有相同的正负号——true（+）或者 false（–）。从图 3.5.2 的目标“EnemyHurt”到模块“ShootEnemy”的连接就是一个例子。对于从模块或目标 B 到 A 的冲突性连接，每一个在 B 的条件列表中的命题在 A 的效果列表中，这样在连接的两端的命题都有不同的正负号。图 3.5.2 所示的目标“NotLowAmmo”到模块“ShootEnemy”的连接就是一个例子。冲突性连接从它们的目标中拿走能量，继承性连接给它们的目标输入能量。这样，一个模块或者目标就会试图抑制那些执行之后减弱其条件的模块，并且试图执行那些在执行之后增强其条件的模块。

每一个资源用一个资源节点表示，并通过一个定义好的函数 $f(s)$来定义。函数指定了在每一种情况 s 下所预期的可获得资源的数量。除了函数之外，每一个节点还有一个变量 bound 用来跟踪绑定的资源的数量，以及一个资源激活阈值$\theta_{\text{Res}}\in(0..\theta)$，其中$\theta$是一个全局的激活阈值。在图 3.5.2 所示中，我们看到在所有的情况下预期要使用的每种资源的数量是常数。这并不奇怪，因为代理在各种情况下身体各部分的数量总是相同的（游戏不考虑丢失肢体或者类似的可怕事件）。

通过资源连接，模块被连接到资源节点。对于一个模块的每一种资源类型来说，都有一个连接来把模块和相关的资源节点连接起来。

控制参数被用来微调网络，并且这些参数值的取值范围是[0，1]。“激活感应”参数γ控制继承性连接的激活。“抑制感应”参数δ用来控制冲突性连接的负激活。惯性β，全局阈值θ，和阈值衰减$\Delta\theta$的意义是明显的。在下一个部分，它们的作用将更加清楚。

2. 行为选择算法

用下面的步骤选择每一次循环执行的模块：

1. 计算每一个模块的激活值（activation）a。

2. 根据它的条件列表，使用三角范数（triangular-norm）计算每一个模块的可执行性（executability）e。

3. 执行值 h(a,e)，通过 a 乘以 e 来计算得到。注意这个值组合了执行一个行为的效力（激活 activation）和成功执行它的可能性（executability）。用这种方法，只要有一个高激活值即使那些条件没有被很好地满足的模块也可能被执行。

4. 对于每一个模块用到的资源，从上一次没有获得的资源开始，模块检查它是否超过了资源阈值，并且检查是否有足够的供模块执行所需要的资源。如果是，它就绑定资源。

5. 如果一个模块已经绑定了所有它需要的资源，那它就执行并且把资源阈值重新设置为全局的资源阈值。

6. 每一个模块取消对它所使用资源的绑定。

资源的阈值随着时间的推移而线性地减小，以确保一个行为最终能够绑定它需要的资源，并且最活跃的行为取得优先权。

公式 3.5.1 显示了在时刻 t，通过一个继承性连接，从一个目标 g_i 到一个模块 k 的激活（activation）的传播过程。函数 f 是一个三角范数，它结合了一个目标的强度（strengh）和动态相关性（relevance）。符号 ex_j 是连接目标的效果命题的值。

$$a_{kg_i}^{t}{}' = \gamma \cdot f(l_{g_i}, r_{g_i}^{t}) \cdot ex_j \tag{3.5.1}$$

Download at www.Pin5i.Com

公式 3.5.2 显示了在时刻 t，通过一个冲突性连接，从一个目标 i 到一个模块 k 的激活（activation）的传播过程。

$$a_{kg_i}^{t}{}'' = -\delta \cdot f(l_{g_i}, r_{g_i}^{t}) \cdot ex_j \tag{3.5.2}$$

公式 3.5.3 显示了在时刻 t，通过一个继承性连接，从一个模块 $succ$ 到一个模块 k 的激活（activation）的传播过程。

$$a_{kg_i}^{t}{}''' = r \cdot \sigma(a_{succ\, g_i}^{t-1}) \cdot ex_j \cdot (1 - \tau(p_{succ}, s)) \tag{3.5.3}$$

p_{succ} 是后继模块的命题，a_{succ} 是后继模块的激活（activation）。τ (p_{succ},s)是 p_{succ} 在情况 s 下的值。当继承性连接的始端的命题满足性降低的时候，激活（activation）的传播增加了。因此，我们可以看到不满足条件时就会增加对网络子目标模块的调用。公式 3.5.4 通过使用高的概率，来使行为模块更具吸引力[Goetz97]。这减少了不必要的行为切换，因为感知的微小变化而中断一个正在执行的行为的可能性将更小。

$$\sigma(x) = \left(1 + e^{k(\mu - x)}\right)^{-1} \tag{3.5.4}$$

公式 3.5.5 显示了通过一个冲实性连接从一个模块开始的激活（activation）传播过程。数据项 a_{conf} 和 p_{conf} 分别表示了连接源头的模块的激活（activation）和命题（proposition）。

$$a_{kg_i}^{t}{}'''' = -\delta \cdot \sigma\left(a_{conf\, g_i}^{t-1}\right) \cdot ex_j \cdot \tau\left(p_{conf}, s\right) \tag{3.5.5}$$

公式 3.5.6 显示了在时刻 t 一个模块 k 的激活（activation）是它在前一个时刻 t–1 的激活（activation）用惯性β加权再加上由每个目标 i 保留的激活（activation）的和。

$$a_k^{t}{}' = \beta a_k^{t-1} + \sum_i a_{kg_i}^{t} \tag{3.5.6}$$

公式 3.5.7 显示了一个模块仅保留从每个目标得到的的最大的激活绝对值。它等价于仅保留从模块到每个目标的最可靠的路径。

$$a_{kg_i}^{t} = \mathrm{abs}(\max\left(a_{kg_i}^{t}{}', a_{kg_i}^{t}{}'', a_{kg_i}^{t}{}''', a_{kg_i}^{t}{}''''\right)) \tag{3.5.7}$$

3.5.2 行为选择的质量

我们进行了一系列的实验来评估使用如图 3.5.3 所示的行为网络来进行行为选择的质量。在实验的过程中，我们给出了一个代理感知到的状态的高层次的描述、它执行的行为和控制参数的值。网络配置的默认值是“γ（ActivationInfluence）= 1.0，δ（InhibitionInfluence）= 0.9，β（Inertia）= 0.5，和θ（Global threshold）= 0.6”。这些参数在大多数情况下都工作得很好。在少数实验中我们测试了一些参数值的极限，特别是 Inertia β 和 inhibition influence δ。下面的每一个讨论都是基于我们对纪录的分析以及对代理行为的直接观察。

1. 行为的全貌

代理行为的全貌可以这样来描述：代理在关卡中游荡并且开始搜索，直到它找到敌人（Explorer）。在找到敌人之后，它开始射击（ShootEnemy）并接近敌人（GotoEnemy），接着它更换武器，并用更有效的武器锤子（FinalizeWithHammer）。在敌人死亡之后，它停止射击（StopShoot）并且再一次开始游荡。当遭到反复射击的时候，它会持续地射击；过一会儿，它停止射击敌人，并且开始躲避接踵而来的子弹。如果敌人停止射击，它会再次向敌人靠近。当代理在战斗中受伤

Download at www.Pin5i.Com

时，如果它知道急救包的位置（GotoKnownMedKit 有一个高的真值），它会去到那儿取急求包，稍后就会恢复自己的健康值。如果在靠近敌人时代理的健康值变得非常低，并且有一个可及的急救包（MedKitReachable），代理就会停止走向敌人并靠近急救包，同时保持射击——除非他已经很接近敌人，在这样的情况下，他会尝试使用锤子杀死敌人（FinalizeWithHammer）。

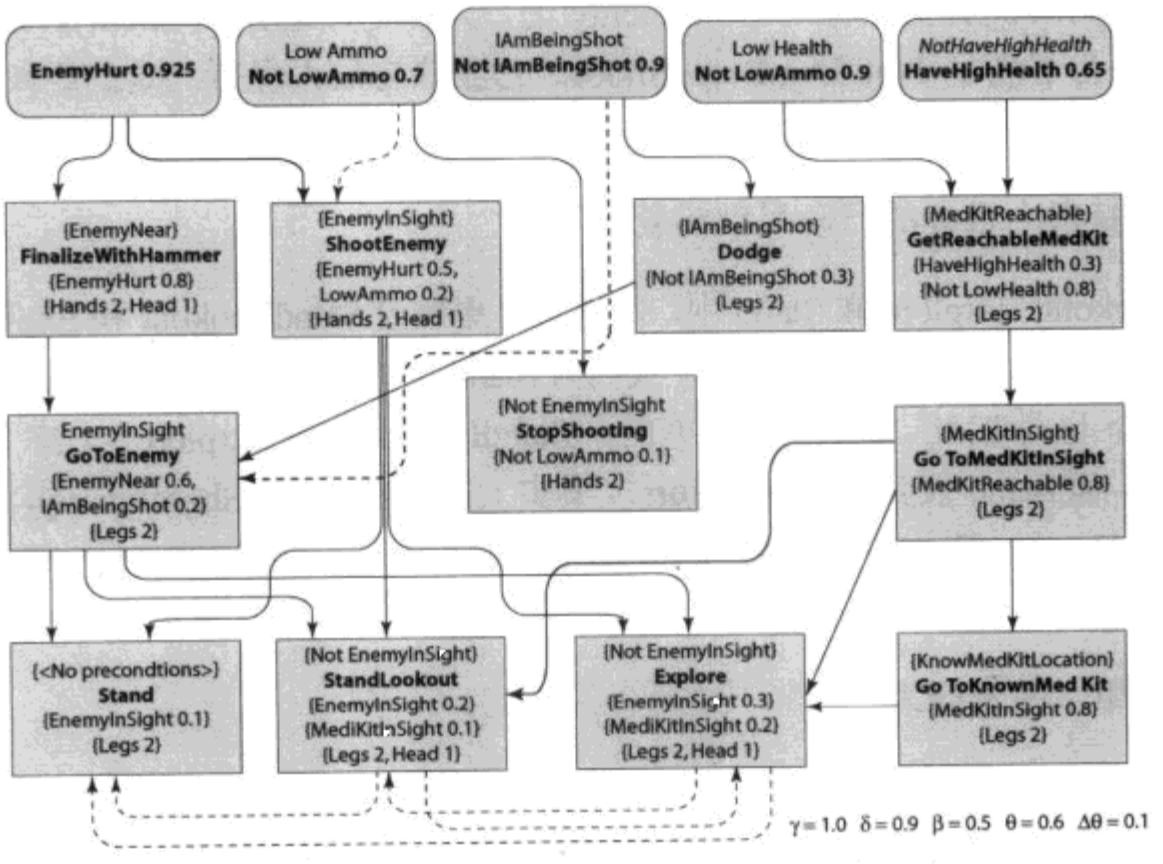


图 3.5.3　在调查行为选择算法质量中所使用的行为网络

2. 行为链

我们观察到 3 种行为序列。序列{StopShooting, Explore, GoToEnemy, ShootEnemy, FinalizeWithHammer}是代理通常的攻击序列，序列{GoToKnownMedKit, GoToMedKitInSight, GetReachableMedKit}是在视线内没有敌人且健康值不高时被执行。较长的序列{Explore, GoToEnemy, ShootEnemy, FinalizeWithHammer, StopShooting, GoToKnownMedKit, GoToMedKitInSight, GetReachableMedKit}基本上是前两个序列排列而成，并且是代理最常见的全局行为。我们看到，尽管代理没有一个正式的计划，代理还是生成了合理长度的、调和的行为链。

3. 反应性和坚定性

我们看到行为序列{Explore, GoToEnemy, FinalizeWithHammer}就像一个计划一样来实现目标{EnemyHurt}。如果在靠近敌人的过程当中代理受到射击，它就会停止行为 GotoEnemy，而去执行行为 Dodge，在躲避射击以后，GotoEnemy 被重新执行。我们看到代理在对一个事件做出反应之后，会重新回到我们看到的“计划”。它表现出了在反应性和坚定性之间的一个很好的折中。对于大的β（Inertia），在受到射击时，代理会滞后一些时间躲避，并且只有在受到多次的射击之后才会开始躲避。

4. 冲突的解决

回顾图 3.5.3。我们看到目标 EnemyHurt 企图让行为 ShootEnemy 执行，而目标 Not

Download at www.Pin5i.com

LowAmmo 企图阻止执行 ShootEnemy。目标 Not LowAmmo 的影响很小，除非代理的弹药开始缺少。当这种情况发生的时候，冲突性的影响 Not LowAmmo 使得代理把武器更换为锤子，因为锤子不需要弹药。这是在目标发生交互的时候发生的一个异常的、理智的节省弹药的方法。（注意在“设计”时节省弹药的方法是使用行为 StopShooting。）

另一个冲突的解决发生在行为 GotoEnemy 上。为了更好地完成 EnemyHurt，代理必须靠近进敌人；但是为了满足 Not IAmBeingShoot，它应该与敌人保持一定的距离。网络能够很好地处理这种冲突，在合理的时候躲避，并且在躲避了子弹之后又重新开始向敌人靠近。

5. 同时支持多个目标的行动受到偏爱

在 StandLookout、Explore 和 Stand 中，网络总是更偏爱 StandLookout 和 Explove，即使它们具备相同的执行力（executability），甚至在我们为 Stand 的效果命题 EnemyInSight 使用了更大的期望值的情况下也是如此。原因是简单的：StandLookout 和 Explore 支持 EnemyHurt 和 HaveHighHealth，而 Stand 仅支持 EnemyHurt。（事实上，它支持的是 ShootEnemy，而 ShootEnemy 接着支持 EnemyHurt）。那些支持更多目标的模块能够积累更多的激活性（activation），并且能够有更多的机会被选择。

6. 合理地组合并发行为

我们发现代理能够很好地使用它的资源并且能够合理地组合它的动作。在射击的时候它同时躲避子弹或者朝敌人靠近，它停止射击的同时会去搜索或者取得一个急救包，甚至在取一个急救包时继续射击。所有的行为组合都是合理的，并且我们能够预见的所有好的行为组合都在试验中实际地观察到了，正如前面的分析所证明的那样。

这部分叙述的一些实验已经表明扩展的行为网络提供了一个有竞争力的解决方案来为一个目标驱动的游戏代理进行行为选择。下面我们将讨论如何调整和修改行为网络来实现不同的个性。

3.5.3 个性设计

对于行为网络控制的代理，我们有三种方法来构建它们的个性：改变全局参数，改变目标强度，改变网络自身的拓扑结构。

1. 操作全局参数

改变全局的参数能够使我们控制两个重要的个性特征：慎重（通过激活阈值θ）和坚定（通过惯量β）。

一个高的激活阈值θ 能够促成更好的行为选择，因为只有那些具有高激活值（activation）的行为才可能被执行——也就是说，一般需要更多的激活传播周期才能够决定下一步做什么，并且也要求每一个模块有更高的执行力（executability）。记住，只有当执行力（executability）乘以模块的激活（activation）高于模块所需要的资源阈值时，它才会被执行。对于一个外部的观察者，这等于一个慎重的行为，因为一个代理大多数情况下都会做那些有效的并且能够有助于达成目标的行为。

一个高β 值可促成坚定的行为：在感知到的信息中如果有大的或者长期的改变时，代理才改变他的行为。假设我们需要为虚幻竞技场游戏建立两个角色 Veteran 和 Novice。Veteran

Download at www.Pin5i.Com

是安静的、理智的，他试图长远地最大化他的所有目标。他有着非常好的自我控制力和坚定性，并且想要杀死尽可能多的敌人，但是决不会以自己的生命为代价。Novice 希望像 Veteran 一样，并且也有着接近的值，但是 Novice 显然缺乏坚定性和合理行为的纪律性。他是冲动的，并且经常采取一些在某个环境中并非最恰当的行为。

我们注意到对 Veteran 的个性要求与很多游戏需要建立能够最大化其积分的代理角色的要求是符合的。这正是图 3.5.3 所示的那个代理。看一下在上节中提到的全局行为的描述，我们发现我们有了一个战斗原型 Veteran 的个性：代理在杀戮的时候非常坚定，在安全的情况下会使自己康复，并且有着持续战斗的耐力，即使是在遭到射击的情况下——没有“恐惧”。

那么，我们如何把这个 Veteran 接着转化为 Novice 呢？为了使代理变得更冲动，我们降低它的惯量（β），使得它变得过于应激。这样的代理当受到射击的时候就迅速地企图躲避。同时，在追踪敌人的时候，如果代理的健康值变得很低并且看到一个路径可及的急救包，代理就会迅速地靠近这个急救包。这导致了一个较差的行为：通常敌人和代理受伤一样严重，用锤子进行一次攻击就可以获得胜利。追踪也是一样，躲避是好的，但是仅仅是为了防止受到过分伤害，因为过分受伤害不能保证成功。

为了增加 Novice 的错误次数，我们降低了全局的阈值。通过这样的的方法，降低了行为选择的质量，因为更多的行为同时超过了阈值；并且在超过阈值的模块中，任何一个都不比其他的优先。现在，即使在它非常地接近敌人，并可以使用锤子来攻击敌人时，代理也经常是射击敌人。对于一个非常低的值 0.1，代理也常常静止地站着（Stand）而不是搜索关卡（Explore）。

我们找到的来实现 Novice 的最好的参数是$\gamma=1.0$，$\delta=0.9$，$\beta=0.1$，$\theta=0.25$，$\Delta\theta=0.1$。

2. 操作目标强度

现在，假设游戏设计师惊叹于我们轻而易举地从原来的 Veteran 中创作出了一个新的角色，于是要求我们塑造另外的两个角色：Samurai 和 Berserker。

Samurai 是冷酷坚定和具有攻击性的。在战斗中战死是他的最高荣誉，并且他喜欢一对一的战斗。杀死他的对手是他的较强的目标，并且他将试图实现这个目标而不惜牺牲自己的生命。在和敌人战斗的时候，他不会停下来去攻击另一个代理，他也不会因为痛苦或者危险而停止攻击。

Berserker 具有攻击性，不守纪律，也没有坚定性。只要在竞技场中，他就会以一种疯狂的方式粗暴地攻击对手。他对痛苦极不敏感，并且在大多数情况下他不会停止攻击以康复自己或者躲避子弹。

为了把 Veteran 变成 Samurai，从目标的强度着手，因为 Veteran 和 Samurai 一样是坚定的、有经验的和慎重的。我们设置 EnemyHurt 为 1.0，NotIAmBeingShot 为 0.6，NotLowHealth 为 0.5 和 HaveHighHealth 为 0.4。在这样的强度作用下，代理将总是接近敌人而不是躲避子弹，并且在战斗中代理不会停下来去取急救包。我们验证了这些参数所造就的代理的行为，每当发现一个敌人，代理就射击着靠近他，并且如果敌人还没有死亡的话，就接着用锤子进行攻击（FinilizeWithHammer）。如果在视线中没有敌人，Samurai 就会寻找急救包来恢复健康值。对于一个玩家来说，Samurai 显示出了应用的行为：在他追踪敌人的时候，他从不为病痛（low health）或者危险（being shot at）干扰，并且他会灵活运用战术（在远处射击而在近处用锤子攻击）。

为了实现 Berserker，我们从 Samurai 开始，因为其目标是类似的。减小目标 LowHealth 的重要性可使代理对痛苦的感受性变得更低。为了实现他的疯狂，我们减小了惯量和他的全

Download at www.Pin5i.Com

局阈值。减小 β 使得代理非常的应激，而减小 θ 使得代理采取了疯狂的行动，比如在近距离射击而不是使用锤子。

Berserker 的全局行为正如所期望的：在战斗中他不会停下来躲避子弹或者进行康复，并且他疯狂地战斗—敲击和射击在他的路径中碰上的任何东西。

3. 添加新的行为模块

这里缺少了在战半环境中的一个基本角色原形：Coward。他的主要目标是不受伤害地活着从战斗中逃脱，所以他将避免和敌人遭遇，并且将优先考虑维持或者恢复健康值。

为了实现 Coward，我们从 Veteran 开始，减小 EnemyHurt 并且提高 HaveHighHealth。我们让全局的常数维持原状，因为像 Samurai 一样，Coward 是慎重的；像 Veteran 一样，他也是坚定的。

Coward 的行为可以这样来描述：他开始搜索关卡直到发现一个敌人。当敌人在视线中时，他开始射击。在射击过程中，如果射向他的子弹没有打中他，他就会开始躲避后来的子弹。如果他被子弹击中，他就会迅速地去取得任何可及的急救包。在没有任何可及的急救包的情况下，他会持续地躲避子弹并战斗，直到他的健康值变得非常低。当这种情况发生时，他会逃离战斗并且去恢复他的健康值，即使是不得不去取一个很远的急救包。

尽管代理已经对他的健康值过分关心，并且也不能够再被描述为勇敢，对于他的说明，有一个非常重要的东西还是被忽略了：主动地避免交战。为了达到这一点，我们为网络实现了一个新的模块：GoAwayFromEnemy。通过添加这个模块，当 Coward 看到一个敌人时，他会边射击边远离敌人（GoAwayFromEnemy 和 ShootEnemy 被同时执行）。图 3.5.4 显示了 Coward 角色的网络。注意，添加新行为是一个简单的模块操作，是一个无关紧要的修改。网络框架很自然地处理模块之间的联系。

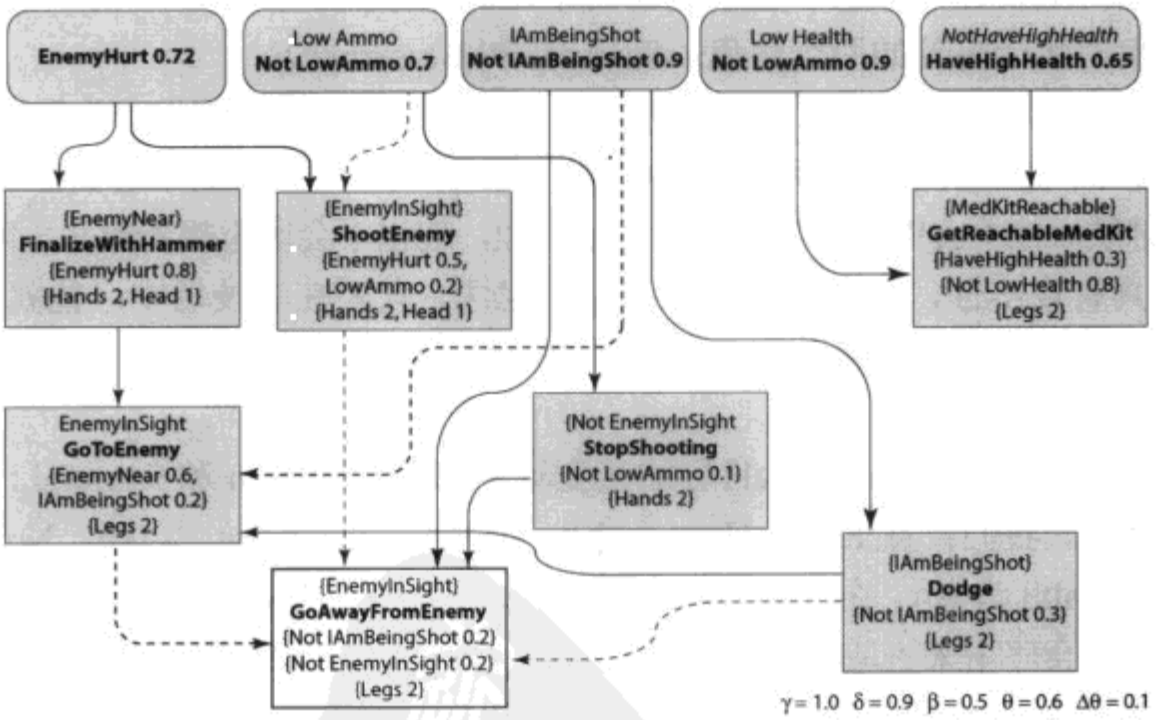


图 3.5.4 Coward 行为网络

3.5.4 结论

我们看到，对于在连续的动态环境中肩负复杂的目标和动作的游戏代理，比如现代的动

Download at www.Pin5i.Com

作游戏，扩展的行为网络是一个好的行为选择机制。

本文验证了在三维动作游戏领域的一个足够出色的行为选择算法的属性，即坚定性、寻找机会、偏爱同时满足多个目标的行动、合理的冲突解决方案、通过动作序列来实现目标和明智地选择并发行为。

从代理设计的角度，扩展的行为网络最突出的一点是它的模块化和能够比较容易地集成新的目标和行为。设计者可以一次开发和测试一个模块，并且建立一个基本行为库。当设计一个代理时，我们能够只需装配模块，而由网络框架去处理它们之间的交互。这就把设计者从必须预见可能的模块间的每一次交互的繁重负担中解脱出来，从而创作出更有趣的具有沉浸性的游戏。

为了塑造一个代理，我们从设置目标开始。目标反映了角色的价值观和动机。接着，我们开始装配一组模块，这些模块能够实现设置的目标（比如，那些把目标的条件作为效果的模块）。最后，通过调整目标的强度值和全局参数，就可以调整代理的全局行为，因而就完成了所需要的个性的设计。

对典型角色设计的实验表明：对于简单的角色，扩展的行为网络是一个有竞争力的解决方案。它们是简单的，并且仅仅通过调整一些常数值，就能够使我们塑造出个性非常不同的代理。

最后，必须指出的是扩展的行为网络满足目标驱动的动作计划框架的要求，并且还有更多的优点：允许设置环境相关的目标、能够很好地处理目标间的冲突，是机器人领域中一项非常成熟的技术。

3.5.5　参考文献

[Dorer99] Dorer, K., "Extended Behavior Networks for the Magma Freiburg Team." RoboCup-99 Team Descriptions for the Simulation League, Linkoping University Press, 1999: pp. 79–83.

[Dorer04] Dorer, K., "Extended Behavior Networks for Behavior Selection in Dynamic and Continuous Domains." *Proceedings of the ECAI workshop, Agents in dynamic domains,* U. Visser, et al. (Hrsg.) Valencia, Spain, 2004.

[Goetz97] Goetz, P., "Attractors in Recurrent Behavior Networks." Ph.D. Thesis, University of New York, Buffalo, 1997.

[Maes89] Maes, P., "How to do The Right Thing." *Connection Science Journal,* Vol. 1, No. 3., 1989.

[Müller01] Müller, K., "Roboterfußball: Multiagentensystem." February 2001. CS Freiburg, Diplomarbeit. University Freiburg, Germany.

[Nebel03] Nebel, B. and Y. Babovich, "Goal-Converging Behavior Networks and Self-Solving Planning Domains, or: How to Become a Successful Soccer Player." s.l. IJCAI03, 2003.

[Pinto05-a] Pinto, H. and L. O. Alvares, "An Extended Behavior Network for a Game Agent." Anais do Quinto Encontro Nacional de Inteligência Artificial, Brazil, 2005-a.

[Pinto05-b] Pinto, H. and L. O. Alvares, "Extended Behavior Networks and Agent Personality: Investigating the Design of Character Stereotypes in the Game Unreal Tournament." *Proceedings of the Fifth International Working Conference on Intelligent Virtual Agents,* Kos, Greece, 2005-b.

Download at www.Pin5i.Com

[Rhodes96] Rhodes, Bradley, "PHISH-Nets: Planning Heuristically in Situated Hybrid Networks." Masters of Science Thesis, Massachusetts Institute of Technology, 1996.

[Tyrrell93] Tyrrell, Toby, "Computational Mechanisms for Action Selection." Ph.D. Thesis, University of Edinburgh, U.K., 1993.

Download at www.Pin5i.Com

3.6 用支持向量机为短期记忆建模

Julien Hamaide,
Elsewhere Entertainment
julien.hamaide@gmail.com

游戏中你能碰到的最有意思的问题就是挑战人工智能的编程。最关键的部分就是学习过程。当游戏玩家不断地变化玩游戏的策略，学习系统就会经历很多的困难。经典的学习系统需要花费很多时间去忘记旧的规则，同时学习新规则。作为一种解决方法，本文为学习分类器引进了一个时间概念。这样，系统就可以很容易地忘记过去的事情。在这里提出了支持向量机（SVMs，Support Vector Machines），这种技术可以应用于其他类型的分类器。

3.6.1 支持向量机简介

支持向量机（SVMs）是学习分类器。例如，神经网络学习分类器。但是它们使用的概念略有不同。支持向量机（SVMs）是一个线性分类器的概括。与神经网络相比，SVMs 关键的优越性是它们的创建和训练算法。这里将快速地介绍一些关键概念。更多的信息可以在[Cristianini01]中找到。首先是介绍线性分类器，然后推广到非线性的问题。这个介绍包含了核心概念和训练算法。

1. 线性分类器

在超平面的帮助下，线性分类器可以把输入空间分成两个部分：类 A 和类 B。位于超平面上部的每一点属于类 A，而位于超平面下部的每一点属于类 B。图 3.6.1 显示了一个已分类的数据集。在方程 3.6.1 定义的平面 D 是分类器的唯一参数。如果点 x_i 被分为类 A 和类 B，如果分类器是经过训练的，这些数据的分类可以由方程 3.6.2 和方程 3.6.3 来实现。

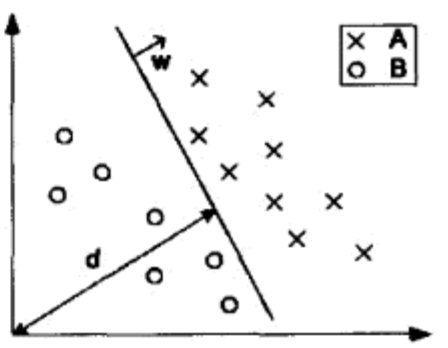


图 3.6.1 线形可分离数据集

$\langle x, y\rangle$ 代表了标准点积 $\langle x, y\rangle = x_1 \cdot y_1 + x_2 \cdot y_2 + \ldots$。

$$D \equiv \langle w, x\rangle + d = 0 \tag{3.6.1}$$

$$x_i \in A \Leftrightarrow \langle w, x_i\rangle + d > 0 \tag{3.6.2}$$

$$x_i \in B \Leftrightarrow \langle w, x_i\rangle + d < 0 \tag{3.6.3}$$

Download at www.Pin5i.Com

然而，由两个约束条件组成的系统的训练并非无足轻重。如果具备方程3.6.4和方程3.6.5，那么分类的条件就变成了方程3.6.6。

$$y_i = 1 \Leftrightarrow x_i \in \mathrm{A} \tag{3.6.4}$$

$$y_i = -1 \Leftrightarrow x_i \in \mathrm{B} \tag{3.6.5}$$

$$y_i \bullet (\langle w, x_i \rangle + d) > 0 \tag{3.6.6}$$

因此，就像方程3.6.6验证的每个点，训练就是要寻找 w 和 d。但是，这里存在着无数个分离平面（见图 3.6.2）。需要从这些平面中选择一个。我们尽量选择一个最能概括分类器的平面，也就是说，使用一些没有包含在训练集合中的数据，也能得到比较好的结果。我们决定使用方程3.6.7而不是方程3.6.6。就像在图3.6.3中显示的那样，有两个分离平面。每一个平面对应一个 y_i 值（1或–1）。平面间的距离称为边界。可以表明，边界是与 $1/|w|$ 成比例的[Moore01]。

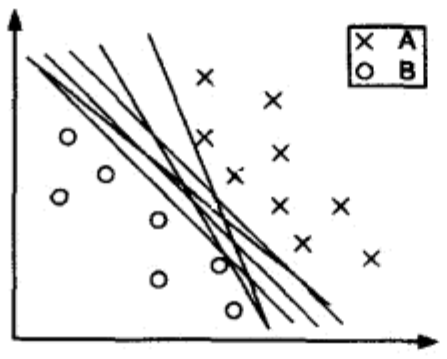


图 3.6.2 几个分离数据集的平面

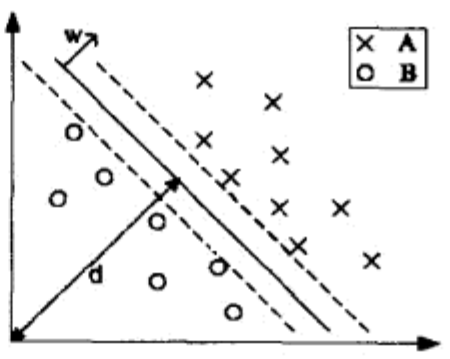


图 3.6.3 最好的分离平面

为了获得更概括的分类器，必须把边缘最大化，或者最小化$|w|$。这将被用于训练算法。

$$y_i \bullet (\langle w, x_i \rangle + d) \geqslant 1 \tag{3.6.7}$$

验证方程3.6.8的那些点 x_i 称为支持向量。这些点支持我们这个解决方法，它们分布在限定平面上，并且位于它们类的边缘。

$$y_i \bullet (\langle w, x_i \rangle + d) = 1 \tag{3.6.8}$$

可以证明，向量 w 是支持向量的一个线性组合[Burges98]。方程3.6.8和方程3.6.9的组合可被改写为方程3.6.10。现在分类的条件与训练数据无关。

$$w = \sum \alpha_j y_j x_j \text{ 且} \alpha_j \geqslant 0 \tag{3.6.9}$$

$$\sum \alpha_i y_i \langle x_i, x \rangle + d \geqslant 1 \tag{3.6.10}$$

现在的训练在于寻找α_i和 d。每一个不能验证方程3.6.8的点，系数α_i将是空的。因此可以把它从解决方法中移走。这是训练所要达到的一个目标。支持向量及其相应的系数用作为分类器的参数。

图 3.6.4 表明的解决平面都依赖于支持向量，具有如下特点：因为它们都共享法线 w，这些平面必定互相平行。可以证明，去掉一个非支持向量的向量，解决方法并不改变。这是合乎逻辑的，因为它们的α_i是空的，所以不会改变分类器的参数。

如果想用一个新数据集来训练系统，没有必要添加整个旧数据集。我们需要的仅仅是旧数据集中的支持向量。

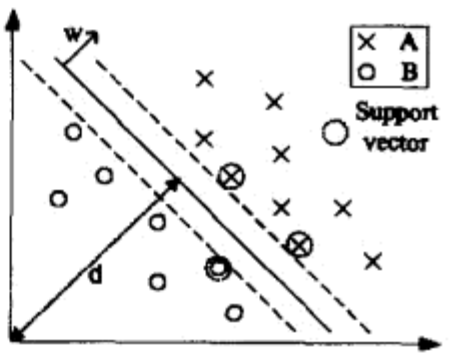


图 3.6.4 在支持向量中的平面

Download at www.Pin5i.Com

2. 非线性可分离数据集的扩展

如果输入空间被转换成一个高维空间，非线性可分离数据集也可以变成线性可分离的。图 3.6.5 就显示了这样一个空间转化的概念。

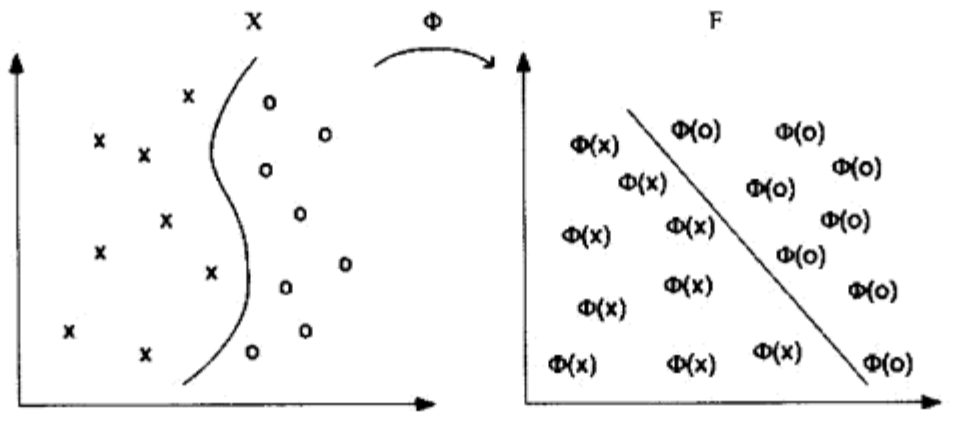


图 3.6.5　输入空间转换

图 3.6.6（a）显示了一个具体的例子。如果你试图用线性分类器为这些数据集分类的话，无法获得分离平面。如果在方程 3.6.11 的帮助下，把一维的空间转换成一个二维的空间，那么这时候的数据集就是线性可分离的。图 3.6.6（b）表示转换过程和一个成功的分离平面。这个例子虽然简单，但也表明这种转化为更高维空间的神奇之处。

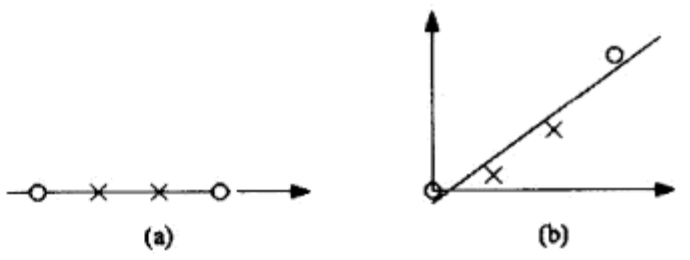


图 3.6.6　通过使用转换成一个更高维空间来解决问题的例子

$$\Phi(x) = (x, x^2) \tag{3.6.11}$$

3. 核心函数

如果转换应用于每一个向量 x_i 和 w，方程 3.6.10 就变成了方程 3.6.12。

$\langle \Phi(x_i), \Phi(x) \rangle$ 是两个转换向量的点积。尽管转换空间已成为高维的，但返回值将总是一个标量值。可以把 $\langle \Phi(x_i), \Phi(x) \rangle$ 当成一个简单的函数，其功能就是要返回一个标量。这个函数称为 *kernel*(*K*)（核心函数）。方程 3.6.13 给出了这个函数 $K(x, y)$的定义。现在不需要强求知道$\Phi(x)$的值。此时的$\Phi(x)$也可以转换数据到一个无限纬度空间（核心指数是这种转换的一个例子）。作为一个例子，方程 3.6.11 表示的等价的核心转换将在方程 3.6.14 中计算。现在我们可以重写分类条件，最后形式如方程 3.6.15 所示。

$$\sum \alpha_i y_i \langle \Phi(x_i), \Phi(x) \rangle + d \geqslant 0 \tag{3.6.12}$$

$$K(x, y) = \langle \Phi(x), \Phi(y) \rangle \tag{3.6.13}$$

$$K(x, y) = \langle \Phi(x), \Phi(y) \rangle = \left\langle \begin{pmatrix} x \\ x^2 \end{pmatrix}, \begin{pmatrix} y \\ y^2 \end{pmatrix} \right\rangle = x \cdot y + x^2 \cdot y^2 \tag{3.6.14}$$

$$\sum \alpha_i y_i K(x_i, x) + d \geqslant 0 \tag{3.6.15}$$

核心函数必须验证在[Burges98]中可以看到的一些条件。这种情况下，一些$\Phi(x)$与核心函数匹配。方程 3.6.16 中的核心函数具有如下的一些重要的属性。

$$K(x, y) = K(y, x)$$

Download at www.Pin5i.Com

$$
\begin{aligned}
&K(x,y) = K_1(x,y) + K_2(x,y)\\
&K(x,y) = \alpha K_1(x,y) \text{ with } \alpha > 0\\
&K(x,y) = K_1(x,y) \cdot K_2(x,y)\\
&K(x,y) = f(x) \cdot f(y)\\
&K(x,y) = K_3(\Phi(x),\Phi(y))
\end{aligned}
\tag{3.6.16}
$$

方程 3.6.17 展示了一些经典的核心函数。它们可以同方程 3.6.16 中显示的属性相结合。

$$
\begin{aligned}
K(x,y) &= \exp\left(-\|x-y\|^2 / 2\sigma^2\right)\\
K(x,y) &= \langle x,y\rangle^n\\
K(x,y) &= (\langle x,y\rangle + 1)^n
\end{aligned}
\tag{3.6.17}
$$

4. 训练

如前所述，在方程 3.6.7 的约束下，训练在于寻找 w 和 b。使用方程 3.6.9，在方程 3.6.15 的约束下，训练就是要寻找 a_i 和 b。目标就是要找到一个最概括的分离平面。必须最大化边界，或者最小化$|w|$。于是算法训练就变成了方程 3.6.18。这是一种有条件的优化。

应用拉格朗日优化理论，方程 3.6.18 就变成了方程 3.6.19[Cristianini01]。方程 3.6.19 是一种二次优化问题。二次优化理论表明问题是凸起的，不存在局部最小化问题。在这个优化问题中，通常可以找到最好的结果。这是支持向量机（SVM）的一个重要优势：当给定数据集并选中核心函数后，通常可以找到最好的结果。多次训练这个系统都得到一个完全相同的结果。虽然二次优化算法有很多种，但已超出了本文的论述范围，此处，我们仅仅介绍了支持向量机。更多的信息可以在[Gould]中找到。

$$
\min_w \frac{1}{2}\|w\|^2 \; with \sum \alpha_i y_i K(x_i,x) + d \geqslant 0 \tag{3.6.18}
$$

$$
\max_a \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i,x_j) \tag{3.6.19}
$$

3.6.2 短期记忆模型化

分类器系统是用收集的数据集来进行训练的。在一个神经网络中，不可能决定或者知道何时网络会忘记过去学到的一些东西。在支持向量机（SVM）中，分类是由支持向量和它们的系数 a_i（见方程 3.6.9）来描述的。这些向量是收集到的数据集的一部分。这样就有可能给每个向量标定日期。然后，经过一定的时间段，为了取消向量的影响，可以把它从解法集中移走。整个数据集就可以不断地减少最老的向量，增加最新的向量。图 3.6.7 显示了移走一个过时的支持向量时，相应的解法集的变化情况。图 3.6.8 显示了添加一个新的支持向量时，相应的解法集的变化情况。

一方面，只要支持向量没有改变（或者变化的新老数据都不是支持向量），训练就会给出完全一样的结果，解法集也不会改变，并且训练速度也非常快。另外一方面，如果移走一个支持向量或者新添一个支持向量，与第一种情况相比，训练需要的时间会更长一些。但是训练速度也很快，因为非支持向量的数据权重（a_i）仍然为零。

Download at www.Pin5i.Com

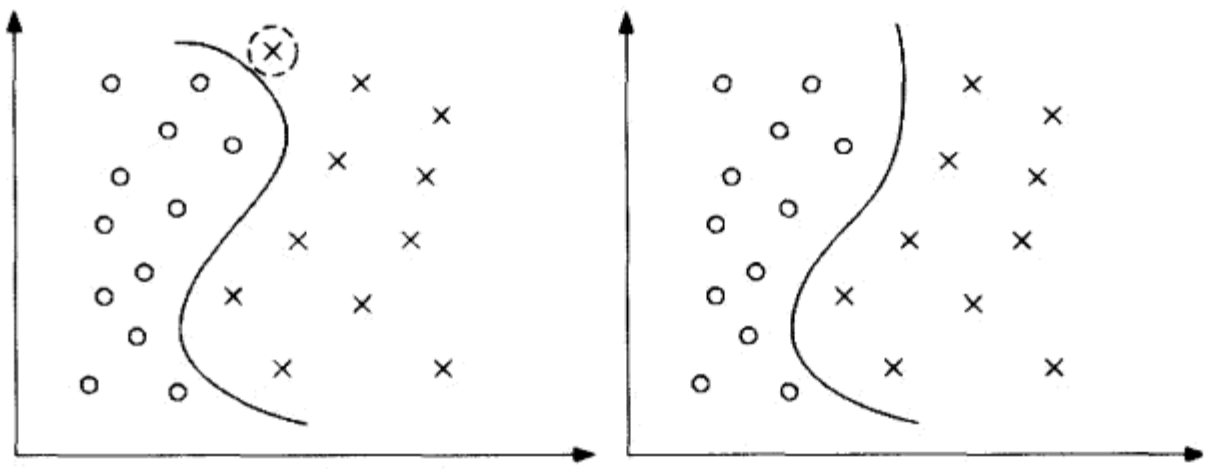


图 3.6.7　移除一个过时的向量

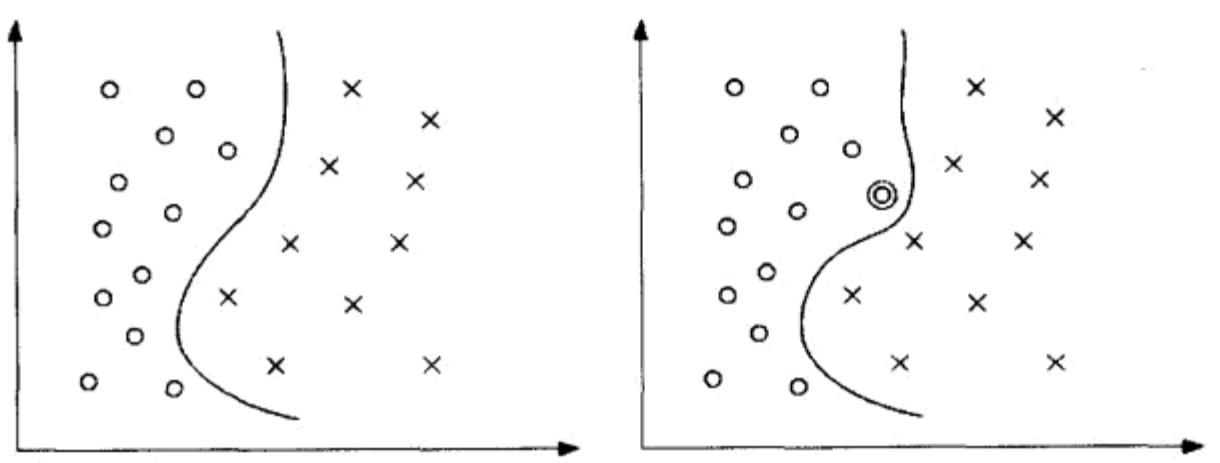


图 3.6.8　添加一个新的支持向量

系统派生可能会是一个问题，而且除了支持向量和它们的系数 a_i，解法也受到所选择使用的核心函数的约束。虽然这或许是一个优点，或许是一个缺点，因为要选择这样的一个核心函数就可能会非常棘手。

这种技术在一个充满变化的环境中相当有用，因为过去学习到的知识对现在不会有无止境的影响。的确，对于神经网络，如果环境发生了变化，一些决策可能就不再有效。有时为了纠正这种趋势，我们会犯多次错误。而对于带有标定日期的支持向量机（SVM），只会有一次错误或一次长时间的等待（例如，过时的向量）。

虽然只在 SVM 中介绍了这种技术，显然，这种技术也可以适用到其他的分类器，只要分类器是用从训练数据集中得到的数据来表示它们的参数，例如最邻近算法（KNN，K-Nearest Neighbors）。在加强学习方面，这种 SVM 可以代替神经网络。

3.6.3　CPU 的消耗限制

去评估点 x 的值，需要计算方程 3.6.20。这种分类需要计算 $K(x, y)$的值和每个支持向量的两个乘积。重要的是限制分类器对 CPU 的消耗。降低 CPU 消耗的唯一方法是限制支持向量的数目，并限制到一个固定的值。使用这些带有日期的系统，支持向量数目太高时，最老的支持向量在评估时可以抛开不计。然而，抛开不计的支持向量不能从训练数据集中移走。如果是这样的话，支持向量必将被另外一个取代。这种行为将会降低分类器的有效性。在游戏中，限制支持向量的数目是非常重要的。因为它在决策中引入了一些噪音，所以效率下降

Download at www.Pin5i.Com

并非总是坏事。

$$\sum \alpha_i y_i K(x_i, x) \geqslant 0 \tag{3.6.20}$$

3.6.4 结论

这篇文章中，除了介绍支持向量机（SVMs），还引出老化数据的概念。尽管这一概念可以用于大多数的分类器系统，但 SVM 是个很好的候选者，因为分类器的参数是用训练数据集来表示的。它还为支持向量的数量限制和 CPU 消耗的限制提出了解决方法。

3.6.5 参考文献

[Burges98] Burges, Christopher, "A Tutorial on Support Vector Machines for Pattern Recognition." 1998. *Knowledge Discovery and Data Mining,* Vol. 2, No. 2: pp. 955–974.

[Cristianini01] Cristianini, Nello, "Tutorial on Support Vector Machines and Kernel Methods." ICML-2001 Tutorials. Available online at *http://www.support-vector. net/icml-tutorial.pdf.*

[Gould] Gould, Nick and Philippe Toint, "A Quadratic Programming Page." Available online at *http://www.numerical.rl.ac.uk/qp/qp.html*.

[Moore01] Moore, Andrew W., "Support Vector Machines." Available online at *http://www.autonlab.org/tutorials/svm.html*.

Download at www.Pin5i.Com

3.7 使用战力值评估模型进行战争役分析

Michael Ramsey

QJM（Quantified Judgment Model）不仅是一种战争模型也是一种战争理论。最初，它是用来模拟历史上的战争的，后来，经过进一步的修改它被用于现代战争的预测。这是预测游戏中潜在胜利者的一个理想系统。在本文中，我们将描述基本的 QJM 公式。可以通过增加模型来进一步地扩充这个基本公式，例如，磨损计算模型、单位的空间效力模型和伤亡效力模型。

只要有冲突存在，就会有不同的理论来指导这些战争，对这些丰富的战争理论史做出卓越贡献的两个人是：亨利·约米尼[Jomini92]和卡尔·冯·克劳塞维茨[Howard83]。约米尼发展了与复杂的几何图形相关的作战理论。而克劳塞维茨则从一个更加哲学和定性的角度来描述战争。克劳塞维茨的定律之一就是“数量法则”。最初，这个法则用来处理两个数量之间简单的关系，使用一个随机变量来说明战争的不确定性。这个数量法则是 QJM 及其潜在的一些理论的精神灵感和基本基础。

QJM 提供了一个相对简单的框架，在这个框架中，通过评估和计算一系列可变因素来对两个对立势力进行比较。QJM 是由狄·恩·杜普伊（T.N.Dupuy）[Dupuy84]发明的，QJM 最初是为军队服务的，用于评估历史上的战争。量化军事战争最容易的部分就是武器，以及它们对战士、地形和结构的影响。一种特定武器的特征通常都有恰当的定义，并且武器通常是服从标准规范的。但是在战争中，武器通常并不遵守每个规范；QJM 会考虑这些“变化”的因素来量化这些武器的用途。QJM 过程的另一个方面包含一些通常被其他理论排除在外的因素，其中包含的一个因素就是行为因素。本文详述了基本的 QJM 公式，以及如何使用它快速、持续、有效地预测战争结果。

3.7.1 基本公式

实际上，QJM 公式非常简单，它集合了一些复杂变量。因而不必留待论文的最后才介绍这个公式，现在，我们就来看看这个公式，然后将在这篇论文的其他部分剖析其细节，最后，分析它的一个例子。

基本的 QJM 公式会生成一个战斗力的值（CP）：

Download at www.Pin5i.Com

$$CP=(ForceStrength(FS))7\times(VariableFactors(VF))\times(CombatEffectiveness(CEV)) \quad (3.7.1)$$

这个公式看起来十分简单，实际上也是如此；但像大多数的数学公式一样，它的细节很重要。贯穿整篇论文，基本公式将发展成为一个简洁的模型，同时，还要识别各个子公式独特的细节。除非另有说明，所有变量都定义为 0 和 1 之间的值。

3.7.2 计算兵力

QJM 的一个组成部分是兵力（FS），它是用来打击对手的所有火力。兵力会因众多变量因素而变更。这些变量的范围包括从气候、地形到后勤这些因素，还有领导能力等。用于计算兵力的基本功公式是：

$$S=(W_N\times V_N)+(W_G\times V_G)+(W_I\times V_I)+(W_Y\times V_Y)+(W_L\times V_L)+(W_O\times V_O) \quad (3.7.2)$$

这里：

S 是计算的兵力；

W 是特定武器累计的作战杀伤力指数（OLI）；

V 是变量的影响因子。

作战杀伤力指数的子标识以及变量的影响因子是：

N 是步兵武器标识符；

G 是炮兵武器标识符；

I 是装甲武器标识符；

Y 是空中支援武器标识符；

L 是反装甲武器标识符；

O 是空中防御武器标识符。

最初的 QJM 兵力（FS）是一个严格的现实世界中的术语，没有理由 FS 不能被科学幻想设定的或者由虚幻设定产生的值来改变或者替代。我们甚至可以很容易地创造出一种卫星武器。唯一的前提条件是：这个公式能被一致地应用于交战双方。

3.7.3 计算潜在兵力

QJM 过程的下一步是产生一个潜在兵力（P）。潜在兵力利用分配给各种作战标识符的值。作战标识符不是个别部门武器的统计，而是从作战优势角度观察到的平均值。这种方法有助于抑制一个部门的武器类型对另一部门武器类型的影响。例如，一个军队某一部门的武器可能是一种不可思议的快速侦察坦克。而大部分部门的武器是慢而笨重的坦克。当然侦察坦克的速度由于全军的整体力量而降低。基本的 QJM 公式也使用操作符，例如士气和兵力的训练。计算潜在兵力最有意思的一个方面是：不需要包含所有的基本标识符；如果你不需要这些标识符，你不使用它就可以了。这一点真正地使 QJM 系统具有天生的即插即用性，而这正是即插即用功能的一个范例。

潜在兵力的公式是：

$$P=S\times M\times L\times T\times U\times B\times US\times RU\times HU\times ZU\times V \quad (3.7.3)$$

潜在兵力的各种组成部分的定义如下：

P 是计算的潜在兵力。

Download at www.Pin5i.Com

S 是从方程 3.7.1 中计算出来的兵力。
M 是作战灵活值。
L 是作战领导值。
I 是作战训练值。
O 是作战士气。
B 是作战后勤标识符，它集合了一些概念，例如供应和通讯。
US 是作战的态势。
RU 是作战的地形变化。
HU 是作战的气候变化。
ZU 是作战的季节因子。
V 是一般作战的弱点。

3.7.4　为武器效力进行建模

武器效力的质量是由其作战杀伤力指数（OLI）决定的。基于历史的观点，作战杀伤力指数最初代表了武器的相对杀伤力。这就允许对历史上的任何武器进行一下比较，例如一个燧发枪，与现代的 M-16，甚至可能是未来的某种武器进行比较。由于武器具有如此悬殊的杀伤能力，因此不可能建立一个精确的 OLI 模型。但是 OLI 为变化的武器类别提供了一个一致的框架。这个框架最基本的方面包含在下列几个变量中：武器发射率、可靠性、准确性、范围、对象和每次射击的目标数。这五个变量是这个统一框架的一部分，它定义了理论上的杀伤力指数（TLI）。TLI 的结果值是用每小时的人员伤亡数来表示的。利用一个预定的分散因子可以把 TLI 转换成 OLI。

TLI 的计算公式是：

$$TLI = RF \times R \times A \times C \times RN \tag{3.7.4}$$

TLI 各组成部分的定义是：
RF 是发射率；
R 是可靠性；
A 是准确性；
C 每次射中需要射击的次数；
RN 是范围。

TLI 每个成分的确定有点不同。发射率 RF 是在一段时间内对准目标有效发射的次数。标准的时间长度通常是一个小时。一件武器的可靠性 R 是它将失败的次数。这里的失败包括一切从武器维修到一般的武器失败。使用直接交手的武器被认为是 100%的可靠。武器的准确性 A 没有人为的因素。武器应该认为是可见的和上了枪栓的。然后，利用预定的射击率和射击时间，当射击一个固定目标时，就能清楚地了解这种武器的精确度。每个目标射击的次数（*C*）是用预定的目标密度来计算的，这个目标密度是用来衡量这个武器可以射中多少个潜在的目标。一把剑的 *C* 值通常是 1，而任何规模的炸弹的 *C* 值通常是数百到数千。

分散因子是这样的一个值：它定义了一些武器种类的继承能力，在更大范围内会造成更大的损失。表 3.7.1 包含了一些分散因子的例子。

Download at www.Pin5i.Com

表 3.7.1 不同军事时代的分散因子

时　期	分散因子	时　期	分散因子
早期的人类（石头武器和棍棒）	1	第一次世界大战	100
中世纪（弓，矛）	2	第二次世界大战	5 000
拿破仑一世时期（火药时代）	10	21 世纪早期（核）	10 000
内战（来复枪和加农炮）	20		

为了把 TLI 转换为一个可用在军事环境的值，它必须转换成作战杀伤力指数。OLI 的公式是：

$$OLI = \frac{TLI}{DF} \tag{3.7.5}$$

需要由类别来组织 OLI 的值。标准分类定义如下：步兵、炮兵、装甲车、空中支援、空中防御和反装甲武器。我们可以增加更多的种类。但是，一般来说，在添加一种新武器之前，要先评估这种新式武器是否属于原先已存在的类别。OLI 的值只需要计算一次，并且可以在多次作战中使用。

3.7.5 获得一个理论上的战争结局

现在，通过评估战斗双方力量的一个简单比率，我们就可以知道一场战争理论上的结局。潜在兵力比（PPR）是：

$$PPR = \frac{(PowerPoterntialForce1)}{(PowerPotentialForce2)} \tag{3.7.6}$$

如果 *PPR* 等于 1.0 或更大的话，那么 *Force*1 将是理论上的胜利者。如果 *PPR* 小于 1.0，则 *Force*2 应该是胜利者。

3.7.6 关于 CEV

就像以前我们说明的那样，QJM 允许添加部件和删除部件。但是像 CEV 这样的组成部分是否也是可选项？答案是肯定的。尽管 CEV 是基本公式的组成部分，它的作用仅仅是当历史上有相应的战争时，用来预测未来的战争。如果我们为一个现实的第二次世界大战游戏编写人工智能（AI）程序的话，我们就可以拿我们的结果与实际的历史战争役相比较。详情可以在[Duputy85]中找到。

3.7.7 一个 QJM 系统的例子

现在，我们进入一个范例来体验一下 QJM 系统。这个例子描绘的是古代发生在一个开阔平原上的两军之间一个战斗，其中一支军队无论在体力上还是在士气上都已经累得筋疲力尽了。由于战争设定发生在古代，其分散因子将为 1。

我们有两支对峙的军队。第一支军队有 5 000 名步兵，持有刀和短柄斧；5 000 名步兵持有矛；5 000 名步兵持有弓箭。第二支军队有 10 000 名步兵持刀和 5 000 名弓箭手。第一步为

Download at www.Pin5i.com

三种不同的武器种类计算 TLI：

配备刀的步兵的 TLI 是：60×1×1×1×1=60。

配备矛的步兵的 TLI 是：30×0.5×1×1×3=45。

配备弓箭的步兵的 TLI 是：20×0.2×1×1×20=80。

第二步是把 TLIs 转换成 OLIs。这将理论杀伤力指数转换为作战杀伤力指数。就像前面指出的那样，这只是通过分散因子划分 TLI。因为这是一个虚幻的设置，分散因子定义为 1。所以配备刀、矛、弓箭的步兵相对应的 OLIs 分别是 60、45 和 80。

下一步是计算 FS（兵力）。由于计算兵力涉及到一些可变的因素，我们必须确定这些因素。这些可变因素包括地形、势态、季节和气候。我们为这个例子定义了地形变量（参见表 3.7.2）。

表 3.7.2　步兵的变化的地形因素。在这种地形步上调遣步兵是非常艰苦的，并且战斗是在给定值接近为零的地形上发生的

地 形 类 型	变量（0.0 到 1.0）	地 形 类 型	变量（0.0 到 1.0）
空旷的/无障碍的	1.0	多山的	0.1
被森林所覆盖的	0.5		

第一支军队的兵力是：

$$((60\times5)\times1)+((45\times5)\times1)+((80\times5)\times1)=300+225+400=925$$

第二支军队的兵力是：

$$((60\times10)\times1)+((80\times5)\times1)=600+400=1000$$

下一步就是计算各支军队的潜在兵力。我们用于计算潜在兵力的值是军队的领导能力、士气和作战天气变化。第一支军队的领导度是 1.0、士气度是 1.0、作战天气变化度是 1.0。第二支军队的领导度是 0.5（因为它在几次战役中打了败仗），士气度是 0.5，作战天气变化度是 1.0。

第一支军队的潜在兵力是：

$$925\times1.0\times1.0\times1.0=925$$

第二支军队的潜在兵力是：

$$1000\times0.5\times0.5\times1.0=250$$

最终获得一个理论上的成果，我们评估一个潜在兵力比为：

$$\frac{925}{250}=3.7$$

像在 PPR（Power Potential Ratio）中定义的那样，如果结果是 1.0 或者更大，那么第一支军队将是胜利者。我们从这个例子中看到，第一支军队是决定性的胜利者。

3.7.8　局限性

虽然 QJM 理论能够进行精确的战争预测，它还有一些需要改善的地方。因为 QJM 本质上是建立在一套等式上的公式，它不能真正确切计算出某特定时间潜在的兵力损失。相反，朗彻斯特微分系统（Lanchester differential system）允许在特定的时间对数据进行评估，因为它是一个简单的、统一的模型。朗彻斯特系统也一定具有它自己的局限性，在《游戏编程精

Download at www.Pin5i.Com

粹》5[Bolton05]中将讨论该系统及其局限性。

3.7.9 结论

本文介绍了这个独特的战争预测公式 QJM。QJM 不仅可以用来重现历史上的战争，也可以用来预测战争的结果。QJM 也具有即插即用的能力，这可以对一些概念进行集成，如磨损计算或空间效力。QJM 还可以进行修改，也可以被扩展以运用于其他的游戏系统中。

3.7.10 参考文献

[Bolton05], Bolton, John, "Using Lanchester Attrition Models to Predict the Results of Combat." *Game Programming Gems 5,* Charles River Media, 2005.

[Dupuy84] Dupuy, Trevor N., *The Evolution of Weapons and Warfare.* Da Capo, 1984.

[Dupuy85] Dupuy, Trevor N., *Numbers, Predictions and War.* Hero Books, 1985.

[Howard83], Howard, Michael, *On War.* Princeton, 1983.

[Jomini92], Jomini, Baron Antoine Henri, *The Art of War.* Greenhill Books, 1992.

Download at www.Pin5i.Com

3.8 设计一个多层可插拔的 AI 引擎

Sebastien Schertenleib,
瑞士联科理工学院
虚拟现实实验室
S é bastien.SchertenIeib@epfl.ch

这篇文章的目的就是为一个数据驱动的 AI 引擎提供一种架构，它能够控制大批独立的角色，能够实时地对环境和用户的交互做出相应的反应。编码的实际引擎用于一个群众仿真系统。本文将概述一个完全实现了的数据驱动 AI 系统，并指出这个系统的优点和劣势，同时，将它与传统的基于编码的 AI 实现进行比较和对比。

实际上，在任何游戏 AI 系统中，你都可以把 AI 看作是两个分离的区域组成的：

逻辑层，也称为“引擎”层：这是一个基础系统层，它控制在游戏世界中设置的 AI 角色。包括像动画选择、路径寻找、避免障碍物等类似的事情。

数据层：这里描述实际的 AI 行为和在游戏世界中移动和前进所需的决策结构。在这一层，我们将描述这样一些事情，如特定动作的执行、AI 应该游历的地方或者是武器的选择。

目前的许多 AI 游戏系统依然完全依赖于基于编码的实现，或者是“单一结构”。在这种系统中，逻辑层和数据层都要编写代码。数据层有任何的变化，系统就需要重新编译。在开发中，这会导致编程的瓶颈问题，因为程序员不仅需要补充系统，而且还要重建系统，以便能检测这些变化。随着系统复杂程度的提高，单一系统将越加难以管理和效率低下。

为避免这一点，我们必须在这两个 AI 层之间实施一个明确的隔离。这样每一层都能比较容易实现、能够分别地进行完善，两者之间也能够更好地配合。如果我们通过一些配置文件和/或脚本定义的数据层来实施这项隔离，那么这就是众所周知的数据驱动架构（[Grimshaw89]，[Bilas02]）。实验证明，这种架构能提供更新型、更有效率的结果[Shumaker04]。任何数据驱动系统需要考虑的一个基本问题是在数据范围内操纵控制流。在本文中介绍的这个系统是围绕着一个多层系统建立的，着重于使基本层的功能最小化，并考虑到特殊情况，可以使用专用的、插件扩展模块。

3.8.1 相关工作

在过去的几年中，引进可编程的图形编程单元（GPUs），以及用于实

Download at www.Pin5i.Com

时图形应用的可编程阴影程序的出现[Lindholm01]，已经表明只要付出较少的努力，一般游戏的图形质量就可以获得非常大的提高。为了生成这些阴影程序（例如[Cg04]、[GLSL04]、[HLSL05]）而引进更高级的编程语言的作法已经证明，只要直接给开发者提供一种更加强有力的工具[Fernando03]，就可以获得更好的图形质量。通过同样的方法，使用 AI 定义语言中更高级的抽象概念，人工智能的质量和复杂度可以获得进一步的改进。许多研究人员认为用于 AI 加速器的专用硬件即将付诸应用[Funge99]。另一个可能的进步是未来带有多个核心处理器的硬件，这就使得 AI 应用可以拥有一个专用的核心处理器。

这类硬件的进步能够创建可视化的大规模群众模拟，这种模拟会有更好的行为逼真度。在过去几年中，脱机技术已经在商业系统[Koeppel02]、[Moltenbrey04]中得到了精雕细琢。不过现在，研究人员和中间件公司正试图将脱机技术跟交互应用相结合。[Champandard03]已经开发了一种框架[Fear02]，允许使用 XML 配置文件去创建 AI 行为。[Kruszewski05]描述了已开发的中间件[AI Implant00]以及它与下一代游戏平台的结合。与[Bilas02]中描述类似的数据驱动结构，已经用于商业产品的开发。

3.8.2 AI 引擎架构

1. 设计原则

该系统包括多个抽象层，通过一致的插件 API 接口，可以把低级功能（例如动画，物理，指导等）和高级功能（如战术意识、认识、记忆等）与每一个代理连接起来。通过把核心 AI 引擎与其他插件模块隔离开来，我们可以在不同项目中重复使用这个核心引擎。事实上，每一个插件模块都有自己的一套约束条件和规则。设计出的模块可以被用于速度、内存性能、高保真渲染、或者是你需要的任何方面。就是因为这种灵活性，基于当前项目的需要（起因于许可证费、可扩展性、平台等），我们就有更多的自由度去选择适当的模块。从而使用同一个核心 AI 功能就可以开发各种不同的实时应用。

2. 系统概况

处理一个有百余个具有高度可塑性的、分布式的实体的群众模拟，需要不断地修改和调整其行为规则，这样才能产生令人信服的结果[Sung04]。每个代理必须能够完成不同的任务：例如行走或与环境对象交互。使用一种更直观的方法定义代理的行为，你必须使用一种更高级的抽象并把细节委托给低层功能完成[Fairclough01]。在我们这个系统中，通过以下的两种机制是可以实现的：充分利用一种脚本语言通过为任务分优先级的方法，以及稍后将在文中解释层次化的模糊状态机（FuSMs）来管理行为规则。

3. 行为引擎

这个系统架构依赖于 C++模板，就像在[Abrahams04]中描述的那样，使用元编程(meta-programming)技术。这个系统中的每一个实体都是从一个公共的核心模板类中派生来的。通过资产设计模型[Gamma95]，这些实体可以借助继承性和授权组合起来。大部分的实体共享类似的属性，并且是从一个动态属性（例如指导行为等）和一个物理属性（例如块头、重力等东西）中派生的。引擎在实体的独立特性之上进行工作，使用共同的接口可以使插件功能更加

Download at www.Pin5i.Com

方便。程序清单 3.8.1 显示了一个典型的接口代码的片段，在这个例子是一个 IAnimation 类。

程序清单 3.8.1　动画可插拔的接口片段

```
Class IAnimation : public IPlugIn
{
    void defaultPosture() = 0;
    //key_frame
    kfActivate(const std::string& keyframeName)=0;
    kfSetTimeScale(vhtReal rTimeScale) =0;
    // ...
    // Walk Animation
    void walkSetDesiredFrontalSpeed(const vhtReal& rSpeed) =0;
    void walkSetPositionReachedTolerance(const vhtReal& rValue) =0;
    void walkSetDesiredPosition( vhdVector3 vecPos) =0;
    // ...
    //look
    void lookSetTargetLocation(const vhdVetor3 & vecPos) =0;
    void lookSetEyeBlinkParameters(vhtReal rCloseEyeAngle,
        vhtReal rPeriod,
        vhtReal rVariance)=0;
    // ...
    // other actions
    // ...
};
```

当启动一个新行为目标时，请求从系统的最高级开始。例如，动画模块可能预示需要把手臂移动到一个特定的位置。在这个例子中的“动画模块”是一个高级的公共接口。实际工作是由动画系统中底层的插件实施过程来完成，我们称之为“具体实施。”使用这种方法，AI 引擎不是绑定于一个专用动画或者物理包。每个包可以自由执行它认为合适的公共接口请求。一个行走动画的计算可以使用关键帧动画或使用基于物理的动态行走。图 3.8.1 显示这个框架的基本情况，清楚地展示了 AI 引擎与各个插件实施过程之间的相互独立性。每个实体有自己的私有行为规则编码，还可访问它的缺省动作，但不能直接控制它的实例化对象。

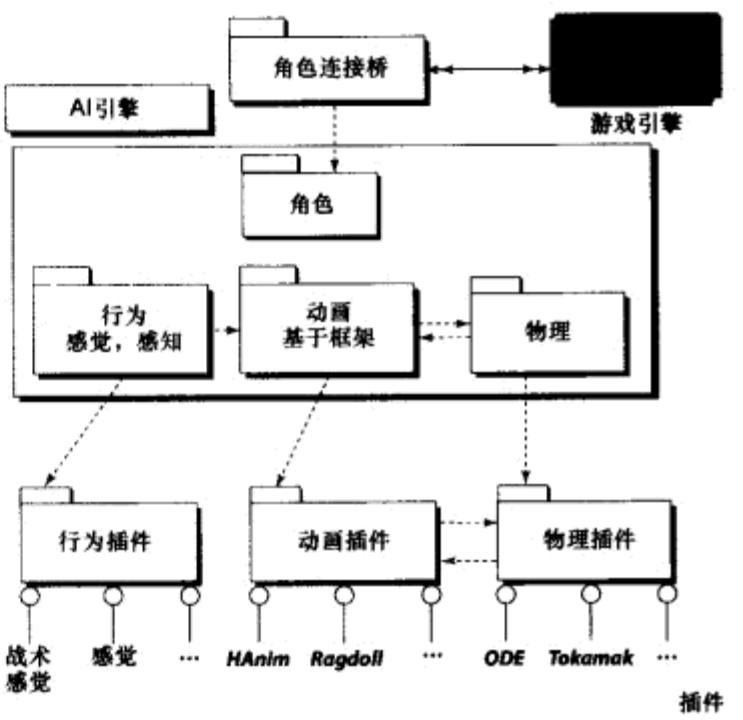


图 3.8.1　特性等级和具体的实例实体

3.8.3　数据驱动类和属性

在这个引擎的设计过程中，更新每个代理的状态所需要的信息不能依赖数据本身。这些不同的抽象层（见图 3.8.2）保证了这个隔离。

- 用 Lua（一种编程语言）中的元表（metatables）定义[Ierusalimschy96]行为规则和模糊状态机（FuSM）的状态。

Download at www.Pin5i.Com

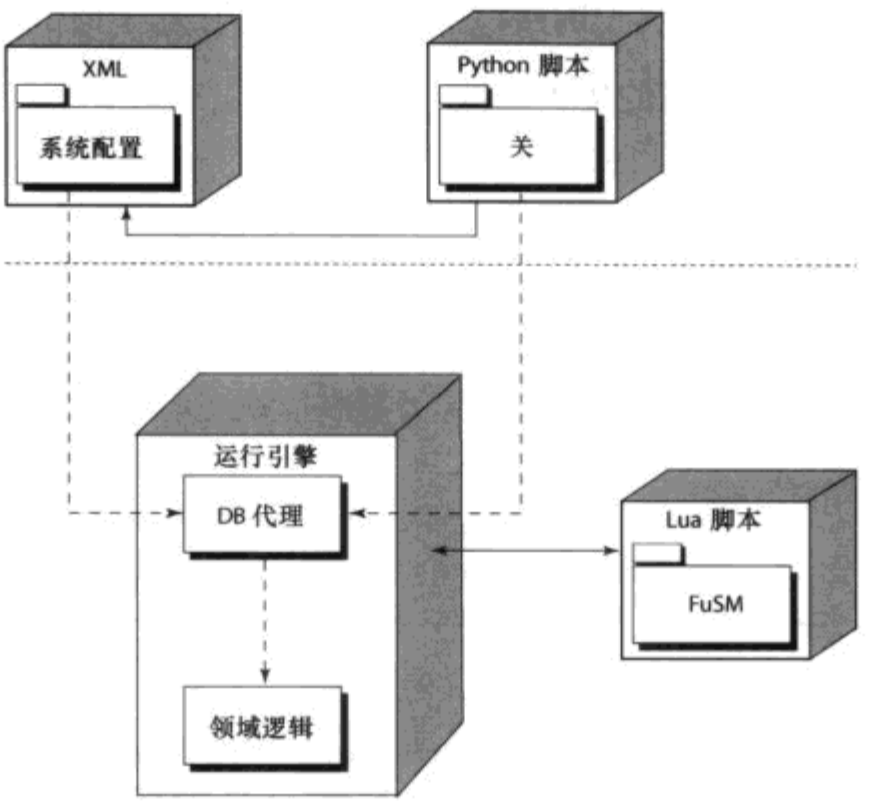


图 3.8.2 多结构系统概括

- 通过 Python（一种编程语言）的微线程（[Hoffert98]，[Python91]）管理事件的传播和游戏世界的相互交互。
- 用 XML 配置文件设定对这个游戏世界的初始描述。

在每一个抽象层，我们利用每种语言独特特点。逻辑层是用 C++编写的，数据操纵是用脚本语言实现的。决定使用两种不同的脚本语言，是基于以下几个原因：首先，一般而言，Lua 是比 Python 更快的一种解释型的语言[Bagley05]。所以我们使用 Lua 操纵和更新每个实体的不同的状态机；因此，在保持一种数据驱动方法时效率还能保持在一定的水平。然而 Python 给我们带来更多可以利用的第三方组件（尤其是数据库和网络管理等）。

虽然，编写这个系统的各种脚本语言之间互不干扰，要想在 Python 和 Lua 之间进行直接地交流，可能的方法是使用双向桥解决方案[Niemeyer03]。

1. 层次化的 FuSM：模糊状态机

FuSMs 是常规有限状态机的一个组合，采用模糊逻辑规则来制定决策，而不是采用布尔逻辑[Kantrowitz97]。因此，模糊状态并不限于开或关，它可以容纳一个中间值。不像有限状态机，在一定程度上，一个模糊状态系统可以具有全部或任何状态。然而这种精细的本质使得 FuSMs 的创建比 vanilla FSM 的创建更加复杂。同时存在的激活状态会大大降低不可能产生的行为的预测性。它也会大大地降低状态机的复杂度，可以用更少的状态来编制出更广范围的不同状态。

本例中，在 Lua 元表（metalables）中定义了应用需要的 FuSM 状态和子状态。在 Lua 中，一个元表就是一个普通的 Lua 表，在特定的操作中，需要使用附加说明去规范其行为（以及如何利用用户提供的数据）。在这个例子中，将涉及为专用状态提供特定的规则，允许采用基于游戏特定情节的规则（例如，不同的环境，实体的数目等），并为访问 C++角色实体的数据结构提供参考。

Download at www.Pin5i.Com

如程序清单 3.8.2 所显示的那样，每个状态描述需要提供 4 个关键字：

姓名：仅仅是状态的名字。

进入：调用进入到这个状态的实体功能。

执行：状态的更新功能。

退出：在退出特定状态的时候被调用。

程序清单 3.8.2 在 Lua 元表中描述的一个 FuSM 状态

```
-- create the State_PathBehaviorActivateGoal state
State_PathBehaviorActivateGoal = {}
State_PathBehaviorActivateGoal["Name"] = "PathBehaviorActivateGoal"
State_PathBehaviorActivateGoal["Enter"] = function(character)
    -- nop
end
State_PathBehaviorActivateGoal["Execute"] = function(character)
    character:walkTo(character:getDesiredPosition(), true)
    character:activateWalk()
    character:getPath():setNextWaypoint()
    if character:getPath():finished() then
        --done
        character:getFSM():changeState(State_PathBehaviorDone)
    else
        character:getFSM():
            changeState(State_PathBehaviorSelectGoal)
    end
end
State_PathBehaviorActivateGoal["Exit"] = function(character)
    -- nop
end
```

2. C++和 Lua 元表（metatable）的联编

核心 C++引擎包含一个 Lua 解释器，并通过使用[Luabind03]类库产生一个“胶水”代码（这个术语是指管理在 C++和 Lua 之间的变量和/或类联编所需要的编码）来展现它的功能。Luabind 与 Boost 类库的元编程（meta-programming）方法有许多共同点[Boost98]，因为它提供了一套扩展 Lua 脚本语言的函数，使其具有面向对象的机制。比如允许 Lua 类可以从其他的 Lua 类或 C++类中派生。因为 Luabind 使用元编程，像[Manzur03]中的一些流行工具，它不需要预处理程序的编译过程。编译器简单地自动产生胶水代码。然而，联编代码的编译过程比较慢。战斗中的一个策略就是把联编代码保存在独立的对象文件中，而不是存放在一个巨大的联编库中。

程序清单 3.8.3 表示了在 Luabind 库中，被 Lua 调用的一个 C++类的例子。

程序清单 3.8.3 Luabind 联编

```
void iCharacter::registerWithLua(lua_State* pLua)
{
    //register the class into the module vhdEl
    luabind::module("vhdEl", pLua)
    [
       //indicate that the resulting Lua class
```

Download at www.Pin5i.com

```
        //inherit from the exported base class
        .def(luabind::class_<iCharacter,
            luabind::bases<vhdEl::iBaseEntity> > ("iCharacter")
        .def(luabind::constructor<const std::string&>),
        .def("setWalkStyle" , &vhdEl::iCharacter::setWalkStyle),
        .def("setWalkSpeed" , &vhdEl::iCharacter::setWalkSpeed),
        .def("lookAt" , &vhdEl::iCharacter::lookAt),
        //...
        .def("performSensing" , &vhdEl::iCharacter::performSensing)
    ];
}
```

就像在清单 3.8.3 中看到的那样，引擎的许多方面可以被脚本语言调用。在这个系统，Lua 脚本中描绘了驱动动画和实体反应的行为规则和 FuSMs。每个实体的状态机保持一个内部的 luabind::object 加速器。它联系 Lua 对象和对应的 C++对象。这个联系变量给用户提供了对引擎代码和 Lua 堆栈之间交互的一个直接连接。由于每个状态包含自己的联编加速器，在访问时间方面，系统不受 FuSM 的规模和复杂度的影响。引擎不需要了解状态本身就能够执行和更新代理的状态。程序清单 3.8.4 显示了引擎中的状态执行控制逻辑的一个小片段。

程序清单 3.8.4 用于更新激活状态机的 C++方法

```
void FSM::update()
{
    //luabind object point to the current Lua metatable
    if (_luaCurrentState.is_valid())
    {
        //_pOwner: pointer to the entity
        _luaCurrentState.at("Execute") (_pOwner)
    }
}

void FSM::change(const luabind::object& newState)
{
    //call exit on the active state
    _luaCurrentSate.at("Exit")(_pOwner);

    //change state
    _luaPreviousState = _luaCurrentState;
    _luaCurrentState = newState;

    //call the entry method for the new state
    _luaCurrentState.at("Enter")(_pOwner);
}
```

3.8.4 分优先级的任务管理器

有多种方法可以把使用脚本语言所导致的性能影响减少到最小。一种方法是利用一个管理器处理 FuSM 系统的工作流，在状态的种类中分配任务，从而避免对不必要的任务的更新。然而，由于引擎不和数据直接相连，为了更加方便，设计者必须用有关种类信息来标注不同的状态。

Download at www.Pin5i.Com

AI 结构利用划分了优先级的任务

为了保证实时的高性能，任何 AI 系统需要在预算的时间帧内完成工作。为了适应可分配到 CPU 时间，需要限制系统行为的数量和/或复杂度[Funkhouse93]。这需要保持 AI 系统对资源需求的相对稳定性，这样游戏的播放就不会因为峰值时刻 CPU 的需求大，导致帧速不连续[Schertenleib02]。通过提供一种高级方法使设计者能在这个系统中组织不同的任务，我们就能将把每个任务的分类从引擎中分离出来。

在特定的帧内，所有任务都可以被激活。我们需要分散它们的调度，这样我们才能确保每帧工作流能保持一定的常量。为了实现这一点，我们把这些任务划分为三类：一类是在每一帧中都需要更新；一类是可以进行定期更新；最后一类是只有在处理时间允许的情况下，才能被执行。

你可以因为不同的原因定期检查特定的任务。在检查中，如果一个关键帧的动画已经完成，你可以使用一个周期性的优先级，因为在一帧一帧的基础上它是不可能改变的。对某些任务进行周期性检查的另外一个原因是：在任务传送之前引进一定时间的延迟。这种延迟可以通过避免实体直接对每个刺激做出反应，增强 AI 效果[Laird01]。要使需要每帧更新的任务数目最小，因为这是从 AI 执行预算总时间中直接减去的。

通过对任务分类，我们才能确保引擎能获得最大效率。我们使用可配置的搜索规则（根据任务的类别来定制）帮助我们在每个行为库中直接查找。我们可以保证越重要的任务会首先出现，而比较早的任务并不被踢出这个系统。图 3.8.3 显示了任务调度的简单概述。

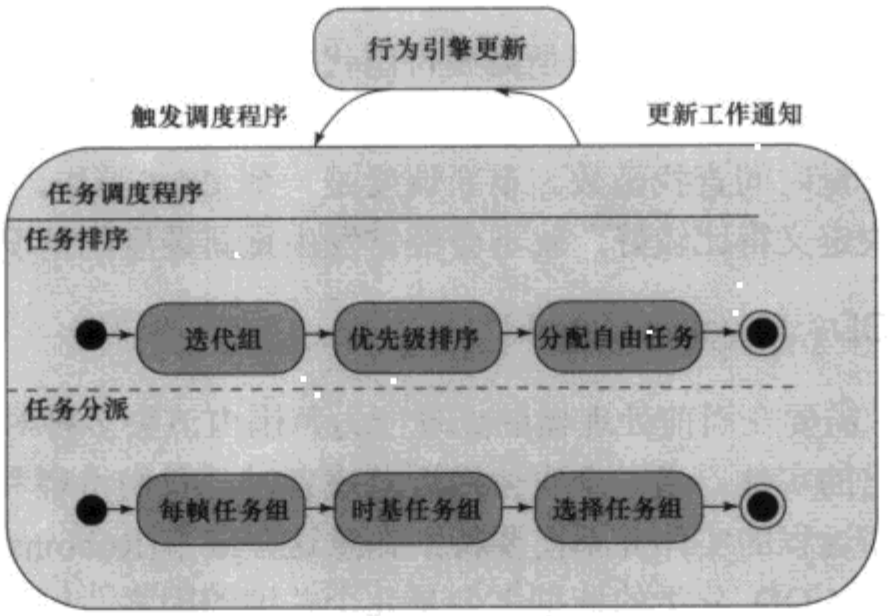


图 3.8.3　任务调度管理

3.8.5　性能问题和技术

在本文所描述的系统中，可能会出现一些性能问题。但是如果比较小心的话，它们会被妥善地处理，甚至可以完全避免。这里我们将讨论处理这些问题的一些方法和技术。

1. 概况

困扰数据驱动系统的一个共同问题就是系统的性能，因为执行脚本程序总是比直接执行

Download at www.Pin5i.Com

代码要慢。通过基于优先级的调度，所有数据驱动引擎必须避免不必要的状态最新，并要限定在脚本级进行处理的一些工作。繁重的浮点操作或大型查询行动应该保持在代码级，通常仅仅在适当的情况下，才用脚本来设置参数。使用这种方法限制脚本，将有益于系统的数据驱动，可以使其从沉重负担解脱出来。

这里描述的系统，储存在C++对象中的信息势必需要脚本接口。任务的分类是一个棘手的问题。因为引擎并不控制任务的复杂度和持续时间，设计者需要为系统提供线索，就像在程序清单3.8.5中所描述的那样。为每个任务收集信息需要一个常规的描述，如在任务执行期间的资源使用情况。依靠这些描绘的结果，任务可以分配到合适的分类组中去。

例如，在执行中变化较小的任务是常量的一个好的替代方法，基于帧率的执行（使用最长持续时间来支持最大基于时间的种类）。对基于试探函数的每个LOD损失进行预测评估，这样用于每个任务管理的不同细节等级（LODs）将会从中受益。

程序清单3.8.5 任务分类可能会不同的LODs

```
State_DoAction["Enter"] = function(character)
    -- hints for classifying tasks
    character.lod[0] = MaxTimeTask(0.01)
    character.lod[1] = MaxTimeTask(0.002)
    -- constant time
    character.lod[2] = ConstantTimeTask(0.0001)
end
```

2. 任务排序

对所有不同的任务进行管理和排序是需要付出代价的。在这个系统中，调度任务可以使用一个基于时间的优先级相关表。为了减少复杂度，我们把每一个任务分成小块。每块将按顺序执行，提供一个常量时间查找函数。重新调度是一个$O(m)$操作，这里的m是这个块中的任务数。如果这些块定义得比较好，就不会经常发生重新调度的情况。

3. 细节等级（LOD，Level of Detail）

和图形引擎类似，需要在当前处理器的处理能力范围内渲染多种对象，AI也必须提供不同等级的抽象来控制它的实体。当一个实体接近游戏者时，我们希望其行为能够比较敏捷而丰富。然而，屏幕外和远程的实体并不需要特别在意这些细节[Robbins03]。

对给定的任务进行LOD分类的依据主要是几个关键的因素，例如整体资源使用率。一个图形的LOD系统，根据距离相机的远近程度就可以选择当前的LOD。声音和行为LOD系统可能会需要更多的专门的可供选择的试探程序。一个AI LOD会考虑执行任务的计算复杂度。在我们这个系统中，动画可以是基于骨骼的动画或者使用系统预先录入的姿态。前者的计算可以基于在这个动画中的骨骼数目。而后者则是一个固定时间的任务。因此，就像在清单3.8.5所显示的那样，系统能基于任务当前的LOD为可用的任务提供更准确的调度。

为工作流排序，需要把任务分成不同的种类。一类任务是需要在一帧一帧的基础上更新。另一类任务是周期性的，还有一类是只有获得处理时间的时候才能被执行。最后一类任务的典型例子与这个世界中居住的虚拟角色有关，但他们不会同游戏者有直接的交互。因此，如果这样的一个角色正在执行空闲动画，并且需要更新其行为，这种任务就很容易被延迟一些

Download at www.Pin5i.Com

帧，而这种延迟游戏者也许不会太在意。在一定程度上，一些任务可以利用引擎收集的一些统计信息，以便在运行过程中被重新分配到更加合适的类中去（例如，特别是周期性的任务）。

3.8.6　工具

随着当今系统复杂度的增加，仅仅开发一个库是不够的。还需要努力去创建一些专用的工具，这些工具将会发挥库的潜力。手工编写成千上万的状态可能会是一个非常痛苦的过程。因此，在图形环境中，FuSMs 的出现可能对生产效率起到重大的推动作用。

1. 脱机处理过程

图 3.8.4 显示了一个用于设置状态转变的定制化的工具。然后把数据直接导到相关的脚本。像在[Darovsky05]或[Jacobs05]中倡导的那样，开放源代码可能是引起读者兴趣的一个很好的出发点。

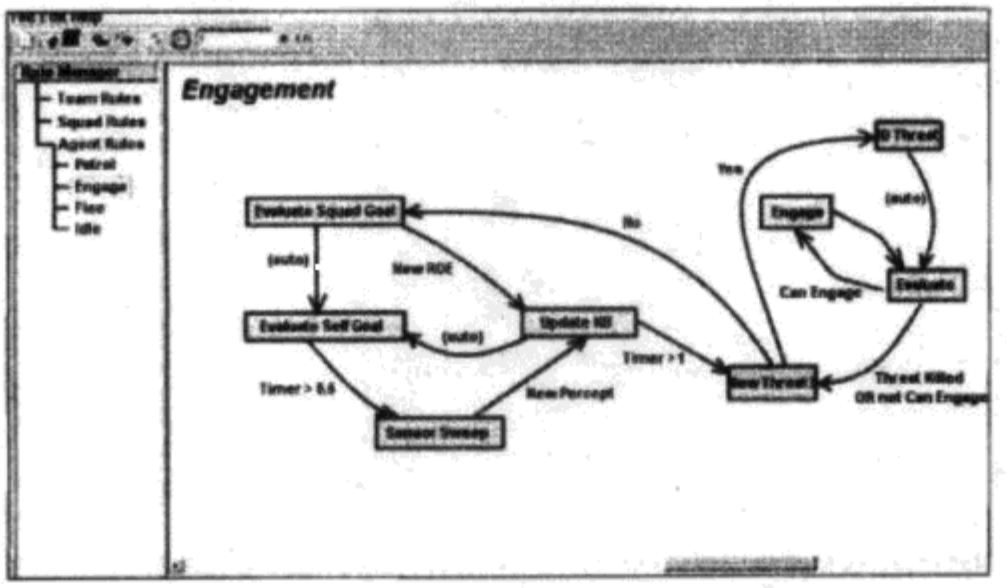


图 3.8.4　用于产生 FuSM 的设计者工具

在图 3.8.4 所显示的 FuSM 的编辑器，其核心思想是把信息储存到 XML 配置文件中，然后一个编译器在 Lua 元表中生成不同的 FuSMs。在中间文件中使用 XML 编码的原因是要使创建工具从引擎数据结构中分离出来。图 3.8.5 描绘了这个处理过程，显示运行引擎将把 Lua 脚本转换成 Lua 字节编码，从而在运行时可以用被 AI 引擎使用。

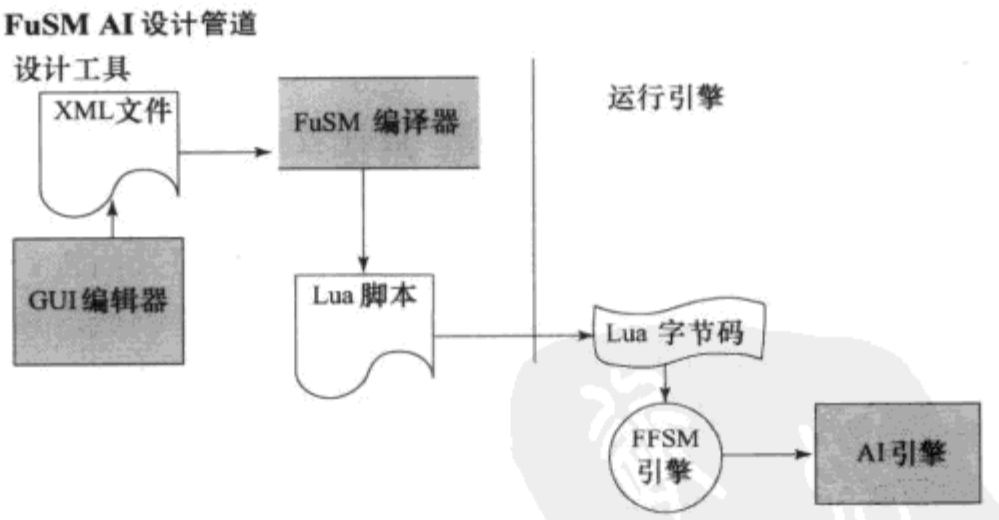


图 3.8.5　用于从 GUI 编译器到运行引擎产生不同 FuSM 的管道

Download at www.Pin5i.Com

2. 运行信息

开发由上百或更多的动态实体（例如人群模拟）组成的 3D 实时应用是一个非常艰难的过程。为了创建一个可信的世界，设计师需要与虚拟环境进行直接交互。仅看屏幕上角色的一般动作是不够的。还需要在运行时修改实体的模拟参数。这个想法的目的就是要产生动态的图形用户界面，它能根据设计者的需要反映当前的数据集。图 3.8.6 就显示了这样的一个图形用户界面的例子。根据路径规划的需要，它为当前的导航图提供了可视的调试信息，以及一套小装置（屏幕菜单）显示控制模拟的反馈信息。用户可以直接与系统、触发事件进行交互，或动态地改变系统的参数。

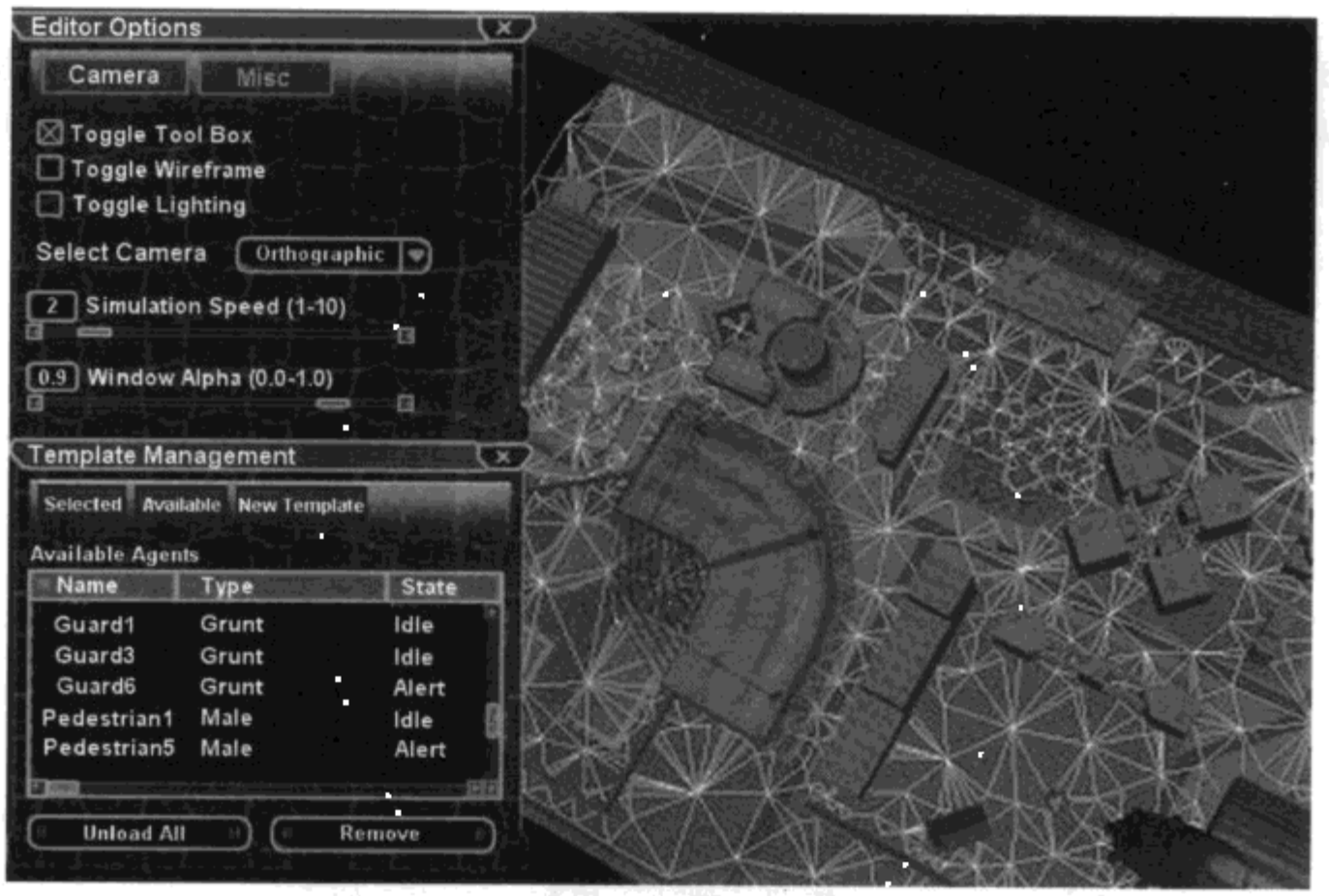


图 3.8.6 用于控制实体状态演变的实时 GUI

3.8.7 结论

本文展示了一个数据驱动 AI 引擎的设计结构。该系统性能能够处理大规模的人群模拟，这完全是通过它的数据来控制的。

然而，这个系统还存在着一些局限性。首先，设计者必须帮助系统按照优先级对不同的任务进行排序。第二，尤其是在不同程序和脚本语言进行调试的时候，多种语言的开发环境更增加了系统的复杂程度。然而，插件接口给系统提供了更大的灵活性，尽管存在着少许的性能影响，但设计者能够重复使用核心引擎。最近，在图像编程领域取得了一些进展，提供了更好的工具和更高级的语言，这很清楚地表明了数据驱动系统的好处。在 AI 领域必须达到一定的标准化水平，这样才能使设计师关注于系统的行为方面，而不是在执行方面。这篇论文正是迈向这个目标的一个尝试，但是还需要利用更加有效的工具去做更多工作来扩展这个系统，并在数据生成处理流程方面付出更多努力。

Download at www.Pin5i.Com

3.8.8 参考文献

[Abrahams04] Abrahams, D., *C++ Template Metaprogramming: Concepts, Tools, and Techniques from Boost and Beyond.* A. Gurtovoy, 2004.

[AI Implant00] A.I.IMPLANT, "Middleware from BioGraphic Technologies for realtime crowd simulation." BioGraphic Technologies, 2000.

[Bagley05] Bagley, D., "The Great Computer Language Shootout Benchmarks." 2005. Available online at *http://shootout.alioth.debian.org.*

[Bilas02] Bilas, S., "A Data-Driven Game Object System." *Gas Powered Games,* 2002.

[Boost98] Boost, "Boost C++ Libraries." 1998.

[Cg04] Cg, "The Cg Shader Programming Language." nVIDIA, 2004.

[Champandard03] Chapandard, A., *AI Game Development, Synthetic Creatures with Learning and Reactive Behaviors.* New Rider, 2003.

[Darovsky05] Darovsky, A., FSME, 2005.

[Fairclough01] Fairclough, C., "Research Directions for AI in Computer Games." Technical Report TCD-CS-2001-29, Trinity College, Dublin, 2001.

[Fear02] Alex Champandard, "Fear Flexible Embodied Animat Architecture." 2002.

[Fernando03] Fernando, Randima and Mark J. Kilgard, *The Cg Tutorial.* Addison-Wesley, 2003.

[Funge99] Funge, J. D., *AI for Games and Animation: a Cognitive Modeling Approach.* A K Peters, 1999.

[Funkhouser93] Funkhouser, T. A., "Adaptive Display Algorithm for Interactive Frame Rates During Visualization of Complex Virtual Environment." C. H. Séquin, University of California at Berkeley, 1993.

[Gamma95] Gamma, E., et. al., *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley Professional, 1995.

[GLSL04] GLSL, "The Shader Programming Language for OpenGL," 2004.

[Grimshaw89] Grimshaw, A. S., "Real-Time Mentat: A Data-Driven Object-Oriented System." Proc. IEEE Globecom: pp. 232–241, 1989.

[HLSL05] HLSL, *The Microsoft DirectX High-Level Shader Programming Language.* Microsoft, 2005.

[Hoffert98] Hoffert, J. and K. Goldman, "Microthread: An Object for Behavioral Pattern for Managing Object Execution." Washington University, Distributed Programming Environments Group, 1998.

[Ierusalimschy96] Ierusalimschy, R., "Lua-an Extensible Extension Language." Software: Practice & Experience, Vol. 26, John Wiley & Sons, 1996.

[Jacobs05] Jacobs, S., "Visual Design of State Machine." *Game Programming Gems 5:* pp. 169–176, Charles River Media, 2005.

Download at www.Pin5i.Com

[Kantrowitz97] Kantrowitz, M., "Fuzzy Logic and Fuzzy Expert Systems." 1997. Available online at *http://www.faqs.org/faqs/fuzzy-logic/part1/*.

[Koeppel02] Koeppel, F., "Massive Attack." Popular Science, 2002. Available online at *http://www.popsci.com/popsci/science/d726359b9fa84010vgnvcm1000004eecbccdrcrd/ 5.html*.

[Kruszewski05] Kruszewski, P. A., "A practical system for real-time crowd simulation on current and next-generation gaming platforms." 2005.

[Laird01] Laird, J., "Toward human-level AI for computer games." Invited presentation, AAAI-2000. Available online at *http://ai.eecs.umich.edu/people/laird/talks/Soar-games/index.htm*.

[Lindholm01] Lindholm, E. and M. J. Kilgard, "A User Programmable Vertex Engine." *Proceedings of SIGGRAPH 2001,* ACM Press/ACM SIGGPRAH: pp. 149–158, 2001.

[Luabind03] Luabind libraries for use with the Lua language, 2003.

[Manzur03] Manzur, A., *toLua++*. 2003.

[Moltenbrey04] Moltenbrey, K., "Digital artists re-create ancient Rome for the epic miniseries Spartacus." *Computer Graphics World,* 2004.

[Niemeyer03] Niemeyer, G. *Lunatic Python.* 2003.

[Python91] *Python Programming Language.* 1991.

[Robbins03] Robbins, J., "Simulation Level of Detail Or Doing As Little Work As Possible." Physically Based Modeling, Simulation, and Animation, 2003.

[Schertenleib02] Schertenleib, "Complex 3D Environments System Rendering and Consistent Frame Rate." 2002.

[Shumaker04] Shumaker, S., "Techniques and Strategies for Data-Driven Design in Game Development." Computer and Information Science, 2004.

[Sung04] Sung, M. and M. Gleicher, "Scalable behaviors for crowd simulation." *Eurographics* 23, 2004.

Download at www.Pin5i.Com

3.9 一个管理场景复杂度的模糊控制方法

Gabriyel Wong,
南洋理工大学
gabriyel@gamail.com
Jialiang Wang,
南洋理工大学

众所周知，游戏应该在预期的渲染负载范围内进行设计。通常，3D 艺术家和与实际的游戏引擎直接相联系的环境细节编辑者一起工作，这使得他们可以提前预览他们的工作，并能检查他们开发的内容是否可以在交互帧速率上运行。而这已成为一种规范，因为单个用户使用一种预定的方法操纵游戏玩法。但日益明显的是动态的环境（例如多游戏玩家的游戏方案）正在挑战这一惯例。在这篇文章中，我们描述一个管理场景复杂度的模糊控制方法，并叙述该方法的性能和图像质量，以及它在动态游戏环境中一些应用。

3.9.1 关键思想

给定一个固定的硬件资源集，实时计算机图形的目标就是在交互帧速率（性能）的范围内要最大化视觉敏锐度（质量）。但现有存在的机制：例如细节等级（LOD）的控制、能见度算法和基于图像的技术，这些技术都力求降低几何复杂度，不可否认的是它们的性能随着场景内容的不同而改变，但在处理整个场景的几何负载控制的时候，它们是目标统一的。

这篇文章并不是描述一个新的降低几何复杂度的算法或试探法。而是说明了管理场景的几何负载的一种新颖方法：使用模糊-逻辑原则去选择 LOD。将渲染过程作为一个“车间”来进行处理，就有可能创建一个基于模糊逻辑的控制器，送到这个车间进行渲染的负载由这个控制器来调节。这个控制器带有反馈机制，可以使用预先建立的规则调整几何负载，这些从开发者的知识中衍生出来的规则是用来有效地选择场景对象的 LOD，负载可以随着性能质量要求的改变而改变。

3.9.2 为什么使用模糊控制?

在控制渲染过程中，采用模糊逻辑[FLDE]有几个好处。首先，与系统控制的常规方法相比，模糊逻辑提供了一种更加快捷和更加方便的设计方法。众所周知渲染过程是非常复杂的，因此，对它进行确切完整的建模是非常重要的。模糊技术允许输入/输出规则建立在专业知识的基础之上，而不是使用那些冗长

Download at www.Pin5i.Com

的或者不可能求出的数学模型。实施过程可以简化为 IF-AND-THEN 语句，这个语句控制着输入/输出的关系。此外，它的语言特性为开发者的设计和用户个性化定制提供了一个简单的平台。因为规则是从语言描述中而不是从数学公式中衍生的。此外，通过开发者对系统的直观了解，模糊逻辑提供了一种可供选择的方法去描述系统中的非线形。因此，上述的好处使得模糊逻辑很有吸引力，可以选择这种模糊逻辑来作为控制一个复杂过程的基础，例如一个实时渲染过程。

3.9.3 工具

有许多创建模糊推理系统（FISs，Fuzzy Inference Systems）的工具，其中包括 Math Works 中的模糊逻辑工具箱，INFORM 公司的 fuzzyTech，Aptronix 公司的 FIDE™（模糊推理开发环境）。同样，开发 FIS 的工作流也常涉及所有的工具。它基本上涉及了输入变量、输出变量的标识和规则集的创建。为了模糊推理系统的工作，需要模糊化控制器的输入和输出，通过映射语言描述（模糊集）成简单的数学范围（如非常低为 0，非常高为 5 等）。上述工具通常提供各种类型的成员函数，帮助输入/输出值映射到它们相应的语言描述中去。图 3.9.1 说明了输入和输出的成员函数的概念。

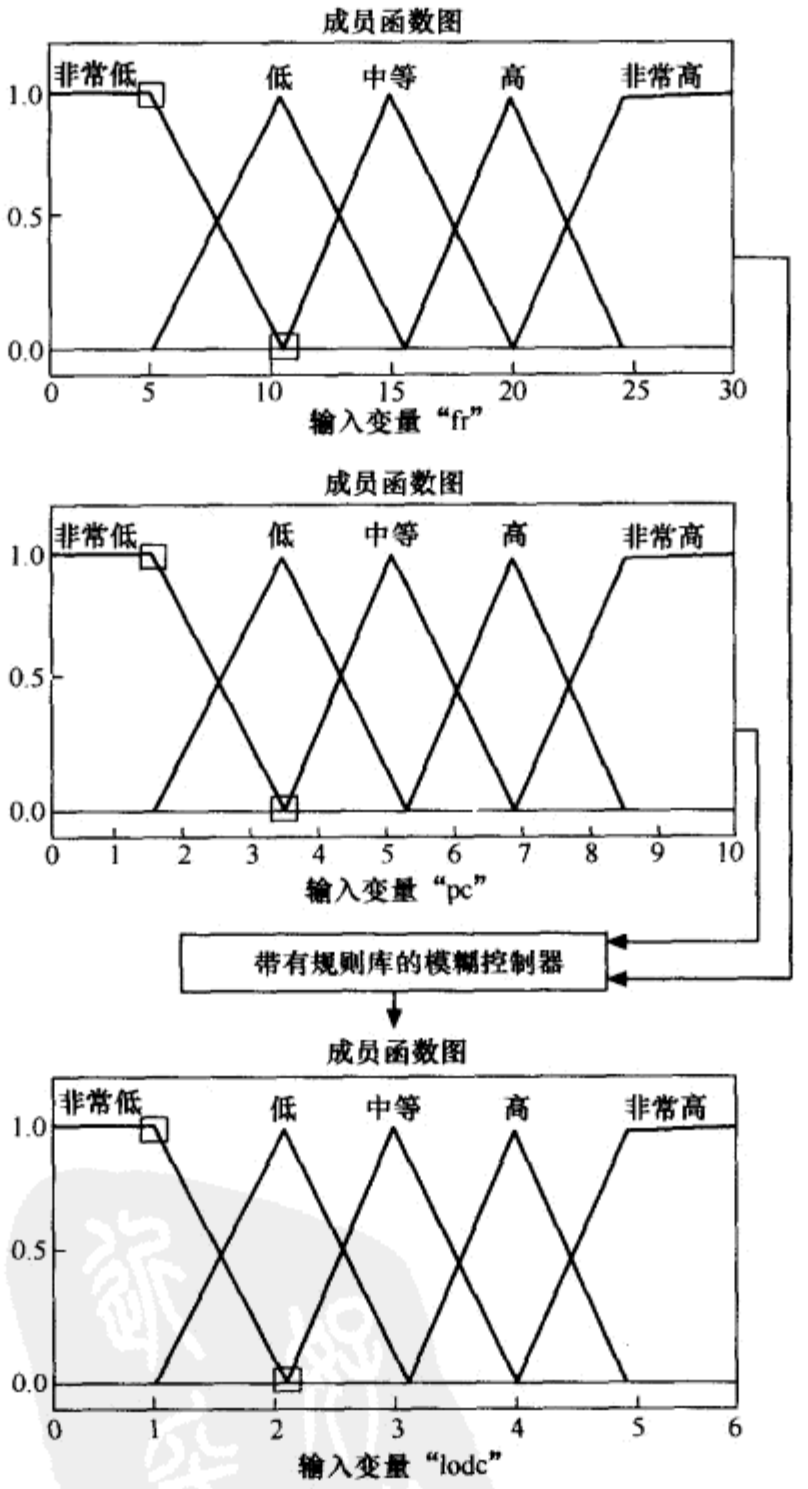


图 3.9.1 FIS 系统的输入输出成员函数

Download at www.Pin5i.Com

基于开发者学识的规则集基本上描述了 FIS 的先行条件（IF 条件）和结果（THEN 输出）之间的关系。同样地，一个好的 FIS 设计工具会提供一个编辑器，这个编辑器可以很方便地创建和修改规则集（见图 3.9.2）。必须指出的是，详细说明了输入/输出参数和规则集并不表明模糊推理系统的开发已经完成。相反，还需要在游戏应用中，对模糊推理系统进行测试，并反复进行调整试验以确保它的有效性和实用性。

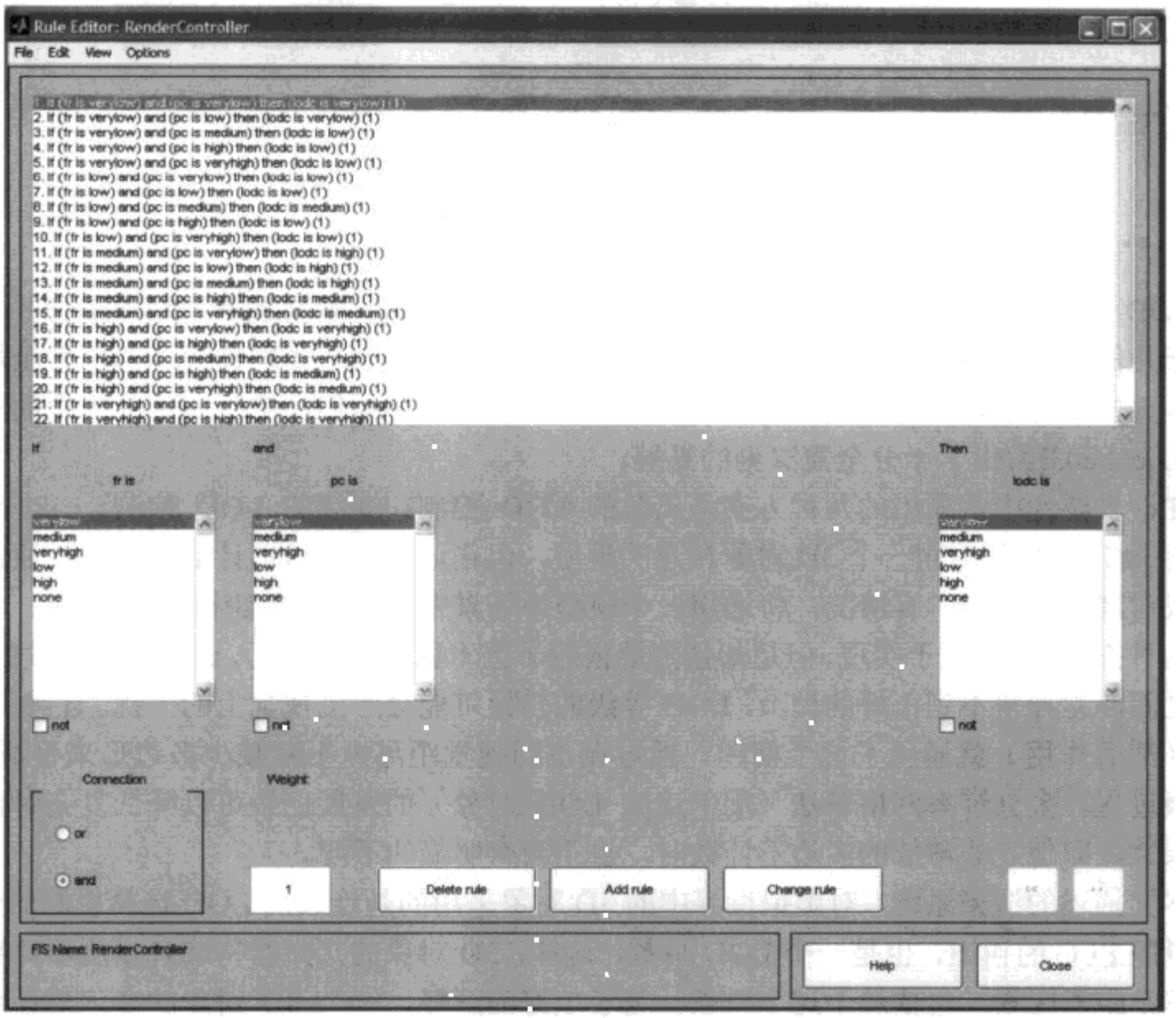


图 3.9.2　管理 FIS 的输入输出关系的一个规则集例子

3.9.4　系统设计

图 3.9.3 说明了一个带有模糊逻辑控制器（FLC）的系统概述。该系统包含一个从车间（渲染过程）到模糊逻辑控制器的反馈分支。选择场景帧率和多边形计数作为反馈信息反馈到 FLC。可配置性是设计这样的一个系统需要说明的重要问题。用户偏爱的帧率和多边形计数可以设置为应用程序行为的参考。这些反馈信息与用户设置的目标值相比，错误被修改后输入到模糊逻辑控制器。

模糊逻辑控制器可以被看作是一个黑盒子，根据这些输入，通过规则集来计算输出。随后，这个输出被传送到车间作为选择 LOD 的一个控制变量，它最终会对发送到渲染处理流水线的多边形负载进行调节。

Download at www.Pin5i.Com

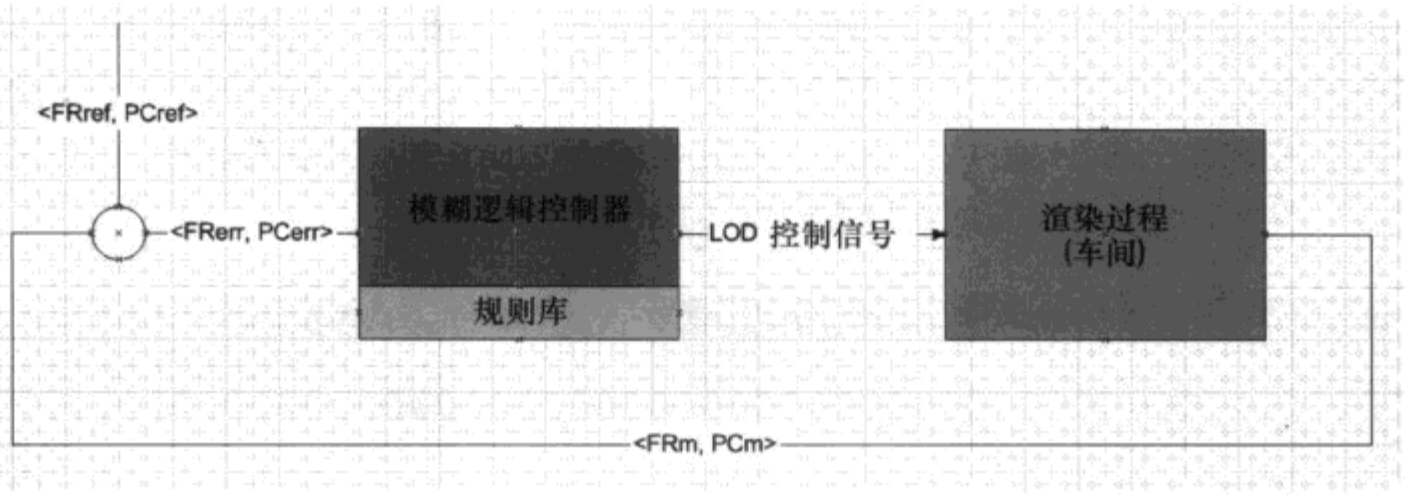


图 3.9.3 代有逻辑控制器的渲染系统

LOD 是使 FLC 能够控制渲染系统几何负载的基本机制。模糊逻辑控制器本身并不是一种降低几何复杂度的算法或者试探法，FLC 是依赖 LOD 机制达到同样的目标。但全局目标也就是要控制整个场景的几何负载。对各种 LOD 技术以及他们在实时图形应用中的相关问题，[Luebke03]给出了十分全面深刻的见解。

在许多游戏中常采用的两种方案是离散的 LOD 控制和连续的 LOD 控制。一个离散的 LOD 控制方案是利用同一个 3D 对象预建的模型，但在运行时多边形计数会变化。通常基于距离或对屏幕空间的占有情况，将选用一个模型为场景中的 3D 对象建模。

这种方法简单、易于实现，但是却强烈地依赖于艺术家的创作技巧。因为如果连续的 LOD 等级之间的差异得不到很好的调节，LOD 等级的转换可能会产生视觉上的干扰。连续的 LOD 方案（滞后作用）就避免了这个问题，通过在不同视觉距离内不断减少多边形来平滑 LOD 的转换过程。多分辨率网格算法（用于连续 LOD 对象）的发展已经可以降低复杂对象的几何复杂性，即使是非常低的多边形计数时，也不会有明显的不同。

本文描述的渲染系统，对虚拟世界中的 3D 对象采用的是连续的 LOD 选择机制。作为控制变量的 FLC 的输出，也是一个 LOD 乘数，这样，3D 对象的几何结构或者多边形计数不仅仅依赖于距离因素，也依赖于这个乘数。为了更好的解释，一个 3D 对象的多边形计数的下降可能相对于距离是线性变化的。通过把源于 FLC 输出的 LOD 因子包括进来作为一个乘数，这个下降就可能呈指数性的变化。因此，根据应用的需要使用 LOD 控制变量，开发者能够很容易地创建许多不同的下降模式。以下代码说明这是如何执行的：

```
void setCLODFalloff(float fallOffMultiplier)
{
    // For each mesh node in the scene
    {
        // Start doing some camera-object distance calculation

        // Compute near distance as ratio of viewing range
        float fRatio = (fDepth-fNear)/(fFar-fNear);

        // If nonlinear drop-off is required
        fRatio = Mathf::Pow(fRatio,2.0f);
```

Download at www.Pin5i.Com

```
            int newIndexCount = (int)(MaxIndex * fRatio *
                                      (1.25/fallOffMultiplier));
            CLODObject->SetIndexCount() = newIndexCount;

            // ...
        }
    }
```

通过使用连续的 LOD，在 3D 对象的 LOD 等级之间可以实现无缝的平滑转换，因此，在运行时可以减少视觉假象。这也遵守了渲染系统的设计原则，因为使用这种 LOD 机制可以保留图象质量因数。创建这个渲染系统使用的是一个开放源代码的图形工具箱。FLC 的开发是基于 Matlab 提供的 FIS 库（C 语言）。

3.9.5 游戏中的应用

[Matlab1]Math Work 中的模糊逻辑工具箱为开发游戏中使用的 FIS 提供了一个良好的开端。直接使用这种 FIS，有几点需要说明：

- FIS 假设包含有关多边形负载和帧率关系的规则。它们看起来是最基本的，但是对不同的应用或硬件设置来说可能并非是最优化的。
- 必须从初始测试结果获得一个评估基准，并用这个 FIS 进一步调整规则集，直至取得令人满意的成果。

Math Works 提供了一个有强有力的工具，通过它的 Simulink 环境[matlab2]为模糊逻辑应用建立了原型。这种应用开发过程通常涉及在 Simulink 中创建“积木”，并把这些积木连在一起成为一个系统。即使一个简单的 Simulink 模型文件也能说明一个带有 FLC 的控制渲染过程的概念。不同的信号发生器可以用来模拟各种变化的几何负载条件，这些变化条件通常是由以下一些因素造成的，例如：相机的移动（例如用户的移动）、新实体的创建或游戏中加入新对象。为了更好地说明 FLC 性能，一个可能的办法就是用一个实际的游戏应用代替这个“车间”，而不是使用这个信号发生器作为变化的输入，做到这一点，需要使用 Simulink S-函数结构在游戏周围生成一个包装。

值得注意的是，没有一个模糊逻辑控制器能处理超出基础硬件能力的负荷。开发者或者用户可以自由地设置目标帧率和多边形计数值，就像图 3.9.3 所示的那样，但是这些值必须在平台的综合计算能力范围内。因此，建议进行初始测试来评估目标硬件平台的负载处理能力，以便为这些参数确定合理的值。

3.9.6 假设

采用模糊逻辑管理场景复杂度的方法，就像在这篇文章中讨论的那样，严重地依赖于一个假设条件，假设几何负载一项几乎就可以完全地表述场景的复杂度。但是，实际上游戏开发可能包括着更多的处理过程：例如网络、人工智能、语音和说明每帧中的时间分配的物理计算。此外，使用如绘图明暗法这种更先进的渲染技术，现在通过使用通常的图像和每个像

Download at www.Pin5i.Com

素遮盖算法可以拆开多边形计数和表面细节之间的关系。

但在游戏软件运行时，模糊逻辑的中心思想在控制各种过程元素（包括图形的和非图形的）方面仍然是非常有用的。如要做到这一点，需要将这些过程的信息联合起来输入到模糊逻辑控制器，并且产生一套与各种输入组合相对应的输出的新规则集，就像以前描述的那样。但随后，这也意味着需要足够的专业知识或用精确的公式来表示出输入/输出的规则，才能使模糊逻辑控制器更有效。

3.9.7 实现考虑

虽然连续 LOD 机制能够提供一种更加准确地控制几何负载的方法，但值得注意的是，计算及转换 LOD 的过程可能会阻碍帧速度。因此，一般不建议每帧都计算 LOD，而是根据一定的时间周期或者在交替帧内计算。支持这种方法的另一个原因是因为场景内容在连续帧之间的连贯性，由此可提高渲染的效率。与此相反，如果在每帧都处理 LOD 转换的时候不够细心，也会导致“闪烁效果”，这就是视觉干扰，尤其是转换距离变化到接近观察点的值的情况下（在繁重负载的条件下）。

另一个在执行过程中非常有用的考虑是要加入连续的 LOD 对象。在将 3D 对象送到渲染处理流水线之前需要为 3D 对象建立顶点连接信息是连续 LOD 控制机制的一个众所周知的弱点。尽管这些过程可以脱机执行，但如果测试频率或者游戏应用程序重启次数太多的话，就会降低生产能力。此外，由于连续 LOD 选择的计算开销非常大，很明显，为了保证性能起见，不提倡把一个场景中所有对象都包含到连续的 LOD 对象中去。

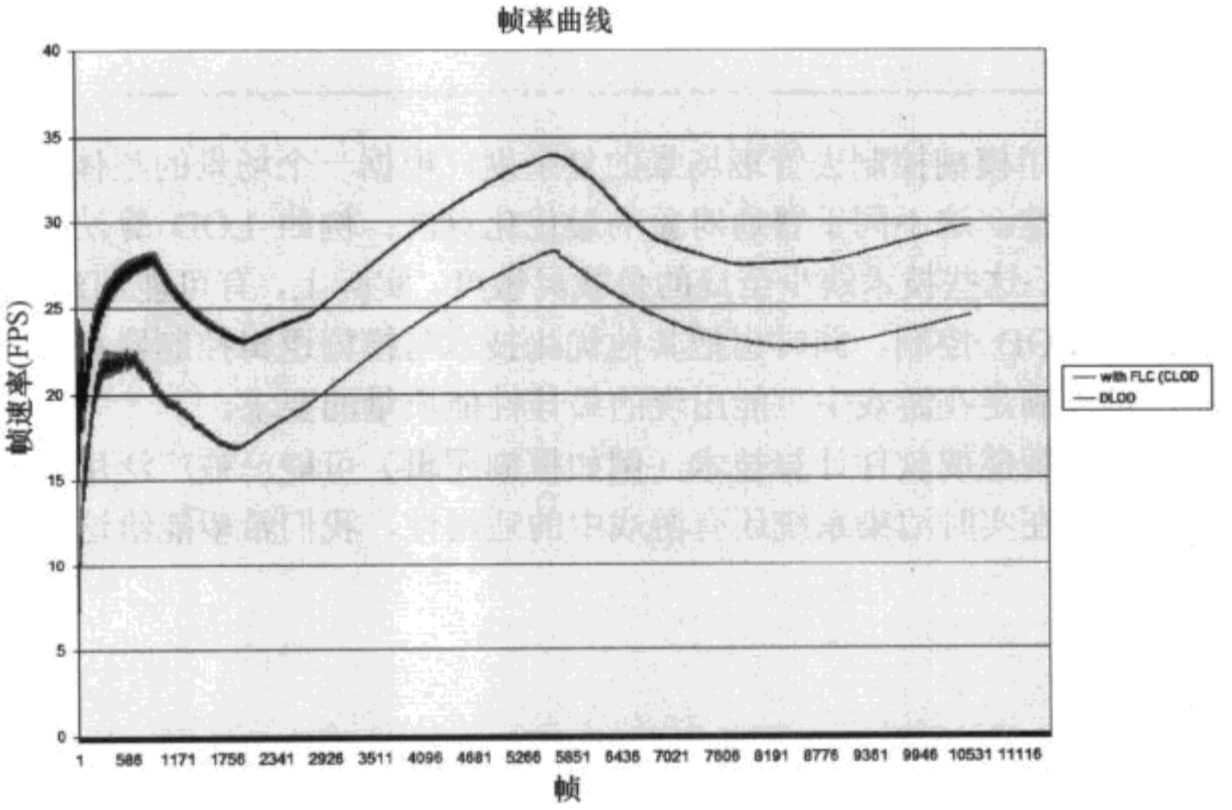
3.9.8 测试和结果

我们开发了一个测试应用实例，是一个由在开阔地带上放养的大批牛组成的三维世界。牛群中大约 40%的牛是由连续的 LOD 创建的，因此这些牛就构成了整个场景几何体中可以由模糊逻辑控制器 FLC 控制的那部分。当然，为了实现最佳场景性能，这个设置值可以根据不同应用中场景内容的状态进行调整。例如，与多边形计数值少的许多小对象相比，如果为了创建相同数量的负载，包含大量几何负载的形状复杂和巨型的对象能更好地由 FLC 来进行管理，因为前者会带来更高的计算机开销。

这次试验的目标是确认一个带有 FLC 的渲染系统的性能收益。同时也测试与仅仅使用离散 LOD（DLOD）的传统方法相比，FLC 的功能和它操纵场景负荷达到用户设定目标的有效性。为了运行测试，使用两种不同的方法来创建一个普通的相机路径。结果见图 3.9.4。

如图 3.9.4 中所示，在整个测试中，我们可以看到带有 FLC 的渲染系统产生的帧率比其他利用传统的 DLOD 的系统高出大约 25%。在测试初始阶段，两个渲染系统会遭受强负载，那时图形负载超过了系统的处理能力（发送到渲染的速率是 30Hz）。图 3.9.4 中所显示的三角计数收益曲线指的是使用两种不同的渲染系统每帧多边形计数的不同。显然，带有 FLC 的系统在这个强负载时期显著地节省了更多的几何负载，尽管帧率短暂地下降到 22Hz 左右。这个系统在测试的其他时期也同样效果明显，收益曲线也取决于场景的组成。图 3.9.5 给出了测试应用的一个屏幕相比的图样，此图片也在调色板 Color Plate 6 上显示。

Download at www.Pin5i.Com



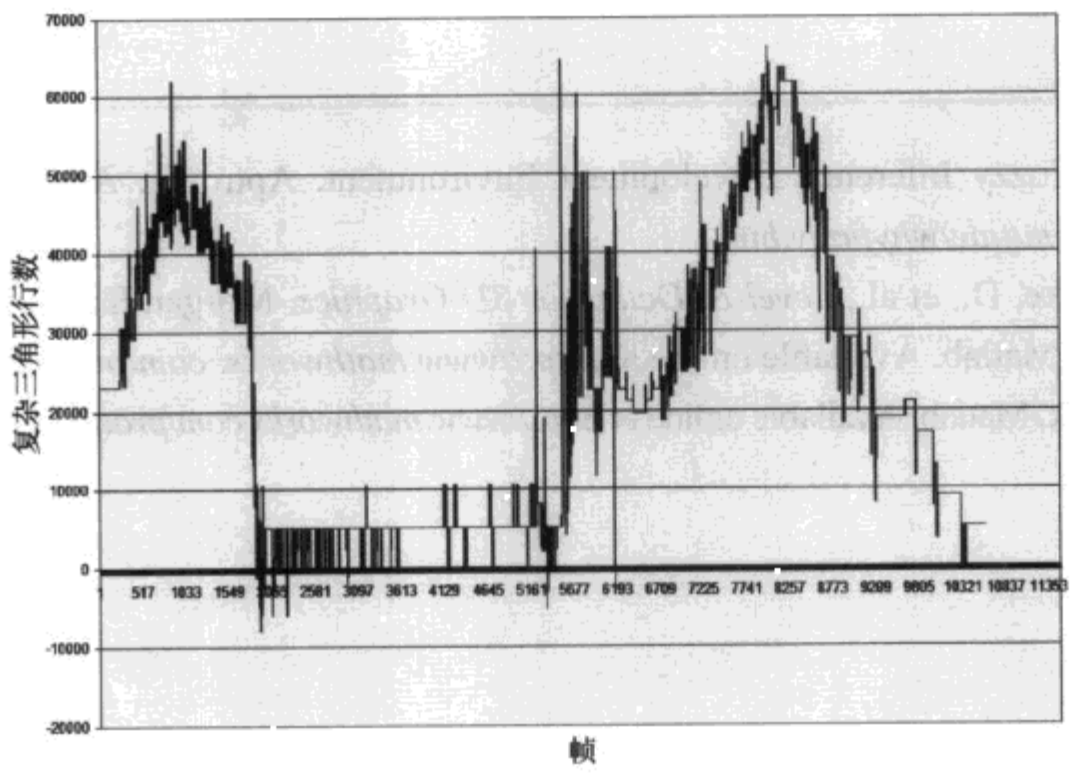


图 3.9.4　两种不同的渲染系统的性能比较

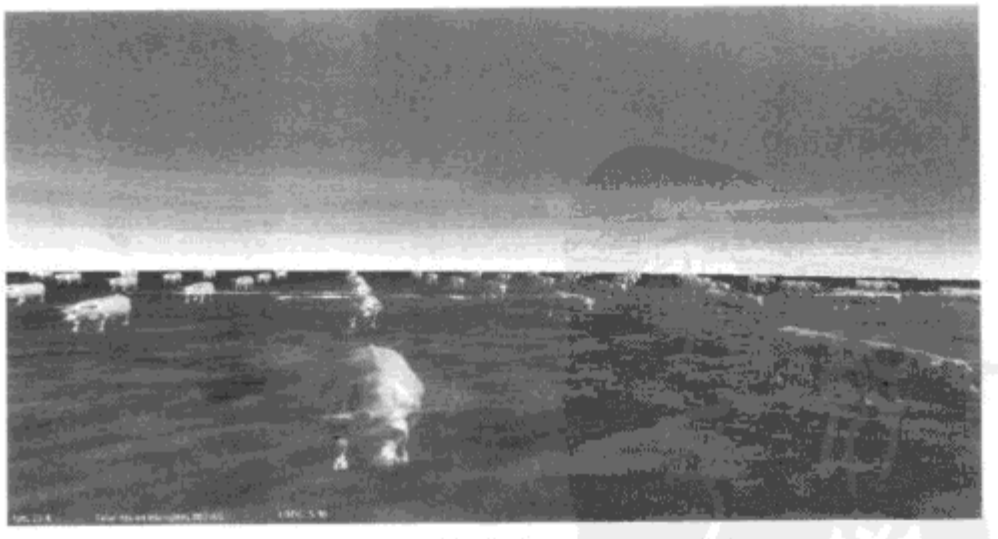


图 3.9.5　测试应用中的一个捕捉图像

Download at www.Pin5i.Com

3.9.9 结论

本篇文章提倡使用模糊控制去管理场景的复杂度，根据一个场景的整体几何负载来实现某种形式的宏观可控性。这不同于普通对象的最优化算法：例如 LOD 算法、可见度计算和基于图像技术的算法。这些技术缺少全局的负载灵敏度。实际上，有可能创建一个软件框架，这种框架不仅仅把 LOD 控制，同时也把其他优化技术与模糊逻辑控制器相联系。这是一种更有力的机制，可以满足在游戏中可能出现的最佳性能质量的要求。

最后，虽然人工智能或软件计算技术（例如模糊逻辑）可能已被广泛用于游戏决策和作战情景。通过剖析它在实时渲染系统还有游戏中的适用性，我们希望能给这个研究领域提供一个全新的水平。

3.9.10 致谢

这项工作是由新加坡国家实验室和支持的，并授与证书 DSOCL01144。

3.9.11 参考文献

[FIDE1] FIDE, Fuzzy Inference Development Environment. Aptronix. Available online at *http://www.aptronix.com/fide/whyfuzzy.htm.*

[Luebke03] Luebke, D., et al., *Level-of-Detail for 3D Graphics.* Morgan Kauffman.[Matlab1] Fuzzy Logic Toolbox. Matlab. Available online at *http://www.mathworks. com/products/fuzzylogic.*

[Matlab2] Simulink. Matlab. Available online at *http://www.mathworks.com/products/simulink.*

Download at www.Pin5i.Com

第 4 章

脚本和数据驱动系统

Download at www.Pin5i.Com

简　介

Graham Rhodes，应用研究协会
grhodes@gsrhodes.com

在视频游戏刚出现的时候，还是很简单，而且很容易开发。最早的游戏基本上不包含任何美术内容，而且经常是靠一名能够利用一些传统的编程语言，如 C 或其他集成语言，来编写代码的程序员独立开发完成的。当时，任何编程语言都可以进行游戏开发，而且那些所谓的玩家并没有任何关于游戏如何输出，游戏的画面应该是怎样的固定看法，即使游戏内没有角色的 AI 也可以通过 Turing 测试。

含有清晰定义过的规则的简单游戏只需要一个闭合的系统。即使今天，在一个游戏开始研发的阶段，这个理论也是成立的。但是随着电视游戏的发展，游戏玩家也在不断地成熟。至今为止，游戏行业已经快速发展了将近 45 年。今天的很多游戏都比较复杂。对于一个给予代码的现代游戏来说，在一个 1～2 年的制作周期内，由一个团队的程序员编出百万行的 C++代码已经是很平常的事情了。现在游戏的美术内容和游戏设计已不由程序人员创造，游戏已经成为一种数据驱动产品。精英玩家们希望游戏的动作做到极尽真实。他们希望在游戏中看到当突发性事件发生时要有动态的音效，由 AI 控制的角色的行为也要越真实越好。现在很多电视游戏的大小已经超出了几张 CD-ROM 的容量范围，游戏中要包含成百上千的游戏物件和角色的 3D 模型、贴图以及动作。无论是从程序员的角度来看，还是从美术师的角度来看，排除游戏的引擎、游戏玩法特点和游戏中的行为不谈，仅管理如此大数量的游戏物件，已在很大程度上增加了游戏内容的复杂性。

在近几年来进入到游戏中的脚本语言在很大程度上为那些制作游戏的人和玩游戏的人提供了好处。例如，程序员可以利用脚本语言来实现引擎特征，或者为游戏构建一个版本 debug，或者在一台并没有安装开发工具和源代码的计算机上对游戏进行 debug。脚本语言还可以帮助程序员很快地开发出一些功能，比如 AI 系统的有限状态机，或者任务管理系统等。脚本语言的语句结构比 C++要简单得多，这种简单性可以允许游戏设计师不用等程序人员花费时间编写 C++功能和重建游戏应用程序就可以自己实现一些游戏玩法特点。同理，那些会使用 3ds Max 和 Maya 数字内容生成工具编号脚本的美术师也可以轻松地学会如何使用游戏中的脚本，并以此来实现动作触发器或者播放游戏内的过场动画，等等。现在几乎全世界每一个角落都有玩家在用脚本语言来编写他们最喜欢的游戏的 MOD。

Download at www.Pin5i.Com

今天，脚本语言和游戏引擎一样，已经很普及了。那种要想提供脚本支持只能从头专门建立一个脚本语言系统的时代已经一去不复返了。现在的状况是，开发者可以根据他们在高性能、高兼容性、多线程、网络化，简单化的需要，从多种高产品质量，现成的脚本系统中挑选一种来使用。

对于现在市面上众多的脚本系统，开发者们又会面临一个问题：怎样才能选出最适合自己的脚本系统，开发者怎样在防止兼容性风险的同时又能最大的发掘系统的特点呢？本章将会提供大量的技巧，来帮助您解决这些问题。

首先，Diego Garcés 提供了游戏开发环境中最受欢迎的几个脚本语言的调查报告，并介绍了哪些特征可以针对解决哪些问题。接下来，Waldemar Ceeles、Luiz Henrique de Figueiredo 和 Roberto Ierusalimschy 提供了关于 Lua 开发的两篇文章。首先，他们介绍了几种在 Lua 中建立 C++对象的方法；然后，他们讨论了多种利用 Lua 中直观列表迭代器的协同例程（co-routine）和 ranging 功能来管理协同多重任务。对于那些需要即时实现几百个模拟任务的游戏，Sébastien Schertenleib 接下来介绍了一个利用 Python 的微线程在开发过程中协同多重任务编码对其进行管理的方法。

脚本系统的最佳工作环境是在一个编写的比较直观，处在最优化状态的 C++引擎中。由于现代游戏数据驱动的这一特性，游戏引擎需要以一种自然的方式来反映游戏内容的本质是很重要的。因此，在开始开发之前进行游戏架构的设计并不是一种琐碎无用的事情，在游戏初期决定的一种不好的设计会导致开发成本和 QA 成本的增加，最终导致游戏的表现糟糕，甚至使整个项目失败。本章有两篇文章涉及到数据驱动的架构，其中将讨论在游戏中表现游戏物件的各种方法。而且这两篇文章所讨论的引擎架构可以很直观地设计成一个脚本系统。Matthew Campbell 和 Curtiss Murphy 介绍了一个架构，该架构可以在某种程度上通过代理对象表现游戏中的内容，以便很容易地将游戏引擎中的对象整合到美术工具（如关卡编辑器等）或者高级运行时子系统中（如消息框架）。Chris Stoy 介绍了一个组织性很强且比较简单的游戏物件组件系统，该系统能够帮助解决由于专门的或者赶工出来的游戏引擎设计引起的 C++对象膨胀问题，并且能够简化运行时的物件合成。

最近一段时间游戏开发者所采用的游戏开发流程比以往任何时候都要复杂微妙；然而，还是有很多团队会在发行商对需求做出变更时面临计划不合理、拖延、反应迟钝的困扰。一种设计合理的数据驱动软件架构，再加上一种灵活的脚本系统，能使您的团队在面对类似情况时快速地恢复高效的开发能力，并且在必要的时候能够很快的转变方向。通过对本章介绍的那些技巧的应用，可以在制定计划、前期开发和为下一个游戏项目做框架准备的时候优化您现有框架。利用这些作者智慧的结晶，确保能够提高团队在紧迫开发环境中完成高质量游戏的能力。

Download at www.Pin5i.Com

4.1 脚本语言总述

Diego Garcés，FX Interactive
diegogares@gmail.com

4.1.1 为什么要使用脚本语言

在过去的几年中，脚本语言成了开发计算机和视频游戏的一种很受欢迎的工具。脚本语言可以在游戏执行时控制游戏的多个方面。例如，可以控制动画的次序、敌人类 NPC 的行为，以及由 AI 控制的队友对玩家的指示的反应，等等。游戏的这些部分都是由美术师、设计师或玩家来创造的。脚本语言能够让这些开发团队的成员修改或创造这些行为。

脚本语言使快速原形开发成为可能，这种开发模式正在被越来越多的游戏开发团体所接受。为你的游戏建立一个游戏原形有助于在开发前期测试游戏中的问题，并试验多种制作方法。根据同样的道理，脚本还允许不经过长时间的汇编就能快速地对游戏进行大的调整，甚至有可能不需要重新读取游戏。

4.1.2 简介

可供游戏开发者选用的脚本语言有很多种。本文不会介绍所有的脚本语言，但是会介绍如下比较受开发者欢迎的脚本语言：

- Python；
- Lua；
- GameMonkey；
- AngelScript。

Lua[Lua05]和 Python[Python05]是两个比较成功的商业游戏，而 GameMonkey[GameMonkey05]和 AngelScript[AngleScript05]比较新，但是也在逐渐被各个游戏开发团队所接受。

脚本语言有很多共性，因此你在选择脚本语言的时候一定要考虑如下 4 点。

- **语言的编码**：语言的特征和编码的易用性。
- **与 C 和 C++的整合**：脚本语言大多是对 C++核心语言的延伸。因此必须要确保在核心引擎和脚本之间的数据转换不需要太多程序方面的努力，并且对于项目的需求来说要功能足够强大，并且足够有效。

Download at www.Pin5i.Com

- **性能**：尽管脚本并不用于游戏中与性能相关性很大的部分，但是，在进行游戏及游戏开发的时候如果消耗过多的内存就是一个比较严重的问题了，另外一些任务的执行过缓还会影响到游戏的桢率。
- **对开发的支持特点**：一些能使脚本语言使用更为简便的外加工具或功能可以大幅度提高开发人员的效率。比较好的 debug、说明图和记录工具可以帮助你更为迅速的解决问题。

4.1.3　语言编码

1. 通用特征

脚本语言都会拥有一些通用的特性，因为在设计脚本语言的时候有很多类似的目标。

脚本语言是翻译语言，不能汇编成机器代码，因此脚本语言需要一台虚拟机来执行。这种特征导致了脚本语言比较慢。但是与此同时也增加了语言的灵活性，这种灵活性表现在游戏运行的同时可以调整代码，并且能够快速地调节算法或者游戏参数。为了减少运行时间，脚本代码一般来说可以编译成字节代码（一个脚本的中间二元形式），尽管字节代码也是运行在虚拟机上。

脚本语言是高级语言。它们会使用自动内存管理，因此可以不用考虑指针、访问冲突和内存泄露等问题。另外一些高级的结构图和结构表可以让程序员将他们的努力专著在算法，而不是数据处理上。

脚本语言能为流控制提供很大的灵活性。流控制和迭代特征在所有的脚本语言中也大致相同；但是它们总是在灵活性和易用性上有所差异（例如，能对结构表和结构图上的元素轻松地进行迭代）。

2. 非通用特征

不同的设计理念会导致不同的脚本语言在解决同一个问题的时候采用不同的方法。本章会针对一些问题解释一下怎样选择相对应的语言。当然决不会巨细无遗，关于某种语言，如 Python，总的特征及延伸，并不在本文讨论范围之内。

3. 类型安全，参数传递，以及内存管理

AngleScript 是一个强类型语言。其中的参数和变量都拥有一个固定的类型，并且不会拥有其他类型的值。脚本被编译成直接代码的时候要进行类型检查。表 4.1.1 对固定型和动态型的语言做了一下比较。

表 4.1.1　　**固定型与动态型语言的比较**

Python	Lua/GameMonkey	AngelScript
f = 3.1415	f = 3.1415;	float f = 3.1415;
f = True	f = true;	f = true; // Error
f = "hello world"	f = "hello world";	

除了 AngelScript 之外的其余语言都是动态型的。简单变量和容器都可以存储其他类型的值。甚至可以存储函数，因为它们被看成是第一级变量。这在传统意义上讲是带有 Lisp 语言

Download at www.Pin5i.Com

的函数式编程的一个很流行的特征，在 AI 研究领域被广泛的应用。

AngelScript 在变量被使用前要先声明变量的优先级，Python、Lua、和 GameMonkey 在变量被指定时就可以生成。在 Lua 中，如果您不希望变量成为全局变量，还可以将其声明为本地变量。在 GameMonkey 中你必须声明某一个变量为全局的，不然其就会被定义为本地变量。

在 Lua、Python 和 GameMonkey 中，函数可以接收任意数量的参数，并可以返回多种值。在 Lua 中，需要声明确定的参数，并且不被支持的参数会指定给 *nil* 值（一个特殊的值，表示该值缺失，与 Boolean 值 false 的用法相同）。表 4.1.2 中给出了这些语言中的多种参数以及返回值的一些例子。

表 4.1.2　　使用多种参数和返回值的例子

Python	Lua

def f(a,b,c = False):
　　if (c):
　　　　return a+b,a*b
　　else:
　　　　return a-b,a/b,a+b
	function f(a,b,c)
	　　if (c) then
	　　　　return a+b,a*b;
	　　else
	　　　　return a-b,a/b,a+b;
	　　end
	end

Example Usage: d,e = f(2,4,True) # Result is 6,8	**Example Usage:** d,e = f(2,4,true) -- Result is 6,8
g,h,i = f(4,2) # Result is 2,2,6	g,h,i = f(4,2) -- Result is 2,2,6
g,h,i = f(4,2,True) # An error results	g,h,i = f(4,2,true) -- Result is 2,2,nil
j,k = f(2,4,True,20,30) # An error results	j,k = f(2,4,true,20,30) -- Result is 6,8
k = f(6,2) # Result is 4,3,8	k = f(6,2) -- Result is 4
k, l = f(4,2) # An error results	k,l = f(6,2) -- Result is 4,3
k = f(b = 2,a = 6) # Result is 4,3,8	k = f(b = 2,a = 6) -- Not possible

由于涉及自动内存管理，我们在这里所讨论的所有语言都会提供某种动态容器。Python 在动态结构尚有很大的灵活性，这些灵活性是通过 list、tuples、set 和 dictionaries 等内建类型所实现的。下表是 Lua 和 GameMonkey 中所使用的容器和物件、模仿图谱、列队、树或者类（如表 4.1.3 所示）。AngelScript 用的是一个动态的内建类型。

表 4.1.3　　Tuple、List 和 Table 举例

Python	GameMonkey
pot1 = HealPotion() pot2 = HealPotion()	pot1 = HealPotion(); pot2 = HealPotion();

Download at www.Pin5i.Com

续表

Python	GameMonkey

pot3 = ManaPotion()
weapon1 = Sword()
gold = 20.5
bag = pot1, pot2, pot3
backpack = [weapon1, bag, gold]
# bag is a tuple 	
# backpack is a list 	
	pot3 = ManaPotion();
	weapon1 = Sword();
	gold = 20.5;
	bag = {pot1, pot2, pot3};
	backpack = {weapon1, bag, gold};
	// bag and backpack both tables

4. 继承与面向对象编程

脚本语言经常被用来建立部分游戏或整个游戏的快速原型。脚本语言中支持的面向对象编程是很重要的，因为现今大部分的游戏都是建立在对象的基础上的。只有 Python 还会提供经典的 out-of-the-box 对象支持。对对象的简单定义及控制可以通过一个和多个继承体系来实现。就像表 4.1.4 中所显示的，Lua 和 GameMonkey 用 table 来模拟类。由于函数是一类值，因此可以在 table 中存储，也可以作为方法来使用。继承在 Lua 中也可以实现，但是需要一些利用到元表的高级编程(例如，对一些情况的 callback 调用，对不存在项的访问)。AngelScript，作为一个扩展语言，只会提供一些对 C++类的访问。虚拟函数一般情况下不会有什么问题，因为容器和函数都可以使用导出值。

表 4.1.4　　本地的和基于 Table 的对象编程比较

Python	Lua

class Soldier:
"Soldier base class"
def TakeDamage(self,iDamage):
self.iHealth-=iDamage;
iHealth=200
def Print(self):
print "Soldier Health:"
print self.iHealth
 	
class Sniper(Soldier):
"Sniper derived class"
def Print(self):
print "Sniper Health:"
print self.iHealth
	Soldier={}
	S_mt={__index=Soldier}
	function Soldier:new()
	return
	setmetatable({iH=200},S_mt)
	end
	
	function Soldier:TakeDamage(iDamage)
	self.iH = self.iH-iDamage
	end
	
	function Soldier:Print()
	print("Soldier Health:",self.iH)
	end
	
	Sniper = {}
	Sn_base_mt={__index=Soldier}
	setmetatable(Sniper,Sn_base_mt)
	Sn_mt={__index=Sniper}
	function Sniper:new()
	return
	setmetatable(Soldier:new(),Sn_mt)
	end
	function Sniper:Print()
	print("Sniper Health:",self.iH)
	end

5. 语言专门性问题

Lua 使用的是一种称为尾部调用的调用机制，该机制允许使用递归算法。它能够为了新

Download at www.Pin5i.Com

的函数反复使用同一个栈，这就消除了建新调用时的大部分花费（overhead）。但是尾部调用要求调用必须是当前函数的最后一个表达式，见程序清单 4.1.1。

程序清单 4.1.1 使用 Lua Tail 调用的例子

```
function Hanoi(n, sA, sB, sC)
    if (n == 1) then
        print("Move disk from",sA,"to",sB)
    else
        Hanoi(n-1,sA,sC,sB)
        print("Move disk from",sA,"to",sB)
        return Hanoi(n-1,sC,sB,sA) -- tail call: reuse of stack
    end
end
```

元表是在某些值上执行的 callback。也是 Lua 的一个特征，最初只适用于 table 和 C 对象。在其后的 5.1 版本中，每一个类型才有其自己的元表。元表的主要用途是绑定 C++中的类并对基于 table 的对象提供支持；当然，元表还有很多其他的潜在用法。

GameMonkey 最初支持的是状态的概念，广泛应用于 AI 的有限状态机中，并且能够实现在一个函数执行过程中的任何一点来终止执行，并跳到另外一个函数的功能，以此来模拟状态变化。

4.1.4 与 C++的整合

本小节将讨论游戏开发者如何将他们的脚本语言与 C++进行整合。在整合过程中需要注意以下几点：

- 全局变量共享；
- 从 C++代码使用脚本；
- 从脚本访问 C++的特征。

1. 一般性介绍

- Python

Python/C 的 API 能够使数据的函数型转换得以实现，通过 C++中建立 Python 值的函数，可以调用 Python 函数，并且对 Python 可调用的 C++的 callback 函数进行注册。将标准 Python 绑定在 C++上并不是一个简单容易的过程。需要很多代码，并且很容易变得很庞大且容易出错。一般来说，为了方便起见，会采用一个能够自动将 Python 整合到基于 C 或 C++的游戏中的系统。如果要是有足够的时间和资源的话也可以自己编写这个系统，或者使用一些免费的软件，如 Boost.Python 或者 Swig。

- Lua

Lua 和 C++的信息交换是通过 Lua 中运行时管理的一个虚拟栈来实现的。每一个函数都会有它自己的栈。这就意味着如果您调用一个 C++函数，你就调用了一个 Lua 函数，同时会建立一个新的栈以交换数值。为了提供更多的灵活性，这个栈不会是一个纯粹的栈，它的数据可以通过索引被访问。正索引从栈的底部访问，而负索引从栈的顶部访问。

Download at www.Pin5i.Com

- AngelScript

AngelScript 和 C++共享调用协议，这也是该脚本语言设计的一个目标。正因为这样，在将 AngelScript 整合到 C++代码的时候就不需要代理函数了，这也减轻了在共享函数方面所作的努力。

- GameMonkey

GemeMonkey 支持对 C++的简单整合，这也是该脚本语言设计的一个目标。尽管 GameMonkey 通过 table 的使用支持有限的面向对象编程，但是它并不像其他面向对象的语言那样缺少对类继承、多形态以及函数超负载的支持。甚至于，可以通过一些额外的编程工作实现对类的支持；如果你不需要复杂的面向对象编程的话，GameMonkey 也可以很完美的满足你的要求。

2. 全局变量共享

Lua、AngelScript 和 GameMonkey 在脚本中定义了对全局变量的完全访问。Lua 和 GameMonkey 还可以让你访问全局变量并保存全局变量，AngelScript 提供 API 函数能够回收一个全局变量的指针。Python 中的全局常量可以通过 C++访问。

在 AngelScript 中，C++定义的变量可以通过访问它们在 AngelScript 中的脚本代码来轻松地实现注册。Lua 和 GameMonkey 使用全局 table 来存储从 C++获得的用户数据，这些数据还可以通过脚本函数访问。

3. 从 C++代码中使用脚本

脚本语言是作为 C++核心引擎的一个延伸来使用的。从 C++的角度看，它需要调用一个脚本函数。整个调用过程由通过 argument、开始执行和回收返回值三部分组成。

Python 程序员在使用脚本语言的时候需要转换所有 Python 对象的参数并且按 tuple 对它们进行分组，这样它们就会以一个特殊的 argument 被通过。在 Lua 中，参数会存储在一个栈里，而 AngelScript 和 GameMonkey 会加入帮助者类，分别是一个 asIScriptContext 或者一个 gmCall，这个过程在函数名被调用的时候就被初始化。

对 Python 和 Lua 来说，调用函数需要进行一次特殊的 API 调用。在 AngelScript 和 Game Monkey 中，会利用帮助者类的方法来开始脚本的执行。

一旦函数执行完成，就需要收集执行结果。GameMonkey 和 AngelScript 只能通过再次访问帮助者类而回收一个返回值。从 C++调用 Python 函数也只能返回一个值，但是这个值可能是一组几个值的一个 tuple。这几个值被放在栈中，并由 C++函数在脚本执行完毕后进行回收。表 4.1.5 比较了调用不同的脚本函数的方法。

4. 从脚本访问 C++的特征

一般来说，调用协议在脚本语言和 C++之间是不同的。也正是因为这样，需要为每一个我们想调用的函数编写代理函数（如表 4.1.6 所示）。当然，AngelScript 是一个特例，因为在设计上 AngelScript 与 C++简单绑定并与 C++共享调用协议。

Download at www.Pin5i.Com

表 4.1.5

Python	Lua

d=PyModule_GetDict(module);
func=PyDict_GetItemString(d,
"Attack");
arg=Py_BuildValue("si","Grunt",
iDice);
res=PyObject_CallObject(func,
arg);
PyArg_ParseTuple(res,"ff",
&fDamageReceived,
&fDamageInflicted);
	lua_pushstring(L,"Attack");
	lua_gettable(L,LUA_GLOBALSINDEX);
	lua_pushstring(L,"Grunt");
	lua_pushnumber(iDice);
	lua_call(L,2,2);
	fDamageInflicted=lua_tonumber(L,-1);
	lua_pop(L,1);
	fDamageReceived=lua_tonumber(L,-1);
	lua_pop(L,1);

GameMonkey	**AngelScript**

gmMachine m;
gmCall call;
call.BeginGlobalFunction(&m,
"Attack"))
call.AddParamString("Grunt");
call.AddParamInt(iDice);
call.End();
call.GetReturnedFloat(fDmgRcvd)
// DamageInflicted cannot be
// obtained
// obtained
	engine->CreateContext(&context);
	fID=engine->GetFunctionIDByDecl(
	"module", "floatAttack(int,int)");
	context->Prepare(fID);
	context->SetArgDWord(0,GRNTID);
	context->SetArgDWord(1,iDice);
	context->Execute();
	fDamageReceived=
	context->GetReturnFloat();
	// DamageInflicted cannot be

表 4.1.6

Python	GameMonkey

PyObject* AttackP(PyObject* self,
PyObject* args)
{
PyArg_ParseTuple(args,"si",
szEnemy,&iDice);
Attack(szEnem,iDice,
fDamageReceived,
fDamageInflicted);
return Py_BuildValue("ff",
fDamageReceived,
fDamageInflicted);
}
	int _cdecl AttackP(gmThread* th)
	{
	GM_CHECK_NUM_PARAMS(2);
	GM_CHECK_STRING_PARAM(szEnem,0);
	GM_CHECK_INT_PARAM(iDice,1);
	Attack(szEnemy,iDice,
	fDamageRec,
	fDamageInf);
	th->PushFloat(fDamageRec);
	th->PushFloat(fDamageInf);
	return GM_OK
	}

在 Python，这些代理函数会收到一个包含有参数的 Python 对象，并且返回一个作为结果的 Python 对象。在 Lua 和 GameMonkey 中，它们会收到一个状态结构——Lua_State 和 gmThread——这个状态结构允许回收参数并发送结果给脚本，另外还有一些其他有用的操作。

对于脚本虚拟机来说，它没有办法靠自己来识别 C++函数。我们必须告诉它在哪里能找到这些函数，以及从脚本的范围来说这些函数的名字是什么。Python 和 GameMonkey 需要开发者自己建立一个包含所有函数的清单或模块，其中的信息包括每个函数的名字和代理函数的指针。开发者必须将这个模块注册成为一个单独的 API 调用。Lua 和 AngelScript 会导出全局函数，并单独的注册每一个函数，因此不需要任何的中间结构。表 4.1.7 比较了这两种注册方式。

Download at www.Pin5i.Com

表 4.1.7

Lua	GameMonkey

lua_register(L,"Attack",
AttackP);
lua_register(L,"Attack2",
AttackP);
	static gmFunctionEntry s_AttLib[]=
	{"Attack",AttackP},
	{"Attack2",Attack2P},
	...
	
	};
	machine->RegisterLibrary(s_AttLib,
	sizeof(s_AttLib)/sizeof(s_AttLib[0]));

AngelScript 允许 C++方法像全局函数那样被注册。Phython、Lua 和 GameMonkey 则只能注册静态方法。这是因为方法的地址只有被声明为静态的时候才能够获得。这些静态方法可以作为中间函数来回收对象范围之外的额外的参数，然后利用这个对象范围调用真正的方法。这样，通过两个间接性来得到一个泛函性。

AngelScript 可以很简单地通过一个单独的 API 调用来输出类。Python、Lua 和 Game Monkey 也有类似的机制，但是需要多余的编程，并且会比较复杂。这些工作一般来说会交给一些绑定工具来完成。

5. 绑定工具

如我们所见，在 C/C++和脚本语言之间交换数据是一个比较复杂的工作。由于这个理由，Python 和 Lua 社区开发出了自动完成脚本语言整合的整合器。GameMonkey 甚至还有一种专门的自动绑定工具，不过还处于开发初期。为了避免大量的由拷贝粘贴造成的错误，因此，花一些力气开发一种绑定工具，或者学习如何有效的利用一些现有的绑定工具还是很重要的，这能够让您通过一种很好的方法在脚本和 C++之间共享特征。

- Python-Swig

Swig 是一个 C 的预处理程序工具。它会将一个端口文件作为输入，由开发者编写，并向 C++和 Python 输出原文件，从而形成游戏汇编的一部分。这个接口文件会解释 Python 将会用到的所有的 C++特征，例如函数、类和全局变量等。

Swig 有一套自动的机制，能为 C++类生成影子类。这些影子类就是拥有和 C++相关类同样行为的 Python 类。通过这种机制，就可以通过 C++的执行获得明确的 Python 代码。由于影子类也是真正的 Python 类，它们也作为基础的类用在继承的层级结构中。

Swig 通过使用 director 来支持虚拟调用的覆盖重载。Director 是一个中间对象，它通过向前的方法调用来实现一个合适的执行，不管该执行是属于 C++还是 Python。这会对内存和 CPU 时间加重一个值得考虑的过载。因此，只有在编写用 Python 覆盖一个 C++方法的时候才被推荐使用。

Swig 还支持超载函数和控制器，但是当 Swig 不能分辨参数类型的时候将不会支持（如，int 和 short）。

例外情况可以通过 C 函数返回偏差值来进行扩散甚至生成，但这需要在接口文件中再作一些工作。表 4.1.8 介绍了一个 Swig 接口的例子。

Download at www.Pin5i.Com

表 4.1.8

C++ Header File (soldier.h)	Swig Interface File

class Soldier
{
public:
void TakeDamage(int iDamage);
void Print();
}
	%module Army
	%{
	#include "soldier.h"
	%}

- Python- Boot.Python

Boot.Python 是依靠于元编程的一个库，用于将 C++和 Python 整合在一起。这个系统的最终目标就是尽量做到无缝。这就意味着它不需要在 C++的代码上做太大的改动就能将 C++的特征导出到 Python 中。

C++的类可以通过编写 C++代码转换到 Python 中，但是句法结构要类似于 Python。程序清单 4.1.2 向我们揭示了这个方法以及相关的数据成员。

程序清单 4.1.2　揭示了 C++类与 Boost.Python

```
#include <boost/python.hpp>
BOOST_PYTHON_MODULE(Army)
{
    class_<Soldier>("Soldier")
       .def("TakeDamage", &Soldier::TakeDamage)
       .def("Print", &Soldier::Print);
}
```

Boot.Python 还支持以参数和数据成员为导出属性的构造函数。 这些属性在 Python 中会被看为公共数据成员，但是内部来说它们还是被当成访问器在使用。

C++的继承必须要在类的定义上体现出来，并且应该在 Python 代码中达到预期的行为。新的 Python 类可以从打过包的 C++类中导出，但是如果我们希望调用 Python 的虚拟的重写方法，我们就必须要建立一个调度器类在 Python 函数存在的情况下进行调用。Pyste，是一个已建立的扩展，可以通过它来简化这项工作。

函数重载和控制器会按照默认参数导出为函数——每一个重载，参数数量都需要一个单独的注册。

在 Python 执行过程中从 C++中调用时产生的异常，会自动被传播和转换。

Boot.Python 还提供一些其他特点，包括从 Python 代码中使用 C++迭代器，使其成为 Python 迭代器，而且支持恢复。对象接口以及它为 Python 通用类型导出的类，也是 Boost.Python 库中的一部分，并且能够将 Python 的数据类型的使用封装在 C++代码中。

- Lua-ToLua++

和 Swig 类似，ToLua++采用以文件包为输入（基本上是一个头文件的列表）并输出 C++代码的方式来实现绑定的。不同的是通过 ToLua++，可以将一些额外的注释信息放到头文件中，并将其打包在文件包中。

通过 ToLua++导出的类不需要太多的努力，只需要对这些类进行一个清楚的 C++定义就可以了，定义中要说明方法和导出的数据成员。这样用户就可以将这些类作为 Lua 类来使用，并对其进行一切允许的操作。另外，ToLua++也支持单独的继承和多态性，但是多重继承在

Download at www.Pin5i.Com

访问其成员的时候要指定额外的基类。

ToLua 支持重载函数和控制器，但是控制器必须是类的一部分，不可以是外部函数。

- Lua-LuaPlus

LuaPlus 所解决的问题和 ToLua++不同。它是一个 C++的包装器，但同时也提供针对 Lua 内核的一些扩展和修改，包括 Lua 的一个修改版。因此，如果您想要使用最新的官方 Lua 版本，LuaPlua 可能不是最好的选择。

LuaPlus 并不会提供完全的 C++导出。LuaPlus 可以让用户注册函数和类方法，并让 Lua 以全局函数的形式识别它们，并自动建立闭包（closure）或者仿函数(functor)。不过，LuaPlus 并不会提供跨语言的继承或函数重载。它的函数分派机制没有 ToLua++强。

LuaPlus 会提供一些额外的功能使在 C++中使用 Lua 更加简单一些，这些功能是其他绑定软件所不具备的。这些额外的功能就是将 Lua 状态（LuaState）和 Lua 对象（LuaObject）进行封装。通过 LuaState，C++程序可以初始化 Lua，利用堆栈来进行操作，还可以很容易地访问全局变量或注册表。LuaObject 能提供非基于堆栈的 Lua 对象的封装。通过这种方法，C++程序员可以忽略掉 Lua 堆栈，从 C++代码中使用 Lua 对象也会变得极其简单。

4.1.5　性能特点

1. 内存管理

这些脚本语言都是高级语言；正因为这样，它们都会进行自动的内存管理。而一些复杂任务，如为结构定位内存，或释放内存，或解决内存泄露以及访问异常，在这些脚本语言中都不会存在。但是，还是需要注意一些内存使用上的一些问题。在非正确的时间释放内存会对桢率造成影响，过多的内存消耗或者内存分段也对游戏机平台上的性能造成很大影响。

- 引用计数

引用计数只是每一个对象都会计算其他对象需要引用它的次数。如果一个对象的引用计算是零，那么就意味着在今后不会有对象需要访问它，因此它可以从内存中被释放掉了。Python 和 AngelScript 都会执行垃圾回收算法，但是在内存管理上两者会比较依赖于这种方式。

通过引用计数，使生成引用循环来防止对象被释放掉成为可能。图 4.1.1 示范了这个例子。在这一个特殊的例子中，由于始终有一个引用被保留（Turret 中的一个引用），就算是 Tank 对象中所有的外部参考都被释放了，对象还是会被保留。这个问题的产生是由于 Turret 是 Soldier 的参考，因此它不会被释放；而 Soldier 又

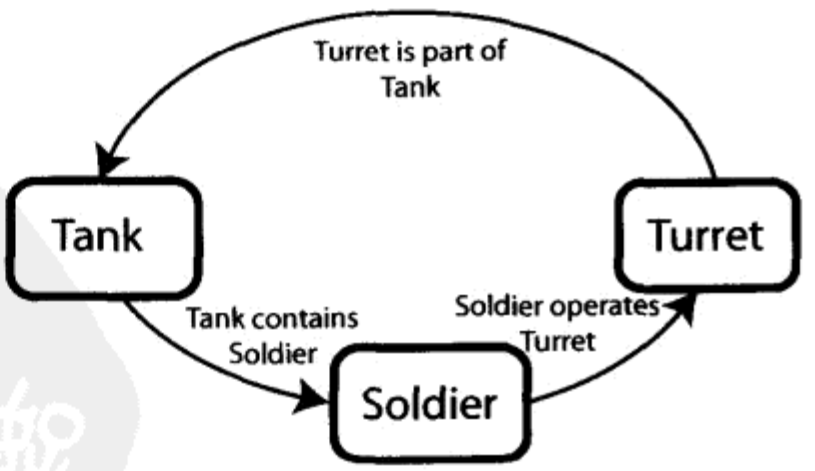


图 4.1.1　使用循环示例

Download at www.Pin5i.Com

是 Tank 的参考，这就完成了一个循环。它们中任何一个都不会被释放掉，即使该循环外的任何对象都没有被使用。如果没有提供其他的自动内存管理系统，那么脚本的编写者必须要小心避免这种情况发生。垃圾收集，我们将会在下面讲解，它会提供一个自动打破引用循环的方法。

释放一大块内存也可以使用类似的方式。正因为这样，引用计数有了一种通过发布式超时来释放内存的优势。每当对象变成非引用状态，就会有一些时间丢失，这要依赖于对象的大小。通常这个时间丢失是可以忽略不计的。

- 垃圾收集

与引用计数不同的是，垃圾收集并不会通过保存外部信息来辨认对象是否处于被引用状态。这种信息是通过计算每一个垃圾回收循环过程从根元素开始漫游整个对象层级来获得的。所谓的根元素（例如，线程和全局变量）将存储一些对象的引用信息，这些信息会一直被访问直到没有新的对象被找到。对象在每一次引用的时候会被打上一个标记，这样脚本语言在运行时就不会试着释放它们。当整个漫游过程结束后，所有没有引用标记的对象会被进行垃圾收集。这个漫游和释放过程通常上被称为标记和清除（mark and sweep）。

图 4.1.2 给出了一个典型的战争游戏的对象层级关系。整个世界由一辆 Tank、一个 Soldier 和一个 Squad，当然，该 Squad 还可以细分为更小的组成部分，包括 Privates。如果想知道一个 Private 对象是否可删除，垃圾收集器必须漫游整个 Squad 对象。Private 对象会打上标记，因此它就不会被删除掉。当这个 Private 死掉的时候，一个 AI 算法将会把它从 Squad 对象的引用移除。由于没有任何对象引用它，Private 将在下一个 mark and sweep 循环中不被标记，这样它就会在内存中被删除掉。

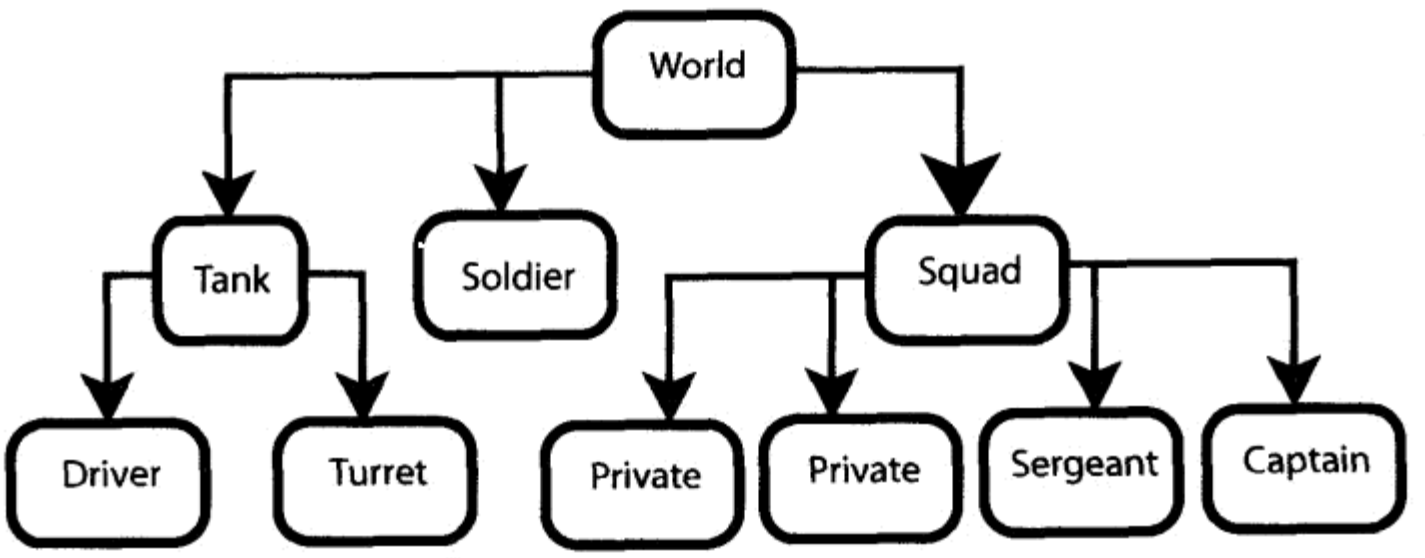


图 4.1.2 层级关系示例

如果 mark and sweep 过程通过一步完成，通常被称作“全垃圾收集”。一个全垃圾收集循环可能会很长时间，特别对于那些比较忙的系统来说，这也是一个不再即时的应用程序（例如，游戏）中使用垃圾收集的传统原因。为了解决这个问题，增量垃圾收集器诞生了。它的整个垃圾收集过程通过每隔几桢来实行的若干垃圾收集过程来实现，这就最大限度地减少了垃圾操作对桢率的影响。

就像前面介绍的，引用循环无法通过单独使用引用计数来解决，因为每一个对象都至少

Download at www.Pin5i.Com

要含有一个引用并且不会被释放掉。垃圾收集对这一问题提供了一种解决方法。Python 和 AngelScript 会执行小的垃圾收集循环来释放引用循环中的对象。

GameMonkey 支持全垃圾收集和增量垃圾收集。垃圾收集过程还可以依照检查每次调用所获得的子对象的个数来自定义，因此对大型的对象层级结构的标记也不会使系统停滞太久。还可以通过设定内存限制来选择什么时候运行垃圾收集器。一个软限制会在超过限制数量的时候告知虚拟机，这样就会启动增量的垃圾收集。可以通过一个硬限制来规定开始全垃圾收集过程的时机。

Lua，从 5.1 版本开始，也开始使用增量垃圾收集。这是一个被要求了很长时间才被加入的功能，因为 Lua 的很多性能问题都是因为使用全垃圾收集造成的。

如果增量垃圾收集还无法满足我们的帧率需求，GameMonkey 和 Lua 提供了在每一次逻辑记号结束后立即执行垃圾收集的函数，这样，可以避免类似于在几帧中间定格的情况发生。

- 可自定义的内存管理器

在游戏机平台上的内存是有限的。正因为这样，一些内存问题，例如将系统内存破碎成小块将会对性能造成毁灭性打击，另外没有足够的连续内存还会造成定位失败。如果游戏所使用的脚本语言是从系统中直接定位内存的话，那么对内存定位的完全控制就会失去，同时，破碎方法还会导致灾难性的应用失败。为了解决这些问题，游戏机游戏经常会使用一个合适的与操作系统独立的内存管理器来定位和释放内存。

GameMonkey 和 AngelScript 允许简单整合的自定义内存管理，并且在内存需要为一个对象保留或释放的时候进行回调。为 Lua 和 Python 增加一个自定义的内存定位系统会更加复杂一些，因为需要改变它们的内部代码。但是，由于它们都是公开源代码的版本，并且文档工作做得很好，因此为 Lua 和 Python 添加自定义的内存管理也是一个可行的方案。

2. 分析

Python 在他的分析任务标准库中为开发者提供了两个模块。Profile 模块允许运行一个函数，并且存储这个运行的分析数据。Pstats 模块提供了检查这些分析数据的方法。

最新版本的 Python 将 Profile 模块替换成了 Hotshot 模块，Hotshot 由 C 语言编写并且提供更加无缝的操作。另外值得一提的模块还有 Trace 和 Timeit 模块，能够跟踪整个脚本的执行并且提供一个很精细的时间测量。此外，还有一次正在开发与测试的 Python 分析器。

在 Lua 和 AngelScript 中的分析可以通过使用上边解释过的 hook 来收集函数或者代码片断的时间信息。LuaProfiler 与 Lua 代码打包。通过它，可以十分简单地得到每一个函数调用的记录和消耗的时间。

4.1.6　开发支持特点

本小节我们会试着解释一些游戏开发者必须注意的问题，因为这些问题会在使用某种语言的结果上造成很大的不同。

多功能操作系统和 CPU 现在是一个标准；然而，开发一个最合适的、好用的多线程应程序的确是一项很难的工作。提供多线程开发特点和功能的语言可以很大程度上帮助提高开发

Download at www.Pin5i.Com

者的效率。

除错器（debugger）和分析器也是一种语言能提供的最重要的功能。好的工具是成功开发一个游戏项目非常重要的因素，但是开发者通常并没有时间，也没有资源来建立一套合适的开发工具。如果脚本语言对内存管理和多线程有很好的内嵌支持——开发者就能节省他们大量的编码时间。

1. 多线程

Python，在它的最近标准版中，还没有达到线程安全。它通过使用一个叫做 GIL 的全局锁定来阻止访问共享资源。这种排外的访问方式是多重操作的需要，就像访问一个全局变量或调用一个 C 函数。

Python 和 Lua 都会提供模拟多线程系统的类。这些类就是 Pyhon 中的 Generator 和 Lua 中的 Coroutines。它们可以实现其他函数的连续执行，但是并不能由系统完成按时间排序。同样的方法也被应用于 GameMonkey 的一个本来就有的线程支持中。GameMonkey 的线程支持也是该语言设计的一个目标，因为这样，GameMonkey 中对线程的支持比其他任何语言都要强。GameMonkey 中的线程可以被阻挡或者设定为休眠，直到某一个游戏或系统事件唤醒它们。

在 Lua 中，如果多重脚本需要使用不同的 Lua 状态，它们也可以在同一时间被多线程执行，因此，有可能为每一个游戏平行地执行一个 Lua 脚本。GameMonkey 也可以支持多线程执行，因为它的函数是线程安全的。但是基于 flex 的脚本的编译是非线程安全的，并且有必要对脚本进行预编译或对游戏引擎中的脚本编译器实行排他式访问设定。

2. Debugging 工具

Debugging 在传统意义上讲是不包含在脚本语言中的，因此在脚本中查找错误会是一种困难且耗时的工作。由于脚本语言被越来越多的应用到游戏环境中，因此拥有一个好的 debugging 工具就更为重要。

Python 有一些现成的模块，例如 Logging 包，能够帮助程序员解决 debugging 问题。Pdb 模块支持建立断点，分级通过代码或检查堆栈桢。这些模块所提供的类扩展性很高，可以让用户根据需要自定义他们的 debugging 功能。

另外，还可以寻求一些外部的 debugger，HAP 就是最著名的一个。HAP 是由 Humongous Entertainment™ 开发的。HAP 有很多功能并且已经被 Python 的使用者，尤其是游戏开发者接受。

Lua 并没有提供任何内建的 debugging 工具。取而代之，Lua 提供 hook 和帮助函数来获取运行堆栈的信息或者在函数未被激活的情况下访问本地变量。另外还有一些 hook，能在函数调用、回归、Lua 函数中新的一行执行，以及大量的指导被执行的时候加入自定义行为。通过这些工具，自建一个 debugger 来简化 debug 工作也不是很困难的（实际上，有一些第三方的 Lua debugger 存在）。如果我们使用的是 LuaPlus 作为绑定接口，我们就能利用它的远程 debugger 作为插件被使用的基于 Windows 的开发环境。

GameMonkey 原本就可以在代码中加入断点。另外，还可以在 GameMonkey 代码中设定断言以保证某一个条件为真，并抛弃其他异常的情况，这个结果会被应用程序获得并且提供

Download at www.Pin5i.Com

一个消息或其他的引擎相关的任务。

GameMonkey 还能在某种特殊情况下为虚拟机提供回调，例如：一个异常被剔除、垃圾收集开始、一个线程的建立，或是一个线程的毁坏，等等。另外，标准 GameMonkey 发布版本中还有一个 debugger 的例子。尽管这个例子比较简单，但是它可以通过一些机制的提供而进行扩展。

AngelScript 依靠回调来提供 debugging 支持。对于每一个执行的语句，都会有一个回调函数，这个函数可以进行任何的 debug 处理，例如在断点暂停执行。另外，最近将会支持检查变量值。

单元测试是最近发展出来的 debug 的另一种机制。在 Python 和 Lua 中都得到了实现。PyUnit 是 Python 标准库中的一个框架。Lunit 和 Luaunit 是 Lua 使用者为了对 Lua 程序进行单元测试而开发的工具。Luaunit 是基于 PyUnit 框架建立的。

4.1.7　总结

本节介绍了 4 种不同的脚本语言——Python、Lua、GameMonkey 和 AngelScript——开发者可以为他们自己的游戏项目选择最合适的一种。

Python 并未被构想成为一个单纯的扩展语言，而扩展语言却是 Lua、GameMonkey 和 AngelScript 的设计初衷。这就使得 Pyhton 在提供功能方面看起来要更加完整更加复杂，尽管它实际的功能性并不比 Lua 和 GameMonkey 强太多。在任何情况下，都需要综合考虑项目的特点以及目标平台，内存消耗等因素来进行选择。

GameMonkey 和 AngelScript 有一个类似于 C++的句法结构。这可能会使非程序员接触起来更困难一些，这些人员可能对伪代码的句法结构更为熟悉。但是，相似的句法结构能够帮助程序员进行 C++代码和脚本的过渡。大部分的复杂脚本都需要至少一名程序员的支持，而改变句法结构就会有可能导致错误和拖慢，这是开发者处在紧迫时间表的压力下所无法承受的。根据工作室的角色分配不同，C++句法结构可能是一个需要考虑的重要因素。

传统上来说，脚本编程是与困难并且乏味的工作联系在一起的，而这正是由于缺乏合适的工具所导致的。今天这个说法再也不成立了。在 Ppython、Lua 和 GameMonkey 中你可以找到很多不错的编辑、debuggin，以及分析工具。AngelScript 尽管现在来说外部工具不是很多，但是随着该语言越来越受欢迎，更好的工具会使其编写游戏脚本更为方便。

4.1.8　参考文献

[AngelScript05] Jönsson, Andreas, “AngelCode Scripting Library (AngelScript).”2005. Available online at *http://www.angelcode.com/angelscript.*

[GameMonkey05] Riek, Matthew and Greg Douglas, “GameMonkey Script.” 2005.Available online at *http://www.somedude.net/gamemonkey*.

Download at www.Pin5i.Com

[Lua05] Ierusalimschy, Roberto, Luiz Henrique de Figueiredo, and Waldemar Celes,"The Programming Language Lua." 2005. Available online at *http://www.lua.org*.

[Python05] van Rossum, Guido, "Python Programming Language." 2005. Available online at *http://www.python.org*.

Download at www.Pin5i.Com

4.2　把 C++对象绑定到 Lua

Waldemar Celes, PUC-Rio
celes@inf.puc-rio.br
Luiz Henrique de Figueiredo, IMPA
lhf@impa.br
Roberto Ierusalimschy, PUC-Rio
roberto@inf.puc-rio.br

游戏中使用的脚本语言已经成为一种标准应用。脚本语言能够在游戏开发中扮演一种重要的角色，并且让数据结构化、计划事件、测试和调试这些工作更加容易。脚本语言也允许像美术、策划这些非程序专家通过一个高层的抽象脚本来为游戏编写代码。这个抽象层的一部分也允许提供给玩家来定制整个游戏。

从程序员的角度来看，把一个脚本语言嵌入到游戏中最主要的问题是，如何为脚本语言提供对宿主对象的访问（通常是 C/C++对象）。在选择一种脚本语言的时候有两个关键的特性：嵌入相关问题和绑定相关问题。而这些是 Lua 语言的设计初衷。但是，Lua 语言并没有提供任何自动创建绑定的工具，因为这是出于另外一种设计初衷：Lua 只是提供机制，而不是策略。

因而，就有许多种策略可以用来在 Lua 中绑定宿主对象。每一种策略都有它的优点和缺点，游戏开发者必须在得到在脚本环境中所需要的功能需求之后确定最好的策略。一些开发者可能只是把 C/C++对象映射成简单的数值，但是其他人可能需要实现运行期类型检查机制，甚至是在 Lua 中扩展宿主的应用。另外一个需要处理的重要问题是，是否允许 Lua 来控制宿主对象的生命周期。在本节中，我们将探究使用 Lua 的 API 来实现不同的宿主对象绑定策略。

4.2.1　绑定函数

为了说明不同策略的实现，让我们考虑把一个简单的 C++类绑定到 Lua 中。实现的目标是在 Lua 中实现对类的访问，因此允许脚本通过导出的函数来使用宿主所提供的服务。这里主要的想法是使用一个简单的类来引导我们的讨论。下面讨论的是一个虚构游戏中的英雄类，有几个将会被映射到 Lua 中的公用方法。

Download at www.Pin5i.Com

```
class Hero {
public:
    Hero (const char* name);
    ~Hero ();
    const char* GetName ();
    void SetEnergy (double energy);
    double GetEnergy ();
};
```

要把类方法绑定到 Lua 中，我们必须使用 Lua 的 API 来编写绑定功能。每一个绑定函数都负责接收 Lua 的值作为输入参数，同时把它们转化成相应的 C/C++值，并且调用实际的函数或者方法，同时把它们的返回值返还给 Lua。从标准发布版本的 Lua 中，Lua API 和辅助库提供了不少方便的函数来实现 Lua 到 C/C++值的转换，同样的，也为 C/C++到 Lua 值的转换提供了函数。例如，函数 luaL_checknumber 提供了把输入参数转换到相对应的浮点值的功能。如果参数不能对应到 Lua 中的数值类型，那么函数将抛出一个异常。相反的，函数 lua_ pushnumber 把给定的浮点值添加到 Lua 参数栈的顶端。还有一系列相类似的函数来映射其他的基本的 Lua 类型和 C/C++数据类型。我们目前最主要的目标是提出不同的策略来扩展标准 Lua 库和它为转换 C/C++类型对象所提供的功能。为了使用 C++的习惯，让我们创建一个叫做 Binder 的类来封装在 Lua 和宿主对象中互相转化值的功能。这个类也提供了一个把将要导出到 Lua 中的模块（类库）初始化的方法。

```
class Binder {

public:
    // 构造函数
    Binder (lua_State* L);
    // 模块（库）初始化
    int init (const char* tname, const luaL_reg* flist);

    // 映射基本数据类型的方法
    void pushnumber (double v);
    double checknumber (int index);
    void pushstring (const char* s);
    const char* checkstring (int index);
    ...
    // 映射类型化的宿主对象的方法
    void pushusertype (void* udata, const char* tname);
    void* checkusertype (int index, const char* tname);

private:
    lua_State* L;
};
```

类的构造函数接收 Lua_state 来映射对象。初始化函数接收了将被限制的类型名，也被表示为库的名称（一个全局变量名来表示在 Lua 中的类表），和绑定函数列表。已实现方法到基本类型的映射，并且直接调用了标准的 Lua 库。例如，映射一个数值到 Lua 中，或者从 Lua 映射出来的方法可能是这样的：

Download at www.Pin5i.Com

```
void Binder::pushnumber (double v) {
    lua_pushnumber(L,v);
}

double Binder::checknumber (int index) {
    return luaL_checknumber(L,index);
}
```

真正的挑战来自把用户自定义类型互相转换的函数：pushusertype 和 checkusertype。这些方法必须保证映射对象的绑定策略和目前使用中的一致。每一种策略都需要不同的库的装载方法，因而要给出初始化方法 init 的不同实现。

一旦我们有了一个 binder 类的实现，那么绑定函数的代码是非常容易编写的。例如，绑定函数相关类的构造函数和析构函数的代码如下：

```
static int bnd_Create (lua_State* L) {
    LuaBinder binder(L);
    Hero* h = new Hero(binder.checkstring(L,1));
    binder.pushusertype(h,"Hero");
    return 1;
}
static int bnd_Destroy (lua_State* L)
{
    LuaBinder binder(L);
    Hero* hero = (Hero*) binder.checkusertype(1,"Hero");
    delete hero;
    return 0;
}
```

同样的，与 GetEnergy 和 SetEnergy 方法的绑定函数能够像如下编码：

```
static int bnd_GetEnergy (lua_State* L) {
    LuaBinder binder(L);
    Hero* hero = (Hero*)binder.checkusertype(1,"Hero");
    binder.pushnumber(hero->GetEnergy());
    return 1;
}
static int bnd_SetEnergy (lua_State* L) {
    LuaBinder binder(L);
    Hero* hero = (Hero*)binder.checkusertype(1,"Hero");
    hero->SetEnergy(binder.checknumber(2));
    return 0;
}
```

注意绑定函数的封装策略将被用于映射对象：宿主对象使用对应的 check 和 push 方法组来进行映射，同时这些方法也用于接收关联类型为输入参数。在我们为所有的绑定函数完成编码后，我们可以来编写打开库的方法：

```
static const luaL_reg herolib[] = {
    {"Create", bnd_Create},
```

Download at www.Pin5i.Com

```
    {"Destroy", bnd_Destroy},
    {"GetName", bnd_GetName},
    {"GetEnergy", bnd_GetEnergy},
    {"SetEnergy", bnd_SetEnergy},
    {NULL, NULL}
};

int luaopen_hero (lua_State* L) {
    LuaBinder binder(L);
    binder.init("Hero",herolib);
    return 1;
}
```

4.2.2 绑定宿主对象和 Lua 数值

将 C/C++对象和 Lua 绑定的方法就是把它的内存地址映射成轻量级的用户数据。一个轻量级用户数据可以用指针来表示（void *）并且它在 Lua 中只是作为一个普通的值。从脚本环境中，能够得到一个对象的值（它的指针），做比较（一个轻量级用户数据等同于任何其他轻量级用户数据和相同的 C/C++地址），并且能够把它传回给宿主。我们要在 binder 类中所实现的这个策略所对应的方法通过直接调用在标准库中已经实现的函数来实现：

```
void Binder::init (const char* tname, const luaL_reg* flist) {
    luaL_register(L,tname,flist);
}

void Binder::pushusertype (void* udata, const char* tname) {
    lua_pushlightuserdata(L,udata);
}

void* Binder::checkusertype (int index, const char* tname) {
    void* udata = lua_touserdata(L,index);
    if (udata==0) luaL_typerror(L,index,tname);
    return udata;
}
```

函数 luaL_typerror 在上面的实现中用于抛出异常，指出输入参数没有一个有效的相关对象。

通过这个映射我们英雄类的策略，以下的 Lua 代码便是可用的：

```
local h = Hero.Create("my hero")

local e = Hero.GetEnergy(h)
Hero.SetEnergy(h,e-1)

Hero.Destroy(h)
```

把对象映射成简单值至少有 3 个好处：简单、高效和小的内存覆盖。就像我们上面所见到的，这种策略是很直截了当的，并且 Lua 和宿主语言之间的通信也是最高效的，那是因为

Download at www.Pin5i.Com

它没有引入任何的间接访问和内存分配。然而，作为一个实现，这种简单的策略因为用户数据的值始终被当成有效的参数而变得不安全；传入任何一个无效的对象都将会导致宿主程序的直接崩溃。

加入类型检查

我们能够实现一个简单的这时类型检查机制来避免在 Lua 环境中导致宿主程序崩溃。当然，加入类型检查会降低效率并且增加了内存的使用。如果脚本只是用在游戏的开发阶段，那么类型检查机制可以在发布之前始终关闭。换句话说，如果脚本工具要提供给最终用户，那么类型检查就变得非常重要而且必须和产品一起发布。

要添加类型检查机制到绑定值的策略中，我们能够创建一个把每一个对象和 Lua 相对应类型名字映射的表（在这篇文章中所有提到的策略里，我们都假定地址是宿主对象的唯一标识）。在这张表中，轻量级的用户数据可以作为一个键，而字符串（类型的名称）可以作为值。初始化方法负责创建这张表，并且让它能够被映射函数调用。然而，保护它的独立性也是非常重要的：从 Lua 环境中访问该表是不被允许的；另外，它仍然有可能在 Lua 脚本中使宿主程序崩溃。使用注册表来存储确保该表保持独立性是一种选择，它是一个全局的可以被 Lua API 单独访问的变量。然而，因为注册表是唯一且全局的，用它来存储我们的映射对象也阻止了其他的 C 程序库使用它来实现其他的控制机制。另一种更好的方案是只给绑定函数提供访问类型检查表的接口。直到 Lua5.0，这个功能才能实现。在 Lua5.1 中，有一种更好的（而且更高效）方法：环境表的使用直接和 C 函数相关。我们把类型检查表设置成绑定函数的环境表。这样，在函数里，我们对表的访问就非常高效了。每一个函数都需要注册到 Lua 中，从当前的函数中去继承它的环境表。因而，只需要改变初始化函数的环境表关联就足够了——并且所有注册过的绑定函数都会拥有同样关联的环境表。

现在，我们可以对 **binder** 类的执行类型检查的方法进行编码了：

```
void Binder::init (const char* tname, const luaL_reg* flist) {
    lua_newtable(L); //创建类型检查表
    lua_replace(L,LUA_ENVIRONINDEX); //把表设置成为环境表
    luaL_register(L,tname,flist); //创建库表
}

void Binder::pushusertype (void* udata, const char* tname) {
    lua_pushlightuserdata(L,udata); //压入地址
    lua_pushvalue(L,-1); //重复地址
    lua_pushstring(L,tname); //压入类型名称
    lua_rawset(L,LUA_ENVIRONINDEX); // envtable[address] =类型名称
}

void* Binder::checkusertype (int index, const char* tname) {
    void* udata = lua_touserdata(L,index);
    if (udata==0 || !checktype(udata,tname))
        luaL_typerror(L,index,tname);
    return udata;
}
```

该代码使用一种私有的方法来实现类型检查：

Download at www.Pin5i.Com

```
int Binder::checktype (void* udata, const char* tname) {
    lua_pushlightuserdata(L,udata); //压入地址
    lua_rawget(L,LUA_ENVIRONINDEX); //得到 envtable[address]
    const char* stored_tname = lua_tostring(L,-1);
    int result = stored_tname && strcmp(stored_tname,tname)==0;
    lua_pop(L,1);
    return result;
}
```

通过这些做法，我们使得绑定策略仍然非常高效。同样的，内存负载也非常低——所有对象只有一个表的实体。然而，为了防止类型检查表的膨胀，我们必须在销毁对象的绑定函数中释放这些表。在 bnd_Destroy 函数中，我们必须调用这个私有方法：

```
void Binder::releaseusertype (void* udata) {
    lua_pushlightuserdata(L,udata);
    lua_pushnil(L);
    lua_settable(L,LUA_ENVIRONINDEX);
}
```

4.2.3 绑定宿主对象和 Lua 对象

把宿主对象绑定到 Lua 中的轻量级用户数据中，尽管简单而且高效，但它仍然会有很多限制。因为把宿主对象映射成 Lua 中的普通值，所以我们无法把它看成一个对象；同样的，我们也无法使用面向对象的语法来引用它的方法，而且在 Lua 中无法去管理对象的生命周期（Lua 中有垃圾回收的功能，我们可以设置当 C/C++对象不再被引用之后，让 Lua 自动地销毁它们）。需要把宿主对象和 Lua 对象对应起来，我们就需要采用另外一种绑定策略。我们可以把一个对象对应到完整的用户数据中，从而取代把对象映射成轻量级的用户数据的策略。Lua API 提供了一个叫做 lua_newuserdata 的函数允许我们来创建一个内存数据块，并且把它作为一个不透明的对象（用户数据类型）映射到 Lua 中。在 Lua 中，无法区分一个轻量级用户数据和一个完整的用户数据。然而，宿主能够把一个完整的用户数据和元数据表关联起来，例如定义特定的行为。不同的对象将会设置成不同的元数据表，当然，相同的对象类型会对应同一张元数据表。建立类型检查机制可以利用已经提供这样功能的辅助函数库，通过函数 luaL_newmetatable 和 luaL_checkudata 实现。对于每个已经创建的元数据表，辅助函数库都会在注册表中添加一个表项，把类型名和元数据表映射起来。这样我们只要给出类型名，就能很轻易的访问到相对应的元数据表了。

在我们的这种情形下，用户数据将按照指针的大小来创建以存储对象的地址。我们可以让用户数据从库中的表继承，这样，无论什么时候用户数据都是被索引的，相对应的库中的表也将被随时访问到。这就允许我们去使用面向对象的语法，就像以下代码一样：

```
local h = Hero.Create("my hero")

local e = h:GetEnergy()
h:SetEnergy(e-1)
```

Download at www.Pin5i.Com

此外，我们可以针对元事件（元表的__gc 字段）编写垃圾回收代码来销毁对应的宿主实例；并且不需要去精确调用 Destroy 方法。不论什么时候收回用户数据，Lua 都会自动地触发相关联的元事件。

使用完整的用户数据表示边界对象将会带来新的挑战。我们不能在一个对象映射到 Lua 中时每次都创建一个新的用户数据。我们必须保证唯一性。如果同一个对象被第二次映射到 Lua 中，我们必须使用相同的用户数据。如果我们使用了不同的用户数据，那么将会引起二义性：Lua 中的两个不同的对象映射到同一个宿主对象上。这样做的效率也非常低下，因为每一次映射都必须创建一个新的动态对象。因此，我们必须跟踪每一个映射到 Lua 中的对象。同样的，这可以用环境表和绑定函数相关联来实现。这个表可以把每一个对象的地址和完整的用户数据关联起来。如果需要 Lua 仍然能够自动地根据用户数据来收集宿主对象，我们就必须把表设置成拥有弱值——就是这个表要保持它的值的一个弱引用(用户数据)。如果这个表是一个普通表，那么用户数据将不会被收集，因为它可能始终只被它自己引用。弱引用将不被垃圾收集器所考虑。

初始化函数现在接收一个额外的参数：和垃圾回收事件（__gc）相关联，用以销毁宿主对象的绑定函数。

```
void Binder::init( const char* tname, const luaL_reg *flist, int (*destroy)
                  (lua_state*) ){
    lua_newtable( L );                    // 创建一个唯一的表
    lua_pushstring(L, "v");
    lua_setfield( L, -2, "__mode" );      // 设置成弱值表
    lua_pushvalue( L, -1 );               // 复制表
    lua_setmetatable( L, -2 );            // 把它自己设置成元表
    lua_replace( L, LUA_ENVIRONINDEX );// 设置表为事件表
    luaL_register( L, tname, flist );     // 创建 libtable
    luaL_newmeatatable( L, tname );          // 为对象创建元表
    lua_pushvalue( L, -2 );
    lua_setfield( L, -2, "__index" );     // mt.__index=libtble
    lua_pushcfunction(L,destroy);
    lua_setfield( L, -2, "__gc" );        // mt.__gc = destroy
    lua_pop( L, 1 );                      // 弹出元表
}
```

这个函数把用户数据压入到 Lua 的栈顶并首先检查这个对象是否已经被映射了。如果没有，则它会创建一个新的用户数据并且把它映射到环境表中。如果对象已经在 Lua 中存在了，这个函数会返回存储在环境表中对应的用户数据。这个函数直接使用辅助函数库中提供的 luaL_checkudata 功能去检查给定用户数据的类型。

```
void Binder::pushusertype (void* udata, const char* tname) {
    lua_pushlightuserdata(L,udata);
    lua_rawget(L,LUA_ENVIRONINDEX);          // 从环境表中得到 udata
    if (lua_isnil(L,-1)) {
        void** ubox = (void**)lua_newuserdata(L,sizeof(void*));
        *ubox = udata;                       // 把 udata 中的地址保存起来
        luaL_getmetatable(L,tname);          // 获得元素
        lua_setmetatable(L,-2);              // 把元素赋给 udata
```

Download at www.Pin5i.Com

```
            lua_pushlightuserdata(L,udata);     // 压入地址
            lua_pushvalue(L,-2);                // 压入 udata
            lua_rawset(L,LUA_ENVIRONINDEX);     // envtable[address] = data
        }
    }

    void* Binder::checkusertype (int index, const char* tname) {
        void** udata = (void**)luaL_checkudata(L,index,tname);
        if (udata==0)
            luaL_typerror(L,index,tname);
        return *udata;
    }
```

把宿主对象绑定到 Lua 对象上会带来不少好处，特别是因为宿主对象能够被看作 Lua 中的任何其他对象。我们使用相同的语法来响应垃圾收集器去释放已经分配的内存。而且，性能损耗只会明显地体现在一个对象的第一次映射（这是因为创建了一个新的用户数据）。内存大小有一种可以理解的限制：每一个对象都对应一个用户数据（存储在一个单一的指针中）和一个表项（保证唯一性）。

1. 加入继承

让我们现在来考虑一个已经存在的类，Actor，它是 Hero 类的父类。要说明这个论述，我们必须把 GetName 方法移到基类中：

```
Class Actor {
public:
    Actor (const char* name);
    ~ Actor ();
    const char* GetName ();
};

class Hero : public Actor {
    ...
};
```

在把这两个类绑定到 Lua 中后，我们自然会在子类中调用父类的方法。因此，Lua 代码如下编写将能够实现：

```
local h = Hero.Create("my hero")
local n = h:GetName()
```

如果使用了到目前为止我们所讨论过的代码，那么我们在调用上面语句 GetName 会得到一个错误：需要一个 Actor 类的对象和一个将接收的 Hero 对象。我们的类型检查机制并没有记录类的体系结构。不过，在我们的绑定代码中加入多态和继承是非常容易的。

首先，我们要修改初始化函数：现在它需要接收基类的类型名称（bname）。接下来，我们在子类的元表里保存和基类相关联的元表 —— 比如说，使用_base 字段。这对于类型检查机制是必须的。最后，我们让保存表从基类的保存表中继承。这些附加的设置已经在之前扩展初始化函数编码中使用到了。

Download at www.Pin5i.Com

```
    ...
    lua_pushcfunction(L,destroy);
    lua_setfield(L,-2,"__gc");          //mt.__gc = destroy
    if (bname) {
        luaL_getmetatable(L,bname);
        lua_setfield(L,-2,"_base");     //mt._base = base mt
    }
    lua_pop(L,1);                       //弹出 metatable
    if (bname) {
        lua_getfield(L,LUA_GLOBALSINDEX,bname);
        lua_setfield(L,-2,"__index");   //libtable.__index = baselib
        lua_pushvalue(L,-1);            //复制 libtable
        lua_setmetatable(L,-2);         //把自己设置成 metatable
    }
}
```

对于检查一个给定参数的类型，我们可以不再使用辅助函数库中所提供的功能，因为当失败的时候，我们还要去检查方法是不是属于基类，并且跟踪保存在元表中的_base 字段的基类链。以下代码实现了这个功能：

```
void* Binder::checkusertype (int index, const char* tname) {
    lua_getfield(L, LUA_REGISTRYINDEX, tname); // 得到 mt 类型
    lua_getmetatable(L,index); // 得到相关的 mt
    while (lua_istable(L,-1)) {
        if (lua_rawequal(L,-1,-2)) {
            lua_pop(L,2);
            return *((void**)lua_touserdata(L,index));
        }
        lua_getfield(L,-1,"_base"); // get mt._base
        lua_replace(L,-2); // replace: mt = mt._base
    }
    luaL_typerror(L,index,tname);
    return NULL;
}
```

能够实现从基类的继承也表明了用宿主对象绑定到 Lua 对象来取代把它们看成普通数据以后获得的强大功能。

2. 添加扩展性

我们能够更进一步。Lua 是一个强大的脚本语言，因为它提供了易于使用，却非常精妙的机制来构造我们的代码和数据。这些高级数据结构的原理和实现在 Lua 显得那样简单，主要是依赖于对表（tables）的使用：相关联的数组可以索引任何值（除了 nil）。对于表（tables）的使用，实际上，极大的简化了对于复杂数据的描述，并且如果我们能够把这种能力加入到我们的绑定系统中将会有特别好处。允许脚本把宿主对象当做一个相关联的数组是我们的目标：能够为对象添加新的属性，重新定义已有的方法，或者扩展对象的行为：

```
local h = Hero.Create("my hero")
```

Download at www.Pin5i.Com

```
h.myFactor = 2 -- stores an extra attribute
h.GetEnergy = -- redefine the GetEnergy method
    function (h)
        return Hero.GetEnergy(h) * h.myFactor
    end

h:GetEnergy() -- call the modified method
```

我们可以通过把 Lua 中的表和对象关联起来以提供这样的扩展性。为了允许为对象添加一个新的字段，我们使用对应的元事件（__newindex 事件）来把新的字段存储到相关联的表中。同时，我们修改了访问元事件（__index 事件）来查找在相关联表中的字段。在 Lua 5.1 中，存储额外字段的关联表，可以像环境表那样和用户数据关联起来。一开始，任何新的从环境表中继承的用户数据都是当前的 C 函数。当__newindex 事件第一次被触发的时候，就把一个新的环境表赋给用户数据。使用这个策略，如果脚本真的使用到了它，我们也只需要创建一个新的表。这种事件的处理方法可以如下编码：

```
static int newindex (lua_State* L) {
    lua_getfenv(L,1); //得到环境表
    if (lua_equal(L,-1,LUA_ENVIRONINDEX)) {
        lua_newtable(L); //创建一个新的表
        lua_pushvalue(L,-1); //复制一个表
        lua_setfenv(L,1); //把环境表和用户数据关联起来
        lua_pushvalue(L,-1); //复制环境表
        lua_setmetatable(L,-2); //把它自己设置成元表
        lua_getfield(L,LUA_ENVIRONINDEX,"__index"); //get libtable
        lua_setfield(L,-2,"__index"); //envtable.__index=libtable
    }
    lua_pushvalue(L,2);
    lua_pushvalue(L,3);
    lua_rawset(L,-3); //把键保存到环境表中
    return 0;
}
```

这个策略的一个缺点就是在访问宿主方法的时候会导致性能瓶颈，因为我们必须首先检查该字段已经在环境表中存在。__index 元事件无法直接映射到库表中，因此必须通过编写一个函数来访问相应的环境表：

```
static int index (lua_State* L) {
    lua_getfenv(L,1);
    lua_pushvalue(L,2);
    lua_gettable(L,-2);    // 访问环境表
    return 1;
}
```

4.2.4 宿主和 Lua 表绑定

当一个应用程序需要频繁的添加额外的字段时，这是一个更加简单并且自然的策

Download at www.Pin5i.Com

略。这个策略把宿主对象和 Lua 表（Table）直接绑定。我们为每一个对象创建一个表，从而取代为每个对象创建一个完整的用户数据。宿主对象的引用会以轻量级用户数据的形式指定给预先定义的字段（比如，__pointer）并保存在表中。由于表在 Lua 中已被看成是一个对象，所以之前讨论的针对用户数据的所有特性都可以在表中应用——如面向对象的语法使用、自动垃圾收集和对额外属性的存储，都能够自然地处理。我们也必须确保唯一性，在代码处理上也和使用完整用户数据的策略相似。初始化函数也和使用完整数据的策略非常相似——就是不需要去为__index 和__newindex 事件编写代码。最主要的不同在于 push 和 check 对象的函数（更加精确，这种方式对象指针（void*）被保存并索引）：

```
void Binder::pushusertype (void* udata, const char* tname) {
    lua_pushlightuserdata(L,udata);
    lua_rawget(L,LUA_ENVIRONINDEX);          //得到在环境表中的对象
    if (lua_isnil(L,-1)) {
        lua_newtable(L);                     //为对象创建一个新的表
        lua_pushlightuserdata(L,udata);      //压入地址
        lua_setfield(L,-2,"_pointer");       // table._pointer=address
        luaL_getmetatable(L,tname);          //得到元表
        lua_setmetatable(L,-2);              //为 table 设置元表
        lua_pushlightuserdata(L,udata);      //压入地址
        lua_pushvalue(L,-2);                 //压入表
        lua_rawset(L,LUA_ENVIRONINDEX);      // envtable[addr]=table
    }
}
```

```
void* Binder::checkusertype (int index, const char* tname) {
    lua_getfield(L, LUA_REGISTRYINDEX, tname); // 得到 mt 的类型
    lua_getmetatable(L,index); //得到相关联的 mt
    while (lua_istable(L,-1)) {
        if (lua_rawequal(L,-1,-2)) {
            lua_pop(L,2); // pop string and mt
            lua_getfield(L,index,"_pointer"); // 获得地址
            void* udata = lua_touserdata(L,-1);
            return udata;
        }
        lua_getfield(L,-1,"_base");     //得到 mt._base
        lua_replace(L,-2);              // replace: mt = mt._base
    }
    luaL_typerror(L,index,tname);
    return NULL;}
```

这个策略的唯一缺点是，和扩展的完整用户数据策略相比，在没有给它添加额外的字段的情况下，绑定一个对象的时候需要分配更多的内存。

4.2.5　总结

我们已经讨论了把宿主对象绑定到 Lua 中的不同的策略。这里面并没有最好的策略。每

Download at www.Pin5i.Com

一个应用程序都会有它的特点，根据比较和衡量来做出选择。Lua 提供机制，但不提供任何策略。然而，任何把所有绑定策略放到一起的比较都至少有这样 3 个论点：灵活性、效率和内存需求。为了能够帮助您在这些策略中选择一个合适的，我们已经按照这 3 个论点针对文中提到策略做了一个比较。

首先，在表 4.2.1 中，我们统计了这些不同的策略所能够提供的特性。当然，所有策略都提供了从脚本环境中对边界对象的直接访问。只有提供了类型检查的策略才能够保证安全的访问。把宿主对象和 Lua 对象绑定允许在 Lua 中能够使用面向对象的编程（和继承）方法。最后，其中两个策略允许把对象看作相关联的数组，因而实现了对对象的扩展。

表 4.2.1　　现有策略约束的特性

Strategy	Access	Type-Checking	OO	Extensibility
轻量用户数据	√			
轻量用户数据类型	√	√		
完整用户数据	√	√	√	
可扩展用户数据	√	√	√	√
表	√	√	√	√

为了测试效率，我们使用不同的策略去调用同一个简单的方法。我们也比较了在最后两个策略中被支持的处理额外属性的效率。我们没有对最终结果进行解释，得到这些结果只是为了做一个提示；它们也可能在您的机器上有细微的不同。这些测试将在实际的应用程序中分开进行。因此，对一个实际的应用程序性能的真正影响并没有考虑。和在宿主语言中执行相比，大多数的时间花在绑定后的方法中（如访问、类型检查和映射）。表 4.2.3 所报告的时间是调用过表 4.2.2 中的代码 100 万次而得到的。

表 4.2.2　　用于测试不同策略效率所用的代码

Method Invocation		Extra Attribute Access
For Binding Objects To Values	For Binding Objects To Objects	
for i = 1, N do 　Hero.SetEnergy(h,i) 　if Hero.GetEnergy(h)~=i 　then 　　error("failed") 　end end	for i = 1, N do 　h:SetEnergy(i) 　if h:GetEnergy()~=i 　then 　　error("failed") 　end end	for i = 1, N do 　h.myattrib = i 　if h.myattrib~=i 　then 　　error("failed") 　end end

表 4.2.3　　调用代码 100 万次耗时

Strategy	Time	
	Method Invocation	Attribute Access
轻量用户数据	0.52	N/A
轻量用户数据类型	0.77	N/A
完整用户数据	0.97	N/A
可扩展用户数据	1.42	0.61
表	1.53	0.16

最后，表 4.2.4 展示了每一个策略对内存的需求量。在该表中，*U* 表示一个完整的用户数据保存在指针中，*T* 表示一个最初的空白表，*E* 表示一个表项。

Download at www.Pin5i.Com

表 4.2.4　内存消耗比较

Strategy	Memory Consumption	
	Per Object	Extra Attributes
用户数据	—	N/A
轻量用户数据类型	E	N/A
完整用户数据	U + E	N/A
可扩展完整用户数据	U + E	T + E per attribute
表	T + E	E per attribute

一个最终，但是很重要的说明是：本节所列出的方法并未涵盖所有 Lua 绑定策略。还有很多其他的不同方法可以去测试，特别是对类体系结构的类型检查。本节最主要的目标就是提供一些 C/C++对象和 Lua 绑定的基本方法，以及一些对实际绑定有用的策略。我们也鼓励开发者去实现不同的策略，实现他们自己游戏中的实际需求。本节中所有列出来的代码都可以在本书的 CD-ROM 中找到。

4.2.6　参考文献

[Ierusalimschy03] Ierusalimschy, R., “Programming in Lua.” Lua.org, 2003. Availableonline at *http://www.lua.org*.

Download at www.Pin5i.Com

4.3 使用 LUA 协同程序实现高级控制机制

Luiz Henrique de Figueiredo, IMPA
lhf@visgraf.impa.br
Waldemar Celes, PUC-Rio
celes@inf.puc-rio.br
Roberto Ierusalimschy, PUC-Rio
roberto@inf.puc-rio.br

协同程序（coroutine）在游戏编程中是一种非常有效的机制。它对于一个多线程系统中的线程来说会更加简单，它使用它自己的本地执行栈，但是会和其他协同程序共享全局环境。而不像被操作系统管理并在理论上并发执行的标准线程，只有一个系统程序能够在指定时间内运行，就像一个标准的函数。但是和函数不同，协同程序能够在任意时候挂起并恢复。因此，协同程序提供一个并发执行的假象而没有任何的消耗和管理同步段的负担。这能够很明显的简化程序结构。

Lua 编程语言实现了协同程序：我们能够在以任意数量嵌套的函数调用中挂起一个协同程序。Lua 协同程序允许程序员实现很多高级的控制机制，像过滤器、迭代器和协作式多任务系统等。协同程序对于实现增量算法（在游戏编程中的一种通常任务）也非常有帮助。

本节我们将讲述如何使用 Lua 协同程序在游戏中实现一些高级的控制机制。我们集中讨论协同程序在 Lua 中的使用，并没有使用 C API。从宿主程序中管理 Lua 协同程序的简单例子，可以参考[Harmon05]。有关协同程序的详细讨论可参考[deMoura05]和[Ierusalimschg03]。

4.3.1 Lua 协同程序

协同程序在 Lua 中是第一类值。这意味着协同程序可以被看成和其他值一样：它能够赋值给变量、通过函数参数传值、用于表实体的键或值。Lua 在协同函数模块中提供函数来维护协同程序。我们调用 coroutine.create 来创建一个新的协同程序。这个函数把即将成为主函数的方法作为输入参数并且返回最新的协同程序。

Lua 协同程序可以有 3 种不同的状态：挂起（suspended）、运行（running）和死亡（dead）（见图 4.3.1）。在它创建以后，一个协同程序是挂起的——这就是说，并没有执行。要开始执行一个协同程序，需要调用 coroutine.

Download at www.Pin5i.Com

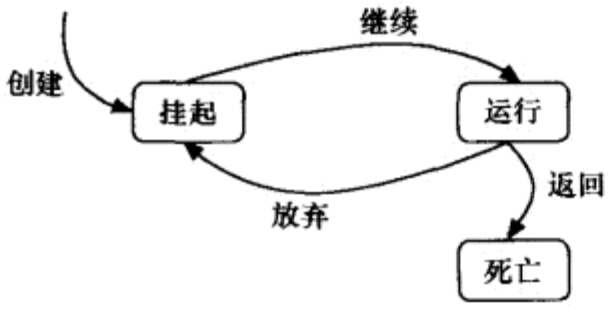


图 4.3.1　Lua 协同程序的状态

resume 函数，并且把协同程序实例作为它的第一个参数。这个协同程序的状态跟着就切换到了运行状态。要挂起一个协同程序，可以在它运行期调用函数 coroutine.yield。这样控制权又回到恢复协同程序的函数手中。一个挂起的协同程序可以再调用 coroutine.resume 恢复，并且它将从最近调用 coroutine.yield 挂起的点恢复运行。当协同程序的主函数返回的时候，协同程序的状态变成了死亡，并且无法再被恢复；控制权又回到了调用者手中。在任意时刻，我们可以通过 coroutine.status 来检索协同程序的状态。

Lua 实现了非对称式协同程序，因为两个不同的函数需要从一个协同程序中传递控制权：恢复和交出。协同程序从属于它的调用者（比如说，唤醒它的函数），而且控制权总是会被传回给调用者。非对称式协同程序，初看起来是一个限制机制，但实际上它非常的易用和强大。例如，非对称式协同程序非常易于在 Lua 协同程序的高层实现[deMoura04]。

Lua 也为协同程序和调用者交换数据提供了一种有效的方式。所有的参数通过在挂起的时候传入，并在对应的唤醒操作中作为返回值传出。相反的，所有的参数通过唤醒函数传入（除了第一个指定要被唤醒的协同程序）然后变成返回值从最后一个挂起函数中传出。作为一个特例，协同程序第一次被唤醒的时候，哪些参数会作为主函数的参数。

4.3.2　过滤器

协同程序的一个简单且有效的应用是过滤器。过滤器处理了这样一个数据流：变换输入数据次序，修改以后生成输出数据，如图 4.3.2 所示。

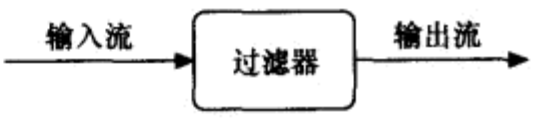


图 4.3.2

尽管这看起来很琐碎，但实际上过滤器是一个生产者-消费者链，一个经典的计算问题。使得这个问题看起来不那么琐碎的原因是生产者和消费者之间有着不一样的工作速度。协同程序能用一种比较自然的方式来管理这种不协调的速度。

事件处理在游戏中是一个简单的过滤器的例子：外部事件需要转换成游戏事件。考虑这样一个游戏中最普通的时间处理流程：

```
while true do
    local ev = GetNextEvent()
    ... -- handle event
end
```

函数 GetNextEvent 负责把外部事件映射成为游戏事件。当每一个外部事件对应单个的游戏事件的时候，实现就非常的直接。如果我们需要把多个外部事件合成为一个游戏事件的话，事件的实现同样也非常直接：在这个函数中，我们保持处理外部事件直到它们能够转换成一个有效的游戏事件。

最大的挑战似乎在于当一个外部事件要转换到两个以上游戏事件，因为 GetNextEvent 一次只能够返回一个事件。要保持代码简单，事件处理循环不能知道每一个游戏事件是如何生成的。这就是协同程序有用的地方。我们需要一种机制来返回已经生成的游戏事件中的下一个——这

Download at www.Pin5i.Com

就是协同程序的优点所在。于是我们能够编写一个事件过滤器当作协同程序。下面的代码处理以下情况：（a）一个外部事件映射成一个游戏事件；（b）两个外部事件映射成一个游戏事件；和（c）一个外部事件映射成两个游戏事件。

```
local EventFilter = function ()
    while true do
        local curr = GetExternalEvent()
        local kind = ClassifyEvent(curr)
        if kind == "one-to-one" then
            coroutine.yield(TranslateEvent(curr))
        elseif kind == "two-to-one" then
            local next = GetExternalEvent()
            coroutine.yield(CombineEvent(curr,next))
        elseif kind == "one-to-two" then
            local ev1, ev2 = SplitEvent(curr)
            coroutine.yield(ev1)
            coroutine.yield(ev2)
        else
            return nil
        end
    end
end
local GetNextEvent = coroutine.wrap(EventFilter)
```

这个协同程序我们调用函数 coroutine.warp 来创建，并同时每次返回一个能恢复协同程序的函数。通过执行这些操作，事件处理循环的编码就像调用一个常规的函数一样。

4.3.3 迭代器

迭代器的实现是另外一个在游戏开发中频繁出现的问题，协同程序同样能够非常方便地实现。迭代器用来实现遍历一个数据结构中存储的所有的元素。真正的迭代器是一个用来遍历（访问）数据结构的高层函数，并且允许通过调用一个指定的函数来处理每一个访问到的元素。由于函数在 Lua 中属于第一类值，所以非常易于实现一个真正的迭代器。让我们来看下面这样一个简单的字符串数组的例子，来继续我们的讨论：

```
local weapons = {"knife", "sword", "gun", "knife", ...}
```

最简单的遍历数组的（如一个数组迭代器）函数实现可以像如下编码。这个函数接收一个数组和一个要对数组元素进行调用的回调函数。

```
local iterator = function (array, cb)
    for i = 1, #array do
        cb(array[i])
    end
end
```

由于 Lua 提供了完整的词法范畴、闭合和匿名函数，使用它们非常简单地就能写出像迭

Download at www.Pin5i.Com

代器要求的那种回调函数。例如，一个统计指定的元素在数据结构中出现次数的函数迭代器，我们可以如下编码：

```
local count = function (container, elem)
    local n = 0
    iterator(container, function (value)
        if value==elem then
            n = n+1
        end
    end)
    return n
end
```

尽管上述的迭代器有效，但它并不是非常的有弹性。举例来说，它并不能同时遍历两个数据结构（例如，需要比较或者合并它们的元素）。此外，代码本身看起来也非常的不自然。程序语言提供了循环结构来遍历一个数据元素的集合，但如果我们能够规范遍历任意数据结构的访问行为的话，那么就能够使它更加灵活易用。正是出于这个原因，在 C++和 Java 中都提供了用于访问“下一个”数据结构元素的迭代器对象。在 Lua 中，用一个每次调用都返回下一个元素的函数来代表这样的迭代器。使用这种方案，我们可以重写上面提到的统计函数如下：

```
local count = function (container, elem)
    local n = 0
    local itr = elements_of(container)
    local value = itr()
    while value do
        if value == elem then
            n = n + 1
        end
        value = itr()
    end
    return n
end
```

这样，element_of 函数就负责创建并返回迭代器（itr）。而使用 Lua 语言提供的通用 for 语句，这些代码可以编写的简练得多（函数调用迭代器是隐藏在循环中的）：

```
local count = function (container, elem)
    local n = 0
    for value in elements_of(container) do
        if value == elem then
            n = n + 1
        end
    end
    return n
end
```

Download at www.Pin5i.Com

element_of 在 Lua 中同样很容易实现。就像上面的代码一样，迭代器必须要保存它的当前状态，使得它每次被调用时继续迭代。这可以在 Lua 的语法范畴内实现：

```
local elements_of = function (array)
    local i=0
    return function () i = i+1 return array[i] end
end
```

所有这些构造都非常的简单。当我们需要为复杂的数据结构创建迭代器的时候，困难随之而来。在这种情况下，迭代器保持跟踪复杂状态，并在语法范畴内不能工作。这就是协同程序能发挥它们的地方。从任意层数的嵌套调用的内部都可以挂起一个协同程序的执行——当然也包括递归调用。让我们考虑一个使用场景图描述虚拟世界的游戏，从而展示如何使用协同程序来遍历层次数据结构。这样一个场景图可以很自然地使用 Lua 的表构造函数来编写，如下所示：

```
myscene = Scene {
    name = "my scene",
    LightSource {
        position = {0.0,5.0,0.0},
        cutoff = 60,
    },
    Transform {
        translation = {1.0, 0.0, 0.0},
        rotation = {45, 0.0,0.0,1.0},
        Entity {
            geometry = Sphere{radius=1.0},
            appearance = Texture{image="myimage.tif"},
        },
        ...
    }
    ...
}
```

使用协同程序，创建一个遍历场景图中所有节点的函数就和编写一个递归访问场景图的函数一样简单。不需要在访问每个元素的时候调用一个回调函数，只需简单的挂起执行，把相对应的元素值作为参数传入：

```
local function traverse (graph)
    coroutine.yield(graph)
    for i=1,#graph do
        traverse (graph[i])
    end
    return nil
end
```

迭代器本身是由恢复协同程序的函数给出的。

```
local nodes_of = function (graph)
    return coroutine.wrap(function () traverse(graph) end)
end
```

Download at www.Pin5i.Com

使用这样一个迭代器，访问所有场景图中节点的循环有一个标准的构造：

```
for n in nodes_of(myscene) do
    -- process node n
    ...
end
```

4.3.4　任务安排

协同程序的另外一个自然的应用是任务安排。这个想法需要创建能容纳多个任务的一个安排表对象。因此安排表对象需要提供按照次序执行已经注册过任务的功能。每一项任务都被实现成一个协同程序，在进行了一些计算后挂起它的执行过程。协同程序实现了非独占式的多线性，因此它需要每一个任务都能够精确挂起自己的执行过程。（非独占式的多线程在消除同步错误上有很大的优势，它能够实现在临界区之外来挂起执行。）

下面的代码展示了如何在 Lua 协同程序的基础上实现一个任务安排表。安排表被表现成控制注册过任务集合的一个对象。我们要为每一项任务给定一个相关的名称。

```
local scheduler = {
    tasks = {}
}

function scheduler:addtask (name, task)
    self.tasks[name] = coroutine.create(task)
end
```

安排表的主循环函数遍历所有的任务并按次序执行它们。在每个任务执行完成以后，安排表会去检查相应的任务协同程序是否已经死亡，如果是，那么将把它从安排表中删去。

```
function scheduler:loop ()
    repeat
        for name,co in pairs(self.tasks) do
            coroutine.resume(co)
            if coroutine.status(co) == "dead" then
                self.tasks[name] = nil
            end
        end
    until not next(self.tasks)
end
```

注意在任务运行时的次序是没有定义的，因为 pairs 可以是任意次序。如果我们确定要指定一个特定的次序，那么我们就必须把任务保存到一个数组中。

主循环函数同样能够用协同程序来实现。这样做的话，我们就可以把其中一个安排表的主循环函数加入到另外一个安排表中，这样我们就实现了多层的任务安排表。

4.3.5　协作式多线程

协同程序能够允许建立一种更加详细的控制机制。下面我们就来举一个例子，将 Lua 协

Download at www.Pin5i.Com

同程序的标准状态进行扩展。而这个扩展的目的就是实现一个协作式多线程。我们使用 Lua 协同程序的数据交换工具进行两项任务之间的消息转换，这种转换经常会用到游戏编程中。例如，游戏中的对话就可以看成是两个角色之间的消息转换。

在这个说明图中，某一项任务可以拥有 4 种状态：活动（awake）、休眠（sleep）、等待（wait）和死亡（dead）（见图 4.3.3）。由一个分发器（Dispatcher）来控制该任务的执行。在我们之前讲到的部分，每一个注册过的任务都会被指派一个名字，这个名字用于在数据转换机制中识别该任务。在每一次迭代中，分发器会访问所有注册过的任务，并且在必要的时候执行。所有处于活动状态的任务都被执行，所有处于休眠状态的任务会被检查，看其是否需要被激活并执行。处在等待状态的任务会被忽略。

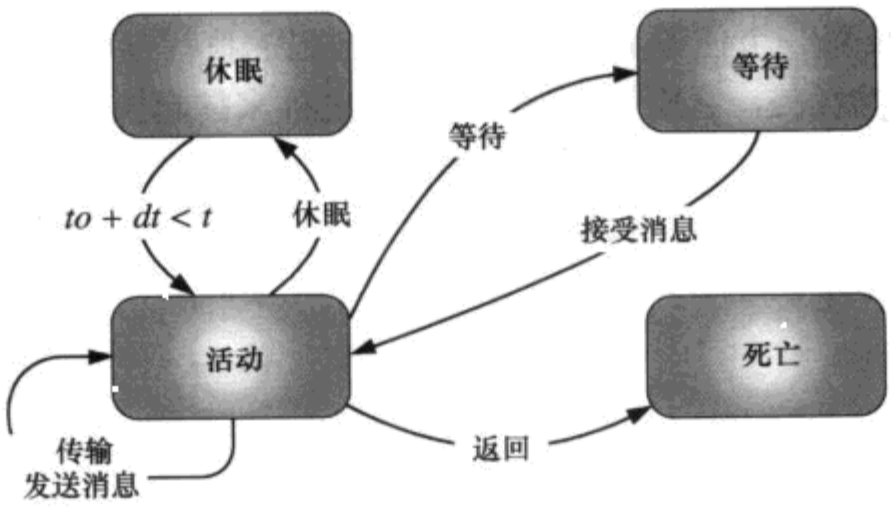


图 4.3.3 多任务合作下的任务状态

每一项任务都会作为一个协同程序被一再地执行。然而，这些任务将不会使用标准的 yield 函数，而是使用分发器提供的下列函数来中断（suspend）它们的执行。

- Transfer（receiver，message）：向接收器（receiver）任务发送一个消息；发送消息的任务处于活动状态。
- Sleep（dtime）：告知分发器在某段时间内忽略该任务；任务进入休眠状态。
- Wait( )：告知分发器忽略任务直到该任务被明确地激活；任务处于等待状态。

如果该任务的主函数返回，那么任务就变成死亡状态，并且从分发器的名单中删除。如果任务处于等待状态，并且接收到了一个消息，它就会被激活，并且重新参与下一次迭代。

不必建立一个单独的分发器对象，我们在这里选择为支持多重分发器而建立一个库。表 Dispatcher 模拟了一个分发类并且在每次建立 Dispatcher 对象时作为元表来使用。

```
-- Exported Dispatcher "class"
Dispatcher = { }
Dispatcher.__index = Dispatcher

-- Create new Dispatcher object
function Dispatcher.create ()
    local obj = {
        reftime = os.time(), -- set initial time
        tasks = {} -- list of tasks
    }
```

Download at www.Pin5i.Com

```
    setmetatable(obj,Dispatcher)
    return obj
end
```

用于在Dispatcher中注册新的任务的method会接收到一个任务名字和与其相关联的主函数，每一项任务都会在一个存储由 Dispatcher 控制执行任务过程中需要的所有信息的表中表现出来。

```
function Dispatcher:addtask (name, task)
    local t = {
        name = name,
        status = "awake",
        co = coroutine.create(task),
        wakeuptime = 0,
        sender = nil,
        message = nil,
    }
    self.tasks[name] = t
    self.active = t
    coroutine.resume(t.co,self)
end
```

注意，Dispatcher 在任务作为一个参数经过自身的时候会调用该任务。这就给了一个对任务进行任何设置的机会。但是之后，任务马上就要回归到 Dispatcher 的控制之下（参见本节最后部分的例子。）

对所有任务执行控制的回路会在所有已注册的任务中进行迭代，执行活动的任务、检查休眠的任务，查看休眠任务是否需要被激活并执行。

```
function Dispatcher:loop ()
    repeat
        self.currtime = os.difftime(os.time(),self.reftime)
        for name,task in pairs(self.tasks) do
            if task.status == "awake" then
                self:execute(task)
            elseif task.status == "sleeping" then
                if task.wakeuptime <= self.currtime then
                    task.status = "awake"
                    self:execute(task)
                end
            end
            task.sender = nil
            task.message = nil
        end
    until not next(self.tasks)
end
```

负责执行任务的补充方法会重新开始一个相应的协同程序，并作为外部参数通过，这些外部参数为发送者的名字和消息。这个函数还会检查任务什么时候死亡，并将其从 dispatcher

Download at www.Pin5i.Com

中删除。

```
function Dispatcher:execute (task)
    self.active = task
    local ok, errmsg =
        coroutine.resume(task.co,task.sender,task.message)
    if not ok then
        error (errmsg)
    end
    if coroutine.status(task.co) == "dead" then
        self.tasks[task.name] = nil
    end
end
```

下边还介绍了一种用于任务执行中断的方法。该方法将消息作为输入参数，接受者任务的名字和消息传给其他的任务。消息和发送者的名字被存储起来，用来重新开始相应的接收者任务。这样如果接收者处于等待状态，那么它就会被激活。

```
function Dispatcher:transfer (receiver, message)
    if receiver and self.tasks[receiver] then
        local task = self.active
        self.tasks[receiver].sender = task.name
        self.tasks[receiver].message = message
        if self.tasks[receiver].status == "waiting" then
            self.tasks[receiver].status = "awake"
        end
        return coroutine.yield()
end
```

下面的方法能为休眠状态的任务存储它们在执行回路中被检查的唤醒时间。最后，这个方法还会将该任务做上等待的标记。

```
function Dispatcher:sleep (dtime)
    local task = self.active
    task.wakeuptime = self.currtime + dtime
    task.status = "sleeping"
    return coroutine.yield()
end

function Dispatcher:wait ()
    local task = self.active
    task.status = "waiting"
    return coroutine.yield()
end
```

注意，被转换或传输的消息的值并不仅仅局限于字符串。消息可以表示为任何 Lua 值，这就能传输比较复杂的数据了（例如：表或者应用程序定义的用户数据）。

为了举一个给予以上这些代码的联合多线程应用的例子，我们将其应用于一个游戏角色

Download at www.Pin5i.Com

上。为了让事情尽量简单化，该角色是被动的，只能回答向它提出的问题。这个简单角色会对一些消息作出反应："hi"、"who"、"advice"、"sleep" 和 "bye"。对于前 3 个消息，角色会发回对应的应答。它对 "sleep" 的反应是进入休眠状态 10s。当休眠的时候，角色是非活动的，并不会接收任何消息。角色接受到了 "bye" 消息后会推出；然后变成死亡状态并且从分发器的任务列表中被删除。

```
local character = function (dispatcher)
    local reply = {
        hi = "Hi",
        who = "I am a scripter",
        advice = "Try Lua",
    }
    local sender, message = dispatcher:wait()
    while message ~= "bye" do
        if message=="sleep" then
            io.write("(Zzz)\n")
            sender, message = dispatcher:sleep(10)
        else
            local answer = reply[message] or "Sorry?"
            sender,message = dispatcher:transfer(sender,answer)
        end
    end
end
```

在一个具体的游戏中，类似于这个角色的行为将会扩展成更加复杂的活动行为。

为了检查角色的行为，让我们假设一个玩家是任务列表中的另外一个 agent。玩家键入消息，并且能看到回答。

```
local player = function (dispatcher)
    local sender, message = dispatcher:transfer()
    while true do
        if sender then
            io.write(sender,": ",message,"\n")
        end
        io.write("Player: ")
        local line = io.read()
        sender, message = dispatcher:transfer("Character",line)
        if line == "bye" then
            return
        end
    end
end
```

为了让这两个 agent 能够共同运作，我们需要建立一个分发器，把它们加入到任务列表中，并且让它们进入分发回路。

```
local d = Dispatcher.create()
d:addtask("Character",character)
d:addtask("Player",player)
```

Download at www.Pin5i.Com

```
d:loop()
```

关于这个技巧的所有代码可以在随书 CD-ROM 中找到。

4.3.6 总结

在本章中，我们介绍了一些在 Lua 协同程序上层编程的高级控制机制。在 Lua 中执行的协同程序都是基于几种简单的概念，但是能提供强大的编程工具。我们鼓励游戏编程人员使用 Lua 的协同程序创建其他的控制方式。游戏编程人员可以使用 Lua 的协同程序在更广阔的领域里解决其他的问题，例如增量模拟、AI 以及文本处理等。

4.3.7 参考文献

[deMoura04] de Moura, A. L., N. Rodriguez, and R. Ierusalimschy, "Coroutines in Lua." *Journal of Universal Computer Science,* 10(7): pp. 910–925, 2004.

[Harmon05] M. Harmon, "Building Lua into Games." *Game Programming Gems 5,* Charles River Media: pp. 115–128, 2005.

[Ierusalimschy03] Ierusalimschy, R., "Programming in Lua." Lua.org, 2003. Available online at *http://www.lua.org*.

Download at www.Pin5i.Com

4.4　在多线程环境里处理高级脚本执行

Sebastien Schertenleib,
瑞士联邦理工学院
虚拟现实实验室
Sebastien.schertenleib@epfl.ch

下一代的专用硬件、高端 PC 和家庭控制台将提供一种多芯的架构[DeLoura05][Intel05][Micorsofe05]。在多芯环境里发展有效的编码是一项复杂的任务，几乎没有开发者能完全掌握。为了能让开发者利用多芯硬件创造和操纵游戏模拟，专门的努力是必不可少的，以便提供一种更加友好的开发环境。因此，游戏开发系统需要为高级的互动行为提供更好的抽象层，比如说行为脚本。

这个技巧描述的是，基于组件的架构如何被采用从而使之从提供多重任务表示形式的脚本界面中获益。一种建立在微线程概念基础上的专门的执行程序[Hoffert98]，这里是这样描述的，通过使用一条完全不同的线程进行每一个流控制克服了一些局限性和性能的消耗。

4.4.1　基于组件的软件和脚本的解释程序

基于组件的软件设计提供了一种层级结构风格，它把系统功能解析成一种层级制度，每一层都与它的上一层和下一层相连接[Rene05]。通过保持应用程序专用代码的最高级状态并且允许开发者可能再利用较低的层级，这种方法倾向于促进循环再利用。底部的层级通常是低级库，它们被优化以便在专用平台上运行，但是上部的层级被用来与系统进行信息交换。这样，一个独立插件可以与任何顶级服务器进行信息交换。

一个插件的架构可以轻易的使脚本界面显示引擎功能。在这个架构中，一个脚本语言解释程序负责 C++对象和脚本语言对象的绑定。C++对象被反映在一个更安全的脚本语言环境里，这使得开发者得以解脱而不必担心通用 C/C++的问题，比如说内存管理或是类型检查。结果，生产能力可以更强大，为测试初期的想法和即将完成的现实游戏编码提供更多的开发时间。

接下来的部分将描述一个标准的解释程序如何被扩展成拥有更多先进功能的、满足专用游戏系统需求的程序。我们将介绍一些建立在 Python 脚本语言基础上的执行程序的细节[Python05]；但是，一些通用的概念也会被其他的脚本语言所采用，比如说 Lua[Lua05]。

Download at www.Pin5i.Com

4.4.2 协同程序与微线程程序

理论上来说，当我们建立一个充满了单独个体的虚拟世界的时候，我们希望每一个个体都能独立的发生互动。因此，最直接的方法就是建立一个单独的流来控制每一个角色。从编程的角度来说，我们可以预想为所有的个体媒介指定一个线程。然而，线程转换的系统开销以及内存的延后导致这种方式被禁止。为了优化性能，重头线程的数量应该与可用的核的数量一致。另外，在多线程环境下，开发和 debugging 要比单线程环境下困难得多。只有少数的团队成员可能拥有多线程基于组件系统开发所需要的技巧和智慧。因此，提供一个能够通过高级抽象进行操作并能够保持即时性能和多重任务表现的软件架构是很重要的。另外，还有一个亟待解决的问题是，为非专家的开发人员提供合作多重任务和多线程功能开发的简单方法，并且解除 debugging 及其他数据同步性冲突这样的繁琐工作。

协同程序的概念，由[Conway63]总结，是一般意义上控制抽象概念的最古老的方法之一。这个概念是在一个协同环境下提供控制行为。但是这个能为程序员提供有力的控制抽象概念的方法被很多通用目的语言设计者们忽略了。近来，脚本语言，例如 Python、Lua 和 Perl 已经将合作式任务管理作为多线程环境的替代方案[Schemenaur01][adya02]。以简单迭代器和生成器为形式的受限协同程序模式被称为微线程。一个微线程指一个拥有自己的私有数据和控制堆栈的执行单元，它与其他微线程共享全局变量。由于采用合作共同模式，一个微线程在其他的微现程可以运行前必须明确地被要求暂停。

在 Python 环境中，一个微线程就是一个包含了 yield 语句的特殊函数[Schemenaur01]。当其被调用的时候，一个微线程函数必须返回一个可在该微线程控制堆栈的程序中任何点上都能恢复的头等对象。由于微线程不会受到昂贵的上下文转换的影响，因此在同时运行几千个微线程的时候并不会受到性能上的惩罚[Tismer00]。

作为微线程的一种天性，微线程允许通过将对象状态信息转移到本地变量来大幅度简化 AI 和对象更新代码[Catter01]。微线程可以用来编写生成一个结果并且 yield 回主程序的函数。主程序就可以稍后再恢复它们，并将它们恢复到其变形前的样子，保留所有的本地变量。由于微线程基于合作多重任务，数据同步性是被限制的。

4.4.3 微线程管理器

为了简化微线程在合作多重任务中的使用，我们引入了控制微线程执行的微线程管理器。该管理器可独立地的控制每一个微线程的状态（例如，运行、暂停或终止）。通过和该管理器通信，任何应用程序都可以与微线程互动，并根据需要改变它们的状态。在开发过程期间，开发者可以在运行时打断脚本，这对原型开发过程是很有用的。

在图 4.4.1 描述的想象图中，可以看到微线程 id1 在它最后被 yield 的位置接受到控制并继续执行。当 yield 语句到达的时候，它会将控制返回给管理器。同时，根据一个外部事件的需要终止微线程 id3。管理器将发送该信息给微线程，并释放其所有的变量状态。当这个操作完成之后，微线程 id3 将会给微线程管理器发送一条信息，允许它更新当然活动及非活动线程的列表。在这个例子中，微线程 id2 是非活动的。

Download at www.Pin5i.com

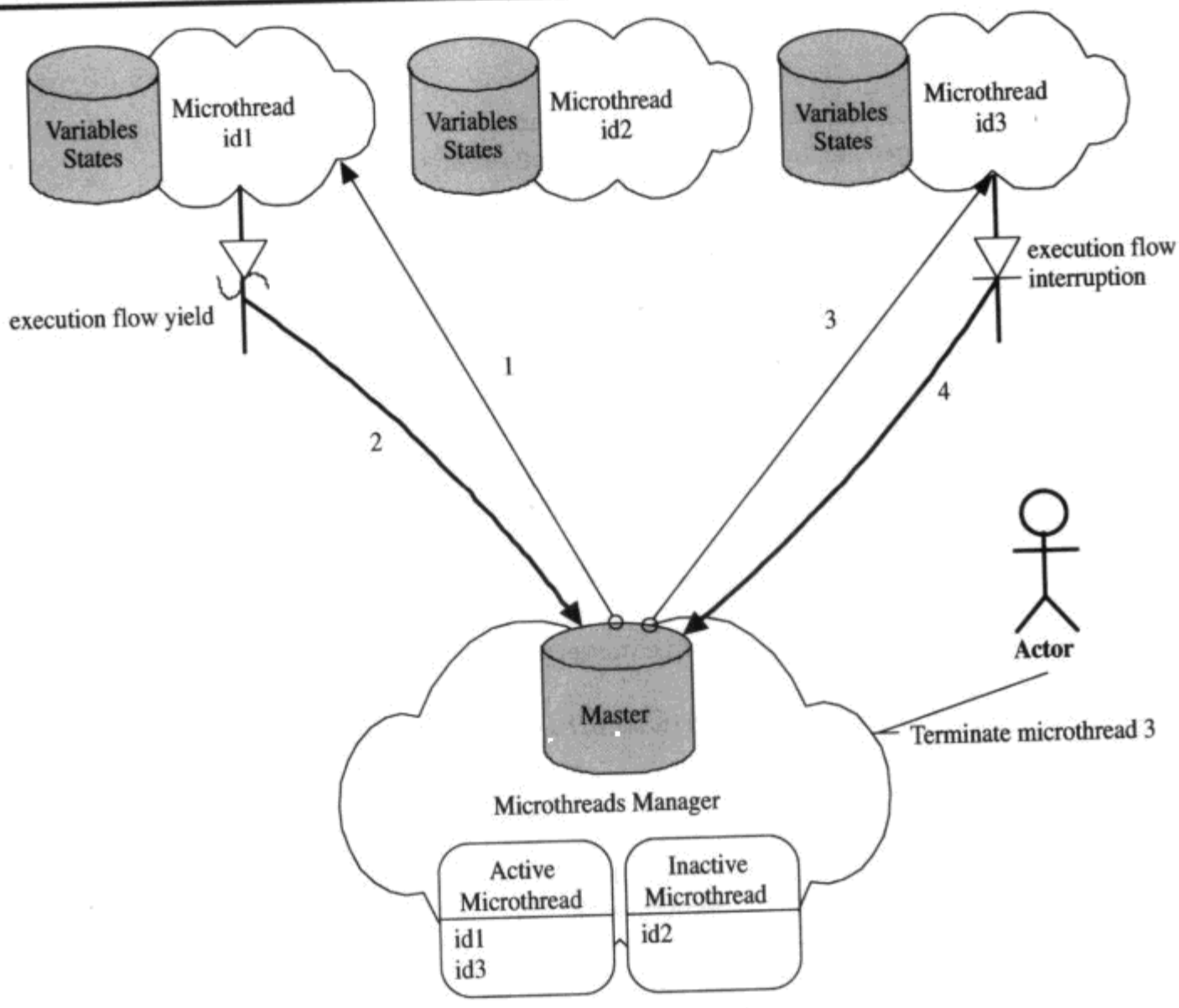


图 4.4.1　微线程运行模型

执行

微线程管理器必须持有一个线程列表和这些线程的状态。它负责生成、运行和销毁每一个活动的微线程。程序清单 4.4.1 说明了一部分执行过程。

程序清单 4.4.1　微线程管理执行细节

```
#The manager is itself a Python generator.
def pyMicroThreadManager():

    '''

    To synchronize the Python micro thread manager with C++ code
    we yield at each iteration, leaving the C++ code triggering the
    .next() statement
    '''

    while 1:
      yield 0
      if (isMicroThreadManagerPaused()):
        # Pause this manager and wait until the app resumes it
        yield 0
      elif (isMicroThreadManagerMustDie()):
        # Break and terminate the microthread generator
```

Download at www.Pin5i.com

```
            break
        else:
            # Safe removal of killed mircrothreads
            for i in range(len(g_pyMicroThreadToBeRemoveFromList)):
                removeMicrothread(g_pyMicroThreadToBeRemoveFromList[i])

            g_pyMicroThreadToBeRemoveFromList = []

            # Manage each microthread status independently.
            for i in range(len(g_pyMicroThreadList)):
                g_pyCurrentMicroThreadInRun=i
                nameOfMicroThread=g_pyMicroThreadNameList[i]
                if (not isMicroThreadPaused(nameOfMicroThread)):
                    # Handle script calling C++ code throwing exceptions
                    try:
                        # If it is already finished, throw exceptions
                        yieldResult = g_pyMicroThreadList[i].next()
                        if isMicroThreadMustDie(nameOfMicroThread)
                            or yieldResult==1:
                            pyEndOfMicroThread(nameOfMicroThread)
                    except ValueError:
                        print "Safely terminate the microthread"
                        pyEndOfMicroThread(nameOfMicroThread)
            # Check if an external event requires termination of the manager
            yield pyEndOfMicroThreadManager()

# First create the manager
g_pyMicroThreadManager=pyMicroThreadManager()
# Create the micro thread list
g_pyMicroThreadList=[]

# A parallel one with names not generator this time
g_pyMicroThreadNameList=[]
'''
Create a list of micro thread that have been killed, to be
safely remove in sync by micro thread manager
'''
g_pyMicroThreadToBeRemoveFromList=[]

'''
Create a variable which contains the ID in the list from the
last microthread called, in order to be able to remove it in case
of C++ throw Exception, we simply kill this microthread
'''
g_pyCurrentMicroThreadInRun=-1
```

4.4.4 嵌入 Python

1. 配置模式

如果核心引擎 API 对脚本开放，那么就可以开发出复杂脚本。一个典型的案例就是编写需要在 C++中运行的更快的算法，并且在脚本中控制它们。由于每一个游戏都有它们自己的

Download at www.Pin5i.Com

需求，因此就开发出了不同的配置模式。这些配置模式允许脚本按不同的方式运行。

- 模块化模式

在这个模式中，如图 4.4.2 所示，在解释程序完成更新之前，主线程会一直处在等待状态，这是通过使用微线程在微线程管理器上注册来实现得。微线程会通知微线程管理它现在的状态。Python 微线程更新回路会以一个与核心 C++引擎不同的频率运转，这样就会在不影响 C++核心路径性能的情况下允许复杂脚本的执行。

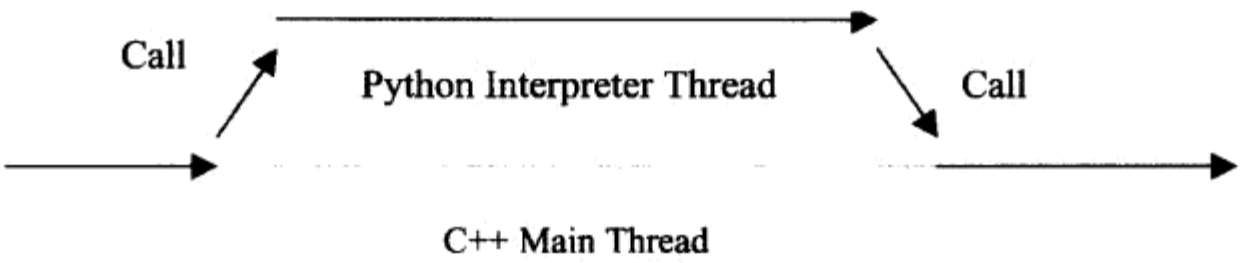


图 4.4.2　案例 1，模块化模式

- 合作多任务模式

在这个模式中，如图 4.4.3 所示，微线程被用来通过合作多重任务运行脚本。这就确保了微线程的更新速度不会比 C++代码路径的频率高，也避免了由于微线程更新比所需要频率更快而占用过多系统资源的情况发生。一般来说，这个模式是最合适的模式。

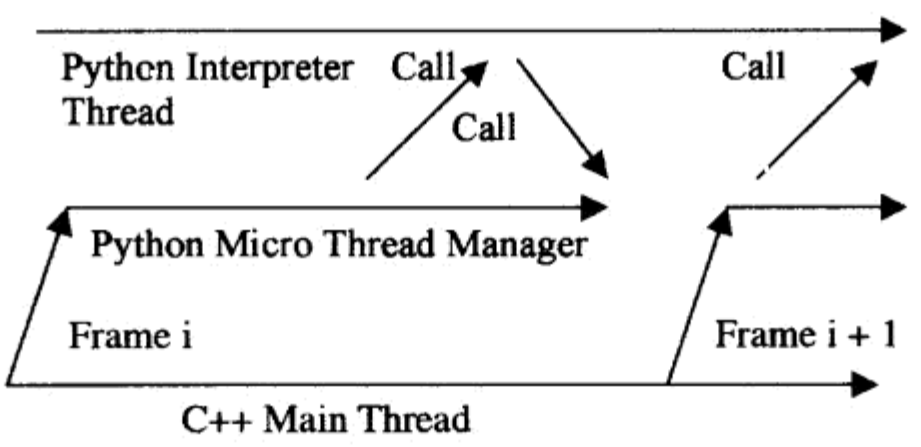


图 4.4.3　案例 2，合作多任务模式

- 并发多任务模式

在这个模式中，如图 4.4.4 所示，Python 多线程管理器在 Python 解释程序线程中生成的线程中运行。这样就很明确地将 C++和 Python 代码的路径区分开来，也避免了整体脚本工作流在基础上的不同造成的桢率不一致。

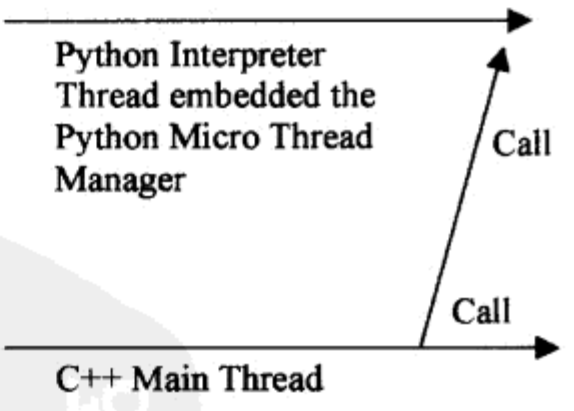


图 4.4.4　并发多任务模式

2. 自定义预处理器和专门化关键词

我们这里所谈到的这个系统的一个目标是，为微线程的使用提供高级抽象概念。实实在在的执行必须要被隐藏起来。系统应该控制所有的变量/对象的生成和管理。并且为了方便脚本开发过程，我们会开发一个自定义的预处理器。这个预处理器的目的是将简单脚本通过关键词转换为自然的

Download at www.Pin5i.com

Python 微线程代码。使用自定义关键词并按照自动预处理步骤的目的是减少手动处理时由于逻辑混乱所产生的错误。这种方法还可以在不破坏脚本的情况下对管理器进行修改。预处理器还会生成 Python 对象绑定于 C++对象在运行时的引用。这就免除了将这些对象绑定到每一个微线程上下文中的需要。

为了简化脚本生成过程，可以建立两个关键词：vhdYIELD 和 vhdRETURN 语句。关键词 vhdYIELD 将控制权交还给微线程管理器，管理器进行堆栈中的下一个微线程的过渡。为了达到微线程的标准，脚本需要拥有至少一个强制性的 vhdYIELD 语句。使用这个关键词可以使一个长运行的程序通过几桢来实现。开发者可以将这个语句设定在长的或者不可预见的算法的关键区域，就像路径计划需求那样。vhdRETURN 关键词会破坏和终止当前的微线程。默认情况下，所有微线程的最后一行代码必须是 vhdRETURN 语句。

为了获得直观的印象，我们来看一下程序清单 4.4.2 中的脚本描述。

程序清单 4.4.2　一个基本的脚本

```
B=0
while (true):
    B=B+1
    if (B==10):
        vhdRETURN
    vhdYIELD
```

这个例子脚本将会在 10 个连续桢中被执行。每一次 Python 解释程序到达关键词 vhdYIELD，微线程都返回 yield 到微线程管理器，管理器最后会运行等待中的下一个微线程。当 B 等于 10 的时候，执行 vhdRETURN 将会通知管理器该微线程必须被终止。

由于这些额外的关键词并不是 Python 语言的一部分，因此它不会和标准 Python 解释程序一起运行。我们前边提到的预处理工具将会把程序清单 4.4.2 中的内容转换成自然的 Python 代码，如程序清单 4.4.3 所示。

程序清单 4.4.3　基于程序清单 4.4.2 预处理脚本的 Python 对应脚本

```
# Microthread definition
def __PythonScript2123():
    '''
    Use a try-and-catch block to avoid that the script calls a
    C++ method to propagate the exception.
    Also, dump the traceback helping the end user
    '''
    try:
        # The original script begins here
        while (true):
            print B
            if B == 10:
                '''
                The preprocessor replaced vhdRETURN with this Python code
                '''
                yield pyEndOfMicroThread(
                    "__generatorObj__PythonScript2123")
            B = B + 1
```

Download at www.Pin5i.Com

```
        '''
        The preprocessor replaced vhdYIELD with this Python code
        '''
    yield 0
except:
    traceback.print_exception(sys.exc_info()[0],
      sys.exc_info()[1], sys.exc_info())
    yield pyEndOfMicroThread("__generatorObj__PythonScript2123")
    # Create the Python generator
    __generatorObj__PythonScript2123=__PythonScript2123()

    # Register the microthread
    g_pyMicroThreadList.append(__generatorObj__PythonScript2123)
    g_pyMicroThreadNameList.append("__generatorObj__PythonScript2123")
```

注意，从运行性能的角度来看，分解脚本只在开发阶段发生。对于应用角度，所有的脚本都应该是纯粹的 Python 字节代码。

我们的预处理系统的运行也有一定缺陷，并且不会发现诸如长运行主回路需要分离成几桢或者脚本作者希望明确终止一个微线程。

3. 可创工具

为了方便开发流程，可创工具是必须的。在我们的执行过程中使用了一个基于文本编辑器的 Python 控制台 GUI（见图 4.4.5），提供了和沟通层以及和游戏引擎的互动。这个编辑器

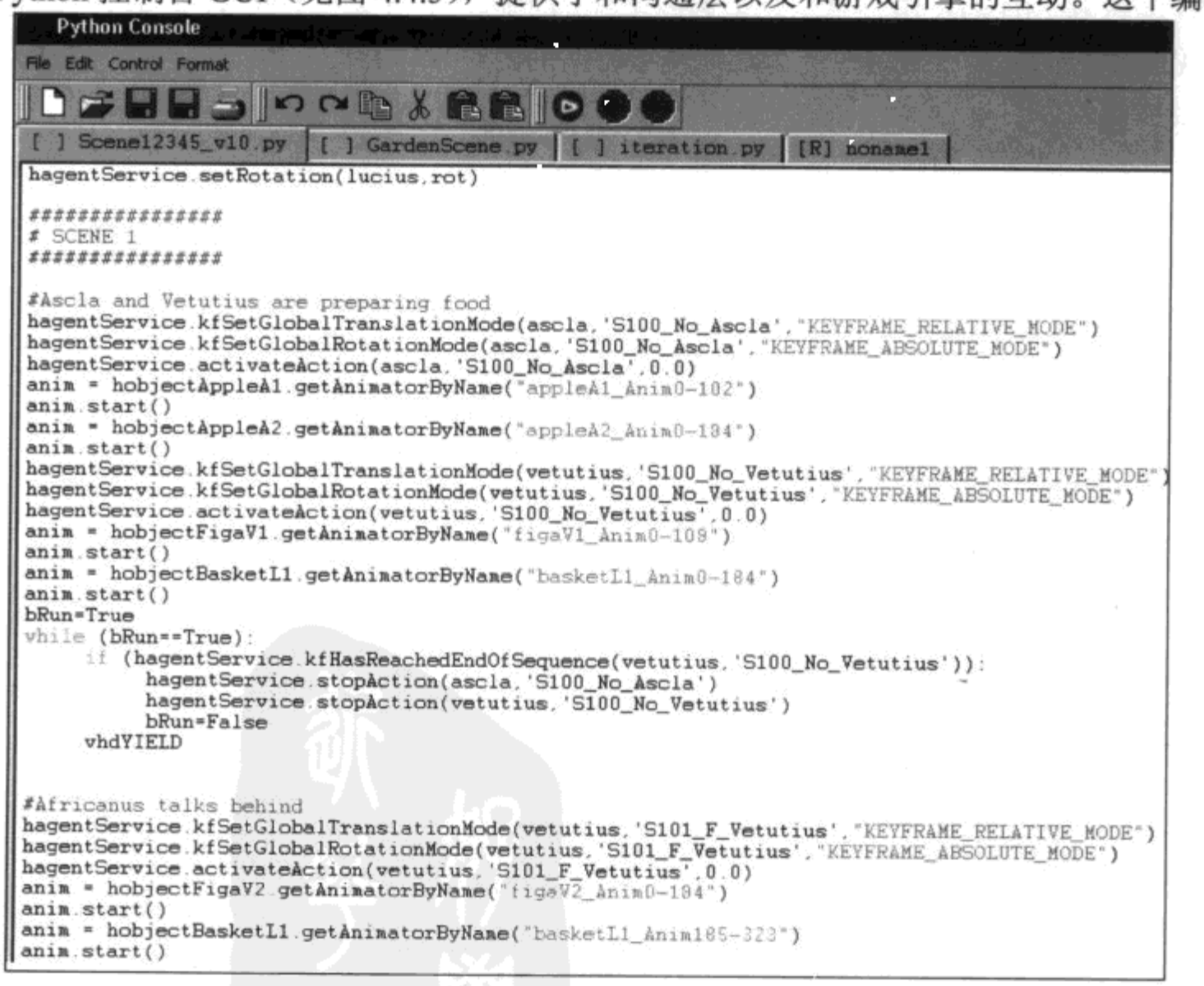


图 4.4.5　Python 控制台

Download at www.Pin5i.Com

给出关于每一个微线程现在状态的回馈并且使用彩色句法结构高亮表示 Python 的关键词和专门的游戏引擎对象。

4.4.5 试验和结果

到现在为止，我们的微线程管理器已经被应用到控制动态组件如云彩的模拟、粒子系统以及对媒介的模拟。然而，这个系统离完美还很远，并且有很多局限性，特别是 Python 的多线程模型。例如，现在还不可能从一个微线程运行环境的外部打断这个微线程。作为一个推论，在某些情况下可能需要专门的控制。作为一个边注，在 Lua 中的实际协同程序的实施能提供这样的功能。

为了分析这个架构的可测量性，很多测试用应用程序被建立起来，以测试在拥挤模拟情况下的适应能力。一个测试台为微线程建立了 800 个独立运行的个体媒介。这个应用程序的目的就是控制媒介行为上的上下文对象，在其中的每一个实体都通过微线程保持它们自己的内部逻辑。各个实体之间体现出来的明显区别提供了实体层面上的粒度。这样，AI 引擎就能控制每一活动实体在即时约束下的时间表，并且根据它们当前的细节等级改变它们的内部逻辑。这也在流执行中提供了更高的灵活性。

为了比较这个系统，使用了各种不同的案例。测试是在一台 Pentium 4 Xeon、2.2GHz、1GB 内存、一个 nVIDIA Quadro Pro 显卡上运行的。图 4.4.6 显示了在这个模拟过程中主要组件的性能影响。3D 渲染组件是消耗最多的组件。动画和 AI 更新与活动的实体数量联系紧密，非可见实体的动画更新是通过一个连续基于时间的函数来保存的（例如，只有骨骼根定向及定位会被修改）。

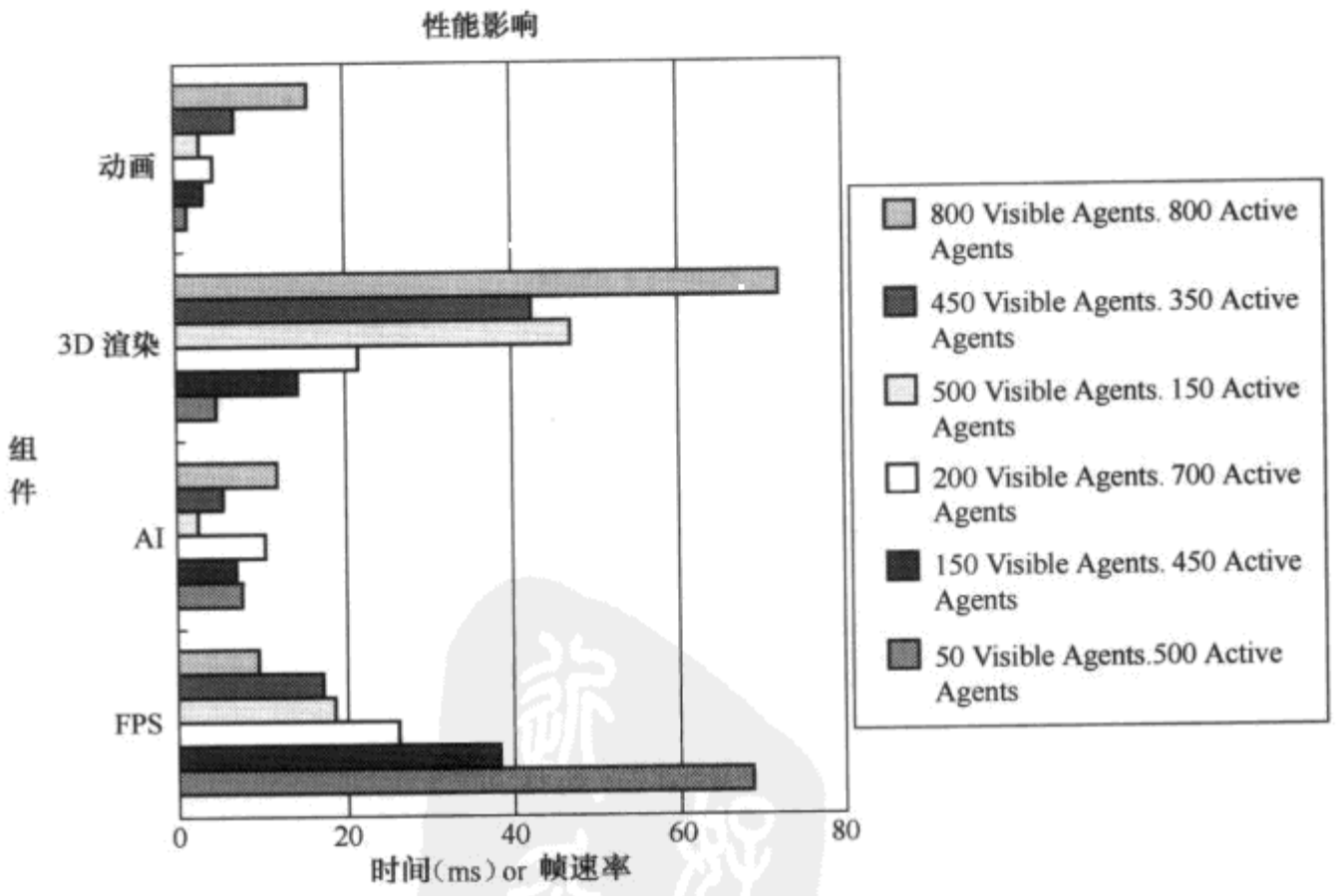


图 4.4.6 性能影响

Download at www.Pin5i.Com

4.4.6　总结

本节展示了可以在现有的基于组件的平台上被建立起来的一个高级的抽象概念。这个系统为非技术用户可以控制多任务环境下虚拟世界提供了一个专门的工具。能够按需要尽可能多地满足专门控制流为世界提供了一个更直观的表现，与此同时并不影响即时性能。

现在也计划通过利用一个更加细化的微线程管理控制方案来扩充现有的微线程管理器，进而扩充整个模型的灵活性，与此同时提供更多的工具来帮助实现 debuging 和模拟。

4.4.7　参考文献

[Adya02] Adya, A. and J. Howell, "Cooperative Task Management without Manual Stack Management." *Proceedings of USENIX*, Annual Technical Conference,Monterrey, California, 2002.

[Carter01] Carter, S., "Managing AI with Micro-Threads." *Game Programming Gems2*, Charles River Media: pp. 265–272, 2001.

[Conway63] Conway, D., "Design of a Separable Transition-Diagram Compiler."CACM, 1963.

[DeLoura05] DeLoura, M., "CELL: A New Platform for Digital Entertainment."GDC, 2005.

[Hoffert98] Hoffert, J. and K. Goldman, "Microthread: An Object for Behavioral Pattern for Managing Object Execution." Washington University, Distributed Programming Environments Group, 1998.

[Intel05] Intel, "Intel Multi-Core Processor Architecture Development Backgrounder."2005.

[Lua05] Ierusalimschy, R., L. H. de Figueiredo, and W. Celes, "The Evolution ofLua," 2005. Available online at *http://www.lua.org*.

[Microsoft05] Microsoft, "Methods and system for general skinning via hardware accelerators." U.S. Patent Application, 2005.

[Python05] Python, "Python Programming Language." 2005. Available online at *http://www.python.org*.

[Rene05] Rene, B., "Component Based Object Management." *Game Programming Gems 5*, Charles River Media, 2005: pp. 25–37.

[Schemenaur01] Schemenaur, N. and T. Peter, "Simple Generators." PEP, 2001. [Tismer00] Tismer, C., "Continuations and Stackless Oython." *Proceedings of the 8th International Python Conference, Arlington*, VA, 2000.

Download at www.Pin5i.Com

4.5 使用非插入型代理导出角色属性

Matthew Campbell 和 Curtiss Murphy,
BMH Associates, Incorporated
campbell@bmh.com, murphy@bmh.com

从某些观点来看，每一个游戏引擎都会在导出角色属性时遇到一些问题。哪怕是最简单的引擎最终都需要拥有一个能给外部工具导出内部游戏角色属性的功能。关卡编辑器需要这些属性帮助设计人员在不依靠开发人员的情况下制作地图。消息传递框架需要这些属性有效地建立通用消息和发送网络更新。加载/保存模块需要它们来实现状态的持久化。尽管随这个功能的需求非常清楚，但不幸的是，这个方案并不总是显而易见的，特别是对于 C++来说。

这个技巧描述了一个用于导出游戏角色属性的灵活且松散的结构。它也将展示如何为您的游戏角色类解决这个问题而不需要做任何改动。有些时候需要通过重构来完成的。然而，在 C++中，这几乎肯定需要在你的角色上做一些改动。因此，这个技巧展示了一个通过在你目前已有的游戏对象之上再创建一个抽象层的非插入型方案来导出游戏角色的属性。在你构建关卡编辑器、游戏管理器甚至是消息框架时，这个方案都是比较理想的。

4.5.1 角色、代理和属性

在这个方案中，有 3 个主要的元素，分别是角色、角色代理和角色属性。本节将介绍这 3 个组件的定义。

1. 角色

角色（Actor）在游戏世界中都是独一无二的。它们构成了游戏的内容，并且包含了许多游戏世界交互的逻辑。车辆、天气系统、发射器、触发器和光这些都是角色。角色可以很简单，像静态的树，或者相当复杂，像一个组装的、可摧毁并带有声音，武器和人工智能的坦克。我们在这里所讨论的角色，是那些有您需要导出的属性的角色。车辆具有最大速度和最大破坏力这样的属性。天气对象有时间、季节和云等状态。触发器有碰撞范围和事件说明。

Download at www.Pin5i.Com

2. 角色代理

一个角色代理（ActorProxy）是一个简单的面向数据的类，该类封装了每一个角色的类。它们非常简单，但是它们通过实施了两个系统中主要的工作而在这个体系里扮演了一个关键的角色。首先，它们为关卡编辑器或者游戏管理器这样的外部工具提供了一个通用而又统一的类，使它们有一个访问和维护属性的途径。其次，它们可以访问到它所封装角色的所有东西，特别是属性。一旦被定义之后，角色代理就可以用来代替角色，特别是对于高端的工具来说。

3. 角色属性

一个角色属性（ActorProperty）是一个单独的、关于角色信息的原子片。角色属性是你所要导出的数据中含金量最高的部分。也是将要被编辑器显示并且有可能被网络层发送的数据。从图 4.5.1 中能够看到，角色属性被定义成一系列的 Set/Get 方法、To/From 字符串方法和像名字、描述、标签这样的标识信息。

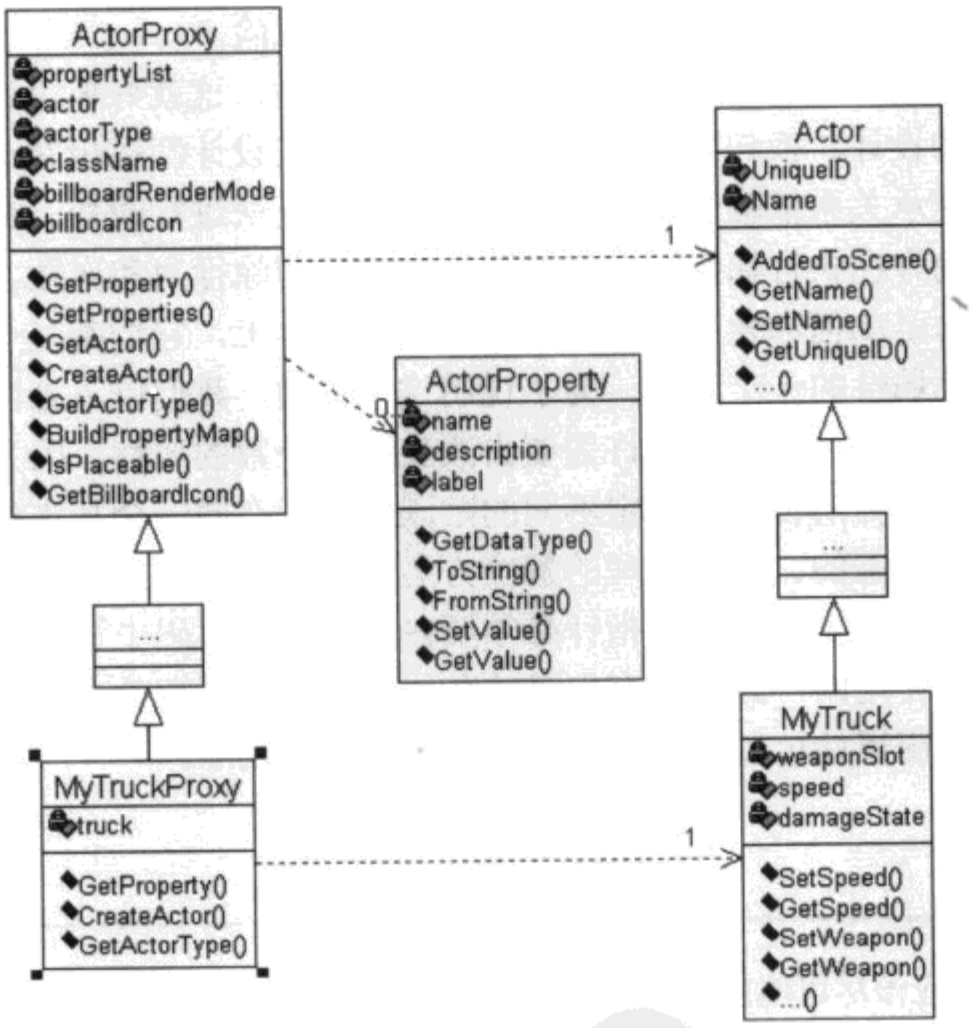


图 4.5.1　角色代理类图

尽管为每一个属性维护这样一个信息会给你的系统增加一些处理损耗，但它能够极大地提高系统弹性和游戏角色的有效性。当你的整个团队打算引入统一使用属性的概念时，这个方法就会特别的有效。想想这种情况：你的一个团队成员创建了一个格子旗的角色作为游戏夺旗模式的旗帜。它只是单一的为了达到这个目的。但是如果把贴图、周期、阶段和振幅这样的数据都保存为属性，那么该成员就能创建一个通用的角色，并在不修改任何代码的情况下将它作为任务 NPC、路点、房间装饰甚至是背包对象！

Download at www.Pin5i.Com

4.5.2 非插入型和动态体系结构

为什么要使用角色代理？毕竟，角色类已经存在了；为什么要创建额外的类或者增加功能性的封装呢？答案是实用性。在理想的世界中，我们可以不需要这样的区分。然而在实际应用中，有很多工程师分别在物理的和组织范围内编写独立无关的模块。换句话说，不可能所有的游戏对象都统一的支持这种行为。幸运的是，角色代理创建了一个非插入型体系来解决这个问题，并且清除了原有对象的属性关联功能。

另外当处理已经存在的游戏引擎或者和现有的系统集成的时候，这个体系结构的非插入型特点就会显得特别的重要。“他只是说现有的？”没错，现有的，就是先前存在的代码。在游戏不断增加大小、成本和复杂度的同时，对现有模块的使用就会变得愈加重要。问题出在现有模块很容易就会有成百上千行代码，这些代码缺少支持导出属性。一种解决办法是为每一个对象去尝试修改引擎的最底层的实现。但这样增加了很可怕的，有可能使现有代码报废的风险。为了避免这种风险，我们推荐一个非插入型属性层。因为这一层是构建在已有代码之上，开发者添加和维护属性不会存在任何令人厌恶的风险。

这种解决方案的另一个重要属性是角色属性的动态性。这就好像争论是采用基于组件的设计还是基于继承的设计。换句话说，任何依赖编译期的设计都很少对已有软件系统的寿命和弹性造成影响。继承关系依赖于编译期而在运行期则无法进行改变。另一个方面，基于组件结构的设计方法允许在执行期针对一个对象添加或者删除行为和属性。

那么，这个概念是如何和角色属性相关联的呢？实际上，它们都是一样的。角色代理在很多情况下都是角色属性的一个组件。为了说明这个原理，我们以下列情况为例：当赋予静态模型角色一个资源属性之后，它也许需要去改变贴图个数的属性。因为这样可以允许一个关卡设计师为一个立方体的各个面指定不同的贴图，或者在同一个基本角色上指定一个门的正、反面。使用一种静态的、没有代理的解决方案可能经常会导致一个指针直接指向它们的数据元素。因此，要导出贴图属性，指针必须要在编译期的代码中注册。这样一个系统是没有办法在上述例子中使用的，因为会有大量的贴图需要在运行期改变。幸运的是，使用这里介绍的属性系统，贴图属性可以在运行期动态地添加或者删除。

4.5.3 角色属性

面向对象的设计应该尽可能的提供复用性和可扩展性。组件之间应该采用简单但同时也是更加灵活的方式互相通信，以允许系统在它的生命周期增长和变化。这使得对对象数据按照通用的方式访问变成可能。为了在游戏引擎中能够按照通用方式访问数据，我们引入了 ActorProperty。角色属性负责在角色之间传递数据。本小节将从基础来构建 ActorProperty，从最基本的构建模块开始。

1. 函数

ActorProperty 最重要的两个特性是它的“getter”和“setter”方法。从最简单的语义上来说，getter 是一个用来检索数据的方法，而 setter 是一个赋予数据的方法。当它们组合在一起，就定义了封装一个原子数据元素所有需要的东西。接下来的问题就是如何使用一种通用的方式导出它

Download at www.Pin5i.Com

们。通常，使用一种称为反射（或自省）的技巧。不幸的是，C++语言并不自发的支持这样一个构造。这就需要我们使用仿函数这一概念。仿函数（Functors），可以看作一个函数对象，一种为了在基本 C 代码中查找回调函数指针的 C++机制。

以下代码是一个用来展示如何使用仿函数简单的例子。更加深入的讨论可以在[Hickey94]中找到。MakeFunctor、Functor0 和其他可用的仿函数都可以在 http://www. delta3d.org 中找到。为了本小节的目的，我们把这些对象看成黑箱，并假设它们的功能已经被解释过了。

```
class MyClass {
public:
    void SayHi() { std::cout << "Hello!" << std::endl;
};

int main(int argc, char *argv[])
{
    MyClass test;
    Functor0 doHello = MakeFunctor(test,&MyClass::SayHi);

    //在控制台打印"Hello!"
    doHello();

    return 0;
}
```

2. Getter/Setter

我们已经讨论过仿函数的用法，现在我们可以使用它们来为 ActorProperty 构造 getter 和 setter 功能。构造 setter 或者 getter 功能是什么意思呢？换句话说，一个 getter 或 setter 函数需要哪些参数并且它的返回值是什么？好的，一个 setter 函数会设置一些数据。因此，它会有一个参数（将要被设置的值）。而 getter 函数，从另外一个方面，能够返回属性的值。因此，它不会有输入的参数，但是会有一个返回值。此外，保持方法签名简单，getter 和 setter 将支持使异常跟踪错误。

下面的代码展示了如何把 getter 和 setter 仿函数结合到一个基本属性类中。由于属性类的基本任务就是和仿函数一起工作（例如，函数对象），因此它就需要保持住这些函数。这个类也将展示模板是如何使用的。模板是你的系统能够通用的支持各种不同数据类型的关键部分，像整型、浮点和字符串。因此，属性类被参数化成两种子类型，SetType 和 GetType。这样子类就能够通过 GetPropertyType 方法和定义 GetType/SetType 指定它们所有支持的数据类型。

```
template <class SetType, class GetType>
class GenericActorProperty : public ActorProperty {
public:

    GenericActorProperty (Functor1<SetType> &set,
    Functor2<GetType> &get)
{
    SetPropFunctor = set;
    GetPropFunctor = get;
}
```

Download at www.Pin5i.Com

```
//子类返回一个指定的数据类型
virtual DataType &GetPropertyType() const = 0;

//通过调用 set 方法来设置这个属性的值
virtual void SetValue(SetType value)
{
    SetPropFunctor(value);
}

    //通过调用 Get 方法来得到这个属性的值
    virtual GetType GetValue() const
    {
        return GetPropFunctor();
    }

private:
    Functor1<SetType> SetPropFunctor;
    Functor2<GetType> GetPropFunctor;
};
```

3. 属性设计

之前的章节定义了仿函数，并且展示了它们是如何在基本的角色属性类中使用的。但是，这里还有一点需要注意的。毕竟 GenericActorProperty 也只是一个基类。这个结构需要支持大量的数据类型，从简单的整型、浮点型到复杂的贴图、网格和声音这样的属性。为此，你需要为每一个属性类型准备一个子类，每一个子类支持且只支持一类数据，图 4.5.2 展示了一个关于属性类的子设定。

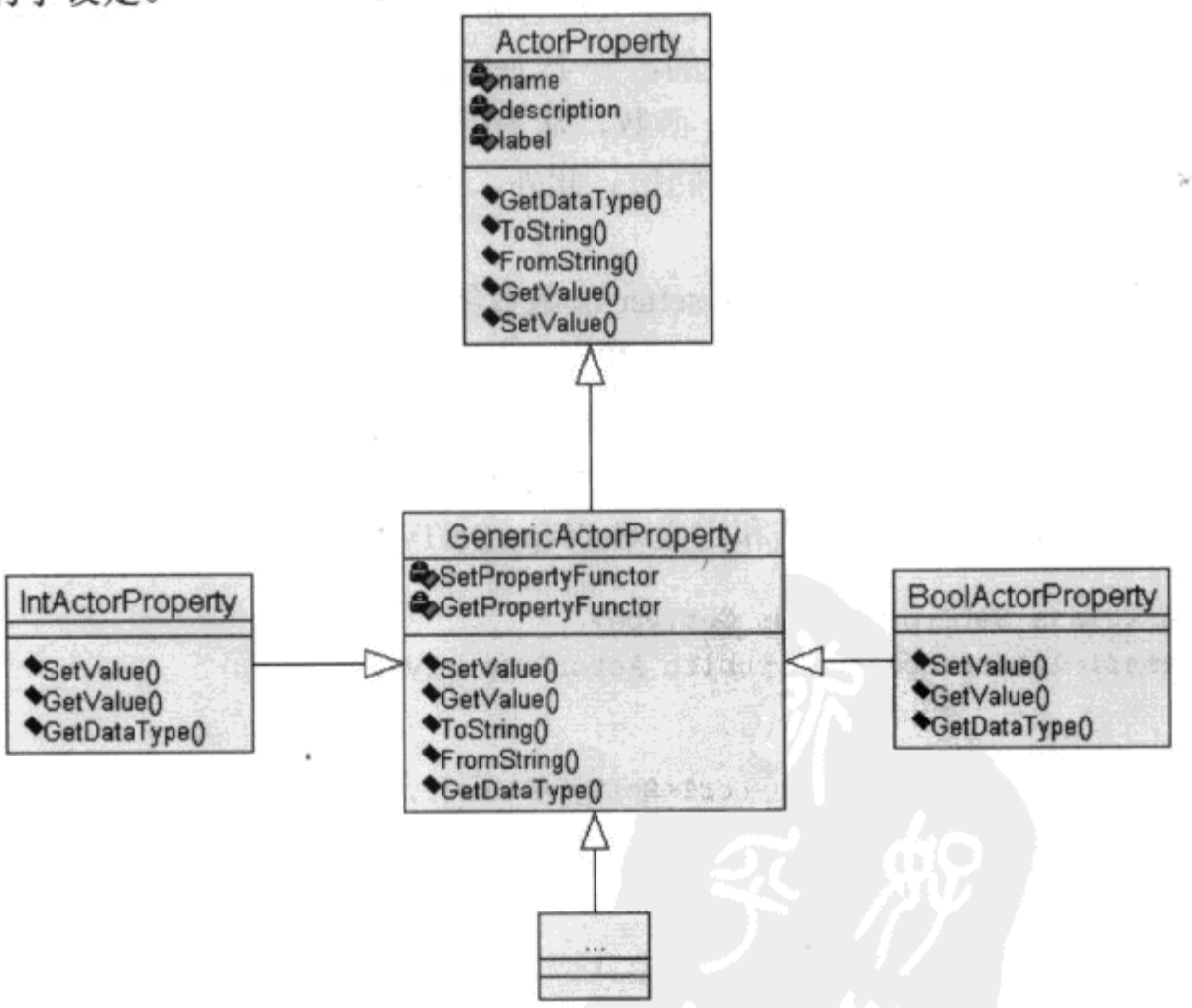


图 4.5.2 ActorProperty 类图

Download at www.Pin5i.Com

这个设计相当的易于理解。ActorProperty 是一个包含了重要元数据（如名字、描述等）的基类。它也包含了把属性转换成或者转换自字符串的接口。这对需要文本解释的操作非常有帮助，如从 XML 文件中加载或者保存属性。尽管没有在图中指出来，但是 ActorProperty 应该支持对二进制数据流的序列化，如网络通信就会使用到它。

最后，我们用 IntActorProperty 类实现的代码片断来总结本小节的内容。ToString 和 FromString 方法没有在这个例子中给出，因为它们和我们所讨论的问题无关。这两个函数相当的简单，仅仅是把整数和字符串互相转换。

```
//整数属性类…
class IntActorProperty : public GenericActorProperty<int,int> {
public:

    IntActorProperty(Functor1<int> set, Functor0Ret<int> get) :
        GenericActorProperty<int,int>(set,get) { }
    const DataType &GetDataType() const
    {
        return DataType::INT;
    }
}
///一个整型属性类的测试类…
class MyClass {
public:
    MyClass(int initialValue) { myValue=initialValue; }
    void Setter(int newValue) { myValue=newValue; }
    int Getter() const { return myValue; }
private:
    int myValue;
};
int main(int argc, char *argv[])
{
    MyClass test(25);
    // 创建一个整数属性的例子。在实际实现中，它存储在管理它的角色属性里表的 ActorProxy 中
    IntegerActorProperty prop(MakeFunctor(test,&MyClass::Setter),
        MakeFunctorRet(test,&MyClass::Getter));

    //打印值 25
    std::cout << "Value = " << prop.GetValue() << std::endl;
    prop.SetValue(10);
    //打印值 10
    std::cout << "Value = " << prop.GetValue() << std::endl;

    return 0;
}
```

4.5.4　角色代理

现在我们明白了 ActorProperty 的机制，是时候让它们使用 ActorProxy 了。幸运的是，ActorProxy 会非常的简单。每一个代理构建了一个属性需要导出的 getter 和 setter 方法的列表。

Download at www.Pin5i.Com

除了属性以外，每一个代理都保存了它对应角色的引用。这就允许了高层工具可以使用 ActorProxy 来替代角色对象。一旦初始化，ActorProxy 就会提供一个统一的行为层来用于你的游戏引擎去创建对象文件的输入/输出（I/O）和事件消息传递。

下列片断展示了一个 BaseLightActorProxy 的 BuildPropertyMap()方法的例子。每一个代理都必须重载这个函数以构建它的属性列表。

```
void BaseLightActorProxy::BuildPropertyMap()
{
    const std::string GROUPNAME = "Light";

    // 得到这个代理封装的角色
    Light *light = dynamic_cast <Light *> (mActor.get());
    if (NULL == light)
        //抛出异常

    // 一个轻量角色中布尔属性的例子
    AddProperty(new BooleanActorProperty("enable","Enabled",
        MakeFunctor(*light,&Light::SetEnabled),
        MakeFunctorRet(*light,& Light::GetEnabled),
        "Sets whether this light is enabled", GROUPNAME));

    // 在封装后的角色中设有一个匹配的单元的整数属性的例子。因此，我们在 Proxy 上增加一个单元来转
       换数据。注意这样的属性非常灵活足以易用在不同对象上
    AddProperty(new IntActorProperty("LightNum","Light Number",
       MakeFunctor(*this,&BaseLightActorProxy::SetNumber),
       MakeFunctorRet(*light,& Light::GetNumber),
       "Sets the light number", GROUPNAME));

    // 例如，枚举是一个让用户从列表上指定一个值的模版化的属性类型
    AddProperty(new EnumActorProperty<LightModeEnum>(
    "Lighting Mode","Lighting Mode",
       MakeFunctor(*this,&BaseLightActorProxy::SetLightMode),
       MakeFunctorRet(*this,&BaseLightActorProxy::GetLightMode),
       "Sets the lighting mode", GROUPNAME));

    // 一个复杂属性资源的例子。贴图资源保存在代理自己的资源路径上。当在代理上调用一个单元时，它自
       动转换资源路径，加载数据，并且调用对应的角色单元
    AddProperty(new ResourceProperty(*this,
       DataType::TEXTURE, "BloomTex", "Bloom Texture",
       MakeFunctor(*this, &BaseLightActorProxy::SetBloomTexture),
       "Sets the bloom filter texture", GROUPNAME));
    ...
}
```

这一个代码片断还有一些事情是需要注意的。首先，每一个代理都会完全控制它所能看到适合于它的属性。这样做可以实现属性分组，就像看到的 Light 组的名称一样。其次，代理所做的远远不只是简单的封装。代理能够做任何它需要做的事情。在这种情形下，光照模式不会存在于 Light 中。是不是在真的角色中存在并不是很重要，代理能够创建属性，或者

Download at www.Pin5i.Com

处理不同情况的方法。

图 4.5.3 展示当 BaseLightActorProxy 导出到关卡编辑器之后是如何显示这些属性的。

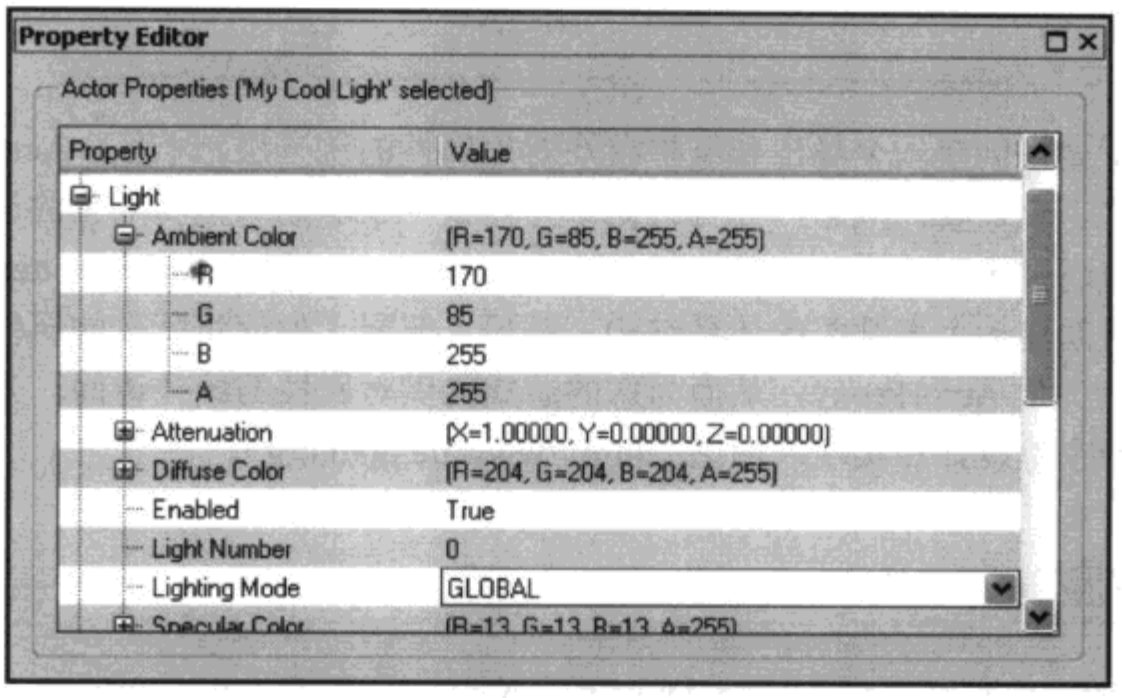


图 4.5.3　LightActorProxy 导出到 Delta3D 后的属性

4.5.5　从理论到实践

这个架构不只是一种理论。它是一种真正有效的系统。实际上，它是动态角色层的基础，这一点可以在开放源代码的引擎游戏 Delta3D 中找到[eMH05]。

很早的时候，我们接到一个在现有引擎之上构建一个关卡编辑器的任务。当我们准备开始工作的时候，我们构建了角色的基类并且把属性直接放到角色中。不幸的是，Delta3D 已经在不少的程序中应用了，而且如果我们引入一个新的统一的基类，将无法维护核心引擎的完整性。这就让情况变得复杂了，因为核心引擎角色和场景图紧密地结合在一起。幸运的是，代理解决方案解决了上述的问题，还同时大大增强了引擎的兼容性。旧的工程不会有影响，新的工程会得到一个非常灵活的工具。

尽管本小节的焦点是 ActorProperty，但利用 ActorProxy 还有很多方式。实际上，Delta3D 引擎在这个新的架构上做了一些扩展。举例来说，关卡编辑器需要一个 2D 公告板来表现所有的角色。而核心角色类并不了解属性并且肯定不会知道关卡编辑器，因此我们把这个行为放置到 ActorProxy 中。另外，我们还在 ActorProxy 的设计中加入了基本的对象消息系统、日志系统，甚至还包括可以调用的消息处理方法。这一整套新的行为并不需要对底层的角色进行额外的修改。

基于 ActorProxy 和 ActorProperty 的设计会有另一件事情。由于代理可以交替并且普遍地使用，这样很明显就可以抽象更高一层。因此，我们创建一个动态的角色层来使在导出库的同时导出一个或多个角色代理。每个库一般都能像 ActorProxy 导出一组属性和属性类型一样导出角色代理和类型的集合。使用这种方法，角色库可以分别由不同的独立的远程小组来创建，而且可以动态地加载到游戏、编辑器和任何您所需要的工具中。所有的库都是在代理层工作，因此和角色的类型完全没有关系。该设计的最后的革新在于根据客户端或者是服务器

Download at www.Pin5i.com

端来动态地加载不同的库。由于角色都是通过 ActorProxy 统一定义的，基本的游戏逻辑将不会了解到它们的不同。

4.5.6 总结

本节讲述了如何创建一个导出和维护游戏对象属性的统一层。讨论了 Actor、ActorProxy、ActorProperty 这 3 个重要的概念，并且讨论了为什么一个非插入型和动态的访问角色策略如此地重要。本节还讲述了使用仿函数如何创建 ActorProperty 的 getter 和 setter 方法，以及如何使用这些技术创建一个属性类的体系结构。最后，解释了如何将所有的属性放在一起，为每一个角色创建一个 ActorProxy，从而为您的游戏提供对属性的统一访问。开源 Delta3D 项目有这个原型的完整设计和实现（网站：http://www.delta3d.org）。

4.5.7 参考文献

[BMH05] BMH Associates, "The Dynamic Actor Layer." July 2005. Available online at *http://www.delta3d.org*.

[Hickey94] Hickey, Rich, "Callbacks in C++ Using Template Functors." 1994. Availableonline at *http://www.tutok.sk/fastgl/callback.html*.

Download at www.Pin5i.Com

4.6　基于组件的游戏对象系统

Chris Stoy, Red Storm Entertainment
cstoy@nc.rr.com

游戏对象是存在于虚拟游戏世界中最基本的对象，它提供了对象在模拟过程中的数据和功能。在 C++中，这些通常都是由基本对象类的子类提供特定的功能和数据来完成的。这样做的问题是，有可能由于为每一种类型的对象实现一个子类而造成类爆炸，重复功能的函数有可能出现在完全没有关联的对象中，并且需要为每一个游戏对象赋予一个预先定义的对象类型。本节会通过定义一个基于组件的游戏对象系统来解决这些问题，对象是由一系列的组件构成，每一个组件都提供特定的功能和数据。这个系统也允许在运行期组合对象或是提供数据驱动系统来组合一个游戏对象。

4.6.1　游戏对象

一个游戏对象表现了在游戏世界中存在的基本实体。在通常情况下，它由一个表示位置和旋转的变换信息、一个唯一标识符、一些定义对象属性的状态信息和用来修改和查询状态的方法构成。能够被表示成游戏对象的例子有人工智能、车辆、树、建筑物、武器和任何能够在游戏世界中单独存在的对象。传统的游戏对象体系问题是对象的数据有可能增长的难以管理，并且功能也会变得越来越复杂。随着复杂性的增长，代码会变得越来越难以理解，这些增长很可能导致一些功能的重复并且错误可能随时出现。

游戏对象组件系统（GOCS）试图解决类爆炸所产生的相关问题，并且强制把游戏对象的功能分解成一系列的只提供特定功能和接口的组件。尽管这种方式初看起来是以一种更加复杂的方式去实现了一些特性，但它强制访问数据的时候采用正确的接口并且把功能划分成更多可管理的部分，使它们更容易维护。

GOCS 的核心是游戏对象（GO）。它是由一个变换（例如，在世界矩阵中的位置和旋转）、一个唯一标识符和一个游戏对象组件（GOC）组成的，如程序清单 4.6.1 所示。

程序清单 4.6.1　一个游戏对象的基本定义

```
typedef std::string go_id_type;
```

Download at www.Pin5i.Com

```
class GameObject
{
    public:
    GameObject( const go_id_type& id );

    const Transform& getTransform() const { return mTransform; }
    void SetTransform( const Transform& xform )
    {
      mTransform = xform;
    }

    const go_id_type& getID() const { return mGOID; }
    void setID( const go_id_type& id ) { mGOID = id; }

    GOComponent* getGOC( const goc_id_type& familyID );
    GOComponent* setGOC( GOComponent* newGOC );
    void clearGOCs();

private:

    Transform mTransform; // 本地到世界的变换的转换
    go_id_type mGOID; // 这个对象的惟一标识符
    typedef std::map<const GOComponent::goc_id_type, GOComponent*>
        component_table_t;

    component_table_t mComponents; // 所有组件的图
};
```

从这个基础定义，我们现在可以创建一个独一无二的对象并且把它放到场景管理系统中；无论如何，现在还没有什么有趣的东西。为了让我们新的对象拥有一些功能，我们必须创建一些 GOC。

4.6.2 基本的游戏对象组件

基本的 GOC 提供了一个所有 GOC 都必须实现的通用接口。这让 GO 可以管理它们并简化它们的使用。每一个 GOC 都为特定的功能定义了特定的数据和行为。像生命、可视化、物理管理、背包和人工智能功能都是 GOC 的例子。基本的 GOC 定义可以参照程序清单 4.6.2。

程序清单 4.6.2 GOC 接口

```
class GOComponent
{
    // GOComponent 界面
public:
    typedef std::string goc_id_type;

    GOComponent() : mOwnerGO(0) {}
    virtual ~GOComponent() = 0 {}
```

Download at www.Pin5i.Com

```
    virtual const goc_id_type& componentID() const = 0;
    virtual const goc_id_type& familyID() const = 0;

    virtual void update() {}

    void setOwnerGO( GameObject* go ) { mOwnerGO = go; }
    GameObject* getOwnerGO() const { return mOwnerGO; }

private:
    GameObject* mOwnerGO; // 这个组件属于的游戏对象
};
```

组件基于通用的组件功能并按照类体系结构组织。那些有关联的组件组我们称之为“家族”。每一个组件都是从一个基本的家族组件中继承而来。一个组件的家族信息从 familyID() 方法来获得。一个组件家族的根接口类必须提供该 ID，而且 ID 必须是在所有家族中唯一的。这样做的方法之一是使用家族的名字作为 ID。

例如，我们希望定义一个负责 GO 渲染的组件家族。我们可以这样做：

```
class gocVisual : public GOComponent
{
    // GOComponent interface
public:
    virtual const goc_id_type& familyID() const
        { return goc_id_type("gocVisual"); }

    // gocVisual interface
public:
    virtual void render() const = 0;
};
```

这里利用简单地重载 familyID()的纯虚方法来返回这个组件类的 ID。我们同时也为这个组件类提供一个特定的新接口，它由一个单独的新方法构成，这个方法就是 render()，用它来渲染出这个组件的可视化表现。你也许觉得我们可以使用 GOComponent 的方法 update()来执行渲染。但是，update 方法一般是用来更新这个组件的状态。举例来说，我们也许会需要在真正渲染一个人类的模型之前来更新它的骨髓位置。

现在我们就能够来定义一个组件类，需要定义这个组件在这个类里面特定的实现。这个功能通过让组件从 GOC 接口类中派生来完成。根据上面 gocVisual 的例子，来具体实现一个用球体来表现对象的组件：

4.6.3　在游戏对象中实现组建管理

在 GO 中管理组件的接口是非常简单的。我们需要能有添加组件、删除组件以及检索并得到一个组件的方法。这些工作可以做得非常有技巧，但是现在我们将用简单的方式来实现这些函数。

有一种必须遵守的规则是，在 GO 实例中不能同时存在两个相同类型的 GOC。因此每个

Download at www.Pin5i.Com

GOC 都要有一个特定的功能，需要 GO 或者对 GOC 有更多了解的高层代码才能使得两个以上相同类型的 GOC 存在。

举例来说，如果在 GO 中我们同时有两个 gocVisual 组件，一个是 gocVisualSphere，一个是 gocVisualCub，使用者需要在渲染的时候决定使用哪种组件。这将会导致暴露过多的信息，而且增加了代码之间的耦合度。在这种情况下，更好的解决方案是提供一个可以容纳其他 gocVisual 组件的 gocVisualContainer 组件，并且在调用 render()方法时决定使用哪一个组件。访问组件的接口非常简单：

```
GOComponent* getGOC( const goc_id_type& familyID );
GOComponent* setGOC( GOComponent* newGOC );
void clearGOCs();
```

getGOC()方法将通过传入 familyID 来返回 GOC，当 GOC 并不存在的时候返回 NULL。同样的，setGOC()方法通过传入 GOC 设置 GO 中的组件，而且如果一个 GOC 的类型并不存在的话，将返回替换了的 GOC 指针。最后，clearGOC()方法会删除在 GO 中所有的 GOC。必须要提醒您的是一定要注意组件的归属，而不要导致不必要的内存泄漏。最简单的方法就是使用带有引用计数的指针类，如 boost:shard_ptr。无论如何，使用适合你项目的解决方案。作为一个例子，让我们创建一个包含 gocVisual 的 GO 并在渲染的时候使用它，如程序清单 4.6.3 所示。

程序清单 4.6.3 创建和使用 QO 与组件

```
void init()
{
// DO INITIALIZATION
// 假定我们有一个场景管理类的全局对象在 gSceneMgr 中
gSceneMgr.clear();

// 创建 10 个随机视觉组件的游戏对象
for ( int ii = 0; ii < 10; ++ii )
{
    GameObject* go = new GameObject( go_id_type( ii ) );
    go->setTransform( /*some random transform*/ );
    GOComponent* gvsis = 0;

    // 随机决定使用何种视觉表现方式
    if ( rand()%2 )
    {
        gvis = new gocVisualSphere(5.0f);
    }
    else
    {
        gvis = new gocVisualCube();
    }

    go->setGOC( gvsphere );
}
```

Download at www.Pin5i.com

```
    // 把游戏对象添加到场景管理中
    gSceneMgr.add( go );

    // FINISH INITILIZATION AND START MAIN LOOP
}

void renderFunc()
{
    // 假定是框架结构中用来渲染场景的方法枚举所有的场景对象，并进行渲染
    GameObject* go = gSceneMgr.beginIteration();
    while ( go )
    {
        // 渲染对象
        GOComponent* goc = go->getGOC( goc_id_type("gocVisual") );
        gocVisual* gvis = static_cast<gocVisual*>(goc);
        if ( gvis )
        {
            // 我们不需要知道使用了哪一个 gocVisual 对象
            gvis->render();
        }
        go = gSceneMgr.nextIteration();
    }
}
```

4.6.4　组件间的通信

尽管 GOC 被尽可能地设计的独立，但在同一个 GO 里的组件有时候也需要和别的组件进行通信。例如，一个 gocAI 组件有可能需要访问保存在 gocHealth 组件中的数据来做一些决定。这就是为什么基本的 GOC 需要保存一个“拥有者”GO。当组件被添加到一个 GO 中的时候，ownerGo 会被设置成新的 GO，并且当组件被删除的时候，ownerGo 会被清除。使用这个变量，组件就可以通过 GO 来访问其他相关的组件。要注意的是，无论如何，组件间的访问都增加了互相之间的耦合。在实际应用中，只要足够的重视，将不会引起太大的问题。让我们用 gocAI 做例子看它们是如何工作的。

```
#include "gocHealth.h"

gocAI::update()
{
    // 查看是否受伤，如果受伤了，就逃跑
    GameObject* go = getOwnerGO();
    GOComponent* goc = go->getGOC( goc_id_type("gocHealth") );
    gocHealth* health = static_cast<gocHealth*>(goc);
    if ( health )
    {
        if ( health->isWounded() )
        {
            setBehavior( cFleeBehavior );
        }
```

Download at www.Pin5i.Com

```
    }
}
```

在这里我们看到了 gocAI 通过它的拥有者 GO 调用了 gocHealth。如果组件存在，那么它通过 gocHealth 来检查伤害信息并决定做什么。

4.6.5 游戏组件模板

GO 由 GOC 组成。到目前为止，我们看到的都是非常简单的例子。但在实际应用中，组件会变得非常复杂，而且它们会管理大量的数据。让我们深入地讨论上面所提到的 gocHealth 组件，如程序清单 4.6.4 所示。

程序清单 4.6.4 定义一个 800Health 组件

```
class gocHealth : public GOComponent
{
    // GOComponent interface
public:
    virtual const goc_id_type& familyID() const
        { return goc_id_type("gocHealth"); }

    // gocHealth interface
public:
    typedef int health_value_t;
    enum bodyPart_e { head=0, torso, leftArm, rightArm,
                      leftLeg, rightLeg, cNumBodyParts };

    gocHealth();

    health_value_t getInitialHealthAt(const bodyPart_e part) const;
    void setInitialHealthAt(const bodyPart_e part,
                            const health_value_t hp);
    health_value_t getHealthAt(const bodyPart_e part) const;
    void setHealthAt(const bodyPart_e part, const health_value_t hp);

    bool isWounded() const;
    void reset();

private:
    health_value_t mCurrentHPs[cNumBodyParts];
    health_value_t mInitialHPs[cNumBodyParts];
};
```

使用这个定义我们可以创建一个带有 gocHealth 属性的新的 GO，如下所示：

```
GameObject* go = new GameObject( go_id_type( "Human" ) );
go->setTransform( /*some random transform*/ );

// 创建个人类的视觉组件
GOComponent* gcvis = new gocVisualHuman();
```

Download at www.Pin5i.Com

```
go->setGOC( gcvis );

// 创建并初始化一个健康状况的组件
GOComponent* gchealth = new gocHealth();
gcHealth->setInitialHealthAt( gocHealth::head, 7 );
gcHealth->setInitialHealthAt( gocHealth::torso, 50 );
gcHealth->setInitialHealthAt( gocHealth::leftArm, 20 );
gcHealth->setInitialHealthAt( gocHealth::rightArm, 20 );
gcHealth->setInitialHealthAt( gocHealth::leftLeg, 30 );
gcHealth->setInitialHealthAt( gocHealth::rightLeg, 30 );
gcHealth->reset();
go->setGOC( gchealth );
```

尽管这个例子易于管理，但想象一下必须编写更多更复杂的组件。尽管这个例子是易于管理的，但想象一下需要编写更多包含复杂代码的组件来初始化大量的数值。而且，我们将很有可能创建一个完全共享同样组件的 GO 类型的多个实例。例如，一群由 AI 控制的敌人和你控制的英雄战斗。不但在创建组件的时候逐个初始化它们非常琐碎，而且也会因为将在每一个组件实例重复公用数据而浪费很多的内存。在这个 gocHealth 的例子中，mInitialHP 是一个所有组件实例共有的一个常量。

游戏组件模板（GCTemplate）通过定义如何初始化一个组件的特定类型来解决这些问题。它也提供了一个存储通用数据和方法的仓库，让实际的 GOC 实例只管理改变过的数据。像 GOC 一样，GCTemplate 都继承自一个基本的接口类：

```
class GCTemplate
{
public:
    GCTemplate() { }
    virtual ~GCTemplate() = 0 { };
    // 返回 GOComponent ID,默认情况下,我们可以注册所创建的模板
    virtual const goc_id_type& componentID() const = 0;
    virtual const goc_id_type& familyID() const = 0;

    virtual GOComponent* makeComponent( GameObject* go ) = 0;
};
```

我们可以为每一个 GOComponent 创建模板。例如：

```
class gctHealth : public GCTemplate
{
    // GCTemplate interface
public:
    // 返回这个模板能够创建的 GOComponent ID
    virtual const goc_id_type& componentID() const
        { return goc_id_type("gocHealth"); }
    virtual const goc_id_type& familyID() const
        { return goc_id_type("gocHealth"); }

    virtual GOComponent* makeComponent()
    {
```

Download at www.Pin5i.Com

```
        gocHealth* goc = new gocHealth(this);
        goc->reset();
        return goc;
    }

    // gctHealth interface
public:
    typedef int health_value_t;
    enum bodyPart_e { head=0, torso, leftArm, rightArm,
            leftLeg, rightLeg, cNumBodyParts };

    health_value_t getInitialHealthAt(const bodyPart_e part) const;
    void setInitialHealthAt(const bodyPart_e part,
        const health_value_t hp);

private:
    health_value_t mInitialHPs[cNumBodyParts];
};
```

注意，在 gctHealth::makeComponent()方法中，我们调用一个使用 gctHealth 模板为参数的特殊构造函数。在这个构造函数中，我们把用来创建 gocHealth 组件的指针存储到 gctHealth 模板中。这样，在 gocHealth::reset()方法中，我们根据 gctHealth 模板来得到初始的生命值。依赖于组件实例之间的数据量是一个常数，这个引用能节约我们的一些内存。通过把这些模板保存到一个管理器中（例如，以一种 createGOC()方法保存到一个全局唯一的 GCTemplace 管理器），我们可以创建并初始化一个 GO，像这样：

```
GameObject* go = new GameObject( go_id_type( "Human" ) );
go->setTransform( /*some transform*/ );

// 创建一个人类的视觉组件
GOComponent* gcvis = gTempltMgr->createGOC( "gocVisualHuman" );
go->setGOC( gcvis );

// 创建并初始化一个健康状况的组件
GOComponent* gchealth = gTempltMgr->createGOC( "gocHealth" );
go->setGOC( gchealth );
```

4.6.6 游戏对象模板

GCTemplate 把 GO 的创建从我们之前看到的硬编码格式简化——但是我们可以做得更好。通过保存我们创建 GO 的组件模板集合，我们可以使创建过程更加容易：

```
class GOTemplate
{
public:
    // GCTemplate 列表类型
    typedef std::list<GCTemplate*> gct_list_t;
```

Download at www.Pin5i.com

```
    ~GOTemplate();

    // 数据访问方法
    void clear();

    const std::string& name() const { return m_name; }
    void setName( const std::string& name ) { m_name = name; }

    gct_list_t& components() { return m_components; }

    void addGCTemplate( const GCTemplate* gcTemplate );
    GCTemplate* getGCTemplate( const goc_id_type& id ) const;

protected:
    GOTemplate( const std::string& name );
    std::string m_name; // 模板名称
    gct_list_t m_components;
};
```

使用模板定义，我们可以创建一个全局的管理器（GOTemplMgr）来保存可用的模板并提供一个工厂方法，createGO()。这让我们很容易地创建出一个新的 GO 实例：

```
GameObject* go = gGOTemplMgr->createGO( "GO Template Name" );
go->setTransform( /*some transform*/ );
```

4.6.7　数据驱动的游戏对象创建

基于组件的游戏对象系统表现出的最强大的功能之一是，现在我们可以定义一个实际的 GO 数据集合。这取决于您希望创建的数据文件类型和您的资源系统。比方说，我们可以在 GO 之上使用 XML 来定义以下数据：

```
<?xml version="1.0"?>
<game_object_template name="Ped_Female ">
    <components>
        <goc component="gocVisualHuman">
            <model name="Ped_Female"/>
            <scale value="1.1"/>
        </goc>
        <goc component="gocHealth">
            <hitpoints
                head="7"
                torso="50"
                leftArm="20"
                rightArm="20"
                leftLeg="30"
                rightLeg="30"/>
        </goc>
        <goc component="gocAIPedestrian"/>
    </components>
```

Download at www.Pin5i.com

```
</game_object_template>
```

假定游戏对象模板管理器（和所有支持类）都实现了解析功能，那么您就可以很容易地从文件中创建任意数量的 GOTemplate，您希望定义不同的对象用在您的游戏中。

4.6.8 总结

本节展示了抽象组成各种在游戏中使用的 GO 组件功能的意义。这让我们可以通过组合不同的组件功能随意地创建新类型的 GO。使用 GCTemplate，我们可以简化创建新 GOC 的过程，同时能够节约一些内存。同样的，我们可以使用 GOTemplate 来定义一个特定的 GO 类型是由什么组件构成的。最后，通过一种正确的输入方式，我们可以把 GO 类型整个的定义成数据文件（比如 XML），使得我们无须修改任何代码就能把各种 GOC 组合成一个独一无二的 GO 类型。

Download at www.Pin5i.Com

第 5 章

图形学

Download at www.Pin5i.Com

简　介

Paul Rowan, Rho, Incorporated
paul@rowandell.com

在刚开始编写游戏的时候，我第一次有机会看到专业人员是如何做的。我起初承担转换 DOS 游戏的任务，将其重新组合为流行的 GUI 界面。这是绝好的查看游戏内部的机会，还能看看产生这些游戏的魔法。记得有时我查看一段代码，仔细解析其内容和功能，然后对自己说，“就这样？”看完后会发现它是明显、直接甚至简单的。我记得单独阅读一些片断的时候会发现它们甚至只是一些琐碎的代码，然而整个不可思议的处理流程正是融入到了这些可信赖、容易理解的片断中了。我开始认识到这些不可思议的事正是一个整体，创建了复杂逼真事物的简单机制的总和——游戏。

围绕着魔术师们的规则是从不揭示他们职业的秘密。这样做会破坏表演的秘诀，暴露其本质——一个消遣和舞台诡计的简单汇集，从观众的角度来看，无法将表演与魔法分辨开。魔法其实就在表演和观众的悬疑中，并不是其带来的秘密技巧。交出这些秘密并不会打破魔术师的规则。实际上，这仅仅提高了魔术的神秘性。魔术是长期的学习，是专业技能和创造魔术的灵巧表演，不是这些技巧本身。

本书的图形部分包含了从场景和几何预处理到高端 GPU 技术的一些章节，比如逐点光照和高动态范围的渲染。我们拥有仅利用一些基本的动画就能渲染广泛的不同人类角色静止动作的技术。在无法执行预先描述的行为时，要创造令人信服的角色动画总是一项挑战。这里可以通过可靠的方法来实现。有两节的内容致力于场景处理。其中 5.2 节介绍了生成包围体二叉树的方法，自适应地调整其启发式算法的标准，通过给出的原几何体选择最优的分割平面。而 5.3 节转动了标准的包围盒，通过沿轴定向的包围盒计算与几何体最匹配的近似体——有向包围盒。另外，5.4 节描述了优化蒙皮动画几何体的技术，可以有效地用 GPU 进行处理。

其他几节提出了一些 GPU 的创新使用，可以通过更统一的着色器模型和高精度的帧缓冲来实现。虽然它们确实与硬件相关，但这些内容是希望说明针对这些问题的一些创新的方法，即便是假设 GPU 的性能提高到几年以后的水平。有两节的内容说明了如何利用当前图形卡中新的顶点纹理特性。一直以来我们都需要在 CPU 和 GPU 的工作中进行协调，现在显卡可以处理所有技术。我们对很多试图创建统一的逐点光照引擎的开发者都面对的难题进行了深入论述。本章中介绍了无须多次渲染几何体就能处理场景中大量光源的解决方案。

Download at www.Pin5i.Com

本章中最具有创新性的内容是渲染有线条画特性的纹理，以前从未有过将其存储为低分辨率纹理的有效方法。本章还介绍了在对任何缩放比例都能锐利地渲染路标以及有相似特性的对象的方法，该方法利用低分辨率纹理和像素着色器，通过类似字体渲染器的方法来实现。本章还对渲染动态天空的有效方法，以及关于高动态范围渲染技术的讨论。

本章的内容只是讲述了一些游戏的技巧和手法，单独地看只是一些舞台技巧和对观众的误导。这里你或许能找到产生非常激烈的反应的方法，这样的反应可能会是“噢！就是这样做的”。魔术就是将这些片断结合起来，制造能挑战你想法的表演，而它们自己的技术只是一些简单的行业工具：魔术就是表演。

Download at www.Pin5i.Com

5.1 交互角色真实的静止动作合成

Arjan Egges, Thomas Di Giacomo 和
Nadia Magnenat-Thalmann
MIRA 实验室，日内瓦大学
egges@miralab.unige.ch,
thomas@miralab.unige.ch, and
thalmann@miralab.unige.ch

在本节中，我们要讨论一类经常在游戏中被忽略但是对于创建真实的角色动作却至关重要的运动：静止动作。实际上静止不动的角色并不存在，但在游戏中我们遇到一种情况：没有为角色停止或者静止的时候（比如等待另一个角色完成它们的部分）预先设计动作。这里我们介绍一种用于自动生成静止动作的框架，同时也可以作为游戏中角色动画引擎的一部分。该方法克服了使用已有剪辑库为基础来合成静止动作的限制。直接使用库中小动画会导致不自然的重复运动，将静止动作插入另一个动画中也会产生不连续性。

我们的动画引擎用指数映射表示旋转，同时应用了主分量分析（PCA，Principal Component Analysis）的方法。PCA 显著减小了动画空间的维度，因此允许在动画数据之间进行更快的转换，该方法对在线游戏十分有用。由于维度的减小，不同动作能够很快生成并保持不同节点间的独立性，最后得到持续真实而广泛多样的动画效果。

5.1.1 简介

虽然虚拟的人类行为模型在实时应用和非实时应用领域都一直在不断提高，但在实际控制和动画制作方面依然存在一些难题。人类总是一直以不同的方式运动着。在对角色制作动画的时候，首先需要解决的问题就是两个剪辑之间动画的缺乏，解决了这个问题就能防止在不同动作之间看上去不自然的静止姿势。我们将发生在动态动作之间的等待或静止状态的细微动作称为“静止动作（Idle Motion）”。静止动作可以分为以下三类。

- **姿势转换**：这一类静止行为涉及从一种到另一种静止姿势的转换。比如说在站立的时候保持平衡，或者是在不同的躺卧或坐姿间切换。
- **连续的小姿势变化**：由于呼吸、保持平衡等因素，人类身体会不断地进行微小的运动。当角色中缺少这些运动的时候，看上去非常不真实。
- **附加静止动作**：这些类型的动作一般都涉及与某人自己的身体交互，

Download at www.Pin5i.Com

比如接触脸或者头发，或者将手放入口袋中。

除了[Perlin95]描述一个基于噪音函数作用于关节的动作产生器外，目前几乎没有关于模拟静止动作方面的工作。不过人类静止动作影响了所有关节，无法通过噪音函数轻易解决。除了对独立关节生成随机运动外，另一种可能性是以捕获的动画文件为基础生成静止动作。这样就能生成影响所有关节的静止动作，但是会非常僵硬而且具有很大的重复性。其次，这种方法也会产生不同动画剪辑间的过渡问题。

我们的方法利用已有的动画库。在我们的例子中，动画通过动作捕获系统记录来生成，当然也可以通过手工直接设计。目前已经有大量的技术可以从动画库中生成新的动画，但是很少应用于创建静止动作。Kovar 等人[Kovar02]提出了动作图方法，基于动作库生成动画及其之间的过渡。该方法可用于示范动作之外其他特殊的问题，然而该技术依赖于经过了预处理阶段的封闭的动作库。我们的关键因素是：希望平稳地将其应用到任意动作捕获系统生成的动画或者是直接计算的动画设计中，其原因是静止动作很少是动画设计者的最终目标，而是在不同活动的动作间建立过渡的一种方便的方法。

因此，需要有激活和停止的功能供 AI 或者脚本来触发。Li 等人[Li02]用线性动态系统建模并将动作分成 Textons。另外，Kim 等人[Kim03]提出了分析声音的方法，基于通过音频流识别的节拍生成有节奏的动作。这些技术最适合于迪斯科舞曲之类具有节奏模式的连续动作。Pulen 等人[Pulen02]提出了辅助建立关键帧动画的方法，该方法基于用户设计的关节动画的子集自动生成全部角色动作。这样动作捕获的数据可用于合成还没有制作动画的关节以及提取可被用户控制关节的纹理（类似噪音）。

我们的方法试图将需要用户干涉的部分最小化，同时直接面向实时应用。Lee 等人[Lee02]提出了对能够通过动作捕获获取的动作进行采样，然后进行动作合成的方法。Arikan 等人[Arikan02] [Arikan03]完成了在带标注的动作库上定义模型图的工作。

5.1.2　人体动画的主分量

目前已有很多为虚拟角色设计动画的技术，其中如下两种是非常常见的技术。

- **关键帧**：动画由一组关键帧（由设计者设计或者自动生成）通过插值的方法组成。虽然该方法可以实现非常灵活的动画，但是如果艺术家们没有投入大量的时间将会导致其真实性较低。
- **预录制的动画**：动画由动作捕获和跟踪系统录制。动画的真实性较高，但并不是很灵活。目前已有一些方法用于克服该问题[Bruderlin95]。

类似 PCA 的方法可以决定数据集中变量间的依赖关系。PCA 的结果是一个矩阵（由一组特征向量构成），它将一组部分依赖的变量转变成另一组具有最大独立性的变量。PC 变量按照它们在数据集中出现的次数排列，较低的 PC 索引表示在数据集中具有高出现率，较高的 PC 索引表示其在数据集中具有较低的出现率。同样的，PCA 的作用是通过移除变量集中高 PC 索引的项来减小数据集变量的维度。和关注类人动画一样，我们也希望尽可能地不受到几何体的约束，根据明确说明类人动物标准骨骼的 H-Anim 标准[H-Anim05]，我们在潜在节点骨骼上执行 PCA。

要执行 PCA，我们需要将数据集里动画序列中的每一帧转换到一个 n 维向量。在我们的

Download at www.Pin5i.Com

例子中，每个姿势（关键帧）表示为 25 个节点（包括根节点）的旋转和根节点的移动。直接在旋转矩阵中应用 PCA 并不能得到满意的结果，其原因是旋转空间存在非欧氏几何体。不过我们可以构造一个线性版本的旋转（或者是正交旋转）矩阵，即“指数映射”旋转的指数映射表示法，通过将矩阵对数应用于正交旋转矩阵上来求得该指数映射的表示。该旋转的表示法对于运动插值[Alexa02][Park97]非常有用，因为它允许在旋转上执行线性操作。旋转的矩阵对数是斜对称矩阵，这样就能表示为一个向量。在我们的例子中，使用 25 个节点的旋转加上一个全局平移来定义一个姿势，因此向量的维度为 78。

虽然方向上的指数映射表示法使得控制和插值更加容易，但是整个范围内也存在一些奇异点。对于任意给定的 $aX+bY+cZ+d=0$ 对数映射可以根据绕轴 v 的 $2n\pi+\theta$ 旋转和轴 $-v$ 的 $2n\pi-\theta$ 旋转将每个方向映射到无穷数量的点上[Grassia98]，因此必须采取措施来保证奇异点不会干涉插值（请参阅[Grassia98]中更详细的讨论）。

我们已经在姿势向量上运用了主分量（PC，Principal Component）分析，其结果是一个 78×78 的主分量矩阵。按照如下方法，可以将由一组表示旋转的四元数和一个全局平移变换表示的姿势转换成 PC 向量：

```
void toPC(Posture* p, PCPosture* g) {

    // 用于与 PC 矩阵相乘的向量
    // PCSIZE = 78
    float* quatTransVector = new float[PCSIZE];

    // 获取全局位移量
    translation root_trans;
    p->getTranslation(root, root_trans);
    quatTransVector[0] = root_trans.x;
    quatTransVector[1] = root_trans.y;
    quatTransVector[2] = root_trans.z;

    // 现在放入所有节点 (PCJOINTS = 25)
    // getRotation 函数计算了节点旋转的指数映射表示
    for (int joint = 0; joint < PCJOINTS; joint++) {
        skewSymTp rot;
        p->getRotation(joint,rot);
        int location = joint*3 + 3;
        quatTransVector[location] = rot.a;
        quatTransVector[location+1] = rot.b;
        quatTransVector[location+2] = rot.c;
    }

    // 计算转置的 pcmatrix 和 quatTransVector 的乘积
    for (int row=0; row<PCSIZE; row++) {
        float currPC = 0.0f;
        for (int column=0; column<PCSIZE; column++)
            currPC += globalpcmatrix_[column][row]
                * quatTransVector[column];
        g->setValue(row,currPC);
```

Download at www.Pin5i.Com

```
        }
        // 清理内存
        delete [] quatTransVector;
    }
```

从 PC 表示法到四元数姿势的转换也是类似做法，可以从随书光盘中找到。一旦你获得姿势的 PC 表示，你就可以从由主分量结构带来的维度减少中受益。

在我们的例子中，索引大于 25 的分量几乎都为零，从而可以将其删除。虽然在某些情况下可能产生错误动作，但是删除这些分量极大地减少了需要处理的数据总量，同时也降低了某些需要通过网络发送数据时对带宽的需求。作为扩展，维度的减少也可以直接与细节层次的算法关联，根据角色在场景中的距离减少姿势维度。

5.1.3　姿势变换

本小节中，我们将提出一种从预先记录的剪辑创建动画数据库的方法，我们可以从该数据库中自动生成新的真实的动画。可以随意激活或者停止这些动画，从而避免两个动画剪辑之间不真实的静态姿势。下一小节中我们将解释如何将用于姿态变换的静止动作加入你的角色动作中。下面以人类站立的记录为例来说明我们的技术。

1. 基础动画结构

由于疲劳等因素的影响，人类站立时其姿势每过一段时间会改变一次。在这些姿态的改变过程中，人物始终处于休息的姿势。我们可以将休息姿势分成多种不同类型，比如左脚平衡、右脚平衡或者双脚平衡，等等。同样的，给定某人站立的记录，我们可以从中提取形成不同类型姿势过渡过程的动画片断。这些动画片断综合起来组成一个用于合成平衡动画的“数据库”。为了使数据库更加有用，每一个可能的类型转换至少需要一个动画，不过每种过渡超过一个动画会更好，因为这样会在后面的动作中产生更多变化。将数据库中的记录剪辑加以融合和修改，从而生成新的平滑过渡的动画。

数据库中每个动画片断都有一个起始姿势和一个终止姿势，在我们想把这些片断连到一起的时候，我们首先需要解决如何在不同动画片断之间平滑过渡的问题。为了实现这一点，我们通过起始和终止姿势间所有过渡来实现数据库的扩展。使用所谓“动画拟合”的过程，利用数据库中已有的动画来创建新的动画（如图 5.1.1 所示）。为了让该技术的结果尽可能真实，我们需要知道数据库中哪段动画能够最好地与起始姿势 p 和终止姿势 q 拟合。为了找出这段动画，系统必须选择第一帧和最后一帧分别与 p 和 q 最接近的动画。假定某姿势可以用一组 PC 值向量表示，定义两个姿势之间的距离为两个 PC 向量之间的加权欧氏距离，用如下例子加以说明：

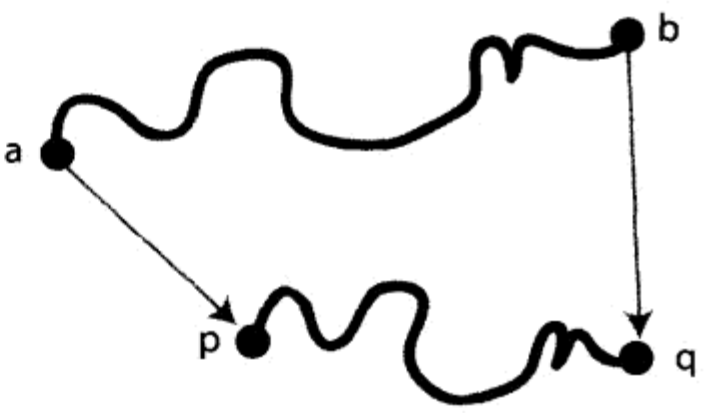


图 5.1.1　简单的例子：从姿势（a, b）到（p,q）的拟合动画

Download at www.Pin5i.Com

```
float distance(PCPosture* p, PCPosture* q, int* weight) {
    float dist = 0.0f;
    for (int i=0; i<PCSIZE; i++) {
        float distComp = p->getValue(i) - q->getValue(i);
        dist += weight[i]*(distComp*distComp);
    }
    return dist;
}
```

利用这些权重我们可以赋予姿势数据中频繁出现的部分更高的重要性，这样姿势数据中相关性较小的部分对姿势距离的影响也会较小。Kovar 等人[Kovar02]已经提出了姿势距离计算中的一些问题，他使用的是骨架驱动的点云。我们的系统中低 PC 索引的项比高索引的项具有更高的权重，这样姿势中最相关的部分也成为影响距离计算最重要的因素。有了最接近的动画，我们就能安排合适的动画，自始至终完成我们想要的动作。

```
void KeyFrameBodyAnimation::fit(KeyFrame* newkf0, KeyFrame* newkf1) {
    PCPosture* newkf0_pc = newkf0->getPCPosture();
    PCPosture* newkf1_pc = newkf1->getPCPosture();

    PCPosture* oldkf0_pc = keyframes_->begin()->getPCPosture();
    PCPosture* oldkf1_pc = keyframes_->end()->getPCPosture();

    float* offset0 = new float[PCSIZE];
    float* offset1 = new float[PCSIZE];

    for (int u=0; u<PCSIZE;u++) {
        offset0[u] = newkf0_pc->getValue(u)-oldkf0_pc->getValue(u);
        offset1[u] = oldkf1_pc->getValue(u)-newkf1_pc->getValue(u);
    }

    int size = keyframes_->size();
    for (int f=0; f<size; f++) {
        PCPosture* currFrame = keyframes_->at(f)->getPCPosture();
        for (int x=0; x<PCSIZE; x++) {

            // 获取原来的 PC 值
            float pcval = currFrame->getValue(x);

            // 应用偏移量
            pcval += (offset0[x]*float(size-f)/float(size);
            pcval += (offset1[x]*float(f)/float(size);
            currFrame->setValue(x,pcval);
        }
    }
}
```

由于我们在动画上仅执行了线性操作，因此使用该方法可以很快地扩展数据库。为了进一步提高速度，可以通过仅在主分量子集上运用拟合过程来充分发挥 PC 的性质。唯一需要注意的事情是动画拟合技术只能作用于位移总量较小的情况，幸运的是静止动作正属于这一类。

Download at www.Pin5i.Com

2. 排除平移项

由于后面我们将使用数据库中的动画来创建平衡变换序列，因此有必要运用规范化的步骤使动画间的相对位移最小。我们以第一帧到最后一帧的均值与原点（0,0,0）的距离作为每个动画序列的平移量（如图 5.1.2 所示）。

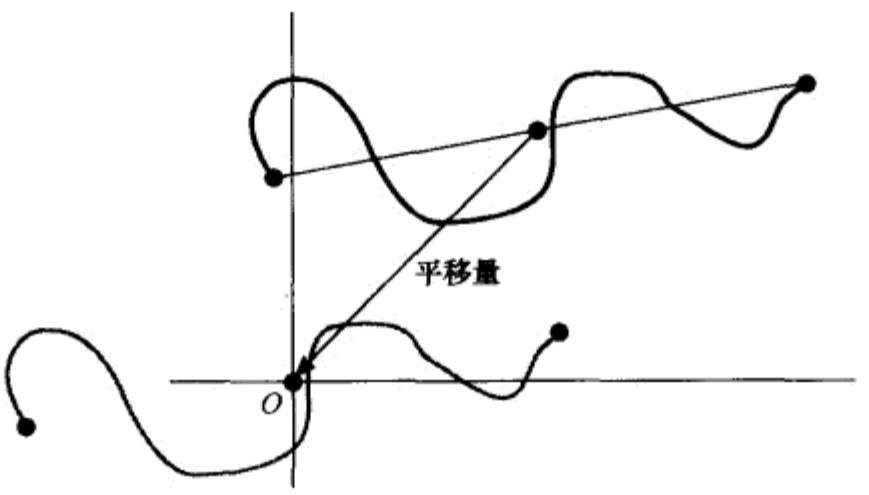


图 5.1.2　该图说明了一个动画序列在（x,z）平面（前侧面）上的平移量

要从动画中排除偏移，我们从每帧的根平移量中减去该偏移值。用下面的例子来说明：

```
void KeyFrameBodyAnimation::removeTransOffset() {
    // 计算偏移量
    translation firstTrans, lastTrans, offsetTrans;
    keyframes_->begin()->getPosture()->getTrans (root,firstTrans);
    keyframes_->back()->getPosture()->getTrans (root,lastTrans);
    offsetTrans = -(firstTrans+lastTrans)/2.0f;

    // 应用偏移量
    for (int i=0; i<keyframes_->size(); i++) {
        Posture* p = keyframes_->at(i)->getPosture();
        translation currTrans;
        p->getTranslation(root,currTrans);
        currTrans += offsetTrans;
        p->setTranslation(root,currTrans);
    }
}
```

当然也有其他计算平移量的方法，比如说在动画中搜索最中间的关键姿势，或者通过时间信息确定姿势变换的中间值。

3. 构造新的动画

利用上文描述的技术，我们现在可以扩展数据库从而实现起始和终止姿势之间的动画。扩展之后的数据库可用于通过序列化姿势变换片断来创建新的动画。在每个姿势变换片断之间插入一个暂停，这样角色就能在在姿势变换间等待一定时间。图 5.1.3 列出了一些我们用该系统创建的例子姿势。随书光盘中有更多示范电影。

Download at www.Pin5i.Com

图 5.1.3 使用静止动作引擎创建的范例姿势

虽然在我们的例子中仅在站立人的数据库中执行了 PCA 操作，但是该技术不局限于仅仅合成站立人姿势。数据库可以进行进一步扩展，不仅包含站立人的平衡姿势变换，同时还包括坐着的人改变姿势的动画，或者躺着的人的姿势变化。这样就会出现另一个问题：姿势的类型与一个或者多个对象相关——比如一把椅子或者一张床。但是，执行上文描述的标准化过程后，静止动作可以相对人在空间中的位置播放。

5.1.4 姿势的连续微小变化

除了我们在前面章节中讨论的平衡变换姿势外，小的姿势变化往往能够极大地提高动画的真实性。由于呼吸或者微小的肌肉收缩，人类不可能像机器人一样严格保持同样的姿势。合成这些小姿势变化的基本原则是使用主分量表示每一个关键帧。由于变化作用于 PC 上而不是直接作用于关节参数，因此可以生成顾及关节之间依赖性的随机变化。另外，因为 PC 表示了数据中不同变量间的依赖性，所以 PC 本身就是具有最大独立性的变量，我们同样也可以为了产生姿势变化单独地对待它们。

产生变化的方法是在每个主分量或者其子集上应用 Perlin 噪音方程[Perlin85]。这里我们提出另一个基于动作捕获数据的曲线形状来生成微小变化的方法。该方法应用统计学模型生成相似（随机）曲线，可以让一个特定对象的姿势变化保持某种已有的趋势，比如典型的头部运动。另外与 Perlin 噪音相比该方法具有完全自动的优势，而且不需要事先定义噪音的频率和幅值。我们分析那些不包含任何平衡变化或者其他没有微小变化的动画片断，将分析的结果用于生成附加在其他姿势上的类似变化。

对所选的动画片断进行规范可以排除基本姿势同时保留变化。因为这些变化会在平衡变化姿势之上合成，所以规范化势在必行。为了规范动画片断，我们计算片断中每个 PC 的均值，并从每个关键帧的 PC 值中减掉。这样就可以排除基本姿势而仅仅保留变化（见图 5.1.4）。毫无疑问，这种方法只能应用于基本姿势不会明显变化的动画序列。

1. 变化预测

现在我们描述如何根据动作捕获的数据预测变化。我们用图 5.1.4 中动画片断的 PC 值来说明这一过程。为了减少 PC 曲线上点的数量同时保留其大体上的形状，将 PC 值转换成一组极大值和极小值，如图 5.1.5 所示。这样我们将说明可以在给定前面点的情况下预测下一个最大值/最小值的技术。为了估算最大值和最小值，可以用一个简单的算法遍历所有点，给定误

Download at www.Pin5i.Com

差值 e，决定点是否是最大值或者最小值。

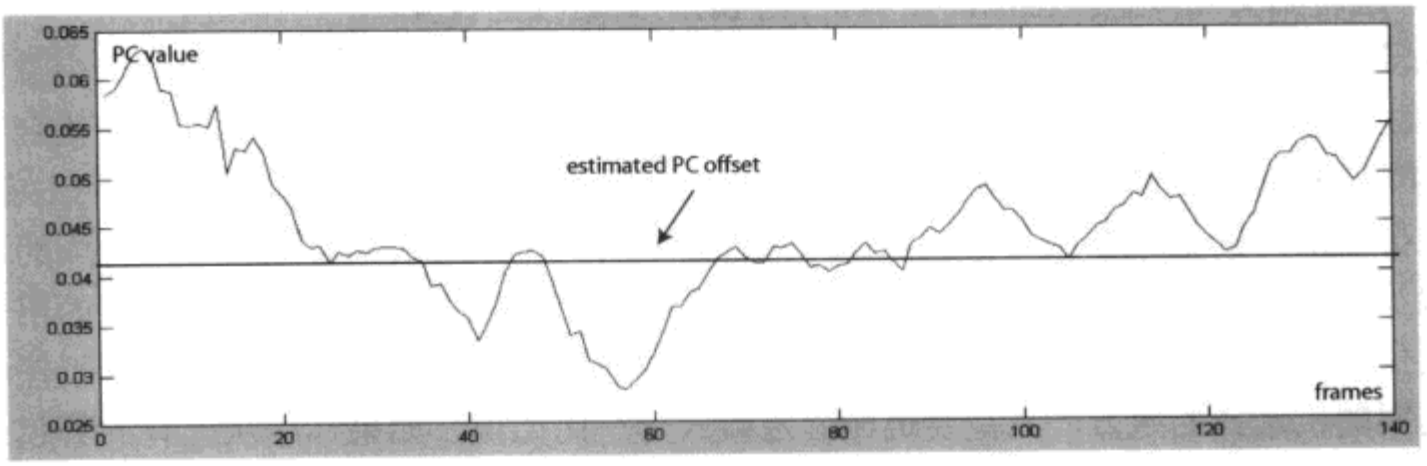


图 5.1.4　该例子表示动画片断中一个主分量的值，根据均值估算的偏移大约是 0.045

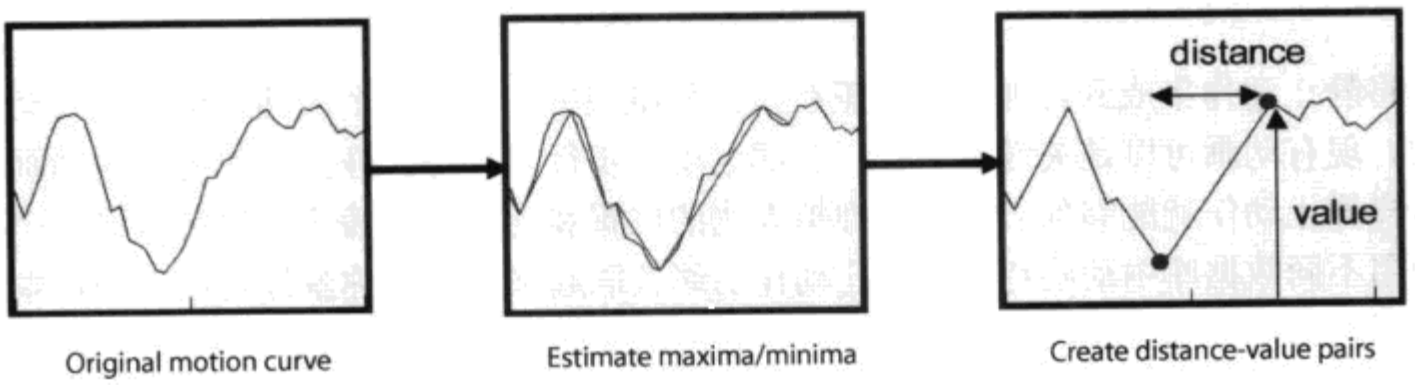


图 5.1.5　对已有微小姿势变化中一个 PC 的分析和分割得到一对“距离-值”

为了从数据中除去绝对时间，对每个最大值/最小值我们以毫秒为单位定义其与前一个最大值/最小值距离的差以及相应的 PC 值，如图 5.1.5 所示。下一步则是构造一个能够给定一对距离-值预测下一对距离-值的系统。这样生成的一对对距离-值就形成新的动画序列。

为了向系统中加入一些随机量，我们并不直接使用点，而是运用了点聚类的算法。在我们的程序中，我们已经应用了最小生成树聚类算法[Jain99]。该算法应用到距离-值空间后，产生一组距离-值配对附近的簇，如图 5.1.6 所示。从动作捕获的数据我们可以知道每对距离-值的后继者。利用这些信息，我们能够定义两个簇之间任意过渡的概率，生成“概率过渡矩阵”。为了确保总是存在过渡的可能性，我们不断合并簇直到不存在不可能从一个簇到另一个簇过渡的情况——这里基本的意义是：没有终点。根据如下算法我们可以得到能用于生成新动画的概率模型。

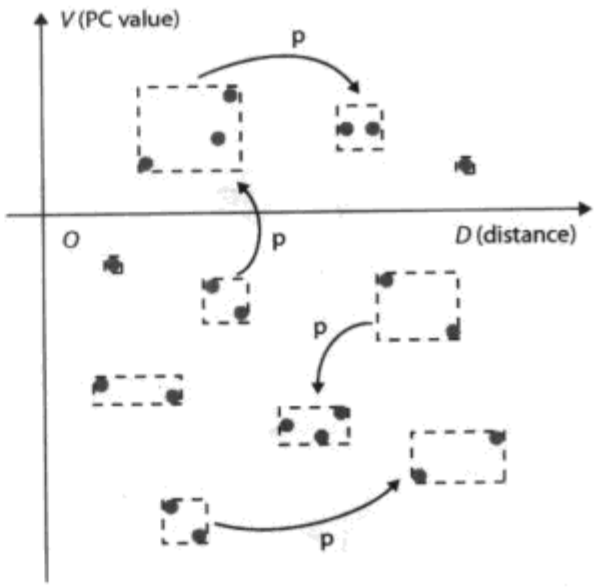


图 5.1.6　从动作捕获的数据获取的距离-值组合的集合，通过最小生成树算法聚类。箭头表示从一个簇过渡到另一个簇发生的可能性

- 设中间点位于时间 t=0 处；
- 随机选择一簇点；
- 随机选择该簇中的一个点（例如一对距离-值）；
- 在动画的时间 t+*distance* 处添加一个关键帧，给定 PC 值；
- 基于最后选择的簇，根据“概率过渡矩阵”选择下一个簇；

Download at www.Pin5i.Com

- 随机选择簇中的一个点（例如一对距离-值）；
- 跳转到第 4 步重复上面的步骤，直到动画到达预定长度。

由于动画并非从原点开始，而是从簇中随机选择的点开始，因此姿势变化很少重复。不过动画上的 PC 值曲线依然与分析的动画采样具有非常相似的结构。为了进行变化合成，需要将本节中描述的动画片断生成方法应用于 PC 值的子集上，对每个主分量独立地生成变化。由于主分量之间的依赖性相对很小，因此这样并不会产生不真实的结果。变化合成的最终结果也很少出现重复。这是因为在每个簇中随机选择点以及单独对待 PC。这里介绍的用于从给定曲线生成相似曲线的方法并不仅仅局限于用来合成静止动作。你可以将其应用于任何或多或少需要随机选择的地方，只是要遵循固定模式——比如说，如果你想从已有的看起来或者听起来一样的数据生成纹理或者声音，结果仍然存在差异。

2. 与实时系统集成

要将静止动作集成到可能有其他正在执行的游戏中，需要一个能够管理与现有动画混合的架构。现有动画可以是关键帧手势、头部运动、步行，等等。静止动作与其他动画混合起来，这样静止动作就能够独立于对应虚拟人物的位置和旋转进行播放。你可以将每个虚拟人物与使用不同数据库中动画片断的静止动作引擎关联起来。这样你就能将在一个场景中放置几个虚拟人物而且都能显示不同的静止动作行为。文献[Egges04a]和[Egges04b]中给出了关于如何将静止动作与其他动作混合起来的更精确的描述。

将静止动作与其他动画混合起来可以分两步进行，如图 5.1.7 所示。首先，将平衡变换动作通过权重因子与其他动画混合起来。该混合作用于关节这一层面，允许定义关节掩码，指定动画中的哪些部分将会在最后的动画中删除。使用掩码非常重要，因为某些特定的动画，比如步行或者跑步，不应该与任何平衡变换动作混合。同样的，掩码也指定了动作中的哪些部分不能改变。例如，在手势动画中，运动的手臂不应该与静止动作中静态手臂姿势混合，这样会形成不明显的手势。

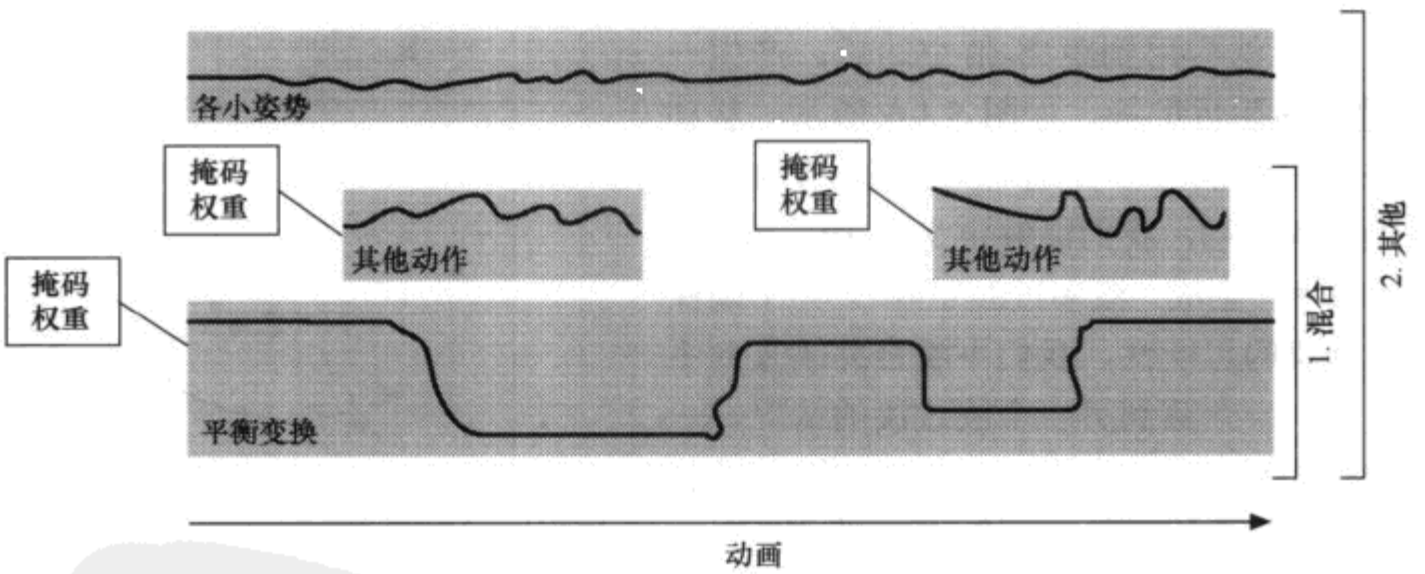


图 5.1.7 将静止动作集成到实时动画框架中

平衡变换动作和其他动画混合后，小姿势变化就被添加到了动画中。小姿势变化的附加部分便可以在 PC 空间中出现。对于动画的每一帧，混合动画和变化动画的 PC 值叠加起来，形成最终的动画。该方法之所以能够良好地工作，是因为即便该变化增加了虚拟人物在静止

Download at www.Pin5i.Com

姿态过程中的真实性，它们在动画片断中的存在也很难被注意到。

5.1.5　总结

本节描述的技术可以建立一个基于动画和姿势的数据库，真实地构造与数据库中的动画和姿势连贯的静止动作的系统。由于最消耗时间的运算已经在数据分析阶段预计算完成，因此该方法可以在任何标准的 2GHz 桌面型 PC 上实时地生成和播放动画。另外，PC 值和旋转矩阵之间的转换通过对每帧应用一个 3×3 的矩阵（或者四元数）的对数/指数运算以及矩阵乘法组成，因此它也遵循实时操作的限制。

5.1.6　参考文献

[Alexa02] Alexa, M., "Linear combination of transformations." *SIGGRAPH 2002*, 2002: pp. 380-387.

[Arikan02] Arikan, O., et al., "Interactive motion generation from examples." *Proceedings of ACM SIGGRAPH 2002,* 2002.

[Arikan03] Arikan, O., et al., "Motion synthesis from annotations." *ACM Transactions on Graphics,* 22(3), pp. 392-401, 2003.

[Bruderlin95] Bruderlin, A., et al., "Motion signal processing." *Computer Graphics,* Vol. 29 (Annual Conference Series), 1995: pp. 97-104.

[Egges04a] Egges, A., et al., "Personalised Real-Time Idle Motion Synthesis." *Pacific Graphics 2004,* Seoul, Korea: pp. 121-130, October 2004.

[Egges04b] Egges, A., et al., "Example-based idle motion synthesis in a real-time application." *CAPTECH Workshop,* 2004: pp. 13-19.

[Grassia98] Grassia, F. S., "Practical parameterization of rotations using the exponential map." *Journal of Graphics Tools,* 3(3): 29-48, 1998.

[H-Anim05] H-ANIM Humanoid Animation Working Group, "Specification for a standard humanoid." August 2005. Available online at *http://www.h-anim.org/*.

[Jain99] Jain, A. K., et al., "Data clustering: a review." *ACM Computing Surveys,* 31(3): pp. 264-323, 1999.

[Kim03] Kim, T. H., et al., "Rhythmic-motion synthesis based on motion-beat analysis." *ACM Transactions on Graphics,* 22(3): pp. 392-401, 2003.

[Kovar02] Kovar, L., et al., "Motion graphs." *Proceedings of SIGGRAPH 2002,* 2002. [Lee02] Lee, J., et al., "Interactive control of avatars animated with human motion data." *Proceedings of SIGGRAPH 2002,* July 2002.

[Li02] Li, Y., et al., "Motion texture: A two-level statistical model for character motion synthesis." *Proceedings of SIGGRAPH 2002,* 2002.

[Park97] Park, F., et al., "Smooth invariant interpolation of rotations." *ACM Transactions on Graphics,* 16(3): pp. 277-295, July 1997.

Download at www.Pin5i.Com

[Perlin85] Perlin, K., "An image synthesizer." *Proceedings of the 12th Annual Conference on Computer Graphics and Interactive Techniques,* ACM Press, 1985: pp. 287-296.

[Perlin95] Perlin, K., "Real time responsive animation with personality." *IEEE Transactions on Visualization and Computer Graphics,* 1(1), 1995.

[Pullen02] Pullen, K., et al., "Motion capture assisted animation: Texturing and synthesis." *Proceedings of SIGGRAPH 2002,* 2002.

Download at www.Pin5i.Com

5.2 用自适应二叉树进行空间剖分

Martin Fleisz

对于空间剖分目前已有多种不同的技术，每种都有各自的优缺点。大多数情况下其不足之处取决于处理的几何体的特性。例如，某种技术对室内场景非常有效，但是可能对大型室外场景毫无作用。

不过，作为另一种可选的剖分方法，Yann Lombard 介绍了自适应二叉树（ABT，Adaptive Binary Tree）的概念[Lombard02]。ABT 与其他现有技术的不同之处在于，ABT 基于场景的几何体对空间进行分割。分割体可以动态改变大小，也能够部分重叠从而减少分割平面的数量。分割平面由一个考虑了多种不同参数的评分系统决定，就像空间中有大量分割面片和分割平面的位置。由于这种自适应的行为，ABT 非常适合剖分任何类型的场景。

5.2.1 如何建立自适应二叉树

首先，我们可以通过生成与坐标轴对齐并且覆盖整个场景的包围盒顺利地创建树的根节点。然后开始递归细分函数，每一步包围盒都由垂直于3 个主坐标轴之一的分割平面分开，产生两个子节点。每个子节点都由与坐标轴对齐的包围盒包围，可以很容易地由视锥体进行校验。函数终止的条件是：如果一个节点中面的数量到达预定限制或者树的深度太大。

目前创建 ABT 相对比较容易，不过仍然有一个问题：如何判断哪样的平面可以有效地分割节点？在 Lombard 首次 ABT 的描述中，他将找到这样平面的过程作为不同标准（以及其对应功能）的数学最小化问题加以说明[Lombard02]。现在我们来进一步了解这些标准。

1. 最优空间定位

主要目标是将得到的子包围盒最大坐标轴最小化。该函数偏向于分割当前节点包围盒最大坐标轴的平面。

$$f_1(pos) = \max(split_x_axis, split_y_axis, split_z_axis) \quad (5.2.1)$$

2. 子节点体积

两个得到的子节点必须具有近似相等的体积。该函数偏向于将当前节点分割成对等体积子节点的平面。

$$f_2(pos) = abs(volume_child1 - volume_child2) \quad (5.2.2)$$

Download at www.Pin5i.Com

3. 面的数量

表面应该被均匀地分布到所有叶子节点上。

$$f_3(pos) = abs(numfaces_child1 - numfaces_child2) \tag{5.2.3}$$

4. 分割面的数量

该函数计算平面分割的面的数量。平面分割的面越多，越不适合作为分割面。

$$f_4(pos) = total_number_of_split_faces \tag{5.2.4}$$

要找到最好的剖分平面，上面所有函数必须最小化。这一点并不是和开始可能听到的一样琐碎，因为函数 f_3 和 f_4 都呈现出离散行为，这就意味着它们能有很多局部极小值（取决于几何拓扑结构）。另外，每个函数都有一个权重因子，它们提供了更可控的创建过程。利用这里所有的信息，我们最后得到如下获取可行平面时必须最小化的方程。

$$f(pos) = f_1(pos) \cdot w1 + f_2(pos) \cdot w2 + f_3(pos) \cdot w3 + f_4(pos) \cdot w4 \tag{5.2.5}$$

理想的权重因子分布取决于引擎的需求。如果场景中多边形数量已经很大，则需要避免创建更多的渲染数据，这样函数 f_4 就需要赋予比其他函数高的权重。

找到分割平面以后，一个节点会分成两部分。虽然函数 f_4 已经在求解面的过程中最小化了，但是仍然会有很多面片被分割。ABT 通过扩充节点的包围盒克服了该问题。该方法的实际原理是 ABT 节点并不需要表示空间中清晰的范围，近似即可[Lombard02]。为了有效地保持树的层次结构，两个子节点也不能过度扩充。Lombard 建议扩充因子的最大比例应为子节点包围盒的体积。使用 5%～10%的因子可以防止将近 90%的面被分割。

现在我们完成了对树的创建。根据 Lombard 建议，在一定环境下，我们必须优化 ABT 的包围体积，不过这样的情况很少发生。最后，我们将会得到一颗完美的包含所有可用于渲染的几何数据的平衡树。

5.2.2 ABT 实现细节

1. 搜索分割面

从表面上看，ABT 背后的概念似乎非常容易而且合乎逻辑，不过在实现过程中事情可能没有起初看上去那么简单。在树的创建过程中最困难的步骤无疑是求解分割平面。同时，这一步也是 ABT 创建过程中最重要的部分。所选平面将当前几何体集合分割的越完美，最后将树用于视锥体裁减时会得到更好的结果。现在我们进一步看看平面选择部分的具体实现，如图 5.2.1 所示。

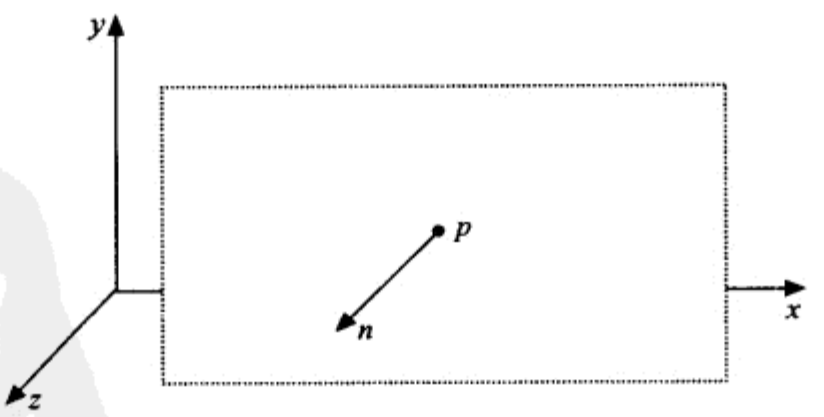


图 5.2.1 由法线 n 和点 p 定义的分割面，与 z 轴垂直

Download at www.Pin5i.Com

如前文所述，分割面的方向总是与 x、y 或者 z 轴垂直。平面可以通过其法向量和穿过平面的点决定。在开始搜索之前，我们已经有了大量关于我们寻找的面的信息。首先我们知道法向量——可以简单地让主坐标轴垂直于我们的平面。其次，需要一个位于平面上的点，或者更确切地说，仅仅需要知道该点的一个坐标，即该点在法线所在坐标轴的位置，其原因是该平面可以生成另外两维，这就意味着我们可以选择相伴坐标的任意值。下面是简化后的关于如何实现平面搜索函数的概要：

```
Plane getBestSplitter(AABB curAABB, Surfaces *pSurfaceList,
    int iNumSurfaces)
{
    Plane bestPlane;
    float fBestPlaneScore = 1000000.0f;

    // 沿每条坐标轴扫描
    for(int iCurAxis = 0; iCurAxis < 3; iCurAxis++)
    {
        // curPlane 的法线和点都初始化为 0/0/0
        Plane curPlane;

        // 初始平面的法线（与扫描轴平行）
        curPlane.Normal.xyz[iCurAxis] = 1.0f;

        // 沿当前坐标轴对一些平面进行采样
        for(int i = 0; i < SAMPLES; i++)
        {
            // 计算缺失的坐标，完成平面计算
            curPlane.Point.xyz[iCurAxis] = getSample(i);

            // 计算该平面分数
            float fScore = getPlaneScore(curAABB, curPlane,
                                         pSurfaceList);

            // 检查是否得到了更好的平面
            if(fScore < fBestPlaneScore)
            {
                bestPlane = curPlane;
                fBestPlaneScore = fScore;
            }
        }
    }
    return bestPlane;
}
```

如果你正在寻找关于本文更详细的信息，可参阅随书光盘上 ABT 演示程序的源代码。

2. 决定平面的得分

既然我们已经选择了一个平面作为潜在的候选分割面，我们必须利用已经描述过的评估

Download at www.Pin5i.Com

函数计算其分数。因为其定义可能有些不清楚，下面是可能的实现方法[Campbell03]：

$$f_1=1-Length\ (ThisAxis)\ /Length\ (LargestAxis) \tag{5.2.6}$$

函数 f_1 计算当前扫描的坐标轴的长度与包围盒最长轴的比例。如果分割最长轴，则结果为 0。如果分割与最长轴大小接近的轴，该函数将返回比短轴小的值。

$$f_2=2*fabs\ (0.5-SplitPercent) \tag{5.2.7}$$

方程 5.2.7 中 f_2 的 *SplitPercent* 参数表示分割在体积上能够到达的纵深距离（比如 0.25 表示四分之一，0.5 表示一半，等等）[Campbell03]。当分割平面与当前包围盒距离增加的时候，该方程返回较大的值。

$$f_3=fabs\ (NumFrontFaces-NumBackFaces)\ /NumFaces \tag{5.2.8}$$

NumFrontFaces 和 *NumBackFaces* 分别表示在分割平面前面和后面的面的数量。差别越大，f_3 得到的结果越差。

$$f_4=NumSplitFaces/NumFaces \tag{5.2.9}$$

方程 5.2.9 评估了当前几何体集合中分割面片（由 *NumSplitFaces* 参数给出）到面片总数的比例。

3. 控制平面选取

ABT 的实现过程中常常碰到的一个问题是不同权重因子的分布问题。对于该问题简短的回答是：没有在所有情形下都起到很好作用的标准。如果你有大量几何数据，应该避免由于分割而产生更多的几何体。因此需要给函数 f_4 比其他高的权重。或者说你想分割从均匀分布的几何高度场数生成的地形。在这种情况下，几乎没有什么需要分割，因此也不必关心函数 f_4。（即使平面没有与几何体完整对齐，相关包围盒也可以避免分割）。你可以用这些信息给函数非常低甚至是 0 的权重，让其他参数显得更重要。

实际上，面评分函数中的权重因子为用户提供了一个非常重要的特性。在它们的帮助下我们可以在不修改任何代码的情况下控制整个树的创建过程。你可以对不同的场景类型运用不同的权重，并且调节到让 ABT 获得最佳性能的位置。

4. 处理“临界面”

如果进一步观察创建的 ABT，你会注意到这样的问题：某些叶子节点包含一些具有相同材质的面片。渲染这些小的数据集对性能有负面的影响，因此应该避免。这里有一些避免“临界面”的解决方法[Lombard03]（见图 5.2.2）。

要处理该问题，第一种可能的方法是扩展分割面的分数计算方程，将材质边界考虑进去。不过，如 Lombard 所说，这种方法是否适用取决于引擎中对材质管理的具体实现。

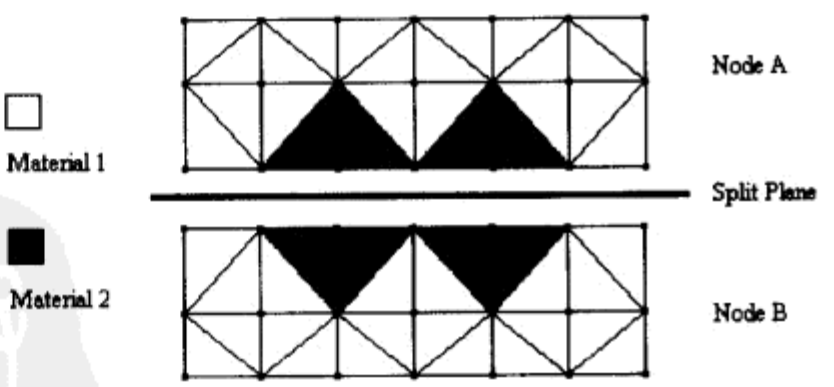


图 5.2.2 将一个节点分成两个子节点后的临界面（材质 1）

Download at www.Pin5i.Com

另外一种克服该问题的方法是重新插入临界面。标记这些面以后，你可以尝试将它们添加到邻近的已经包含具有这些材质的面的节点中。不过，该方法会导致其他节点的扩充，这样对树的定位有不利的影响。注意，根据 ABT 创建过程最后的描述，你必须对树的包围盒进行优化。

Lombard 使用的第三种方法与其他方法完全不同。标记临界面后，将它们存放到“隔离列表”中，并从叶子节点中移除。创建过程完成后，我们得到了沿着一列难以归类的面片的主 ABT。现在我们在临界面上再次创建 ABT。得到的树将会比主 ABT 树浅很多，并且包含更少的面片。共享同一材质的面现在合并到了一个节点内，这样的节点会有更大的体积。当然，现在我们必须在渲染时遍历两棵树，从性能的方面来说也会增加一点点开销。另一方面，渲染数据现在对硬件更友好了，可以全面提升性能。

5.2.3　找到合适的分割面

创建 ABT 的过程中最困难的部分就是找到最合适的分割面。空间中可能存在无数的分割面，因此我们必须选择一个小的子集进行操作。Lombard 的第一个 ABT 实现使用神经网络来找到合适的面。不过该方法的缺点是难以实行，尤其是在数据量较大的时候[Lombard03]。因为分割面的选择在 ABT 的创建过程中非常重要，下面几小节说明了一些确定合适分割面的途径。

1. 线性采样

线性采样是直接快速的找到分割面的方法。这样一个集合可能包含将当前的体积分割成 10%、20%或者在你期望的轴上任意长度比例。虽然该方法非常容易实现，但是我们仍然不推荐使用该方法建立 ABT。大多数节点会包含“热点区”，这意味着其几何体集中在一些小的最容易被该方法遗漏的子区域[Lombard03]。

2. 连续逼近

获得分割平面的另一种方法是用连续逼近的方法。我们首先选择沿三主轴之一在中间位置分割的平面，并计算其得分。现在选择两个分割了四分之一体积的子平面，同样也是沿着第一个平面的方向。选择下一个最高分的面作为中心平面，重复前面的步骤，唯一的区别是与新的子平面的距离现在切成了一半（仅有原体积大小的 1/8）。图 5.2.3 显示了该方法的前 5 步。逼近可以停止于数次迭代，或者是如果两个子平面都比中心平面得分更低。你可能已经注意到该过程与二分查找有些类似。

3. 随机采样

另外一种简单易行的方法是随机采样，简单选择随机体中随机位置的平面并计算其得分。最后将得分最高的平面作为分割面。这种技术的缺点之一是可能遗漏热点区域——与线性采样的情形一样。使用好的随机数生成器和足够的采样点可以避免该问题。

Download at www.Pin5i.Com

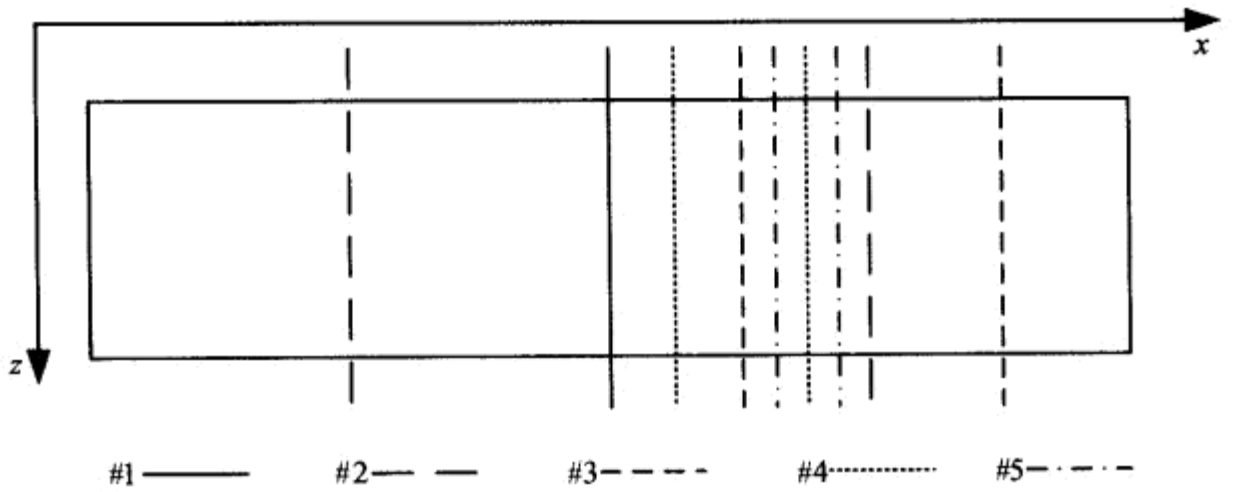


图 5.2.3 用交替的子平面选择进行连续逼近，从右边开始

4. 不均匀采样

正如已经提到的一样，利用均匀采样的策略并不是找到分割面最好的方法。Mirko Teran-Ravniker 在其 ABT 创建中使用基于正弦波的非均匀采样的方法。通过这种方法，测试平面会集中在中间，因而得到了相当好的结果。

5. 所有顶点法

在 Lombard 的研究中，他提出了下面这种数学上完美的解决方法。如果进一步研究，我们可以对评分方程中的两种不同的方法加以区分。首先，我们有普通的线性函数，比如“在空间中的位置”和“子体积”函数（f_1 和 f_2）。第二种称为“分段定义”的函数，例如“子面片”和“分割面片”函数（f_3 和 f_4）。如果将这些函数用图表绘出，给出在重要的坐标轴上所有可能的位置，我们会得到量化的图表。这也意味着你将仅得到包含特定长度上的常量的整型数字，并突然跳跃到另一个整数。参考图 5.2.4 和图 5.2.5 说明了分段定义的函数 f_4。

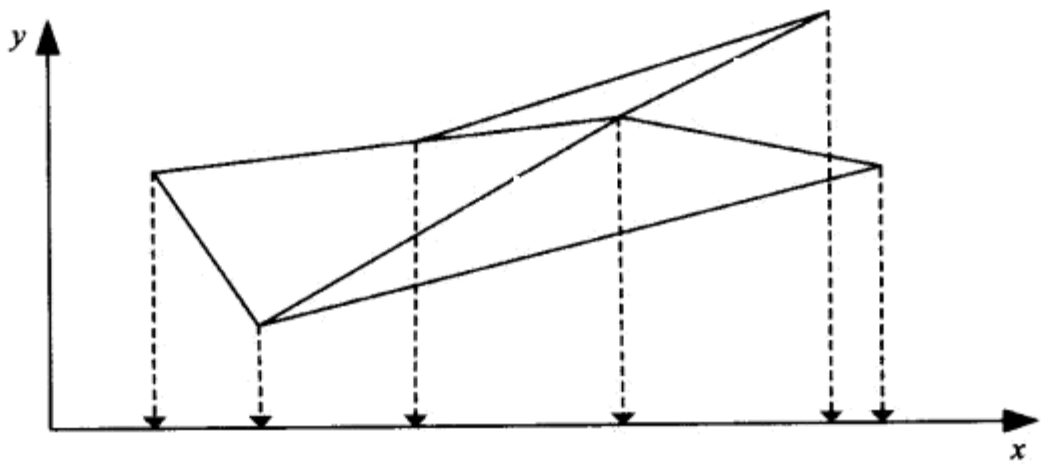


图 5.2.4 将一些三角形的顶点投影到 x 轴上

从图 5.2.5 中我们可以看出，函数 f_4 在图上的跳跃不是随机的，它们总是与面片上投影到正在扫描的坐标轴上的顶点的位置保持一致。这说明了两点，分割面片的数量是常数，f_4 将总是返回同样的结果。如果我们现在从分析上解出两点间的线性方程，我们将会得到局部最小值。如果我们为所有其他顶点重复该操作，我们将得到完美的全局最小值。

使用该方法我们可以创建完美的 ABT。不过它也有一个缺点：非常消耗时间。在最坏的

Download at www.Pin5i.Com

情况下，如果场景包含 n 个顶点，你将需要计算 $3\times(n-1)$ 个平面的分数。因为很多顶点投影到正在扫描的坐标轴上相同的点上，因此这个数量通常会小一点。通过仅扫描最长轴来寻找分割面的方法可以进一步地减少这个数量。记住在这种情况下，选择权重因子时平面方程中 f_1 的参数将总是 0，不会在渲染中起作用。

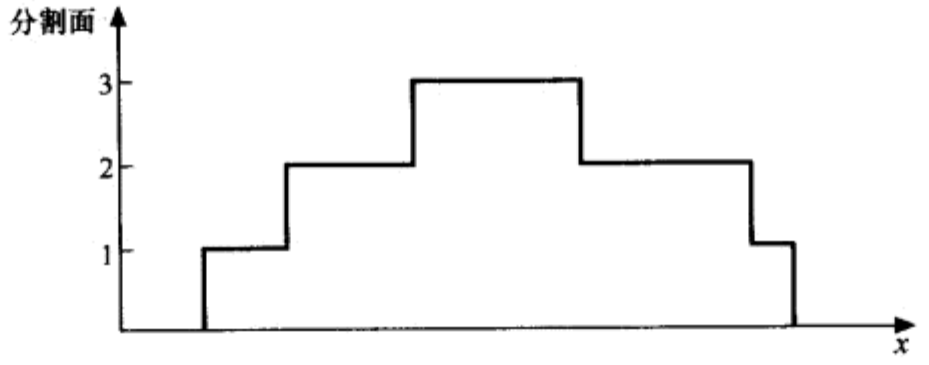


图 5.2.5　相应的分段定义的函数 f_4

6. 统计学的加权多重采样

Andy Campbell 提出了非常快速而且令人惊奇的工作良好的方法来找到合适的用于分割的面。首先，我们必须计算当前几何体集合的均值点。然后计算标准差，从而得到顶点在空间中分布的统计数据。下一步是分别在均值两边标准差的范围内采样 10 个平面。平面可以相互独立地均匀地采样，也可以增加距离让采样点距离均值更远一些。

和所有顶点法相比，该方法的优点是它只需要对更少的面进行评估。也有可能将这种方法和其他任何技术合并。该方法并没有对整个体积进行采样，你仅需对与均值在标准差范围内的面积进行采样。该技术的结果和所有顶点法得到的结果几乎一样好。

5.2.4　在动态场景中使用 ABT

当前状态下的 ABT 非常适合渲染静态几何体。但是现代游戏中，用户期望有些特定的动态物体，这样就需要游戏开发者在引擎中加入动态对象。和静态几何体相比，动态对象能够在每一帧中改变它们的位置。如果我们应用一些微小的改动，我们依然可以用 ABT 处理动态对象[Lombard02]。

我们假设每个动态几何体块有一个与坐标轴对齐的包围盒，并且不管对象怎么移动总是跟随对象。我们现在所要做的是再联合叶子与根节点之间所有节点的包围体，如图 5.2.6 所示。

你会立刻注意到父节点包围盒的大小会随着对象的移动增加。该技术的问题是随着时间的流逝，如果一个对象移动了很长一段距离，ABT 会慢慢地退化掉。这就意味着树裁剪的效率会进一步降低，造成性能上的影响。Lombard 提出的解决方法是对每个节点引入一个退化因子，表明当前节点与其原体积相比的扩展程度。如果该因子超过了某一阈值，整个分支需要立即重建[Lombard02]。

另外一种处理动态对象的方法是将对象移到树上面更高一级的位置，这样它们就符合在节点包围体中的条件。这种方法的问题是对象将很快移到树的上端，使得树不再可用。这是因为两个局部接近的节点在层次型树结构中不一定是接近的。因此，如果一个对象跨越了两

Download at www.Pin5i.Com

个这样的节点的边界，它将在树上向上滑动，最后停止在根节点中。其结果是大量对象将会被定位在树的上层区域中[Lombard03]。

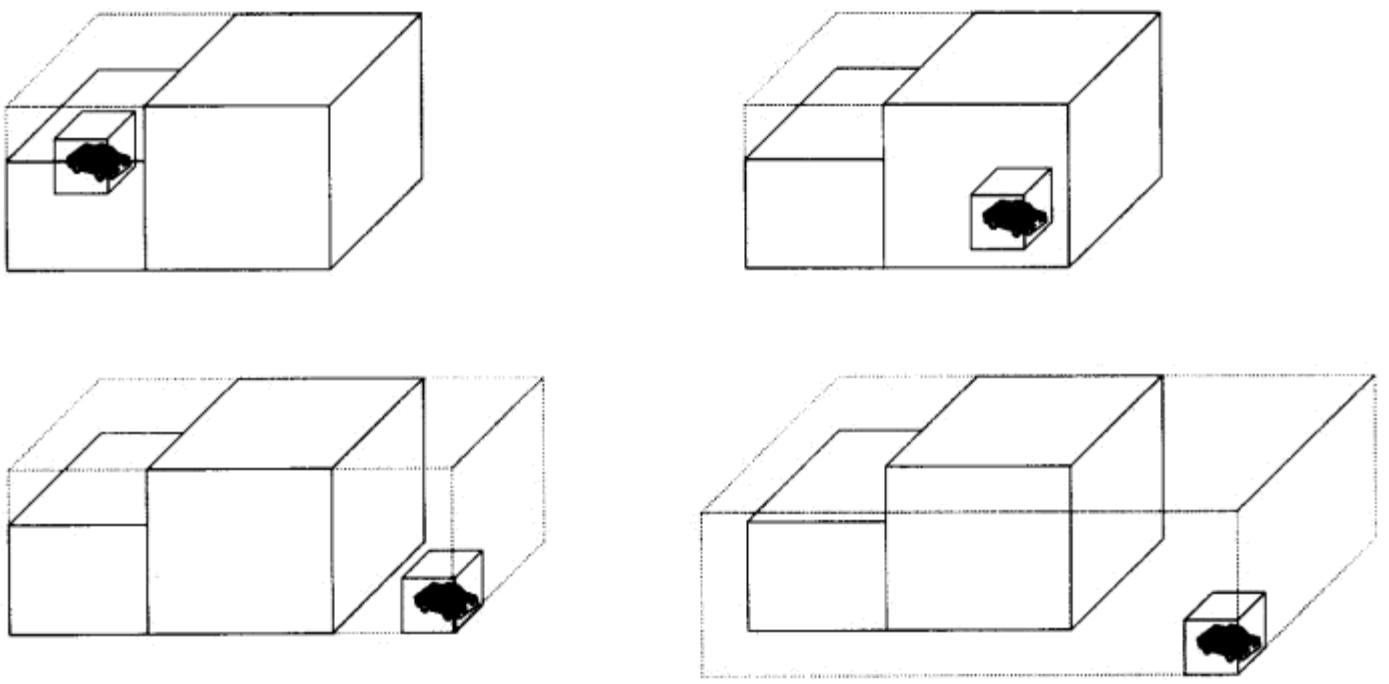


图 5.2.6 在 ABT 中移动的动态对象

可以看到，ABT 可用于管理包含动态对象的场景。不过，在这一方面仍然有可以提升的空间。通常情况下，将这些对象从它们的静态副本分离开并将其放入它们自己的空间树中是一种很好的方法，不过这样也需要处理和管理第二棵树。

5.2.5 渲染 ABT

渲染 ABT 非常简单而且极其高效。从根节点开始，你所要做的事情只是检查两个子节点的包围盒是否在视锥体之内。对每个通过检测的子节点，重复这一步骤直到任一子节点都不可见（例如，其包围盒位于视锥体之外）或者你到达了一个叶子节点。第二种情况下你仅需简单地渲染存储的几何数据。

不过，渲染不仅包括将渲染数据送到 GPU 中，同时还并有状态改变。叶子节点很可能包含使用几种不同的纹理、顶点或者像素着色器的几何体。所有这些状态的改变都非常耗时，因此应该保持完全最小化。Lombard 给出了一个关于如何提高 ABT 渲染效率的很好的例子[Lombard02]。现在假设我们有如表 5.2.1 中所示的在树创建之后有序的面的集合，这里的几个状态改变通过着色器状态 ID 来引用。

表 5.2.1 与叶子节点和着色器关联的面列表

面片	叶子 ID	着色器状态 ID
1	123	0x12345678
2	123	0xABCDEF00
3	123	0xABCDEF00
4	123	0xABCDEF00
5	200	0x12345678
6	200	0x12345678
7	200	0xABCDEF00

Download at www.Pin5i.Com

从这个列表中我们能够建立如表 5.2.2 中的面片列表，然后附加到一个特定的节点。

表 5.2.2　　有附加面片的节点列表

叶子 ID	开始面片	面片数	着色器状态 ID
123	1	1	0x12345678
	2	3	0xABCDEF00
200	5	2	0x12345678
	7	1	0xABCDEF00

如果我们正常遍历树并且渲染每个节点，在上面表格的例子中我们必须进行 3 次状态改变，尽管相同的工作可以通过一次改变完成。因此，我们并不遍历树并立即渲染每个节点，我们创建一个按照着色器状态 ID 排序的渲染状态列表（参见表 5.2.3）。

表 5.2.3　　最后按照着色器 ID 排序的渲染状态列表

开始面片	面片数	着色器状态 ID
1	1	0x12345678
5	2	0x12345678
2	3	0xABCDEF00
7	1	0xABCDEF00

另外，保存列表中顶点数据引用的方法也是可取的，同时也可以进一步提高效率。下面的伪码片断说明如何使用最后的渲染状态列表来渲染 ABT 数据[Lombard02]。

```
for(each entry in render state list)
{
    if(Entry->ShaderStateID != LastEntry->ShaderStateID)
    {
        SetNewState(Entry->ShaderStateID);
    }
    RenderVertexArray(Entry->VAPointer);
    LastEntry = Entry;
    Entry++;
}
```

尤其是在有很多材质的场景中，从减少 API 调用上获得的性能提升将会超过由对列表排序带来的开销。当然，这种方法需要一种允许很简单地对比表面着色参数的材质管理。

5.2.6　总结

由于在创建过程中的自适应性，ABT 是一种非常有效的空间剖分技术。每个节点被分割成两个子节点，直到一个节点上面片的数量达到预定的阈值。将当前节点体积切割成两部分的平面可以通过不同的方法确定。包括简单的线性采样法和连续逼近的方法，或者是一些更复杂的方法。ABT 也能处理动态对象，不过在这种情况下还有一些事情需要考虑。虽然起初这些实现看上去很复杂，但是实际上相当容易，同时每一种图形引擎都能从这种类型的空间数据结构中获益。

Download at www.Pin5i.Com

5.2.7 致谢

感谢 Anthony Whitaker 和 Yann Lombard 在本文写作过程中的帮助。

5.2.8 参考文献

[Campbell03] Campbell, Andy, "ABT questions: construction and use." June 16, 2003. Available online at *http://www.gamedev.net/community/forums/topic.asp? topic_id=163240.*

[Lombard02] Lombard, Yann, "ABT—What it is, and how to build them." November 14, 2002. Available online at *http://www.gamedev.net/community/forums/viewreply.asp?ID=675554.*

[Lombard03] Lombard, Yann, "Answers to ABT questions." June 21, 2003. Available online at *http://www.gamedev.net/community/forums/viewreply.asp?ID=961492.*

[Teran-Ravniker04] Teran-Ravniker, Mirko, "Tranquility Demo" February 23, 2004. Available online at *http://fly.to/teran.*

Download at www.Pin5i.Com

5.3 用有向包围盒增强对象裁减

Ben St. John，西门子
ben.stjohn@siemens.com

对象级的裁减是在不检查每个对象的三角形或者点的情况下识别一个对象是否在照相机中不可见。这种能力是任何图形系统中重要的一部分，它们希望能够有效地显示复杂场景。裁减系统的价值在于对象可见性计算与不断写入渲染管线的开销之间的联系；一般这意味着在可见性计算的精确性和复杂性之间找到正确的折衷。根据以往的经验，这只需非常简单（如果容许较小的不精确性）的测试。

本节说明了一种提高对象包围盒精确性的技术，在所有形式的操作中都非常有用，包括碰撞检测、视线、拾取、闭合以及视锥体裁减。我们将主要集中在视锥体裁减以及一个层次型且顶端相当紧凑的有向包围盒的方法，并实现快速精确的可见性计算。我们构造一个半有向的包围盒，它在 x 和 y 轴上是有向的，但是仅仅与 z 轴对齐。这就（有很多面的情况下）保证裁减质量没有显著下降的同时极大地简化了包围盒的计算。

5.3.1 方法概要

本节核心算法是计算“严格”与 x 轴和 y 轴对齐，但是只是简单地与 z 轴对齐的包围盒。当包围盒实际上用于确定与视锥体交互时，将其视为一个普通的有向包围盒。由于对象一般都能够自由旋转，这样可以避免引入额外的能引起坐标轴对齐测试的体积。基本上我们假设包围盒能够通过仅绕 z 轴旋转找到。

这种假设有多糟糕呢？一般来说一点也不糟糕。正如我们这个行星上大多数对象受重力支配一样，它们趋向于一个准确稳定的垂直部分，这里是指能够很好地适合它们的包围盒。只有与图 5.3.1 中类似的有明显垂直倾斜的对象会从自由地选择包围盒的轴中受益。即使你的模型来自另外的太空，但你的美工（可能）不是，因此你应该没关系。我们的方法与包围球相结合一样，在最坏的情况下会产生稍微不太理想的匹配，但

图 5.3.1 投影到 x-y 平面后产生的较差包围盒的例子

Download at www.Pin5i.Com

是也仅仅是 z 方向。

我们的第二个假设是对象或多或少地以原点为中心，通常情况下也是如此。这样简化了方程，在本节最后我们会看到如果需要的话如何修正，因此这样的假设也是没有坏处的。

5.3.2 传统技术

一般有两种方法可用于生成对象的包围盒。第一种方法由统计学得到[VanVerth04]，称之为“协方差矩阵”。其基本思想是确定数据中具有最大和最小偏差的坐标轴或者方向。适用于 2D 点集的协方差矩阵是：

$$\text{cov} = \begin{bmatrix} S_{xx} & S_{xy} \\ S_{xy} & S_{yy} \end{bmatrix}, \text{以及} \tag{5.3.1}$$

$$S_{yy} = \sum_{pts} \left(p_y^2 - \overline{p}_y \right) \tag{5.3.2}$$

这里 $\overline{P}_y$ 是点 y 值的平均值，既然我们假设对象以原点为中心，则该值近似为 0。因此，协方差矩阵中的条件简化为：

$$S_{xx} = \sum_{pts} p_x^2, S_{yy} = \sum_{pts} p_y^2, S_{xy} = \sum_{pts} p_x * p_y \tag{5.3.3}$$

另一种方法来自物理学，其主轴从对象计算得出。主轴可以被想象成对象首选的旋转轴——或者更精确地说，是对象转动惯量最小化的方向。转动惯量张量在等式 5.3.4 中定义，是质量在绕坐标轴旋转的距离平方上的积分。它有点像对象的质量，但是适用于旋转。

$$\text{I} = \begin{bmatrix} S_{yy} & -S_{xy} \\ -S_{xy} & S_{xx} \end{bmatrix} \tag{5.3.4}$$

除了它们需要在每一点乘以质量之外，这里的子项和协方差矩阵中的一致。在简单例子中，每个点的质量是相同的（也是任意的），因此我们将其设为 1 并且忽略。后面我们将回到这一简化再看看如何改进我们的模型。

两种方法的目的都是矩阵对角化——也就是找到点的方向从而使对角以外的区域（S_{xy}）变成 0。这种定向能够通过变换世界坐标系的 x 和 y 轴的旋转矩阵来表示，使得它们可以和对象主轴联系起来。物理学和统计学两种技术都利用对角化来最大化或者最小化与其中一个轴的距离。

5.3.3 针对二维的有效解决方案

事实上，对于二维的情况，物理学和统计学两种途径实际上都归结到计算同样的事情上。一个矩阵只是经过缩放的另一个矩阵的逆矩阵，解决一个矩阵的方法同样也可用于另一个。这里主要涉及一些线性代数问题，如果信任我们的结论你可以略过这些讨论。最后，我们计算能使点的包围盒紧密围绕的旋转角度。但是，对于想知道这些是从何而来的勇敢者，接着往下读……

Download at www.Pin5i.Com

两种方法都采用对称矩阵而且尽量对角化它。线性代数中有一个很方便的理论，如果矩阵是对称的而且对角项为正值时，矩阵总是可以对角化。在我们的例子中，等式 5.3.1 和 5.3.4 中的矩阵都是对称定义的，而且对角项均为平方和，也总是正值——这样看来我们运气不错。

对角化一个矩阵也就是解方程 5.3.5：

$$\mathrm{I}=\mathrm{RDR}^{-1}=\mathrm{RDR}^{\mathrm{T}} \tag{5.3.5}$$

这里 **I** 是我们的惯性张量，**R** 是旋转矩阵，**D** 是对角矩阵。

由于 **R** 表示二维旋转，因此可以写成：

$$\mathrm{R}=\begin{bmatrix} c & -s \\ s & c \end{bmatrix} \text{ where } c=\cos\theta, s=\sin\theta \tag{5.3.6}$$

同时由于旋转一般都是正交的，其逆矩阵也就是其转置矩阵。扩展所有矩阵我们可以得到：

$$\begin{bmatrix} c & -s \\ s & c \end{bmatrix}\begin{bmatrix} D_x & 0 \\ 0 & D_y \end{bmatrix}\begin{bmatrix} c & s \\ -s & c \end{bmatrix}=\begin{bmatrix} S_{yy} & -S_{xy} \\ -S_{xy} & S_{xx} \end{bmatrix}, \tag{5.3.7}$$

通过一些代数运算，我们可以解出：

$$\theta=\frac{1}{2}\tan^{-1}\left(\frac{2S_{xy}}{S_{xx}-S_{yy}}\right) \tag{5.3.8}$$

在三维的情况下，我们需要解出一个 3×3 矩阵的特征向量，这需要首先解出三次方程，然后解出特征向量，这两种方法都相当复杂而且浪费大量算术运算[Eberly02]。更进一步来看，协方差矩阵和惯性张量之间的简单联系会变得更复杂。

我们实际上需要的并不是角度本身，而是最后的旋转矩阵，因此我们可以避免代价较高的三角函数及其反函数，直接解出 θ 的正弦和余弦值。首先我们通过方程 5.3.7 解出 $\cos 2\theta$，然后使用半角恒等式：

$$\cos 2\theta=\frac{S_{xx}-S_{yy}}{D_y-D_x}=\frac{S_{xx}-S_{yy}}{\sqrt{(S_{xx}-S_{yy})^2+4S_{xy}^2}}\text{，和} \tag{5.3.9}$$

$$\cos\theta=\sqrt{\frac{1+\cos 2\theta}{2}},\ \sin\theta=\sqrt{\frac{1-\cos 2\theta}{2}} \tag{5.3.10}$$

因此，整个系统可以通过三次平方根求解——即使是在没有浮点能力的系统上也是相对进行较低开销的操作。

最后还有一个重要的注意事项：我们现在解决了将世界坐标系的 x 和 y 轴变换到对象主轴的旋转问题。要正确地对点进行旋转，我们需要通过该矩阵的逆矩阵（或转置矩阵）进行变换，将点映射到 x 轴和 y 轴上。下面的具体实例应该可以帮助弄清这一点。

实例

我们将该技术应用到下面的简单问题上。以矩形盒子为例进行旋转，然后裁去一角，令其稍微复杂一点（参见表 5.3.1）。这里我们已经知道了正确的解。

包围球相当大，比真正的包围盒大 3 倍，因此毫无价值。即使轴对齐的包围盒改进这种情形，也没有太大帮助。计算的包围盒的体积相当好，虽然它仍然比实际最小值大 11%。最重要的因素是我们得到的解是 31.5°——已经接近了，但是可能比我们期望的仍有一些差距。

Download at www.Pin5i.Com

那么误差是怎么来的呢？

表 5.3.1 旋转 30° 的缺角盒

原 始 点	经过旋转的点	原 始 点	经过旋转的点
(4.0, 0.7)	(3.11, 2.61)	包围半径	4.12
(3.0, 1.0)	(2.10, 2.37)	包围球面积	53.41
(–4.0, 1.0)	(–3.96, –1.13)	AABB 面积	43.40
(–4.0, –1.0)	(–2.96, –2.87)	θ	31.5°
(4.9, –1.0)	(3.96, 1.13)	计算的包围盒面积	17.75
包围盒面积	16.00		

5.3.4 传统技术上的改进

有两种误差来源，我们可以对其中一种采取一些措施，但是另外一种误差相对不是很重要。在某种意义上存在我们没法处理的误差，这是因为我们正在解决的问题其实并不是我们实际上希望解决的问题。我们正在解决协方差或者转动惯量的问题而不是在计算点集的最小包围盒。两种技术都使用对象中的所有点，找到可以平衡所有点的旋转矩阵。例如，这就是说如果旋转的一个解导致大量点轻微地靠近轴，同时有一个单独的点严重远离，由于总和最小，因此该解仍然是首选的。对于最小化包围盒，我们实际上关心的仅仅是最后的值。虽然可能听起来这是技术上无法避免的缺点，但在我们最后的版本中将仅仅存在最小限度的影响。

第二种缺点更为重要；我们在修正过程中开发了具有更好更快结果的技术，也从本质上排除了由第一种缺点引入的误差。目前我们在两个模型中都考虑了所有点，而且都平等地对待这些点。在转动惯量模型中，每个点都有相同的权重。但是，因为我们需要的只是包围盒，所有内部的点都是无用的、错误包含的信息。因此，我们不需要内部点。

其次，在同一个 *x-y* 平面上位置接近的点会产生比其本身更大的影响，因此都会歪曲结果。实际上一个点足够定义最大的 *x* 或者 *y* 值；额外的临近点只是可能在已将其他单独点移动到更远处为代价的情况下，得到能够使它们更靠近某根轴的旋转角度。

现在我们能做什么呢？对于三维的情况，Haines 和 Akenine-Möller[Moeller02]建议使用 Gottschalk 的方法[Gottschalk99]，他采用对象凸包，然后根据表面积对凸包中每个面的中点赋予一定权重。这样，我们就可以得到满意的映射，解决寻找薄壁中空形状的转动惯量的问题。它同时消除了由多个临近点引入的误差，因为靠得越近，它们所属的三角形面片越小。但是这样显然是一个复杂的过程。单单决定凸包问题的时间复杂度就是 $O(n \log n)$，这还没有处理更复杂的决定如何从点构造面以及实际上需要做的其他计算。

这就是我们在仅考虑 *x-y* 平面投影的情况下独有的关键简化。因为我们仅仅需要考虑连接包围点的直线长度，无须考虑面的问题，而且包围点具有清晰的顺序。例如，我们可以只对边界进行逆时针处理即可。现在我们仍然存在首先计算凸包的问题，可以通过简单近似来回避该问题。

我们可以认为，点集的凸包由相对所有坐标轴最小的和最大的点组成。这意味着凸包上的每个点都是某一方向上的最大值。通过限制方向的数量，我们可以减少大部分的工作，同时在有限的范围内降低了精度。由于我们能够约束该误差，因此我们可以通过相应增加包围盒的大小使所有的点都包含在里面。

Download at www.Pin5i.Com

1. 简化的凸包

我们要做什么呢？我们来考虑 4 个轴，8 个点的情况。设 x 轴和 y 轴由直线 $x+y=0$ 和 $x-y=0$ 形成。在实平面上这就对应于找到 x 轴和 y 轴上沿对角线最远的点（如图 5.3.2 所示）。

该计算会非常快：

```
For each point:
    x = p.x
    y = p.y
    z = p.z
    r2 = x*x + y*y + z*z
    if (r2 > rMax)
        rMax = r2
    For x, y, z, x+y, and x-y
        if ([x] > [xMax])
            记录点的索引并更新 [xMax]
```

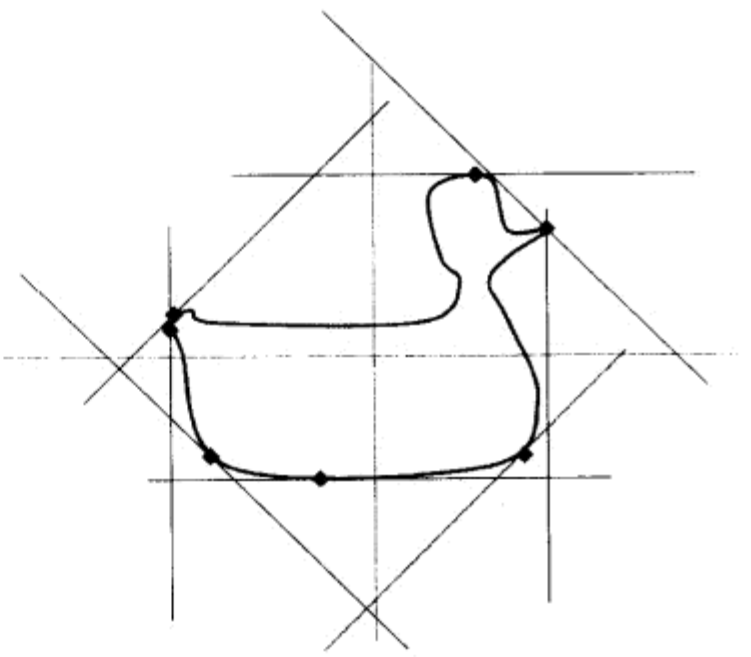


图 5.3.2　鸭子的简化凸包

这种情况下的最大误差是 (1–cos 22.5) / cos 22.5，或者大致上是 8%。为了确保我们的包围体包含了所有点，我们需要在此数值上缩放 x 和 y 的范围，最后包围体增加了约 16%。如果有必要，我们可以将所有点变换到包围盒的方向上，并找到精确的最大和最小值，但是看起来这样并不值得。

同样的，如果考虑由 $x+/-2y$ 和 $2x+/-y$ 形成的轴，可以显著地减小误差，但随之而来的额外的复杂计算使得该方法的价值需要重新考虑。它确实能够将最大误差减小到(1–cos 11.25)/cos 11.25，即 2%，这也同样是相当显著的提高。

这样一个包围体称为 *k-polytope* 或者 *k-DOP*（DOP，Discrete Oriented Polytope），这里 k 是极大极小值的个数——在我们的例子中是 8。一般来说使用的包围面是精确的，而包围点则不是。不过，使用点更简单而且也需要更少的存储空间。该技术带来了许多优点。第一，我们有复杂度为 O(n)的技术来得到（近似的）凸包。第二，操作非常简单（大 O 标注后面的常量表示）——仅有比较和添加操作，即使是在没有浮点支持的硬件上开销也很小。第三，对于后面一个更复杂的计算，我们仅需考虑 8 个（或者更少的）点。第四，我们需要自由排列的点，因此对于分配权重的直线段的计算变得并不重要。

2. 小结

现在我们来考虑整个过程。对于所有的计算，我们假设点的原点或多或少与中心重合。

首先，我们检查完整的点列表。对于每一个点我们计算其与原点的距离（平方值）以及其沿每个轴 ($x, y, z, x+y, x-y$) 的投影，记录所有这些数据的最大和最小值。

使用 8 个（或者更少的）x-y 凸点构造惯性张量矩阵，并用直线段的长度对每个点赋予权重。点的权重由其到上一个点和下一个点的距离和给出，这里不可避免地需要平方根操作。

然后我们解旋转矩阵并将其应用到 8 个包围点上。在这里我们决定了每个坐标轴方向上的最大长度，通常称之为“半长”，并表示为 h_a、h_b 和 h_z。我们将 x-y 向量的长度扩展 8%，

Download at www.Pin5i.Com

这是前面计算的误差限。通常情况下在轴和长度的形式上实际保持半长是有用的，因为我们会使用这两个值但是并不想重新计算独立的值。

当我们将改进的方法应用到我们原先的问题上时，我们得到的解是 29.7°，比较满意地接近了准确值，结果体是 16.33。现有误差实际上来自我们的假设：点都以原点为中心。由于我们切掉了盒子的一个角，因此该假设并不成立。

在这一点上，我们应该对结果相当满意。我们有一个紧密的包围盒，计算开销为 O(n)，非常低的常数。我们避免了所有的三角函数，只需要处理一些平方根。现在的问题是，我们如何利用这些信息，如何针对该盒子实际筛选呢？

5.3.5 对包围盒进行筛选

首先我们想确定使用有向包围盒作为包围体是否有价值。例如，如果对象是球体，其包围盒实际上比其包围球大（惊奇还是意外）。这一点可以很快确定。由于包围球检查非常快，因此我们可以在任何情况下进行，同时仅在需要的情况下执行开销更大的包围盒测试。

为了确定包围盒是否实际上紧密符合，我们简单地比较两个包围形状的体积。如果等式 5.3.11 成立，则球更适合。

$$\frac{4}{3}\pi r^3 \leqslant 8h_a h_b h_z \tag{5.3.11}$$

这一点可以通过各种方法记录下来，取决于代码的整洁和内存开销之间的平衡。我们同时注意到最短边，其长度对应于最小球体。如果对象中心比该点到视锥体的距离近，我们便知道该对象与视锥体相交；因此执行包围盒交叉测试则是浪费的。

要检查包围盒自己是否与视锥体相交，我们需要进行适当的定向。对每个对象自身（例如，旋转、平移和缩放）其坐标轴和长度经过严格变换。如果有必要，由于我们知道向量有特殊形式（一个 z 向量和两个 x-y 向量），可以对这种乘积进行优化，不过可能并不需要。然后我们有了照相机空间中的 3 个向量（h'_a、h'_b、h'_z）。然后我们将每个半长投影到视锥平面的法线 n_{fi} 上，给出到该平面的距离。对于离平面最近的夹角，

$$d_{\text{nearest}} = d_{\text{center}} - \left(\left|h'_a \cdot \mathrm{n}_f\right| + \left|h'_b \cdot \mathrm{n}_f\right| + \left|h'_z \cdot \mathrm{n}_f\right| \right)。 \tag{5.3.12}$$

如果小于 0，则包围盒与视锥部分相交（实际在角上稍微有点不准确）。在绝大多数情况下，当我们计算一个对象是否在视锥体之外时，我们仅需考虑视锥体上距离对象中心最近的 3 个平面（参见[Moeller02]），其中一个与 z 平面是近还是远，因此总体上的计算开销相当小。

综合起来考虑，我们用下面的伪码程序段来检查是否与包围体相交。通常情况下，一旦我们知道对象位于视体之外，我们就可以返回；但是如果有可能在里面，我们需要应用更精确的测试。

```
// 可以首先进行球体测试，但这里暂时忽略这一过程
// 在照相机空间中的对象
for zNear, zFar, and nearest of left/right and top/bottom plane pairs
    d = 与平面的距离（带符号）
    if (d > radius) // 球在外面
```

Download at www.Pin5i.Com

```
            return EOutside;
        if (d > innerRadius) // 需要和包围盒对比检查
            记录面的索引和距离
                \
if (BoxSmallerThanSphere() && (记录的平面索引 != 0))
    将半长变换到相机空间
    for all planes
        if (plane index)
        dA = dot(n,hA) * lenA
        dB = dot(n,hB) * lenB
        dZ = dot(n,hZ) * lenZ
        dCorner = fabs(dA) + fabs(dB) + fabs(dZ)
        if (distanceToPlane > dCorner)
            return EOutside

return EIntersecting
```

要注意，由于近裁剪面和远裁剪面非常简单，其他平面是对称的，对于所有对象视锥体都是一致的，这样该例程依然有大量的优化机会。

5.3.6 进一步改进

虽然我们在建立包围体及其测试上已经有了相当高的精度和速度很快的技术，这里仍然有许多可能改进的地方。

1. 中心偏移向量

对象的原点是否可以明显远离其中心，中心偏移向量（center offset vector）可能很有用。这仅仅是简单的从原点到物质中心的位移（在对象局部坐标系中）。中心偏移必须作为裁减数据结构的一部分保存，但其变换和使用则相当直接。这必然将额外的工作和复杂性引入裁减计算中，因此需要仔细考虑是否需要实现这一优化。

2. 检测完全在视锥体内的对象

上文忽略了一个重要的附加的优化：识别对象是否完全位于视锥体内部。在这种情况下我们可以忽略开销常常很大的对单独三角形的裁剪测试。执行这样一个测试稍微增加了视锥算法的复杂性，它并不和判断一个对象是否位于视锥体之外那样简单，因为你需要确保对象完全位于所有平面内。但是单独测试相当类似，上面描述的改进（例如，首先进行球测试，然后进行内球测试，接下来进行有向包围盒测试）仍然有效。

3. 投影到不同平面上

x-y 平面的选择相当随意。如果模型一般用与其垂直方向不同的坐标轴，则可利用这一点。另外，要自动化该过程，可以计算投影到 *x-z* 或者 *y-z* 平面的值，从而得到最好的结果。作为准则，截项（我们这里是 S_{xy}）表明旋转是否可以给出更好的包围盒。利用这一点，不再需要完全解出每个可能的投影。

Download at www.Pin5i.Com

4. 其他裁减增强技术

当一个对象被确定完全在视体之外时，[Assarsson99]通常可以通过记录是由哪一个平面导致的来加速，下一次检查对象可见性的时候首先测试该平面。这称之为平面一致性优化。该方法是因为从一帧到另一帧，切割平面总是保持不变。这一额外的复杂性是否有必要应该通过分析才能得知。

5.3.7 总结

我们已经描述了既可以有效计算对象包围体又可以在当前照相机视锥体中检测该包围体的技术。包围体是一个在 *x-y* 平面内的旋转盒，通过确定对象凸包的主惯性轴来计算。该方法具有小常量 $O(n)$时间复杂度；避免了使用任何超越方程；由于仅使用对象凸包（近似凸包），因此在处理不规则形状对象时具有相当的健壮性。

然后，我们描述了一个层次型的测试对象可见性的方法。首先尝试排除基于其包围球的对象。然后，如果需要且可能成功，执行包围盒测试。按照这种方法，可以在不需要手工创建包围体的情况下迅速确定可见对象的保守集合。

这些增强的包围体可以被用于其他用途，比如碰撞检测、光线相交、封闭裁减，希望可以提高所有这些任务的效率。

5.3.8 参考文献

[Assarsson99] Assarsson, Ulf and Tomas Möller, "Optimized View Frustum Culling Algorithms." Technical Report 99-3, Department of Computer Engineering, Chalmers University of Technology. Available online at *http://www.ce.chalmers.se/~uffe/vfc.pdf*.

[Eberly02] Eberly, David, "Eigensystems for 3 × 3 Symmetric Matrices." Available online at *http://www.geometrictools.com/Documentation.html*.

[Gottschalk99] Gottschalk, Stefan, "Collision Queries using Oriented Bounding Boxes." Ph.D. Thesis, Department of Computer Science, University of North Carolina at Chapel Hill, 1999.

[Moeller02] Akenine-Möller, Tomas and Eric Haines, *Real-Time Rendering*, 2nd Ed. A. K. Peters, 2002.

[VanVerth04] Van Verth, Jim, "Using the Covariance Matrix for Better-Fitting Bounding Objects." *Game Programming Gems 4*, Charles River Media, 2004.

Download at www.Pin5i.Com

5.4　皮肤分离的优化渲染

Dominic Filion
dfilion@hotmail.com

5.4.1　简介

渲染蒙皮网格形成了当今三维游戏引擎中的一个中心部分。对于某些即时战略游戏（RTS，Real-Time Strategy），蒙皮网格实际上形成了屏幕上渲染的所有数据中一个相当大的子集，对蒙皮几何体渲染的优化变得极为重要。目前通常将所有顶点处理放入 GPU 子系统中进行，蒙皮网格也不例外。不过，蒙皮网格在一定程度上造成了皮肤分离过程的复杂性。

下面简要回顾一下皮肤分离过程，蒙皮网格上的顶点有一些附加在每个顶点上额外的信息，以骨骼索引和骨骼权重来表示。骨骼矩阵对网格的影响被插入到有限大小的骨骼调色板中，每个顶点赋予影响它的骨骼索引。骨骼矩阵则通过各自的权重加权、连接和应用到每个顶点上。

GPU 去皮算法的一个限制是其必须处理有限大小的骨骼调色板。顶点着色器 2.0 现在已将可用的顶点寄存器的数量从 96 个增加到 256 个。不过，这些骨骼矩阵必须与其他信息共存，比如光照参数、材质信息、雾的约束，等等。每个骨骼矩阵由一个 4×4 的矩阵组成，由于最后一列总是$[0\ 0\ 0\ 1]^T$，因此可以将其忽略。将得到的 4×3 矩阵进行转置得到一个每个骨骼有 3 个向量寄存器的 3×4 矩阵。另外，至少需要保留 9 个寄存器给投影矩阵、材质颜色，还有比如两个漫反射光（例如，每个光源有颜色和位置信息）等。因此，实际上允许的骨骼调色板的最大骨骼数量在顶点着色器 1.1 上是 29 个，在顶点着色器 2.0 上是 82 个。如果由于非皮肤的原因需要使用其他参数，例如光照，则最大骨骼调色板的大小则会进一步减小。

但是，通常我们并不希望将一个单独角色骨骼的最大数量限制到如此低的数值上，因此有必要进行某种形式的网格分离。有大量骨骼的网格需要被分割成几个区域，每个区域使用一个有最大大小的唯一的骨骼调色板。虽然该过程可能会转移到美工身上，但是我们更期望这一过程得以自动完成。

即使整个骨骼集适合于一个单独的骨骼调色板，网格分离可能仍是值得做的。这源于在大多数作品流水线上，影响一个单独顶点的骨骼数量可能从 0～4 不同。这可以利用一个四权重的顶点着色器，给当前没有使用的额外权重位置赋 0 来处理。不过这并不是最佳方案，这样浪费了处理权重的 GPU 时钟，而且没有任何作用。更优化的方法是进一步将网格分割成几节，用相同数量的权重，在那些专门为计算一个、两个、三个或者四个权重的顶点着色器之间切换。

Download at www.Pin5i.Com

注意，这样是否能够加速取决于图形流水线的特性。对于 DirectX，在顶点着色器之间切换的开销与着色器本身相关（虽然随着新版本 API 的发布开销在逐步下降）。在其他平台，例如 PlayStation®2（PS2），当着色器类型改变时，新 VU（向量单元，Vector Unit）微代码必须上传到向量单元中并激活。因此，分割低细节的网格并不总是能够加速。对骨骼调色板分割和骨骼权重分割都理想的情况是高细节、更复杂的网格，每个骨骼都影响大量顶点。

因此，我们这里讲的是网格的分割，首先根据影响每个顶点的骨骼的数量分割，然后根据适合骨骼调色板的骨骼数量。本节的中心是展示在这方面的一些启发式算法，同时说明如何实现网格分割。

5.4.2 分割的概念

实际上在绝大多数图形管线中，依赖于各种参数的网格分割是普遍现象，所以这里有必要首先简要介绍一下分割的概念。本小节中心是基于有限蒙皮的网格分割，其他网格分割的情况可能是：

- 分割具有超过一定数量的优化顶点或者三角形的网格。
- 分割在美工编辑工具中给不同网格子区域赋予不同材质或者纹理的网格。
- 分割受不同用于静态光照预计算的光源影响的静态网格。
- 分割需要实时沿着预定义的物理破裂路径将对象分割成残骸的网格。

本小节所述的网格分割总是沿着三角形边界发生。我们只是将网格三角形分割成不同的部分，在分割过程中并不增加新的三角形、删除或者改变三角形。

分割过程考虑网格中的每个三角形，并将其赋予到某一个“桶”中。每个桶就是网格的一个子集，同时拥有其自身的索引和顶点缓冲。图 5.4.1 是一个简单的网格分割，及其相应的顶点和索引缓冲结构的例子。

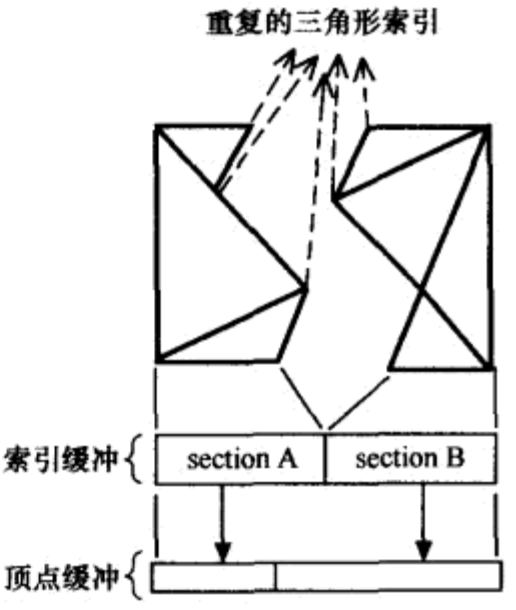


图 5.4.1 简单网格分割及相关结构

由于某些三角形是重复的，分割网格一般需要消耗更多内存。因为两个骨骼调色板之间的顶点着色器需要使用不同的本地骨骼调色板索引来访问不同调色板中相同的骨骼，所以一些顶点同样也会重复。

在分割过程中，我们必须给每个三角形指定需要放入的桶。因此我们定义下面的 CTriBucketInfo 结构体：

程序清单 5.4.1 CTriBucketInfo 结构

```
struct CTriBucketInfo
{
    CBonePalette*     pBonePalette;
    int               WeightCount;
};
```

Download at www.Pin5i.com

这一结构给出了一个三角形属于哪个桶所需的所有信息。当且仅当两个三角形有同样的 CTriBucketInfo 结构时它们属于同一个桶。

在网格分割算法的计划阶段，我们分配一个 CTriBucketInfo 数组，为网格中的每个三角形创建一个 CTriBucketInfo。分割的主要启发式算法是填充每个 CTriBucketInfo 结构。

5.4.3　权重分割的启发式算法

根据影响每个三角形的权重数量进行分割相当简单。

算法 5.4.1　权重分割

```
For each triangle
    For each of the 3 vertices on the triangle
        将三角形权重数量设为 0
        For each of the 4 weights on each vertex
            将顶点权重数量设为 0
            If vertex weight is nonzero
                Then 递增顶点权重数量
    If 顶点权重数量 > 三角形权重数量
        Then 将顶点权重数量赋予三角形权重数量
将三角形权重数量存储于 CTriBucketInfo 数组中，供后面的网格分割使用
```

算法 5.4.1 相当直接。对每个三角形上每个顶点的两重循环迭代查找每个顶点，找到具有最大皮肤权重数量的顶点。接下来将该数值存储于该三角形的 CTriBucketInfo 结构中。

5.4.4　骨骼调色板的启发式算法

骨骼调色板分割的启发式算法有一些棘手，同时也需要非常仔细的分析。复杂网格可以分割成两个、三个、四个或者更多的骨骼调色板；目前也有多种调整网格片断的方法。我们可能无法很直接明显地得知为什么以及在何种情形下某种网格分割方案会比另一种更优，因此本小节中我们探讨更多关于为什么启发式算法重要的细节问题。

如果对蒙皮网格进行了不正确的划分，得到的子区域会比应有的更多。如果没有正确分析，一个应该适应于两个骨骼调色板的网格将会分散到三或四个骨骼调色板中。更多的网格区域意味着更多的渲染处理、索引和顶点缓冲中更多的重复以及更低的效率。

一旦我们为一块骨骼赋予了一个骨骼调色板，我们就期望不将其赋到其他任何骨骼调色板上，但是事实总不是这样。如图 5.4.2 所示，以一个典型的游戏角色手臂骨架为例。

图 5.4.2　简单手臂骨架

如果我们将手臂骨骼赋予某个骨骼调色板，如图所示，这将允许我们利用该骨架调色板为任何仅被该手臂骨骼影响的顶点蒙皮。然而，要对肘部顶点蒙皮，我们既需要手臂骨骼也需要前臂骨骼，它们需要出现在同一个骨骼调色板中。如果我们沿手臂移动的更远一点，我们可以看到对手腕蒙皮既需要前臂也需要手部骨骼出现在同一调色板中，依此类推。

因此，将一块骨骼赋予一个调色板并非意味着我们已经完成了对这块骨骼的操作，它可

Download at www.Pin5i.com

能与其他骨骼重叠，由于骨骼调色板可能已经填满，我们还可能不能将重叠的骨骼放入同一个骨骼调色板中。

对于分割算法的每次迭代，我们必须选择想要将哪块骨骼添加到当前的骨骼调色板中。一旦我们将一块骨骼加入骨骼调色板，我们要尽可能地避免将同一块骨骼插入不同的骨骼调色板。将骨骼包含到多个调色板中会产生低效的划分。一些重叠骨骼出现在多个调色板中是不可避免的，这正如所有影响邻接重叠区域的骨骼必须包含到相同的调色板中。我们不能将整个骨架放入一个单独的调色板，因此必须在某点进行中断，同时一块骨骼必须被再次包含到不同的调色板中。

骨架中任何骨骼都会对邻接重叠区域产生影响。对于每个受到骨骼影响的顶点，我们注意到其他骨骼也会影响该顶点。我们将影响一个特定顶点的骨骼集合称为“影响集”。在大多数系统中，影响一个特定顶点的骨骼的最大数量限制为 4，因此影响集将包含不超过 4 块骨骼。

我们能推断和注意到特定骨骼出现处的所有影响集。例如，手臂骨骼可能伴随以下形式出现：仅有手臂骨骼、受前臂影响的手臂骨骼或者受手部骨骼影响的手臂骨骼。每块骨骼的影响区域都与相邻对象重叠，我们得到新的组合影响集。现在我们将参考作为能影响网格的骨骼组合的可能的集合。因此，我们的手臂骨骼有 3 种组合方式。

在创建骨骼调色板时，我们必须确保网格所有可能的骨骼组合出现在至少一个骨骼调色板中。对于手臂骨骼，至少有一个调色板必须包含手臂骨骼和前臂骨骼，同时至少一个调色板（可能是同一个）必须包含手臂骨骼和手部骨骼。否则无法渲染这些骨骼重叠的区域，其原因是没有调色板包含我们需要的骨骼集合（同时我们会被迫创建另一个包含这些骨骼的调色板）。

因此，要将骨骼从进一步处理中排除，我们必须将骨骼连同所有其他出现在同一个骨骼组合中的骨骼作为正在处理的骨骼加入现有的骨骼调色板。换句话说，所有骨骼组合包含的骨骼必须放入同一个骨骼调色板。这样看起来有理由寻找允许我们尽早避免骨骼被进一步处理的启发式算法。

最后我们有了寻找合适的启发式算法的线索：我们需要首先将具有最小骨骼组合的骨骼加入骨骼调色板。这样我们只用将最小数量的其他骨骼加入骨骼调色板从而将这些骨骼从进一步处理中排除。

这样通常会得到一种内向骨骼划分的策略。像手指这样在外肢上的骨骼需要首先加入，像躯干（一般与很多其他骨骼重叠，比如腿、头和手臂）这样的中间部分要最后加入。不过我们可能已经通过简单沿着骨架加入了几块骨骼首先处理了一些骨骼，或者是通过将骨骼组合的数量作为启发式算法的基础，我们有一个更通用的不依赖于骨骼特定拓扑结构的算法。

这样考虑的话，这里是基于骨骼组合的启发式算法概要：

- 为网格计算所有骨骼组合；
- 找到最小组合的骨骼；
- 将该骨骼加入调色板；
- 尽可能多地将该组合中的骨骼加入调色板；
- 删除已经完全包含在该骨骼调色板中的组合；
- 重复操作直到该网格所有骨骼组合完全包含到了某个调色板中。

Download at www.Pin5i.Com

5.4.5　启发式算法的细节

本小节将描述启发式算法的简单实现。我们已经回顾了一些启发式算法背后的理论，下面的部分将会有更多的代码示例。

要在调色板中找到最好的候选骨骼，第一步是记录骨架中每个可能的骨骼组合。然后我们才能找到哪块骨骼使用了最少数量的骨骼组合。因此，正如前面给出的例子，一块手臂骨骼很可能有 3 个组合：仅被手臂骨骼影响的顶点、既被手臂又被前臂影响的顶点以及既被手臂又被肩部影响的顶点。如图 5.4.3 所示，如果一块骨骼被一个三角形的顶点 1 和 2 使用，另一块骨骼被该三角形的顶点 3 使用，这两块骨骼都必须出现在同一个骨骼调色板中才能渲染这一个单独的三角形。

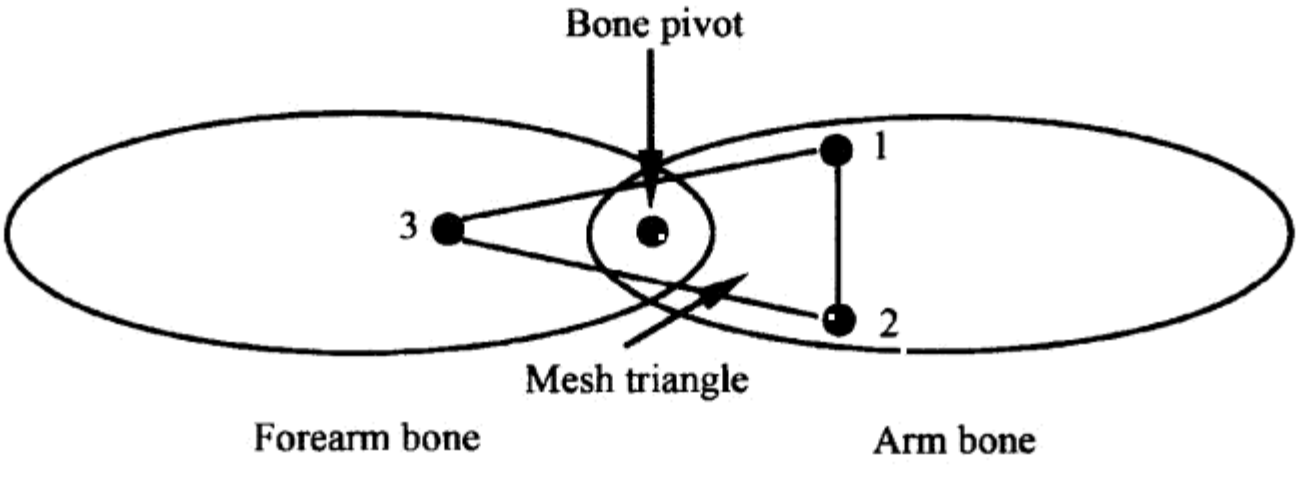


图 5.4.3　一个三角形共享的骨骼

因此，要计算出所有使用的骨骼组合，我们首先需要记录每个三角形使用了哪些骨骼。由于每个三角形有 3 个顶点，因此每个三角形使用的骨骼可以上升到 12 块。这一骨骼索引集合（上至 12 块）便能存储到唯一的组合中。算法 5.4.2 说明了如何实现这一点。

算法 5.4.2　初始的启发式算法设置

```
Const DWORD MAX_BONES = 256; // 允许的最大总骨骼数
struct BoneCombination
{
    DWORD BoneIndices[12];
};
BoneCombination AllBoneCombinations[TriangleCount];
set<BoneCombination> BoneCombinationSet;
bool BonesUsed[MAX_BONES];

For each triangle
将 BonesUsed 数组初始化为 0
    用特殊值 FREE_SLOT 填充 BoneCombination 入口

    For each of the 3 vertices on the triangle
        获取顶点骨骼索引以及权重
        For each of the 4 vertex weights
            如果权重非零，则将 BonesUsed 数组中该索引标记为已使用
```

Download at www.Pin5i.com

```
For each entry in the BonesUsed array
    If 骨骼索引已使用（BonesUsed 入口为 true）
      根据在 AllBoneCombinations 数组中的三角形将索引存储到 BoneCombination 结构第一个可
      用位置上
      在 BoneCombinationSet 中插入包含当前三角形信息的 BoneCombination 结构
```

需要提及的是该算法后半部分的一些注意点。我们首先将正在处理的一个特定的三角形使用的所有骨骼打上标签，然后进行第二轮遍历，迭代添加所有骨骼，这样该三角形使用的骨骼会被记录到一个 BoneCombination 结构体中。骨骼首先被标记，然后添加的目的是便于对骨骼索引进行快速排序，这样以相同顺序出现的具有相同索引的两个完全同样的骨骼组合将会使用一样次序的骨骼索引，和它们在 BoneCombination 结构中排序一样。在 C++中，可以用 STL 的 set 来记录所有骨骼组合；使用 set 后，当我们在不同的三角形中碰到骨骼组合时，多次添加同样的骨骼组合 set 将仅记录该组合一次。

如算法 5.4.3 所示，在记录了所有骨骼组合之后，我们便能计算每块骨骼出现在多少个骨骼组合中。

算法 5.4.3 计算骨骼组合

```
// 本算法中使用的一个存储骨骼组合数量的数组
int BoneCombinationCount[MAX_BONES];

将 BoneCombinationCount 初始化为 0
For each BoneCombination in BoneCombinationSet
    For each of the 12 bone indices within the BoneCombination
        Increment the BoneCombinationCount entry corresponding to the
        current bone index
    Ignore the entry if it is marked as FREE_SLOT
```

这里我们检查了所有组合，并递增了出现在一个组合中每个骨骼的组合数量。掌握这些信息后，我们最后便能实现我们的启发式算法，找到有最小组合的骨骼，并作为添加到我们当前骨骼调色板的候选对象来使用，如算法 5.4.4 所示。

算法 5.4.4 找到最好的候选骨骼

```
int BonesInPalette = 0;
bool AddedBones[MAX_BONES];

For each bone palette
    将 AddedBones 数组初始化为 0
    即然 BonesInPalette 小于调色板允许的最大骨骼数
        遍历 BoneCombinationCount 中的所有值并找到大于 0 的最小值，保存该最小值以及数组索引

        如果 BoneCombinationCount 中所有入口都是 0
            完成；退出

        添加骨骼

    完成骨骼调色板
```

Download at www.Pin5i.Com

该算法中绝大部分步骤都非常容易理解。

在找到候选骨骼后，我们便能将共享该组合的所有骨骼加入到骨骼调色板中。通过迭代所有骨骼组合，我们可以识别出包含当前正在处理的骨骼的组合，然后将该组合中的其他骨骼添加到骨骼调色板中。

算法 5.4.5　添加骨骼

```
// 用于骨骼调色板的方便的容器
list<DWORD>    BonePalette;

如果在 AddedBones 数组中骨骼目前还没有标记为已添加
    将骨骼加入 BonePalette
    在 AddedBones 数组中将骨骼标记为已添加
    递增 BonesInPalette

For each BoneCombination within BoneCombinationSet
    If current bone index is assigned to any of the 12 slots within
    BoneCombination
        For each of the 12 slots within BoneCombination
            If bone index in that slot is not marked as already added
            in the AddedBones array
                将骨骼添加到 BonePalette 中
                在 AddedBones 数组中将骨骼标记为已添加
                    递增 BonesInPalette
                    如果 BonesInPalette 等于调色板中允许的最大骨骼数
            终止当前循环
```

一旦完成，我们则再次进行所有骨骼组合的循环，从骨骼组合集中删除完全在骨骼调色板中的骨骼组合。这样确保启发式算法的正确性，并且最终确保每个骨骼组合出现在一个调色板中。

算法 5.4.6　删除经过处理的组合

```
// 该 map 将为每个骨骼组合保存其所在的骨骼调色板
map< BoneCombination, BonePalette*> CombinationsMap;

For each BoneCombination in BoneCombinationSet
    如果 BoneCombination 中所有骨骼索引都在 AddedBones 数组中已经标记为已添加（不要忘记忽略
FREE_SLOT! ）
        For each slot within BoneCombination
            递减 slot 中该骨骼索引的 BoneCombinationCount
    将当前 BoneCombination 与当前调色板索引加入 CombinationsMap 中

    将当前 BoneCombination 从 BoneCombinationSet 中删除
```

一旦骨骼调色板完成，由于其已经被填满或者没有更多的骨骼需要处理，我们必须将一些特定的与调色板相关的信息存储起来重新调整骨骼索引。由于我们将大的骨骼集合缩小到小调色板上，需要将网格顶点反映的全局骨骼索引重新映射到顶点使用的局部骨骼调色板的有效索引上。算法 5.4.7 说明了存储需要进行这些步骤的信息的算法。

Download at www.Pin5i.Com

算法 5.4.7 完成骨骼调色板

```
// 用于结束所有操作的有用的结构体
struct BonePalette
{
    int* BoneIndices;
    int* GlobalBoneIndexToPaletteBoneIndex;
}
```

```
根据所需索引数量在骨骼调色板中申请空间
保存骨骼索引
重新映射骨架中未归调色板前的原始骨骼索引并存储到顶点缓冲
将 BonePalette 添加到全局调色板列表中
```

当前调色板的索引同样存储于 STL 的 map 中，其关键字是当前骨骼组合。后面我们将用该 map 找到每个骨骼组合用到了哪些骨骼调色板。

数组 pGlobalBonesToLocal 包含了在调色板中找到的每个全局骨骼索引的局部骨骼索引。这样为新的骨骼调色板重建顶点缓冲时可以利用这些信息重新映射骨骼索引。在将每个骨骼组合指定到骨骼调色板后，我们便能使用这些信息根据其指定的骨骼调色板将每个三角形规类到不同的三角形桶中。对于每个三角形，我们将 CTriBucketInfo 结构中的 pBonePalette 入口设置到该三角形对应的调色板上。

所有 CTriBucketInfo 结构都填充后还有一些琐碎的事情，重新组织网格的索引缓冲并根据指定的 CTriBucketInfo 调整三角形索引，为每个三角形桶存储不同的三角形批处理信息。顶点缓冲同样需要重建，这是因为由两个或者更多骨骼调色板共享的顶点必然使用不同的局部骨骼索引来访问不同调色板中的同一块骨骼；在这些情况下，顶点一定会重复。索引和顶点缓冲结构根据应用程序的不同而不同，且依赖于特别的需求。算法 5.4.8 说明了重建索引和顶点缓冲的过程。

算法 5.4.8 重建网格——更好、更强、更快

```
为每个三解形
    将三角形添加到索引缓冲中——利用 map 将独立的索引缓冲和对应的 CTriBucketInfo 结构联系起来
    将顶点添加到顶点缓冲中——再次利用该 map
```

5.4.6 总结

这里介绍的启发式算法通常用于逻辑组织的蒙皮划分，而且能得到最少的骨骼调色板总数。它可用于任意复杂的骨架，不受任何骨架布局例外的限制。其实现非常快，添加到产品线后在导出时间上没有任何明显的延迟。

Download at www.Pin5i.Com

5.5　GPU 地形渲染

Harald Vistnes
harald@vistnes.org

大多数地形渲染算法都是由高度场（heightfield）生成高度逼真的三角网格。为了避免实现过程中生成太多三角形，需要采取基于视点的细节层次(LOD)控制。在相机移动的同时对三角网格进行更新，每次更新（一般每帧一次）都从高度场中查找高程数据并重新计算顶点位置。这种逐帧更新顶点的方法必然消耗相当可观的 CPU 时间。如果将高度场查找从 CPU 转移到 GPU 中，顶点位置就能通过顶点着色器（vertex shader）进行计算，这样就不再需要逐帧更新顶点缓冲。

本节将介绍一种易于实现的 GPU 地形渲染算法。GPU 地形渲染算法的基础是 Shader Model 3.0 中顶点纹理（vertex texture）读取的特性。这里介绍的算法通过重复使用一小块顶点缓冲来减少对内存的需求，其中每个顶点仅由 3 个浮点数组成；通过将高度场存储到顶点纹理中并在顶点着色器中查找高程数据的方法，将 CPU 从逐帧的顶点缓冲更新中释放出来。

5.5.1　算法

一般算法的主要思想是将地形分割成一组单元格，每个单元格包含固定数量的三角形。这些单元格中的顶点按照 $(2^k+1) \times (2^k+1)$ 的规则网格进行组织，这样每个顶点都与高度场 $(2^n+1) \times (2^n+1)$ 个元素其中之一相对应。所有送入 3D API 处理的单元格都是正方形平面，然后通过在顶点着色器中读取存有高度场数据的顶点纹理上相应位置的高程对顶点进行移动。由于顶点的位置是在顶点着色器而不是在 CPU 中计算，而且每个单元格由相同数量的顶点组成，因此所有单元格可以重复使用同一个顶点缓冲。除了顶点缓冲之外，还需要一个单独的索引缓冲，并可以通过有效组织索引值使整个单元格作为一个单一的长三角形带进行渲染。

当用同样的顶点缓冲和索引缓冲渲染不同的单元格时，仅需对两个恒定参数进行调整。相对于每个单元格都有一个顶点缓冲的方法而言，这种方法可以显著节约内存，当然这也取决于单元格的大小。

每个单元格都必须对两个恒定参数进行设定，其中包括一组比例和偏移参数，它们用于指定单元格的面积及其在地形上的位置。

图 5.5.1 表示一个 33×33 的高度场和两个 5×5 并且具有不同比例和偏

Download at www.Pin5i.Com

移参数的单元格。高度场数据由两个参数 u 和 v 索引，这里 $0 \leqslant u \leqslant 1$，$0 \leqslant v \leqslant 1$。与之类似，单元格顶点的水平位置也由两个参数 s 和 t 给出，这里 $0 \leqslant s \leqslant 1$，$0 \leqslant t \leqslant 1$。对于一个给定的单元格，比例和偏移参数表示了 $\{s,t\} \Rightarrow \{u,v\}$ 的一个映射。使用新的 $\{u,v\}$ 对高度场纹理进行采样从而得到高程值 h，这里 $0 \leqslant h \leqslant 1$。三元组 $\{u,h,v\}$ 表示顶点在局部空间中的位置[1]。由于坐标 $\{s,t\}$ 同时用于计算顶点的位置和纹理坐标，因此在顶点缓冲中 s 可作为顶点的 x 坐标，t 作为 z 坐标。实际上只需要顶点的位置数据保存到顶点缓冲中即可，因此每个顶点仅需 3 个浮点数。

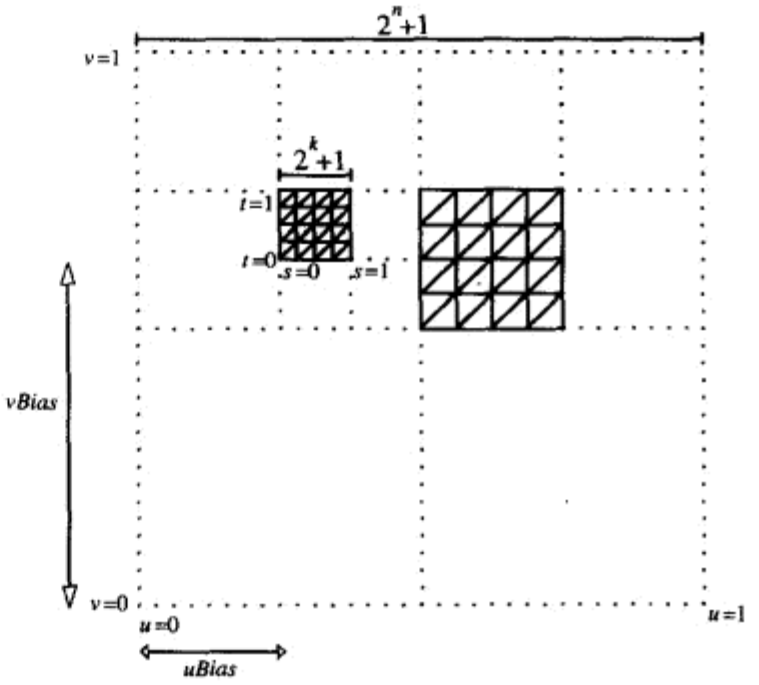


图 5.5.1 两个覆盖地形的单元格。通过比例和偏移两个参数即可将单元格映射到地形上一块指定的区域。注意单元格顶点和高度场元素之间的对应关系

以下代码列举了使用 HLSL 编写的最小的顶点着色器，说明如何实现顶点位置的变换。uScale、vScale、uBias、vBias、heightfield 和 matWorldViewProj 是恒定参数。注意目前没有用到输入位置的 y 坐标，后面会用到。

```
float4 VS(float3 pos : POSITION) : POSITION
{
    float s = pos.x;
    float t = pos.z;
    float u = s * uScale + uBias;
    float v = t * vScale + vBias;
    float4 tex = float4(u, v, 0.0f, 0.0f);
    float h = tex2Dlod(heightfield, tex).x;
    pos = float3(u, h, v);
    float4 pos0 = mul(float4(pos,1.0f), matWorldViewProj);
    return pos0;
}
```

覆盖整个地形的单元格的比例参数是 1.0，覆盖 1/4 地形的单元格的两个比例参数都是 0.5。图 5.5.1 中较大单元格的参数是 0.25，较小的是 0.125。较大单元格的两个方向上的偏移参数都是 0.5；较小单元格 uBias 是 0.25，vBias 是 0.625。

5.5.2 细节层次

如上所述，通过等分比例参数可将单元格覆盖的面积减小到 1/4。因此，一个单元格可以被分成 4 个比例参数是一半的小单元格。如果从一个覆盖整个地形的单元格开始，递归地将其分割成 4 个子单元格直到单元格与高度场的分辨率一致，我们就可以有效地创建单元格四叉树。

[1] 用非统一缩放的矩阵获取水平和垂直方向范围内的相应关系，从而得到世界空间中的位置。

Download at www.Pin5i.com

你可能已经猜到可以利用这一点来控制地形的细节层次（LOD）。通过递归地对每个单元格进行评估，可以决定是进行渲染还是继续细分成更小的单元格，这样我们就可以在减少渲染的三角形总数的同时得到漂亮的地形，不过这取决于评估标准和单元格的大小。

本小节中我们将使用简单的基于距离的评估标准，距离相机较远的单元格比较近的单元格覆盖更多的面积，因此接近相机的地方会有更多的细节。更好的方法则是对单元格使用屏幕空间顶点误差的最大值作为评估标准，不仅考虑了与相机的距离，还考虑了地形表面的粗糙程度。利用文献[Ulrich02]中使用的方法可以很容易地实现这种评估标准。

基于距离的评估标准可以由下面的简单校验给出：

$$\frac{l}{d} < C$$

如果该校验失败，则渲染单元格；如果校验成功则将其分割成 4 个小单元格并递归地进行校验。如图 5.5.2 所示，参数 l 表示从单元格中心到相机的距离，d 表示一个三角形在世界空间中的大小，该基于距离的标准来自文献[Röttger98]。C 是用于控制地形渲染质量的可调常量，C 越大需要细化的单元格越多，从而需要渲染更多的三角形。在本书附带光盘的演示程序中，可以通过滑动控件方便地调整该质量参数。

程序清单 5.5.1 说明如何在细节层次的控制下渲染地形。世界矩阵用于获取世界空间中的距离。这里设定世界矩阵是一个缩放矩阵，参数 fMinU 和 fMinV 的初始值是 0.0；fMaxU、fMaxV 和 fScale 的初始值是 1.0；iLevel 表示四叉树的深度。

程序清单 5.5.1

```
void Render(float fMinU, float fMinV,
    float fMaxU, float fMaxV,
    int iLevel, float fScale)
{
    float fHalfU = (fMinU + fMaxU) * 0.5f;
    float fHalfV = (fMinV + fMaxV) * 0.5f;
    float d = (fMaxU-fMinU)*m_matWorld._11/(m_iBlockSize-1.0f);

    D3DXVECTOR3 c(fHalfU*m_matWorld._11,0,fHalfV*m_matWorld._33);
    D3DXVECTOR3 v = c - g_Camera.GetPos();
    float l = D3DXVec3Length(&v);

    float f = l / d;

    if (f > m_fLOD || iLevel < 1) {
        Draw(fMinU, fMinV, fMaxU, fMaxV, iLevel);
    } else {
        Render(fMinU, fMinV, fHalfU, fHalfV, iLevel-1, fScale/2);
        Render(fHalfU, fMinV, fMaxU, fHalfV, iLevel-1, fScale/2);
        Render(fMinU, fHalfV, fHalfU, fMaxV, iLevel-1, fScale/2);
        Render(fHalfU, fHalfV, fMaxU, fMaxV, iLevel-1, fScale/2);
    }
}
```

Download at www.Pin5i.Com

由于相机移动的过程中单元格细化参数会突然变化，因此会看到众所周知的视觉跳变问题。可以通过对不同细节层次之间顶点的的高程进行柔变处理来防止跳变。方法是对每个顶点读取两个不同参数顶点纹理中的高程，然后按照柔变参数进行线性插值。

上面讨论了细节层次，最后我们来讨论单元格的大小。一般我们希望通过批处理大量的三角形来减少渲染 API 的调用次数。但是较小的单元格允许在细节层次方面有更多的变化。如果单元格太大，同样大小的三角形就会覆盖太大面积，根据 LOD 常数的不同，结果可能太粗糙或者三角化得过于精细。另外，越小的单元格越容易完全处在视锥体之外，在四叉树遍历的过程中越容易裁减掉更多的单元格。在演示程序中，我们发现单元格的大小为 17×17 和 33×33 时可以在渲染的调用次数和三角形大小的多样化之间得到一个折衷。

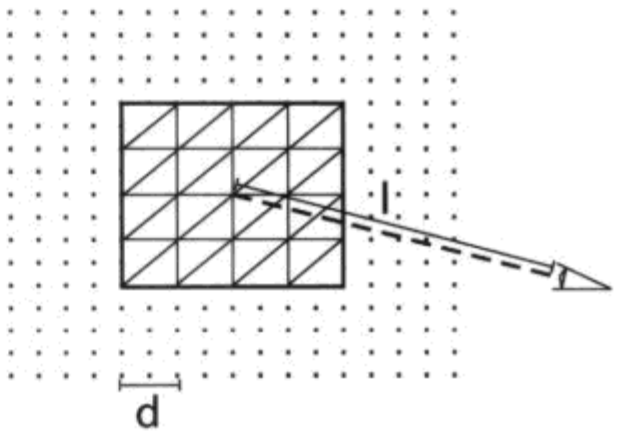


图 5.5.2 基于单元格中心到相机距离和三角形在世界空间中大小的细节层次评估标准

5.5.3 避免裂缝

裂缝一般出现在不同细节层次的单元格之间的边界上，同时是所有地形 LOD 算法中必须处理的问题。有很多不同的方法避免地形上这些令人恐惧的洞。我们选择一种简单高效的方法：沿单元格边界的“垂直裙摆”。文献[Ulrich02]描述并使用了该方法。如图 5.5.3 所示，每个单元格都有沿垂直边界进行扩展的“裙摆”，当两个不同细节层次的单元格相接时，任何可能的裂缝都会被裙摆几何体填充，即便垂直裙摆可能引起一些光照和纹理拉伸的问题，填充的裂缝一般都很小而且不容易被注意到。

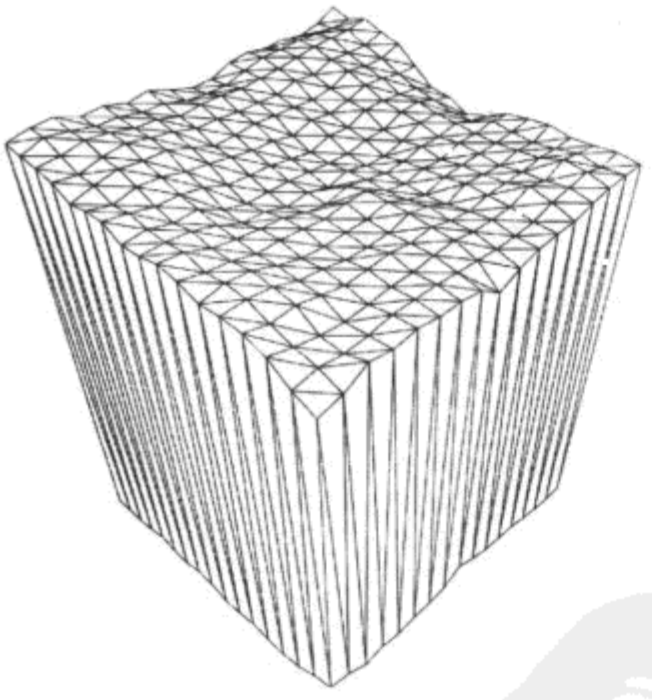


图 5.5.3 每个单元格都有沿边界扩展的垂直裙摆

我们希望在不影响现有算法的基础上添加裙摆。仍然使用简单的顶点缓冲，只是在缓冲最后添加一些额外的顶点，区别于“地形顶点”，我们称之为“裙摆顶点”。索引缓冲同样能够将所有顶点索引表示为一个长三角形带。

但是我们在顶点着色器中如何区别这些地形顶点和裙摆顶点呢？事实上并不需要区分。在顶点着色器的程序清单中可以看出，顶点位置的 y 坐标没有使用过，现在我们要利用它了。初始化顶点缓冲时，将地形顶点的 y 坐标设置为 1.0，裙摆顶点的 y 坐标设置为–1.0，在顶点着色器中将这一行：

```
pos = float3(u, h, v);
```

修改为：

```
pos = float3(u, h*pos.y, v);
```

Download at www.Pin5i.Com

这样，裙摆顶点自动获取负向高程。裙摆顶点的水平位置等于对应的地形边界顶点。不需要对裙摆顶点进行其他的特殊处理；唯一的开销就是沿着边界的额外的三角形。

注意该技巧假设高度场不包含负高程，负高程的典型应用是低于海平面的地形。这个问题可以通过偏移所有高程直到没有负值来解决，有效地伸出海平面。

5.5.4　视锥体裁减

我们可以在遍历四叉树的过程中检测到单元格是否位于视锥体之内。如果一个单元格完全处在视锥体之外，我们可以通过中止递归略过它。该检测可以在程序清单 5.5.1 的函数 Render()开始的时候进行。

这里最方便使用的包围体是与坐标轴对齐的包围盒。单元格最小的和最大的 u 和 v 坐标给出了局部空间中平面的范围，但是最小和最大的高程则不易找到。最精确的方法是在四叉树初始化的过程中计算所有单元格的最小值和最大值。本书附带光盘的演示程序中，我们对所有包围盒使用整个地形的最大和最小高程。这将导致包围盒比实际的高很多，以及无法裁减掉一些不可见的单元格。在实际的实现过程中，应该选择更精确的方案。

5.5.5　法线计算

如果需要动态光照或者环境贴图的效果，我们还需计算地形的法线。如文献[Shankel02]所述，可以对高度场法线的计算进行明显的优化，只需 4 次高度场查找和两次减法。利用这一点，在顶点着色器中，可以通过除了在顶点纹理上采样顶点本身高程之外，还对四邻域高程进行采样并计算顶点法线。目前的硬件，顶点纹理读取开销较大，这种方法可能过于耗时。但未来的硬件可能会优化顶点纹理的读取，因此这可能只是一个短期的问题。

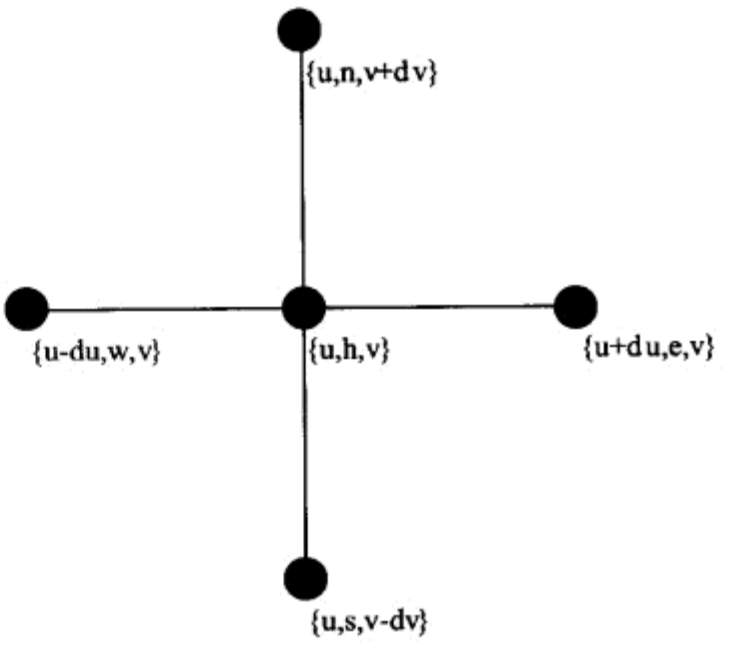


图 5.5.4　计算顶点法线所需的四邻域顶点高程

如图 5.5.4 所示，给出顶点及最近的邻接点，顶点法线可以表示为[Shankel02]：

$$N=\{(w-e),2d,(s-n)\},$$

这里 d 和高程具有相同的单位，表示顶点之间的距离。

然而这种方法的缺点就是法线依赖于细节层次，导致在相机移动引发细节层次变化时光照效果发生变化。

为了避免该问题，可以预先计算所有顶点的法线并存储到一张法线贴图中。法线可以在像素着色器中进行查找，使地形的光照独立于潜在的三角化。由于顶点纹理采样存在较大开

Download at www.Pin5i.com

销，该方法也比为每个顶点计算法线的方法更快，地形光照的逼真度也越高。其弊端是，需要显著增加一些用于存储法线贴图的纹理内存。

在随书演示程序的代码中同时实现了这两种技术，你可以按 F5 键进行切换。从彩页 8 中可以很容易地看到不同技术的效果。最上方的截图中，法线是在顶点着色器中计算的；中间的截图是通过像素着色器对法线贴图进行采样地结果。你可以很清晰地在第一张截图中看出不同细节层次之间的光照区别。

5.5.6 碰撞检测

要是一个游戏仅有漂亮的地形，一定无法令人动心。我们需要在地形上面放置一些对象，比如树木、建筑物、汽车、敌人，等等。我们也决不希望相机穿过地形而让用户看到三角形的背面。为了保证这些约束，我们还要在指定位置找到地形的高程。

典型的实现方法是在 CPU 中对高度场进行采样，可能还需要对临近的 4 个点进行插值。由于系统内存中没有高度场数据的复本，因此在我们的情形下这种方法并不理想。

有一种完美的方法不需要高度场的第二份拷贝。该技巧是将高度场的查询渲染到一张纹理上。为了防止相机与地形交叉，创建一张 1×1 的纹理作为渲染目标。创建一个动态的仅包含一个有位置数据顶点的顶点缓冲。在需要执行高度场查询时，将预期的$\{u,v\}$坐标设为顶点的 x 和 z 坐标并对顶点缓冲进行更新。设置纹理作为渲染目标。将顶点缓冲作为点列表进行渲染。在顶点着色器中将位置的 x 和 z 坐标作为纹理坐标传入像素着色器。这样在像素着色器中，在期望的坐标中对高度场进行采样并将高度值作为像素的颜色返回。渲染目标纹理中唯一的一个像素将包含地形在期望位置的高程。

随书演示程序采用了该方法来防止相机移动到地形的下方，每帧执行一次高度场查询。

当然，该方法并不仅限于对相机所在位置的高程进行 1×1 的查询，你可以生成一张更大的渲染目标和顶点缓冲，在一次调用中同时对几个位置进行查询，这样该方法就可用于同时查询地形中大量对象的位置。即使是用更传统的地形渲染算法，利用硬件滤波该方法可能会比传统的 CPU 地形采样更快。

5.5.7 实现细节

本小节不包含教你如何使用顶点纹理的内容。文献[Gerasimov05]或者 DirectX 的文档[D3DX05]包含如何设置你的应用程序并使用顶点纹理的信息。随书演示程序使用 DirectX9.0，OpenGL 同样支持顶点纹理，因此，任选一种 3D API 都可以实现本文算法。这里我们仅仅考虑 DirectX9.0。

将高度场存储到纹理中的缺点是，高度场的大小受限于图形硬件支持的最大纹理。当前显卡支持的最大纹理是 4096×4096，因此无法直接支持像 16K×16K 大小的超大地形。可采取的解决方法是将地形分割成几块纹理，在需要的时候加载到内存中。但是对于典型的游戏来说最大纹理的大小已经足够了。

Download at www.Pin5i.Com

目前图形硬件刚开始支持顶点纹理，还没有支持众多的纹理格式。例如，NVIDA 的 GeForce 6800 的顶点纹理仅支持 D3DFMT_R32F 和 D3DFMT_A32B32G32R32F 纹理格式。演示中使用的是 D3DFMT_R32F，即每个像素是一个 32 位的浮点数，但是高度场通常使用 8 位或者 16 位整数作为高程值。这样，顶点纹理对内存的需求比高度场本身精度所需要的更多，未来的硬件可能支持更多的顶点纹理格式。

在 GeForce 6800 上，仅在支持的顶点纹理格式上支持邻近点滤波。如果需要进行线性或者三线性滤波，你必须在顶点着色器中实现。这样就需要读取更多的顶点纹理，并对性能形成负面影响。

由于当前顶点纹理支持的局限性，因此必须对不支持顶点纹理的硬件做向下兼容的实现。当然，在这种情况下可以使用任何传统的地形渲染算法。较为省事的方法是对每个单元格使用一个顶点缓冲，在 CPU 中填充高程数据。除了顶点纹理的读取之外，这里讨论的大多数其他的部分都可以在更老的硬件上实现。

5.5.8　运行结果

随书附带光盘演示程序实现 GPU 地形渲染算法，还可以对一些参数进行配置。高度场是 2049×2049 大小的 D3DFMT_R32F 的 DDS 纹理，由高度场生成的法线贴图存储为 D3DFMT_V8U8 的有符号纹理格式。法线贴图忽略了 y 坐标，取而代之的是在像素着色器中通过 x 和 z 坐标进行计算。

演示程序中的两个可选参数是单元格的大小和法线计算方法。表 5.5.1 和表 5.5.2 列举了规定时间内单元格大小不同时的结果，分别采用法线贴图和顶点着色器法线计算。

表 5.5.1　规定时间内使用法线贴图时不同单元格大小的渲染结果

	9×9	17×17	33×33	65×65
Fps:	43	85	78	89
Tris/sec:	35M	70M	73M	78M

该测试运行的软件环境是窗口模式的应用程序，硬件环境是安装有 NVIDIA GeForce 6800 Go 图形适配器的便携式电脑。LOD 常数设置为缺省值。从结果中我们可以看出，9×9 的单元格大小明显太小，渲染调用的次数太多。虽然 65×65 的单元格大小在本测试中法线贴图情况下有最好结果，但是它产生了面积太大的等大三角形。我们发现 17×17 是在渲染调用次数和单元格大小之间的较好折衷。

表 5.5.2　规定时间内使用顶点着色器计算法线时不同单元格大小的渲染结果

	9×9	17×17	33×33	65×65
Fps:	43	54	37	40
Tris/sec:	35M	45M	34M	34M

除了单元格大小之外，我们还可以看出用法线贴图大致上比用顶点着色器计算法线快两倍，如彩页 8 所示，它同时还具有更出众的视觉质量，孰优孰劣显而易见。

Download at www.Pin5i.com

5.5.9 总结

本节阐述了一种易于实现的基于 GPU 渲染地形的算法。算法的关键点是将高度场保存为顶点纹理，并对整个地形重复使用一块小的顶点缓冲。低内存需求和低 CPU 消耗显示出巨大的优势，将可用资源留给游戏中物理、人工智能等其他更重要的任务。该算法的速度比其他地形算法都要快。当 Shader Model 3.0 成为图形硬件的标准以后，基于 GPU 的地形渲染可能会淘汰如今在游戏中广泛使用的地形渲染算法。

5.5.10 参考文献

[D3DX05] Microsoft DirectX Developer Center, DirectX C++ documentation. Available online at *http://msdn.microsoft.com/directx/.*

[Gerasimov05] Gerasimov, Phillip, et al., "Shader Model 3.0——Using Vertex Textures." NVIDIA whitepaper, 2005. Available online at *http://developer.nvidia. com/object/using_vertex_textures.html.*

[Röttger98] Röttger, S., et. al., "Real-time Generation of Continuous Levels of Detail for Height Fields." *Proceedings of WSCG '98,* 1998: pp. 315-322.

[Shankel02] Shankel, Jason, "Fast Heightfield Normal Calculation." *Game Programming Gems 3,* Charles River Media, 2002.

[Ulrich02] Ulrich, Thatcher, "Rendering Massive Terrains using Chunked Level of Detail Control." April 2002. Available online at *http://cvs.sourceforge.net/viewcvs. py/*checkout*/tu-testbed /tu-testbed/docs/sig-notes.pdf?rev=HEAD.*

Download at www.Pin5i.Com

5.6 基于 GPU 的交互式流体动力学与渲染

Frank Luna
frank@moon-labs.com

大部分技术采用了正弦波叠加的近似方法（参考[Vlachos02]和[Finch04]），来实现较为简单的 3D 流体渲染。由于这些技术采用的是基于顶点着色器的静态几何动画，因此它们是固定在场景中的，用户不能明显地互动和打扰流体。交互式流体效果的通常实现方法如下：先利用 CPU 进行相关的物理计算，然后使用动态顶点缓冲器来更新几何图形。这种方法在[Lengye02]中有专门的讨论。这种方法的缺点就在于每次更新模拟的时候，都需要利用 CPU 进行物理计算，然后把新的几何图形传回显卡。

而如今，伴随着图形硬件对 Vertex Shader 3.0（特别是顶点纹理抓取功能）的支持，流体的物理和渲染都可以完全通过 GPU 的通用计算技术来实现。通过把流体动力学计算转移到 GPU，就避免了动态顶点缓冲器的更新，而充分利用了强大的 GPU 来处理高负载计算；而 CPU 也可以空出来处理其他的任务。在本节将会深入讨论如何使用 DirectX 9.0c 来实现一个动态流体系统，该系统将会利用 GPU 进行物理计算和渲染。图 5.6.1 和彩图 9 是本书光盘中相关演示程序的截屏，演示了波纹是如何在 GPU 上更新和渲染的。

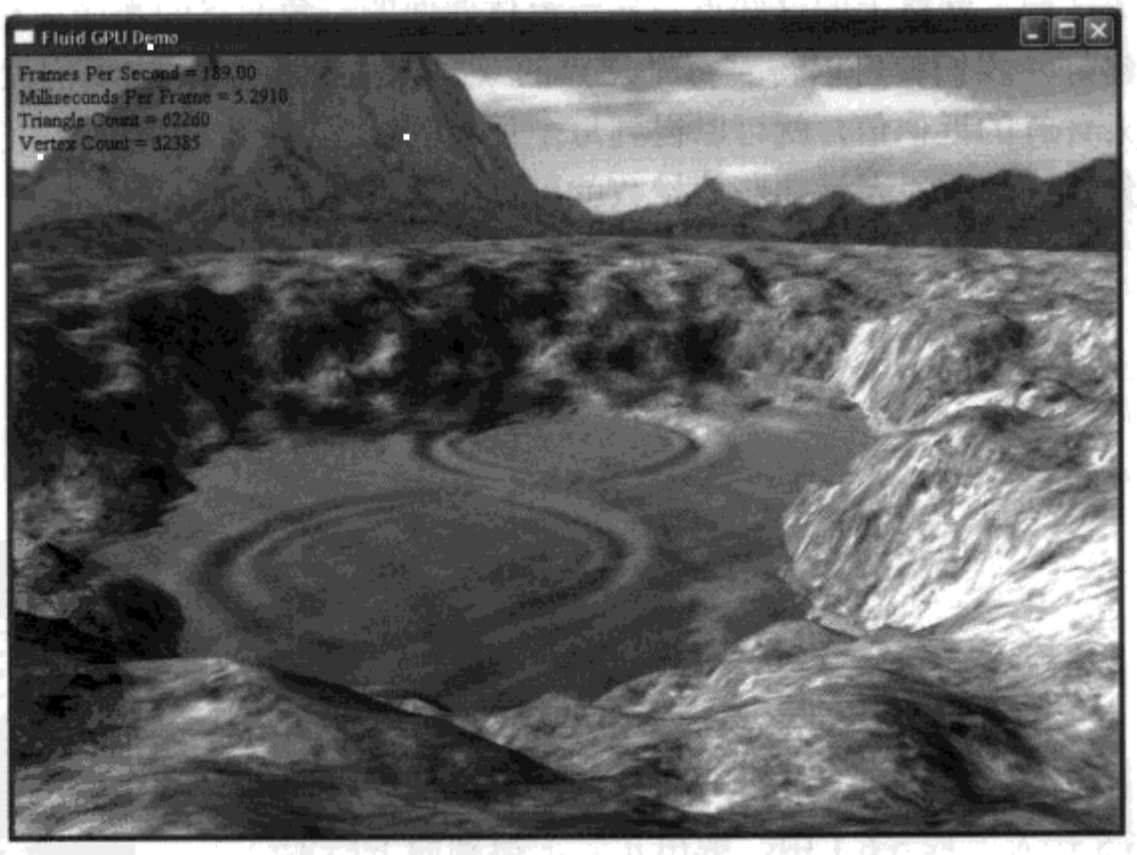


图 5.6.1 用户通过交互式的输入在池塘中产生波纹（该模拟由 GPU 完成）

Download at www.Pin5i.Com

5.6.1 数学背景知识

本小节将会介绍实现流体模拟所需要的基本数学知识。我们的讨论仅是概要性的介绍而不涉及相关的数学细节知识，因为这些不是本文的主题。

1. 2D 波动方程

偏微分方程（Partial Differential Equation）：

$$\frac{\partial^2 h}{\partial t^2}(x,z,t)=c^2\left(\frac{\partial^2 h}{\partial x^2}(x,z,t)+\frac{\partial^2 h}{\partial z^2}\right)-\mu\frac{\partial h}{\partial t}(x,z,t) \qquad (5.6.1)$$

该方程描述了 2D 水波的运动，阻力是$-bv$，基于以下假设：

- 固定均匀的质量密度；
- 固定均匀的表面张力（包括波运动期间）；
- 完全柔性表面（抗弯强度为 0），因此张力总是位于表面的切线上；
- 倾斜角足够小，因此 $\tan\theta\approx\sin\theta$。

因此，本数学模型能够很好地对较小的水波（如水坑、池塘和相对平静的湖面）进行流体模拟。在[Kreyszig62]中可以找到基于该偏微分方程的一个优秀推导公式（无阻尼系数）。

参数 c 和μ分别代表水波速度和阻尼系数，修改它们的值可以实现对水波如同艺术般的控制。不过，必须确保修改值符合稳定性条件（参考下面的稳定性部分的内容）。

2. 显示的有限差分法

方程 5.6.1 的解是 $y=h(x,y,z)$ 某个函数，该函数反映的是在某个特殊时刻（即水波横向运动时），水膜上每个点的高度。为了获得方程 5.6.1 的近似解，我们假设有一块放置了网格的方形区域；网格上的这些点与描绘流体曲面的三角网络的那些顶点是一一对应的。一旦网格构建好，我们就可以得到每个顶点的 x,y 坐标值；接下来我们只需要为每个网格点求出 $y=h(x,y,z)$ 的近似值，就可以得到每个顶点的高度（y 坐标值）。我们只需要用以下的这些有限差分公式来近似表示那些偏导数就可以求得近似值：

$$\frac{\partial^2 h}{\partial t^2}(x_i,z_j,t_k)\approx\frac{h_{i,j}^{k+1}-2h_{i,j}^{k}+h_{i,j}^{k-1}}{(\Delta t)^2} \qquad (5.6.2)$$

$$\frac{\partial^2 h}{\partial x^2}(x_i,z_j,t_k)\approx\frac{h_{i+1,j}^{k}-2h_{i,j}^{k}+h_{i-1,j}^{k}}{(\Delta x)^2} \qquad (5.6.3)$$

$$\frac{\partial^2 h}{\partial z^2}(x_i,z_j,t_k)\approx\frac{h_{i,j+1}^{k}-2h_{i,j}^{k}+h_{i,j-1}^{k}}{(\Delta z)^2} \qquad (5.6.4)$$

$$\frac{\partial h}{\partial t}(x_i,z_j,t_k)\approx\frac{h_{i,j}^{k+1}-h_{i,j}^{k-1}}{2(\Delta t)} \qquad (5.6.5)$$

上述公式中的 $h_{i,j}^{k}=h(x_i,z_j,t_k)$。为了更简单，我们规定$\Delta x=\Delta z$。将上述等式 5.6.2 至等式 5.6.5 代入方程 5.6.1 中，解出 $h_{i,j}^{k+1}$，得到如下显示公式：

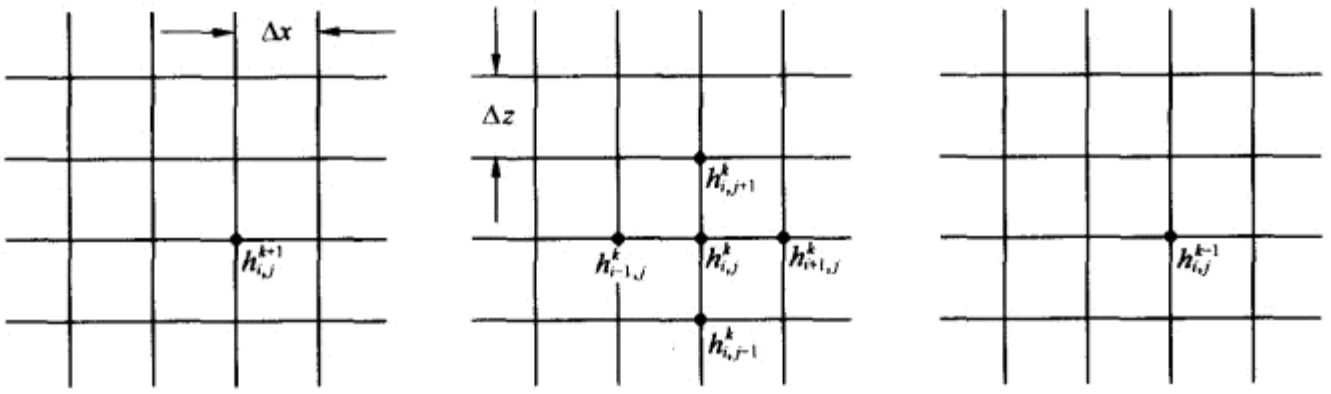
Download at www.Pin5i.com

$$h_{i,j}^{k+1} = k_1 h_{i,j}^{k-1} + k_2 h_{i,j}^{k} + k_3 \left[h_{i+1,j}^{k} + h_{i-1,j}^{k} + h_{i,j+1}^{k} + h_{i,j-1}^{k} \right] \tag{5.6.6}$$

其中

$$k_1 = \frac{\mu(\Delta t) - 2}{\mu(\Delta t) + 2}, k_2 = \frac{4 - 8 \times c^2 \dfrac{(\Delta t)^2}{(\Delta x)^2}}{2 + \mu(\Delta t)} \text{和} k_3 = \frac{2c^2 (\Delta t)^2}{(\Delta x)^2 (\mu(\Delta t) + 2)}$$

换句话说，现在我们根据已知的高度值（也就是在第 $k-1$，k 个时间间隔某些网格点的高度），可以求得在第 $k+1$ 个时间间隔位于 (x_i, y_j) 处的网格点所对应的函数值 $y=h(x, z, t)$。图 5.6.2 展示了求解在第 $k+1$ 个时间间隔位于 (x_i, y_j) 处的网格点的高度所需的先前网格点的高度值。经过Δt 时间（某种时间单位）后，根据等式 5.6.6，我们可以容易地求出第 t_{k+1} 个时间间隔时每个网格点的新高度值。由于等式 5.6.6 依赖于每个网格点的前两个时间间隔的高度值，因此，我们需要维护两个额外的网格缓冲区来存储第 t_k 和 t_{k-1} 个时间间隔的网格点高度。



下一次 $k+1$ 解　　当前 k 解　　前一次 $k-1$ 解

图 5.6.2　t_{k+1} 时刻的高度值依赖于 t_k 和 t_{k-1} 时刻的高度值

为了使得这种方法能够工作，我们必须拥有最早两个时间间隔的所有网格点的初始高度值。我们可以简单地去用各种方法来手动设置这些初始值：对于流体从未分布状态开始运动的情况，可以设置为 0；另外，其他方法产生的初始水波数据也可以用来进行设置，比如输入一张高度图（Heightmap）或者将正弦波叠加。

公式 5.6.2、5.6.3、5.6.4 和 5.6.5 所给出的有限差分法的偏差（带误差信息）可以在[Burden01]中找到。[Lengyel02]同样提供了一个直观的几何偏差，但是缺乏误差信息。

3. 稳定性

由方程 5.6.2 得到的显示公式并不总是稳定的；[Lengyel02]中给出了以下稳定条件：水波的速度 c 和时间间隔Δt 必须同时满足以下不等式：

$$0 < c < \frac{\Delta x}{2(\Delta t)} \sqrt{\mu(\Delta t) + 2} \text{和} 0 < \Delta t < \frac{\mu + \sqrt{\mu^2 + 32 \dfrac{c^2}{(\Delta x)^2}}}{8 \dfrac{c^2}{(\Delta x)^2}}$$

4. 切线与法向量

对于法线贴图，我们需要流体网络中每个顶点的正切基（Tangent Basis）。函数 $h(x, z, t)$

Download at www.Pin5i.Com

描述了位于某点 (x, z, t) 曲面的高度。而偏导数

$$\frac{\partial h}{\partial x}(x,z,t) \text{ and } \frac{\partial h}{\partial z}(x,z,t)$$

分别定义了过 x 与 z 方向上某点的切线斜率。因此，分别得到 x 与 z 方向的切向量：

$$\mathrm{v} = \left(1, \frac{\partial h}{\partial x}(x,z,t), 0\right) \tag{5.6.7}$$

和

$$\mathrm{u} = \left(0, \frac{\partial h}{\partial z}(x,z,t), 1\right) \tag{5.6.8}$$

详情，请参考图 5.6.3。

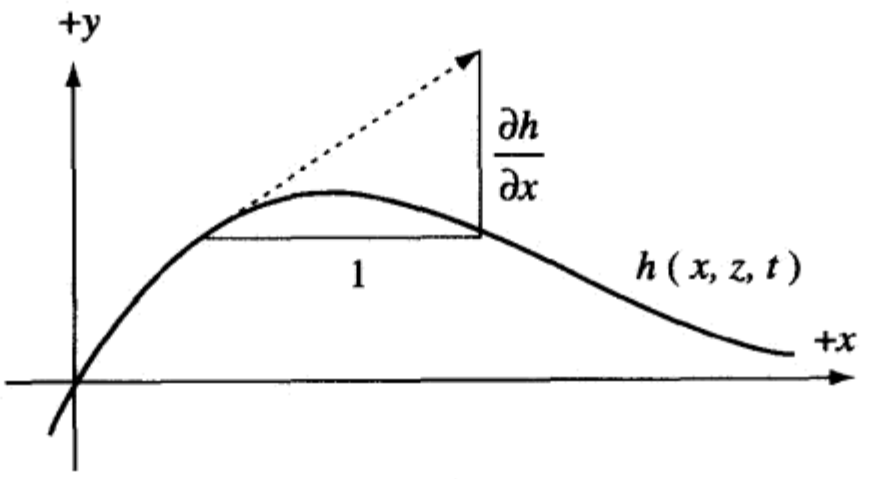


图 5.6.3 偏导函数描述了过曲线上某点的切线斜率。此处，根据斜率的定义（纵距离与横距离之比）可以得知：如果横向移动 1 个单位，纵向就移动 $\partial h/\partial x$ 个单位。该定义同样适用于 z 方向的切向量

注意图 5.6.3 中的 u 和 v 都不是单位向量。给定切向量，通过简单的叉乘，就可以得到法向量：

$$\mathrm{n} = (\mathrm{u} \times \mathrm{v}) / \|\mathrm{u} \times \mathrm{v}\|$$

该等式使用了左手坐标系统。

方程 5.6.7 和 5.6.8 中的偏导函数可以用以下的有限差分公式进行近似：

$$\frac{\partial h}{\partial x}(x_i, z_j, t_k) \approx \frac{h^k_{i+1,j} - h^k_{i-1,j}}{2(\Delta x)} \tag{5.6.9}$$

和

$$\frac{\partial h}{\partial z}(x_i, z_j, t_k) \approx \frac{h^k_{i,j+1} - h^k_{i,j-1}}{2(\Delta z)} \tag{5.6.10}$$

5.6.2 GPU 实现

1. 纹理表示网格

纹理在 GPU 中以数组形式表示。我们通过 render-to-texture（渲染到纹理）功能可以写入纹理（GPU 转换为数组写入），而通过像素着色器的采样可以读入纹理（GPU 转换为数组读入）。需要注意的是，同时对同一个纹理进行渲染和采样的行为是未定义的

Download at www.Pin5i.Com

[Harris04]。

我们需要 3 个网格来实现等式 5.6.6 中的有限差分法：第一个用来存储下一次解，第二个用来存储当前解，第三个用来存储前一次解。因此，我们需要 3 个沸点 render-target（渲染目标）纹理。

```
DrawableTex2D* mPrevStepMap;
DrawableTex2D* mCurrStepMap;
DrawableTex2D* mNextStepMap;
```

网格中的每个元素仅仅存储了高度值，因此没有必要使用 1D 元素格式（1D Element Format，如 D3DFMT_R16F）；不过，目前的硬件（如 Geforce 6800 GT）并不支持对 D3DFMT_R16F render target（渲染目标）的渲染。我们可以使用 D3DFMT_R32F 来代替前面这种格式，但是目前的硬件并不支持用该格式的α混合（alpha blending）来对波进行干扰，也就是 D3DUSAGE_QUERY_POSTPIXELSHADER_BLENDING。因此，我们使用了 D3DFMT_A16B16G16R16F 这种格式；但是只在红色信道（red channel）存储了高度值，而不使用绿色、蓝色和α信道，这的确很浪费空间。

2. 利用 GPU 进行模拟

每经过Δt 时间，需要进行模拟更新以及为网格中的每个点重新计算新的高度值。我们的目的是完全利用 GPU 来完成上述计算工作：GPU 将会用于进行更新网格高度值所需要的通用计算，而不是用来输出色彩信息。

现在，我们给定前两个时刻(t_{k-1}、t_k)的所有网格点的高度值（存于 mPrevStepMap 和 mCurrStepMap），要求 t_{k+1} 时刻的高度值（存于 mNextStepMap）。在 CPU 的实现中，我们需要遍历每个网格点应用等式 5.6.6。那么在 GPU 中，我们怎么做呢？像素着色器（pixel shader）会为每一个绘制到 render target 上的像素执行一次计算；因此，我们可以把一个像素着色器当成一个循环体（loop body）：

For each pixel P_{ij}

　　Execute Pixel Shader on P_{ij}

因此，我们按如下 5 步处理。

- 将 mNextStepMap 设置为 render target。
- 将 mPrevStepMap 和 mCurrStepMap 设置为纹理效果参数，用来采样到像素着色器（也就是，它们构成了计算下组高度值的输入值）。
- 在整个投影窗口上绘制一个单元方块，这可以确保像素着色器为 render target 中的每个像素都执行一次计算。
- 在像素着色器中使用公式 5.6.6。
- 更新完毕；通过循环 render targets 为下次更新做准备：

```
DrawableTex2D* temp = mPrevStepMap;
    mPrevStepMap = mCurrStepMap;
    mCurrStepMap = mNextStepMap;
    mNextStepMap = temp;
```

Download at www.Pin5i.Com

以下是完成上述计算的实际顶点和像素着色器的实现代码：

```
// 位于纹理空间[0, 1]单元的纹理像素大小
uniform extern float gTexelSize;

// 等式5.6.6的预计算常量
uniform extern float gK1;
uniform extern float gK2;
uniform extern float gK3;

// 前一次网格的计算结果
uniform extern texture gPrevStepTex;
uniform extern texture gCurrStepTex;

sampler gPrevStepSmplr = sampler_state
{
    Texture = <gPrevStepTex>;
     [...]
};

sampler gCurrStepSmplr = sampler_state
{
    Texture = <gCurrStepTex>;
     [...]
};
```

```
void PhysicsVS(float3 posH : POSITION0,
    out float4 oPosH : POSITION0,
    out float2 oUV : TEXCOORD0)
{
    // 投影窗口已定义的顶点
    oPosH = float4(posH, 1.0f);

    // 转换投影坐标系([-1,1],y轴向上)到纹理坐标系([0, 1],y轴向下)
    oUV = float2(posH.x, posH.y) * float2(0.5, -0.5) + 0.5;
}

float4 PhysicsPS(float2 uv : TEXCOORD0) : COLOR
{
    // 世界值清零
    if(uv.x < 0.001f || uv.x > 0.985f ||
        uv.y < 0.001f || uv.y > 0.985f)
        return float4(0.0f, 0.0f, 0.0f, 0.0f);
    else
    {
        // 前一次和当前时刻所采样的网格点
        float c0 = tex2D(gPrevStepSmplr, uv).r;
        float c1 = tex2D(gCurrStepSmplr, uv).r;
        float t = tex2D(gCurrStepSmplr,
```

Download at www.Pin5i.Com

```
            float2(uv.x, uv.y - gTexelSize)).r;
        float b = tex2D(gCurrStepSmplr,
            float2(uv.x, uv.y + gTexelSize)).r;
        float l = tex2D(gCurrStepSmplr,
            float2(uv.x - gTexelSize, uv.y)).r;
        float r = tex2D(gCurrStepSmplr,
            float2(uv.x + gTexelSize, uv.y)).r;

        // 返回该网格点的下一次计算结果(等式 5.6.6)
        return float4(gK1*c0 + gK2*c1 + gK3*(t+b+l+r),
            0.0f, 0.0f, 0.0f);
    }
}
```

如我们在像素着色器 PhysicsPS 中所见，需要对边界进行清零（如果你不喜欢“零边界”，你可以在投影窗口的边上绘制线条，该线条实际上就是描述了边界条件）。另外，你可能会关心从纹理贴图中取走的采样点。处理该问题的方法有很多：你可以设置圆柱型的纹理包装（texture wrap），或者把数值钳制到边界值。最后，参数 gTexelSize 表示在纹理贴图上对临近的网格点进行采样取值时，上下左右所移动的距离（参考图 5.6.2）；通常定义为 1/TextureSize。如果你的纹理不是正方形的，那么在水平和垂直两个方向上的偏移值是不一样的：分别是 1/TextrueWidth 和 1/TextureHeight。

3. 利用 GPU 计算切向量和法向量

在模拟更新完毕后，基于新的顶点高度可以计算出新的切向量。为此，我们首先需要增加另一个网格数据结构来存储切向量数据：

```
DrawableTex2D* mTangentMap;
```

对于切向量，我们只需要存储偏导数（也就是等式 5.6.7 和 5.6.8 中的 y 分量部分）；我们可以在任何时候，利用给定的这个数据重新构造出切向量。由于我们没有必要对切向量的纹理进行混合操作（blending operations），因此我们可以用 D3DFMT_G16R16F 格式来解决这个问题：

- 红色部分存储$\partial h/\partial x$。
- 绿色部分存储$\partial h/\partial z$。

接下来，为了计算出切向量数据，我们遍历每个网格元素（更新过），对其应用公式 5.6.9 和 5.6.10，把最终的结果写入 mTangentMap（mTangentMap 是 render target）。从而引出以下计算切向量数据的像素着色器代码（在 PhysicsVS 中，顶点着色器仅仅渲染了一个方块）：

```
// 本地空间的时间段大小
// (用于等式 5.6.9 和等式 5.6.10)
uniform extern float gStepSizeL;

// 我们正在计算正切信息的顶点高度
uniform extern texture gCurrStepTex;

sampler gCurrStepSmplr = sampler_state
```

Download at www.Pin5i.com

```
{
    Texture = <gCurrStepTex>;
    [...]
};

float4 CalcTangentsPS(float2 uv : TEXCOORD0) : COLOR
{
// 世界清零
    if(uv.x < 0.001f || uv.x > 0.985f ||
        uv.y < 0.001f || uv.y > 0.985f)
        return float4(0.0f, 0.0f, 0.0f, 0.0f);
    else
    {
        float t = tex2D(gCurrStepSmplr,
            float2(uv.x, uv.y - gTexelSize)).r;
        float b = tex2D(gCurrStepSmplr,
            float2(uv.x, uv.y + gTexelSize)).r;
        float l = tex2D(gCurrStepSmplr,
            float2(uv.x - gTexelSize, uv.y)).r;
        float r = tex2D(gCurrStepSmplr,
            float2(uv.x + gTexelSize, uv.y)).r;

        // 应用等式 9 和等式 10
        float xtanY = (r - l) / (2*gStepSizeL);
        float ztanY = (t - b) / (2*gStepSizeL);

        return float4(xtanY, ztanY, 0.0f, 0.0f);
    }
}
```

4. 利用 GPU 复制纹理

回想一下我们之前一直使用的 16 位浮点 render-target 格式。不过，目前支持顶点纹理拾取功能的硬件（如 Geforce 6800 GT）仅仅适用于 32 位浮点 render-target 格式（如 D3DFMT_R32F 和 D3DFMT_A32B32G32R32F）。因此，在我们能够做置换贴图（displacement mapping）之前，我们需要将当前的网格数据（也就是高度和切向量偏导数）复制到 D3DFMT_A32B32G32R32-F 格式的纹理中。我们按照以下方式放置数据。

- 红色信道存储顶点高度。
- 绿色信道存储$\partial h/\partial x$。
- 蓝色信道存储$\partial h/\partial z$。
- α信道未使用。

以下是进行复制的像素着色器代码（在 PhysicsVS 中，顶点着色器再次仅仅是渲染一个方块）：

```
// 用于复制的数据源
uniform extern texture gHeightMap;
uniform extern texture gTangentMap;
```

Download at www.Pin5i.Com

```
sampler gHeightSmplr = sampler_state
{
    Texture = <gHeightMap>;
    [...]
};

sampler gTangentSmplr = sampler_state
{
    Texture = <gTangentMap>;
    [...]
};

float4 CopyTexPS(float2 uv : TEXCOORD0) : COLOR
{
    // (r1, 0, 0, 0) + (r2, g, 0, 0) = (r1, r2, g, 0).
    float height = tex2D(gHeightSmplr, uv).r;
    float2 tangents = tex2D(gTangentSmplr, uv).rg;
    return float4(height, tangents, 1.0f);
}
```

上述代码将数据复制到 D3DFMT_A32B32G32R32F 格式的 render target 中：

```
DrawableTex2D* mVertexDataMap;
```

5. 位移贴图

让我们回顾以下目前为止我们所完成的工作。首先，在 GPU 实现中，纹理替代了数组；我们通过纹理渲染来实现写入，通过纹理采样来实现读取。其次，每个时刻（即经过Δt 时间间隔后），我们为波动方程计算新时刻的高度值；然后我们循环使用这些计算出来的高度值，为下次更新作准备：当前高度值变成前一次的，下次高度值变成当前的，而下次高度值之后所存放的旧有的前一次 render target 的无用高度值，可以在下个时刻被覆写。一旦新的顶点高度计算完毕，我们进入切向量基本数据（即 y 分量部分的偏导数）的计算过程。因为我们无法在 16 位的浮点纹理上使用顶点纹理拾取功能，我们接下来便是将存储顶点高度和切向量数据的纹理复制到 32 位的浮点格式的纹理中。

因此，此时我们把当前顶点高度和基本的切向量数据全部存储在单个的纹理中，用 mVertexDataMap 标识。我们现在可以将数据传入顶点着色器（vertex shader）中来实际设置顶点高度及顶点正切基。要这么做的话，我们需要将 mVertexDataMap 设为纹理效果形参，然后在顶点着色器中用其自带的 tex2Dlod 函数对其采样；为了能够访问最顶端 mip 贴图的细节等级（Level Of Details，LOD，实际上只有一个细节等级），我们在函数 tex2Dlod 中将第二个形参的 w 部分指定为 0。然后，我们可以从 RGB 组件中取出数据：

```
OutVS GPUFluidRenderingVS(float3 posL : POSITION0,
    float2 uv : TEXCOORD0)
{
    OutVS outVS = (OutVS)0;
```

Download at www.Pin5i.Com

```
    // 为位移采样高度图
    float4 data = tex2Dlod(gDispSmplr, float4(uv, 0.0f, 0.0f));

    // r 分量存储高度
    posL.y = data.r;
    outVS.posH = mul(float4(posL, 1.0f), gWVP);

    // 构建切线和法向量
    float3 xtan = normalize(float3(1.0f, data.g, 0.0f));
    float3 ztan = normalize(float3(0.0f, data.b, 1.0f));
    float3 normalL = normalize(cross(ztan, xtan));

    ... Do Other Vertex Shader Work.

}
```

基于输入纹理的顶点位置修改被称为位移贴图（displacement mapping），可以通过顶点纹理拾取功能实现。早期的硬件不支持在顶点着色器中进行纹理采样。

5.6.3 流体互动

1. 利用 GPU 干扰水波

目前为止，通过手动初始化第一和第二个解，然后应用等式 5.6.6 来计算后续解。不过，我们的实际目的是能够在运行时利用 GPU 干扰流体。例如：喷泉的场景中，你希望基于水滴动态地产生水波；湖面的场景中，你希望基于移动的船只或者游戏角色涉水经过，来动态地产生波浪。

我们可以把等式 5.6.6 看做是每个时刻解决一个新问题：它利用前两个等式解，计算出新解，并且它并不在意之前的解是什么。因此，其中的 trick 就在于手动修改前两个解来添加新的 disturbance（即置换受到干扰影响的顶点的网格高度值）；然后，在进行 simulation 更新的时候，新添加的 disturbance 将会被考虑进来进行流体更新，因为等式 5.6.6 是依赖于以前的一组网格解的。注意我们并不想简单地在某个特殊区域覆盖前一次的水波运动；替而代之的是，从上次运动中加上/减去某个位移值（displacement）。另外，当我们干扰某个区域的网格点时，我们应当以一种衰减的方式干扰周围的网格点，以保证没有太多的突变。

起先这看起来应当是很简单的（本应如此），但是我们的网格是存储在显存中的纹理，因此我们不能从 CPU 对它们进行写入。但我们回想起纹理渲染其实就是写数组的 GPU 等价。因此，我们可以将前两次解设置成 render target，然后在投影窗口上进行三角形绘制（圆、方或任何你所期望的 disturbances 的形状），该投影窗口仅仅包含了与我们所期望干扰的网格点所对应的像素点。此外，因为我们是直接在投影窗口上进行三角形渲染的，因此 z 分量是空闲的；我们可以用 z 分量来存储高度位移的大小。请参考以下着色器代码：

```
void DisturbVS(float3 posH : POSITION0,
    out float4 oPosH : POSITION0,
    out float oHeight : TEXCOORD0)
{
```

Download at www.Pin5i.Com

```
    // 投影窗口已定义的顶点
    oPosH = float4(posH.x, posH.y, 0.0f, 1.0f);
    oHeight = posH.z;
}

float4 DisturbPS(float height : TEXCOORD0) : COLOR
{
    return float4(height, 0.0f, 0.0f, 1.0f);
}
```

此外，我们启动了α混合，并将源和目的地的混合标记设为1；这可以确保高度是累加，而不是覆盖的：

```
// f = src * srcBlend + dest + destBlend
// = src * 1 + dest * 1
// = src + dest (i.e., accumulate the heights)
AlphaBlendEnable = true;
SrcBlend = One;
DestBlend = One;
```

好好回忆一下“纹理表示网格（Textures Represent Grids）”，我们在其中使用了D3DFMT_A16B16G16R16FL来替代D3DFMT_R32F来支持混合技术。

2. Barrier Maps（障碍图）

目前，只有清零（zeroed-out）的边界条件表示了障碍物；也就是，当波浪达到边界的时候，它们会简单地进行试探，就像前面有堵墙一样。其他需要障碍物的实例并不难举出。例如，水体本身可能并不是真正的矩形，而是不规则形状的池塘，当水到达边界的时候，应当进行检测；或者水体中央可能有许多小岛，波浪同样应当检测这些岛屿的边界；或者其他诸如船只和桥墩的障碍物对象。

为了处理这个问题，[Tessendorf04]中建议使用barrier maps来标志出出现障碍物的网格点。核心思想就是给每个网格点赋一个[0，1]区间的权值。如果网格点是一个障碍物，它的权值就是0；而它相邻网格点的权值应当是渐变的（除非也是一个障碍物），确保平滑的跌落。然后，当计算顶点高度的时候，我们从barrier map中取出对应的权值作为影响的因素（清零或者减小高度）。如果障碍物的位置发生变化（如船只发生移动），那么你可以动态地通过渲染来创建barrier map。

5.6.4 补充材料

[Tessendorf04]使用了线性伯努利偏微分方程，从而推导出一个显示的公式。作为另一种可选的公式，同样可以使用本文所讨论的相同策略进行GPU编程。换句话说，[Zelsnack04]解决了我们在本文中所讨论的同样的2D波动方程，但是它使用了Verlet Integration数值法，而不是有限差分法。

我们的显示法并不总是稳定的。无限制的稳定方法还是存在的（如Crank-Nicholson法），

Download at www.Pin5i.Com

但是这些方法大都是隐式的，并且需要在每个时刻对方程组进行求解。[Krüger05]开发了一个利用 GPU 求解线性方程组的 framework，该 framework 可用于隐式法的实现。

通过下述方法可以获得漂亮而寂静（good-looking static）的海浪：滚动表示海浪高度的高度图，及通过顶点纹理拾取功能（见[Kryachko05]）进行位移贴图。我们可以设计一种混合的效果：利用本文之前所描述的动态生成的波浪，为周围环境的普通波浪添加上述技术。

经典的水面渲染（water-rendering）技术包括利用菲涅耳技术（fresnel term）来组合反射和折射贴图，该技术中的滚动法线贴图是用来进行纹理查找的（实现细节请见[Vlachos02]、[Pelzer04]、[Sousa05]）。焦散效果（Caustics，即聚焦光射到水下表面）可以通过滚动焦散贴图实现，或者通过一种更精巧的方法实现（见[Guardado04]或[Ernst05]）。

最后，[Tessendorf02]中提供了一份非常棒的关于海水渲染技术的调查报告。

5.6.5 总结

我们在本节中展示了如何在 GPU 上完整地实现交互式流体模拟特效。在实现环节中，我们避免了基于 CPU 实现所带来的一系列典型问题：大量的 CPU 开销（[Blythe05]中提及游戏主要是受到 CPU 执行瓶颈限制（CPU-limited）），和系统内存到显存的传输。对于大网格来说，后一种问题是最主要的问题；即使上传一个中等大小的顶点网格（如 256×256），所传输的数据包括位置数据、纹理坐标、法向量和切向量，这样的网格通常也是一笔不小的开销。更通常一点讲，本节中所介绍的技术为其他 simulation 任务（如粒子系统（particle system））提供了从 CPU 实现转为 GPU 实现的参考。

5.6.6 参考文献

[Blythe05] Blythe, David, "DirectX: The Next Step." Meltdown 2005 Presentation, 2005.

[Burden01] Burden, Richard L. and J. Douglas Faires, *Numerical Analysis*, 7th Ed. Brooks/Cole, 2001.

[Ernst05] Ernst, Manfred, Tomas Akenine-Möller, and Henrik Wann Jensen, "Interactive Rendering of Caustics using Interpolated Warped Volumes." *Graphics Interface,* 2005. Available online at *http://graphics.ucsd.edu/~henrik/papers/ interactive_caustics/.*

[Finch04] Finch, Mark, "Effective Water Simulation from Physical Models." *GPU Gems: Programming Techniques, Tips, and Tricks for Real-Time Graphics*, Addison-Wesley, 2004.

[Guardado04] Guardado, Juan and Daniel Sánchez-Crespo, "Rendering Water Caustics." *GPU Gems: Programming Techniques, Tips, and Tricks for Real-Time Graphics*, Addison-Wesley, 2004.

[Harris04] Harris, Mark J., "Fast Fluid Dynamics Simulation on the GPU." *GPU Gems: Programming Techniques, Tips, and Tricks for Real-Time Graphics*, Addison-Wesley, 2004.

[Kreyszig62] Kreyszig, Erwin, *Advanced Engineering Mathematics,* John Wiley and Sons, 1962.

[Krüger05] Krüger, Jens and Rüdiger Westermann. "A GPU Framework for Solving Systems

Download at www.Pin5i.Com

of Linear Equations." *GPU Gems 2: Programming Techniques for High-Performance Graphics and General Purpose Computation*, Addison-Wesley, 2005.

[Kryachko05] Kryachko, Yuri, "Using Vertex Texture Displacement for Realistic Water Rendering." *GPU Gems 2: Programming Techniques for High-Performance Graphics and General Purpose Computation*, Addison-Wesley, 2005.

[Lengyel02] Lengyel, Eric, *Mathematics for 3D Game Programming and Computer Graphics,* Charles River Media, 2002.

[Pelzer04] Pelzer, Kurt, "Advanced Water Effects." *ShaderX²: Shader Programming Tips & Tricks with DirectX 9*, Wordware Publishing, 2004.

[Sousa05] Sousa, Tiago, "Generic Refraction Simulation." *GPU Gems 2: Programming Techniques for High-Performance Graphics and General Purpose Computation,* Addison-Wesley, 2005.

[Tessendorf02] Tessendorf, Jerry, "Simulating Ocean Water." Simulating Nature, SIGGRAPH Course Notes, 2002. Available online at *http://users.adelphia.net/~tessendorf/docs/coursenotes2002.pdf.*

[Tessendorf04] Tessendorf, Jerry, "Interactive Water Surfaces." *Game Programming Gems 4,* Charles River Media, 2004.

[Vlachos02] Vlachos, Alex, John Isidoro, and Chris Oat, "Rippling Reflective and Refractive Water." *Direct3D ShaderX: Vertex and Pixel Shader Tips and Tricks,* Wordware Publishing, 2002.

[Zelsnack04] Zelsnack, Jeremy, "Vertex Texture Fetch Water." 2004. Available online at *http://download.developer.nvidia.com/developer/SDK/Individual_Samples/DEMOS/Direct3D9/src/VertexTextureFetchWater/docs/VertexTextureFetchWater.pdf.*

Download at www.Pin5i.Com

5.7 基于多光源的快速逐像素光照渲染

Frank Puig Placeres，古巴信息科技大学
fpuig@fpuig.cjb.net

光照是随着图形硬件技术的发展而得到大大增强的技术之一。几年前，典型的游戏仅仅使用静态光照，该技术通过 light maps（光效图）和少量的逐顶点（per-vertex）照亮的物体来实现。不过，游戏图形学已经进入了完全逐像素（per-pixel）光照的世界，其中大部分光源都是动态的。

处理动态光源的标准光照解决方案通常包括光照物体的多遍渲染，每遍渲染所使用的着色器程序会计算出少量光源所产生的影响。除了多遍渲染的负面影响外，这种方法最主要的缺点在于要为所有可能的光源组合编写复杂的着色器，如只适用于泛光源（omni light）的着色器，或者同时适用于泛光源（omni light）和聚光源（spot light）的着色器。

如今的图形材质可以处理复杂的特效，比如 Steep Parallax Mapping 和 Multilayered Texture Coordinate Animation。我们可能会使用若干个上述特效的组合，而这只会导致更多的渲染遍数。用来描述这些材质的着色器指令的数目是很高的，而且随着硬件能力的增长而增长，可以用来支持更为复杂的特效。将不同光照模型的组合的处理代码进行叠加，便清晰地克服了图形硬件所导致的着色器复杂性瓶颈问题。

本节中提出了改善标准延迟光照方法（standard deferred lighting approach）的构想，不仅仅可以降低对硬件体和表示复杂场景所需的着色器数目的要求，而且提高了诸如视图色深（depth of view）、头像朦胧（head haze）和雾化这些后处理特效（post-processing effects）的能力。

5.7.1 延迟解决方案

传统的光照解决方案的做法是提交物体的几何数据后，马上应用光照进行处理。当处理关于某个物体的大量的光照效果时，渲染过程将会分为若干遍进行。每遍渲染中，物体的几何数据都要提交给图形硬件流水线，并且计算光源组所产生的效果。每遍渲染的结果都会合并到 fragment buffer（片断缓冲器）中，因此最终的全部光照效果就产生了。

延迟渲染（deferred rendering）作为一种处理大量光源而无须多次提交物体几何数据的解决方案，是非常有用的。如图 5.7.1 所示，延迟渲染仅需要两遍就可以处理完场景，而与光源的复杂程度和数目无关。第一遍过程

Download at www.Pin5i.Com

中，我们没有存储每个屏幕像素的颜色，替而代之的是将每个像素的法向量（normal）、镜面属性（specularity）、发光因子（glow factor）和遮挡项值（occlusion term）这些表面属性存储在 fragment buffer 中。

第二遍过程中，对于每个光源而言，fragment buffer 就是一个纹理；而方形平面区域将会以纹理像素-屏幕像素的映射方式，绘制到屏幕上。某个程序片断将会从每个屏幕像素的缓冲器中解析出表面属性，然后利用光源的位置、颜色等信息作为参数，求解光照方程（lighting equation）；最终的计算结果会被合并到 fragment buffer 中，从而产生完全照亮的场景。

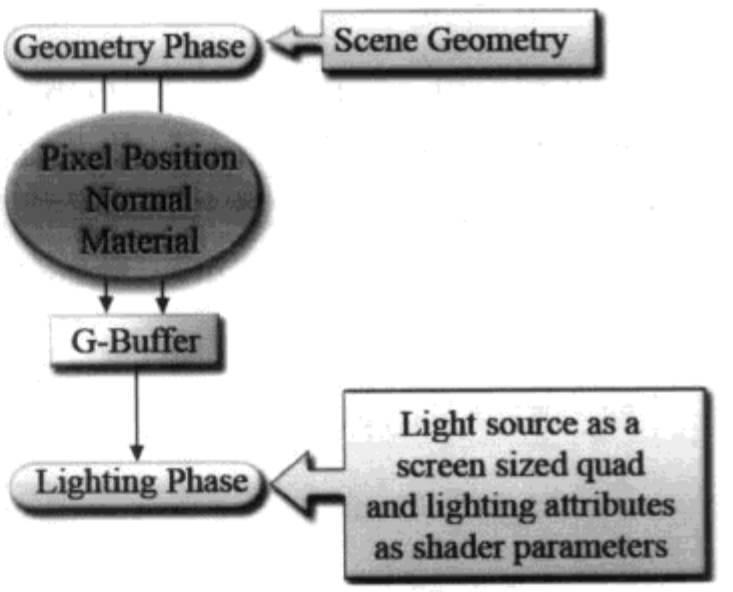


图 5.7.1　延迟着色流水线

5.7.2　高端硬件的延迟着色实现

如前所述，经典的延迟着色实现包括两个主要阶段：几何阶段和光照阶段；它们之间通过几何缓冲器（Geometry Buffer，G-Buffer）。本书附带光盘的“Deferred Shading MRT”目录下提供了一个关于延迟渲染的简单示例程序。

1. 几何缓冲器

构建延迟渲染系统过程中的最重要步骤之一是，决定需要哪些参数进行逐像素光照方程的计算。在第一个示例程序中，我们仅仅使用了像素的位置和法向量这些信息，原因是该例子仅仅展示了具有漫射性质（diffuse components）的点光源（point lights）。如果光照方程需要产生镜面反射或者其他高级的光照特效，其他诸如镜面反射能力和发光因子这些参数是需要额外存储的。

一旦所需要的参数定义完成，我们就成功构建了一个几何缓冲器。该缓冲器就是存储每个屏幕像素详细信息的地方。一般地，几何缓冲器是由若干个 render target 纹理实现的，几何阶段的像素着色器则会把选择好的参数存储在其中。

当涉及中高端的硬件时，我们完全有可能使用浮点纹理来存储位置和法向量信息。然而，通常我们有太多的数值不能全部存储在一个纹理中，因此，我们需要分几遍来传输不同纹理中的数值。但是，如果图形硬件支持多渲染目标（Multiple Render Targets，MRT），我们就有可能以一次把所有需要的参数都存储完毕。图 5.7.2 展示了光盘中的第一个示例程序是如何在两个浮点纹理中组织必要的像素数值的。

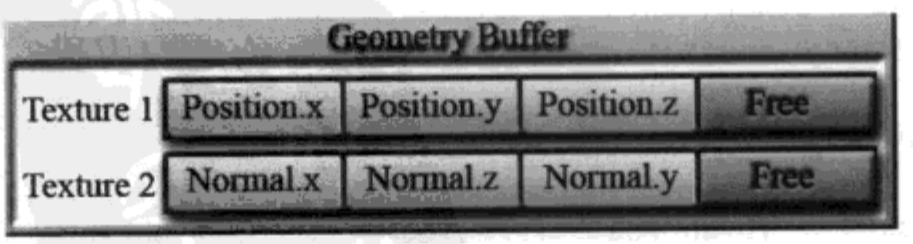


图 5.7.2　将位置和法向量存放到两个浮点纹理中

Download at www.Pin5i.Com

2. 几何阶段

在几何阶段，G-Buffer 被视为 render target，整个场景被提交到流水线。本阶段执行的着色器把所有必须的像素的几何以及表面属性打包到 G-Buffer 中。整个过程应当与第一个示例中的一样简单：像素的位置和法向量信息被发送到像素着色器，着色器则会轮流把这些数据打包成颜色输出（color output），最后存储在 G-Buffer 中。在第一个示例中，G-Buffer 由两个纹理组成，可参考图 5.7.2。

```
PS_OUTPUT ps_main( PS_INPUT Input )
{
    PS_OUTPUT o;

    o.Color0.xyz = Input.WorldPos;
    o.Color1.xyz = normalize( Input.WorldNormal );

    o.Color0.w = 0; // 虽然本例中这两个值未被使用
    o.Color1.w = 0; // 但它们很易符合镜面高光功率和吸收条件

    return o;
}
```

诸如贴皮（skinning）和位移贴图（displacement mapping）这些特效，都是转换场景的几何性质，因此都是在本阶段进行处理的。

3. 光照阶段

光照阶段是延迟着色最重要的阶段。目前，只有光照影响是需要计算的。其他的特效，如阴影（shadow）和视图色深（depth of view）都是应用在本阶段的。

在所有的参数保存到 G-Buffer 中后，组成 G-Buffer 的纹理将会作为着色器程序的输入参数而发送到本阶段。我们将会遍历所有的光源，对于每个光源而言：描述该光源的光照方程将会为每个屏幕像素求值。具体做法就是绘制一个覆盖屏幕的方形区域，然后为其中的每个像素执行着色器程序。

```
float3 LightPos;
float InvSqrLightRange;
float4 DiffuseLightColor;

float4 ps_main( float2 texCoord : TEXCOORD0 ) : COLOR
{
    // 屏幕像素位置和法线取自 G-Buffer

    float3 Normal = tex2D(Texture_Normals, texCoord);
    float3 PixelPos = tex2D(Texture_Pos, texCoord);

    // 计算光线衰减和方向

    float3 LightDir = (LightPos - PixelPos) * InvSqrLightRange;
    float Attenuation = saturate(1-dot(LightDir, LightDir));
```

Download at www.Pin5i.Com

```
    LightDir = normalize(LightDir);

    // 光照方程

    float DiffuseInfluence = dot(LightDir, Normal)*Attenuation;
    return DiffuseLightColor * DiffuseInfluence;
}
```

一旦完成对所有光源的方形区域的渲染和每个光源着色影响的计算，结果将会被混合到帧缓冲器（frame buffer）中；最后，屏幕便包含逐像素光照渲染后的完整场景。值得注意的是，当进行延迟着色的时候，我们可以随心所欲地使用任意光照模型的组合，而不会带来任何额外的开销。目前为止，我们仅仅展示了漫射点光源的处理，而对于诸如聚光源（spot light）或镜面泛光源（specular omni light）的处理，我们只需要编写额外的着色器程序，然后用相应的着色器程序渲染屏幕大小的方形区域即可。

5.7.3　基本存储优化

多渲染目标目前仍未普遍支持。这意味着对于缺乏 MRT 支持的系统，我们必须实现一种应变方案。该方案包括多次执行几何阶段，每一阶段都会存储不同的参数到 G-Buffer 中。多次执行几何阶段意味着反复地处理场景几何，渲染系统将会遇到 high-poly 场景中常见的性能问题，玩家则期待下代游戏的诞生。幸运的是，我们有避免多遍渲染的方法：巧妙地将参数封装到一个纹理之中！

以下列出了 G-Buffer 中通常存储的参数。

- 法向量。
- 位置。
- 材质属性，如放射性（emission）、镜面反射力（specular power），等等。

1. 法向量压缩

因为像素法向量参数是一个三维向量，因此我们需要在 G-Buffer 中存储 3 个数值。然而，法向量必须是单位向量，这允许我们跳过其中的一个数值，因为我们可以利用如下公式稍后将其恢复出来：

$$z = \pm\sqrt{1 - x^2 - y^2}$$

仅仅依靠该公式的问题在于 z 可以是正数或者负数，因此我们必须同时存储原来 z 的符号。围绕着这个额外的位，我们想到一种跳过它的方法。目前为止，示例所运用的光照方程，其采用的都是世界空间向量（world space vectors）。迁移到视图空间（view space）将会带来一些额外的益处。例如，如果所有的屏幕像素属于正面多边形，那么它们的法向量将会有同样的符号。当部分屏幕像素属于背面多边形且存在双面光照时，我们可以替而代之地在几何阶段存储它们的法向量，就像它们属于正面多边形一样。

尽管将法向量按照上述方法压缩成两个数值节省了部分存储空间，但是这种做法同样增加了额外的解压指令。幸运的是，当我们有额外的内存空间时，可以去掉这些额外的指令代

Download at www.Pin5i.Com

码。尽管法向量的 x，y 分量本应该属于区间[−1,1]，但是我们通常以[0,1]区间的数值将它们存储到 G-Buffer 中，这样做通常很有用且很有必要（稍后我们会介绍原因）。应用该转换后，我们可以使用二维的纹理坐标进行纹理查找（lookup texture）；每个像素的正规化法向量的 x，y 分量则分别代表纹理坐标系统中的 u，v 分量。

使用这种纹理以及 G-Buffer 中的 x，y 法向量分量，仅用一次纹理读取，我们就可以得到正规化的法向量，不再需要解压的数学代码，这为我们节省了最多 5 条算术指令。光盘中提供了一个关于产生任意大小纹理的示例程序（请参见“Compute Normal Texture”），同时还提供了第二个延迟纹理示例。

2. 位置压缩

与法向量一样，像素位置也是一个三维向量，但是与法向量不一样的是：它不是单位向量。因此，毕达哥拉斯（Pythagorean）技巧不能适用于像素位置。不过，仍然有办法可以减少存储到 G-Buffer 中的位置信息的数据量。

给定一个像素在屏幕上的位置，可以构建出从眼睛到那个位置的光线。如果已知从眼睛到像素的距离，要得到原始的位置，只要将正规化的光线乘以给定的距离，如图 5.7.3 所示。在光照阶段，因为我们可以从相机所在位置到屏幕像素创建一条光线，因此唯一真正需要存储到 G-Buffer 中的数据就是眼睛到像素的距离。需要存储的像素到摄像机的距离的计算方法很简单，其实就是计算表示视图空间位置的向量的长度。不过，构建 screen-eye 光线还是有些复杂的。

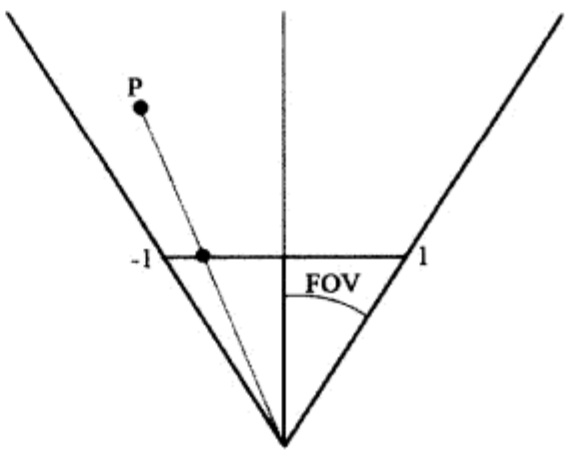


图 5.7.3 只给定距离，推导出像素的位置

在光照阶段，像素着色器可以从 G-Buffer 中取得像素的纹理坐标值。将取得的纹理坐标值从[0,1]的纹理坐标空间转换到[−1,1]的正规化屏幕空间，便可以求出原始投影像素位置。

第一眼看到正规化屏幕的 x-y 坐标系的时候，我们就知道坐标值可以直接赋给光线向量的 x，y 分量。但是 z 分量的值的计算就有些技巧性了。当定义视角投影矩阵的时候，我们使用了视图角度部分（field of view angle）。如图 5.7.3 所示，我们完全有可能使用该角度的正切值计算出相关的距离（该段距离上的半屏长度是单位 1）。该距离就是所求的光线向量的 z 分量值。

即使当我们计算出光线向量的各个分量，但是我们并没有考虑屏幕的长度和宽度是不一样的。解决这个问题非常简单：只要把光线向量中的 x 分量乘以用以构建投影矩阵的视图纵横比即可。下列代码展示了在光照阶段的顶点和像素着色器中构建 eye-screen 光线的过程。

几何阶段的顶点着色器：

```
o.EyeRay = float3(inPos.x * ViewAspect, inPos.y, TanFOV);
o.Depth = length( ViewSpacePosition );
```

光照阶段的像素着色器：

```
float3 PixelPos = normalize(Input.EyeRay) * Depth;
```

Download at www.Pin5i.com

一个完整的关于位置压缩过程的示例可以在光盘中所带的第二个延迟示例中找到。

3. 材质压缩

材质特性（material attributes）是一组定义表面属性的数值。这些数值包括辐射颜色（emission color）、镜面反射能力（specular power）和发光因子（glow factor）。每一种属性的存储都需要占用 G-Buffer 大量的存储空间。一种避免这种恶劣的存储消耗的方法就是忽略其中的部分属性数值，并且不在光照方程中使用，尽管这将无法避免地导致光照质量的严重下降。

注意观察一下材质特性的特点，就会发现在大多数情况下，这些数值并不是逐像素改变的，而是逐表面。假定材质的数量有限，压缩特性的另外一种方法就是把它们存入一个辅助的材质数组中，该数组包含了每种材质所需的所有特性；在光照着色器中，我们把该数组当做材质调色板，用来从中解压出所有的材质属性。这种方法的好处就在于，只需要将数组中材质的位置存储在 G-Buffer 中。

由于延迟渲染系统所支持的材质的数量有限，因此我们有两种方法将材质调色板提交给光照着色器。如果调色板的长度不超过着色器的常量寄存器的数目，渲染系统会把材质特性数据存放到寄存器中，那么光照像素着色器可以通过索引查找来访问需要的材质。但是，如果调色板的长度超过了现有的常量寄存器的数目，那么材质特性数据就会被存储到一个额外的纹理中，而该纹理中的每列代表一个不同的材质。在光照着色器中，表面属性必须通过使用纹理拾取来访问。如果材质的数量或者任意表面特性在模拟期间都不发生改变，那么材质纹理就没有必要在运行时创建，而是在编译游戏关卡时。

5.7.4　着色器优化和硬件限制

由于需要存储的数据量的降低，使得我们在 G-Buffer 中只要使用单个纹理即可。不过，由于浮点纹理并不是广泛支持的，因此，密切关注每个参数实际所需的精度将会大大降低对目标硬件的要求。接下来，我们将要介绍一些关于如何减少位存储位数的例子。

法向量的 x，y 分量可以以 8 位整数表示，而不会损失太多的质量。实际上，在一些场景中，它们甚至以 5 位或者 6 位整数表示而无须担心质量的损失；不过由于 8 位整数与凹凸（bump）和法线（normal）纹理的精度一致，因此它是更常见的选择。

与前面一样，位置向量可以编码成一个表示视点到像素距离的数值。降低色深值的精度对场景质量有显著的影响；不过，对于中低质量的图形选项而言，使用 24～16 位的数据来表示色深的确是一种应变方案。参见图 5.7.4 和彩图 10a、10b，展示了不同色深精度下的外观上的区别。

光盘上的第三个延迟示例程序演示了如何使用 16 位来存储像素色深，1 字节来存储法向量分量，这样，单个被广泛支持的 A8R8B8G8 纹理就可以存下这个数据。如果需要使用不同的材质配置来存储镜面反射能力或者漫射颜色，那么只要将法向量分量的精度降低到 6～7 位，就可以在 G-Buffer 中额外存储 4～15 种不同的材质组合。

Download at www.Pin5i.Com

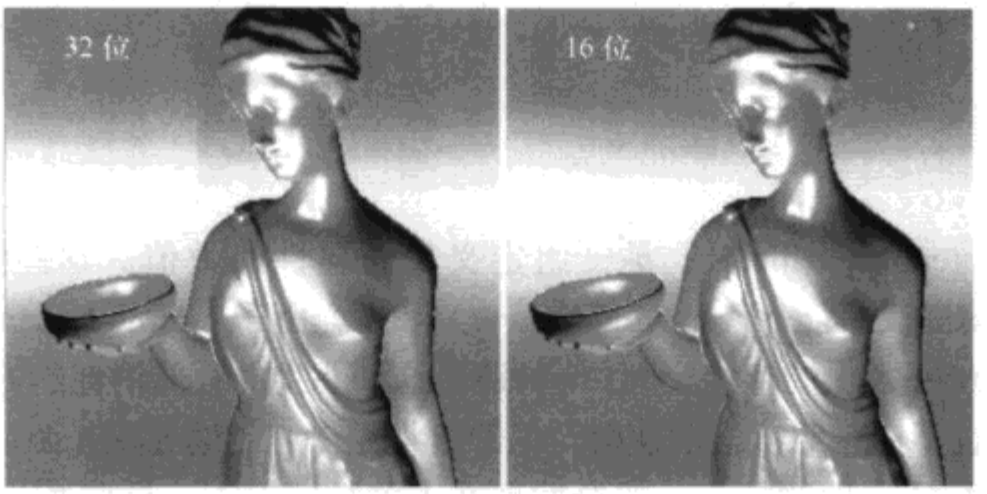


图 5.7.4 使用不同色深精度进行场景渲染

对于中低端的硬件而言，光照的速度非常重要。一种基本的解决方案就是优化着色器代码。例如采用部分常见的光照着色器改善方案。衰减因子（attenuation factor）可以在一维纹理中预先计算好，然后通过纹理拾取获得，这样可以避免使用一些算术指令。一旦不需要额外的精度，我们就可以像上个示例一样，用 half-data 类型替代 float 类型，从而改善某些显卡上的着色器程序。指令顺序的重排（rearranging）或者指令的重新生成（reformulating）有助于编译器产生更快的程序。一般而言，大部分常见的着色器优化都可以适用于延迟着色程序。

1. 高级延迟着色优化

仅仅对延迟渲染框架不同阶段所包含的着色器进行优化，就可以支持大量的图形硬件，并且提高整个系统的性能。但是，如果渲染过程严格按照本节描述的实现，且每个不支持高级优化的光源作为一个方形区域发送到光照阶段后，系统会遭受填充率瓶颈问题。

在下面的小节中，围绕着 Light Manager（见图 5.7.5），介绍了一些高级优化技术。该 Light Manager 是工作于渲染系统和光照阶段着色器之间的防火墙，通过 social 和 individual 这两个主要阶段实现。

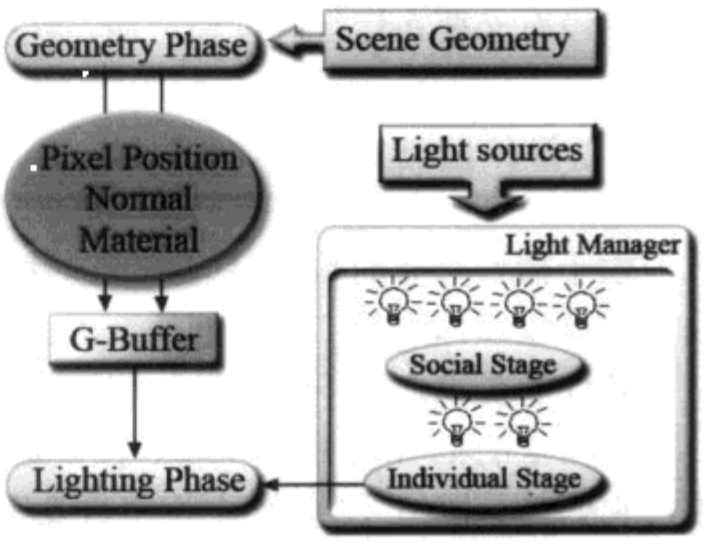


图 5.7.5 使用高级 Light Manager 的延迟着色流水线

2. Social 阶段

在本阶段（第一阶段），系统会访问所有的光源，对它们使用通用的优化来降低所需处理的光源的数量。首先，使用某些算法找出哪些光源是被场景几何所遮蔽或者不在相机的可视区域内。这些算法会对其使用的光源限制对象（light-bounding objects）进行适当地删减。那样的话，只有对可视屏幕区域产生影响的光线才会被使用。但是，不是所有可视光源对于场景都是有重大意义的。当系统出于性能的考虑，需要损失一定质量和精度时，我们可以忽略掉那些只对极少数像素有影响的光源。其他高级优化的方法就是避免处理那些在屏幕空间中非常接近且影响的几乎是同一个区域的光源。我们可以小心地把这些光源合并成一个明亮

Download at www.Pin5i.Com

的大光源，而不会带来许多致命错误。如果光源离开相机位置较远，以上的优化可以很好地工作。

虽然目前为止我们介绍的上述步骤可以大大地减少需要处理的光源的数量，但是系统的整体性能可能仍然非常低。因此，我们可以将影响每个区域的最大光源数量调整为 8～10 个。类似地，如果光源数量增长过大，那么只有最大且最亮或者最近的可视光源才会被处理。

3. Individual 阶段

一旦 social 阶段结束，就决定了一组光源。Individual 阶段会遍历这组光源，并对每个光源的处理开销进行优化。创建场景时，光源被分为两类：全局光源和局部光源。

全局光源是指影响整个场景的光源。它们是代价最大的光源之一，因为它们需要处理每个屏幕像素，而且一般而言是不能进行高级优化的。这些光源是通过光照着色器所绘制的屏幕大小的方向区域而实现的。与全局光源相反的是，局部光源有位置和形状。由于局部光源仅仅影响光源限制对象内部的像素，大部分时间都不会影响到所有的屏幕像素，因此性能提升的余地很大。

例如，将裁剪测试（scissors test）配置到光源限制对象的周围后，就可以快速地忽略那些裁剪区域（scissor rectangle）之外的像素。即使是限制对象（bounding object）本身也可以用直接渲染来替代前面所描述的使用屏幕大小的方形区域。动态分支（dynamic branching）可以忽略那些未点亮的像素，具体做法是：如果是以光源为中心的球体，球体半径之外的像素全部被忽略；如果是普通的 mesh，限制对象之外的像素全部被忽略。[ATI05]中介绍了一种在不支持动态分支的硬件上，利用裁剪测试来实现对其支持的技术。

光照层次细节是另一种可以在 Individual 阶段实现的功能，该功能允许在系统性能和画面质量之间做相应地调整。这种方法使得远处的光源计算质量没有必要与相机附近的光源一样。图 5.7.6 中表示了 3 种光照级别的细节效果，包括“无光源”级别。如图所示，第二级别的细节实际上包含了一个乘法因子 t。通过把 t 设置成 0～1 之间的数值，可以像图中所示的那样，平滑地从一个级别变成下一个级别。因此，在第二级别细节的开始处，可以清楚地看到镜面反射；随着逐渐往下一级别过渡，镜面反射会逐渐淡出；当到达下一级别时，镜面发射就彻底消失了。那样的话，从一个级别的细节到下一个级别，是不会出现引人注目的光照突变的。

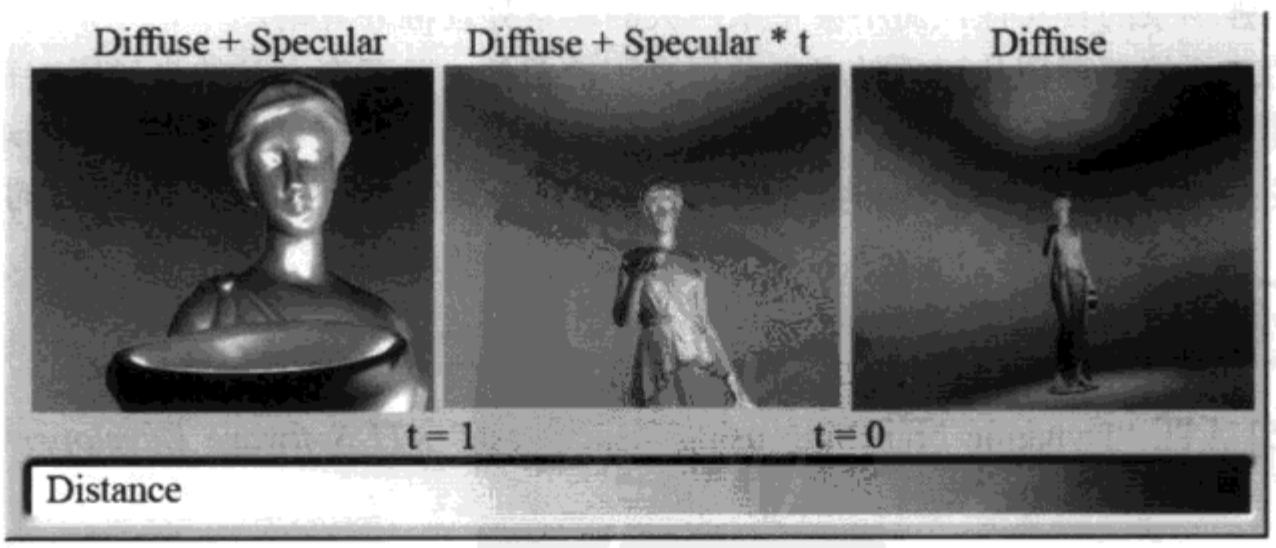


图 5.7.6　光照层次细节示例

Download at www.Pin5i.Com

5.7.5 扩展图像空间（Extending Image-Space）的后处理特效（Post-Processing Effects）

在本节中，术语“延迟着色”和“延迟光照”是作为同一项技术的同义词而使用的。目前为止，虽然我们仅仅讨论了光照技术，但是对于延迟着色而言，光照只是其一，而不是全部。延迟解决方案（deferred solutions）通过将屏内几何（in-screen geometry）的控制权赋给图像空间的后处理程序，就可以获得更好的效果。最后一项功能是将以后处理特效的方式来进行光照。

既然通过访问 G-Buffer 可以知道每个像素的距离，那么增加诸如逐像素雾化（per-pixel fog）或者体积雾化（volumetric fog）这些特效就变得简单而迅速。雾化转变成一种特殊的全局光源，这种光源不是起照明（illuminating）作用，而是简单地根据像素色深（pixel depth），褪变为预定义的颜色（predefined color）。尽管以标准渲染方式来使用正向阴影贴图（forward shadow mapping）和景深（depth of field）并不是直接的方式，但是两者却非常适合延迟着色流水线（deferred shading pipeline）。

延迟着色的真实威力不仅仅是实现多种多样的后处理特效，而且还有可能组合这些特效。将诸如热雾变形（heat haze distortion）这些特效引入渲染流水线后，不再需要多遍渲染；而是简单地实现一个利用 G-Buffer 进行特效处理的精妙着色器，然后通过引入一个新光源将其添加到系统中。通过将光源转换为局部光源，就可以将其优化成仅仅影响世界中某个界定区域的光源。

延迟着色大大增强了图像空间后处理特效的能力，从而在没有进一步几何处理的前提下，混和使用这些特效来进行场景渲染。为了完成最终图像真实性所需要的着色，真实感场景需要大量的后处理特效。带着对渲染技术的上述期望，你会发现延迟着色无疑是目前最快、最简单的实现大量特效的技术之一。

5.7.6 总结

如前面所述，延迟着色技术使得我们可以用大量的光源来渲染场景，而无需担心每个对象上的光源数量以及所使用的光源组合类型。它同样可以利用其他后处理特效来进行光源组合，从而增强场景的真实性，而不会带来巨大的场景处理额外开销。

下代游戏系统更期望从这项技术中获得真正的益处，除了有大量的光源影响每个单表面外，还会使用非常复杂的着色器。目前为止，只有延迟着色系统可以应付此种需求，而不会被场景的复杂性拖垮。不管你期望场景看起来是如何的真实，延迟着色都是很好的解决方案。

5.7.7 参考文献

[ATI05] ATI, “Dynamic branching using stencil test.” *ATI Software Developer's Kit,* June 2005.

Download at www.Pin5i.Com

5.8　路标渲染的清晰化

Jörn Loviscach，德国不莱梅应用技术大学
jlovisca@informatik.hs-bremen.de

标准的纹理化（standard texturing）不能很好地利用清晰特征（sharp feature）来处理图像。交通标志、路标、广告牌或者标签上的文字，都会随着观察者靠近而逐渐模糊。不过，小型像素着色器经过特殊预处理的纹理，可以生成与矢量图形相媲美的照片质量级的特效。本节中将要介绍的技术是采用阈值划分（thresholding）标准纹理来生成二进制图像。该技术适用于颜色种类稀少的纹理，如文本标签（黑、白）、无烟标志（黑、红、白），以及路标（白色透明）。

标准纹理的模糊度在于位图格式图像的性质：在一定放大比例下，不管纹理图像的分辨率有多高，它的像素（即纹理像素）是可分辨的，参见图 5.8.1 和彩图 11。图形硬件中普遍适用的双线性插值法（bilinear interpolation）可以避免这种情形下的像素阶跃（pixel staircasing，又叫做 jaggies）。但是，它总是消除那些原本应该很清晰的边，比如字体符号的轮廓线。

如果纹理仅仅包含纯黑和纯白两种颜色，基于以下原则的方法简单地解决了上述问题：使用像素着色器对双线性插值纹理进行阈值划分。也就是，如果取得的灰度级别高于中等灰度，则将其转变为全黑色；反之则转变为全白色。因此，所有的灰色像素都会消除，而黑白区域之间将会建立明显的分界线。

不过，事情并不是那么简单。如图 5.8.2 所示，上述方法生成的轮廓线不仅失真（wiggles），呈现锯齿（jaggies），并且从一定距离观看，演变成一块盐、胡椒状的奇怪云块。

我们可以采用[Loviscach05]中的方法简单地解决上述 3 种问题：无须增大纹理的大小，无须为每个像素增加额外的纹理，但需要为像素着色器增加一些占用很小开销的代码。从这种考虑出发，该技术与相关工作是不一样的。例如，[Ramanarayanan04]、[Sen04]、[Tarini05]和[Tumblin04]中使用内置的清晰功能（sharp feature）来处理全彩纹理，相关技术的计算开销非常巨大。

与上述相反的是，[Loviscach04]和[Loop05]中利用同等效率的像素着色器来构建三次曲线（cubic curves）所界定的形状。不过，这两种做法都依赖于沿着边界曲线放置三角形，这对于复杂的字体而言，可能需要成百上千的三角形。这对于当今的用户级显卡而言不是很好的选择，因为这些

Download at www.Pin5i.Com

显卡对纹理的支持明显优于几何。

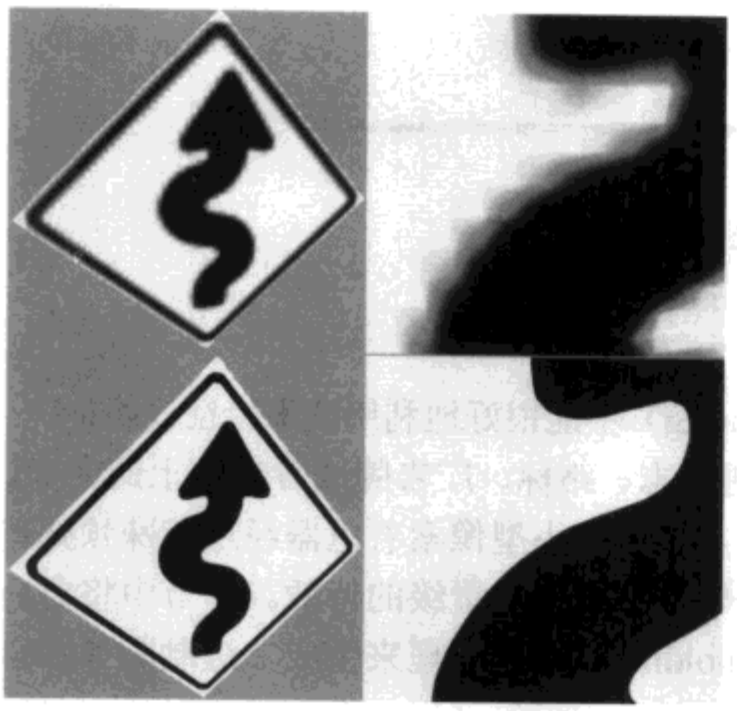


图 5.8.1 双线性插值法得到的标准纹理往往产生模糊字体（上图，64×64 纹理像素）。本节介绍的技术使用了一个简短的像素着色器和一个预处理的纹理来产生清晰的纹理（下图，同样的纹理大小）

图 5.8.2 把标准纹理（左上方）交给硬阈值划分（右上方）处理，会产生锯齿（右上方的 8 倍放大插图），振荡（左下图），远处的盐、胡椒特效（右下方及其中的插图）

5.8.1 反走样阈值划分纹理（Antialiasing Thresholded Textures）

为了保证放大条件（屏幕像素小于纹理像素）和缩小条件（屏幕像素大于纹理像素）下纹理的高质量，我们需要使用一些特殊的度量方法。放大条件下，可以消除锯齿；缩小条件下，纹理会粗糙得像涂抹过一样，没有像素，而是由全白或者全黑替代。

1. 软阈值划分（Soft Thresholding）

由于不具备能够柔化黑白像素梯度（staircase）的灰度（grayscale）像素，阈值划分（thresholding）马上导致锯齿现象。对付这种问题的一种简单的方法就是放弃纯阈值划分（pure thresholding），但是会更加增大对比度。这种情况下，通过双线性插值得到的灰度值 t 不会进行阈值划分：

```
float c = 0.0;
if(t > 0.5)
    c = 1.0;
```

而是提高对比度：

```
float c = clamp(0.5 + a*(t-0.5), 0.0, 1.0);
```

其中 a 控制放大数量。数值 1 重新产生输入；随着 a 趋向无穷，强对比度逐渐淡化为阈值划分。通常 *clamp* 指令会被忽略，因为图形硬件将会自己钳制（clamp）颜色。

a 的数值控制了边的清晰度。目的是以一种消除锯齿的方式来设置 a，但始终保持高清晰度。因此，无论什么比例，黑色与白色之间的转换总是在屏幕空间有相同的宽

Download at www.Pin5i.Com

度。在上述方法的试验中，1/3 像素的宽度最后在锯齿度和清晰度之间的做出了最优的权衡。

黑白变化开始于一个纹理像素过渡到另一个时。如果视点靠近纹理，纹理就被放大。这两个纹理像素可能被映射到其所位于的像素上，比如 10 个分开的屏幕空间像素。将 a 设置为 1 将会导致扩展到 10 个像素上的黑白转变。不过，我们所希望的是 4 5/6 个像素是黑色，1/3 个像素是转变区域，4 5/6 个像素是白色。因此，a 必须设置为 10×3：4 5/6 个像素后，t =29/60，灰度值在黑色那边开始增长（如图 5.8.3 所示）。通常，这意味着将 a 设置为 3×s，其中 s 是像素的屏幕空间大小。

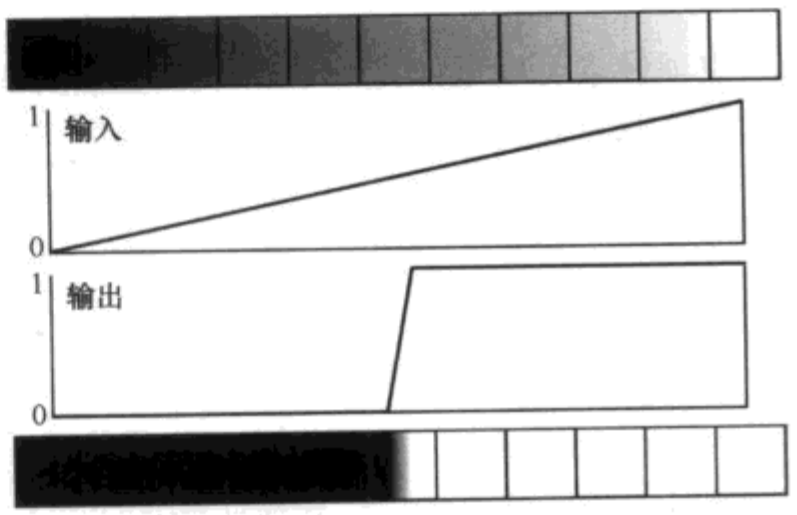


图 5.8.3　为了将 10 个像素的屏幕空间距离转换为 1/3 像素的过渡区域，扩大倍数要设置为 30

2. 纹理大小

为了正确地调节（scale）颜色，需要在运行时决定相邻纹理像素的屏幕空间距离。像素着色器提供了特殊的辅助指令。在 HLSL（High Level Shading Language，高级着色语言）中，这些指令叫做 ddx 和 ddy，用于为相关的参数计算分别关于屏幕空间 x 和 y 的导数。为了在设置中使用这些指令，可以用额外的纹理坐标（uvSca led）来装备对象。这些纹理坐标不是位于 1×1 的范围内，而是宽×高。除此之外，还可以用标准纹理坐标 uv 加一次额外的二维向量乘法，来形成 uv*float2（宽，高）。

二维向量 ddx（uvScaled）和 ddy（uvScaled）横跨了某个屏幕空间像素在纹理空间（以纹理像素单位度量）的椭圆形轨迹，如图 5.8.4 所示。在斜视图（oblique view）或者变形纹理坐标中，轨迹可以变成强力拉长的椭圆形；该椭圆沿主坐标轴方向比次标轴方向横跨更多的纹理像素。

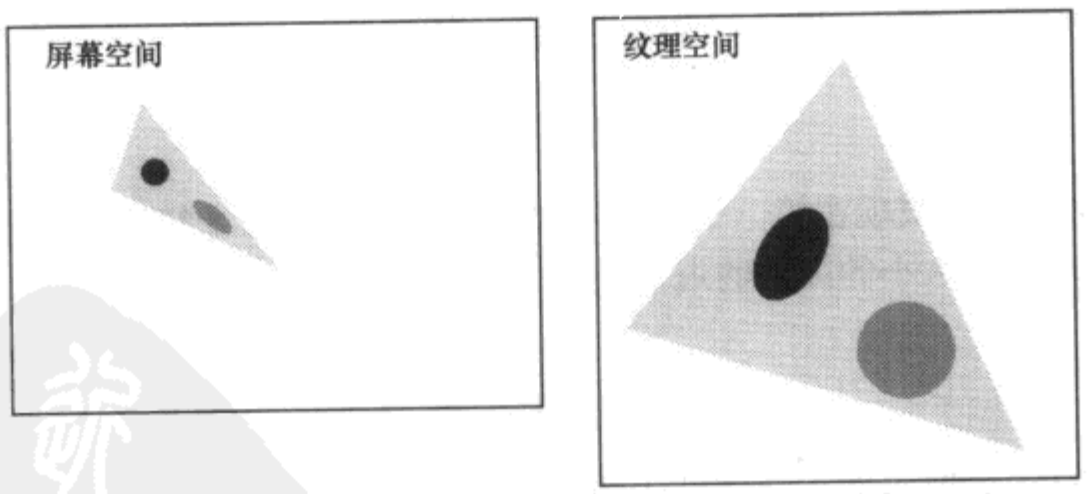


图 5.8.4　圆形的屏幕空间像素在纹理空间的轨迹是一个椭圆，反之亦然

为了计算放大系数 a，需要每个纹理像素的屏幕像素数目，也就是必须知道每个纹理像素在屏幕空间的轨迹。这是一个反转设置（inverse setting），用图 5.8.4 中的中等灰度的圆状物体（circle and disk）表示。出于方便，我们称 ddx（uvScaled）的分量为 e，f；ddy

Download at www.Pin5i.Com

（uvScaled）的为 g，h。因此，将 x-y 屏幕坐标空间的本地变化转换为量化纹理坐标空间的变化的矩阵是：

$$\begin{pmatrix} e & g \\ f & h \end{pmatrix}$$

纹理像素可以近似为 uvScaled 纹理空间内的半径为 1/2 的圆盘，而其在屏幕空间内的轨迹可以近似为椭圆盘。该圆盘由所有偏离中心$(\Delta x, \Delta y)$的点组成，也就是：

$$\left|\begin{pmatrix} e & g \\ f & h \end{pmatrix}\begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix}\right| \leqslant \frac{1}{2}$$

可以写做：

$$\begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix}\cdot\begin{pmatrix} e & g \\ f & h \end{pmatrix}^{\mathrm{T}}\begin{pmatrix} e & g \\ f & h \end{pmatrix}\begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} \leqslant \frac{1}{4} \tag{5.8.1}$$

圆盘的形状由矩阵的特征值决定：

$$\begin{pmatrix} e & g \\ f & h \end{pmatrix}^{\mathrm{T}}\begin{pmatrix} e & g \\ f & h \end{pmatrix} = \begin{pmatrix} e^2+f^2 & eg+fh \\ eg+fh & g^2+h^2 \end{pmatrix}$$

该特征值表示为：

$$\lambda_{1,2} = \frac{1}{2}(e^2+f^2+g^2+h^2) \pm \sqrt{\frac{1}{4}(e^2+f^2+g^2+h^2)^2 - e^2h^2 - f^2g^2 + 2efgh} \tag{5.8.2}$$

沿着 x-y 屏幕空间内的两个主要坐标轴之一的某个向量将会与相应的λ相乘。从椭圆的中心到边界扩展出一条主要坐标轴，而假设 v 是沿该轴的向量，其长度$|v|$等于椭圆的主半径或次半径。根据等式 5.8.1 得出：

$$\lambda_{1,2}|\mathrm{v}|^2 = \frac{1}{4}$$

因此

$$|\mathrm{v}| = \frac{1}{2\sqrt{\lambda_{1,2}}}$$

因此，椭圆的主直径和次直径则是$|v|$的两倍，分别是：

$$\frac{1}{\sqrt{\lambda_1}} \text{ 和 } \frac{1}{\sqrt{\lambda_2}}$$

等式 5.8.2 中的平方根项在第一个解中以加号出现，在第二个解中则以减号出现。因此，为了给出关于纹理像素大小的简单的近似公式，我们一并忽略了上述平方根项：

$$s = \frac{\sqrt{2}}{\sqrt{e^2+f^2+g^2+h^2}}$$

上述等式可以快速地通过 4 维向量计算求得，因此我们如下设置：

```
float s = 1.4/length(float4(ddx(uvScaled), ddy(uvScaled)));
```

不过，如果因为偏爱清晰度而使用算术平均或者主直径的话，会招致长时间地计算等式 5.8.2 中的平方根项。

Download at www.Pin5i.Com

3. Mip-Mapping 转换

如果离开一定距离观察纹理，其将会转变或大量的灰色像素，就像使用标准 mip-mapping 一样。如果坚持基于可控对比度增强的主要思想，其将由如下方法实现：

```
float c = clamp(0.5 + a*(t-0.5), 0.0, 1.0)
```

通过 mip-mapped 纹理，可以取得 t，而 a 则取 1.0（屏幕空间的纹理像素远比像素本身小时），从而容易地求出上述等式的值。

给定前面计算得到的 s，可以简单地设置：

```
a = max(1.0, 3.0*s);
```

因此，纹理像素大小缩小到 1/3 像素水平之下时，像素着色器的结果等价于标准 mip-map。图 5.8.5 显示了 s 和 a 之间的依赖性。

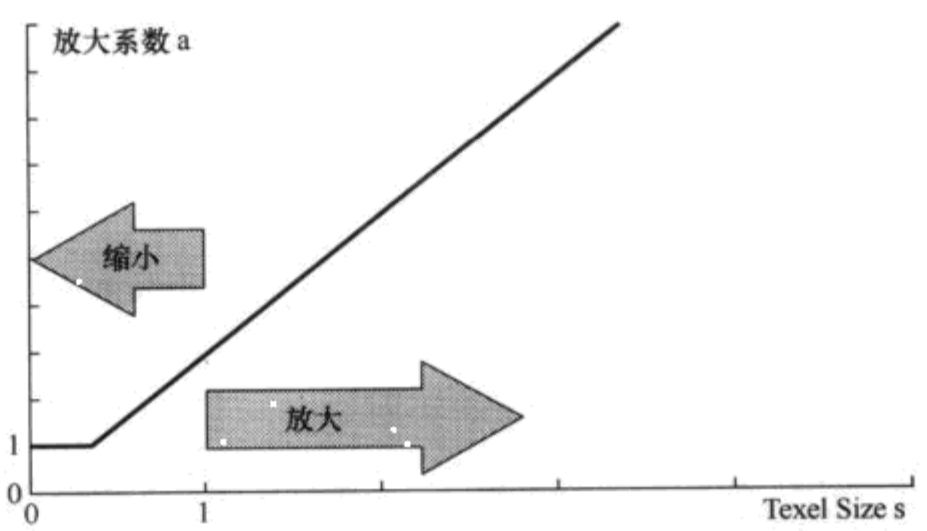


图 5.8.5　放大时，放大系数是纹理像素大小的 3 倍；缩得够小时，返回纹理的原始值

综上所述（加上一些优化手段后），像素着色器如下：

```
half4 t = tex2D(mySampler, IN.uv);
float2 ef = ddx(IN.uvScaled);
float2 gh = ddy(IN.uvScaled);
half sInv = length(float4(ef, gh));
half a = max(1.0, (3*1.4)/sInv);
half c = 0.5 + a*(t.r-0.5);
OUT.color = half4(c, c, c, 1.0);
return OUT;
```

上述程序编译成 11 条 ps_2_x 版本像素着色器的汇编指令。为了在放大率的阈值划分和缩小率的标准 mip-mapping 之间实现一个干净的转换，控制放大率是远远不够的。在转换范围内，a 等价于 1.0，因此纹理中的中等灰度级别的像素变化不大。这种效果导致靠近黑白分界处，出现令人讨厌的灰色斑点，如图 5.8.6 所示。

如果 mip-map 的最优级别不包含中等灰度级别，上述问题可以迎刃而解。实验表明，标准 8 位数据格式可以禁止掉 256 个级别的中间 32 个（即[0～255]区间中的[112～143]子区间）。

Download at www.Pin5i.Com

下面小节中要讨论的填充 mip-map 级别 0 的优化过程，正是考虑了这一点。

图 5.8.6 如果存储的纹理（左边）包含的灰度值接近中等灰度，对比度增强的图像在放大系数 a 接近 0 的时候，可能会显示出灰色的光晕或洞（右边）

5.8.2 阈值划分的优化纹理(Optimal Textures for Thresholding)

本小节处理的纹理通常都通过矢量图形生成，例如通过图像软件的位图导出功能。但是采用阈值划分技术的话，这些位图会导致轮廓线上的凹凸不平。可以将产生不同的纹理图像这一特性作为自己的优势。

1. 几何

我们期望由阈值划分双线性插值纹理所形成的形状由 polyline（不规则的连续的线）界定：线性操作应该产生什么？然而，这对于三角形网格而言是成立的；对于标准纹理贴图中使用的方形网格而言是不成立的（如图 5.8.7 所示）。

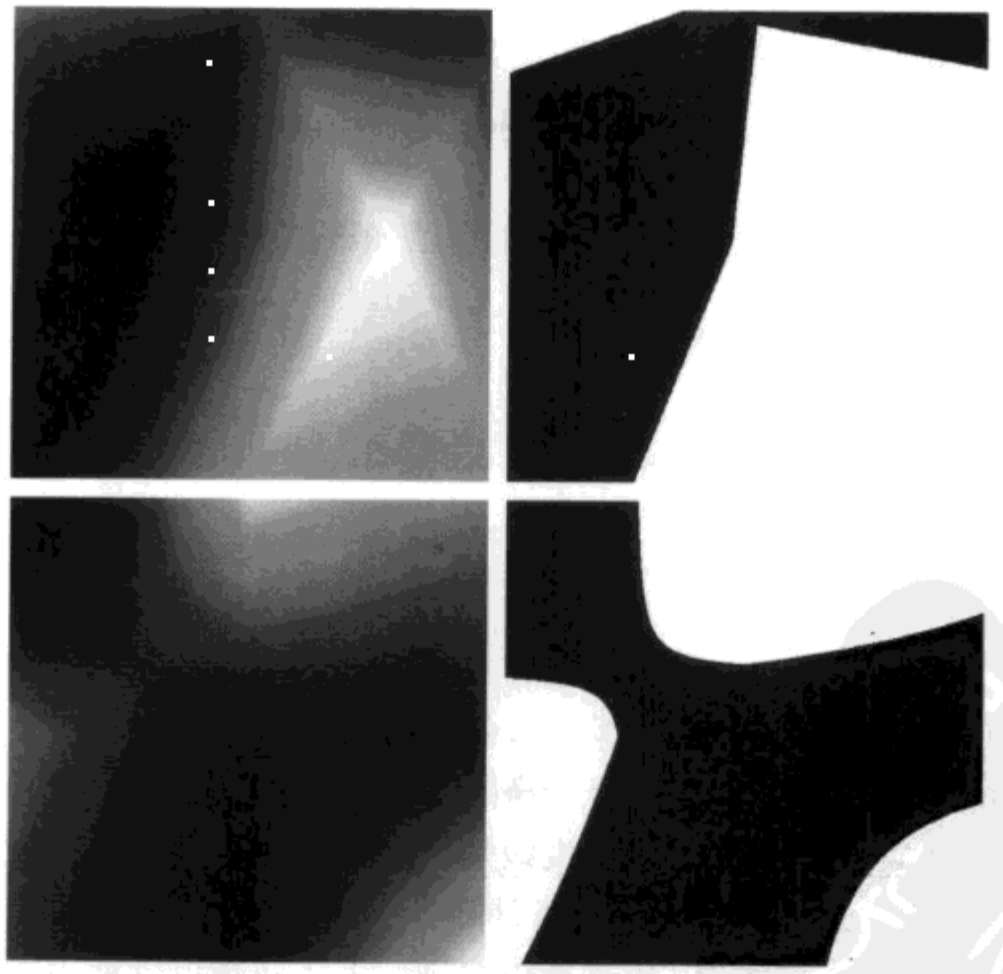


图 5.8.7 在三角形网格中，阈值划分的双线性插值将产生由 polyline 所界定的形状（右上方）。但是，在正方形网格中，轮廓线会变成有理曲线（下方）

Download at www.Pin5i.Com

如果 a、b、c 分别是三角形的内角，双线性插值导致：

$$(1-u-v)a+ub+vc,$$

其中 u、v、（$1-u-v$）是三角形的重心坐标（barycentric coordinates）。然而，如果 a、b、c、d 是正方形的内角，双线性插值导致：

$$(1-u)(1-v)a+u(1-v)b+uvc+(1-u)vd,$$

可以整理成：

$$a+u(-a+b)+v(-a+d)+uv(a-b+c-d).$$

注意公式中出现的乘式 uv，其会将公式转变成非线性表达式。

阈值划分纹理的轮廓线（contour）经过点（u, v），此点处上述表达式的值等于 1/2。对于这个曲线，我们可以为 v 找到关于 u 的有理函数关系（rational dependency）：

$$v=\frac{\frac{1}{2}-a-u(-a+b)}{(-a+d)+u(a-b+c-d)} \tag{5.8.3}$$

如果上述等式的分母不为 0。

我们可以解压出关于几何的精确数据。假设轮廓线与 b 值纹理像素和 a 值纹理像素之间的边相交。这意味着要么 b 小于 1/2，a 大于 1/2；或者反之亦然。轮廓线与边相交于点（u,v），其中：

$$u=1, v=\frac{b-\frac{1}{2}}{b-c}$$

通过等式 5.8.3 对 u 求导数，可以计算出轮廓线与边相交点的角度。导函数值等于曲线与水平方向交角的正切函数值。u=1 时的导函数值：

$$\frac{ac-bd-\frac{1}{2}(a-b+c-d)}{(b-c)^2}=\frac{\left(a-\frac{1}{2}\right)\left(c-\frac{1}{2}\right)-\left(b-\frac{1}{2}\right)\left(d-\frac{1}{2}\right)}{(b-c)^2}$$

与其他边相交点的等式与上述类似。

2. 优化

由于轮廓线是有理曲线（rational curve），因此不容易控制。我们为任务提供了两组参数：第一组，轮廓线与纹理像素网格边的交点位置；第二组，交点处的轮廓线曲线方向。两种类型的参数都可以很容易地按照前面描述的方法计算出来。同样的数据从原始矢量图形中读取出来作为比较的依据。为此，我们可以对高分辨率位图版本的原始图形采用标准的图像处理技术。

目的是以这样一种方式计算纹理，以便原始图像的位置和角度数据可以在阈值划分后很好地再生。由于轮廓线的细节信息只对放大视图有意义，因此，较高（即较粗糙）的 mip-map 级别可以照常用 box filtering 填充。只有最优级别（即 mip-map 级别 0）会发生改变。

结果是只有在异常情况下，无法以这种方式构建纹理，以至于实现了对所有位置和角度的约束。如果轮廓线与纹理像素网格的 n 条内边相交，就会产生 n 个位置与 n 个角度条件。相邻纹理像素的灰度值可以有效地进行调整。如果轮廓线没有很急的转弯，纹理像素的数目

Download at www.Pin5i.Com

是 $2n$（如图 5.8.8 所示）；否则，这个数目就有点低了。由于当所有的灰度值定量为 1/2 左右时，阈值划分图像（thresholded image）不会发生变化，因此，我们会损失更多的自由度。

这些图示表明：约束条件通常都是不会被完全遵循的。我们需要适当地使用一种错误度量法来指导优化过程，在下面的小节中会介绍这种度量方法。这种优化过程速度非常快，因为它仅仅影响与轮廓线相邻的纹理像素。绝大部分的纹理像素可以保持纯黑色或者纯白色。

更为显著的是，优化导致更尖锐的内角。如图 5.8.9 所示，它会自动地用抵衡纹理像素（counterbalancing texels）来处理内角。

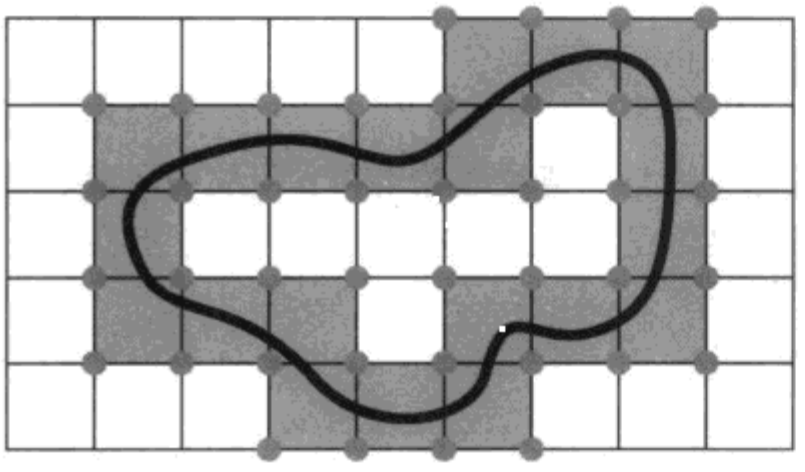


图 5.8.8 与 n 条边相交的轮廓线通常受 $2n$ 个纹理像素影响。本例中 n=20，有贡献的纹理像素就是 40

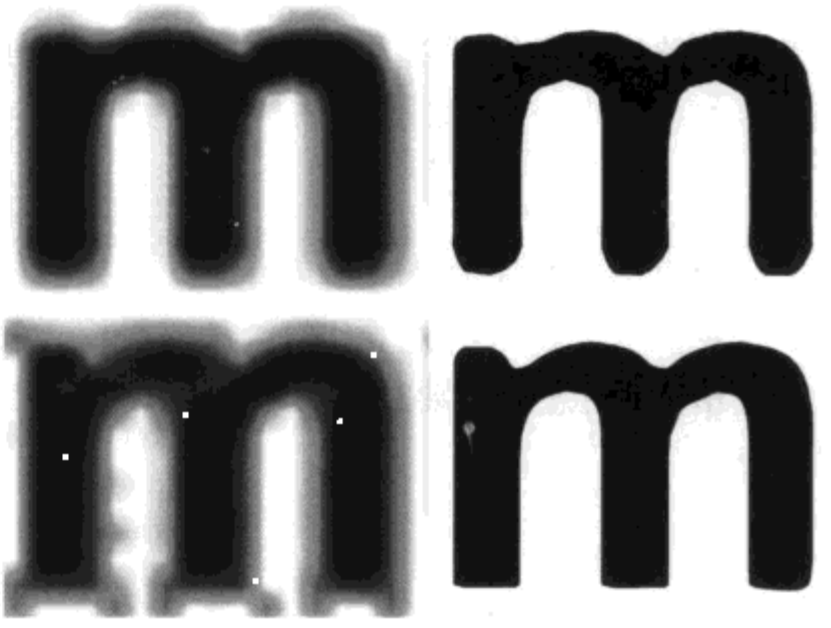


图 5.8.9 在拐角的临近处，优化产生的图样复杂，允许急转弯（上方：未优化的和阈值划分的；下方：已优化的和阈值划分的）。注意，纹理像素的大小

3. 错误度量

我们为错误度量提供了两种简单的选择：第一，轮廓线与纹理像素网格边交点的期望位置与实际位置之间的平方误差和；第二，轮廓线在交点处的期望角度与实际角度之间的平方误差和。

实验表明，以上两种度量方法都会导致令人讨厌的轮廓线错误，如图 5.8.10 所示。虽然第一种度量方法可以保证所有期望交点的命中率非常准确，但是它会导致中间出现明显的 Z 字形图案。第二种度量方法得到的轮廓线可以很好地再生软曲率（soft curvature）；不过，字体会变凸，且字干宽度是不规则的。

因此，建议的解决方案是综合这两种度量方法。通常，更倾向于角度的精度。不过，角的一边与纹理像素网格边越接近垂直，位置的精度越是被重视。这将得到很平滑的轮廓线，但是同时也很好地模仿了字体的垂直和水平线，如图 5.8.10 所示。

在以下程序代码中，这就归结为对每个交点的错误度量的额外支持。

```
double r = Math.Min(1.0, Math.Abs(angleOriginal));
return r*angleDistance2 + (1.0-r)*positionDistance2;
```

其中 angleOriginal 是期望角度（弧度为单位），angleDistance2 是交点处的实际进出角与期望角之间差的平方和，positionDistance2 是期望位置与实际位置之间的平方差。

Download at www.Pin5i.Com

图 5.8.10　单个的基于交点位置（左上方）或者交点角度（右上方）的度量都会导致讨厌的错误。综合的度量法（下方）可以避免大部分错误

5.8.3　创作程序（The Authoring Application）

光盘中带了一个创作程序，包括源代码（见图 5.8.11）。该程序为阈值划分产生最优化的 DirectDraw Surface（.dds）纹理文件，并且通过像素着色器提供了三维预览。该程序的开发平台是.NET 1.1 和托管的 DirectX 9.0c，开发语言是 C#和 HLSL。

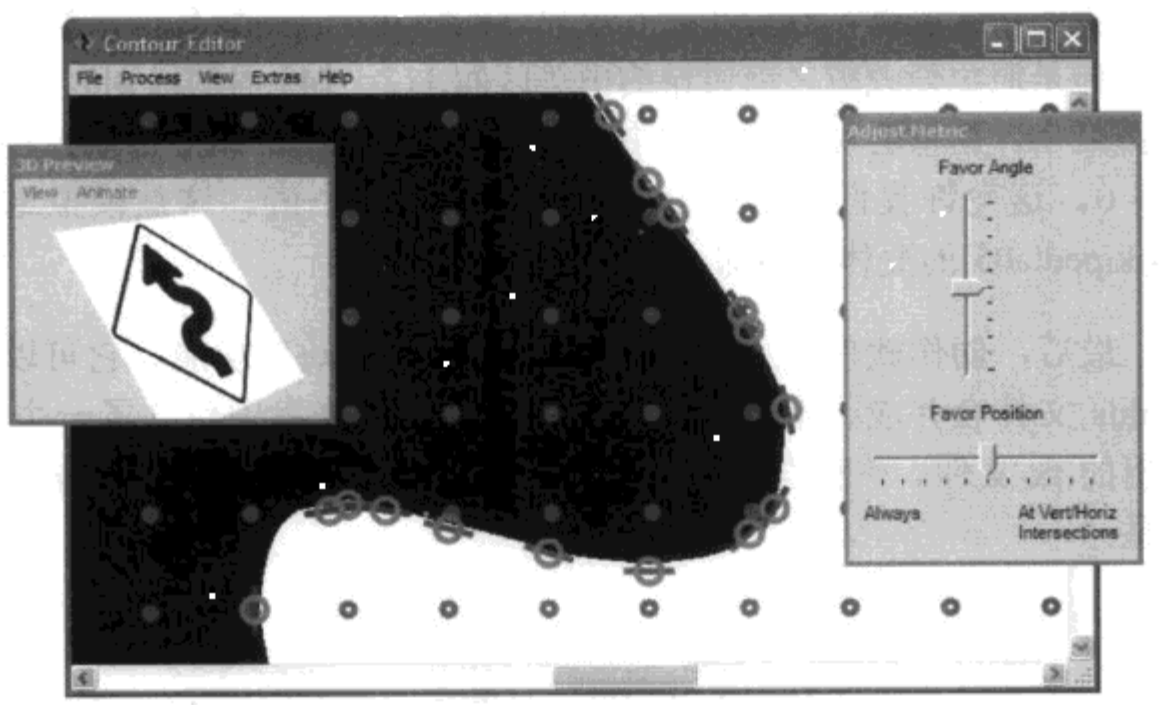


图 5.8.11　光盘中所带的纹理创作应用程序支持互动地调整度量，以及编辑位置与角度的约束条件。这些都是预先自动提取出来的

软件允许用户加载高分辨率的位图文件，选择结果的期望粗糙度（降采样系数），然后把结果保存成.dds 文件。用户可以交互式地将错误度量修改为偏向位置精度，或者角度精度，或者将基于角度的度量和几乎垂直于纹理像素网格边交点的位置错误度量合并。在这之上，用户可以使用鼠标选中相关的边，并且用方向键和 Page UP/Down 键来编辑位置/角度的约束条件。

当轮廓线与纹理像素网格边之间的某个交点碰巧几乎平行于这条边，或者如果若干个交点位于很临近的位置，那么约束条件的编辑就变得非常有用。为了降低这种情况下轮廓线的振荡，用户可以移动位置约束条件，旋转角度限制条件。优化程序以后台线程方式运行，用

Download at www.Pin5i.Com

以提供对编辑生成的实际轮廓线的互动式预览。目前版本的创作程序不允许直接编辑结果纹理的灰度值：由于边界曲线的有理性质，这通常导致复杂的变化。图 5.8.12 和彩图 12 是一个关于编辑产生的结果的示例。

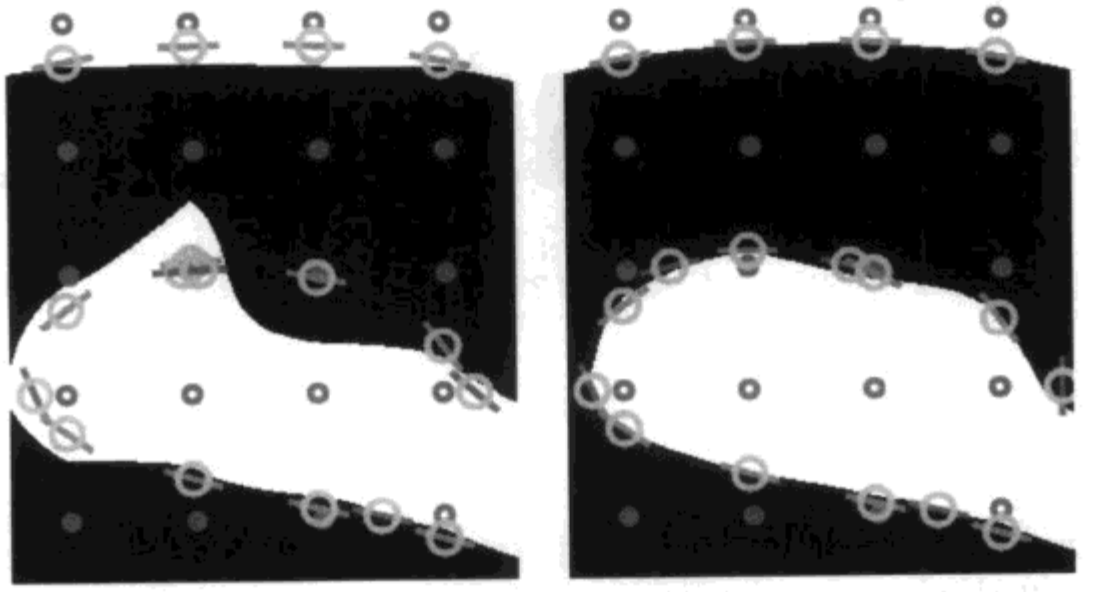


图 5.8.12 用户可能通过编辑位置和角度约束条件来解决临界情况，例如接近切向的相交会使优化器负担过重（左边：解决前，右边：解决后）。位置和角边的约束条件在图中以圆圈显示

优化过程通常与编辑并发运行，但也是不确定的。软件随机地在边界附近选择一个纹理像素，计算其周围的错误度量，然后在 256 个允许的级别中，对纹理像素的灰度级别进行±5 范围内的修改（但是避免修改成之前讨论的中等区间 112～143 内的级别）。通过这次修改，如果错误度量降低了，新值会保留在纹理中；否则，变化保留的概率很低，在遇到大量错误时候迅速变为 0。这允许我们在错误度量中避免本地最小值。这是模拟退火（simulated annealing）[Wikipedia05]的变体，但却不会降低温度。

ON THE CD

最后，创作软件提供了“Multicolor Combine”功能。它可以将两个或者 4 个.dds 文件合并成一个供多彩图使用的文件。光盘中包含了一个演示该种纹理使用的.fx 文件。一种合并若干彩色图层（如用黑色和红色合成无烟标志）的方法是：

```
OUT.color = float4(black*red, black, black, 1.0);
```

不过，这会产生令人讨厌的图像错误问题，见图 5.8.13 和彩图 13。黑色的香烟必须精确地终止于红色叉叉（strike-through）开始的位置。尽管无法保证这个要求，但是我们可以用红色来标记黑色。

```
OUT.color = float4(1.0-(1.0-black)*red, black*red, black*red, 1.0);
```

然后把完整的香烟形状放于黑色图层中，也就是在红色叉叉（strike-through）下面。

ON THE CD

对于像素着色器而言，处理四色图层与处理单色图层的开销是一样的，因为可以用 float4 替代 float 来并行地完成阈值划分计算（thresholding computation）。注意，对比度增强显示地在像素着色器中完成后，诸如 black*red 这样的表达式需要进行颜色钳制（clamping of the colors）。详情请参考光盘中的示例文件。

Download at www.Pin5i.Com

另外一个 Multicolor Combine 的创作程序加入了一个为阈值划分优化的纹理和一个标准纹理。阈值划分纹理可以在另一个纹理上作为蜡纸使用，例如创建路标（见图 5.8.14）。

图 5.8.13　缺乏技巧地使用多种颜色的图像（上方）会在几种颜色交汇的地方产生令人讨厌的错误（见放大部分）。将上述图像反向工程后放入单色图的堆栈，可以解决这个问题（下方：64×64 纹理像素）

图 5.8.14　阈值划分的纹理可以作为标准纹理的蜡纸使用（大小：128×128 个纹理像素）

5.8.4　总结与展望

交通标志和路标使用了视觉上精练的字体和形状。利用从 64×64 到 256×256 个纹理像素所组成的纹理，阈值划分技术可以很好地再现这些效果。处理过的纹理可以很好地进行缩小和放大，任意地放大而不会损失清晰度。由于像素着色器简短且仅请求了一个纹理，因此渲染速度几乎没有受到影响。利用这种技术，当前的显卡大概可以达到每秒填充 10 亿个像素的速率。

上述介绍的方法不能在单个纹理像素中处理两个轮廓线曲线。因此，像衬线字体、尖状体这些复杂的形状，就需要用超高分辨率的纹理来真实的表示。不过，如果用户适当地调整位置和角度约束设置，即使是纹理像素表示范围之下的微小特性有时也会再生出来。

如何包含光照及其他标准特效（如基于阈值划分纹理的深度表阴影（depthmap shadows））是非常简单的想法。但是其可以推广得更远，例如，三维纹理可以用来创建动画轮廓线。另一种扩展的方法就是将这种想法应用到全彩纹理上（full-color texture）。可以尝试将一幅仅有温和颜色梯度（soft-color gradients）的图像与另外一幅含有利边（sharp edges）的图形合并来表示这种纹理。前者可以直接以低分辨率存储，后者可以转换后使用阈值划分。

5.8.5　参考文献

[Loop05] Loop, Charles and Jim Blinn, "Resolution Independent Curve Rendering Using Programmable Graphics Hardware." *ACM Transactions on Graphics,* Vol. 4, No. 3 (SIGGRAPH 2005): pp. 1000-1009.

[Loviscach04] Loviscach, Jörn, "Paving Roads with Pixel Shaders." WSCG 2004 Full Papers: pp. 39-46.

[Loviscach05] Loviscach, Jörn, "Efficient Magnification of Bi-Level Textures." SIGGRAPH 2005 Sketch.

Download at www.Pin5i.Com

[Ramanarayanan04] Ramanarayanan, Ganesh, et al., "Feature-Based Textures." Eurographics Workshop on Rendering 2004: pp. 265-274.

[Sen04] Pradeep, Sen, "Silhouette Maps for Improved Texture Magnification." ACM SIGGRAPH/Eurographics Conference on Graphics Hardware 2004: pp. 65-73.

[Tarini05] Tarini, Marco and Paolo Cignoni, "Pinchmaps: Textures with Customizable Discontinuities." *Computer Graphics Forum,* Vol. 24, No. 3 (Eurographics 2005), pp. 557-568.

[Tumblin04] Tumblin, Jack and Prason Choudhury, "Bixels: Picture Samples with Sharp Embedded Boundaries." Eurographics Workshop on Rendering 2004: pp. 186–196.

[Wikipedia05] Wikipedia contributors, "Simulated Annealing." Wikipedia: The Free Encyclopedia. Accessed August 12, 2005. Available online at *http://en.wikipedia. org/wiki/Simulated _annealing*.

Download at www.Pin5i.Com

5.9 天空渲染在游戏中的实际运用

Aurelio Reis，Raven 软件
AurelioReis@gmail.com

随着越来越多的游戏开始重视广阔的户外环境，天空在场景中的作用也变得日益重要。如果你知道天空渲染技术的计算开销是多么的昂贵，你就会意识到高效地进行天空渲染的重要性。本节将逐一介绍各种通用的、特殊的优化技术，你可以将它们应用到各种天空程序中。即使是高分辨率、全屏视图下的天空，这些优化技术都能保证很好的帧速。

5.9.1 所需与所得相悖

关于动态天空渲染技术的基础知识，建议阅读一下由[Nishita00]、[Preetham99]或[Jensen01]所完成的相关研究。大多数的有效方法都相当复杂，因为它们试图仿真的是真实世界的物理现象。用大量的大气参数来模拟天空一直都是非常困难的，直到最近才出现了针对离线渲染器保留的方法。但随着我们逐渐过渡到完全加速的图形硬件环境，这些技术变得越来越可用。

直到几年前，使用预渲染的天空（类似[Terragen]中的离线渲染器）对天空体（sky boxes）进行纹理化，是在互动的帧速下表示天空最普遍的方法。由于高级飞行模拟器的限制，动态天空超出了大多数游戏的掌控范围之外。不过，随着 MMOG（Massively Multiplayer Online Games，大型多人在线游戏）和侠盗车手 3（Grand Theft Auto 3）这样的游戏的出现，昼夜交替是一个不得不考虑的问题，开发者必须投入到真实自然场景的研究中去。视觉质量（visual quality）和系统性能之间的权衡一直都是一个巨大的挑战，显示质量（presentation quality）以前也碰到过类似的问题。

现代的图形硬件所支持的技术与游戏中应该采用的技术之间有一个鲜明的对比。在大多数的天空渲染演示中，出现的都是关于天空技术本身的介绍。但对于开发者而言，这些技术并不是像所见到的那么有用。[ATI Atmosphere Demo]演示的大气散射虽然给人留下了相当深刻的印象，但并没有呈现出一个完全互动的场景。由于大量地使用着色器指令，互动场景的额外开销肯定非常昂贵。

采用这些技术的大多数演示程序都使用了一个稀疏世界（sparse world），并没有显示出真实游戏中的那种细节程度的场景。这类场景是由

Download at www.Pin5i.Com

其他交互式三维对象所组成的环境，这些对象是从 CPU/GPU 中获得相关处理能力的。在一个真实游戏中，你通常会遇到贴有皮肤（skinned）的三维人物、帧缓冲器的多种混合操作、巨大数量的三角形，以及逐像素的超负荷使用。

本节并没有介绍一种新的、更好的天空建模（sky-modeling）技术，而是介绍了许多让现有技术更好地适用于游戏的方法。关键要在技术和理论之间找到一条路，然后深入钻研影响真实游戏实现的瓶颈是什么，如何才能克服它们。本节介绍的优化方法完全可以运用到实际的商业产品中。虽然这里所探讨的主要主题是高效渲染天空色彩与云朵，但仍简单地涉及了处理恒星所发出的变化光源的高效光照技术。总之，天空技术除了带来视觉上的美感，还应当可以高效地进行渲染。这为渲染引擎中插入一个游戏留下了足够的时钟周期。

5.9.2 天空的组成

让我们首先介绍一下动态天空的组成部分，然后回顾一下将要使用的术语。首先是天空的色彩，通常用天幕（sky dome）来表示，其实就是一个半球状的三维模型。天幕的表面会逐顶点或者逐像素将计算分解成离散的点。

决定某点的天空颜色的有效方法可以是统计的（即使用观察和再生的数据）或者分析的（即基于自然属性）。例如，一个分析模型，通常需要计算出某个特定点的米尔及瑞利散射（Mie-Rayleigh scattering），以及臭氧（ozone）和发光（radiant）。不过，应当注意的是，大多数的实现都会把等式简化为由几个最具重要的因素所构成，通常是米尔及瑞利散射。大气条件的物理性质有非常好的文档说明，前面小节中提到的图形化表示方法并不是什么稀奇的事情。我们将会使用本文中的天空实现所涉及的少量基本源代码，这些代码都得到了其他作者的授权。

对于天空颜色渲染，我们将会使用[Spoerl04]介绍的优化方法，它本质上是将[Nishita00]中介绍的方程简化为仅仅需要考虑简化的 sky-color 算法。在这种算法中，只有米尔及瑞利散射项会被计算（简化的形式），光学深度本质上是被一个基于当前采样点高度的缩放因子所替代。[Nielsen03]中也推荐了这种方法，但是他很好的补充了现有的这种近似方法，使得可以动态地改变光学深度来模拟变化的高度情况下，所观察到的大气的变暗。为了找到相对于观察者位置的天空中太阳的位置，我们将会使用[Jensen01]中所概括的算法。为了得到一天中特定时间的太阳的颜色，我们将会使用[Preetham99]中概括的方程。

云雾是大气中非常重要的部分，但是我们不会在本文中介绍。高质量的云雾渲染方面进行过大量的研究；但是，大部分建议的算法仅仅因为速度不够快而作为实时使用。[Dube05]中介绍一个令人印象深刻的云雾渲染实现，其实际上是一个像素着色器中的光线跟踪器，用来计算在不同的深度情况下，被云雾吸收的光线的数量。唯一的限制就是我们不能定义显示的阳光方向。[Reis05]介绍了一种基于[McGuire05]的近似实现，它考虑了光线方向。一种高效的解决方案就是使用传统的天空滚动（scrolling-sky）的平面技巧：纹理映射到平面上，滚动平面的纹理坐标来模拟移动。该纹理同样可以投影到地面上来模拟云雾的阴影。

最后，太阳和月亮是整体不可或缺的部分。由于我们仅仅讨论了天幕（sky dome）法，因此，得到移动的太阳和月亮的最容易方法是，通过一个投影纹理矩阵将太阳或者月亮的图像投影到几何中。矩阵的计算需要相当多的时间，每天对结果进行若干次的缓存占据了相当

Download at www.Pin5i.com

多的内存(依赖于采样频率)。一种可能的优化方法就是将太阳和月亮绘制到立方体贴图(cube map)，然后由游戏来加载，根据一天中的当前时间进行旋转。因为着色器中旋转立方体贴图需要相当多的指令，因此绘制太阳和月亮的高效方法很难找到。

5.9.3　瓶颈

渲染天空时，会遇到许多性能相关的问题，这取决于你所使用的技术。如果你的方法需要完全或部分在 CPU 上完成，你将会有大量的 CPU 相关的瓶颈问题需要解决。另一方面，如果你完全利用了 3D 硬件加速的优势，你也会遇到一组完全不同的性能瓶颈问题。

下一代游戏广泛地使用了像素着色器，它们中的大部分都很有可能变成 pixel-bound（像素限制)。场景中的顶点数量是一个变量，依赖于场景中的三角形是如何构造的（如，通过 triangle lists 或 strips)；虽然屏幕上的像素数量是和屏幕的分辨率成比例的，但是由于透支，像素着色器为屏幕上的每个物理像素所需要执行的次数也是一个变量。当网格发送到显卡开始绘制的时候，它的顶点会投影到屏幕上，且必须填充。即使显卡试图相对智能地丢弃那些不可见的像素，但是显卡必须单独考虑每个像素的事实还是会导致性能代价（perfromance penalty)。虽然像素可以不绘制，但是它还是会影响性能；一些像素（由于混合）总是被考虑的，因此我们特别当心地来优化它们的着色器。Early depth rejection（通过 pre-z pass)，通过纹理查找（非常迅速）来减少指令数目，这些通常都是减少着色器开销和复杂性的方法。

让我们天空的实现在哪些地方会受到限制。在天空体的早期，最需要关心的问题之一就是使用何种分辨率。假设使用一个纹理分辨率为 512×512 的立方体贴图。依赖于天空中那部分的大小和到那部分的距离，以及所使用的纹理像素填充方法，你可以潜在地绘制大于 512 个像素，或者小于 512 个像素。这会导致填充率问题，因为在高分辨率设置的条件下（更多的像素需要填充)，大量的纹理样品会潜在地耗尽你的性能。

权衡基于顶点/像素的天空的利弊

在目前的情形下，我们可以逐像素地进行大量的计算，这对性能的损伤是巨大的。明显的解决方案是逐像素地转移计算，然后仅取像素着色器抽样命中的纹理。不过，这种方案有其独特的一组问题。假设我们至少使用双线性过滤，颜色值会插值到其所有三角形之间的一个三角形的整个表面；虽然这有助于模拟出一个平滑区间的值，但因为低多边形网格的低采样率，还是会导致粗糙的、人造似的结果。这明显会丧失在整个天空拥有良好颜色梯度的机会。

那么，导致这种结果的原因是什么呢？嗯，我们可以勉为其难地接受高质量所带来的潜在开销，并且附带一个低分辨率却可以逐像素完成我们工作的网格；或者可以使用中等到高等多边形的网格来逐顶点地完成计算。最终由你来根据主要瓶颈的发生地（顶点或者像素）来做出选择。不过，当我们讨论广义方程（generalized equations）时，为了简便起见，我们假设将要讨论的是顶点着色器版本。两种方法在实际中都是有用的，因为它们会根据不同的硬件配置进行有效地缩放。向用户呈现各种不同的选项太过于复杂，这并不是我们所期望的。为此，我们提供了一个通用的“sky detail”滑动条（slider）作为图形选项，它可以悄悄地在各种有效方法之间进行转换，默认值是根据硬件检测的结果设置的。

Download at www.Pin5i.Com

需要考虑的另一件事情就是决定如何表示预先计算的常量和为着色器进行查表。光盘中附带的本文 demo，展示了如何预先计算着色器所需要的伪光学（pseudo-optical）深度，用以模拟地平线附近的大气的强烈色彩，以及随着海拔高度的增高而带来的天空暗化（为了实现的简单起见，因此放弃了计算伪光学色深变化所带来的海拔变化。[Nielsen03]中描述了相关的一种有效模拟方法）。

圆顶的顶点缓冲器中的顶点格式必须能够接受光学深度作为额外的纹理坐标统一参数。以下列出了圆顶的顶点声明：

```
const D3DVERTEXELEMENT9 g_DomeVertexDecl[] =
{
    { 0, 0, D3DDECLTYPE_FLOAT3, D3DDECLMETHOD_DEFAULT,
    D3DDECLUSAGE_POSITION, 0 },
    { 0, 12, D3DDECLTYPE_FLOAT3, D3DDECLMETHOD_DEFAULT,
    D3DDECLUSAGE_NORMAL, 0 },
    { 0, 24, D3DDECLTYPE_FLOAT3, D3DDECLMETHOD_DEFAULT,
    D3DDECLUSAGE_TANGENT, 0 },
    { 0, 36, D3DDECLTYPE_FLOAT3, D3DDECLMETHOD_DEFAULT,
    D3DDECLUSAGE_BINORMAL, 0 },
    { 0, 48, D3DDECLTYPE_FLOAT2, D3DDECLMETHOD_DEFAULT,
    D3DDECLUSAGE_TEXCOORD, 0 },
    { 0, 56, D3DDECLTYPE_FLOAT1, D3DDECLMETHOD_DEFAULT,
    D3DDECLUSAGE_TEXCOORD, 1 },
    D3DDECL_END()
};
```

虽然通过顶点流（vertex stream）来定义额外的顶点元素可能是一种为顶点格式添加额外元素的较好方法，但我们还是选择仅仅使用一个不同的利用指数（usage index）。下面展示了如何为模型顶点缓冲器（model vertex buffers）填充数值：

```
// 预计算圆屋顶模型的任意常量
const float fInv = 1.0f / 5.0f;
for ( int i = 0; i < 3; i++ )
{
    DomeVertex *pVertices;
    g_DomeModels[ i ].LockVertices( (void **) &pVertices );

    int iNumVerts = g_DomeModels[ i ].GetNumVerts();
    for ( int j = 0; j < iNumVerts; j++, ++pVertices )
    {
        pVertices->fOpticalDepth
            = 1.7f - powf( fabs( pVertices->vNormal.y ), fInv );
    }

    g_DomeModels[ i ].UnlockVertices();
}
```

着色器中，顶点的输入结构中包含了光学深度作为输入参数：

```
float OpticalDepth : TEXCOORD1;
```

Download at www.Pin5i.Com

最后，光学深度会在顶点着色器中直接读取；然后要么直接使用，要么作为纹理坐标发送到像素着色器进行逐像素插值。这提出了一个很好的观点。光学深度值最初在顶点程序中直接使用顶点位置计算而得。但是将顶点位置传到像素着色器中进行计算，通常得到的是更为平滑的数值。问题的答案很明显。只要从顶点程序中通过纹理坐标向像素着色器传递一个数值，图形硬件就会使用该值进行插值。因此，当传递该数值到像素着色器进行逐像素光照时，需要重新标准化光线和视点向量。只要处理任何在逐顶点的基本要素中所获得的数据，我们都需要考虑这个因素。使用这种优化技术，将会产生一个低多边形的网格（小于等于 100 个多边形）。这种网格所提供的数据范围是不充分的，因此无法有效地使用顶点着色器技术来表示天空效果。

虽然我们给出了许多提高天空实现性能的方法，但是正如前面陈述的那样，正在完成的最密集的操作就是每帧需要完成的那些（逐像素或者逐顶点）。我们最需要做的就是找出某种缓存特定天幕状态的方法，这样的话，我们至少还能使用几帧。其实，问题就在于我们的场景是振动（vibrant）且交互式的，因此视图会经常变化。最直接的选择就是将三维环境直接捕捉到某个纹理中，然后将其投影到稍微有点真实的场景中。我们该怎么做呢？

5.9.4　立方体天空贴图概要

立方体天空贴图（cubic sky map）本质上就是一个表示天空的立方体贴图，准时地在特定点缓存下来的。通过将天幕渲染到立方体贴图的 6 个面上，我们其实已经能够捕捉它的当前状态，且在许多帧中使用。虽然我们仍然导致了纹理采样时的填充率开销问题，但是这已经远比每帧（逐像素或逐顶点）过去完成所有计算所需的开销低得多。在天幕（dome）渲染到立方体贴图上后，它会映射成一个非常简单的盒子。该盒子用以下 pass-through 式的顶点着色器进行渲染：

```
VS_OUTPUT_CUBEPASSTHROUGH RenderCubeMapPassthroughVP(VS_INPUT IN)
{
    VS_OUTPUT_CUBEPASSTHROUGH OUT;

    // 将位置从对象空间转换到齐次投影空间
    OUT.Position = mul( IN.Position, g_mWorldViewProjection );

    // 通过视点向量
    OUT.TexCoord0 = g_vLocalEyePos - IN.Position;

    return OUT;
}
```

下面是像素着色器：

```
PS_OUTPUT RenderCubeMapPassthruPS( VS_OUTPUT_CUBEPASSTHROUGH IN )
{
    PS_OUTPUT OUT;
    // 使用给定向量直接进行采样(通过立方体贴图隐式地进行标准化)
```

Download at www.Pin5i.com

```
    OUT.Color = texCUBE( CubeMapSampler, IN.TexCoord0 );

    return OUT;
}
```

如你所见，我们把这称为“pass-through”立方体贴图着色器，因为它所做的就是计算视点向量光线（即从观察者的视点位置到顶点位置的向量），然后将其作为插值向量（直接在那个方向上的立方体贴图中进行采样）传递。由于立方体贴图本来就会标准化向量，因此没有必要重新标准化。

立方体贴图所能达到的最大分辨率——512 的高度是察觉不出有何变化的；但是，256～128 的高度却是可以接受的，而且根据用户的硬件规格进行缩放是完全值得的。在 64 左右时，沿着天空色彩梯度方向，你会注意到严重的带状干扰（banding and artifacts），因此质量会严重地下降。这往往发生在天幕的边沿地带，但是无论如何玩家应该从未看到过它们，因此是毫无关系的。任何高过 512 的分辨率都会招致严重的开销，因此我们建议采用 256 左右的分辨率。当然如果你能得到足够的“马力”，尽管尝试 512。

现在，虽然缓存结果是一种很好的优化技术，但如果每帧都需要渲染立方体贴图，这种技术就几乎毫无用处。开销的原因就在于我们要为立方体每个面渲染圆顶，总共 6 次（实际上是 5 次，如果你够聪明的话，就应当知道没有必要考虑靠近地表的面）。那么，我们如何才能高效地保持立方体贴图是最新的呢？我们零星地进行渲染。根据当前时间（time-of-day）实现所采用的时间比例类别（假设你正在使用其擅长的动态天空），你将会期望立方体贴图能够保持一定的更新率（一旦时间变化足以察觉时，立刻刷新）。

5.9.5 特殊时间

一个合理的时间比例大约是现实世界中的 1 秒钟相当于游戏世界中的 1 分钟。因此，现实世界的 1 分钟相当于游戏世界的 1 个小时，现实世界中的 24 分钟就相当于游戏世界中的 1 天。按照这种速度，游戏世界中大概每隔 30 分钟的时间变化是可以察觉的。为了能够更清楚地区分出黄昏或者黎明，我们假设游戏世界中每隔 15 分钟是比较有把握的做法。在这种速率下，你需要每隔一刻钟的游戏世界时间就更新立方体贴图，也就是每隔 15 秒钟的现实世界时间。老实说，这将花费很多时间，但是我们遇到了另外一个问题。不考虑实际场景本身的视图，更新立方体贴图的 6 个面，会对帧速率造成很大的影响。我们所能做的就是将开销展开，具体做法就是用许多帧来更新每个立方体表面。

目前在固定的时间内，我们可以连续或者零星地进行渲染。前一种方法仅仅包括一帧帧地渲染每个立方体视图。后一种方法（首选）实际上是计算出 15 秒时间内每次渲染的间隔，便于我们展开计算。假设我们仅仅更新 5 个面（应该如此），那么就是每隔 3 秒钟进行一次立方体面更新。这无疑很好地展开了开销，但是仍然还有其他问题。虽然我们并不打算在渲染出当前时间后更新时间，但是我们在不同时期把天空的不同状态渲染到每个独立面上。如前面所述，如果天空中可察觉的变化很多时，这对于一天中的每个时期都很明显。不幸的是，唯一的选择是使用类别交换链（a swap chain of sorts）：一个立方体贴图进行渲染时，另外一个则被绘制到天空体上，下次更新时则交换工作类别。这样做可以确保我们不会从更新过的

Download at www.Pin5i.Com

立方体面上得到怪异的颜色变化。

5.9.6　演示程序的黎明

本小节所包含的演示程序和附带光盘中的源代码展示了迄今为止所讨论的许多方法和技术。代码的文档化程度很高，在整个代码中进行探索应该相对来说比较容易。使用法线贴图化的地形使得地表面稍微有趣些，尽管它本不该是演示程序的焦点。

地形高度图由 World Machine Baisc 生成；低、高多边形网格和漫射纹理由 T2 Terrian Texture Generator 生成；最后，法线贴图由 nVIDIA Melody 生成。我们使用 3 个天幕来表示由逐顶点或者逐像素技术所带来的潜在锯齿干扰（aliasing artifacts）。如果你实际选择使用严格基于顶点的实现，所生成的圆顶中边附近的顶点数目指数倍的大于顶部附近的顶点数目。这么做的原因是，尽管顶部附近的颜色变化不多，但是黄昏和黎明期间的边附近有很鲜明的色彩反差；因此如果能有更多的顶点来模拟这些数值，就能很好地描绘这种现象。这种方法同样适用于之前介绍的伪光学深度。

如果对光盘中的代码特别兴趣的话，你可能希望看看类 CSceneView 是如何工作的，主视图是如何生成的，立方体贴图渲染器子视图又是如何附加到主视图上的。场景视图系统主要包含一个主视图和任意数目的子视图，这些子视图可以以任意的顺序或层次附加到主视图上（有相关的限制）。每个视图都包含了许多用以描述如何进行视图渲染的参数。视图目标类型（描述了视图将要绘制到的地方）包含了视图转换与投影矩阵这样的数值。这允许将依赖性的子视图或者主视图绘制到任意数目的纹理中，用以实现水平面反射（planar water reflections）、立方体贴图反射（cube map refelections）、（正）GUI 视图，甚至是场景中的后处理特效。

随着时间的变化，太阳、月亮的位置和颜色都会发生变化。在我们的场景中，我们仅仅考虑来自天空物体的全局照明。基于阴影化的像素虽然可以简单地近似出环境因素，但是地形的光照还是基本的逐像素补色渲染（phong shading）。取得我们已经生成的立方体天空贴图，然后将其作为我们环境的色彩向导，这是一种非常有趣的技术。例如，黄昏时的天空有各种各样的颜色，然而光照不到的地方则只披上了上方天空一角映射过来的浅淡的着色，而不是周围环境的色彩。通过旋转立方体贴图，可以模拟出更适宜的光线漫射。这样做的代价非常昂贵，因为它需要多点的采样，然后运行时对平均结果求和；或者在 CPU 中（非常昂贵的代价）进行下载和旋转立方体贴图。

在世界空间中单独采样立方体贴图并不是想象中的那么容易，因此这仅仅是一项高级技术的建议（未必适用于游戏）。使用启发式的方法，可以基于时间来修改周围环境的光线值。这有助于在一天内标准化光照，这样的话，在太阳的最高点，场景不会太亮；而晚上的时候，场景则不会太暗。同样的，基于低多边形（low-polygon）模型进行法线贴图时，如果使用某种常量光线来计算伪凹凸贴图（bump-mapped）的光照将有助于保持增强的细

Download at www.Pin5i.Com

程度。

对于霾/雾（haze/fog），我们实现了一个简单的消光模型（extinction model），它用来计算场景中的所有对象的数值。我们原本打算实现[Sun05]中介绍的散射模型（scattering model），但最后还是否决了，因为霾的实现代价本身就很昂贵。霾的密度值等其他图形选项可以容易地在 GUI 上修改。不过，我们还是非常希望读者能够深入研究一下在保持合理性能的前提下，提高空间透视（aerial perspective）视觉真实性的方法。

5.9.7 进入地平线之下

关于演示程序的最后一个话题就是高动态范围光照渲染（high dynamic range rendering）。真实世界有一个惊人的动态范围：根据[Reinhard02]，从阴影到光亮处大概有 10 级的绝对范围和 4 级的动态范围。在户外环境中模拟出这种范围是非常重要的，特别是你想将后处理特效（如 glare）应用到场景中的光亮部分时（如白天的太阳和夜晚的月亮，它们分别是那个时候天空中最亮的部分）。由于利用浮点缓冲器（或浮点立方体贴图）进行色调贴图（tone-mapping）是很昂贵的方法，因此我们发出了相关警告。在场景上使用标准的发光或者规定亮度范围的闪耀（存储于 alpha 通道）将会破坏重要的细节部分；但至少它最后可以提供更好的性能。确保选择一个能够表示场景中所有对象的标准化范围。尽管整个白天最亮的物体是太阳，但是在室内（going inside）会彻底地改变数值映射的方式，因此必须特别小心使用高亮度光线值。

尽管演示程序非常接近游戏环境，但是如果不实际做一个游戏，是无法表示出所有的游戏元素的，而游戏开发过程大强度是众所周知的。你可以自己实践一下，但是需要注意的是，往移动、振动的环境中加入栩栩活力对性能的要求非常高。如果我们继续努力实现让人更加沉浸、真实的环境，我们必须权衡图形流水线和技术来充分利用硬件能力。虽然我们说明了高级对象是如何影响性能的，但还有许多未处理的小型性能杀手。采取保守一些的做法从长远看来是有好处的，因此优化天空实现理所当然比任何现有的附加特征都重要。

5.9.8 总结

本节中我们回顾了许多通用的实际优化技术和提高天空实现性能的特殊技巧，并且在所提供的天空实现中展示了这些技术。我们还研究了图形流水线的某些部分和渲染技术，解释了性能剧烈下降的原因以及性能瓶颈的来源。

我们希望你能喜欢这节的内容，从中获得你的天空实现所需要的内容。虽然在视觉美之上我们强调性能，但是更为真实的动态天空是游戏的崇高目标，我们相信只要特别小心、聪明地在现代系统上对性能进行最佳优化，这些完全是可行的。谢谢你，祝你的天空实现顺利！

Download at www.Pin5i.Com

5.9.9　参考文献

[ATI Atmosphere demo] Available online at *http://www.ati.com/developer/demos/ r8000.html.*

[Dube05] Dube, Jean-Francois, "Realistic Cloud Rendering on Modern GPUs." *Game Programming Gems 5,* Kim Pallister, Ed., Charles River Media, 2005. Intel Vtune™. Available online at *http://www.intel.com/cd/software/products/asmo-na/ eng/vtune/index.htm.*

[Jensen01] Jensen, Henrik Wann., et al., "A Physically-Based Nightsky Model." Available online at *http://graphics.stanford.edu/~henrik/papers/nightsky/.*

[McGuire05] McGuire, Morgan and Max McGuire, "Steep Parallax Mapping." Available online at *http://graphics.cs.brown.edu/games/SteepParallax/.*

[Nielsen03] Nielsen, Ralf Stokholm, "Real Time Rendering of Atmospheric Scattering Effects for Flight Simulators." Available online at *http://www2.imm.dtu.dk/pubdb/views/publication_details.php?id=2554.*

[Nishita00] Nishita, Tomoyuki., et al. "Display Method of the Sky Color Taking into Account Multiple Scattering." Available online at *http://nis-lab.is.s.u-tokyo.ac.jp/~nis/abs_cgi.html.*

[Preetham99] Preetham, A. J., Peter Shirley, and Brian Smits, "Practical Analytic Model for Daylight." Available online at *http://www.cs.utah.edu/vissim/papers/sunsky/.*

[Reinhard02] Reinhard, Erik., et al., "Photographic Tone Reproduction for Digital Images." Available online at *http://www.cs.utah.edu/~reinhard/cdrom/.*

[Reis05] Reis, Aurelio, Clouds-rendering demo. Available online at *http://www.codefortress.com.*

[Spoerl04] Spoerl, Marco and Kurt Perlzer, "Advanced Sky Dome Rendering." *ShaderX*^{2}, Wolfgang F. Engel, Ed. Wordware Publishing, 2004.

[Sun05] Sun, Bo., et al., "A Practical Analytic Single Scattering Model for Real Time Rendering." Proceedings of SIGGRAPH 2005. Available online at *http:// www1.cs.columbia.edu/~bosun/sig05.htm.*

[Terragen] Terragen™, Scenery Generator. Available online at *http://www.planetside.co.uk/terragen/.*

Download at www.Pin5i.Com

5.10 基于 OpenGL 帧缓冲区对象的高动态范围渲染

Allen Sherrod，终极游戏开发公司
ProgrammingAce@UltimateGameProgramming.com

最近几年，游戏开发中的许多特效都可以通过后处理实时生成。随着图形学的日益复杂，其实现代码和相关硬件也同样变得无比复杂。由于渲染场景中加入的特效越来越多，从而导致更多离屏的渲染与处理的发生。本节将会介绍一种在 OpenGL 程序中进行离屏渲染的崭新、高效且快速的方法：最近一次新增的帧缓冲区对象。本书光盘中的程序演示了如何使用 OpenGL 帧缓冲区对象（FBO）实现高动态后处理特效。

5.10.1 帧缓冲区对象简介

OpenGL 提供了许多进行离屏曲面渲染的方法。每种方法都有各自的优势与弊端，我们在选择合适方法的时候，必须进行仔细的权衡，其中最主要的因素就是时间与空间开销。

首先介绍第一种渲染方法：glReadPixels()和 glTexImage*D()。这种方法的主要缺陷是速度慢，用它实现的游戏引擎也非常低效。因此，这种方法是最不推荐使用的。第二种方法是 glCopyTexSubImage2D()，它比 glReadPixels()要快得多，但是它的特效处理依旧没有达到特定的速率。第三种方法是一种叫做像素缓冲区（pixel-buffers，简称 p-buffers）的技术。像素缓冲区（p-buffers）是一种非常好的方法，速度上比前面这些方法都要快。另外，通过使用一种不同于原始窗口的像素格式，我们可以很方便地创建不同于原始屏幕分辨率的多维离屏曲面，而不用变戏法似地处理各种 OpenGL 状态的切换。尽管 p-buffers 是一种很好的解决方案，但与之而来的问题也是不容忽视的：巨大的内存消耗、昂贵的上下文切换、艰难的代码管理，以及移植到其他操作系统的复杂性。

尽管 p-buffers 有很多问题，但它也并不是一无是处：通过把设置与使用 p-buffer 的代码进行抽象，便可以得到一个可重用类。这种做法非常符合面向对象技术的开发人员，因为他们只需要将每个所希望支持的平台上的 p-buffer 的创建、维护以及销毁的代码封装成一个类。这种做的问题在于我们要花费额外的工作。

Download at www.Pin5i.Com

我们如何才能高效地解决上述难题？p-buffers 本身已经足够快了，但是我们无法忽视它所导致的棘手问题。值得庆幸的是，OpenGL 的帧缓冲区对象（Frame Buffer Object，简称 FBO）技术可以解决上述难题。FBO 技术有很多优秀的地方，它比前面讨论的方法更易为使用。例如：

- 支持以纹理方式直接读取渲染结果；
- 简单而快速的上下文切换；
- 对象之间的共享纹理和缓冲区占据更少的内存；
- 通过驱动程序的支持，可以工作在任何硬件环境下；
- 设置与清除更为方便；
- 无须额外的步骤来支持不同平台间的代码移植。

OpenGL FBO 使用 EXT_framebuffer_object 扩展。该扩展引入了两种新的对象:帧缓冲区和渲染缓冲区。帧缓冲区是一组缓冲区（例如，颜色、色深、模板），而渲染缓冲区则是一组存储渲染效果的简单的 2D 图片。一旦被绑定于某个帧缓冲区，它所附带的图片将会成为所有分段操作的来源和目的地，除非你解除绑定或者绑定另外一个帧缓冲区。帧缓冲区的建立不需要渲染缓冲区的支持，反之则不行。必须牢记这一点。

众所周知，OpenGL 提供了许多进行离屏渲染（Off-Screen Rendering）的手段。但是综合考虑移植性、易用性和性能的话，FBO 无疑是最佳的选择。nVIDIA 的用户需要安装 77.72 版本以上的驱动，而 ATI 的用户则需要等到相关产品驱动支持的发布（由于 FBO 是很新的技术，ATI 用户等待的时间应当不会太长）。确保将你硬件的驱动升级到最新版本。

5.10.2　设置帧缓冲区对象

为了在程序中使用帧缓冲区对象（FBO），我们必须检查以确保系统支持 EXT_framebuffer_object 以及拥有最新版本的“glext.h”。这样做很有必要，因为当涉及到 Windows 操作系统的时候，我们需要得到某些必要函数的函数指针。一旦所有的必备条件都到位后，我们就可以开始编写我们的程序了。

帧缓冲区对象的创建、使用和销毁，与纹理和着色器对象非常相似，这需要程序清单 5.10.1 中的 OpenGL 函数的支持。其中 *n* 代表需要创建或者销毁的对象的个数；framebuffer(s)是一个无符号整形变量（数组），存储了对象的 ID；target 是 GL_FRAMEBUFFER_EXT。

程序清单 5.10.1　创建、使用和删除帧缓冲区

```
void GenFramebuffersEXT(sizei n, uint *framebuffers);
void DeleteFramebuffersEXT(sizei n, const uint *framebuffers);
void BindFramebufferEXT(enum target, uint framebuffer);
bool IsFramebufferEXT(uint framebuffer);
```

函数 GenFramebuffersEXT 和 DeleteFramebuffersEXT 的运作方式与 glGenTextures 和 glDeleteTextures 非常相似，用户可以用来创建/销毁（FBO）。使用 BindFramebufferEXT 绑定某个 FBO 后，所有的操作结果将会体现在该对象所附带的图像上。如果传给该函数的

Download at www.Pin5i.Com

framebuffer 的值为 0/NULL，OpenGL 将会使用系统默认的缓冲区，并且停止对当前的帧缓冲区对象进行渲染。IsFramebufferEXT 可以用来检查某个帧缓冲区对象是否有效。

通过调用函数 FramebufferTexture2DEXT（enum target，enum attachment，enum textarget，uint texture，int level），可以将纹理附加到某个帧缓冲区对象上。形参说明如下：

- target 取 GL_FRAMEBUFFER_EXT，
- attachment 取 GL_COLOR_ATTACHMENT0_EXT...GL_COLOR_ATTACHMENTn_EXT，GL_DEPTH-ATTACHMENT_EXT，或 GL-_STENCIL_ATTACHMENT_EXT，
- textarget 取 GL_TEXTURE_2D，GL_TEXTURE_RECTANGLE，GL_TEXTURE_CUBE_MAP_POSITIVE_X，GL_CUBE_MAP_NEG-ATIVE_X 等。
- texture 是通过调用 glGenTextures()创建的纹理对象。
- level 代表当前附加的纹理的 mip-map 级别。

通过调用函数 enum Check FramebufferStatusEXT（enum target），我们可以检查 FBO 的状态。本函数只能在创建 FBO 后调用。CheckFramebuffer-StatusEXT 将会返回如下取值：

- GL_FRAMEBUFFER_COMPLETE
- GL_FRAMEBUFFER_INCOMPLETE_ATTACHMENT
- GL_FRAMEBUFFER_INCOMPLETE_MISSING_ATTACHMENT
- GL_FRAMEBUFFER_INCOMPLETE_DUPLICATE_ATTACHMENT
- GL_FRAMEBUFFER_INCOMPLETE_DIMENSIONS_EXT
- GL_FRAMEBUFFER_INCOMPLETE_FORMATS_EXT
- GL_FRAMEBUFFER_INCOMPLETE_DRAW_BUFFER_EXT
- GL_FRAMEBUFFER_INCOMPLETE_READ_BUFFER_EXT
- GL_FRAMEBUFFER_UNSUPPORTED
- GL_FRAMEBUFFER_STATUS_ERROR

接下来介绍渲染缓冲区 API，它与前面介绍的帧缓冲区 API 很相似。详情请见程序清单 5.10.2 形参说明如下：

- n 代表希望创建/销毁的对象的个数。
- renderbuffers 是一个无符号整形的变量/数组。
- target 取值必须为 GL_RENDERBUFFER_EXT。
- internalformat 可以是任何法线纹理格式(例如，GL_RGB，GL_RGBA，等等)。
- width 和 height 代表缓冲区的尺寸。
- pname 取 GL_RENDERBUFFER_WIDTH_EXT，GL_RENDERBUFFER_HEIGHT_EXT，或 GL_RENDERBUFFER_INTERNAL_FORM-AT_EXT。
- params 的取值取决于 pname。如果 pname 取 GL_RENDERBUFFER_WIDTH_EXT，相应函数调用后，params 将取得到缓冲区的宽度尺寸；如果 pname 取 GL_RENDERBUFFER_HEIGHT_EXT，相应函数调用后，params 将取得缓冲区的高度尺寸。

程序清单 5.10.2 渲染缓冲区

```
void GenRenderbuffersEXT(sizei n, uint *renderbuffers);
void DeleteRenderbuffersEXT(sizei n, const uint *renderbuffers);
void BindRenderbufferEXT(enum target, uint renderbuffers);
boolean IsRenderbufferEXT(uint renderbuffer);
void RenderbufferStorageEXT(enum target, enum internalformat, sizei width,sizei height);
void GetRenderbufferParameterivEXT(enum target, enum pname, int *params);
```

Download at www.Pin5i.Com

函数 GenRenderbufferEXT()和 DeleteRenderbufferEXT()分别用来创建和删除渲染缓冲区对象。BindRenderbufferEXT()用来绑定渲染缓冲区，就像 glBindTexture()用来绑定纹理对象一样。IsRenderbufferEXT()用来检查传入参数（对象 ID）所代表的渲染缓冲对象是否有效。RenderbufferStorageEXT()用来定义渲染缓冲区的格式和尺寸。GetRenderbufferParameterivEXT()用来获得关于渲染缓冲区的属性信息。例如，如果想要知道渲染缓冲区的宽度，只要在调用 GetRenderbufferParameterivEXT()的时候，传入 GL_RENDERBUFFER_WIDTH_EXT 作为参数即可。

接下来介绍一下如何将渲染缓冲区附加到帧缓冲区上。这其实是通过调 void Framebuffer-RenderbufferEXT（enu-m target，enum attachment，enum renderbuffertarget，uint renderbuffer）实现的。形参说明如下：

- target 必须取 GL_FRAMEBUFFER_EXT，
- attachment 取 GL_COLOR_ATTACHMENT0_EXT … GL_COLOR_ATTACHMENTn_EXT，
- renderbuffertarget 必须取 GL_RENDERBUFFER_EXT，
- renderbuffer 是渲染缓冲区对象的 ID，用来绑定到当前的帧缓冲区。

通过调用函数 void GenerateMipmapEXT（enum target），就可以从绑定到目标对象上的所有纹理图像生成 mip-maps。形参 target 可以取 GL_FRAMEBUFFER_EXT 或 GL_RENDERB-UFFER_EXT。

5.10.3　基于帧缓冲区对象的高动态范围渲染

为了演示如何使用 OpenGL 的帧缓冲区对象（FBO），本小节介绍了一种简单的高动态范围（HDR）渲染应用程序。计算机图形中，屏幕上显示的场景的亮度值被限制在 0.0～1.0 之间的低动态范围内。现实世界中，我们人类眼睛所能察觉到的亮度值介于一个更高的范围内，而这通常在照片和计算机图形中损失。在视频游戏和其他的图形应用程序中进行场景渲染时，表示每种颜色成分的比特数是有限的。

使用的位数越少，其所表示的数值的精度越低。早期的图形硬件只能使用有限的整数长度的位数来表示每种颜色成分，而现在我们可以使用更多的位数来表示更高精度的数值，这使得我们拥有了更高精度动态范围的亮度颜色。由于浮点数可以存储比整数更高精度的数值，因此我们用浮点型缓冲区替代了整形缓冲区。

高动态范围光照（High Dynamic Range，HDR）渲染通常先将某个场景渲染到某个浮点型缓冲区。浮点型缓冲区的高精度特点允许我们存储比传统的整数型缓冲区更高范围的数值。通过使用浮点型缓冲区来表示高动态范围光照场景后，我们可以向其中增加各式各样的后处理特效来增强场景。例如，通过增加火焰（flare）、流光（bloom）以及其他特效，可以增强场景的真实和细腻感。将高动态范围光照渲染的场景应用到一个全屏的四边形（quad），将其渲染成一幅纹理图像，这就是后处理特效的通常做法。通过这种方法，我们几乎可以实现无限种实时特效。

Download at www.Pin5i.Com

为了实现某种特效，我们通常需要进行多遍渲染。例如，流光特效（bloom effect）是用来在场景中的光亮区域创建光晕，通过以下几个步骤实现：第一，将场景渲染后存放到某个浮点型缓冲区中；第二，将一个全屏的四边形（quad）渲染成纹理图像后存放到另外一个缓冲区中（初始大小的 1/16）；第三，将水平与垂直方向的两种模糊滤镜（blur filter）分别应用到这个亚抽样的场景上；第四，前后两组渲染器生成的结果必须在色调映射（tone-mapped）后，才能显示在屏幕上。

完成上述这一种特效就需要进行大概 5 遍渲染，而其他场景特效（火焰、景深）则需要单独处理。

一旦场景完成渲染，就可以显示在屏幕上。由于场景本身使用的是一组高动态范围的数值，因此需要对结果图像进行色调映射（tone-mapped），以确保显示器或者电视机屏幕可以将它们显示出来。由于底层硬件不支持高动态范围格式的数据，因此我们需要进行色调映射以避免渲染后的场景看起来像一幅曝光过度（overexposed）的图像。

虽然早期的硬件只支持整形缓冲区，但借助各种不同的技巧方法，我们同样可以实现高动态范围渲染。但这种方法往往不仅仅损失了精度，而且性能也非常糟糕。损失精度往往意味着画面质量的下降。部分技术和数据格式支持使用整形缓冲区进行高动态范围渲染，如 Radiance RGBE8 格式。RGBE8 格式所表示的图像中，红（R）、绿（G）、蓝（B）3 种颜色分别用 8 位表示；E 则代表共享的指数值，因此 3 个浮点数可以被压缩存放到红（R）、绿（G）、蓝（B）所对应的 3 个字节中。具体步骤是：加载 32 位的图像→应用程序解码→高动态范围渲染场景。尽管这些图像损失了一定的精度，却没有导致画面质量的巨大下降。

到目前为止，你可能觉得高动态范围渲染的每帧画面做的离屏渲染越多越好。越是高级的高动态范围场景，需要进行的处理也就越多。如果你希望在场景中加入非高动态范围（non-HDR）的特效，例如景深特效（depth-field effect），那么所需要的离屏渲染的次数甚至会更多。速度往往意味着一切；采用现代渲染技术尽管可以尽可能地得到性能上的大幅提升，但其带来的内存的巨大消耗也是很棘手的问题。值得庆幸的是，帧缓冲区对象（FBO）是我们在任何情况下的最佳选择。与前面提到的像素缓冲区（p-buffers）等其他技术不一样的是，FBO 可以在配置 SLI（nVidia's Scalable Link Interface）多 GPU 的系统上完美地运行。实际上，对于使用离屏渲染（off-screen rendering）技术的系统而言，我们也是强烈推荐使用 FBO。

光盘中附带了一个针对本文的高动态范围演示程序：首先将场景渲染后存放到某个离屏（off-screen）的浮点型缓冲区中，然后对结果进行色调映射（tone-mapped）后显示在屏幕上。场景由一个三维物体构成，该物体用一种被称为“光探测器”（light probe）的高动态范围图像纹理化而成。“光探测器”（light probe）其实是采用高动态范围数值存储场景信息的图像，比如立方面贴图或者球面贴图。如果有许多图像需要转换成高动态范围立方体贴图或者球面贴图，我们可以借助一个叫做“HDR Shop”的工具。尽管我们可以不这么做，但是高动态范围图像可以在三维场景中进行真实且高质量的反射贴图（reflection-mapping）。

Download at www.Pin5i.Com

场景一旦被渲染后存放到浮点型缓冲区后，需要对其进行色调映射后才能显示在屏幕上。色调映射可以将高动态范围的图像调节成低动态范围（LDR）的图像，从而渲染输出到屏幕上（无曝光过渡）。本文所介绍的色调映射（tone-mapping）技术非常简单：仅仅将颜色值（color value）与某个呈指数级下降的曝光级别值（exposure level）相乘。

演示程序还使用了 OpenGL 着色语言（Shading Language）来对高级的顶点和像素着色器进行编程。程序运行的时候：用“U”和“D”键来控制曝光级别，从而得到不同的运行结果；用方向键来旋转物体；用 ESC 键来退出程序。

5.10.4　总结

OpenGL 的帧缓冲区扩展是对 OpenGL API 的一个很大的增强。从整体上来看，使用 FBO 并不会破坏你的程序，反而有助于你的程序的设计与实现。本节所附带的演示程序在一个 OpenGL GLUT 所创建的窗口中进行了高动态范围（HDR）渲染，该程序已经在配置 GeForce 6800 GT 显卡的 Windows 桌面系统上测试过。

注意事项

以下是使用 OpenGL FBO 时需要牢记的几点事项：

- 不要每帧都创建和销毁 FBO，否则应用程序的性能将会大打折扣。
- 当修改作为渲染目标的纹理的内容时，避免使用类似 glTexCopy()或 glCopyTexImage() 这样的函数。
- 帧缓冲区对象可以在多个渲染目标之间快速切换。
- 可以用惟一的帧缓冲区对象为多个纹理进行渲染，或者每个纹理单独使用不同的帧缓冲区对象进行渲染。
- 无须为了使用帧缓冲区对象而创建渲染缓冲区对象（RBO），而使用渲染缓冲区对象必须创建帧缓冲区对象。

5.10.5　补充材料

关于 OpenGL 帧缓冲区对象的补充材料可参考 nVIDIA 公司的 Simon Green 的介绍 nVIDIA SDK 9.5 附带的程序展示了一种简单而直接地使用 FBO 的方法。

nVIDIA SDK 中同样包含了关于 SLI 的演示程序和白皮书。白皮书 *SLI Best Practices* 下载地址为：http://download.developer.nvidia.com/developer/SDK/Individual_Samples/screenshots/samples/slibestpract.html。

Paul Debevec 的网站（http://www.debevec.org）上提供了高动态范围纹理图像的下载。读者自行下载创建自定义的高动态范围图像的工具 HDR Shop。

关于高动态范围渲染的深入读物可以参考 Wolfgang Engel 的 *Programming Vertex and Pixel Shaders*（Charles River Media，2004）。

Download at www.Pin5i.Com

5.10.6 参考文献

Green, Simon, "The OpenGL Frame Buffer Object Extension." August 8, 2005.Available online at *http://download.nvidia.com/developer/presentations/2005/GDC/ OpenGL_Day/OpenGL_FrameBuffer_Object.pdf.*

Young, Paul, "SLI Best Practices." August 25, 2005. Available online at *http://download.developer.nvidia.com/developer/SDK/Individual_Samples/DEMOS/ Direct3D9/src/slibestpract/docs/slibestpract.pdf.*

Download at www.Pin5i.Com

第 6 章

音频

Download at www.Pin5i.Com

简　介

Alexander Brandon,Midway Home Entertainment
abrandon@midway.com

音频编程领域在过去的三年里有着巨大的发展。我们可以把在一个点声源（point source）简单地“播放一个声音”等同于简单的多边形和低分辨率贴图。随着真实物理技术和大规律资源管理（massive asset resource management）的出现以及消费者对于真实程度更高的期望，主要的全球发行商开始聘用全职音频程序员来处理这些新的挑战。在这一节中，既有那些从一个简单概念引申出新方法的内容，也有些对全新的先进技术的完整介绍，所有这些都会对我们加强游戏中的声音引擎有所帮助。

在这些内容中最重要的词汇莫过于“实时”了。大多数传统游戏都会持续地对图形进行实时渲染（如果不考虑贴图的话），而它们却很少进行实时声音合成，即使有，所占的比例也不会很多。在下一代声音引擎中，这一情况会慢慢改变。处于这一趋势的前沿意味着我们需要把声音子系统更多地和 AI、物理以及渲染联系起来。

Marq Singer 编写的《由可变形网格实时生成声音》一书在游戏声音合成方面迈进了一大步，他所介绍的方法很可能从来没有人想到过。想象一下两辆车相撞时我们可以实时地生成具有真实感的声音吧。虽然我们已经知道目前要在一个预算和开发周期都受到限制的游戏中使用可变型网格（deformable mesh）会遇到很多问题，但是对于那些通过类似于 Havok 之类的引擎来进行（超越普通物理计算之上的）真实仿真的下一代游戏来说，这将会很寻常。而即使在使用了 Havok 的情况下，最初并没有人真正地考虑到声音。Marq 的想法以及介绍对于任何一本声音编程方面的权威书籍来说都是受欢迎的。目前为止，只有屈指可数的游戏使用了基于物理的声音系统；如果我们可以在下一个游戏中用到这种技术，就可以开辟一块新的天地。

Frank Luchs 在《实时音效轻量级生成器》中对游戏中的实时声音合成技术进行了介绍。这并不是一种新的想法；在一段很长的时间里，由 Analog Devices 开发的 SoundMAX® 一直被认为是有史以来最强大的游戏声音合成引擎。然而 Frank 在本文中给出的代码示例很可能会被下一代产品所使用。实时特效和实时生成是强大的图形引擎的特点，而声音和图形相比，再也不能像以前那样处于次要地位了。Frank 所介绍的实时合成技术不仅可以使用在环境音效（例如水声或是蟋蟀声）上，还可以用于生成那些更常用的声音（如脚步、闪电或是火的声音）。即使在 Xbox 360 上有 24MB 内存

Download at www.Pin5i.Com

供你任意使用时也需要非常小心地使用内存。试试本节所介绍的技术吧。

James Boer 在《实时混音总线》一文中，对当前的热点技术“实时混音”进行了深入探索。在要求声效品质达到电影标准的下一代游戏开发过程中，工程师使用的音频工具在功能上需要达到不亚于电影后期制作所使用的工具。这样的标准，James 阐述了许多可以帮助游戏音频工程师开发出这类音频工具的卓越技术。希望这样的工具最终可以完成商业开发。

Dominic Filion 在《可听集》中所介绍的技术在音频中所起的作用与渲染引擎中的隐藏面消除一样。这项技术为“按需载入（load-on-demand）”功能铺平了道路，从而使我们可以把宝贵的内存和缓存着重用于那些位于玩家听觉感知范围内的事件。PAS 最重要的一点是它对隔离（occlusion）和衍射（obstruction）的运用，而它们已经成为第一人称游戏的标准了。

Julien Hamaide 给我们的礼物则是《一种开销较低的多普勒效果实现方法》。为什么说它是一份礼物呢？还记得在 *Half-Life*®2 中所使用的多普勒效应吗？还记得当火车接近并越过时的声音吗？还记得当飞行电动圆锯（manhack）在距离我们头部只有几英寸处高速越过，然后重新接近时所发出的令人恐惧的嗡嗡声吗？现在我们也可以实现同样的效果了！Julien 的文章还包含了大量的代码、数学分析和示例，这很可能是本章中介绍得最详尽的内容了。事实上，他为我们写下了所有的细节并且把它们装在银质托盘里端了上来。去实现它吧！

Robert Sparks 的文章《仿造实时 DSP 效果》并不很像编程高手的作品，但是他提出了一种非常聪明的方法，使我们不必在动态载入中加入过大负荷，从而避免因为 DVD 缓存不足而导致那些我们所熟悉的跳跃和爆音。通过使用多声道文件，我们可以以任意方式为任意声音加入特效。想在游戏里加入 Lexicon 混音吗？我们不知道有谁曾经尝试过，但是现在我们能够做到了。

Download at www.Pin5i.Com

6.1 由可变形网格（Deformable Meshes）实时生成声音

Marq Singer，红色风暴娱乐公司
marqs@redstorm.com

在本节中我们将介绍一种技术，它可以在体网格（volumetric meshes.）变形的基础上生成具有真实感的声音。我们对物体建模的方式和进行变形/断裂（fracture）仿真时所采用的方式一样：创建大量四面体（tetrahedral）网格来近似一个实心内部结构。我们使用一种名为“模态分析（modal analysis）”的技术来对这些网格进行预先处理以生成离散的振动模态（vibrational mode）。在实施仿真时，如果有外力施加在这一模型上，就会导致其位移和内部变形。我们会把这种撞击力投影到预先计算好的频率模态上，然后再根据这些模态的权对它们进行缩放后求和，这样就可以得到一个复杂的波形。随着仿真的运行，我们可以把生成的波形发送给音频渲染器（audio renderer）来产生与在游戏中作用于这一模型的力相对应的真实声音。

6.1.1 对《游戏编程精粹 4》的回顾

在《游戏编程精粹 4》中，[O'Brien04]介绍了怎样使用模态分析对复杂模型进行实时变形。我们会先回顾一下其中介绍的技术。通过使用这些方法中的数据并对其进行一些简单的操纵，我们就可以把变形的振动模型组合成复杂的波形数据并且由音频渲染器进行处理。我们只需要在前述面形计算的基础上付出非常小的代价就可以获得极为显著而真实的效果。

正如计算机游戏领域中很多应用那样，我们所介绍的过程是以很多现有的技术为基础并将它们用一种全新的方式进行组合而成的。我们通过把仿真中所使用的模型空间细化（volumetrically tessellate）为四面体来构造功能单元（functional unit）。我们使用类似于柔体（soft body）仿真中所采用的方法为这些单元确定约束（constraint）系统。有很多关于物体动态断裂计算的文献已经对这些技术的组合进行了介绍。请参见[O'Brien99]以获得与这些四面体网格的计算和使用相关的详细描述。

本节我们会对模态分析和模态综合（modal synthesis）进行大量的讨论，从而使用它们来预先确定物体的状态。前人所完成的工作主要研究了怎样使用模态技术和预计算模态来对物体进行动态变形（而不是断裂）。《游戏编程精粹 4》中的一个例子采用了这种方法以在一个极具真实感的仿真中

Download at www.Pin5i.Com

为一只（被原作者称之为 dodo 的）橡皮鸭子建模。我们可以假设使用这种方法进行建模的物体不会破碎或断裂，在作用于它们之上的力消失以后，它们最终会回复到某种舒展（rest）状态。如果它们不会破碎，我们就不需要对网格重新进行细分。这样的重构需要对物体新产生的部分进行大量的计算，并且无法利用游戏制作时所进行的预先计算。此外，如果我们要求一个物体必须回到它最终的形状，我们可以假设在它回到这个状态前可能经过的中间形状是有限的。在这里，我们称它们为“模态（mode）”，并且会以它们为基础来创建那些成为最终声音的波形。

6.1.2 概述

基本上，这一过程非常简单。我们对所要仿真的物体预先计算出其振动模态。作用于模型的力会导致网格定点的振动。我们会对模态集合进行分析和筛选。因为我们的目标是创建可以被听见的声音，我们可以忽略任何在人类听觉范围（20～20 000Hz）之外的振动模态。不仅如此，如果与某些模态相关的力不在仿真的合理范围之内，我们也可以忽略它们。例如，在对风铃进行仿真时我们可能不必考虑一颗手榴弹爆炸所产生的力。我们可以使用一个独立的刚体对它的移动和碰撞进行仿真。当仿真中的某个物体发生碰撞时，我们会提取整个碰撞点上的模态，然后对计算出的碰撞振幅和频率求和以得到一个复杂的波形。最后我们把对碰撞进行计算所得到的新数据送入音频渲染器作为一个声音播放。

6.1.3 对模态分析的简要回顾

相比我们的简要回顾，《游戏编程精粹 5》以及本节最后参考文献中所包含的文章对模态分解（decomposition）和模态仿真过程所作的描述要更为详细。然而，对一个非线性系统怎样才能变换为一个由相互独立的谐振子组成的矩阵有所了解也是非常重要的。这一小节将对此进行简要回顾以确保本文的完整性。

模态分解就是对系统进行线性化并将其分解为离散的模态，而模态综合则是对相关模态进行加权合并的产物。为了完成仿真中的变形部分，我们必须同时对它们加以使用。

对于一个使用类似于有限元（finite element）之类的技术进行离散化的特定物理系统来说，我们通常可以使用下面的形式对它们进行描述：

$$K(d)+C(d,\dot{d})+M(\ddot{d})=f \tag{6.1.1}$$

在此，d 是一个参数向量，它描述了仿真的配置（如，节点位置），上面带有圆点的变量则是 d 相对时间的一阶和二阶导数。K 是一个非线性函数，它以 d 为输入，并且返回弹力（elastic force）。如果这是一个弹簧-质点系统（spring-mass system），我们可以用 K 来计算出作用于所有节点上的力。C 也是一个非线性函数，它以速度为参数，返回阻尼力（dampening force），而 f 则是一个向量，其元素为作用于系统的外力（如，碰撞或是用户输入）。函数 M 则以加速度（$\ddot{d}$）为参数并返回产生它们所需要的力。由牛顿第二定律我们可以得知：$f=ma$。

虽然等式 6.1.1 通常是非线性的，如果我们假设位移（displacement）相对较小，我们可

Download at www.Pin5i.Com

以线性化表示系统的舒展配置：

$$Kd + C\dot{d} + M\ddot{d} = f \tag{6.1.2}$$

这里 K、C、M 分别指系统的硬度、阻尼和质量（mass）矩阵。

下一步就是将等式 6.1.2 对角化。通过使用一种名为“Raleigh Dampening”的技术，我们可以把 C 表示为质量和硬度矩阵的线性组合，如下所示：

$$C = \alpha_1 K + \alpha_2 M \tag{6.1.3}$$

其中α1 和α2 都是特定的值。[O'Brien02]也提到了这并不是唯一可行的阻尼技术，我们也可以使用其他技术。Raleigh Dampening 的优势在于它相对来说比较简单直接。

用上述等式替换 C 就可以得到：

$$K(d + \alpha_1\dot{d}) + M(\alpha_2\dot{d} + \ddot{d}) = f \tag{6.1.4}$$

M 是一个正定对称（positive definite symmetric）矩阵，我们可以使用 Cholesky 分解法来对 M 进行分解得到 $M=LL^{\mathrm{T}}$。如果我们另外定义一个变量 $y=L^{\mathrm{T}}d$，随后在等式 6.1.4 两边左乘 L^{-1}，并且用 y 来表示 M，就可以得到：

$$L^{-1}KL^{-}(y + \alpha_1\dot{y}) + (\alpha_2\dot{y} + \ddot{y}) = L^{-1}f \tag{6.1.5}$$

我们可以对矩阵 $\mathrm{L}^{-1}\mathrm{KL}^{-\mathrm{T}}\mathrm{L}^{-1}\mathrm{KL}^{-\mathrm{T}}$ 进一步进行分解以得到类似于 $\mathrm{L}^{-1}\mathrm{KL}^{-\mathrm{T}}=\mathrm{V}\wedge\mathrm{V}^{\mathrm{T}}$ 的等式，这里 V 是一个正交矩阵，其列是 $\mathrm{L}^{-1}\mathrm{KL}^{-\mathrm{T}}\,\mathrm{L}^{-1}\mathrm{KL}^{-\mathrm{T}}$ 的特征向量，而 V^{T} 则是由特征值组成的对角矩阵。我们可以使用 y 来定义另一个变量 z，使得 $z=\mathrm{V}^{\mathrm{T}}y$。通过对 6.1.5 左乘 V^{T} 我们可以得到：

$$\Lambda(z + \alpha_1\dot{z}) + (\alpha_2\dot{z} + \ddot{z}) = \mathrm{V}\ \mathrm{L}^{-1}f \tag{6.1.6}$$

通过简单的运算我们可以得到：

$$Az + (\alpha_1\Lambda + \alpha_2\mathrm{I})\dot{z} + \ddot{z} = g \tag{6.1.7}$$

这里 $g=\mathrm{V}^{\mathrm{T}}\mathrm{L}^{-1}f$。

虽然这些步骤有点复杂，但是最终结果相对来说还算简单。由线性化得到的等式 6.1.4 被转换为一个对角矩阵，这个矩阵包含了一系列经过分解的谐振子。在等式 6.1.7 所描写的系统中，每一列都是一个二阶线性常微分方程，第 i 列的方程式为：

$$\lambda_i z_i + (\alpha_1\lambda_i + \alpha_2)\dot{z}_i + \ddot{z}_i = g_i \tag{6.1.8}$$

这里，λ_i 是矩阵 Λ 的第 i 个元。有很多方法可以对等式 6.1.8 求解，本文中我们将着重使用解析（analytic）方法：

$$z_i = c_1 e^{t\omega_i^{+}} + c_2 e^{t\omega_i^{-}} \tag{6.1.9}$$

c_1、c_2 是特定的复数常量，ω是复数频率，并且满足：

$$\omega_i^{\pm} = \frac{-(\alpha_1\lambda_i + \alpha_2) \pm \sqrt{(\alpha_1\lambda_i + \alpha_2)^2 - 4\lambda_i}}{2} \tag{6.1.10}$$

ω虚部的绝对值对应于某个模态的频率（单位为弧度/秒），实部则是该模态的衰减率（rate of decay）。

$\mathrm{L}^{-\mathrm{T}}\mathrm{V}$ 的列是我们所研究的物体的振动模态。模态之间并不会产生交互。这一特性让我们可以把系统分解为独立的谐振子。每个模态的特征值就是该模态弹性硬度与质量的比率，它等于该模态自然频率的平方。

可参见[O'Brien02]以获得对变形方法和怎样选择 K、C、M 矩阵的讨论。

Download at www.Pin5i.com

6.1.4　声音要求

在确定使用什么来生成声音时，我们不仅必须考虑很多因素，还必须根据我们的计算复杂度要求对其进行修改。

任何声音仿真都会受到人类听觉范围的限制。在 20Hz 和 20 000Hz 之间的频率可以产生能够被人类听到的声音。正如前面曾经提到的，这对我们是有利的，因为我们可以忽略在这个范围以外的模态。这也意味着为了能够得到这一频率分布中的高频部分，我们的仿真必须使用一个足够小的时间片（大约 10^{-5}s）向前推进。这个长度非常小以至于对大多数实时系统来说都过于“离散”了。我们或许也可以接受在仿真中使用较大的时间片来生成声音，但是由于高频部分的缺失，最终的声音效果会比较沉闷。

我们的技术利用了模型变形时发生的振动。无论是在现实生活中还是对我们的模型来说，物体发出的大部分声音都是由弹性形变所激发的振动而来的。因此，我们必须使用一些技术（如模态综合）来重现这些内部变形。刚体仿真（rigid-body simulator）或是非惯性技术（inertialess techniques）并不适合用在这种类型的仿真上，这并不是说我们不必使用刚体引擎。大部分这类仿真都会使用某种类型的经典物理引擎来处理物体移动和碰撞。这种方法的一个好处是，整体模型的刚体计算与变型中所使用的粒子计算是无关的。

如果我们已经使用了一个可以对物体的动态变形进行建模的物理引擎，那么我们几乎不必为声音的生成付出任何代价。用于变形的技术已经会对那些可以作为离散振动状态并且能够被加和的模态进行了描述。我们可以把由此生成的复杂波形方便地表达为声波而不需要经过任何额外的处理。因为我们只对物体的振动建模，而不是对这些振动怎样传输到空气中或是在其他物体上反射进行建模，因此我们所建模的声音本质上是全方位（omnidirectional）的。我们也可以通过使用现有的音频技术来提高声音的真实度（如加入衰减和反射），但是这需要付出更多的代价。

6.1.5　从变形到声音

现在我们已经做好了进行声音仿真的所有准备工作。我们预先计算好了物体的模态，我们已拥有一个刚体仿真来施加外力。在这部分仿真中，我们会把可变形物体视为刚体（很可能使用一种不同的表示）直到发生碰撞为止。

我们对仿真所关心的每一个物体都要预先计算出模态分解部分中所介绍的矩阵。我们把物体的振动模态（也就是矩阵的列 $L^{-T}V$）保存下来以在系统运行时使用。

回忆一下，在等式 6.1.10 中，我们定义虚部的绝对值为模态的频率。我们会检查 $L^{-T}V$ 的每一列来找出那些在 20～20 000Hz（3.18～3 180 弧度/秒）范围内的值。所有其他的值将被忽略。虽然在这一步中会忽略大量的模态，但是剩余的模态数量仍然非常大。在实际应用中，我们发现少量的模态（最前面的 800 个甚至更少）已经足以生成声音了。

在通常情况下，我们运行实时的刚体仿真。一旦发生碰撞，所产生的力将会被投影到从前述步骤中获得的模态中。其反应由等式 6.1.9 决定。

表面振动（surface vibration）是振动从一个物体传播到另一个介质中（如空气）最关键

Download at www.Pin5i.com

的一环。我们会根据模态让物体的表面产生多大的变形来对它们进行缩放，然后把缩放所得的结果累加起来，接着把累加的结果发送给声音渲染器（很可能会先对具有方位的声音做一些额外的处理）。

在等式 6.1.9 中，我们把 c_1 和 c_2 定义为任意的复合常量。现在，我们已经完成了所有准备工作，只需要代入下面的等式，我们就可以定义一个模态对于一个经过投影的脉冲所做出的反应：

$$c_1 = \frac{2\Delta t g_i}{\omega_i^+ - \omega_i^-} \tag{6.1.11}$$

和

$$c_2 = \frac{2\Delta t g_i}{\omega_i^- - \omega_i^+} \tag{6.1.12}$$

这里，Δt 是施力的时间长度，t 则是施加脉冲的时刻。经过简化，我们得到以下等式：

$$z_i = \frac{2\Delta t g_i}{\left|\mathrm{Im}(\omega_i)\right|} e^{t\,\mathrm{Re}(\omega_i)} \sin(t\left|\mathrm{Im}(\omega_i)\right|) \tag{6.1.13}$$

对每个采样都计算等式 6.1.13 将付出很大的代价。幸运的是，通过使用等式 $e^{\omega(t+s)}=e^{\omega t}e^{\omega s}$，只需要做一次复数乘法我们就可以通过采样的值来获得下一个采样。

到了这一步，我们只需要把累加起来的频率发送给声音渲染器（可能要经过额外的处理）就行了。虽然声音生成可以被视为变形的一个副产品，然而我们也可以把注意力从视觉上转移到听觉上。如果我们所建模的物体（如金属或木材）相对来说是刚性的，并且我们可以预见所获得的振动频率会很高，我们可以通过省略对变形的渲染来节约一些资源。生成那些我们最感兴趣的声音的频率通常不会导致物体发生足够的变形来引起视觉上的注意。因此，可能有人会想把这一技术完全用于声音生成上。

6.1.6 总结

这种声音生成技术非常简洁。如果我们已经试图在方针中为动态变形建模，我们只需要付出很小的代价就可以顺带着获得一个声音生成的实现。虽然这种方法不仅简洁，还通过预先计算模态来节约运行时的开销，但是它仍然需要大量的计算，因而可能不能应用于低端硬件。随着在新的游戏机平台和 PC 上所展现的并行趋势，一些数学上的限制变得不再是那么致命了。很可能在接下来的几年中这里所讨论的技术以及其他相关技术（如动态断裂）会变得很平常。

6.1.7 进一步阅读

Hauser, K., C. Shen, and J. F. O'Brien, "Interactive Deformations Using Modal Analysis With Constraints." *Graphics Interface 2003* (June), Halifax, Nova Scotia: pp. 247–256.

O'Brien, J. F. and J. K. Hodgins, "Animating Fracture." *Communications of the ACM*, 43(7): pp. 68–75, July 2000.

O'Brien, J. and J. Hodgins, J., "Synthesizing Sounds from Physically Based Motion." *Proceedings*

Download at www.Pin5i.Com

of SIGGRAPH 2001, Los Angeles, California, August 12–17. ACM Press, New York, 1999: pp. 529–536.

Pentland, A. and J. Williams, J., "Good vibrations: Modal dynamics for graphics and animation," 1989. *Proceedings of SIGGRAPH 89*, Computer Graphics Proceedings Annual Conference Series: pp. 215–222.

6.1.8 参考文献

[O'Brien99] O'Brien, J. F. and J. K. Hodgins, "Graphical modeling and animation of brittle fracture." *Proceedings of SIGGRAPH 99*, Computer Graphics Proceedings Annual Conference Series: pp. 137–146.

[O'Brien02] O'Brien, J. F., C. Chen, and C. M. Gatchalian, "Synthesizing Sounds from Rigid-Body Simulations." *Proceedings of SIGGRAPH 2002,* Los Angeles, California, August 12–17. ACM Press, New York, 1999: pp. 529–536.

[O'Brien04] O'Brien, James F., "Modal Analysis for Fast, Stable Deformation." *Game Programming Gems 4*, Charles River Media, 2004.

Download at www.Pin5i.Com

6.2 实时音效轻量级生成器

Frank Luchs, Visiomedio 有限公司.
gameprogramminggems@visionmedia.com

本节将演示一个有效的音频合成方法，用于实时合成交互式应用软件场景中的环境音效。通常，一种声学环境由大量发声体组成，这将导致很繁重的运算负荷。我们希望建立一个更为经济的模型。这里，我们运用一个简单波形发生器来产生立体声，这种方式能合成大部分用于构成自然和人工合成声音的音频素材。除了提出一种抽象的解决方案外，我们还将着重演示了如何模拟一些自然界的声音，例如，蟋蟀和鸟叫声。

下面将从引擎的结构开始，简要阐述其合成方法，然后说明它和传统方式的区别。最后，将结合实例深入研究在使用合成方法还原真实世界声音的过程中所遇到的问题。通过实时地在预制声音序列中的不同音效之间切换，引擎能够实现多种功能效果。大量可调参数也使其能够实现交互并可根据玩家所在的位置和面朝的方向来进行声音的调整。

6.2.1 环境声引擎

本节所涉及的引擎使用扩展版本的 PortAudio 库[Bencina05]。我们创建了基于 PortAudio 缓存处理之上的一个小的扩展，用于将合成方法定义为音频单元。同时，还用 Freeverb 源代码 [Wakefield00]制作了一个滤波器。本引擎具有一个可以在不同部分间操控的粒子序列器、一个易用的序列发生器以及滤波器系统。这样的粒子序列预处理方式是创建一个有效并且低处理负荷模型的关键。

6.2.2 声音合成

在计算环境音效波形时，一定要把周期和随机的成分区分开来。本小节论述的合成方法的特性就在于它是基于声音的粒子序列的合成器，其中包含随机噪音生成器和确定的类正弦曲线（半正弦波）生成器的信息。

我们的粒子有如下主要属性：

- 频率；
- 时长；
- 电平；

Download at www.Pin5i.Com

- 噪声电平；
- 包络；
- 泛音。

为提高效率，波形生成器使用整数累加器（integer accumulator）而不是浮点矢量（floating-point phasor）。该累加器自动覆盖从 0 到最大 2 ^ 32（4 294 967 296.0）的范围。增量根据样本时长事先计算好并储存在粒子中。在缓冲器循环内，这一增量将被加到一个状态值上。然后使用该状态值在一个正弦表中查找我们需要的样本。

对于大部分声音而言，通过叠加简单正弦波的方式生成已经足够了，但粒子还允许有最多 3 个额外的有它们自己的比率和电平的泛音定义。在粒子链播放时，系统会在不同泛音的音高和电平间进行插值运算。有了泛音，就可以生成主要用于低频范围的磨具声（sharper sound），无固定音高的金属声（metallic sound）及其构成。例如，一个“a”的发音有就 800Hz、1 150Hz、2 900Hz 和 3 900Hz 这几个主要频点。如果参与合成的各正弦波频率之间为整数倍率关系，将合成有音高的声音，非整数倍率则产生无固定音高的声音。

通过在音轨里预制期望的音高信息，可以实现复杂的频率调制、颤音以及滑音效果。我们的粒子大小可变，缺省值为 107 个采样点。在 44 100Hz 的采样率下，这一缺省值相当于 3.33 毫秒的声音信息。人耳很容易察觉循环的声音，但我们的方式可以避免让这样的循环段落明显易听出来。另外，我们可以生成长达几秒的素材，对静态的声音（static sound）来说，这将更为有效。

图 6.2.1 所示宏观地描绘了声音合成系统的主要结构。区别于传统的粒子合成，这里没有使用搭接（overlapping）和窗口（windowing）技术。同样适用于只有一个粒子的情况。为了防止粒子间衔接产生爆音，只在整数相位间切换。通过这样的方式，可以轻松合成清晰有谐音的音色。

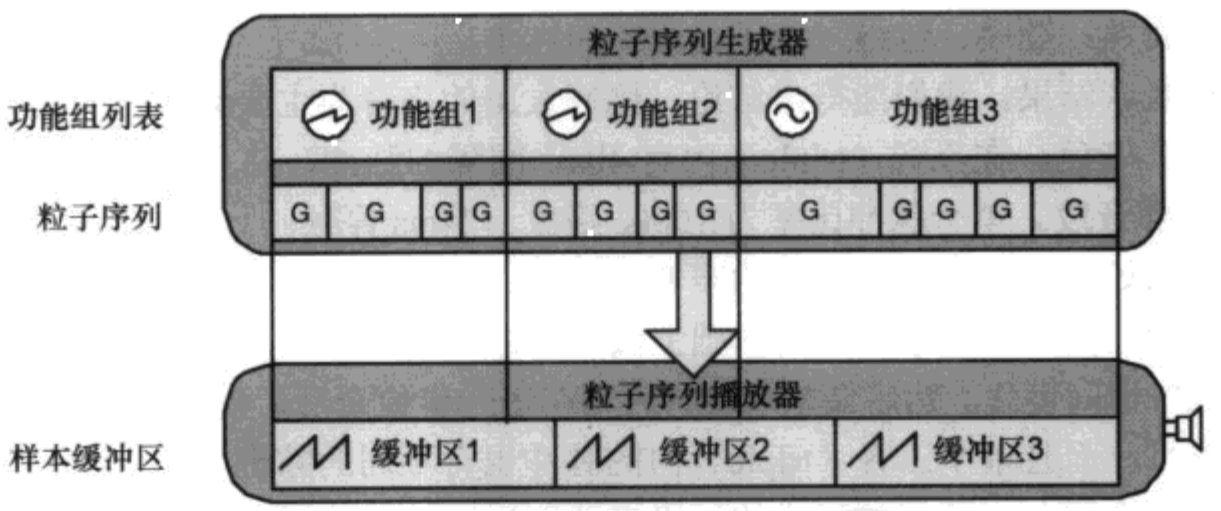


图 6.2.1　声音合成系统的结构

除了主电平外，粒子的组成中还有一个索引可以在一张查找表里查找不同的单极规格化曲线。这个特性主要应用在一些时值较长的粒子上，通过这种方式，我们可以产生过渡平滑的包络曲线，而不需要通过将一个完整的片断切成较小片断并对这些小片断设定不同的振幅值来实现。通常，我们使用的曲线为四分之一周期的正弦函数或是指数函数。指数函数是最常用的曲线，因为它们代表了声学体系的自然衰减图形。为了能用一个特定的包络控制一组粒子，我们有一个选项可以为组里每一个粒子设定一个标示和一个片断。举例来说，一个有

Download at www.Pin5i.Com

4 个粒子的组将有一个标示为 4 和一个升序的片断索引 0、1、2、3。一个有自己包络的单一粒子标示为 1，片断索引为 0。

这种方法为什么运行起来如此迅速？这是因为我们致力于表现前景事件的声音，而并不是让大量的场景中存在的声音同时产生。因为使用小的粒子，我们可以迅速地在所有不同的声音间切换。这样的技术运用到了心理声学理论中的掩蔽效果，通过忽略不容易察觉的部分，节省了大量的运算时间。

关于这种效果的一个很好的样本范例是雷电交加的雨天环境声片断，该范例可以通过访问 http://www.visiomedia.com 在线获得。虽然这类声音你可能经常听到，但请注意，这里并不是将 3 个不同声轨预先混音完成所得到的结果。你听到的效果是由一个波形振荡器迅速在雷、电、雨这 3 种声音元素中快速切换产生的。

通过应用滤波器和轻微的混响，使得声音效果听起来更有层次。采取预计算的方式事先算出一定时间点后将用到的声音数据，可避免不必要的以及资源占用率过高的样本计算处理。我们能够在不消耗过多处理器资源的前提下，实时地合成大量的声音素材。在创建声轨的时候，大部分的计算已经完成；总的来说，此生成器具有和样本回放系统同样的效率，而且有更出色的柔性。其他方法将很难实现这样的可测量性，特别是有多层静态声音样本的情况。

6.2.3 真实世界范例

1. 蟋蟀

对我们的合成系统来说，蟋蟀声音是一个很好的范例，因为它在易于理解的同时包含了自然界大部分声音的一些共同的特性。这些特性之一就是随机值的不规则产生。为了生成可信度高的类似重复音调序列，我们必须在不同的层上变化音高和振幅。我们的引擎可以在如下 5 层上改变属性：顺序、包络、粒子、周期和样本。

通过分析录音结果，我们能得出结论：蟋蟀发出的声音为主要频率点 4 200Hz 短促清晰的三音（triple）音色。在三音（triple）内有 40ms 间隔的脉冲，并且在三音（triple）之间有持续 240ms 的静音段。单个脉冲比率为 16ms 播放，24ms 停顿。

使用上述原始数值，最初重合成的声音听起来有些像廉价的警报钟声，为了让它更接近真实蟋蟀声，我们需要在这个声音的基础上做些改进。最重要的是让三音（triple）间的间隔在 160ms～320ms 间变化。然后调整响度，因为在进一步检查后，我们看到不同三音（triple）组有不同的电平，而且单个三音（triple）内的脉冲也有不同的随机电平。

我们使用两个四分正弦周期形式的包络来调制蟋蟀脉冲粒子的振幅。在音头段我们使用四分正弦周期的第一段，衰减段我们使用已规格化为 0～1 范围的四分正弦周期的第三段（负值）。和原始声音比较，合成的蟋蟀声听起来过于干净。于是我们用一个噪声电平为 10%的粒子来进行补偿，这样能增加每个周期上的轻微的振幅变化。这种方式给声音增加了一个全局噪声。我们可以在每个粒子上调整噪音的量值。

这样声音听起来有很大的改进了。但还缺少一个最能体现真实感的元素。在原始声音中，间隔间是有背景噪音的，而我们合成的声音在间隔间则是绝对安静。所以我们再添加一个峰值频率为 240Hz 的柔和的粉红噪声（pink noise），这样听起来就不一样了！

Download at www.Pin5i.Com

通过间隔的以随机的位置放置一个电平值较低的三音(triple)，并且增加原始三音(triple)的随机程度的方式，可以在此达到掩蔽效果。整合起来，有了每个粒子间随机的平衡，我们将能听到分散在一片区域内的许多蟋蟀同时发声的效果。

2. 鸟鸣声

鸟鸣声是一种迷人的声学现象。它们是音响范围中一个重要且独特的部分。鸟鸣声的合成原理和蟋蟀声类似，但是因为需要更有节奏和旋律性，所以相对复杂一些。音色间轻微的交叠，并且在喳喳声（chirp）的音头有些迅速的滑音效果。

我们的鸟鸣声合成范例产生一组由 15 个主要的喳喳声（chirp）构成的序列，每一个喳喳声（chirp）的时长为 330ms。主要脉冲的时长为 110ms 且频率范围为 2 800Hz～5 600Hz。在这个序列中，还有时长为 110±50ms 的双脉冲及 3×50ms 的三脉冲。典型的单脉冲音头部分为 15ms，衰减部分为 95ms。在音头部分，频率在 7 000Hz～4 300Hz 间滑动，但并不是所有脉冲都有这样的滑音效果。

下面列出 15 个脉冲及它们的频率：

1. 4 331
2. 3 340～4 062
3. 3 337～4 071
4. 4 290
5. 3 399～3 916
6. 4 311
7. 5 544～3 989
8. 4 371～4 154
9. 5 620～4 007
10. 3 707
11. 2 910～3 776
12. 4 252
13. 5 272～4 044～3 899
14. 3 707～3 899
15. 2 767～3 899

为实现滑音效果，我们使用 8 个 15ms 长的微小粒子。这样的粒子太小，不能激活组件包络，但可以通过粒子淡入。在每一个粒子间，我们降低频率，从 7 000Hz 开始，直到到达衰减段时的 4 331Hz，衰减段固定频率持续 90ms，使用一个指数函数包络进行调制。通过单个的喳喳声中频率的快速变化来达到鸟鸣声的特点。快速地在喳喳声的重叠双脉冲间切换，可以实现场景中同时有很多不同的鸟在鸣叫的效果。除非前景元素走到台前，否则你不会有任何间断的感觉。

6.2.4 总结

本小节中的合成方法能产生丰富多样的环境噪声，可用于生成水声、雨声和气泡声、类

Download at www.Pin5i.Com

似雷电的天气效果，以及爆发、岩浆和火的声音。此环境噪声软件合成器 Saccara [Saccara05] 基于音频粒子，尤其擅长产生风和水的效果。

我们考虑得更多的是如何让声音更可信，虽然并没有完全符合科学的“原理”，但合成产生的声音结果却是非常有真实感的，特别是那种比较密集的复杂的环境声音，而且仍然有足够的动态范围可以让单个元素凸现出来。使用一个不相重叠的例子序列，能省掉手动样本选择、编辑、分层所带来的复杂且需花费大量时间的处理。我们的技术可以实现在交互式的速率下模拟复杂的声学场景。

6.2.5 范例

可以在我们的网站上获取范例，该范例为一段整合不同游戏场景环境声的无重复流。我们已经预制了如下序列：引序列、山洞、流体、玻璃、随机分音、岩浆、脚步、呼啸的风、蟋蟀、棕柳莺和雷电。范例和源代码都可在 http://www.visiomedia.com 上获取。

6.2.6 参考文献

[Bencina05] Bencina, Ross and Phil Burk, PortAudio, 2005. Available online at *http://www.portaudio.com.*

[Saccara05] Saccara, 2005. Available at *http://www.visiomedia.com.*

[Wakefield00] Wakefield, Jezar, Freeverb, 2000. Available online at *http://ccrma. stanford.edu/~nando/clm/freeverb/.*

Download at www.Pin5i.Com

6.3　实时混音总线

James Boer, Arenanet
author@boarslair.com

程序员也许并不常编写音频子系统中的音量控制代码。对大多数程序员而言，这看起来好像十分简单。然而，一旦音频内容开发商要求在更短的开发周期内将音量控制模块应用到不同产品中时，我们就不难理解全面统一的机制对开发者的益处了。即便在基本的系统架构已经完成很久之后，通过更加灵活的方式依然可以让程序员轻松地在引擎中加入音量相关的功能模块。下面将要研究的是在一个音频库中实现音量控制的基本技巧，即通过简单的“链总线（chained buss）”机制实现定制化的音量控制。

6.3.1　并非不重要的任务

音频库类似管线，沿着它的轨迹数字波形数据将经过不同的控制、效果和混音处理。最终，这些波形将被送入数模转换器，通过这个模块数字化的近似值被转换为真实的物理波形，之后，这些物理波形通过信号放大后送入扬声器。在送入最终的混音器和转换器之前，这条虚拟管线上有一些不同的点，在这些点上我们希望能以一定的方式来控制音量。这些控制方式列举如下。

- 许多音频系统使用每一资源（per-source）音量控制，这种方式可以让声音设计师无须编辑波形数据本身就能实现音量调整。
- 除此之外，希望提供每一资源（per-source）ADSR（音头、衰减、延音、释音）包络控制。
- 不同的声音应以组的形式分类和控制（例如，音效、对话、音乐等）。
- 个别的组需要自动音量控制，或是 ducking 组；例如，当语音对话播放时，音乐音量需要适当拉低，待播放完后再让音乐音量恢复正常。
- 许多游戏需要某类主音量控制；一个手动音量功能需要有一种控制方式。
- 如果希望有自动主音量功能（类似一个控制所有正在播放声音的音量控制器），这时需要为此设定一种音量控制方式。

或许读者会惊异于这里轻易就能列出 6 种不同的沿着基本音频渲染管线的音量控制方式。实际上除此之外还有一个更基本的问题。关于这些音量控制如何安排，每个游戏都有潜在的不同设置需求。

举一个简单的例子，相对其他类型，一些游戏可能更需要有不同的音

Download at www.Pin5i.Com

频通道的逻辑组。一个棒球游戏可能需要将解说员、人群和背景声、游戏中的声音效果以及音乐分成不同的音频组；而格斗类游戏只要有音乐和音效的音量控制就够了。一个音乐类游戏则需要有更复杂些的控制了，会使用大量不同的音量总线。

读者也许会有这样的疑问："为什么不创建一个简单的固化（hard-coded）的总线设置，其中包含有能满足潜在需求的足够多的音量控制呢？"实际上，这种方式早已被考虑过并且应用于大部分的库里。通常在一个音频 API 里，这些总线被卷标为音频通道的"组"或者"分类"——例如音乐、音效或者对话。除此之外，有时也会有一个"主"音量控制。不幸的是，一旦游戏需要一个和音频系统默认方式不同的其他总线设置方式时，上述方法就很难适用了。

例如游戏需要在一个单个"组"上实现两种不同的音频控制。这类情况出现在上文中我们提到过的"对话"通道里。当评论语音音轨切换进来时，通常需要降低所有非语音通道的音量，之后再让这些音量恢复正常。如果使用一个标准的音频音量系统，这些怎样才能实现呢？大多数情况下，这涉及到复杂的编码，而且还需要在音频库层级上做大量的修改。

为什么固化编码的音量控制不是一种合式的方式呢？简单的回答类似于为什么固化编码同样不适合游戏开发的其他方面：我们几乎不可能预测一个需要很长开发周期的单个游戏的需求，更别说是每一个将来可能会使用这个库的游戏的需求了。其实建立一个灵活、弹性而且几乎可以有无限设置的系统并不需要耗费更多的工作量，下面将会探讨如何实现。

6.3.2 实现音量总线链

上文已说明了对音频库来说，对固定组和总线的固化编码控制并非好的方式。但究竟怎样才是好的选择呢？

基本的方式其实相当简单：每一个音量控制是一个小的对象，该对象简单来说相当于音量控制链上的一个链接。这些音量控制全部都链在管线上，处理的先后顺序和音频数据移动方向一样，都是从最初的源文件直到最终的混音缓冲。独特的音量控制总线可以不限数量的被创建并链接到父总线上，形成类似树的结构——虽然仅从叶子到根遍历（traversed）。和典型的树不同，这里父总线没有子总线的信息。图 6.3.1 演示了如何设定一组音量总线。

创建的每一个音量总线仅包含单一的总线信息，表示隶属于管线的下一层级——类似于树结构的低一层。这样允许创建离散的"音量总线组"，例如图 6.3.1 中所示的"对话"、"音乐"和"音效"，以及针对单个音频素材的特定音量控制对象。这些控制通常以易用的类似音量推子的形式提供给最终用户。

然而，需要注意的是这里在组这一层级还有更深的控制，例如链接在除对话之外其他组之下的一个叫做"非对话"的单一总线。这条总线可用于在对话出现时自动降低音量。如果需要的话，游戏也可提供一个不影响其他通道音量的类似"偶然对话（Incidental Dialogue）"的组。类似如此的简单机制，为你的游戏音频系统设置提供相当大的弹性。

遍历整个音量控制对象链允许简单地计算当前播放缓冲器应被设定的最终音量。整合这些计算中一个值得注意的效果是，对实际声音素材的音量仅仅申请一次。这种方式应用得很好，同样的，许多音频 SDK（例如 Direct-Sound®或 OpenAL）中音量只能在一个唯一的点（每一声音通道）上申请。

Download at www.Pin5i.Com

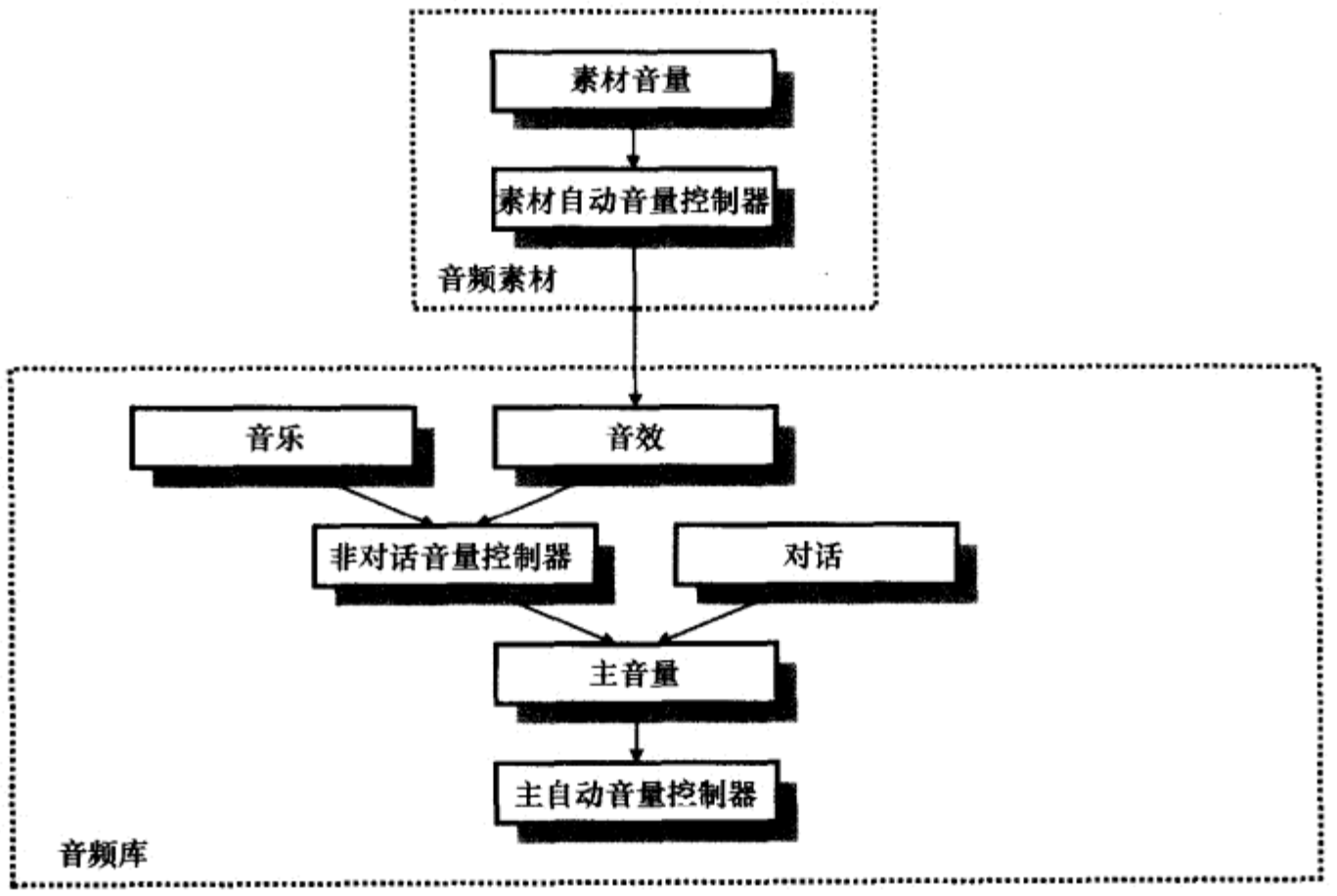


图 6.3.1　运用于游戏中的典型总线设置

这些音量总线可以以希望的任何方式创建和使用，在一些特殊的场合中使用它们将非常有意义。其中之一为，当一组总线构成音频库的核心时，可能通过一个类似字符串或者整数值唯一的 ID 进行关联和管理。当创建音频总线控制单个的声音素材时，这个 ID 值就被指定为每个单个声音的一个创建参数，以这样的方式让新创建的音量总线将自身链接到一个特定的构成音频系统的音量总线上。

实现音频总线的代码实际上相当简单。一般来说，创建一个音频总线不过是建立一个其中包含一个音量变量和一个指向自己类（或结构）的指针的类而已。指针用做树结构中从子目录到父目录的链接，这里和典型的树结构是相反的。程序清单 6.3.1 给出了一个音量总线类的基本实现方式：

程序清单 6.3.1

```
class VolumeBuss
{
public:
    VolumeBuss()
    {
        m_Volume = 0.0f;
        m_DownstreamBuss = NULL;
    }
    void SetDownstreamBuss(VolumeBuss * buss)
    {
        m_DownstreamBuss = buss;
    }
    void SetVolume(float volume)
    {
```

Download at www.Pin5i.Com

```
            m_Volume = volume;
        }
        float GetVolume() const
        {
            float vol = m_Volume;
            if(m_DownstreamBuss)
                vol *= m_DownstreamBuss->GetVolume();
            return vol;
        }
    private:
        float m_Volume;
        VolumeBuss * m_DownstreamBuss;
};
```

从上可以看出，音量总线最简单的形式无非是一个能链在其他类似变量之上的单一变量。

6.3.3 以比率或分贝的形式来表示音量

为了易于计算沿着给定链的每个音量控制的总合结果，我们首先需要理解数字世界里音量控制的特性。现代硬件和 API 中，几乎所有的音频音量都采用减法运算。初始化创建的波形，其音量级一般被设定为有很好的信噪比（或有时是相互之间有好的动态范围）。音量只能程序化的从初始值降低——换句话说，放大原始波形一般不采用。通常，描述这类减低特性为术语分贝（dB）、对数标量，但有时也用比例来给定值——0 到 1 间的浮点数值。

不管用什么方式，合并两个音量值相对容易。如果是比率，简单的将两个值相乘。如果是对数标量，以分贝单位将总负值相加。举例来说，如果一个音量控制设定为–3dB，另一个为–4dB，则总音量就为–7dB。

6.3.4 执行效率问题

如果是在软件中使用音频混音模块，例如 PC 平台，那就应该考虑到如果每次更新都要改变音量将导致执行效率低下。正因如此，需要考虑跟踪经由音量总线链计算出的最终音量值，并且与上一个已知值进行比较。这样，可以在必要时才去改变活动声音缓冲器的音量控制。

实际运用没有听上去那么复杂。程序清单 6.3.2 为伪代码（pseudo code）的大致形式。

程序清单 6.3.2

```
// 在定时更新函数中
NewVolume = VolumeBuss.GetVolume();
if (NewVolume != CachedVolume)
{
    SoundBuffer.SetVolume(NewVolume);
    CachedVolume = NewVolume;
}
```

可选地，这里能增强音频总线类使其决定是否音量与上次检测时相比较已发生变化。

Download at www.Pin5i.Com

6.3.5　其他增强

专门用来控制淡入淡出和包络控制系统的音量总线是本系统最有实用价值的功能之一。事实上，在音量总线类本体中设定这样的自动处理方式是非常值得的。这样一来，我们就可以指定链中的任意节点作为一个机制，容易地、自动地创建一个简单的淡入淡出，甚至一个复杂完整的 ADSR 包络[Boer04]。淡出所有声音用如下一行简单的 C++代码就能实现：

```
// 让所有声音在两秒钟内淡出
AudioManager::getVolumeBuss(ID_MASTER)->fadeout(2.0);
```

6.3.6　总结

音频库中的音量控制虽然表面上看只是一种简单的功能，但如果在设计时没有经过充分地考虑，可能会导致游戏音频设计中不必要的限制和妥协。基本的音量控制总线链设计能让你的音频库无须重写代码就能轻松应对将来任何设计上的挑战。

此外，通过本节实例说明了即便在类似音量控制这样的简单组件设计中，我们也应该多做一些深入的思考。早期植入系统的哪怕微小的灵活性都可能为将来节省大量重写代码的工作。

6.3.7　参考文献

[Boer04] Boer, James, "Dynamic Variables and Audio Programming." *Game Programming Gems 4,* Charles River Media, 2004.

Download at www.Pin5i.Com

6.4 可听集（Potentially Audible Sets）

Dominic Filion
dfilion@hotmail.com

如果在树林中的某棵树倒下时没有人在附近，它会发出声音吗？

随着 EAX 以及具有 5.1 环绕声道的游戏系统的出现，近年来我们在为玩家创建精确的声音体验方面做出了大量的进展。但是，事实上用于声音环境的算法无论在数量还是在复杂度上都无法与那些通常用于视觉效果的算法相比。然而，我们可以对后者中的大部分进行改进从而以某种方式应用于音频领域中。

隐藏面消除（hidden surface removal）是图形开发人员常常使用的关键算法之一。它采用一系列不同的算法对场景中的物体进行处理来确定哪些物体可能会被用户看见。对于图像来说，现在我们主要把隐藏面消除算法作为一种优化手段，因为无论高层的隐藏面消除算法实现得多么不好，显卡的深度缓冲（z-buffer）总是可以确保场景的正确渲染。在音频领域中与隐藏面消除所对应的是隐藏声消除（hidden sound removal），这一技术可以有效地增加声音环境的真实度。通过采用这样一种技术，我们可以对具有成百上千个并发声音的声音环境进行真实的建模，并使用这一算法来找出哪些声音与玩家的当前视野相关。

6.4.1 PVS 入门

PVS（可见集，Potentially Visual Set）是最常用的隐藏面消除算法之一，我们会对它进行简要介绍，然后再研究怎样把同样的算法应用到音频领域中。

在一个 PVS 中，我们会把整个场景划分为一定数量的区域（zone/region）。在导出场景时，我们会对每个区域预先计算出一些信息。这些信息可以告诉我们当玩家在这个区域中的任意位置时，他可能可以看见的其他区域。我们把这些信息称为这个区域的 PVS。在渲染时，系统会找出玩家当前所处的区域，并且不会再对任何不包含在当前区域 PVS 中的区域进行进一步的处理。稍后我们将介绍怎样使用类似的概念来定义一个 PAS。

本质上说，PVS 就是对于物体是否可见的一个粗略估计。玩家可能会处于某一区域中的任意位置上，因此要对此进行精确的计算是不可能的。最重要的是，每个可能被看见的物体都处于 PVS 中（如图 6.4.1 所示）。

Download at www.Pin5i.Com

组成 PVS 的区域通常都是凸的。因为对于一个凸区域来说，无论玩家处于其内部的哪个位置，它都不会被自身所遮挡。凸区域的这一固有特性可以大大地简化我们对 PVS 进行预先计算的算法。

虽然可以让游戏设计人员手工定义构成 PVS 的区域，但通常我们都会让这个过程自动化，使用 BSP（Binary Space Partitioning）来把场景自动地分割为凸区域。

值得一提的是，PVS 有一个重要的限制：我们最好把它应用于静态（static）或半静态（semistatic）的环境中。它可以很好地对门以及其他可能的连接（如可以被打碎的墙等）进行处理，但是对于那些包含了大型可移动障碍物的声音环境来说，这个算法可能并不合适。

6.4.2　PAS 基础

我们可以把 PVS 中的大部分概念应用到声音环境中。在图 6.4.1 中，我们可以看到在同一环境中光和声音的行为。

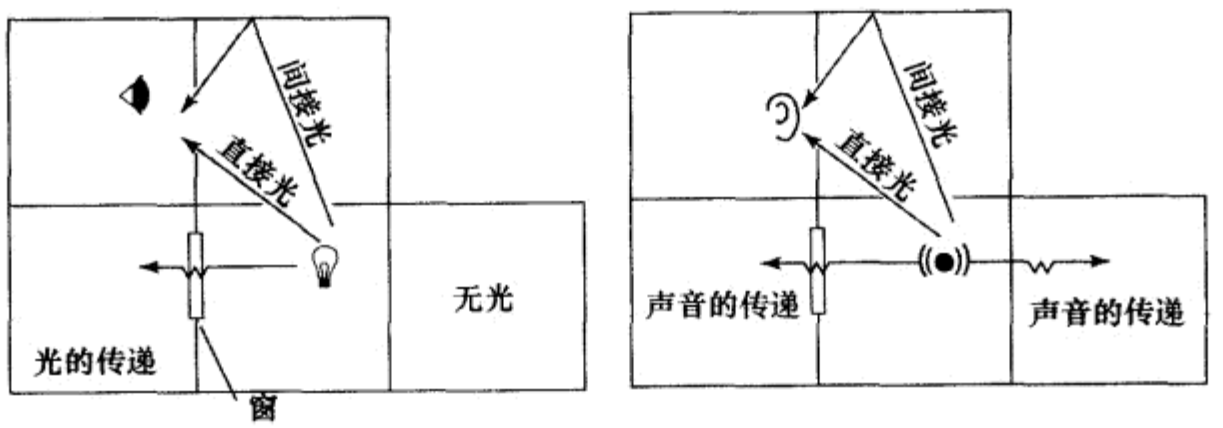


图 6.4.1　光和声音在行为上的相似之处

环境中的光既可以直接进入眼睛也可以通过墙壁反射后再进入眼睛，这产生了类似于反射和环境光（ambient illumination）之类的效果。光也可以在某些类型的媒介中传播，如玻璃（在图 6.4.1 中的窗户），在通过这些媒介时光还会改变颜色（也就是改变光波的频率）。

声音具有类似的特性：它们可以直接进入耳朵或是经过墙的反射，这产生了回响（reverberation）以及反射声波（reflected sound waves）。相对于光来说，声音可以在更多的物质中传播，通常声音在这些物质中传播时会产生低通滤波（low-pass filter）的效果。因为不同类型声音效果的行为和声音特性（sonic property）具有很大的区别，我们往往会对直接、间接和在媒介中传播的声波分别进行处理。

6.4.3　直接声音路径

那些在发声者和接收者之间存在一条直接路径的声音通常可以不经修改地传播到接收者那里。我们还可以使用某些特效来实现远距离声音的衰减和高频抑制（dampening）。为了确定在声源和接收者之间是否存在一条直接路径，我们必需对它们之间的射线进行追踪以检查是否存在阻挡物。

尽管可以对声音建模（sound geometry）的每个多边形都进行一次射线追踪来找到一个交

Download at www.Pin5i.Com

点，我们也可以通过一个基于直接声音路径（direct-sound-path）的 PAS 来迅速排除大部分与声源之间不存在直接路径的情况。可以使用 BSP 把声音建模划分成区域，对每个区域我们都可以确定其他哪些区域中与它有直接声音路径。这样，当接收者在区域 A 时，仅当区域 A 可听集中的某个区域里有声音播放时我们才需要检查 BSP，我们可以安全地忽略其他所有区域。

对于声音来说我们并不需要使用非常精确的环境模型，通常我们可以使用场景建模的简化版本来为声音环境建模。为了简便起见，我们可以限定所有的 BSP 建模都是由与坐标轴垂直的平面构成的，并且它们可以把声音所处的房间或区域完全地包含在里面。

这样我们就可以得到一系列用来代表声音环境的长方体。环境细节的缺失并不会以任何可察觉的方式对声音环境的再现起到影响。通过使用比长方体更为复杂的形状，我们或许可以设计一个能够更为精确地剔除隐藏声音的算法，但是我们也必须为剔除这些声音所需要的大量计算付出代价；很可能我们在性能上所失去的远比采用一种“精确”的解决方案所获得的性能提升要多。

生成 BSP 时我们会把声音建模递归地切割为对等的区域。首先，我们找出整个场景建模的全局包围长方体（Bounding Box），然后，沿着声音建模中某个多边形的平面把它分为两半。随后根据多边形在这个平面的哪一边对所有的多边形进行分类，然后沿着多边形平面对每一部分进一步细分（subdivide），直到每个区域仅仅包含位于边界上的多边形为止。对于 BSP 特性的完整说明已经超出了本文的范围；可以参考[BSP01]来获得更多的信息。要完全掌握本文所介绍的方法，首先必需对 BSP 基础知识有良好的理解，在此我强烈建议读者在有必要时复习一下关于 BSP 的知识。

这样得到的 BSP 就是一棵包含了很多平面的树，它对声音环境中的多边形进行了细分。从这颗 BSP 中我们可以获得描述声音建模的凸区域。因为我们限制这些声音建模只能是与坐标轴垂直的多边形，因此这些 BSP 区域都是不同大小的长方体，这些长方体的每个面都是由一个矩形多边形所定义的。

我们可以把每个 BSP 区域标记为实心的（solid）或是空心的（empty）；我们在计算 PAS 时将会用到这些信息。我们可以认为 BSP 树中所有前向的 BSP 叶节点（front BSP leaves）都是空心区域。与之相反的是，所有后向的 BSP 叶节点（back BSP leaves）都是实心区域。

通常，BSP 树中平面的方向都是和它们所代表的多边形的法向量方向一致的；因此，我们可以安全地假设所有位于 BSP 分割平面后方的区域是实心区域。要得到这点，我们还必需假设场景的声音建模是正确的：不仅没有任何缝隙，还必需完全封闭。

一旦得到正确标记了的区域，我们就需要找出门户（portal），也就是定义两个区域之间连接的多边形。要找到区域之间所有的门户非常简单，我们只需要对所有定义了某个区域的多边形进行检查并且把那些连接两个空心区域的多边形标为门户就行了。与之相反，我们可以把那些连接空心区域和实心区域的多边形看作为墙面。

现在我们得到一系列区域以及连接这些不同区域的门户信息。这是我们进行 PAS 计算所需要的所有数据，如图 6.4.2 所示。

让我们看一下图 6.4.3 中的这个例子。如果区域 A 中的声音能够被另一区域听见，它必须穿过区域 A 的某个门户。因此，如果我们可以从另一个区域中“看到”区域 A 的某个门户，那么我们就与区域 A 中的这个声音有一条直接路径。下面列出了用来确定 PAS 的规则。

Download at www.Pin5i.Com

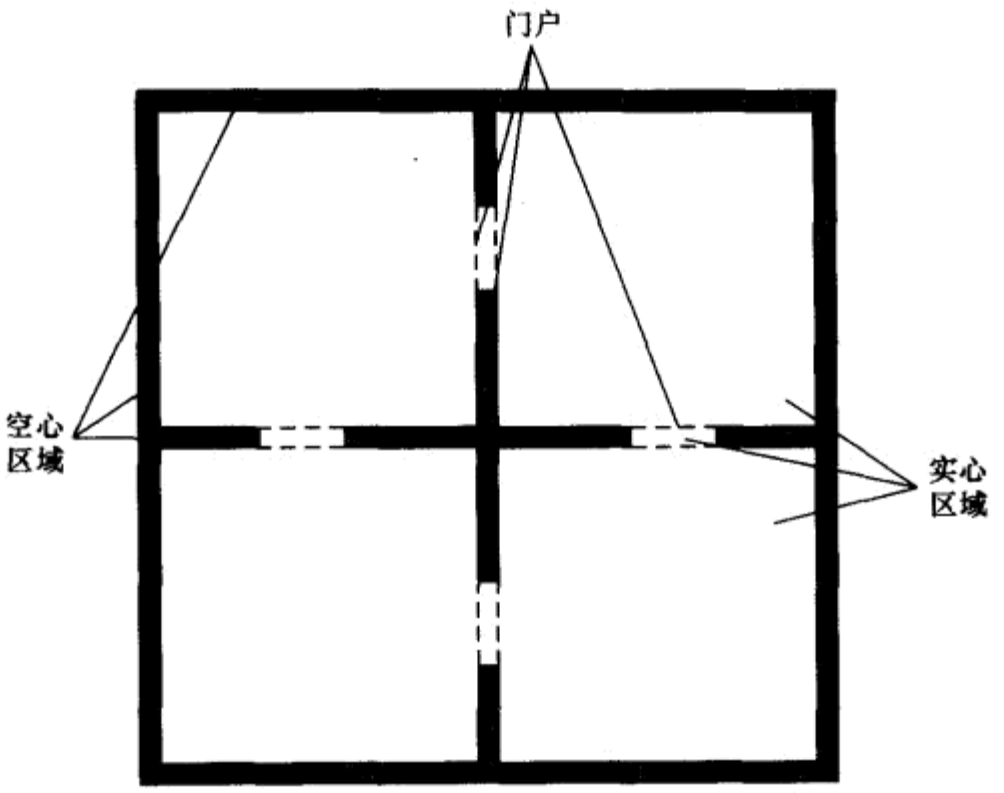


图 6.4.2　空心区域、实心区域和门户

如果一个声音在区域 A 中播放，那么：

- 它能被区域 A 所听见；
- 它能被所有与区域 A 通过门户连接并且离区域 A 的层次不超过 2 的区域听见；
- 给定一个不包含 A 的区域集 S，如果 S 中的某个区域存在一个门户位于由区域 A 的某个门户和与 S 相连的某个区域的门户所形成的平截体（frustum）中，那么在这个区域中可能可以听见这一声音。

好了，我们已经说得够多了。第一种情况非常简单：如果我们和声源位于同一区域中，我们就可以通过直接路径听见它。第二种情况也不是很难：如果我们站在与区域 A 相邻的区域 B 中，或是与区域 B 相邻的区域 C 中，我们都有可能可以获得一条直接声音路径。记住，我们感兴趣的仅仅是排除那些无论声源位于区域 A 中的哪个位置，都不可能存在一条到声源的直接路径的情况。因此，很容易就可以看到如果声源位于区域 A 的某个门户旁，那么区域 B 和区域 C 可以直接听到它。第三种情况是最难的。

在图 6.4.3 中，平截体就是由虚线标出的区域。它通过门户 1 和门户 2 定义。我们通过创建一个穿越了门户 1 的左边和门户 2 的右边，并且穿越了门户 1 的右边和门户 2 的左边的平面来找出这个平截体（在三维情况下我们还要对上下边进行类似的处理）。这个平截体定义了，当站在区域 A 中时，我们穿过门户 1 和门户 2 所能看到的最大范围。如果区域 C 中的某点不在平截体范围内，那么我们不能从区域 A 画出一条到这个点的直线。

观察图 6.4.3，我们可以注意到门户 3 不在平截体范围内。因此，在这种情况下区域 D 中的任意物体和区域 A 中的某个物体间不可能存在一条直接路径。我们还必须考虑有些门户可能会部分遮挡住狭长地带中更远的门户，就像图 6.4.4 中那样。

门户 2 是一个狭小的缝隙，通过它我们只能看见/直接听见一块很小的区域。这样一个有限的平截体会减小我们通过门户 3 所能看到的区域(注意到门户 3 本可以一直延伸到最左边，但是却被门户 2 的平截体裁减掉了)。这减小了门户 3 的平截体（这种情况下，门户 2 和门户 3 的平截体共享相同左边界）。因为门户 4 不在平截体中，门户 4 以外的区域不可能为门户 1

Download at www.Pin5i.com

内的区域所见。

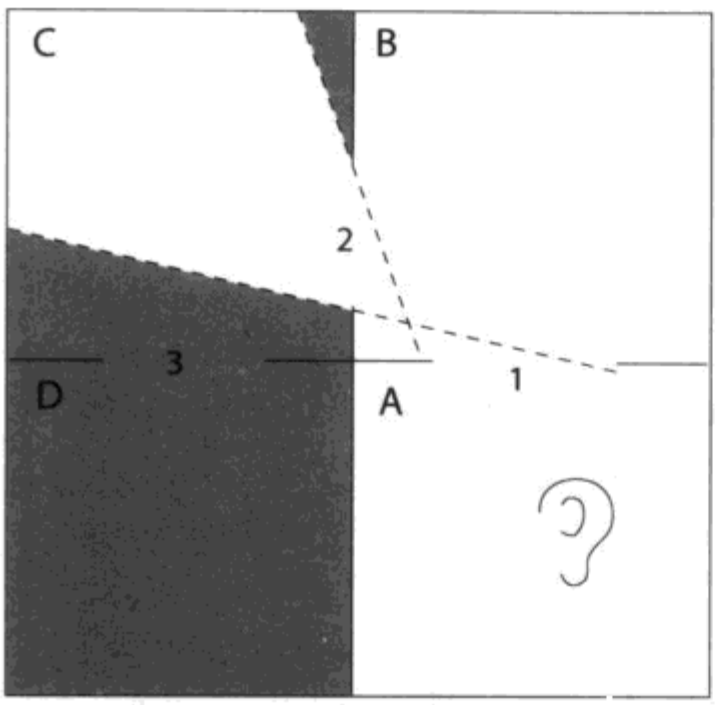


图 6.4.3 区域 A 的直接路径平截体

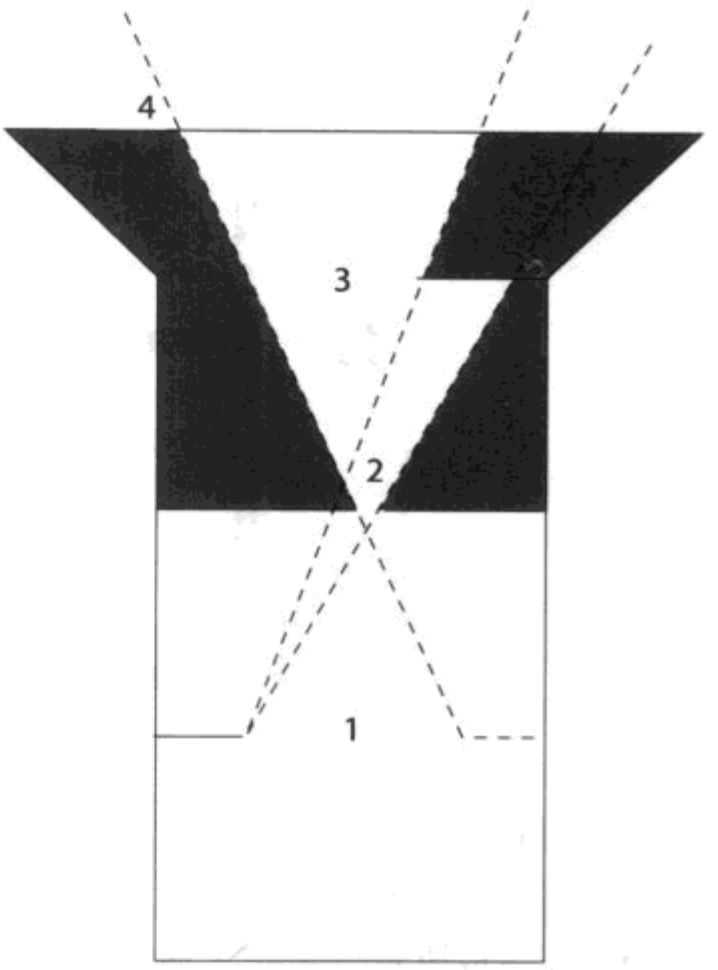


图 6.4.4 门户效果的累积

为了对这一情况进行处理，每当我们处理完某个门户后，在对这一门户以外的门户进行处理时，我们必需不断地用已经处理过的门户的平截体去裁减那些尚未处理的门户。程序清单 6.4.1 中列出了从一个区域创建 PAS 的伪码。

程序清单 6.4.1 创建 PAS 的伪代码

```
for ( DWORD i = 0; i < regionCount; i++ )
{
    CRegion* pRegionA = regions[i];
    for ( DWORD j = 0; j < pRegionA->GetPortalCount(); j++ )
    {
        CPortal* pPortal = pRegionA->GetPortal( j );
        pPortal->BeingTested( true );
        CRegion* pRegionB = pPortal->GetOtherSideRegion();
        pRegionA->AddToPAS( pRegionB );

        for ( DWORD k = 0; k < pRegionB->GetPortalCount(); k++)
        {
            CPortal* pPortal2 = pRegionB->GetPortal( k );
            CRegion* pRegionC = pPortal2->GetOtherSideRegion();
            pPortal->BeingTested( true );
            pRegionA->AddToPAS( pRegionC );

            TestRecursivePortals( pPortal, pPortal2 );
```

Download at www.Pin5i.Com

```
            pPortal->BeingTested( false );
        }

        pPortal->BeingTested( false );
    }
}

void TestRecursivePortals( CPortal* pPortal, CPortal* pPortal2 )
{
    CRegion* pRegionA = pPortal->GetThisSideRegion();
    CRegion* pRegionC = pPortal2->GetOtherSideRegion();
    CFrustum* pFrustum = FormFrustum( pPortal, pPortal2 );

    for ( DWORD i = 0; i < pRegionC->GetPortalCount(); i++ )
    {
        CPortal* pPortal3 = pRegionC->GetPortal( i );
        if ( pPortal3->IsBeingTested() )
            continue;
        pPortal3->BeingTested( true );
        // 如果在对门户进行裁减时没有把组成门户的多边形完全裁减掉，下面这个函数返回 true
        CPortal* pClippedPortal;
        if ( pFrustum->ClipPortal( pPortal3, pClippedPortal ) )
        {
            CRegion* pRegionD = pPortal3->GetOtherSideRegion();
            pRegionA->AddToPAS( pRegionD );

            TestRecursivePortals( pPortal, pClippedPortal );
        }
        pPortal3->BeingTested( false );
    }
}
```

这一算法的最终结果是对声音建模中的每个区域，我们都获得了一个预先计算好的区域集合，这个集合中的每个区域与这一区域都可能存在一条直接声音路径。因此，在运行时我们可以通过 BSP 来确定玩家位于哪个区域中，然后就可以忽略所有不在玩家 PAS 中播放的声音，这就是 PAS 算法的精髓所在。接下来我们会根据各种不同的声音行为为这一系统创建一系列有用的扩展。

6.4.4 为门窗创建动态 PAS

虽然 PAS 需要预先计算好，但是在整个游戏进程中它并不必保持不变。打开或是关闭门窗也可以打开或关闭不同的声音路径。

要处理这些改变，我们可以先在导出时为场景预先计算当某扇门开着时的 PAS。随后，我们把这扇门关上，并且重新对这一情况计算 PAS。我们可以通过下面这一规则来避免对整个场景的 PAS 进行重新计算：任何能够看到这扇门的区域必须重新计算它的 PAS，其他区域保持不变。每扇门两边各有一个区域，如果某个区域属于这两个区域之一的 PAS，那么我们

Download at www.Pin5i.Com

也必须重新计算它的 PAS。

最终对整个场景我们可以预先计算出两组 PAS：一组对应于门打开的情况，另一组对应于门关闭的情况。因为这两个 PAS 集合非常相似（场景中的大部分区域不会受到某扇特定的门是否关闭的影响），我们只需要保存门打开时的 PAS 以及门关闭时的 PAS 与打开时不同的部分。在运行时，我们可以随着门的打开和关闭在这两组 PAS 中进行切换。

6.4.5 PAS 扩展：传导（transmission）

基本的 PAS 结构为我们迅速找到与当前位置可能具有直接路径的区域建立了良好的基础。然而，声音也可以穿越固体物质，本文把这种现象称为传导。声音穿越固体物质后一部分声波会被滤去（filter），因此听上去会有一些模糊。

我们可以使用生成 PAS 所用到的区域和门户结构来找到那些可以通过固体物质传导的声音。每个区域都包含了所有门户（两个空心区域之间的连接）的列表。我们也可以使用同样的结构来生成该区域中所有墙（可以把它们看作为实心门户，也就是空心区域和实心区域之间的连接）的列表。

当我们播放一个声音时，它可以从发声区域所连接的所有实心门户递归地向外传导。如程序清单 6.4.2 所示，随着声音穿过每个实心门户，它会逐渐模糊直到衰减为 0。

程序清单 6.4.2 声音传导的伪代码

```
bool PropagateTransmittedSound( CRegion* pSrcRegion )
{
if ( pRegion->PlayerIsInPAS() )
    Return true; // 存在直接声音路径，不需要传导
    for ( DWORD i = 0; i < pSrcRegion->GetSolidPortalCount();
    i++ )
    {
        CPortal* pSolidPortal =
            pSrcRegion->GetSolidPortal( i );
        CRegion* pRegion =
            pSolidPortal->GetOtherSideRegion();
        MuffleSound(
            pSolidPortal ->GetMufflingCoefficients() );

        if ( pRegion->PlayerIsInPAS() )
            {
                PlayMuffledSound();
                return true;
            }

        if ( pRegion->DistanceFromSource() <
            pSound->GetMaxDistance() &&
            PropagateTransmittedSound( pRegion ) )
                return true;

            UnMuffleSound( pPortal->GetMufflingCoefficients() );
```

Download at www.Pin5i.Com

```
    }
return false;
}
```

我们也可以对声音在一个区域到另一个区域间的模糊信息进行预先计算，并且保存在另一个为声音传导专门创建的 PAS 中。

6.4.6　PAS 扩展：反射

第三种类型的声音效果是反射，它也是物理上最复杂的一种。这一效果还包括回响，如那些被墙反射回来的声音。在一个具有大量障碍物的声音环境里，声音在狭长地带中很容易产生回响效果，因此我们通常可以近似地认为在声音播放时它会沿着这一狭长地带向所有方向传播。

要实现这项功能，我们可以对声音周围的区域进行“填充（flood-fill）”，直到到达接收者或是达到声音衰减的极限。这样我们就可以估计出声音在穿越狭长地带时所使用的最短路径。我们可以根据最短路径到接收者之间的距离估算出反射声音的音量。

我们还希望能够正确地放置反射声音的位置。如果一个声音在狭长地带中反射时改变了方向，我们需要在离接收者最近的门户上（而不是在其原始位置）以当前的衰减来播放它，程序清单 6.4.3 对此进行了实现。注意我们仅在开始播放新的声音或是玩家改变区域时才需要执行这一代码，这相对来说并不频繁。如果玩家所在的区域保持不变，这一函数的结果也会在若干帧内保持不变。

程序清单 6.4.3　声音反射的伪代码

```
Void FindShortestDistanceReflectedSound( CRegion* pSrcRegion )
{
CPortal* pClosestPortal = NULL;
    float fShortestDistance = FLT_MAX;
    if ( PropagateReflectedSound( pSrcRegion, NULL, 0.0f,
        fShortestDistance,
        pClosestPortal ) )
    {
        PlayReflectedSound(pClosestPortal, fShortestDistance );
    }
}

bool PropagateReflectedSound( CRegion* pSrcRegion, CPortal*
    pSrcPortal,
    float fCurrentDistance,
    float& fShortestDistance,
    CPortal*& pClosestPortal )
{
    if ( pSrcRegion->PlayerIsInRoom() )
        return true;
    if (fCurrentDistance < fShortestDistance )
    {
        fShortestDistance = fCurrentDistance;
```

Download at www.Pin5i.Com

```
            pClosestPortal = pSrcPortal;
        }
        if ( fCurrentDistance > pSound->GetMaxDistance() )
            return false;
        for ( DWORD i = 0; i < pSrcRegion->GetPortalCount(); i++ )
        {
            CPortal* pPortal = pSrcRegion->GetPortal( i );
            CRegion* pRegion = pPortal->GetOtherSideRegion();

            if ( !pSrcPortal )
            {
            if ( PropagateReflectedSound(pRegion, pPortal,
            fCurrentDistance +
            pSrcPortal->DistTo( pSound->GetPosition() ) ) )
            return true;
            }
            else
            {
            if ( PropagateReflectedSound(pRegion, pPortal,
                fCurrentDistance +
                pSrcPortal->DistTo( pPortal ) ) )
                return true;
            }
        }
    }
```

6.4.7 总结

本节所描述的系统为模拟声音播放的环境特效提供了完整的环境，它可以被应用于具有任意复杂度的环境。通过采用计算机图形学领域中所使用的 PVS 方法，我们可以迅速地确认声音环境中的哪些声音可以被直接听到。我们还可以对 PAS 以及相应的 BSP 结构进行扩展来处理传导和反射等效果。这使得我们能够实现具有成百上千个声音同时播放的复杂声音环境，并且使用 PAS 系统来迅速地找到与接收者相关的声音。

6.4.8 参考文献

[BSP01] BSP Trees FAQ. Available online at *http://www.faqs.org/faqs/graphics/bsptree-faq*.

Download at www.Pin5i.Com

6.5　一种开销较低的多普勒效果

Julien Hamaide,
Elsewhere 娱乐公司
julien.hamaide@gmail.com

三维图形渲染引擎的质量一直在不断地向真实质量（realistic quality）靠近。然而对视频游戏的其他部分来说却并不是这样。本节的目的就是要向读者介绍一种可以增强游戏音频真实性的技术。通过模拟多普勒（Doppler）效果，我们就可以对发出的声音和接受的声音进行动态修改。这样一来，那些发声实体就会具有动感。我们也可以在游戏之外（譬如说，在为视频创建特效时）使用这项技术。不仅如此，这项技术实现起来也非常简捷。

6.5.1　多普勒效果

多普勒效果就是指当音源和观测者之间存在相对位移时，音源发出的声音在频率上体现出的明显变化。这种感觉是由于所感知的波长变化导致的。图 6.5.1 中表示了当音源移动时在特定时刻发出的声波。如果观测者和音源互相接近，所感知的波长就会变短，因此观测者听到的频率较高。当一辆拉响警报器的救护车向我们开来时，这一效果会尤为明显。当救护车经过我们身边并且逐渐远去时，声波要传播的距离不断增加，于是观测者可以感觉到波长变长了，从而感觉这个声音的频率降低了。

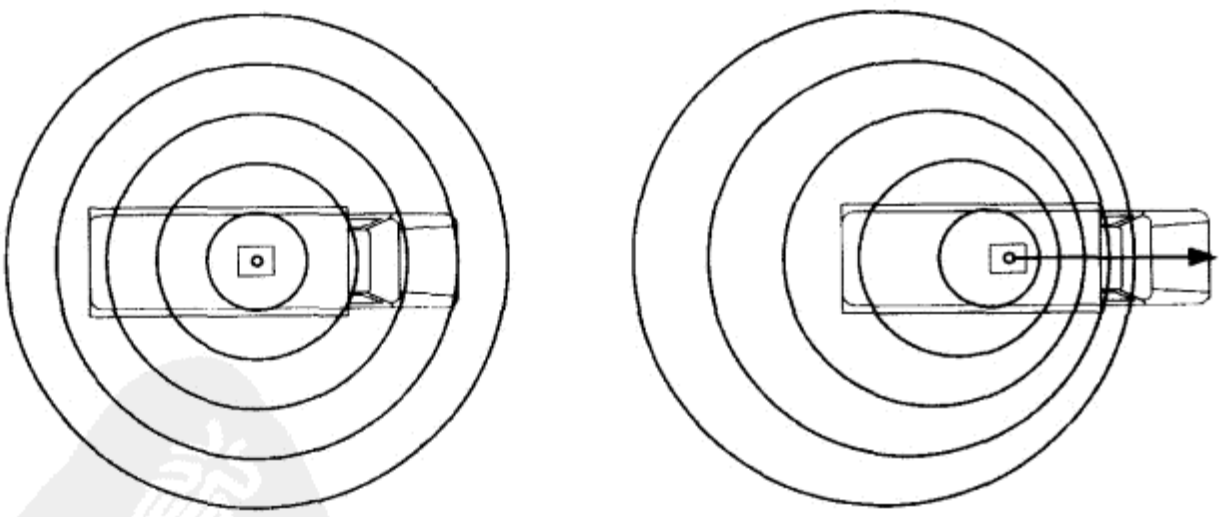


图 6.5.1　移动音源的声波传播

下面我们将要介绍计算多普勒效果所必需的公式。在图 6.5.2 所示中，音源以速度 V_s 向着观测者移动。音源发出声波的频率为 f_0、波长为 λ_0、周期为 T_0。第一个波面（wave front）发出后，就以速度 v（≈340m/s）进行移动。

Download at www.Pin5i.com

我们要使用的速度值等于移动物体的速度在连接音源和观测者之间的轴上的投影。T_0 秒后，音源发出另一个波面。此时第一个波面的传播距离是 $v*T_0=\lambda_0$。而音源的移动距离是 V_s*T_0。我们可以使用等式 6.5.1 来计算所感知的波长。

$$\lambda' = v*T_0 - V_s*T_0 = (v - V_s)*T_0 \tag{6.5.1}$$

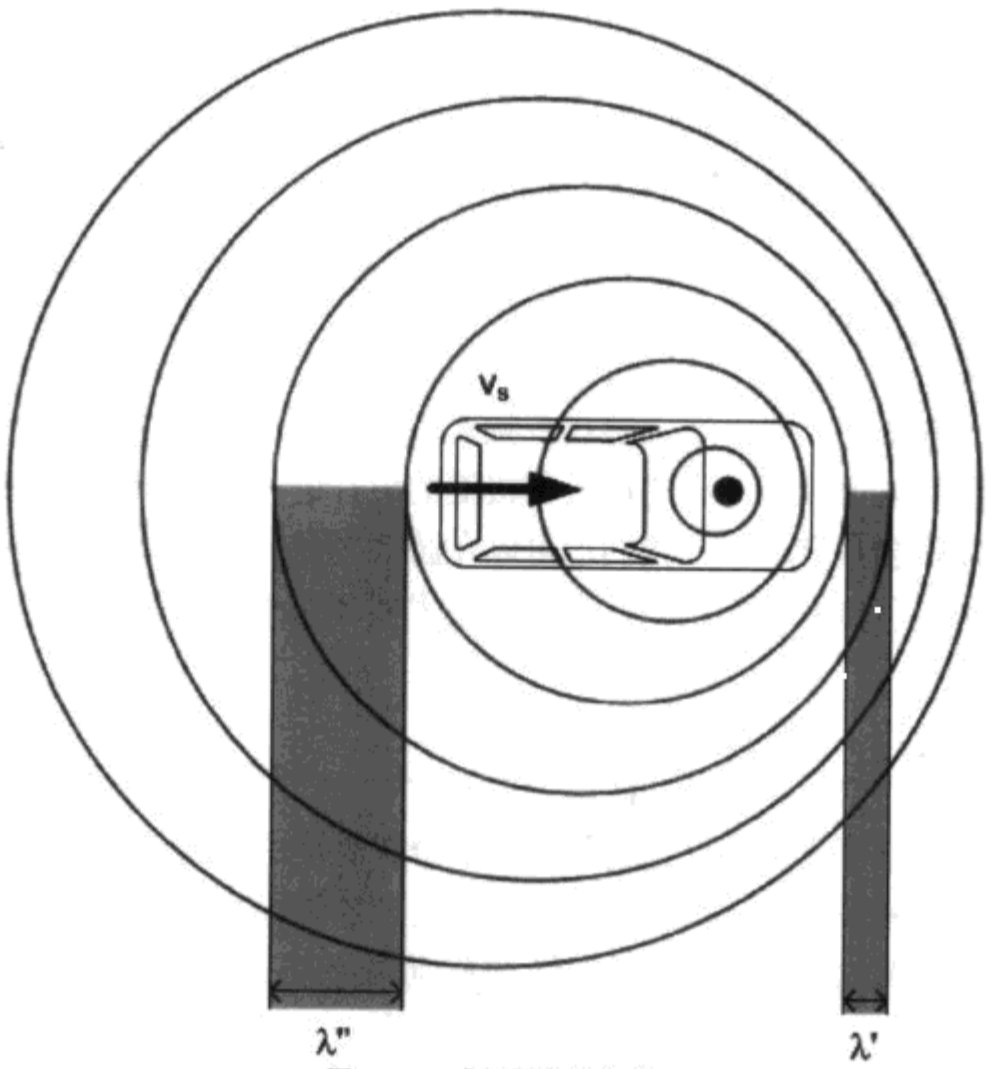


图 6.5.2 感知频率的计算

通常，如果音源和观测者之间相互接近，速度值就是正的；当它们越来越远时速度值则是负的。在后一种情况下，速度将会带一个负号，而 λ'会因此变大。通过把 λ'除以 v，我们可以得出等式 6.5.2 和等式 6.5.3。

$$T' = T_0 \frac{v - V_s}{v} \tag{6.5.2}$$

$$f' = f_0 \frac{v}{v - V_s} \tag{6.5.3}$$

当观测者也在移动的情况下，我们可以使用同样的方式得出等式 6.5.4 和等式 6.5.5，其中音源速度是 V_s，观测者的速度是 V_o。

$$f' = \frac{v + V_o}{v - V_s} f_0 \tag{6.5.4}$$

$$T' = \frac{v - V_s}{v + V_o} T_0 \tag{6.5.5}$$

从实践角度来看，多普勒效果和声音回放速度的提高/降低相关，就像唱机中的碟片一样。如果它转得快一点，声音的频率就会显得比较高；如果它转得慢一点，声音的频率就会显得比较低。这和我们创建受到多普勒效果影响的声音时所采用的原理是一样的。

Download at www.Pin5i.Com

示例

我们使用一个简单的例子来介绍这一理论。整个布局就像图 6.5.3 所表示的那样。只有声源在移动。等式 6.5.4 和等式 6.5.5 中所需要的速度就是声源速度在声源和观测者之间连线上的投影大小，其正负值和前面所描述的一致。这里，由于只有声源在移动，因此速度就等于声源和接收者在距离上的改变。在这个例子中，我们使用距离来计算速度，因为这样一来我们就可以很方便地求出速度的微分表达式。在实际情况下，使用速度向量在两者连线上的投影会更为简便。

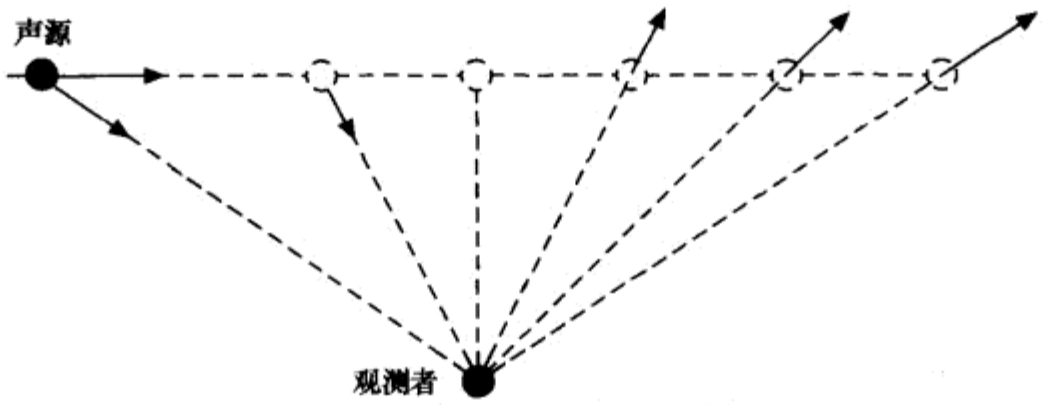


图 6.5.3　只有声源移动时的多普勒效果

声源的位置 P_s 和接收者的位置 P_o 由等式 6.5.6 和等式 6.5.7 定义。

$$P_s = (10 \cdot t, 1, 0) \tag{6.5.6}$$

$$P_o = (0,\ 0,\ 0) \tag{6.5.7}$$

我们使用等式 6.5.8 计算距离。

$$D = \sqrt{(10 \cdot t)^2 + 1} \tag{6.5.8}$$

等式 6.5.9 中是计算出的距离。等式中的负号来自于我们所选择的规则。事实上，当距离 D 减小时，它的导数将会是负的。但是我们的规则要求当声源和接收者之间相互接近时速度是正的，因此我们使用距离导数的相反值。

$$V_{s/o} = -\frac{dD}{dt} = \frac{100 \cdot t}{\sqrt{(10 \cdot t)^2 + 1}} \tag{6.5.9}$$

图 6.5.4a 中是声源的速度，图 6.5.4b 则表示了频率变化。读者可以在随书光盘上找到带有多普勒效果的 WAV 文件。

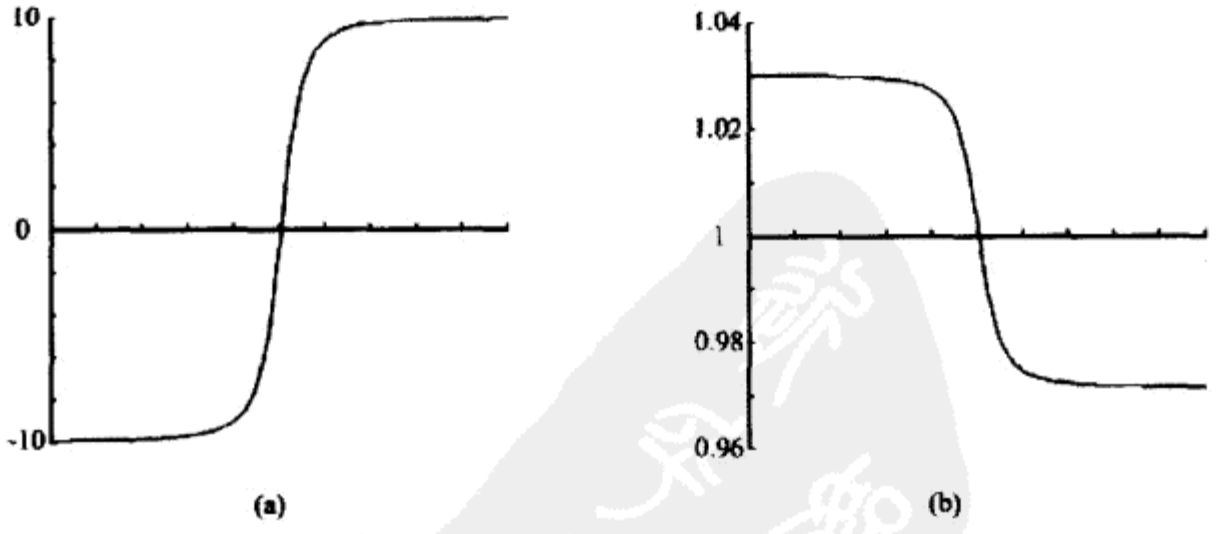


图 6.5.4　（a）相对速度　（b）频率变化

Download at www.Pin5i.Com

6.5.2 编写多普勒效果程序

一段音频记录由保存在数组中的一系列数值表示。这些数值之间间隔一段特定的时间，我们把这一时间标为 T_s，这就是采样周期，等于采样频率 F_s 的倒数。采样频率通常是 44.1kHz 或 22.05kHz。为了运用多普勒效果，我们必须修改所发出声音信号的回放速度。这对应于节拍（temporal）的延长和缩短。我们必须对最终被听到的信号采用和原始信号一样的频率进行采样。对于特定的信号和采样频率来说，其采样时间和在采样数值数组中的下标之间存在一个简单的关系（参见等式 6.5.10）。这里，i 表示数组的下标，它必须是一个整数。

$$t = i \cdot T_s \tag{6.5.10}$$

正如前面所介绍的，改变频率对应于节拍的延长和缩短。等式 6.5.5 告诉我们这一比率 T'_1/T_0 仅仅依赖于 v（音速）、V_s 和 V_o。因此频率比率/周期比率和信号本身是独立的。

为了改变频率，我们根据等式 6.5.4 来对节拍进行简单的修改。我们使用等式 6.5.11 来实现这一变换。根据等式 6.5.10，当原始信号和最终所听到的信号具有相同的采样频率（也就是两者的 T_s 相等时），我们可以用 i 来代替 t。

$$t_{\text{src}} = \frac{v + V_o}{v - V_s} \cdot t_{\text{dst}} \equiv i_{\text{src}} = \frac{v + V_o}{v - V_s} \cdot i_{\text{dst}} \equiv i_{\text{src}} = R \cdot i_{\text{dst}} \tag{6.5.11}$$

具有 src 后缀的时间和下标变量与发出的信号有关；具有 dst 后缀的变量则和最终听到的信号有关。现在我们完全可以使用等式 6.5.12 根据原始数值来得到最终所听到的信号。

$$S_{\text{dst}}(i_{\text{dst}}) = S_{\text{src}}(i_{\text{src}}) = S_{\text{src}}(i_{\text{dst}} \cdot R) \tag{6.5.12}$$

这里，i_{dst} 是一个整数而 R 不是，因此 $i_{\text{dst}} \cdot R$ 也不是一个整数。这样一来，$S_{\text{src}}(i_{\text{dst}} \cdot R)$ 并不总是存在。为了解决这一问题，我们必须使用插值。本节将会介绍一种可行的技术，它不仅在计算时非常简捷，所得到的结果也很理想。

和这一算法等价的代码是：

```
void apply_doppler( int * src_signal, int *dst_signal,
    float ratio, int frame_sample_count )
{
    for( int i=0; i< frame_sample_count; i++)
    {
        dst_signal[i] = src_signal[i*ratio];
    }
}
```

如前所示，i*ratio 并不总是一个整数。我们将在下面解决这一问题。

6.5.3 线性插值

图 6.5.5a 中是原始信号，图中的点表示采样距离。我们仅能使用原始信号的这些采样值。

我们使用上一个和下一个值来插出所需要的值。我们会使用数组的下标，而两个数值下标之间的距离等于 1。这一算法会根据需要对下标进行舍入，然后根据在下标距离之间位置

Download at www.Pin5i.Com

的比例计算出一个值。下面是这一算法的代码。

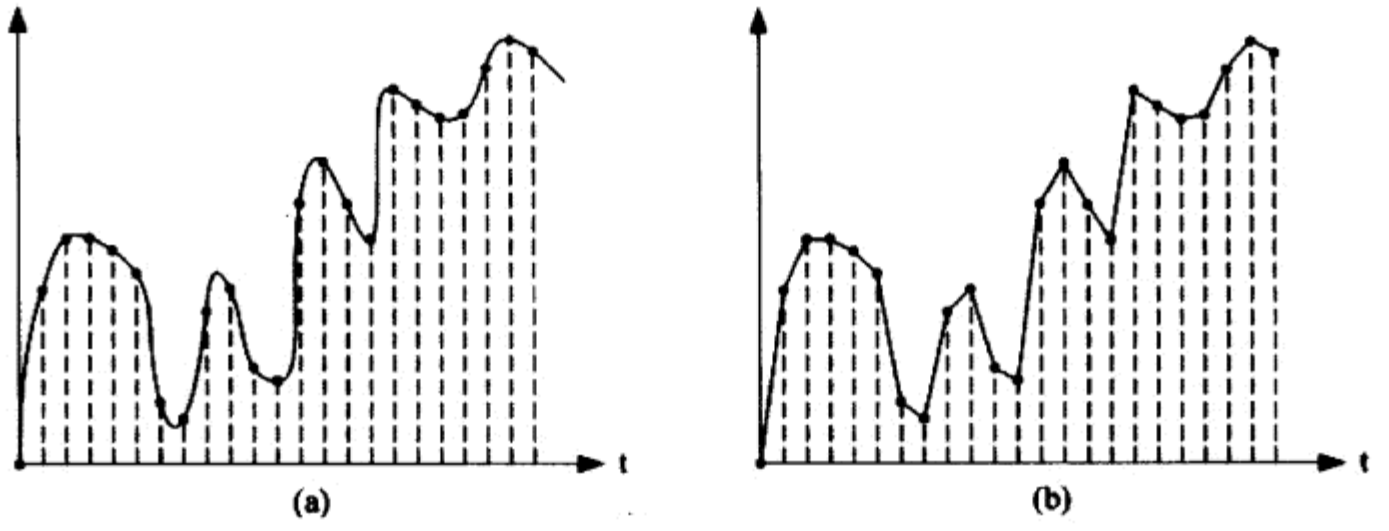


图 6.5.5　(a) 采样信号　(b) 线性插值

```
int interpolate( int* source_array, float wished_index )
{
    int index = (int) floor( wished_index );
    float delta = wished_index -index;

    return (int) ( source_array[index] * ( 1.0f - delta )
        + source_array[index+1] * ( delta ) );
}
```

前面示例代码中的

```
dst_signal[i] = src_signal[ i*ratio];
```

变成了：

```
dst_signal[i] = interpolate( src_signal, i*ratio );
```

6.5.4　根据 R 来计算下标

根据等式 6.5.11，i_{dst} 是由 i_{src} 和 R 计算出来的。使用这种方法时，每个采样都需要一次乘法运算。由于下一个采样的 i_{dst} 总是比上一个增加 1。根据等式 6.5.13，我们只需要在 i_{src} 上加上 R 就行了。这样一来，这个乘法就变成了一次加法。

$$(i+1)_{src} = R \cdot (i+1)_{dst} = R \cdot (i_{dst}+1) = R \cdot i_{dst} + R = i_{src} + R \qquad (6.5.13)$$

于是代码变为：

```
void apply_doppler( int * src_signal, int *dst_signal,
    float ratio, int frame_sample_count )
{
    float isrc;
    for( int i=0, isrc=i; i< frame_sample_count; i++, isrc +=
        ratio)
```

Download at www.Pin5i.Com

```
    {
        dst_signal[i] = interpolate( src_signal, isrc );
    }
}
```

6.5.5 非恒定速度

虽然有些情况下声源和接收者之间的相对速度是一个常量，但通常情况下并不是这样。由图 6.5.4a 可以得知声源的相对速度是变化的，因此它不是一个常量。而根据等式 6.5.10，我们可以知道频率比率也在采样之间不断变化。为每个采样都计算出这一比率可以给出最好的结果。然而，这一方法的 CPU 开销太大了，因为对每个采样都需要进行一次除法。

由图 6.5.4b 可知，前面这个例子中的比率（ratio）是不断变化的。为了节约大量的运算，我们将对比率函数分段线性化（linearize）。我们使用一系列长度很小的线性区间来对这一比率进行近似。在每一帧中，我们都会计算出其开始和结束时的比率。然后对这两个值进行线性插值以求出其他值。因为每一帧的长度都很短，这样的线性化不会带来任何问题，也不会在回放时被注意到。如图 6.5.6 所示，为了让这一现象更为明显，我们把每帧的长度都增加了。在图中我们也可以看到每一帧的长度并不是一个常数，正如游戏引擎在运行时那样。

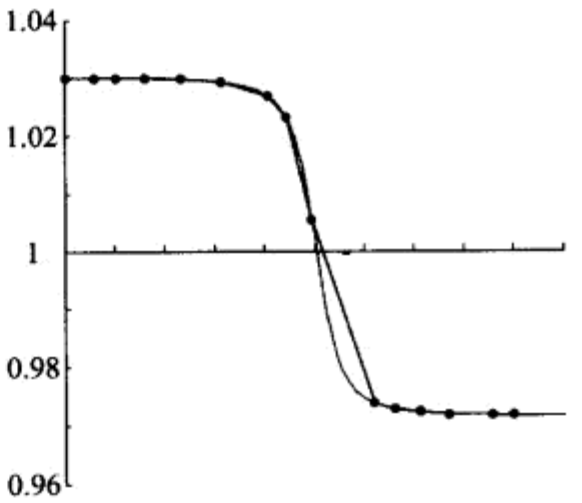


图 6.5.6 频率比率分段线性化

于是代码变为：

```
void apply_doppler( int * src_signal, int *dst_signal, float
ratio_start, float ratio_end, int frame_sample_count )
{
    float isrc;
    float delta_ratio = ( ratio_end - ratio_start )/
        (sample_count-1);
    float ratio = ratio_start;

    for( int i=0, isrc=i; i< frame_sample_count;
        i++, isrc += ratio)
    {
        dst_signal[i] = interpolate( src_signal, isrc );
        ratio += delta_ratio;
    }
}
```

完整的代码在配套光盘中。

我们可以通过降低每次调用函数时所处理的采样点数量（frame_sample_count）来拔高转换精度，最好是能够根据等式 6.5.10 来计算每一个新采样的比率。

Download at www.Pin5i.com

6.5.6　信号对齐（Aliasing）

对音频信号进行变换并不是完全安全的，我们仍然受到采样理论的约束。进行拉伸和缩短其实就是对频率空间（frequential space）进行变换，而加快声音的回放速度会拉伸频谱中的高频部分。图 6.5.7 中展示了一个声音的频率分布，它还使用虚线表示了这个声音经过加速运算后的频率分布。减慢信号的速度也会产生同样的效果，只不过频谱被压缩了。

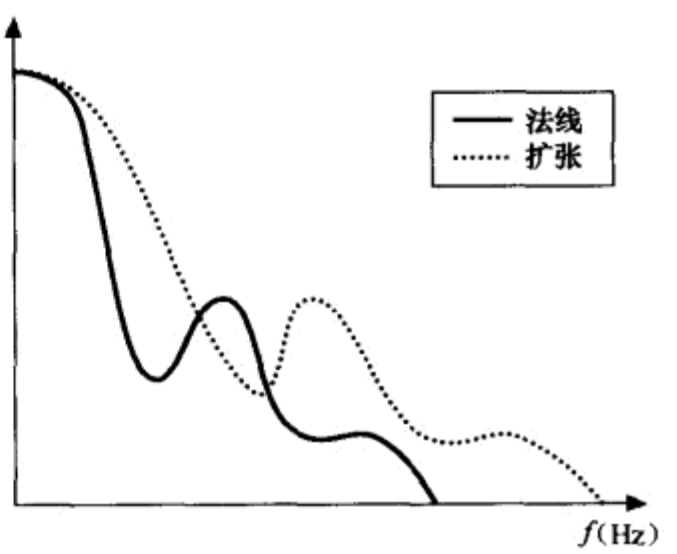


图 6.5.7　频率范围的拉伸

我们还会遇到一个问题，就是信号对齐。根据内奎斯特定律，如果信号的部分频率高于采样频率的一半，这一部分就会表现为 0 到采样频率一半中的某个频率，从而在信号中加入异常的声音（噪声）。读者可以在[Wiki05]中找到关于信号对齐的实例及其数学背景。

在我们所讨论的情况中，如果信号被加速得较大，频率范围就会被扩张并且达到信号对齐的极限。我们必需确保这一比率的最大值不能达到极限以避免出现信号对齐问题。因此，我们应该把信号的频率分布限制在一个已知的值以控制可能出现的问题。

然而，如果声源以 100m/s 的速度移动，比率 R 将等于 0.7（参见等式 6.5.14）。这一比率会保持在 1.0 附近，仅当物体移动得非常快时才会达到极限条件。如果声音的频率分布和内奎斯特定律的理论极限并不接近，那么永远都不会产生这一问题。

$$R=\frac{v+V_o}{v-V_s}=\frac{340+0}{340-100}\approx 1.4$$
$$R=\frac{v+V_o}{v-V_s}=\frac{340+0}{340+100}\approx 0.8 \qquad (6.5.14)$$

6.5.7　实现

ON THE CD

配套光盘上所提供的实现试图模仿游戏引擎中的真实情况。我们以 40ms/帧（25 帧/s）的间隔对声音信号进行处理。我们还使用了一个随机值对每帧的长度进行少量改变以模拟游戏引擎中真实情况下帧长的变化。代码中的主要方法（apply_doppler）可以直接用在真实游戏中。

6.5.8　总结

多普勒效果就是指这样一种物理现象：当声源和观测者之间存在相对运动时会改变观测者对声音的感知。这一现象普遍存在，但是仅当速度很快时才会被察觉。本节提出了一种简

Download at www.Pin5i.Com

单高效的技术来模拟这一效果。这一技术不仅让我们能够以很低的代价来提高声音的真实度，还可以在制作游戏音效中使用。尝试一下吧！

6.5.9 资源

[Wiki05] Wikipedia, "Aliasing." December 2004. Available online at *http://en. wikipedia.org/wiki/Aliasing.*

Download at www.Pin5i.Com

6.6　仿造实时 DSP 效果

Robert Sparks, Radical Entertainment
sparks.robert@gmail.com

想象一下在视频游戏中的某个房间里，有一个正在播放音乐的收音机。当玩家与这个收音机在同一个房间时，它的声音听上去清晰明亮。然而，当玩家在这个房间外面时，在真实情况下收音机的声音将会有所衰减并且变得模糊。这样的模糊和衰减是一种动态的实时效果，它被称为“隔离（occlusion）”。这种效果为下一代的游戏机所支持。但是，当目标硬件还不支持我们想要实现的实时 DSP 效果或是我们想要实现某些特殊的功能时，我们该怎么办呢？

我们可以仿造（fake）它。

本节将介绍怎样在任何平台上对任何效果进行仿造。这一技术虽然实现起来非常方便，但是极为强大。

6.6.1　仿造

对实时效果进行仿造的关键在于，大部分工作都是在我们创建游戏内容时预先完成的。我们会创建经过特殊设计的、包含多个声道的声音文件，而不是像通常一样创建单声道的声音文件。在这个多声道的文件中，我们使用一个声道来保存未经处理的原始声音，我们在其他声道中保存对原始声音进行处理后得到的结果。譬如说，一个声道听上去可能就像未经处理的收音机声音一样，另一个声道听上去就像收音机在墙壁的另一边播放，还有一个声道听上去可能就像收音机在水底播放一样。

当我们在游戏中播放多声道声音文件时，我们把每个声道都看成是一个处于某个特定位置上的声音，这个位置就是与这一声音相连的对象的位置。每个声道都使用独立的音量进行播放。因为所有声道的播放都是同步的，并且它们处于相同的位置，它们发出的声音组合在一起形成了这一对象的整体声音。当我们改变这些声道的音量时，我们也改变了它们所组成的声音。使用这种方法得到的声音和硬件生成的 DSP 效果惊人地相似。

6.6.2　例子：收音机在房间中播放音乐

为了更好地理解这一技术，让我们回到本文开始时所介绍的例子上。我们不仅会在下面对其进行介绍，还在配套光盘上提供了示例代码和 WAV 源文件以便读者可以亲手进行演示。

Download at www.Pin5i.Com

想象一下在一个视频游戏中的某个房间里，一台收音机正在播放音乐（如图 6.6.1 所示）。开始时收听者处于收音机所在的房间里，收音机的声音听上去清晰明亮。当收听者离开这个房间时，收音机的声音很快就会衰减并且变得模糊。

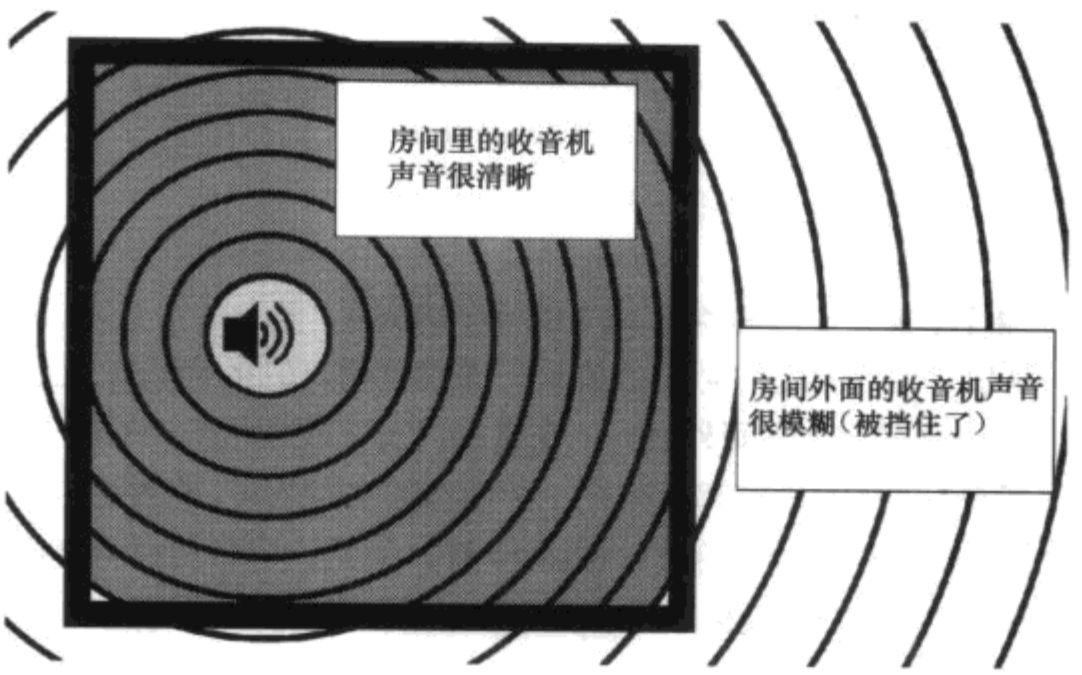


图 6.6.1 房间里的收音机在房间里听起来非常清晰，而在房间外面听上去就像声音被挡住了一样

要实现这个例子，我们需要一个具有两个声道的声音文件。这个文件的一个声道中包含了在房间内部所听到的、未经处理的收音机声音。在另一个声道中则包含了在房间外面听到的收音机声音（参见图 6.6.2 所示）。

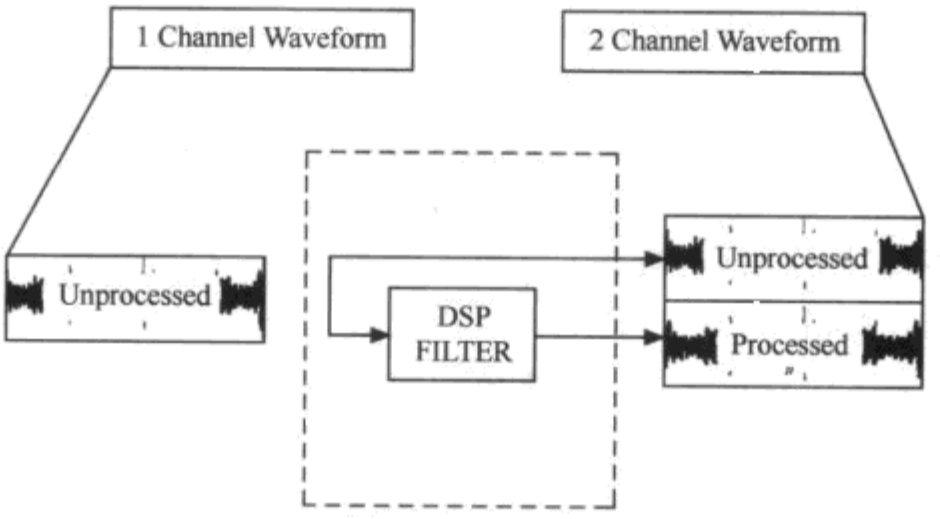


图 6.6.2 制作一个多声道的文件

当我们在游戏中播放这个表示收音机的多声道文件时，我们把每个声道都作为一个处于特定位置的声音来播放，它们的位置就是收音机对象所处的位置。因为开始时收听者处于收音机所在的房间里，我们提高未经处理的声道的音量并且降低经过处理的声道的音量（参见图 6.6.3 中收听者位置 1）。收音机的声音听上去清晰而响亮，因为我们只听到文件中未经处理的那个声道。

当收听者离开这个房间时，我们让未经处理声道的音量淡出，并且让经过处理的声道淡入（参见图 6.6.3 中收听者位置 2）。收音机的整体声音很快就变得像声音被挡住一样；现在我们只能听到那个经过处理的声道。

Download at www.Pin5i.Com

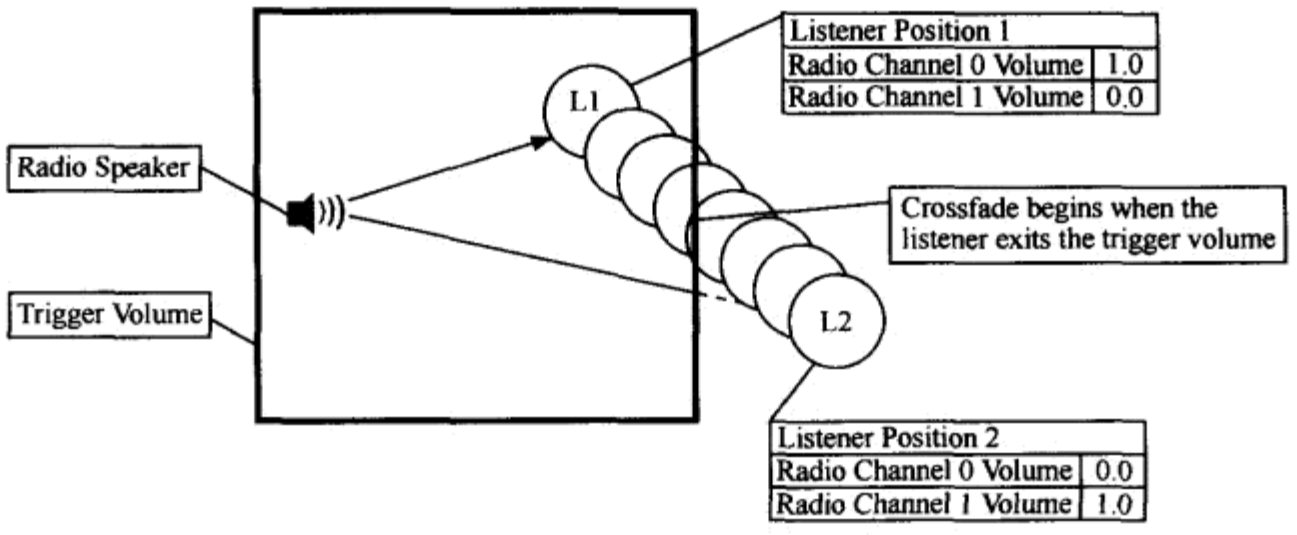


图 6.6.3　通过控制不同声道的音量来进行阻塞处理

在这个例子中，从一个声道到另一个声道的淡入淡出可以在固定的时间内完成，这样做的听觉效果很好。

6.6.3　声音能量恒定曲线（Constant Power Volume Curve）

当我们调整多声道文件中各个声道的音量时，我们希望玩家可以听到这一文件所播放出的整体声音的改变，但是并不能意识到我们在对声音进行替换。为了隐藏这一替换，我们必须确保在任何时刻所有声道所输出的总能量和单一声道所能输出的最大能量相等，如图 6.6.4 所示。在实现这里所介绍的技术时，我们必须确保所有声道的总能量是恒定的。

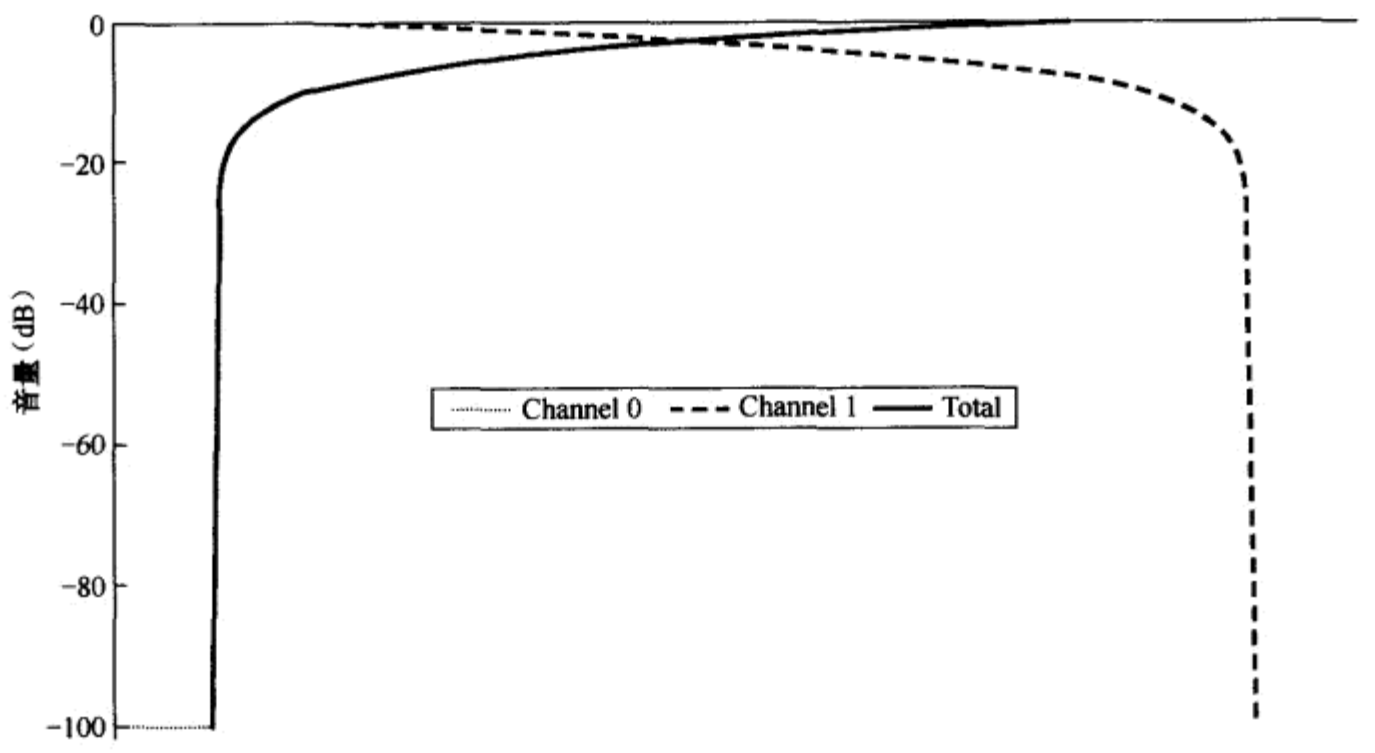


图 6.6.4　两个声道音量输出的总能量保持恒定

6.6.4　对声道音量进行进一步控制

在我们所使用的简单示例中，我们仅仅在收听者离开一个触发范围时通过一个固定长度的淡入淡出来对音量进行控制。这是一种非常简单有效的技术，尤其适用于收听者在从一个

Download at www.Pin5i.com

环境移动到另一个环境时会跨越一个定义良好的限制的情况。

通过把收听者在某个属性上的变化映射到淡入淡出的变化上，我们可以创建更为精妙的效果。这样一来，淡入淡出也可以反映其他属性的变化，譬如说收听者移动的速度。

我们可以使用任意属性来控制声道的音量，而不仅仅是收听者的位置。譬如说，我们可以使用收听者的生命值来控制音量。

6.6.5 在 DirectSound 中播放多声道文件

使用 DirectSound 来实现本文所介绍的技术需要进行特殊的处理。DirectSound 不支持把包含多个声道的文件作为一个处于特定位置的声音来播放。在本书配套光盘中所附的例子提供了对 DirectSound 的包装类（Wrapper Class）来提供这一额外功能。

6.6.6 代价和好处

这一技术最主要的好处在于它可以在任何目标硬件上实现我们可以想象到的任何效果。我们可以使用任意方法来创建这些效果。譬如说，我们可以使用自然的方法来创建特效。我们可以使用多个处于不同声学环境（acoustical environment）中的麦克风同时录制声音效果。

我们也可以使用多个单声道文件而不是一个多声道文件来实现这一技术。然而，使用多声道文件可以使得声音在硬盘上动态载入时不需要进行额外的寻道。

这一技术的代价是，我们必须在内存中保留多个版本的声音并且同时对它们进行播放。因此，与通常情况相比，采用这种技术时内存和硬件声音（hardware voices）会消耗得更快。我们很可能不能在所有时间对游戏中的所有声音都使用这种技术。我们可以把注意力放在少数突出的、使用其他方法无法实现其特效的声音上，从而创建出最生动的结果。

6.6.7 总结

使用本节所介绍的技术，我们可以在任何目标硬件上仿造任何 DSP 效果。它为声音制作所带来的灵活性可能是其他方式无法实现的。

6.6.8 致谢

感谢 Bill Gates、Corinna Hagel 和 Borut Pfeifer 对本文的写作做出的帮助；感谢 Tim Hinds 教会我很多声音编程知识。

Download at www.Pin5i.Com

第 7 章

网络及多人在线

Download at www.Pin5i.Com

简　介

Scott Jacobs，Virtual Heroes
scott@escherichia.net

一直以来，网络/多人在线专家所追求的东西看起来一直都没有变过。带宽总是看起来唾手可得，但是你只要接近它，它就会像海市蜃楼一样消失得无影无踪。新的图形技术以及模拟技术对数据量的需求越来越高，与此同时游戏又在不断地向手机等新的平台延伸，这一切都改变了人们对带宽需求和处理能力的期望。Thomas Di Giacomo 和他的合著者为如何解决 3D 动画数据的带宽和处理能力瓶颈提供了一些方法，并且这些方法依旧是凭借现有带宽来实现数据压缩以及提高计算潜力。

大规模多人在线游戏一直是游戏网络专家们所热衷的领域，他们所关心的问题涵盖很广，细到如何最大化的利用几十个比特，大到如何设计一个复杂的系统等问题。而我们这一节所介绍的技巧对这大小两个方面都会有所涉及。Viknashvaran Narayanasamy 和他的合著者针对 MMOG 的一个高级系统架构阐述如何实现一个复杂的系统，力求创建出系统的复杂性和行为的自然性。Yongha Kim 将针对我们所面临的带宽制约，在如何优化利用每一个比特上给了我们一些建议，并同时介绍了一些适合 MMO 游戏的关于如何创建系统带宽和特殊游戏对象识别器等方法。

而 Peter Smith 的建议把我们的注意力从系统架构及比特利用上移开——毫无贬低之前技巧之意——并转移到如何加速游戏开发早期的调研阶段上来。他介绍了一个 MMO 的游戏原形制作方案，关于这个方案中的系统架构、比特的优化利用和很多复杂的细节都通过利用一个现有的、可扩展的 MMO 系统（Linden Lab 的 Second Life）得以实现。

如果要是玩家互相之间不能连通的话，设计、架构、原形以及制作开发将不会实现哪怕一分钟的多人在线游戏乐趣。作为对 Jon Watte 在《游戏编程精粹 5》（中文版已由人民邮电出版社出版）中的补充，Larry Shi 和 Ying Sha 提供了一项技巧。将 TCP 加入协议家族中将很有可能通过现在大部分的商业 NAT 硬件来实现。

以下这些技巧适用于游戏开发专业人士所面对的很多问题。无论这部分的读者是首席程序师、游戏设计师或者网络软件工程师，你都将会找到指导你的甚至能直接应用到的技巧。

Download at www.Pin5i.Com

7.1　3D 动画角色数据的动态自适应流

Thomas Di Giacomo, HyungSeok Kim,Stephane Garchery 和 Nadia Magnenat-Thalmann, MIRA 实验室, C.U.I; 日内瓦大学
Thomas@miralab.unige.ch
kim@mirabal,unige.ch
Stephane@ miralab.unige.ch
thalmann@miralab.unge.ch
Chris Joslin, 渥太华卡尔顿大学，信息技术学院
cjoslin@connect.carleton.ca

7.1.1　简介

在线游戏出现已经很久了，并且随着最近的网络带宽的增加，其受欢迎的程度也在成几何级数的增长。现在的在线游戏的规模可以从在线人数几十个到几百几千不等，这就预示了一个很大的 3D 数据使用量。鉴于 3D 数据所占用的空间和带宽的大小，它的传输必然被限制，因此现在大多数的在线游戏会使用预存储的 3D 模型而不是动态地去传输 3D 模型。这一约束会给互动造成很大的局限性，不过这也是 3D 传输内容的一个特性。如果使用与音频和视频数据传输类似的方法（尽管 3D 数据通常情况下会更小一些），将传输内容进行自适应压缩，有可能是压缩传输数据、节约带宽需求的一种有效手段。

基于应用在 3D 数据上的 MPEG 压缩算法，本章将会讨论怎么为互动角色建立可缩放数据，以及根据现有带宽、客户端支持的多边形数以及角色在场景中重要性等因素，如何更加合适、更加动态化地减低 mesh 和动画数据传输中的延时。我们不但在考虑视觉效果的前提下控制数据的大小，同时，我们也尝试着为了实现更多的同屏角色和 PDA 调光设备而控制渲染的复杂性。

7.1.2　背景介绍与相关方法

首先，我们需要一些能够实现多重解析度的方案，并以自适应的方式来表现几何和动画数据。然后，基于这个可缩放大小的传输内容，按照实际情况动态地为自适应及决策机制选择最合适的解决方案。下边的内容我们将讨论一些能够以多重解析度来表现几何及动画信息的一些方法，也就是对现有工作的数据的自适应改编（特别是与 MPEG 压缩相关的一些方法，其中很多活动都是针对流自适应改变）。

Download at www.Pin5i.Com

1. 本地计算机图形的 LOD

从一些关于计算机图形技术的参考资料中我们可以举出很多的 LOD（Level of Design）方面的著作，例如[Hopper96]中建立多解析度模板的方法。这个方法通过某一标准，例如摄像机的距离，来精炼或者简化多边形 mesh，以达到在渲染时节约计算量的目的，或者为了达到某些时间上的要求。[Garland97]中也曾经提到过一种利用误差曲面来简化 3D 模型表面的方法。作为这一方法的实践应用，[Fei99]中应用 LOD 建立人体 mesh，[Seo00]中建立人脸 mesh。要注意的是，在建立角色（或其他类似的物体）时一个需要考虑的主要问题是，为角色的关节预留出专门的多边形量（或者面部表情的控制点）以确保动画制作的质量。另外一个简化多角色即时渲染的方法就是将 3D 对象转化成 2D 的，这一方法在[Techia02]中被提到过。

2. 编码媒体的自适应改编

尽管 3D 数据一般都是本地保存，但是基于流的数据传递也是网络游戏的一种比较好的解决方案。因此，数据压缩就成了游戏系统的一个重要组成部分。在这里我们不会提供一种过于详尽的方案；而是直接使用一些已经很成熟的标准化方案，并通过这些方案中的工具和一般性格式来处理可协同操作的数据内容。在 3D 领域，这些方案包括：VRML、X3D 和 U3D。另外还有 MPEG，尽管 MPEG 最早是为了压缩视频和音频文件的，但是 3D 图形技术的发展也通过将 BiFS（Binary Format For Scenes）整合到 MPEG 核心的规则，借鉴了 MPEG 很多概念（参考[Walsh02]关于 MPEG-4 的详细介绍），形成了以 SNHC（Synthetic and Natural Hybrid Coding，合成自然混编码）组来定义 AFX（动画结构外延）的标准。就像[Preda04]中所说的那样，这个标准是专为骨骼动画设计的，但是，MPEG-4 中也有一些关于面部动画的规则。因此，我们这里为什么会从 MPEG-4 着手的原因可以归纳为：未经压缩的 3D 数据在 MPEG-4 中可以粗略地认为与 VRML 类似；MPEG 可以提供一整套媒体框架和有效的压缩方案；另外 MPEG-7 和 MPEG-21 还允许符号性的媒体对象，而这一过程中所用到的机制，就如自适应改编一样。

当考虑到 3D 图形传输的时候，比特流的自适应改编就成了极为重要的一个因素，因为它为传输提供了很多的可能性，例如单数据内容对多设备传输的实现，或者按需求（网络带宽与终端能力限制）压缩数据内容的复杂性和大小。比特流自适应改编需要为改编准备一个可缩放编码数据的解释。[Amielh02]中曾经提到过通过 gBSDL（generic Bitstream Description Language，通用比特流说明语言）来完成这一解释。自适应改编可以通过不同的媒体形式来运作，例如[Aggarwal01]中所提到的音频或者[Kim03]中所提到的视频。但是在改编中要充分考虑到每种媒体格式的特点，以保证自适应改编在数据内容的质量和改编的准确性上最佳效果。事实上，通过处理可缩放的 3D 图形数据来完成传输早已经取得了方向性的进展了——例如[Van Raemdonck02]和[DiGiacomo04]。

7.1.3 处理可缩放 3D 数据的准备和实施

在这一部分，我们会讨论怎样准备可缩放的几何和动画数据。这是从可缩放表现进行数据改编的第一个必要步骤。

Download at www.Pin5i.Com

1. Mesh 的可缩放表现

我们利用一个聚类分级模型来设计这个自适应改编方法。假设对所有数据进行聚类，这样就会使一种复杂的情况可以通过一组简单的聚类来实现。如果 V_n 是一组顶点，F_n 是一组面，那么复杂的 mesh $M_n(V_n, F_n)$就可以通过简单的 mesh 的排列来表示 M_{n-1}，....M_1，M_0。

这样一个代表简化过的排列的多重分辨率模型就会生成或至少会生成一系列的顶点 V 和面 M，合集可以记作“+”，交集可以记作“–”：

$$V=\sum_{i=0}^{n}V_i\ ,\quad M=\sum_{i=0}^{n}M_i \tag{7.1.1}$$

V 和 M 可以分区到一组聚类中。第一类是一组从第 i 层 mesh 转移出的用来生成第 i–1 层的 mesh 的顶点和向量，记作 $C(i)$。另外一类是通过简化重新生成的一组顶点和面，记作 $N(i)$。由此，一个第 i 层的 mesh 就是：

$$M_i=M_0+(\sum_{j=1}^{i}C(j)-\sum_{j=1}^{i}N(j)) \tag{7.1.2}$$

类似的处理可以通过很多种操作器来进行简化，例如，大规模处理、区域合并和细分。在这里我们采用 half-edge collapsing 操作器和曲面误差矩阵来处理（如图 7.1.1 所示）（见[Garland97]）。

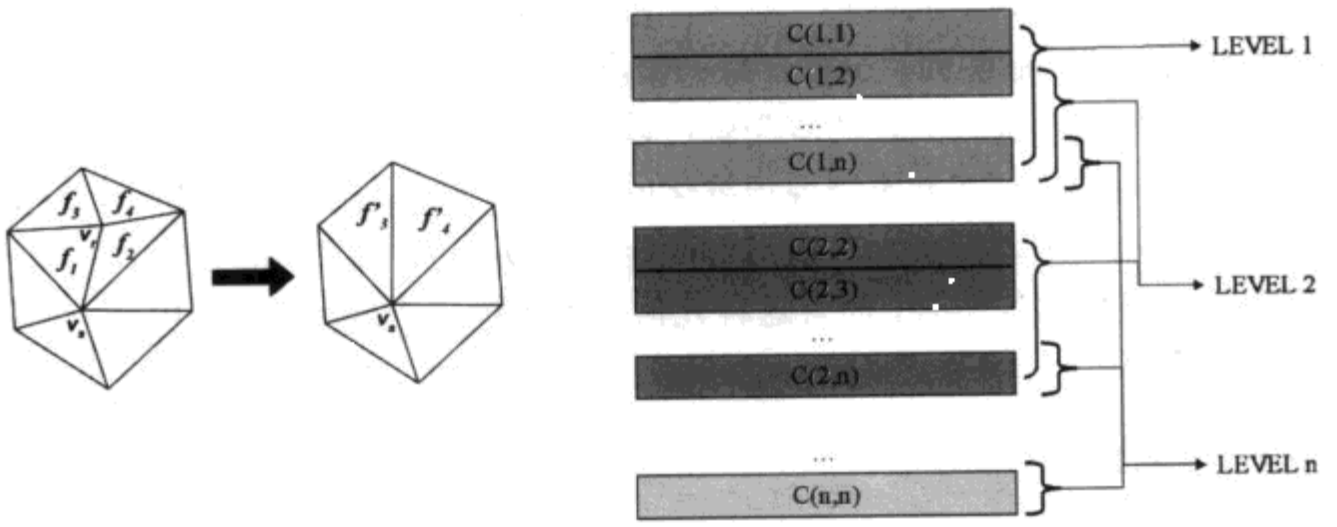


图 7.1.1　edge collapsing 示意图及聚类结构

通过一个 edge-collapsing 操作器，一个边缘（v_r，v_s）就被压缩到了顶点 v_s。通过图 7.1.1 你能发现，面 f_1，f_2 从 mesh 中被移除了，而 f_3，f_4 被转换到 f_3' 和 f_4' 中，这一聚类就可以定义为

$$C(i)=(f_1,f_2,f_3,f_4) \tag{7.1.3}$$

和

$$N(i)=(f_3',f_4') \tag{7.1.4}$$

如果想要对等式 7.1.2 进行评估，我们还需要一些相关的操作：设定一些交集和并集。由简化的特性可以得出，$N(i)$是 M_0,$C(1)$,...,$C(i-1)$ 的并集的子集。通过对这一特性的使用，聚类 $C(i)$ 可以再次聚类成为一组 $C(i,j)$，属于 $N(j)$，这里 $j>i$ 且 $C(i,j)$ 不包含于任何 $N(j)$。这个和 M_0=$C(0)$ 时的 M_0 是一样的。因此第 i 层的 mesh 就可以通过下面这个等式表示，这个等式也需要对选择过程进行简化。

$$M_i=\sum_{k=0}^{i}(C(k,k)+\sum_{j=i+1}^{n}C(k,j)) \tag{7.1.5}$$

Download at www.Pin5i.Com

最后一个步骤就是通过排序来降低在自适应改编过程中做选择的次数。对定点数据进行排序会是比较简单易懂的，因为 edge-collapsing 操作器（v_i，v_s）能够保证 $C(i)$ 与顶点 $v_iC(i, i)$ 有单一联系。通过对一系列顶点 $C(i, i)$ 进行 i 的升序排列，关于层 i 的顶点数据在选择的时候就能够实现整块的连续数据，$v_0,v_1,....v_i$。为了对面进行索引设定，每一个 $C(i)$ 都依照 $C(i, j)$ 进行 j 的升序排列。这样的话，对于层 i 的自适应改编就会最多有 $3i+1$ 种选择，或者最多 $2n$ 种移除方式，这里的 n 是层的数量。

到此为止，我们所讨论的都是指用到了顶点位置和面相关信息的情况。在 mesh 中，还有一些其他的属性必须要考虑到，例如法线、颜色，以及贴图的调整。因为这些属性都和顶点关系密切，因此可以用一种类似于处理顶点的方法来处理这些元素。不过以下情况例外：（1）对于同一个元素，有两个或更多的顶点位置取值相同。或者是（2）同一顶点对于该属性所取的值在两个或两个以上。在以上两种情况中，每一对顶点/面对于属性取值都是存在特殊映射关系的。对于一个相关于聚类 $C(i)$ 中一组面/顶点的某个属性，它的映射应该属于（v_i，f_j），其中 $v_i \in C(i)$。如果某一个属性 p 属于多于一个的顶点，例如（v_i，f_1）$\rightarrow p$，（v_i，f_{21}）$\rightarrow p$，p 被限定在聚类 $C(j)$ 中，且 $j<i$。通过这种排序，当有一个顶点还是以 p 作为属性，那么 p 就一直处在活动状态。因此，对于每一层 i，都会有一个可用的聚类。

图 7.1.1 的右半部分显示了一个了聚类的概念。每一个聚类都有一套顶点和顶点属性，例如定点法线、颜色，及贴图的调整，等等。根据顶点信息，这一聚类还包含了一系列索引面、法线面、色彩面，及贴图面。另外，每一聚类还包含该聚类的材质、贴图的一些子片断。因此，选择每一层只需要选择大块的聚类就可以了。

在很多实际应用中，其实并不需要很细化的生成每一层。在少数多边形上的变化并不会导致在最终表现或质量上差别太大。通过对既定表现的使用，模型系统可以生成一组模型或应用程序所需要的任何层的组合，而且这种几何上的可缩放表现可以被 MPEG 压缩方案编译出来。

2. 可缩放动画数据的准备

我们已经讨论过几何上的缩放问题了，我们现在需要使脸部和身体的动画数据也能够进行缩放，这样才能实现自适应编码。

对于面部动画，我们可以用 MPEG-4 面部动画参数（FAP）来驱动面部动画引擎。尽管这些参数不会提供任何关于相邻顶点的置换信息，但是会提供关于面部特点的置换信息（例如，在做动画实现面部变形）。从 FAP 信息中，我们可以定义每一个置换——也就是根据 FAP 的强度，可以得出哪些顶点被影响到，并指向哪个方向。

借助于 FAP 信息处理面部动画有两个基本途径。第一种是对 FAP 控制点进行片断线形内插值。这种方法可以减少动画过程中的计算量需求，但是对建模阶段要求比较复杂，因为每一个面部特点都必须定义出一个影响区域。另外，这种处理要精细到每一个模型，每一层的细节。

另外一种方法就是通过几何形变算法来计算一个 FAP 的影响（参见[Magnenat-Thalmann04]第 6 章）。这种方法允许一个由面部定义参数（FDP）自动影响顶点的过程。在几秒钟之内，一个面部动画引擎会根据相关信息创造出描述 FAP 点的变化以及内插值信息的面部动画图（FAT）。FAT 会从一个高解析度的模型上自动建立。由于模型上的顶点已经由 FDP 和它们的变化确定了，

Download at www.Pin5i.Com

我们可以很简单地从每一个高解析度模型的高阶 FAT 图中提取出相关的 FAT 信息；这样，也只有全局 FAT 表中的相关部分才会被传送到客户端。

面部的不同区域会根据每个区域不同层的细化程度来进行预定义。彩色图版 14 显示了一个高解析度的运动的脸部模型。图 7.1.2 和彩图 15 显示了最初建立的脸的各个部分。晚些时候我们会对这两部分中具体的定义及应用（如在网络游戏中）进行详细地说明并在之前讨论不同的清晰等级值（LOA）。

如图 7.1.2 所示，整个面部被分成了若干个片断。这种分段方式使根据 FAP 所影响的可变性区域来对 FAP 进行分组成为可能。对于每一个分段的区域来说，我们还要定义不同的复杂度。一个区域片断对应一个 MPEG-4 的 FAP 群组，尽管我们把舌头和嘴唇分在了一组，但是这些置换还是广泛联系的。这些区域的定义如表 7.1.1 所示。

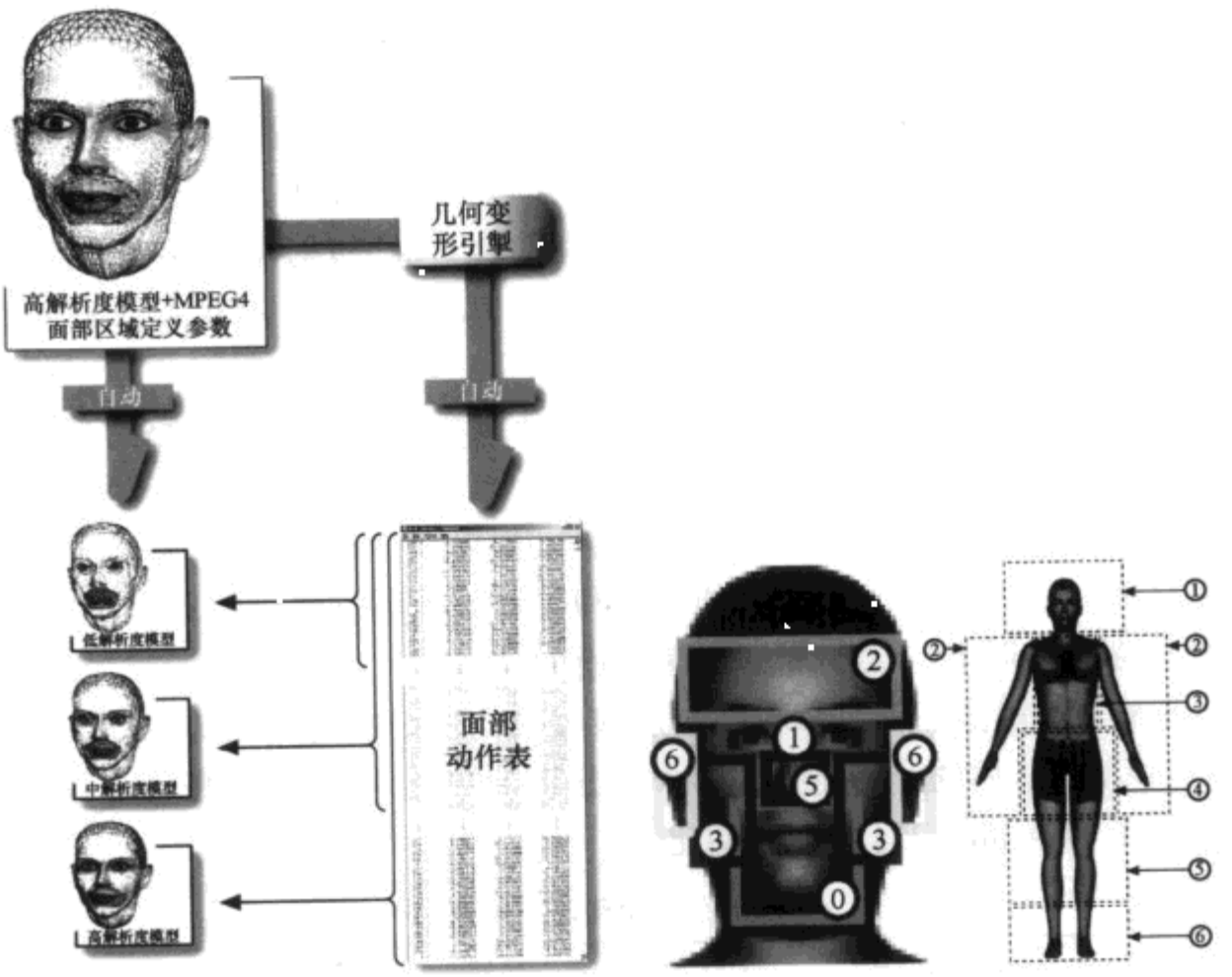


图 7.1.2

表 7.1.1　面部基本区域如表：最终组成全局区域的各个基本区域，例如，“眼睛”由 2+3 这两个基本区域组成；“下半部脸”由基本区域 4+5 组成

区域	关节点	说　明	区域	关节点	说　明
1	31	下颚（下颚、下巴、嘴唇、舌头）	5	3	头部的旋转
2	12	眼球（眼球和眼睑）	6	4	鼻子
3	8	眉毛	7	4	耳朵
4	4	面颊			

需要注意的是，LOA 并不会影响头部转动的值。另外，从尊重用户的价值角度考虑，我们还可以提高或降低某个区域的复杂度。例如，一个表现人物演讲的游戏画面就需要嘴唇周围区域的动画多一些，而面部其他部分的动画少一些。在这种情况下，眼球、眉毛、鼻子和耳朵等周围区域的复杂度就会被降到一个很低的水平，面颊部分可以使用中等的复杂度，下

Download at www.Pin5i.Com

颌部分用高等的复杂度。

表 7.1.2 中定义了面部动画的四级复杂度，这种分级和身体部分的复杂度分级类似，但是由预期的复杂度和游戏内容决定的 FAP 是这个分级的基础。为了降低复杂度等级，可以采取两种方法：第一种是将分组后的 FAP 值组合在一起（例如：所有的上嘴唇值都可以组成一个值），第二种是将已经被相邻的面部特点区域影响到的特点区域删除掉。

表 7.1.2 面部动画的预期和清晰度解析

清晰度等级	FAP 的数量	说　明
非常低	14	
低	26	压缩不必要的 FAP，完全对称
中	44	不对称，把中等级 FAP 建组每两个 FAP 一组，做组等价值
高	68	所有的 FAP 值

最后，经过改编的 FAP 流可以被用在以下两个方面。第一个是根据有关清晰度的规则对数据流进行解码并重建所有的 FAP 数据。在这种情况下我们只需要压缩比特流量就可以了。另外一个方法就是简化变形的过程。在 MPEG-4 面部动画引擎中，变形过程是将每一个 FAP 值进行变形计算并将它们分组。为了简化这一过程，可以通过对每一层变形区域复杂程度的外部设计来获得自适应改编的 FAP 流。这一过程可以通过预加工步骤来自动实现，或者可以和模型一同传输。

对于身体动作来说，我们仍然可以用 MPEG 来表示和压缩基于骨骼的动画（Bone-Based Animation，BBA）动画数据。BBA 是一种关于层级的一般概念，我们这里所说的身体动作引擎是一个基于 H-Anim 1.1 层级（例如：跟据骨骼驱动的动作来定义特定的关节和骨骼）的引擎。

一般来说，身体动作是由一个分级的骨骼和关节结构的动画来实现的，这个分级结构各个等级的复杂性应该是随着层级的加深而简单递增的。

依照 H-Anim 1.1 规范，就可以根据自适应改变的需要来定义 LOA 了（参见表 7.1.3）。例如，最低级 LOA 包括肩膀和臀部，这两个部分是该层上动作最小的部分。另外，中级 LOA 的定义基本上就是 H-Anim 中对脊椎的简化。

表 7.1.3 身体动作预期和清晰度解析

清晰度等级	节点数量	说　明
非常低	5	肩膀，臀部和脚部关节
低	10	非常低等级的关节，加上脖子、肘和膝
中	35	非常低等级的节点，另外手指，脚趾和脊椎都融合成一个关节
高	88	全部层级结构——所有关节

这些复杂性分级可以用来选择一个虚拟人模的动作在自适应改编过程中的细节程度。例如，有些虚拟人模可能是坐在体育馆里的部分观众，他们就不需要特别高的细节程度。因此，用一个非常低的 LOA 分级对他们来说已经足够了；而主角将会运动、讲话，他会是玩家注意的焦点，他就需要一个很高的细节程度，也就是使用中级或高级 LOA。尽管针对个体虚拟人模来说，该方法并不会起到很显著的效果，但是这个分级会很大程度影响到对整个人群动作进行控制时的效率，或者是影响到为了网络传输对动画数据进行自适应改变的效率。

表 7.1.4 定义了关心区域列表，这个列表可以让引擎的使用者选取他们所最关心的身体部分。例如，如果用这个角色讲解烹调一道菜肴的过程，但是引擎的使用者只关心所能听到

Download at www.Pin5i.Com

的演讲部分而不是动作部分，这样动作就可以通过自适应改编而只保留上身甚至只保留脸部及肩膀的动作就可以了。另外一个例子，如果一个导游手指向他正要介绍的几个地点，例如剧院或者公园。引擎使用者可能只主要到这个角色的胳膊和手的动作。图 7.1.2 介绍了 6 个低层级关心区域，这些区域也可以调解成高层级区域。

表 7.1.4　身体基本区域列表：最终组成全局区域的各基本区域，例如，“上身”由 1+2+3+4 这 4 个基本区域组成；“脸和肩膀”由 1+2+3 组成

区　域	节 点 数	说　明	区　域	节 点 数	说　明
1	2	仅面部	4	4	骨盆
2	49	左右臂	5	4	腿
3	25	躯干	6	4	脚

7.1.4　自适应数据的传输

之前的部分介绍了如何为角色的几何及动作创建可缩放数据。图 7.1.3 和彩图 16 则是一个关于实现这样的可缩放数据用例；现在我们来讨论一下自适应改编的整个过程。

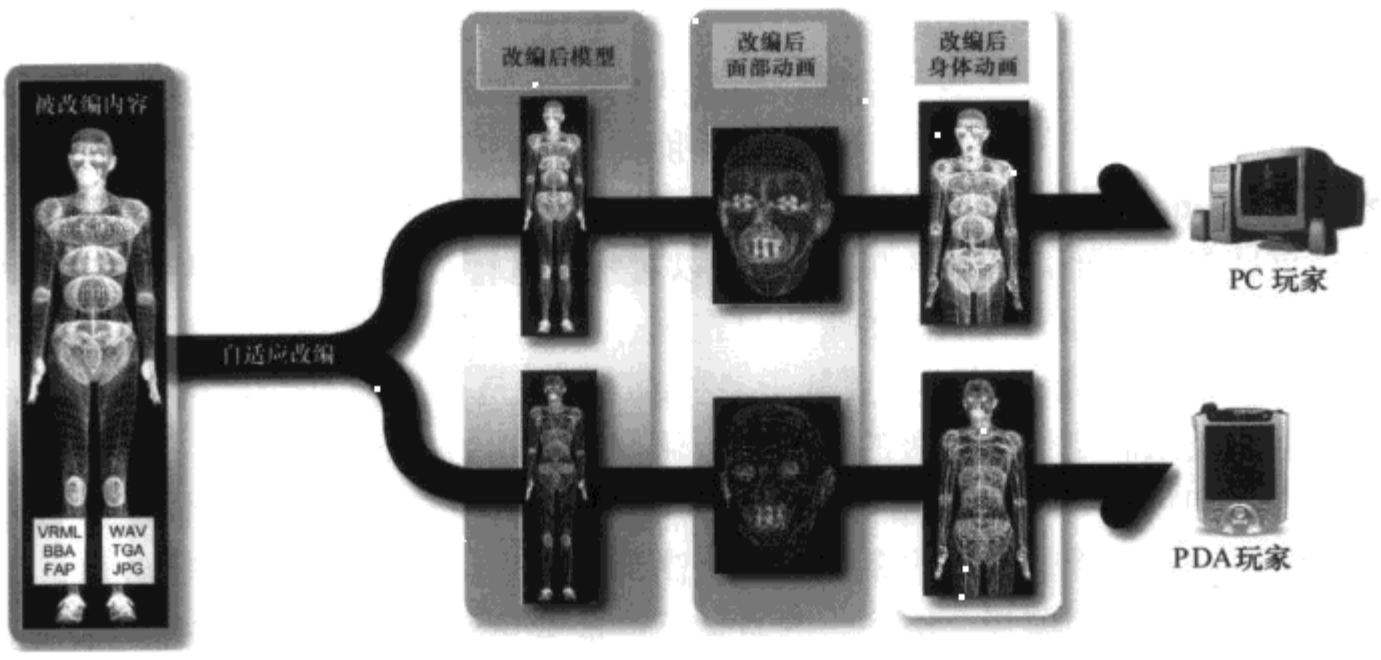


图 7.1.3　从数据库到用户的数据流示意图

数据流的自适应改编基本上就是对比特流进行传输时的修改。图 7.1.4 显示了如何对一个最高内容解析度比特流进行改编，使其符合网络传输需要或者客户端限制。

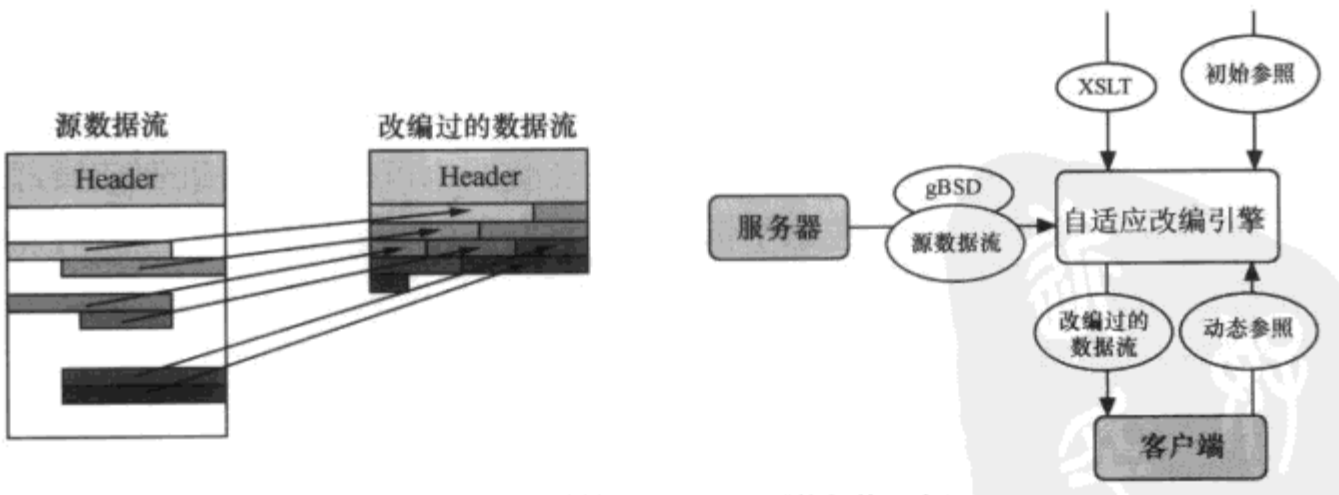


图 7.1.4　数据改编原则（左）及网络架构（右）。
注意，自适应改编引擎也可以作为服务器的一部分而存在或者为了不同的改编而导致多个改编引擎共存

Download at www.Pin5i.Com

利用 MPRG-21 实现自适应改编的原则就是利用 XML 方案（如以前介绍过的 gBDSL），对编码的数据进行描述。在你的编码器输出压缩过的几何和动画数据的同时，也会生成一个 gBSD 文件对数据流进行描述。在随书光盘中可以找到一个关于两帧的身体动画的例子。自适应改编引擎可以将比特流进行转化，并且还会提供一个合适 XML 规则说明，该说明将包含如何根据一组参数将通过改编引擎的数据流进行自适应改编的信息。这些参数为自适应改编提供了前提条件，并且这些参数会涉及到多种信息，例如，网络带宽、玩家获得的帧率，等等。关于比特流的自适应改编过程所涉及到的信息和方法，以及该过程必须要遵循可缩放性，这些我们在前面的部分已经提到过了，这一切都是考虑到这些可缩放特性并为其提供低层级的解决方案。

原始的 FAP 和 BBA MPEG 代码并不能显示出更多的可缩放优势。尽管为了编码而对参数进行分组比较适合我们前面所谈到的那些区域概念，但是并不适合 LOA。这就意味着我们必须将我们的方案定义在一个很低的层级上以利用 LOA 进行自适应改编。

自适应改编过程的参照

你想要多少种参照就有多少种参照，但是我们还是要克制自己一下，所以我们这里所讨论的参考只是定义为客户端能力（特别是渲染能力）、网络能力，以及用户偏爱或情境因素。

另外，对于 mesh 和动画的改编，也有很多不同的参照。而改编的主要目的也是为了尽可能地满足这些参照标准。在网络方面，你可以对带宽进行测定以确认网络可用的承载能力。另外，用户的偏好，或即时方面的约束也是在进行 LOD 选择时的外部条件（在应用程序允许的情况下）。不过最终能力的确认要基于多种因素，还是很复杂的。在这种情况下，可以通过对设备进行比较来确定何种层级的缩放适合何种客户端的承载能力。

接下来，我们将会讨论如何近似地确定要压缩的内容复杂层级的一个流程。首先，需要压缩的内容要在一台标准的测试设备上以其最高的解析度运行，以其作为比较基准。这台测试设备要求能够对该内容解译、渲染，并且运行时能够达到令人满意的帧率，这个帧率的值为 F_N（例如，该帧率要高于 25）。然后，将标准测试机与一般的测试环境做比较，这就需要把当前运行的应用程序的数量降到一个最小值。通过一个较合理的比较过程——如果可能的话要多做比较——会得出一个值 B_N。最后，作为 runtime 运行的一部分，目标设备还要再一次进行比较（可以先作 runtime 或者将 runtime 作为初始化进程的一部分来做）就会得出一个值 B_T。B_N 和 B_T 应该是同一种类型的值，无论是在单独的匹配比较或是加权比较（要包含相对应的元素）。这样我们就可以计算出自适应改编的比率：如果目标设备需要在值 F_T 情况下运行，那么 mesh 和动作应该以 R_A 为比率进行压缩（我们也可以利用最近似值原理，直接利用这个参数来减少多边形的数量）。在这个情况下，我们需要考虑在标准机上的机能剩余和目标机上的可能局限；这样就会得出等式 7.1.6——mesh 和动作的压缩比率。

$$R_A = \frac{F_N}{F_T} \times \frac{B_T}{B_N} \times C_R \qquad (7.1.6)$$

然后，对于所有 R_A 小于 1.0 的情况下，自适应改编都是必需的。在这些情况下，mesh 和动作在表现上并不会发生变化。C_R 是一个外加的因子，可以用常量 1.0，另外它也可以让

Download at www.Pin5i.Com

用户明确地知道在处理某一个专门的内容时会占用多少计算时间。这个外加因子的出现就是因为很多内容在同一设备上同时被解译或渲染——例如，一个图形动画或者音频流。通过这种方法，可以很简单地说明两个或两个以上的内容在同一台机器上运行的情况。另外 C_R 还可以用来分析每一个内容的比重，因为所有的 C_R 之和为 1.0。这种方法在提高效率方面很有用，因为它可以使改编过程不需要考虑：(a) 对一台电脑内多种效率因素的理解（例如：是 CPU 类型或是频率型，理解这个问题需要硬件架构的知识），(b) 将处理过程简化成一个近似的线性过程。

在针对网络带宽进行自适应改编的时候，网络的承载能力直接由一个可用带宽 C_B 来限制，若编码文件的大小为 F_S，下载时间为 T（针对 mesh 来说，因为不是动态下载），这个时间可以是用户需要等待的时间，也可以是你认为合理的等待时间。见公式 7.1.7。

$$\frac{C_B}{F_S} \times T = R_A \tag{7.1.7}$$

R_A 提供一个从原文件到压缩后文件的比率；如果编码过程已经被做得很好的话，那么 R_A 就是决定大小的一个很重要因素了，这个大小是 mesh 大小，也就是数据流被减小的多少。F_S、C_B，甚至 T 都是很容易就能得到的，因此，R_A 也很容易算出来。

最后，几何上的自适应改编过程不需要过多的内存，除了存储改编处理时涉及到的聚类信息和 mesh 数据。另外，可以通过一个“客户端自适应改编”来建立一套低细节层级的聚类，这些低细节的类聚可以在渲染的同时就被动态改编，就像传统的 LOD 系统的工作原理一样。

7.1.5　总结

我们在全球收集到结果是，通过这种方法可以近似获得总体效率 30%的提高，以及 25%到 40%的数据压缩比。比特率对于网络应用程序和网络游戏来说总是很重要的。所以说 3D 数据传输时流量大小就需要被考虑、被控制，并且尽可能的降低。自适应改编是一个必由之路。通过处理不同的层级和压缩方案，自适应改编和压缩将会是处理 3D 数据传输的绝佳组合。

数据流的压缩不但有利于管理比特率，在 3D 方面它还能够提供一种依照客户端能力以及当前场景需求对渲染的复杂性进行管理的好方法。

7.1.6　参考文献

[Aggarwal01] Aggarwal, Ashish, et al., “Compander Domain Approach to Scalable AAC.” *110th Audio Engineering Society Convention,* 2001.

[Amielh02] Amielh, Myriam, et al., “Bitstream Syntax Description Language: Application of XML-Schema to Multimedia Content Adaptation.” International World Wide Web Conference (WWW 2002).

[Di Giacomo04] Di Giacomo, Thomas, et al., “Adaptation of Virtual Human Animation and Representation for MPEG.” *Elsevier Computer & Graphics,* 2004, Vol. 28, No. 4: pp. 65–74.

[Fei99] Fei, Guangzheng, et al., “A Real-Time Generation Algorithm of Progressive Mesh

Download at www.Pin5i.Com

with Multiple Properties." ACM Symposium on Virtual Reality Software and Technology, 1999: pp. 178–179.

[Garland97] Garland, Michael, et al., "Surface Simplification Using Quadric Error Metrics." SIGGRAPH 1997: pp. 209–216.

[Hoppe96] Hoppe, Hugues, "Progressive Meshes." SIGGRAPH 1996: pp. 99–108. [Kim03] Kim, Jae-Gon, et al., "Content-Adaptive Utility Based Video Adaptation." IEEE International Conference on Multimedia & Expo (ICME 2003): pp. 85–94.

[Magnenat-Thalmann04] Magnenat-Thalmann, Nadia, et al., *Handbook of Virtual Human,* John Wiley & Sons, 2004.

[Preda04] Preda, Marius, et al., "Virtual Character within MPEG-4 Animation Framework eXtension." IEEE Transactions on Circuits and Systems for Video Technology, 2004, Vol. 14, No. 7: pp. 975–988.

[Seo00] Seo, Hyewon, et al., "LoD Management on Animating Face Models." IEEE Virtual Reality 2000: pp. 161–168.

[Tecchia02] Tecchia, Franco, et al., "Image-Based Crowd Rendering." IEEE Computer Graphics & Applications, 2002: pp. 36–43.

[Van Raemdonck02] Van Raemdonck, Wolfgang, et al., "Content-Adaptive Utility Based Video Adaptation." IEEE International Conference on Multimedia & Expo (ICME 2002): pp. 369–372.

[Walsh02] Walsh, Aaron, et al., *The MPEG-4 Jump-Start.* Prentice Hall PTR, 2002.

Download at www.Pin5i.Com

7.2 大规模多人在线游戏基于复杂系统的高阶架构

Viknashavaran Narayanasamy,
Kok-Wai Wong 和 Chun Che Fung, Murdoch University
viknash@hobbiz.com, k.wang@ieee.otg
langceccfung@ieee.org

最近，为了能够设计出一个令人兴奋的 Massively Multiplayer（MMP）——即大规模多人在线游戏——来满足玩家不断增长的预期值，设计的复杂程度也在成几何级数增长。这主要是因为 MMP 游戏大规模连线的特质允许了上千名玩家同步游戏。然而，由于现今的游戏架构是基于已经预设好了的玩家行为，因此，在大规模多人在线游戏上的扩展性不是很好。我们在本节会介绍一个基于复杂系统的突发事件处理的架构，来最小化 MMP 游戏中判断性行为。我们会通过一个由下至上的，依赖于异类 agent、自我组织、强化互动、回馈，以及突发事件的设计方法来为下一代 MMP 游戏设计一个多重架构。

由于一般游戏架构都是大量的其他游戏架构中的通用特点的应用和集合，因此我们这里只会涉及到本架构中的独有的一些特点。该技巧将会从软件工程的角度提供一个游戏架构的高阶抽象，以便任何人都能把它用到其自己的 MMP 游戏中去。

7.2.1 复杂系统和突发性事件

复杂系统是指由大量的互动元素组织起来的，在深度和广度上都有着很高的多样性的高度组织的系统。复杂系统被建立的目的就是处理单一的规则或者单层级的解释，并不能处理一些变化[Kirschbaum98]。另外，在自然和生物科学中，组建一个复杂系统也是为了研究寄生、共生、繁殖、同一性，有丝分裂以及优胜劣汰等问题[Odell02]。一个系统如果被称之为复杂系统，那么它必须包括一系列的已定义的定义特性、已命名的自我组织结构、非线性关系、有序或混乱的动态特征，以及突发性事件等特征[Kirschbaum98]。Wikipedia[Wikipedia05]上关于复杂系统做了如下解释，复杂系统区分与一般系统的特性还应该包括：回馈回路的存在、开放式环境以及模糊的系统边界。

MMP 游戏所展现出来的很多特性和行为都适用于复杂系统。其中包括千人的高互动性环境、基于 AI 的 NPC、大规模世界、模糊的游戏状态，以及由不同的社会阶层、文化背景、道德准则的游戏玩家对于系统的多重

Download at www.Pin5i.Com

不可预见输入，等等。

本节所讲解的基于复杂系统的游戏架构将会促进 MMP 游戏对一些近年来一直没法实现的游戏特点的融合。这个架构也会促进对真实世界及突发性行为的模拟。这个系统的独特性还在于对整个游戏生命周期中，原来游戏设计里未包括的突发性事件的处理行为。这些行为能为游戏行为提供一个不可预见性和随机性层面，这对于下一代 MMP 游戏来说是极为重要的。

7.2.2 多重架构

基于复杂系统的多重架构由 4 个层级组成：环境层、物件层、agent 层以及监查层（如图 7.2.1 所示）。传统的多重分级式架构会采取正交的方法来减少不同层级之间的耦合。我们这个架构中，正交性只会被应用到环境层和其他层之间。这样做是为了在其他的层级进行变化时，保证环境层的稳定性与独立性。另一方面，物件、agent 和监查层将会协调工作，并且互相之间紧密结合，以实现游戏中物件的高度互动性。

物件层由零件层和合成层组成。突发性事件会从物件层的零件层开始发生，并通过这些零件的互动与集合向上传递。agent 曾代表了游戏中所有拥有更高级功能的游戏物件。包括了 NPC、玩家以及其他拥有很高的人工智能的物件。物件层和 agent 层能让设计师适用简单的游戏规则和物件层中的零件来创造出有机的、系统的高阶游戏行为。监查层是构架中最后也是最重要的一层。它会通过 agent 层来实现复合的人工游戏控制，和基于一些规则的对游戏环境中所有物件的控制。

1. 环境层

游戏的环境定义了游戏所模拟的虚拟世界中全局的特性。环境层包含这些全局特性的同时还包括了游戏引擎的执行。在环境层的游戏引擎是由模拟、图形、声音、物理及 AI 引擎组成的，此外，还包括事件及输入控制器、资源及硬件驱动层。然而，与其他游戏架构不同的是，我们的架构是不允许直接获取游戏引擎资源和游戏数据资源的。这样做是为了 MMP 游戏中的物件（参见物件层）能够同步运行在不同的客户端和服务器上，而不需要明确知道客户端所运行的平台的相关信息。这也可以允许游戏在不同的平台上运行（例如，游戏机、PC、Web、移动平台，等等），因为只有环境层需要根据游戏平台而发生变化。对于客户端来说，在客户端机器上只需要下载相关的物件的子设定，来满足客户端对游戏环境的现实，以及维持与服务器端同步的物件状态。客户端环境层与服务器端环境层之间的通信秩序要解决在 MMP 游戏运行的时候游戏数据（如等级、图形等）的接收和更新等问题。图 7.2.2 描述了客户端和服务器端运行的图解。

游戏中各个物件在环境层与其他各层的互动过程只会通过事件处理器、AI 引擎和资源抽象层来完成，如图 7.2.3 所示。AI 引擎会应用监查层中的监查器来对客户端环境的能力和限制建立一系列基本的规则。另外它还会对一些类似于游戏内经济系统、物理、环境边界等问题建立一系列基本的规则。输入控制器则会接收玩家的输入，并生成事件让游戏引擎来处理输入。

Download at www.Pin5i.Com

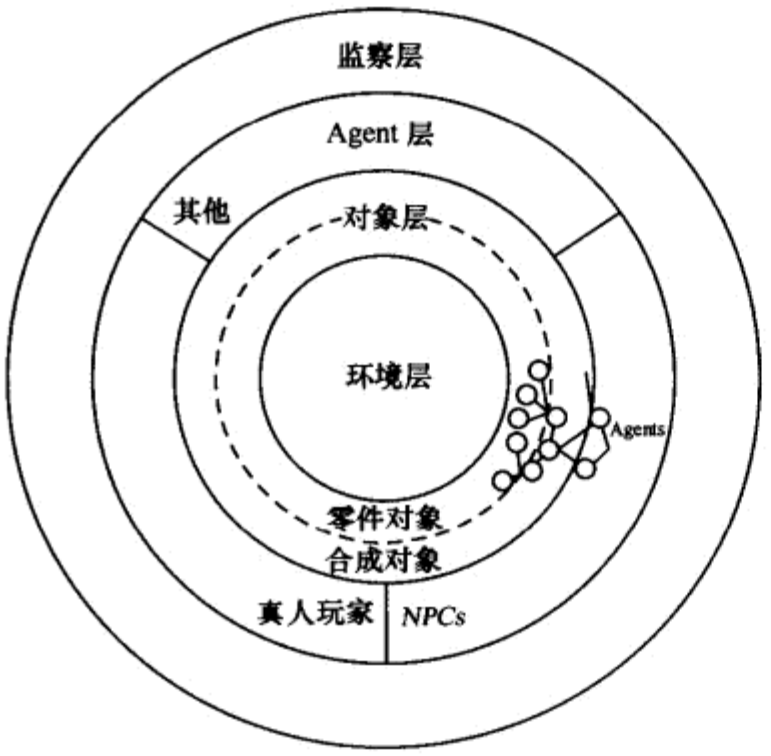


图 7.2.1　多重架构

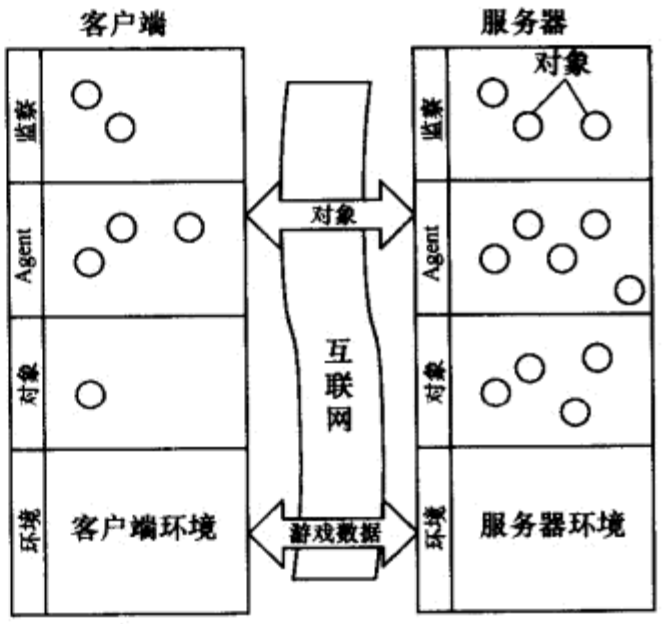


图 7.2.2

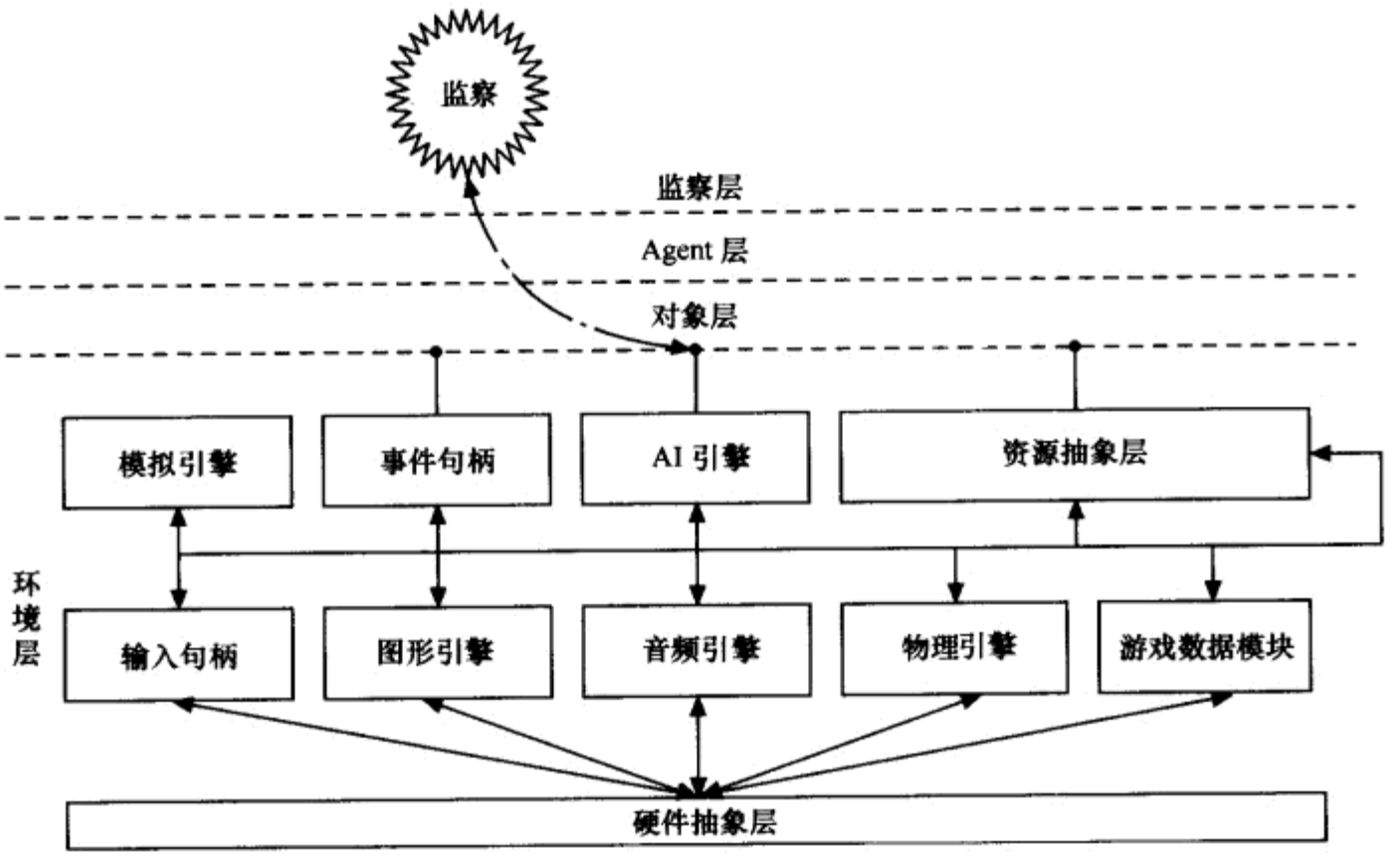


图 7.2.3　环境层

与传统的游戏构架不同，物理引擎将作为一个独立的功能层，成为游戏引擎与硬件的借口，因为现在独立物理计算与 PPU（物理处理单元）的出现已经成为一种趋势[Ageia05]。物理引擎、声音引擎和游戏数据模块在功能上与其他游戏架构并没有太大区别，所以这里关于细节的功能组件就不再赘诉了。事件处理控制器与模拟引擎同步工作，将物件和玩家事件翻译成模拟引擎可以接收的事件。事件处理引擎还会根据发生于某一物件的事件的特别类型对事件进行发送。

模拟引擎，换句话说就是游戏回路。它通过离散事件模拟（Discrete-Event Simulation，

Download at www.Pin5i.Com

DES）模型来工作[Garcia04]。设计 DES 模型的初衷是由于大量的物件存在使事件执行的顺序未知或不可预见时对一个复杂系统实施模拟。在这样一个假想下，现在大部分游戏的模拟模式都依靠于一个不断对不能维持系统因果关系的事件进行检测和执行的过程。作为一个离散事件模拟器中的对这些离散事件执行的模拟过程，需要对这些事件的时间顺序进行维持。这样一个紧密连续的事件执行会提高系统对模拟事件的反应。

如图 7.2.4 所示，图形和模拟的处理是同步进行的。允许图形渲染率能够独立于模拟速度。这一模式将会在下一代游戏硬件及其所支持的性能中实现。

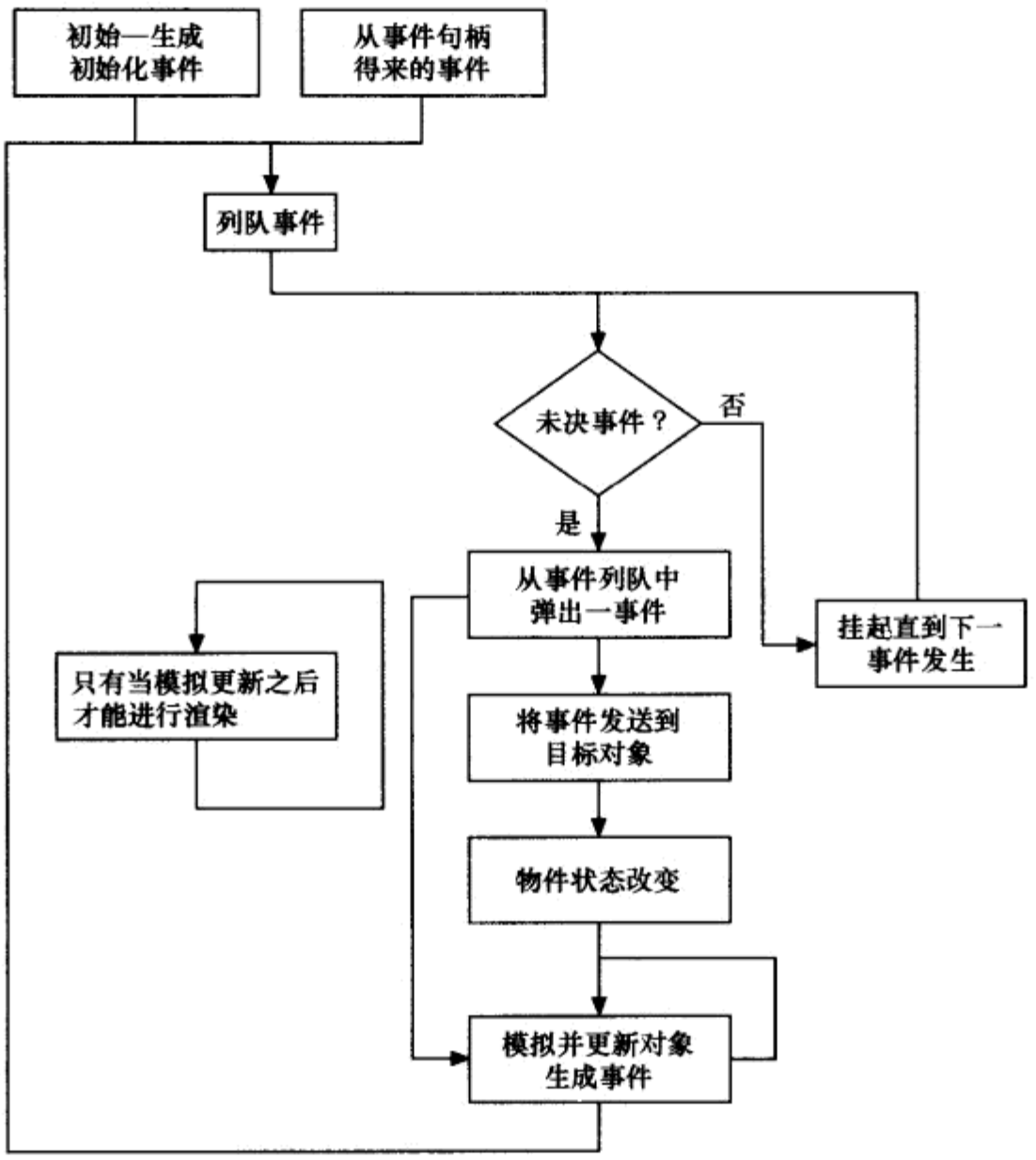


图 7.2.4

由玩家和程序生成的事件将会在处理时进行排序。当序列中没有事件时，系统就会进入休眠模式直到序列中有了新的事件。需要持续进行更新的动态模拟能够在一个可以由用户自定义频率的系统时钟的帮助下生成周期性事件，这些周期性事件是独立于其他模拟处理和渲染子系统的。例如，模拟一个人持续走动。模拟过程是有序的，而且通过列队的方法可以生成事件的优先级。

资源的定位和维护可以通过一个叫做资源抽象层（Resource Abstraction Layer，RAL）来完成。对于有些资源来说（如图形或网络），因为即时性的需求，所以会导致不同的物件因为完成同一个或不同的任务来调用同一个资源。RAL 中一些基于避免不公平情况的简单规则的算法是不够的，因为游戏中不断增加的突发性事件会很快导致这层中规则的溢出。为了避免

Download at www.Pin5i.Com

该资源争夺问题，可以通过一个分配 agent 算法和合作任务 agent 来完成[Seow02]。

这里的分配 agent 算法是基于[Seow03]中提到的多重 agent 任务算法（MA^3）来完成的。在这个算法中，对每一个游戏物件的处理只处理需要的物件的局部信息，并且在发布资源变更意图之前就要做 BDI（Belief、Desire、Intention）[Geiss99]。图 7.2.5 中所示的仲裁 agent 就会根据意图以及相关资源进行判断。MA^3 算法很适合这个应用，因为它能够通过探索式的选择过程选择最接近理想的方案来加速冗长的仲裁。另外，该算法也允许游戏物件平行工作，并在合作的资源参与仲裁之后找回资源、分配资源、分散资源。

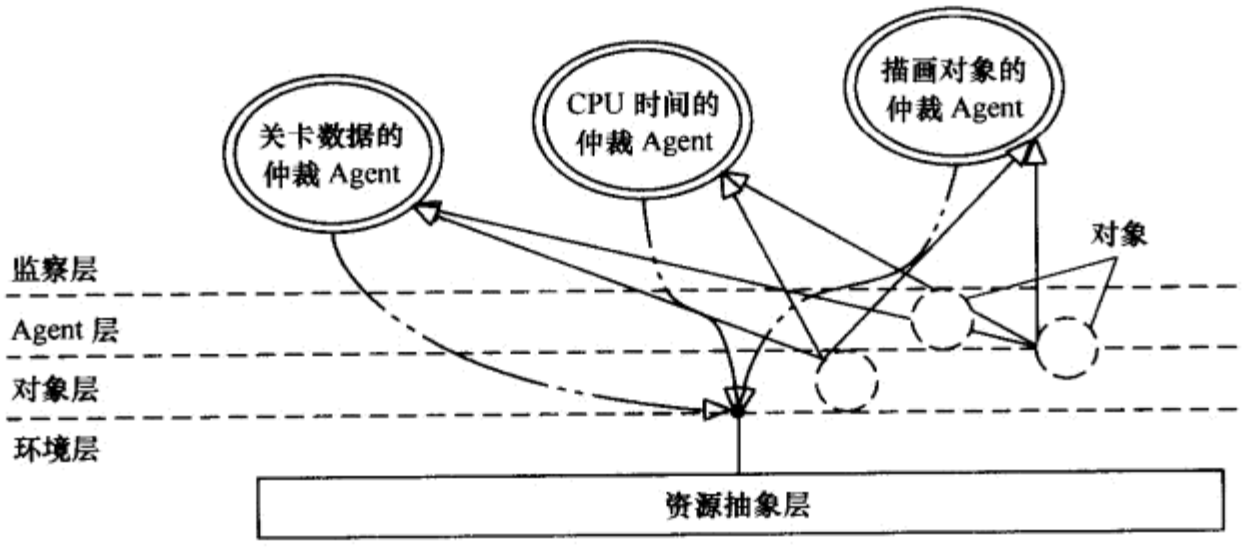


图 7.2.5　利用 MA^3 Agent 能够进行协作资源分配

游戏物件一起通过参与谈判来取得一个互动的一致，这对于所有的 agent 来说都是可以接受的。在仲裁 agent 存在的情况下，每一个游戏物件都要通过一个被提议的资源交流来选择及再选择不同的资源。在这种情况下，当每一个游戏物件都在尝试达到一个让所有的资源交换意图都被合理完成的时候，会体现出一个很强的自我组织能力（self-organization）。

2. 物件层

物件层由零件层和合成层构成。这部分将会介绍怎样将突发事件通过一个由下至上的方式编码成游戏物件，并在物件上再建立物件，形成合成物件，以及最终的为了实现突发行为的职能 agent 物件。

可以这样说，游戏在执行时的复杂性由来就是游戏物件结构的复杂性以及它们之间的内部联系[Wilkinson93]。当 MMP 游戏要引入突发性事件时，设计成拥有大量的属性和操作的游戏物件，并且显示大量的物件之间不同关系就会很轻易的提高开发的复杂性。为解决这个问题，该架构应用了软件工程学中的合成设计模板来处理游戏物件和游戏物件的合作结构。

设计模板是已经被证实过的软件工程学工具，它能够成功的对设计及架构进行再利用。另外一个关于合成设计模板和其他设计模板可以参考[Gamma94]。这里我们用 UML 的类图来表示合成物件的属性、操作、约束以及联系的组织形式。关于 UML 类图可以参照[Booch98]。

许多游戏物件可以被看成是组件和组件的集合（高级结构合成游戏物件）。这些物件在基于物件编程中被称为“合成物件”[Rumbaugh94]。组件静态的内部联合与行为上的互动，以及合成物件的全局性制约影响就类似于我们这里中所谈到的复杂系统和 MMP 游戏的架构[Ramazani94]。图 7.2.6 显示了一个简化了的类图，通过这个类图，可以看出游戏物件是如何在架构中形成合成物件的。合成的游戏物件将会使所有的游戏物件通过回归合成过程统一的

Download at www.Pin5i.com

进行互动。图 7.2.7 描绘了由简单的组件性物件组成的高级结构。

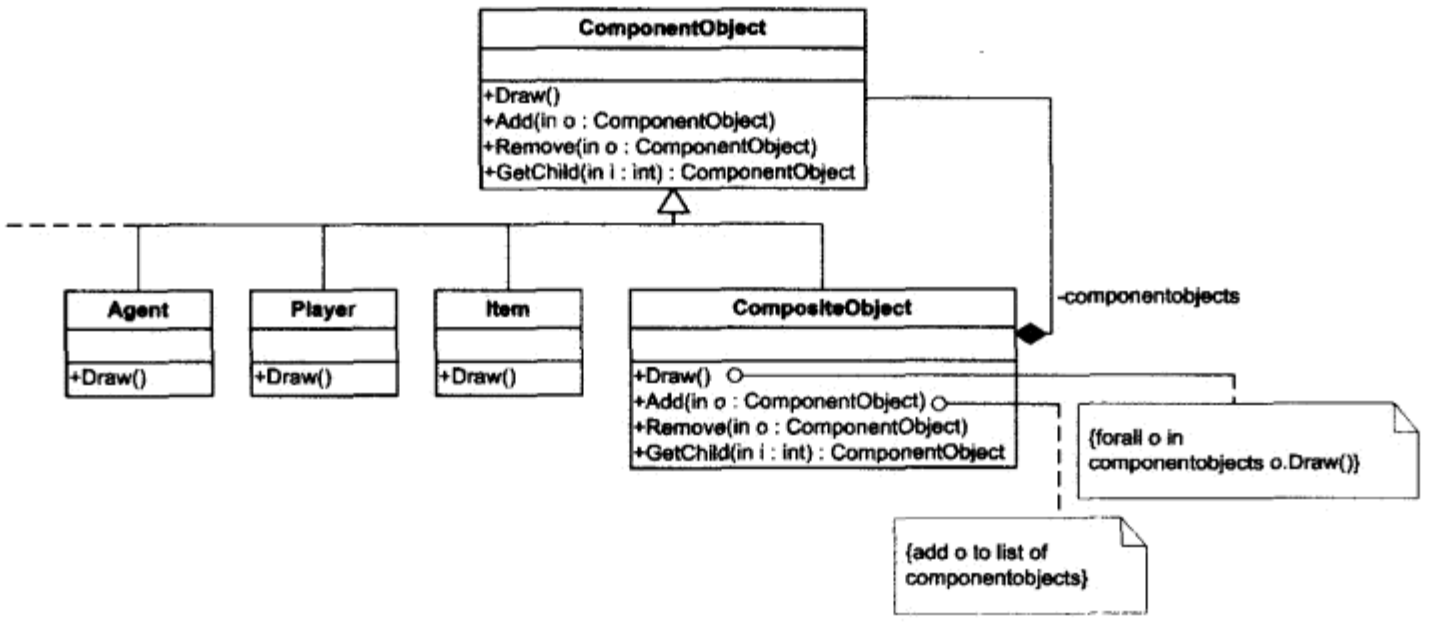


图 7.2.6 一个 NMP 架构中的合成对象

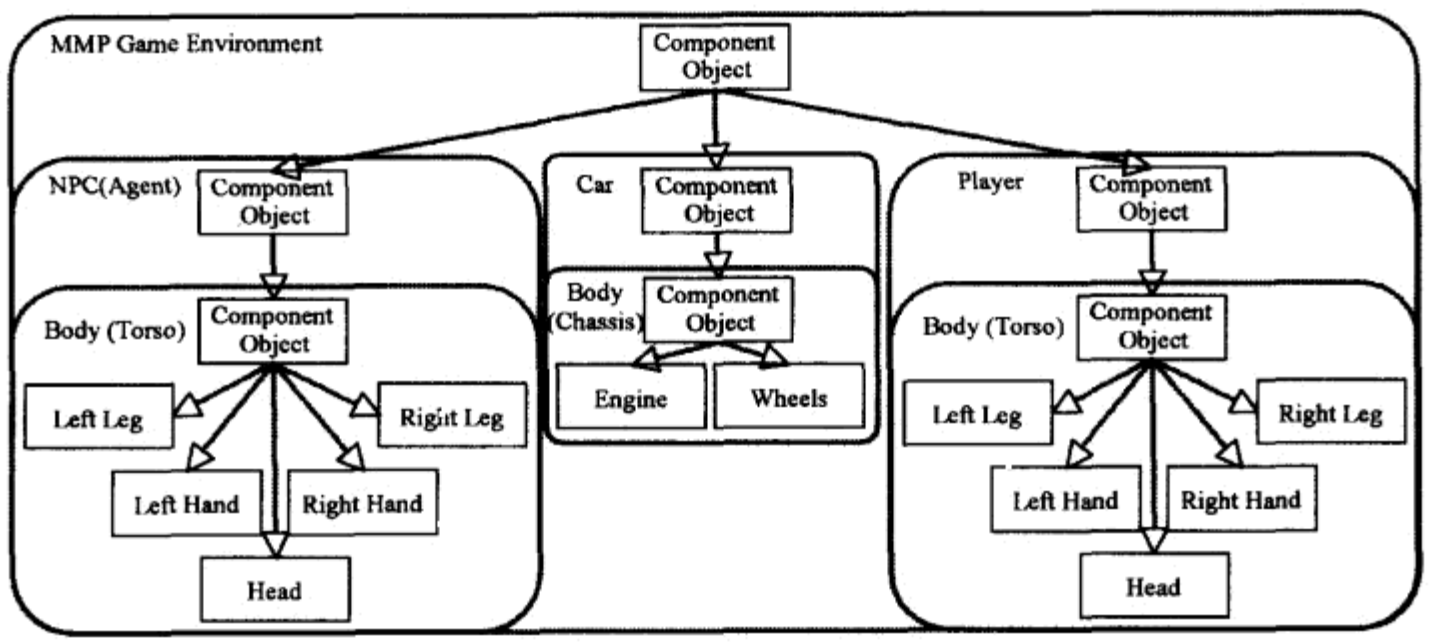


图 7.2.7 由组件对象组成合成对象的高级层级结构

合成物件为游戏物件和它们的语义学关系按组织学形成组织结构提供了一种方法。而属性、操作、关系是如何从简单的组件型物件发展成更复杂的合成物件的是解决问题的核心。实现这个，就能实现有效的突发性事件。

属性和操作可以通过很多方法由低级向高级传播。例如，当物理上的重量是所有游戏物件的通用性属性时，合成物件的重量就是其所有组件性物件的简单重量的和。然而，另外一个例子中，当我们考虑到一个房间中的一盏灯的光照时（例如，房间是合成物件，灯是组件性物件），这个房间需要从那盏灯的环境光照情况中继承他的光照。为了区别属性、操作，以及从组件性物件的传播关系，也为了能提供一个有利于这种传播在复杂环境中进行的构架，就有必要通过它们的内在性质、聚合关系，以及突发性质对属性、操作及关系进行分区[Ramazani94]。

内在性质，游戏物件的内在性质在架构中是指当属性直接涉及到游戏中的合成物件时的逻辑意义。从图 7.2.8 中的例子，我们可以看出，属性的内在性质——汽车颜色从一个有相似内在性质的属性库里被挑选了出来。在这个例子中，汽车作为游戏物件决定着底盘中的颜色。

Download at www.Pin5i.com

为了获取这个属性的内在性质，使用了一个操作 GetColor()，该操作会在汽车这一合成物件中使用底盘这一个组件性物件。因为 GetColor()被定义为它的组合物件中的一个，这就使该操作成为了一个合成的汽车物件的内在性质的操作。实际上，在一些组合型物件的组件性物件中（本例中的底盘），只有当该物件参与了一些联系的时候才会成为该组合型物件的内在性质物件。在游戏运行时，这些内在联系是普遍存在的，因为它们实施起来会很简单；但是，它们并不能够帮助每一个组件性物件建立最终的高级结构（例如：汽车）。所以说它们对 MMP 游戏架构的作用并不大，因为在 MMP 游戏的高级架构中，会经常发生突发性事件。

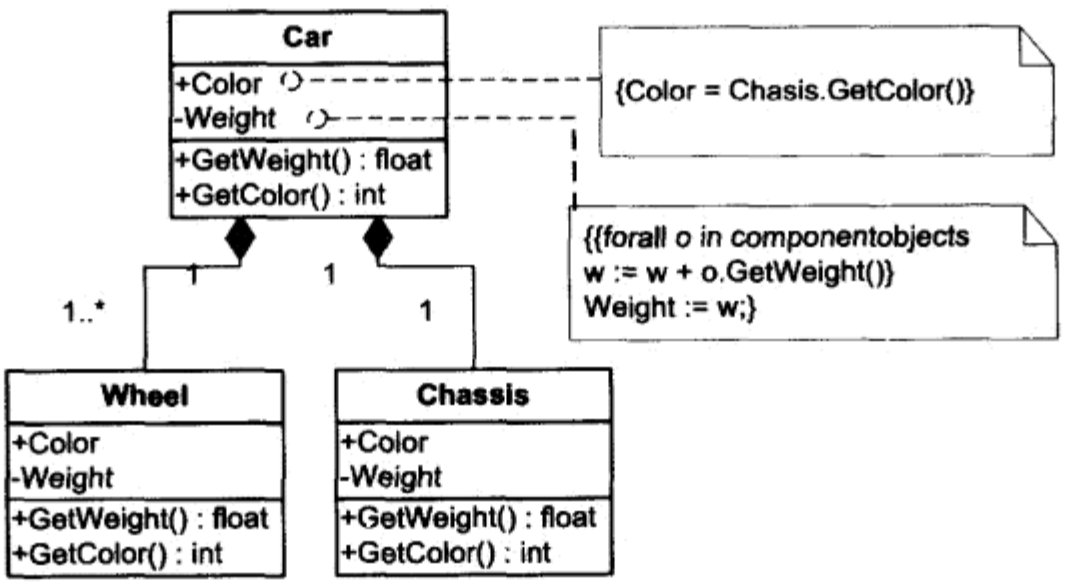


图 7.2.8　一个合成的汽车对象的聚合属件的组成

聚合关系（aggregate relationship），相对来说，对游戏架构会更加有价值，因为在关系上的约束会影响所有的组件性物件。在图 7.2.8 中，聚合属性 Weight 和聚合操作 GetWeight()就描述了这一过程。要创建一个新的聚合操作 GetWeight()就需要将合成性物件——汽车——的所有组件性物件的 Weight 属性求和并导出。同样的，一个新的 Weight 属性值也是通过这一过程导出的。一个合成性物件中的组件性物件的聚合属性的改变也会导致组件性物件的聚合属性的变化，同时，该变化还会传播给合成性物件的其他子物件。另外还需要注意的是，只有当组件性物件和合成性物件组合在一起时（例如：只有当轮子、底盘、引擎等都装在一起时，汽车这一合成性物件才会产生重量），一个新的聚合关系才会产生。聚合关系对基于复杂系统的 MMP 游戏架构来说有很大的好处，新的行为会在组件性物件的聚合的已有属性中产生，但是对于合成性物件来说，并不会产生新的属性。

在图 7.2.9 中，当 Car 这个合成性物件获得一个新的 Engine 组件性物件时，就会获得速度和方向这两个突发性属性（emergent attributes）。操作 move ()就会出现在这个突发性关系之外，被称为突发性操作（emergent operation）。突发性属性由于能够定义合成性物件全局而不是局部的属性，因此突发性属性都是唯一的。然而，突发性操作的唯一性则是因为其并不依赖于任何一个独立组件性物件对全局建立一个全新的操作。例如，汽车只有在引擎、轮子、底盘和其他相关部分存在的情况下才能够拥有移动的能力。

由于支持由下至上建立游戏物件，因此突发性关系能够使基于复杂系统的 MMP 架构更为灵活。由于客户端的物件只会和具有突发性属性的合成性物件互动，因此突发性关系可以将合成性物件的组件进行打包。另外，由于突发性关系能够在合成性物件是个体物件的时候让开发者来处理，所以能够降低开发的复杂性。

Download at www.Pin5i.Com

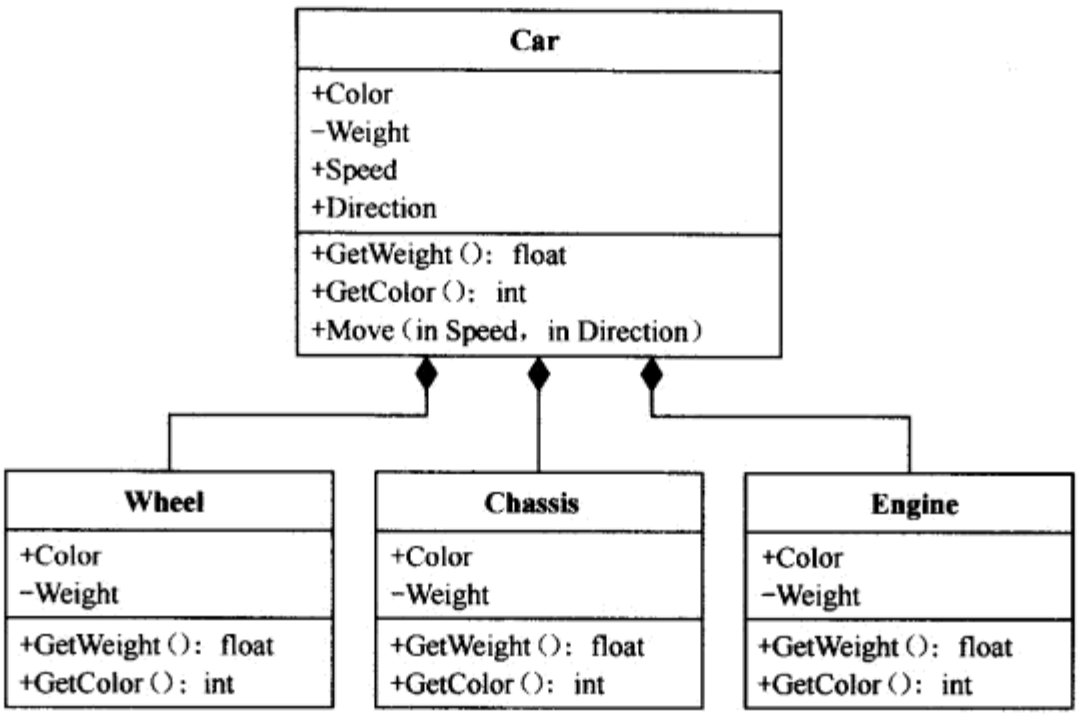


图 7.2.9 一个合成的汽车对象的突发性属性

然而，这样一个架构中，最难实现的就是游戏过程中突发性事件功能上的有效性。一般来说，会通过确保组件性物件和合成性物件之间的突发性关系的功能意义的合理性来实现。如图 7.2.10 所示。这一过程可以很容易地通过一个突发性关系表来实现。例如，当一个汽车的引擎由于撞到了墙而损坏，那么相关的一些特殊联系可以记为失效，另外通往突发性事件 Speed、Direction 和 Move ()的通道也被取消。另外，可以用观察器设计图与突发性关系表共用来为每一个组件性物件添加一个观察器物件。观察器物件可以在物件中保持一个单对多的依赖性，这样的话发生在被观察的组件性物件上的变化就能够与相对应的合成性物件进行沟通[Gamma94]。

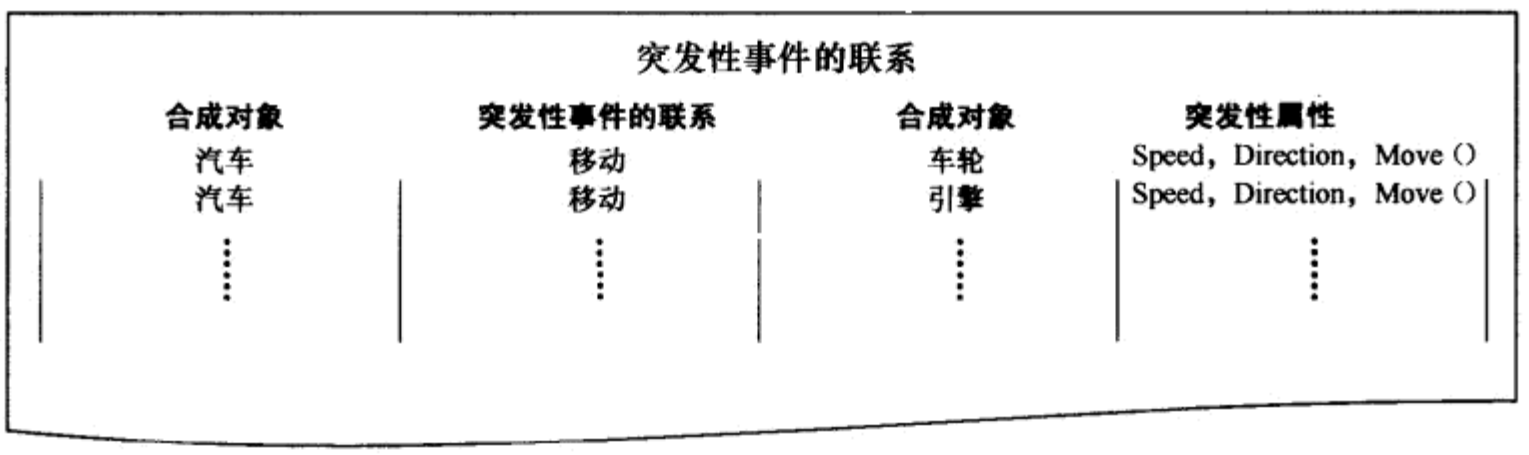


图 7.2.10 突发性关系表

3. Agent 层

Agent 层能够组织 NPC、人类的玩家，以及其他的一些物件，并且让 MMP 架构展现出卓著的人工智能。每一个 agent 物件都包含一个微型的游戏引擎（如图 7.2.11 所示），该引擎又包含了游戏引擎中 agent 的属性和专为该 agent 设置的路由。从概念上来说，每一个 agent 都会在它自身内存储自己的游戏数据。但是在实际实现上会有所差异，因为游戏数据和所有的游戏资源为了优化内存管理都是集中存储在资源池中的。最初，agent 会通过仲裁 agent 访问资源池中的资源。当资源被定位后，agent 就会直接通过资源抽象层来提取被定位的资源。

Download at www.Pin5i.Com

资源包括运行自定义游戏回路的流程和线索，以及在每个 agent 物件中的操作。Agent 物件中的事件句柄可以通过环境层中的事件句柄来处理相关事件。agent 还可以通过事件发生器来创建事件进行更新及其他操作。这会促使模拟引擎与其他引擎协作工作。

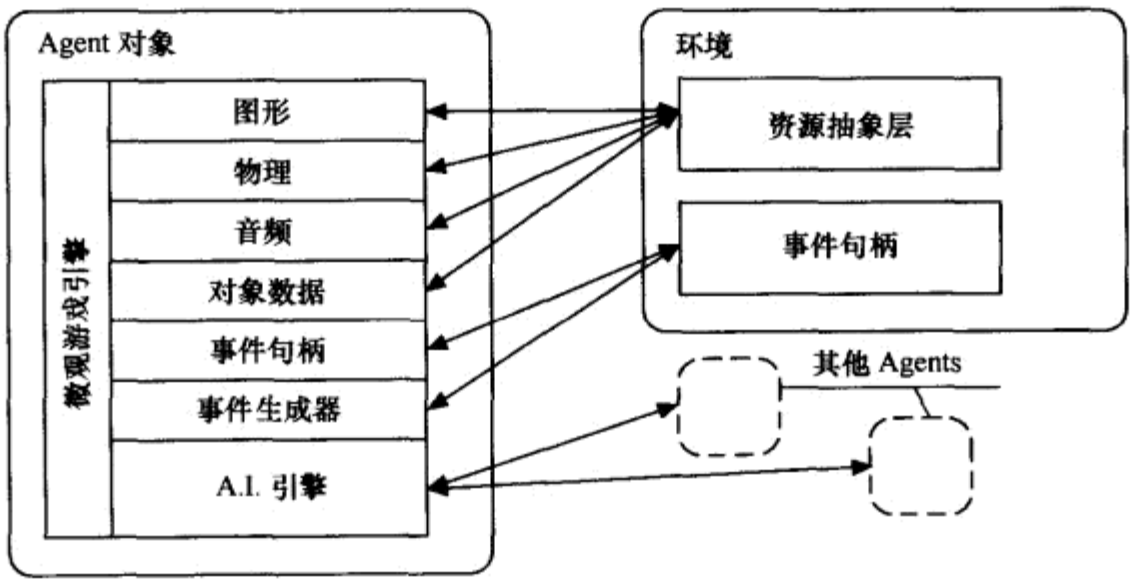


图 7.2.11　一个 Agment 的组成

AI 引擎会与其他引擎沟通并影响它们的决策过程（参见基于回馈的决策系统）。在设计 Agent 层的时候要做到与平台独立，这样复杂系统中高级游戏概念可以很容易地进行扩展，也很容易提高 MMP 游戏的游戏性。

4. 监察层

大部分复杂系统所采用的分散式控制方法也会被这个架构所使用。为了实现在一定规则情况下控制虚拟游戏环境中的游戏物件，就需要一些能够直接影响到其他 agent 活动的 agent。而这些 agent 就会被称为 overseer。这些 overseer 在管理规则方面起了很大作用，并通过它们将那些可能会导致游戏系统出现荒谬行为的突发性事件移除。然而，在引入规则的时候要十分小心，要确定这些规则不会压制住突发性事件的产生。

多重 overseer 会允许不同的规则制定者的不同规则来影响针对不同市场的玩家。对玩家的分段和分组可以从多角度出发。可以从玩家的文化多样性来分组，以确保文化价值被切实地体现出来。另外也可以通过玩家的年龄来分组，以保证一部分游戏内容不会被年轻的玩家接触到。为了允许多重个体规则制定者影响 agent 的行为，也为了保证复杂环境中的多对多联系，overseer 需要在不同的程度上来影响 agent 的行为。

如图 7.2.12 所示，随着空间区别的增加，overseer 对其他的游戏物件的影响会减小。NPC E 在很大程度上被 overseer1 的游戏规则所影响，而局部被 overseer2 的游戏规则所影响。NPC F 则局部被两个 overseer 的游戏规则影响，NPC D 只被 overseer2 的规则影响。空间上的划分可能会发生于很多方面。

例如，overseer1 能够被用来在 MMORTS 即时战略游戏中控制部队 A，同时，overseer2 能够用来控制部队 B。两个 overseer 同时对两个部队实施共同的规则。当两个部队发生战斗的时候，可以通过一个空间的划分——通过每个士兵所服役的年限长度（例如：现役的）来决定每一个士兵变节的可能性。由于两个 overseer 的物理距离不断变小（当部队相互靠拢时），

Download at www.Pin5i.Com

每一个 overseer 都会努力驱使对方部队中的现役士兵变节。

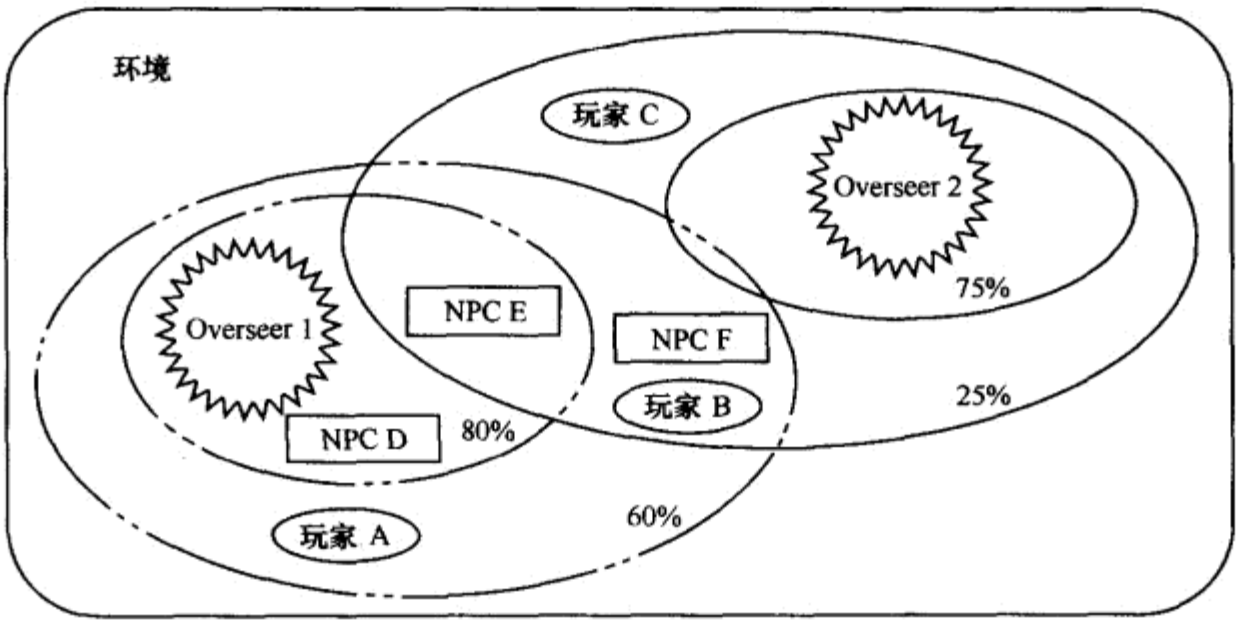


图 7.2.12 监察通过基于协议的控制来影响 Agent 行为

另外一个例子中，overseer 可以用来实现游戏中的经济规则。如果玩家 B 的经济模型被 overseer1 所影响的水平在 60%～80%，而被 overseer2 所影响的水平在 25%～75%，我们可能就会发现玩家 B 的经济模型被 overseer1 控制多过于被 overseer2 控制。

另外还有一些游戏规则会作为 overseer 规则来使用。包括鼓励公平游戏的规则、防止玩家碰撞的规则、跟踪反常行为的规则以及侦测作弊的规则，等等。

7.2.3 基于回馈的决策系统

基于回馈的决策系统是 agent 层和 overseer 层 AI 引擎的一部分。大部分游戏设计上，为了在需要做决策的时候 agent 层能够只基于游戏当前所处的状态来作出判断，都会引入回馈系统。但是类似的架构是刚性的，并且很少支持游戏物件的进化特征，因为这些游戏物件只会被非进化的游戏规则所影响。为了打破这个基于规则的范例，人们做了很多尝试。例如机械化学习[Alexander02]和神经网络[Manslow02]就被用来为系统提供一些随机性。我们的游戏架构中所用到的回馈模式，为了和其他的一些技巧保持协调，提高了系统行为的突发性。在图 7.2.13 中所说明的基于回馈的 agent 对 agent 互动模式，agent 的行为并不单存的受 AI 引擎与其自身回馈的影响，还会被其他 agent 的行为所影响，与此同时，它也会影响其他的 agent，这就对游戏环境造成了跨期间诱导发生特征[LeBlanc00]。

游戏状态会包含虚拟世界中的所有属性的状态。当 agent 做出判断时，它们需要全局的当前状态的视角，外部输入会触发决策过程，并且其他的 agent 会对类似的行为作出反应。当拥有不同决策过程的异类 agent 互相之间产生互动的时候，不可预料决策的制定就会增加。

当然，这需要假设人类玩家除了游戏对他们行为的反馈之外，只会通过其他 agent 的行为来进行学习，就同这个架构中其他的 agent 一样。然而，事实并不完全是这样，因为人类在做决定之前会参考他们现实世界中的经历。这就使虚拟游戏环境存在一种自然的随机性成为可能。Overseer 也是这个回馈模型的一部分。它们的任务就是允许且只允许可以接受的

Download at www.Pin5i.Com

agent 行为在其他的 agent 中间传播。实现图 7.2.13 中的基于回馈的决策模型有很多种方法。在实际 MMP 架构中所用到的会比较复杂，也超出了我们这里所讲述的范围。[Norlig00]和[Rogova03]提供了一些关于这些复杂的决策过程的深入研究。[Evans02]中也提到一个利用 BDI（Belief-Desire-Intention，信任-渴望-意图）[Geiss99]推理建立起来的简单一些的架构，另外，还有一个可以用于提高决策效率，对回馈和输入进行权重分析的感知器模型。

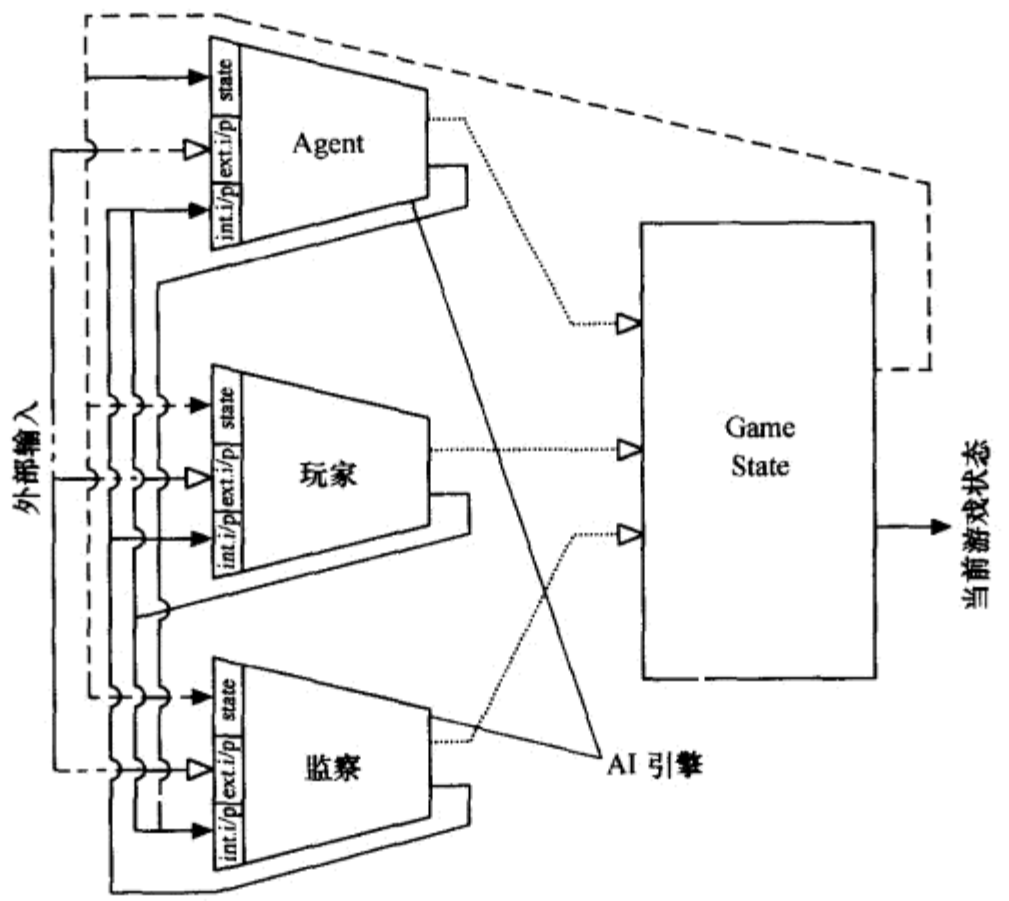


图 7.2.13　一个基于回馈的决策模型

7.2.4　总结

开发大规模多人游戏，展现给玩家全新的有趣的游戏挑战、游戏行为、游戏内容并不是一个琐碎的、微不足道的工作。通过我们这里介绍的架构，可以为在线游戏实现突发性事件和行为，而这正是下一代大规模多人游戏所必需的。

7.2.5　参考文献

[Ageia05] Ageia, "A White Paper: Physics, Gameplay and the Physics Processing Unit." March 2005. Available online at *http://www.ageia.com/pdf/wp_2005_3_physics_gameplay.pdf.*

[Alexander02] Alexander, Thor, "GoCap: Game Observation Capture." *AI Programming Wisdom,* Charles River Media, 2002.

[Booch98] Booch, Grady, *The Unified Modeling Language User Guide.* Addison Wesley, 1998.

[Evans02] Evans, Richard, "Varieties of Learning." *AI Programming Wisdom,* Charles River Media, 2002.

[Gamma94] Gamma, Erich, et al., *Design Patterns.* Addison-Wesley Longman, 1994.

Download at www.Pin5i.Com

[Garcia04] Garcia, Inmaculada, Ramon Molla, and Toni Barella, "GDESK: Game Discrete Event Simulation Kernel." *Proceedings of the 12th International Conference in Central Europe on Computer Graphics, Visualization and Computer Vision 2004* (WSCG 2004). Available online at *http://wscg.zcu.cz/wscg2004/Papers_2004_Full/E67.pdf.*

[Geiss99] Geiss, W., *Multiagent System: A Modern Approach to Distributed Artificial Intelligence.* The MIT Press, 1999.

[Kirschbaum98] Kirschbaum, David, "Introduction to Complex Systems." October 1998. Available online at *http://www.calresco.org/intro.htm.*

[LeBlanc00] LeBlanc, Marc, "Formal Design Tools—Emergent Complexity & Emergent Narrative." *Proceedings of the Game Developer's Conference* (GDC 2000).

[Manslow02] Manslow, John, "Imitating Random Variables in Behavior Using a Neural Network." *AI Programming Wisdom,* Charles River Media, 2002.

[Norlig00] Norling, E., L. Sonenberg, and R. Ronnquitst, R., "Enhancing Multi-Agent Based Simulation with Human-Like Decision Making Strategies." *Proceedings of the Second International Workshop on Multi-Agent-Based Simulation 2000,* Boston. Available online at *http://citeseer.ist.psu.edu/norling00enhancing.html.*

[Odell02] Odell, James, "Agents and Complex Systems." *Journal of Object Technology,* Vol. 1, No. 2: pp. 35–45, July 2002. Available online at *http://www.jot.fm/issues/issue_2002_07/column3.*

[Ramazani94] Ramazani, Dunia, "Contribution of Object-Oriented Methodologies to the Specification of Complex Systems." *IEEE Engineering of Complex Computer Systems* (ECCS 1995): pp. 183–186.

[Rogova03] Rogova, G., C. Lollett, and P. Scott, "Utility-Based Sequential Decision-Making In Evidential Cooperative Multi-Agent Systems." *Proceedings of the Sixth International Conference of Information Fusion,* 2003.

[Rumbaugh94] Rumbaugh, J., "Building boxes: Composite Objects." *JOOP,* Vol. 7, No. 7: pp. 12–22, November 1994.

[Seow02] Seow, Kiam Tian and Khee Yin How, "Collaborative Assignment: A Multiagent Negotiation Approach Using BDI Concepts." *Proceedings of the First International Conference on Autonomous Agents and Multiagent Systems: Part 1* (ICAA 2002): pp. 256–263.

[Seow03] Seow, Kiam Tian and Kok-Wai Wong, "Collaborative Assignment: Using Arbitrated Self-Optimal Initializations for Faster Negotiation." *Proceedings of the International Conference on Computational Intelligence, Robotics and Autonomous Systems* (CIRAS'03), 2003.

[Wikipedia05] Wikipedia, "Complex system." July 20, 2005. Available online at *http://en.wikipedia.org/wiki/Complex_system.*

[Wilkinson93] Wilkinson, M. and P. Byers, "The Engineering of Complex Systems," *IEEE Computing & Control Engineering Journal,* August 1993: pp. 187–189.

Download at www.Pin5i.Com

7.3　为游戏物件生成全局唯一标识符

Yongha Kim, Nexon Corporation
ysoyax@gmail.com

通过网络同步传输的游戏物件需要一个互相之间能够进行区分的唯一性标识符（ID）。在今天日益增加的拥有着大量用户通道的共用多重服务器系统中[Svarovky02]，每分钟会有成百万的游戏物件被建立和删除，另外它们中的一些还需要被储存起来，或者和数据库同步。如果每一个游戏物件不能够被唯一地标识，就会发生物件冲突，并且为这些物件保存副本也是不可能的。这就会导致服务期间的游戏物件的同步性的实现十分困难。

因为这个问题，建立一个全局唯一的标识符（GUID）对于在线游戏来说是一个非常必要的特征。另外，如果 GUID 能够包含有用的信息，例如物件种类和生成时间，那么 GUID 就能够很有效地优化物件跟踪和管理过程。本文将会介绍一种利用 ID 服务器和物件生成器来安全建立 64 位 GUID 的方法。

7.3.1　游戏物件 GUID 建立的需求

首先，让我们来看一下建立游戏 GUID 的需求，以及我们可以通过何种方法来满足这些需求。

1. 足够的 GUID 空间

为了管理游戏物件，对游戏物件指针发放一个 ID 句柄不单是在网络游戏中，即使在单机游戏中也会被用到。一般来说，一个 32 位的无正负的整数型数据类型就足够了，因为在大部分平台上，这类数据可以生成 40 亿个 ID。在大部分的单机游戏中，这些 ID 足够分配给游戏中生成的所有物件了。有些开发者甚至将这 32 位数据分成两个 16 位数据，用前 16 位来保存物件类别，用后 16 位来保存实际物件的 ID。然而，对于在线游戏，尤其是 MMORPG 来说，40 亿的 GUID 空间是不能满足需求的。在极端的情况下，10 000 个游戏物件需要每秒钟就建立一个 GUID，这就意味着 40 亿的 ID 会在 5 天之内就被充满。

自从在 1996 年合伙发行了 *Nexus TK*（第一个 MMORPG）以来，Nexon 已经发行了多部需要大数量级的唯一物件标识符的游戏。这些游戏中的一部分是使用了 32 位的 ID；在游戏运行了几个月到 3 年之后，这些游戏都

Download at www.Pin5i.com

耗尽了 32 位的数据空间。有些应用了 64 位 ID 的，在这些位未恰当应用的情况下，也耗尽了它们的潜在空间。当游戏运行过程中 ID 系统被耗尽的时候，就不得不改变 ID 发放方式，并且置换以前发放的每一个 ID，这确实是一件很痛苦的工作。因此，在发放 ID 的时候，一定要确保能有一个可以至少使用 10 年的空间大小。

2. 效率

通过复合时间或者 MAC 地址来建立一个 128 位的 GUID 也可以实现[ITU-TX.667]。但是，用一个 16 字节的 ID 在面对处理、永久使用存储空间，以及网络带宽方面都是比较消耗资源的。另外，现阶段的 128 位 GUID 的生成方式太复杂，不适用于需要每秒钟产生 10 000 个物件的物件生成器。对于游戏物件的 ID 来说，比较需要更容易实现的数据类型和更简单的生成方式。

3. 全局唯一性特征

由于在线游戏中的游戏物件需要在很多系统之间移动和同步，那些在生成时本地唯一的 ID 在同步时就会产生冲撞。如果两个不同的游戏物件拥有同一个 ID，那么这两个不同的系统在运行的时候就会使用不同的物件属性，甚至于使用不同的物件类别，而这就会直接导致发生错误，而且这种错误很难检测出来，更难修复。

下面说一下一个好的 GUID 系统所需要的条件。

- 过去和将来 ID 的唯一性必须被保证。
- 被不同的物件生成器生成的物件 ID 的唯一性必须被保证。
- 如果 ID 需要跨数据库或者跨系统同步时，那么这些数据库中的同一 ID 所指的必须是同一个游戏物件。

4. 关于物件的信息

在 ID 中包含游戏物件的一些信息会比没有包含这些信息的 ID 在很多方面存在优势。例如，在大多数环境中，系统日志只会记录下 ID，而并不会记录下游戏物件本身。因此，ID 中语义上的信息就能在利用这些日志进行错误诊断时提供很大的帮助。ID 中可以包含的信息有生成时间、生成器的名称，以及游戏物件的类型，等等。

7.3.2 生成 GUID

现在我们来解释一下怎样生成一个 64 位的 GUID 来满足我们上边谈到过的这些需求。如图 7.3.1 所示，这个 64 位的 GUID 由两个独立生成的 32 位的部分组成。上边 32 位是时间标记，有一个全局所属的系统——ID 服务器被生成，生成的时间标记马上被提供给了物件生成器。物件生成器会生成下边 32 位的序列数。时间标记和序列数组成最终的 64 位 GUID。

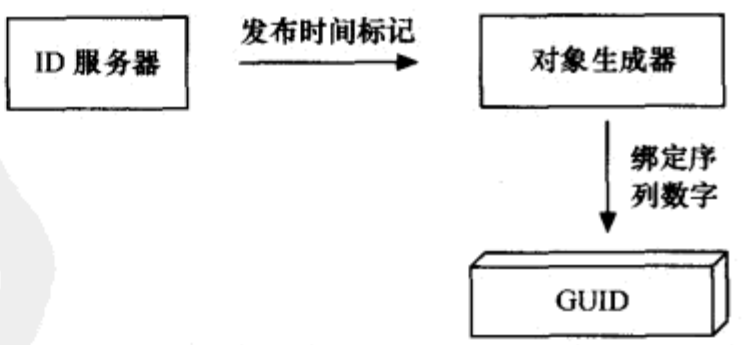


图 7.3.1 GUID 的生成。一个全局的 ID 服务器会生成时间标记并将它提供给对象生成器，其后对象生成器会生成一组系列数字，程序列数字会与时间标记绑定在一起，最后生成一组比位的 GUID

由于时间标记能够保证全局的唯一性，

Download at www.Pin5i.Com

所以本地唯一的序列数将完成最终的全局 GUID。另外，ID 服务器会负责时间标记的产生，所以物件生成器只需要单纯的考虑生成序列数就可以了。这样的一个系统就比较简单、有规则、也比较清晰，同时在管理 ID 生成上也比较有效。

1. 生成时间标记

如图 7.3.2 所示，时间标记由一组 20 位的时间印记和一组 12 位的数字块组成。时间印记用 0x0001 来标记 2005 年 1 月 1 日。这个值会每 10min 增加一次，可以使用 20 年。

如果游戏需要在服务器中持续更长时间，可以降低时间间隔的粒度，例如每隔 20min 增加一次的话可以使用 40 年。而数字块就是同一时间印记过程中时间标记个数的积累值。它会随着物件生成器每次向 ID 服务器要求时间标记而递增。当时间印记变化时，数字块会被重置为 0x000。

通过上述方法可以在 20 年内每 10min 生成 4 096 个时间标记。时间标记的全局唯一性是十分重要的，因此，只允许一个 ID 服务器能够发放时间标记。如果这个比较难于实现的话，可以从数字块中提取一部分用来标记每一个 ID 服务器。

2. 生成序列数

序列数由物件生成器生成，分成三部分：8 位的物件类型，8 位的保留部分，还有 16 位的实际序号（如图 7.3.3 所示）。物件类型所占用的位数可以允许 GUID 反映出游戏物件所指向的 GUID。可以为每一类游戏物件指定物件类型，不过这也不是必须的。在 MMORPG 中，最容易出问题，生成频率最高的游戏物件类型是道具，即使所有的道具都属于一个大类，也可以细分它们以便于管理。保留部分是为了将来可能会出现的需求预留出空间。如果不需要的话，可以用它来表示物件生成器或物件生成器类型的唯一性信息。

图 7.3.2　时间标识信息

图 7.3.3　序列数字的生成

实际序号，就像时间标记中的数字块一样，会随着每一次序列数的发放而递增。当物件生成器向 ID 服务器请求时间标记的时候，实际序号就会被重置归零。如果需要一个单独的实际序号来记录游戏物件的类型，物件生成器可以为一个时间标记生成 n×65 536 个序列数，这里的 n 就是物件类型。

7.3.3　对于特殊情况的处理

当需求并没有被合理估计和控制时，一个全局 GUID 系统会出现很多问题，包括溢出和安全问题。这里我们来简要描述一下如何应对这些问题。

1. 生成用于客户端物件生成器的临时 ID

一个游戏玩家的客户端可能会有为了复制（同步）和控制游戏物件而建立的物件生成器，但是处于安全性考虑，客户端的物件生成器没有权利生成新的可以影响全局状态的游戏物件。

Download at www.Pin5i.Com

不过有时候，也会为了需要会产生与其他进程不同步的临时物件——例如，UI 管理和对于本地事件通过一个游戏物件进行反应。在这种情况下，赋予物件生成器一个受限的权利以便产生 ID 也不失为一个解决方案。对于这种游戏物件，需要在同步处理上将时间标记中的时间印记定为 0x00 000 并声明序列数中的物件类型与全局游戏物件不同。

2. 数字块溢出

当数字块达到 0xFFE 就会导致数字块溢出。这种情况一般会发生在物件生成器过快地耗尽了实际序号，或者过多的物件生成器同时向 ID 服务器要求时间标记。这种情况要尽量避免，如果需要的话，甚至可以对物件生成器访问 ID 服务器的通路设置限制。例如，一般用户客户端物件生成器不应该有访问 ID 服务器的权限。并且 ID 服务器能够提供时间标记的物件生成器的数量应该保持在 100 以下。

如果这种情况无法避免的话，可以增加时间印记并重置数字块。如果时间印记已经被由于到时而自动增加，那么就没有必要再重新增加了。

3. 实际序号溢出

物件生成器只能根据给定的时间标记来组合序列号，所以说当实际序号达到最大值的时候，就无法生成更多的 GUID 了。当这种情况发生的时候，物件生成器可以向 ID 服务器申请一个新的时间标记。这种处理也许不能即时地解决问题，也不能立即生成新的 GUID，因此可以使用保留部分来生成新的实际序列号，以解决实际序列号溢出问题。为了防止实际序列号溢出，可以设定在序列号超过某一值的时候，如 0x8 000，物件生成器自动向 ID 服务器申请新的时间标记。

7.3.4 总结

生成一个新的 ID 并不是很难，但是想要设计一个在运行过程中不需要修改的 ID 问题系统就不那么简单了。我们希望我们的这些从失败中吸取的教训可以帮助你们轻松地解决类似的问题。

7.3.5 参考文献

[ITU-T X.667] ISO/IEC 9834-8:2004 Information Technology, "Procedures for the operation of OSI Registration Authorities: Generation and Registration of Universally Unique Identifiers (UUIDS) and their use as ASN.1 Object Identifier Components." ITU-T Rec. X.667, 2004.

[Svarovsky02] Svarovsky, Jan, "Minimizing Latency in Real-Time Strategy Games." *Game Programming Gems 3,* Charles River Media, 2002.

Download at www.Pin5i.Com

7.4　利用 Second Life 为大规模多人在线游戏原形设计游戏概念原形

Peter A. Smith,
University of Central Florida
peter@smithpa.com

7.4.1　简介

随着大规模多人在线游戏的出现（MMOG），因为玩家不断对一个更成熟的、更能沉醉其中的虚拟世界的追求，游戏制作上的平均大小和复杂性都在不断提升。新的基于预定的服务模式被大多数此类游戏所采用，为了保证持续的资金流需要建立一个在创造性和革新性上来说都十分完美的游戏环境以取得游戏的繁荣。然而，这并不一定正确，因为现在 MMOG 所处的一个两难境地就是游戏开发和运营的成本不断增加，而出版商比以往更加注重他为项目做的每一份投资。因此，开发商如何在完全没有 MMOG 基础设施的时候验证一个革命性的 MMOG 流派，而该 MMOG 流派能否立刻取得显著的资金回流呢？他们可以利用其他公司的优势技术，例如 Linden Labs 的 MMOG 环境 *Second Life*。

7.4.2　为什么要用到 Second Life

Second Life 的核心是一个 MMOG 环境。它通过“Linden Dollars”建立其内部经济并为玩家提供其他 MMOG 所不具备的特殊体验。*Second Life* 除去它的商业副本之外的另一个主要特征就是，整个世界以玩家创造的内容运行，为了做到这一点，Linden Labs 决定允许玩家拥有他在游戏中创造出来的内容的知识产权[Ondrejka04]。这实际上是在游戏开发领域从来没有出现过的。因为大部分公司都会保留为他们自己的软件创造 mod 的权利。而 Linden Labs 的独特战略保证了该公司在竞争日益激烈的 MMOG 领域能够持续发展。*Second Life* 在 2005 年 8 月用户总数已经达到了 40 000[SecondLife05b]。参考图 7.4.1 可以看到 *Second Life* 现在市场所占的位置。

Second Life 甚至还为创造者提供买卖、赠送、交易其物品的经济机制，并且丝毫没有对该流程进行控制。这个特点就导致了一个比其他任何一个 MMPG 都要稳固的经济系统。在作者撰写本文的时候，每千元 Linden Dollar 在 Gaming Open Market 中市值相当于 4 美元[GOM05]。*Second Life* 已经成

Download at www.Pin5i.Com

为一个你永远也不知道将会发生什么的世界。当然，*Second Life* 并不是唯一一个拥有玩家创造这一概念的游戏。

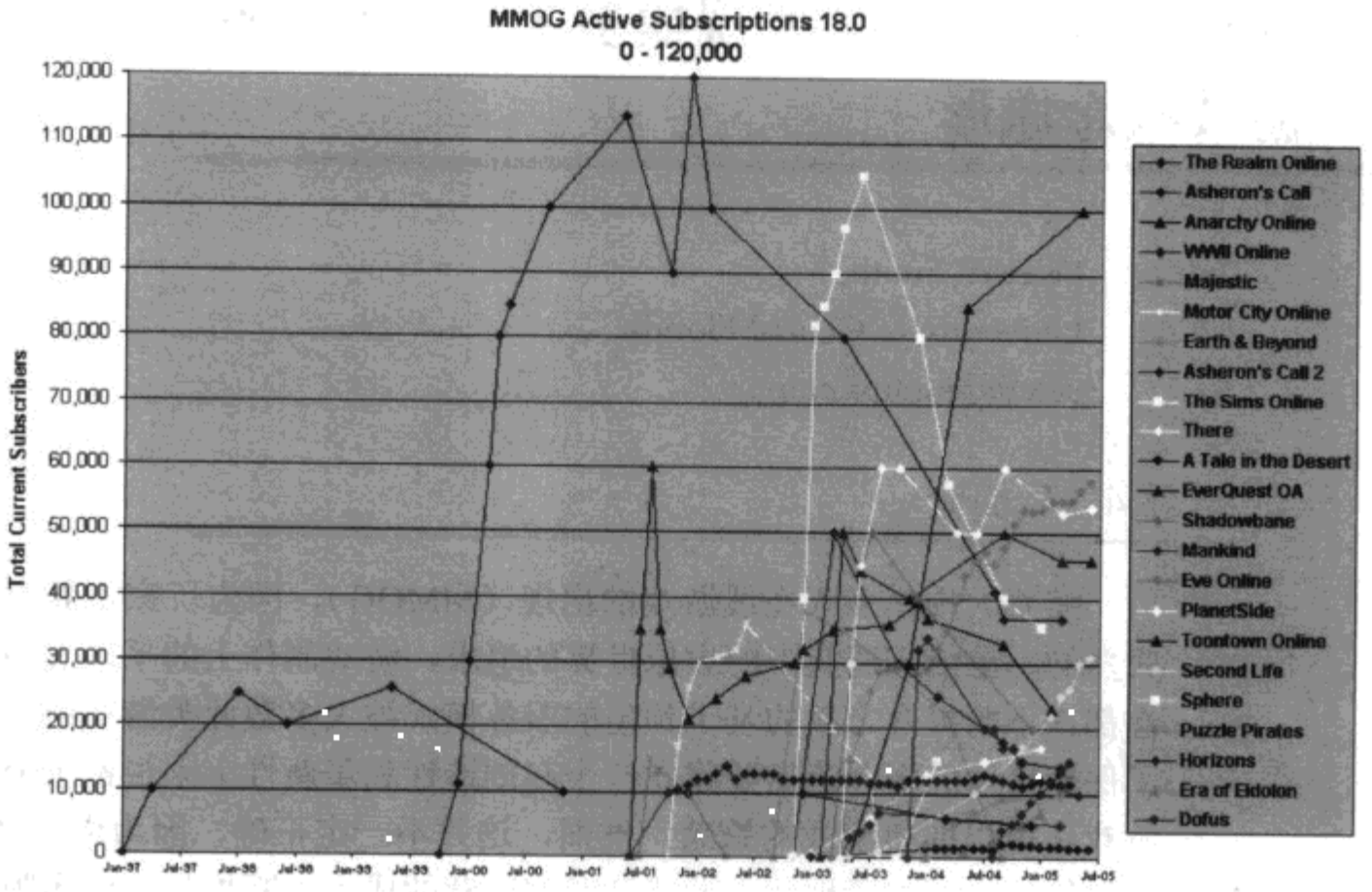


图 7.4.1 MMOG 活动的账号[*Woodcock05*]

1. 现有的其他选择

另外一些可以替换 *Second Life* 的游戏还有 *There* 和 *Action Worlds*。*There*（www.there.com）与 2003 年发布，并且有很多和 *Second Life* 相似的优点。玩家可以在 There.com 中建立属于自己的游戏内容。其缺点是创建游戏内容需要通过一个审批过程，这会耗费珍贵的开发时间来确保没有不合适的内容被放到了游戏世界中。*Second Life* 并不存在这样的限制。因此可以说，There.com 更像是 *The Sims* 而不像 *Second Life*[Book05]。

Active World（www.activeworld.com）比大部分的 MMOG 有着更长的历史。它开始于 1997 年，*Active World* 为学术领域提供了很多的基础性支持。其中最引人注目的就是可以在它的服务器上建立自己的世界。当然，定价模式也和任何专门的服务器定价一样。在这种模式中，你可以建立属于自己的游戏世界的内容，或者是可以允许被批准的玩家访问的广域的内容。这可以作为 *Second Life* 在隐私性方面的一个参考，然而，*Active World* 缺少一个内建的可以由玩家进行测试的机制。虚拟世界中可获得玩家的认识对于进行 Beta 测试是很重要的[Book05]。

当对这些可以作为游戏原形的选择进行评估时，你可以从 Virtual World Review 中找到更多的信息。不过最近的一次评估表示，*Second Life* 在很大范围内拥有很强的特点，能为用户带来很大好处，因此它是现阶段的最好选择。

2. *Second Life* 的特点

Second Life 有很多特点使其成为所有有抱负的 MMOG 开发商的一个有力工具。*Second*

Download at www.Pin5i.Com

Life 的核心是一个即时的、可无限扩展的流动的 3D 世界。其底层技术是一个专门基于网格的网络配置。这对于开发者来说，当玩家在游戏世界中移动的时候，他可以在各台服务器之间毫无延迟和读取的进行无缝移动，这就使巨大游戏世界成为可能。在图 7.4.2 中，可以看到一些正方形将整个 *Second Life* 的游戏地图分成若干块。每一个正方形都代表一个不同的区域。每一个区域等于现实世界或专用服务器中的 16 平方米。这样你就会通过计算整个世界地图中区域的数量来获取一个关于 *Second Life* 的游戏世界大小的概念了[Onderjka04b]。

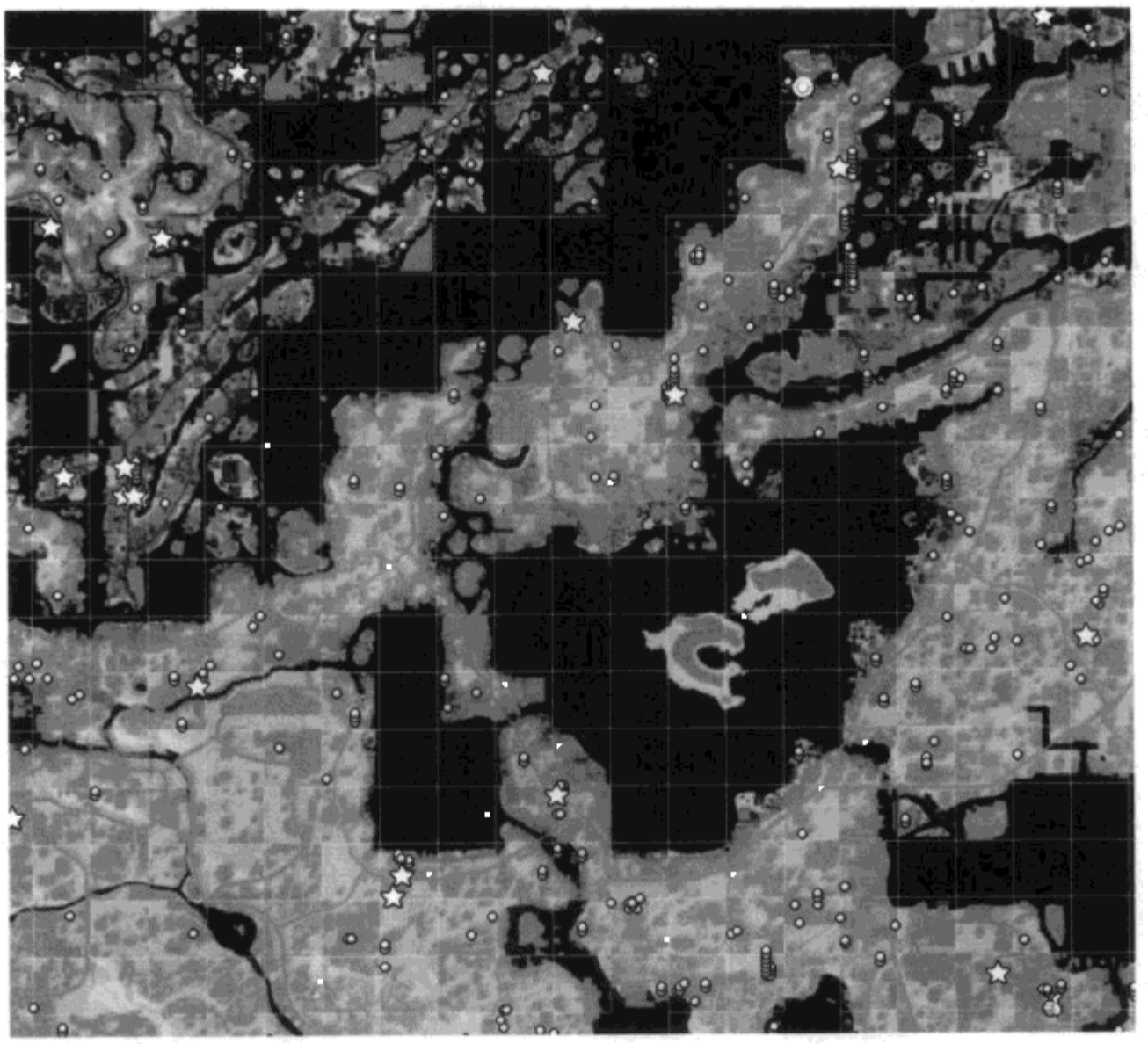


图 7.4.2　“第二人生（Seconal Life）”世界地图的一个区域展现了任务的位置和条件。该图引自第二人生，林登实验室

在 *Second Life* 中的 Avatar 也可以被无限制地自定义。除了一些可以改变玩家 Avatar 形状的简单工具外，还可以为他们提供自定义的服装和动作。甚至还可以从一些流行的开发工具（例如 Poser™ 和 PhotoShop）中导入这些动作和服装。在自定义玩家角色的同时，开发者还可以建立和更改他们自己的物件，以供角色与之互动。简单易用的几何建筑工具可以通过玩家控制台来使用。如果这些工具还不能满足你的要求的话，你甚至还可以通过一些流行的模型包来读取模型[SecondLife04]。

Second Life 中的一切无论从外观还是行为上都能够遵照创造者的意愿。包括拥有可自定义的角色和物件的脚本语言，该脚本语言是基于 C++编程语言的一个松散结构[Brashears03]。通过 Linden 脚本语言，开发者可以为世界中的每一个物件制定一个复杂的行为。只要你敢想，从简单的棒球到复杂的汽车都可以在机中为其编写脚本。*Second Life* 的另一个很强的特点就是整合了 Havok 物理引擎。这个单机的中间件曾经为 *Half-Life2* 和 *Full Spectrum Warrior* 等很

Download at www.Pin5i.Com

多受欢迎的游戏提供了及其精确的物理模拟效果[Havok05]。一般来说开发团队要想利用该技术，所花费的成本是很大的。关于 Havok 的更多信息请参考 www.havok.com。

7.4.3 初试“第二人生（Second Life）”

开始接触 *Second Life* 可能是一种令人畏缩的任务。首先要登录 www.secondlife.com，注册一个账号，然后下载软件。当玩家第一次进入到游戏世界时可以通过帮助教程学习如何开始。但是非常遗憾的是，这个教程并不会帮助你，也不会使你学到如何在 *Second Life* 中进行开发。因为其类似于一个用户中心，Linden Labs 打算将玩家训练成为一个 *Second Life* 的居民。好处是，这些用户基础教学非常贴心也非常全面，有利于你将来逐渐对游戏进行深层次的挖掘。另外，现有的一些各式各样的资源也会帮助新玩家尽快地成长。一些训练视频、文档、活动教程/教程程序等多种训练选择，*Second Life* 也会提供[Second Life04]。

一开始，最好先看一编新手指南，它对如何使用 *Second Life* 进行了很好的介绍。新手指南之后是“脚本编辑新手指南”。然后，您最好能够看一编 Wiki 并参加一些 Learning Center 提供的教学，其中一个 Learning Center 在游戏中的位置如图 7.4.3 所示。玩家社区对这两个部分的选择也支持得比较好。除了这些资源之外，在 *Second Life* 社区中，玩家也有一种喜欢帮助他人的趋势。你只需要在虚拟世界中对人们说话，询问你所需要的信息就可以了。

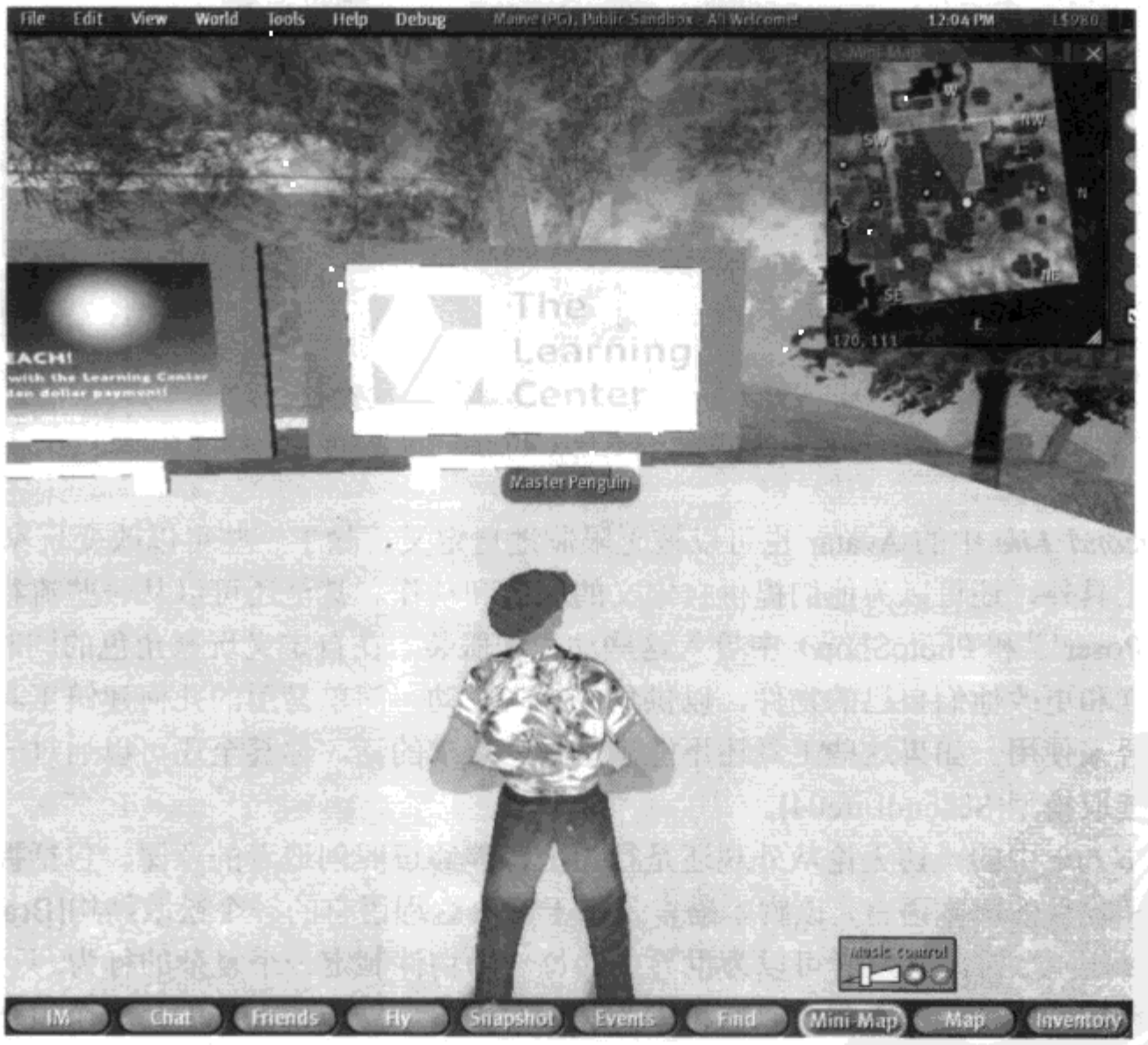


图 7.4.3 人物站在 Learning Center 前，截图引自“第二人生”，林登实验室

Download at www.Pin5i.Com

7.4.4　在 Second Life 中的设计要点

当游戏设计师在设计一个游戏的时候，需要考虑到这个开发平台的优势和制约因素。并不是每一种游戏都可以很好地转换到另外一个系统中，*Second Life* 也不例外。

1. 优势

Second Life 是一个独特的开发环境，适用于各个风格的 3D 游戏。从简单游戏到复杂的 RPG，*Second Life* 能够适合任何游戏类型。*Second Life* 的一个主要强项就是它有能力承受游戏环境中的多玩家数量，以及其内建经济模型。

很多成功的 *Second Life* 游戏都被集中到赌博游戏、FPS 游戏以及 puzzle 游戏。*Second Life* 也是第一个向游戏市场开发多人休闲游戏的游戏。而所有这些游戏的社会学设计都能吸引 *Second Life* 玩家。你甚至可以利用你的时间去钓鱼，如图 7.4.4 所示。

图 7.4.4　在游戏“第二人生”中钓鱼都是独特的体验，截图引自“第二人生”，林登实验室

在 *Second Life* 中利用 Havok 的技术进行物理模拟可以模拟真实的汽车、枪和玩具。从大炮中射出人到汽车竞速，就像是一个 *Unreal* 的 *Second Life* 版本，*Second Life* 中的物理模拟可以做到极尽真实。*Second Life* 中一个很受欢迎的场景就是能让玩家驾驶飞机，这种让人印象深刻的玩法就依赖于物理模拟。

另外一个 *Second Life* 的很有趣的优势是，这个游戏能够改变它所支持的玩家组的大小。如果 10 000 想要同时玩一个游戏，游戏可以在不同的区域进行。所有的物件都可以被复制并重复使用，不用考虑开发成本的影响。

Download at www.Pin5i.Com

2. 制约因素

尽管 *Second Life* 看似开发 MMOG 原形的一个完美的解决方案，但这也并不是说它没有任何局限性。就像大多数的在线游戏环境一样，*Second Life* 也会受到网络延迟的困扰。当玩家进入一个新的区域的时候，会很奇怪地看到建筑物凭空出现。人物有时候也会裸体一段时间后再慢慢地穿上衣服。墙上的图片也会比较模糊，并且只有当玩家在这个场景中呆了一段时间之后才会逐渐清晰起来。网络延迟是在游戏设计时必须要考虑的，否则就会导致如图 7.4.5 中两个图的那种明显的放置差异。任何一个需要用到快速更新、大量物理程序、复杂脚本或者大量物件的游戏都会导致不恰当的网络延迟[SecondLife05a]。

图 7.4.5 网络延迟是游戏开发商面临的主要问题。截图引自“第二人生”，林登实验室

围绕网络延迟进行设计可能是一种比较困难的过程，但也不是没法解决的。*Second Life* 在这方面做得还不错，一般来说玩家并不会感觉到网络延迟的影响。实际上，建筑物一个一个凭空出现的感觉会增加人们走进梦幻的感觉。另外，你可以在 Wiki 主页中找到一个一个更加详细的处理网络延迟的列表[SexondLife05a]。

7.4.5 原形的开发

在开发一个原形的时候，最好从一个简单的物件开始慢慢建立 *Second Life* 中的整个游戏。游戏可以由任何数量的脚本、原始模型以及贴图组成。下面这个例子介绍了如何创建一个简单的玩具——Infinity Ball 的过程（请不要和一个玩具的注册商标搞混）。参见图 7.4.6 来了解一个 Infinity Ball 是怎样创建的。

利用 *Second Life* 开发的一个主要好处就是，所有一些你看到和创建的都在游戏世界中。在图 7.4.6 中，包括贴图、几何、文件和脚本的窗口。另外还有其他很多窗口可以使用。特别是 debug 菜单，这些平常是见不到的，你可以按 Ctrl+Alt+Shift+D 键来激活 Debug 菜单。Debug 菜单可以打开错误信息窗口。Debug 窗口还可以用于离开地形的贴图，这样查找几何上的缺失就会更加容易。它还可以用来监视环境变量，例如风。

Download at www.Pin5i.Com

图 7.4.6　Infinity ball 的创造物。该界面允许直接访问所有部分的各个进展。截图引自“第二人生”，林登实验室

为了创建你自己的 Infinity Ball，可以按如下步骤进行：

- 首先，进入游戏地图中允许创建（Create）的区域。这里大概类似于一个公共的沙盘（像这样的沙盘，游戏中还有很多），或者为你想要创建的项目购买一块土地。
- 右键单击地形并选择弹出窗口中的创建（Create）按钮。这个操作会为你提供一个可以创建的原始物件（primitive object），原始物件的范围可以从一个盒子到一棵树。
- 选择球体，鼠标指针就会变成一个魔杖的形状。
- 点击地形，一个球将被创建出来。
- 现在为这个球获取贴图。可以在文件菜单中选择上传图片（Upload Image）。

- 在随书光盘中找到贴图文件 Infinity_Ball_Texture.tga 并选择它。
- 在视图（View）菜单中选择清单（Inventory）。
- 点击贴图（Texture）文件夹并拖曳 Infinity_Ball_Texture 到球上，这时球体就会被加上贴图了。
- 现在，用创建（Create）窗口中的移动（Move）工具来调整球的位置。
- 回到清单（Inventory）窗口，选择创建（Create），然后再选新建脚本（New Script）。
- 新的脚本就将会出现在清单（Inventory）窗口的脚本（Script）文件夹中。
- 右键点击新建脚本（New Script）并重命名为 Infinity Ball Script。

Download at www.Pin5i.Com

- 双击 Infinity Ball Script 打开脚本编辑器（Script Editor）窗口。
- 在脚本编辑器（Script Editor）窗口，粘贴下列代码：

```
float max = 8.0;
default
{
    // This code runs when a player touches the object
    touch_start(integer total_number)
    {
        float choice;
        integer result;
        // llFrand creates a random number
        choice = llFrand(max);
        // Casting the float as an integer
        // truncates the decimal
        result = (integer)choice;

        if(result == 0) llSay(0, "Yes, of course");
        else if(result == 1) llSay(0, "Can not predict now");
        else if(result == 2) llSay(0, "Signs point to yes");
        else if(result == 3) llSay(0, "Not looking good");
        else if(result == 4) llSay(0, "You can count on it");
        else if(result == 5) llSay(0, "It is certain");
        else if(result == 6) llSay(0, "No");
        else if(result == 7) llSay(0, "YES");
    }
}
```

- 最后，将 Infinity Ball Script 从清单（Inventory）窗口拖曳到 Infinity Ball 上。

利用这个脚本，如果玩家碰到了球，球就会从 8 种反应中随即播放一个，就像现在流行的玩具一样。以上是一个简单的实验，但是它包含了用 *Second Life* 创建互动型物件的基本过程。通过慢慢联系，一切皆有可能。

7.4.6 一个成功的例子

在 *Second Life* 社区中最成功的应该算是 *Tringo* 了。*Tringo* 是一个 bingo 和 *Tetrise* 的奇妙的混合体。它取得的巨大成功使其在现实世界中也获得了商业发行的许可。这个游戏是由一位 30 岁的澳大利亚程序员 Nathan Keir 在学校圣诞节假期的时候设计的，一开始 *Tringo* 出现的时候，没有人会预料到它的成功。Donnerwood Media 已经准备在移动和 Internet 游戏平台上发行[Grimeos]。

Tringo 在处理 *Second Life* 游戏环境的优势和劣势的平衡上做得十分完美。当付费进去玩游戏后，玩家持有一张类似于 bingo 卡的分数卡，这些卡会覆盖在类似于 Tetris 的小碎块上。游戏会不断调出这些小碎块，不过这些小碎块并不能旋转。首先用这些碎块填满他们卡片的玩家要喊“Tringo!”然后他就会赢得奖金。这个东西很简单也很让人上瘾，特别是在玩的过程中还可以和其他的玩家进行一些交流[Grimes05]。

Download at www.Pin5i.com

图 7.4.7　人物在玩 Tringo 游戏。截图引自“第二人生”，林登实验室

7.4.7　总结

Tringo 是一个简单的天才之作，它也是对“*Second Life* 能实现什么”的一次测试。*Second Life* 还处在开发的早期阶段，但是也会出现像 *Tringo* 这样的杰作，现在被人们意识到的“*Second Life* 能实现什么”只是冰山一角。下一个百万美元级的宏大游戏很可能会在 *Second Life* 中实现。有可能下一个超级休闲游戏也会在 *Second Life* 中出炉。一直使自己处于新事物的边缘，并且拥有实际上无限的资源去进行实验，就会在 MMOG 的历史上创造出很多重量级的作品。因此，这也是将会面临的一个大问题：你怎么来度过自己的第二生命（*Second Life*）？

7.4.8　参考文献

[Book05] Book, Betsy, “Virtual Worlds Review.” August 15, 2005. Available online at *http://www.virtualworldsreview.com*.

[Brashears03] Brashears, Aaron, et al., “Linden Scripting Language Guide.” 2003. Available online at *https://secondlife.com/download/guides/LSLGuide.pdf*.

Download at www.Pin5i.Com

[GOM05] Gaming Open Market, "Gaming Open Market Homepage." August 15, 2005. Available online at *http://www.gamingopenmarket.com/*.

[Grimes05] Grimes, Ann, "Tetris Meets Bingo." *The Wall Street Journal,* March 3, 2006: p. B3.

[Havok05] Havok.com, "Havok: Dynamic Gameplay." August 15, 2005. Available online at *http://www.havok.com*.

[Ondrejka04a] Ondrejka, Cory, "Aviators, Moguls, Fashionistas and Barons: Economics and Ownership in Second Life." September 23, 2004. Available online at *http://www.gamasutra.com/resource_guide/20040920/ondrejka_01.shtml*.

[Ondrejka04b] Ondrejka, Cory, "A Piece of Place: Modeling the Digital on the Real in Second Life." June 7, 2004. Available online at *http://papers.ssrn.com/sol3/papers.cfm?abstract_id=555883*.

[SecondLife04] "Second Life Starter Guide." June, 2004. Available online at *https://secondlife.com/download/guides/starter_guide.pdf*.

[SecondLife05a] Second Life Community, "LSL Wiki: Home Page." August 15, 2005. Available online at *http://secondlife.com/badgeo/wakka.php?wakka=HomePage*.

[SecondLife05b] Linden Labs, Incorporated, "Second Life Home Page." August 15, 2005. Available online at *http://www.secondlife.com*.

[Woodcock05] Woodcock, Bruce Sterling, "MMOG Active Subscriptions 0–150,000." August 15, 2005. Available online at *http://www.mmogchart.com/*.

Download at www.Pin5i.Com

7.5　稳定的 P2P 游戏 TCP 连接及敏感 NAT

Larry Shi
shiw@cc.gatech.edu

在Internet 游戏技巧中，我们将会介绍一种在家庭和 ISP 网络地址转换器（NAT）后台的工作站之间建立 P2P 网络进程的技术。该技术也适用于很多基于客户端-服务器架构的在线游戏，因为一个独立的在当前互动的客户端之间的 P2P 连接能够减轻某些服务器的负担。游戏开发者可以利用 P2P 频道来传输声音、聊天、文本以及实时视频等数据，并以此来增强玩家的游戏体验。我们会讨论 NAT 如何影响 P2P 连接，介绍一些在 NAT 后台建立稳定的 P2P 连接的技巧，并提供一些能直接被游戏开发者使用的源代码。我们希望随着连通性的增强，游戏将不仅仅变得更有趣，而且会将更多的人带到游戏中来。

7.5.1　问题

作为一名游戏开发者，如果你想让你的游戏应用程序能够以 P2P 模式互相连接并不占用服务器的带宽，无论你怎么设计你的解决方案，都会必然的要面对 NAT 这道墙。几乎所有的商业家用网络路由，例如 Linksys、Netgear 和很多 ISP 都要依靠 NAT 将家用计算机连接到 Internet 上。通过 NAT，ISP 可以避免为每一个用户确定静态 IP 地址。这就允许 ISP 扩展他们的 IP 地址，并能够支持超出他们的静态地址范围的用户。NAT 也允许家庭用户通过一个独立的专网地址空间来设置他们的家庭网络，并对外部网络隐藏家庭网络的物理布局。NAT 能够实现这些是因为它使用动态地址和端口转换。NAT 的一个四段的 IP 地址（本地 IP 地址、本地端口、远程 IP 地址、远程端口数）唯一的定义了每一个网络连接或进程。为了在家庭网络和外部网络（如 ISP 或者全局 Internet）之间建立一座桥梁，一台 NAT 设备会从它自身的端口空间为每一个通过它的向外连接都定位一个端口。NAT 设备会自动地将每一个向外发送的信息包的源 IP 地址替换成 NAT 设备的 IP 地址，并将源端口重新定位为外部端口号。

图 7.5.1 解释了一个典型的家庭网络系统，该网络使用家用路由器作为一组计算机和其他计算机设备的 NAT 设备。这台 NAT 设备有一个 ISP 指定的、全局的路由 IP 地址 172.168.5.23。在 NAT 设备后面是一个使用本地 IP 地址空间 10.0.1.x 的个人网络。假设家用计算机，10.0.1.3 打算连到一个全局地址为 23.24.1.2 的服务，端口号是 1234。NAT 设备就会截取这个连

Download at www.Pin5i.Com

接请求（包括 TCP SYN 数据包）。根据 NAT 协议，为了建立一个新的连接，这个连接请求会被定位到一个新的端口号上，例如 5462。然后它会替换向外发送的数据包中的源 IP 地址，将其换为自身的全局 IP，并且源端口号也会被替换成新定位的端口号，5462。这样，NAT 就会用（10.0.1.3:31420、172.168.5.23:5462）来唯一地标识这个连接以及为了保持这一连接所作的转换工作。如图 7.5.1 所示，每一个带着 10.0.1.3:31420 这一源地址的向外部发送的数据包，NAT 设备都会将这个地址转换成 172.168.5.23:5462。而每一个从 172.168.5.23:5462 向内发送的数据包，NAT 都会将其转换成 10.0.1.3:31420。

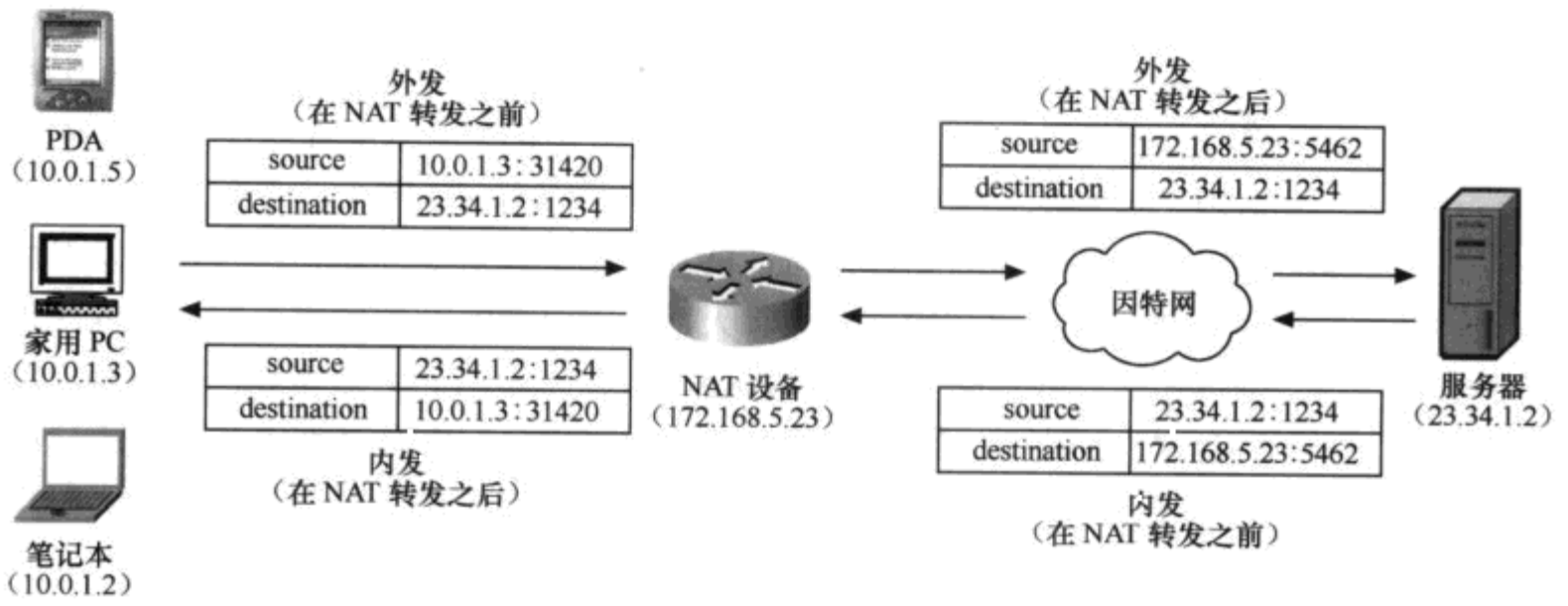


图 7.5.1 网络地址转发设备

但是当 NAT 应用到在线游戏时有一点复杂，因为只有源于内部网络的连接可以被建立。NAT 设备会丢失所有由外部网络向内部网络发送的连接请求，因为没有一个已经建立起来的序列被记录。这些丢失的请求被称为主动连接尝试（unsolicited connection attempts）。换句话说，就是在 NAT 之后的计算机无法连接到另外一个 NAT 之后的计算机上。当然，纯粹的基于客户端-服务器的游戏不会存在这个问题，因为服务器通常都会使用全局 IP 地址而不是 NAT。

7.5.2 技术水平

由于一些新技术的应用，如 P2P 文件共享、VoIP 电话会议以及在线游戏，这些技术都需要突破由于 NAT 所导致的连接限制问题。幸运的是，最近已经出现了大量针对这一问题的解决方案。其中有一项为两台均在 NAT 后的计算机解决 P2P 连接问题的技术被称为“hole punching”（打孔）。hole Punching 可以在 UDP[Guha05]和 TCP 环境下工作[Biggadike05][Eppinger05][Ford05]。在 hole punching 中，两台计算机在初始时都会使用对方终端的私有（内部）地址和公共（外部，转换过的）地址，成为一个为了另外一个终端发送向外连接意图的终端。这样，向外发送的需求就会很有效地在 NAT 上打孔，或者换句话说，将完全转换过的 IP 地址和端口号加到 NAT 上，这就使 NAT 能够接受从另外一个终端向内传输的数据包了。hole Punching 对于应用程序来说是简单而且透明的。它能够很有效且稳定地在大多数表现良好的商业 NAT 设备上工作。

另外还有一些进行 TCP hole punching 的方法。NAT-BLASTER 论文[Biggadike05]中介绍

Download at www.Pin5i.Com

过另外一个 TCP hole punching 的手段，它通过一个源 IP 的欺骗服务器和顺序号协调器来建立两台 NAT 后的计算机的 TCP 连接。[Eppinger05]中还介绍了一个利用常规 socket 编程和一组“协奏（orchestrated）”事件来连接两台 NAT 后的终端计算机。一个最新的研究报告[Ford05]还讨论了利用特殊的 SO_REUSEADDR 和 SO_REUSEPORT socket 选项来建立 P2P TCP 连接的可能性。我们将基于[Ford05]来说明怎样建立 TCP hole punching。将会涉及到的技术包括 TCP/IP 网络、NAT 操作原理，以及 socket 编程。

7.5.3　方法

图 7.5.2 描绘了一个目标网络系统的设置。为了不失去论证的一般性，我们假设有两台客户端计算机，分别是节点 A 和节点 B。网络和 NAT 后面的设备分别是 NA 和 NB。另外，假设节点 A 处于一个专门的家庭网络 10.0.1.x 中，其内部 IP 地址是 10.0.1.5。节点 B 是在一个内部网络 192.168.0.x 上，内部 IP 地址为 192.168.0.10。NA 的全局路由地址为 172.168.5.23，NB 的全局路由地址为 155.55.80.5。我们的目的是利用 NAT hole-punching 的方法建立节点 A 和节点 B 之间传输在线游戏数据的 TCP 连接。按照图 7.5.3 中的步骤，我们将使用典型的 socket 编程来实现这一目标。

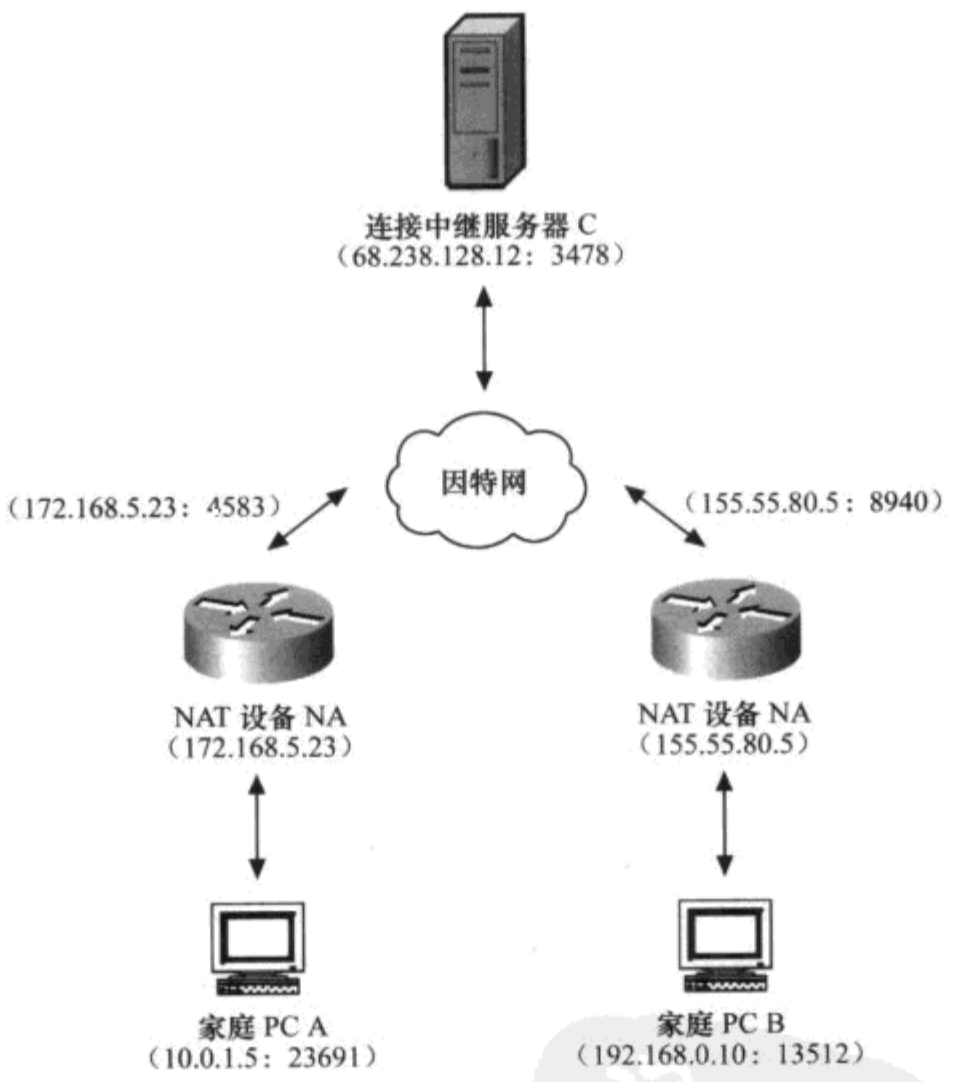


图 7.5.2　两个客户端机器和一个连接服务器

如图 7.5.2 所示，为了让 hole-punching 能够正常运作，我们需要一台连接中继服务器 C（Connection Broke Server C）来帮助建立 A 和 B 之间的连接。C 必须有一个全局路由 IP 地址和一个已知的端口号作为连接中继服务器的进入点。假设 C 的全局 IP 地址为

Download at www.Pin5i.com

68.238.128.12，用作连接中继服务器公用的端口号为 3478。进行 hole-punching 的步骤，参见图 7.5.3 所示。

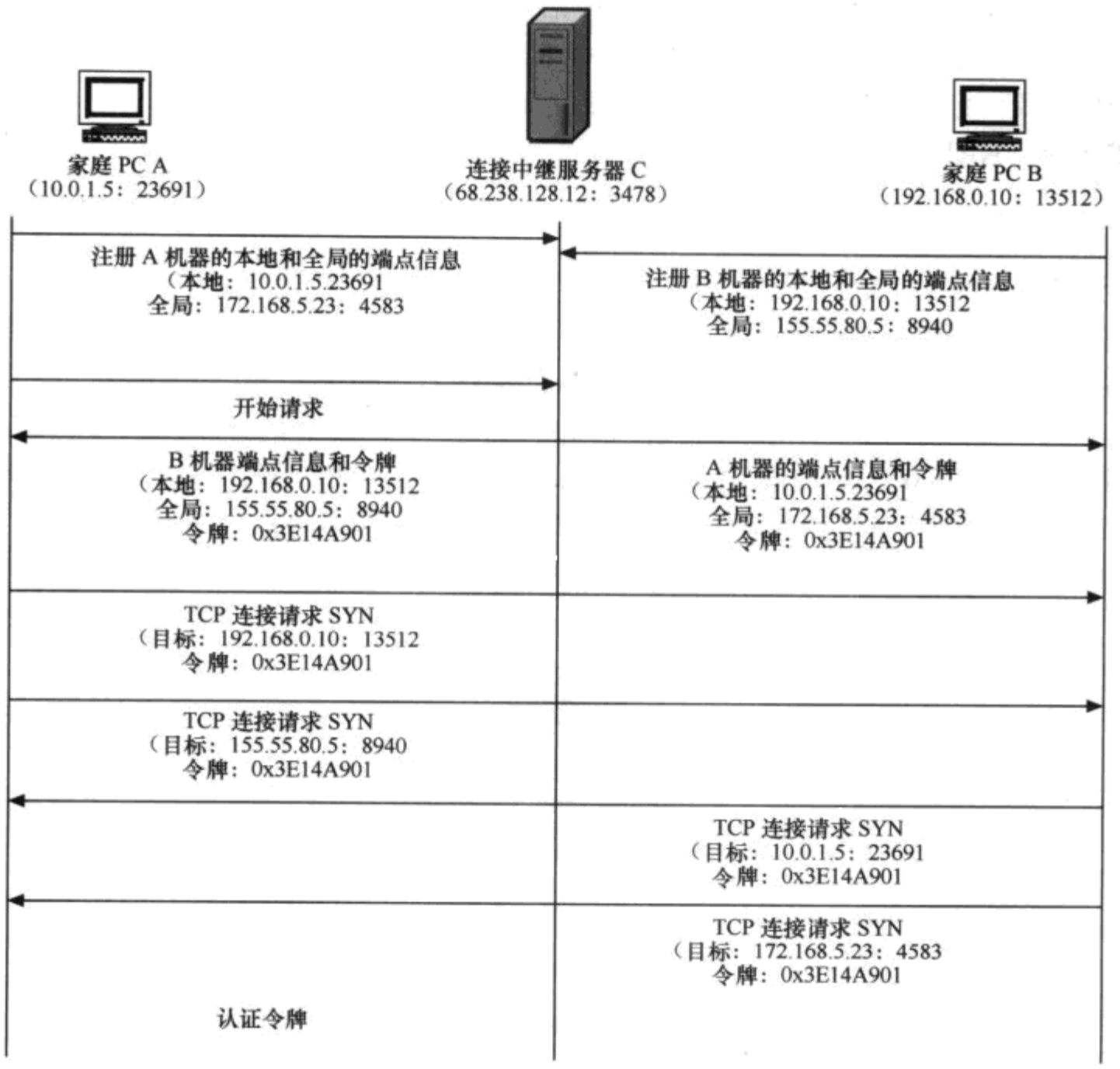


图 7.5.3 信息穿透传递步骤，1）端点注册阶段，2）启动阶段，3）连接阶段，4）认证阶段构成

为了让 NAT 接受从 NAT 设备外某一源地址发送的、以 NAT 设备内的客户端计算机和服务连接端口为目标的传入连接意图，客户端的计算机首先要建立一个通向 NAT 设备的终端网络地址转换表的转换入口。这个转换入口会将一个内部 IP 地址和端口号映射到一个外部 IP 地址和未被使用的 NAT 端口号上。为了实现这一步骤，客户端计算机会以预期的接收数据包的全局源地址和端口号为目的的地址和端口号，发送一个有优先权的外出数据包。

我们前边曾经提到过，一个由本地地址、本地端口号、远程地址和远程端口号所组成的序列能够唯一地标识一个网络连接或会话。NAT 设备会记录每一个外出意图的序列。随后，当 NAT 设备收到向内的数据包与记录的网络连接或会话相同的话，它就会将数据包的目的地址和端口号转换成在 NAT 设备内的可路由的本地的私有地址和端口号，并将数据包发送到 Internet 上的目的地。这个有优先权的外发数据包会在 NAT 设备上建立一个新的连接（或会话记录），然后当 NAT 设备遇到从预期连接意图方法过来的数据包时，它就会认为该数据包是属于已经被激活的连接的一部分，并接收。这就是 hole-punching 的基本原理。

Download at www.Pin5i.Com

1. 终端注册阶段

在终端注册阶段，每一个客户端的计算机上都会建立一个与连接中继服务器相联系的 TCP 连接，并且注册它的内部地址及端口号和外部公共地址及端口号。客户端可以通过 getsochetname() sochet API 调用它的本地 IP 地址和端口号。如果成功的话，这个调用将会提供一个 16 位的端口号和一个 32 位的 IP 地址。然后，客户端的计算机就会将这一 IP 地址通过一个位操作 XOR 及一个连接服务器与客户端都允许的 4 位 magic 数值将 IP 地址改变。改变 IP 地址的过程必须要注意“边际错误”(corner case)，避免 NAT 设备盲目地扫描有效负载及转换 IP 地址。经过这个过程，就会建立一个下文中所定义的数据包的有效负载，并且连接到连接中继服务器，并发送该有效负载。

```
typedef struct {
    unsigned long port;
    unsigned char address[4];
} local_end_point_info_t;

local_end_point_info_t payload;
unsigned char magic[4] = { 0xde, 0xad, 0xbe, 0xef };
struct sockaddr_in sin;
int server_tcp_conn_sock;

// allocate and bind server_tcp_conn_sock ...
int sinlen = sizeof(sin);
getsockname(server_tcp_conn_sock,(struct sockaddr*)&sin, &sinlen);
payload.port = sin.sin_port;
// XORing each byte obfuscates IP addresses in order to thwart
// NAT devices which replace IP addresses
// found in packet payloads.
payload.address[0] = BYTE0(sin.sin_addr) ^ magic[0];
payload.address[1] = BYTE1(sin.sin_addr) ^ magic[1];
payload.address[2] = BYTE2(sin.sin_addr) ^ magic[2];
payload.address[3] = BYTE3(sin.sin_addr) ^ magic[3];
```

一旦中继服务器接收到客户端的连接，中继服务器会知道客户端的公共地址和端口号（通过一次或几次 NAT 转换）。并且从数据包的有效负载中提取出客户端计算机的私有地址和端口号。

```
typedef struct {
    unsigned long local_port;
    unsigned char local_address[4];
    unsigned long remote_port;
    unsigned char remote_address[4];
} end_point_info_t;

end_point_info_t    end_point;
struct sockaddr_in  sin;
socklen_t           sinlen;
int                 server_sock;
local_end_point_info_t payload;
```

Download at www.Pin5i.Com

```
//...
sinlen = sizeof(sin);
accept(server_sock, (struct sockaddr*)&sin, &sinlen);
end_point.remote_port = sin.port;
end_point.remote_address[0] = BYTE0(sin.sin_addr);
//...
point_point.remote_address[3] = BYTE3(sin.sin_addr);
GetPayload(server_sock, payload, sizeof(end_point_info_t));
end_point.local_port = payload.port;
end_point.local_address[0] = payload.address[0] ^ magic[0];
//...
end_point.local_address[3] = payload.address[3] ^ magic[3];
```

在这一步中，尽管不一定需要同步，但是两台客户端计算机都要在连接中继服务器上注册它们的本地及远程地址。

2. 启动阶段

假设节点 A 想要建立一个与节点 B 的 P2P 连接。在启动阶段，A 就需要向连接中继服务器 C 发送一个连接请求，说明它需要和 B 建立连接。作为对这个请求的应答，C 就会发回给 A 关于 B 的终端信息（例如，C 所观察到的公共地址和端口号以及 B 所报告的本地地址和端口号）。同样的，C 也会把 A 的终端信息发给 B。另外，连接中继还会生成一个唯一的记号（一个序列号、时间表及简单的随机数字）并将这个记号与其他信息一起发给 A 和 B。在下一个连接阶段，A 和 B 将会用这个记号来证明它们对对方连接需求的真实性。尽管很少发生，但是缺少真实性认证的过时连接需求所产生的伪造连接也会发生。另外，如果两个分别在不同 NAT 内的节点使用了同样的私有地址空间，也可能会导致错误连接。为了使连接更为有效和可信，需要对记号进行数字签名。在这里介绍一种使用公共/私有密钥认证方案来对记号进行签名的解决方案。连接服务器可以为所有的客户端计算机计算公共/私有密钥对，包括 A 和 B。当连接服务器回答 A 的启动需求时，同时发送一个私有密钥签名语句。例如，连接服务器可以建立一个语句“对在 172.168.5.23 上的客户（A）向在 155.55.80.5 上的客户（B）发送的连接请求（ID=12345），在 68.238.128.12 上的服务器对 A 和 B 发送记号 0x3E14A901”。然后服务器 C 会利用无序算法 SHA-1，为该语句计算一个无序码摘要，并通过私钥对该无序码进行数字签名。C 会将语句和签署过的无序码发送给 A 和 B。整个签名允许客户端通过计算它们自己的无序码并将其于 C 公钥解密后的无序码进行比较后完成。

3. 连接阶段

连接阶段在 A 和 B 都从 C 接收到终端信息和记号后开始。A 和 B 将从它们收到的语句中提取记号的值。然后它们就可以按与 C 一样的方法来重新计算一个无序摘要以核实真实性，用 C 的公钥来对无序签名进行解密，最后将解密所得的无序码和重新计算得出的无序码进行比较。如果这两个无序码匹配，从 C 得到的信息就是真实的。两个客户端都会记录 32 位的记号值 0x3E14A901。

接下来，A 会建立一个 TCP 的 socket，对与 C 连接的端口进行监听。然后 A 会分配另外两个 TCP socket，仍然绑定在同一个连接在 C 的端口上，分别负责向 B 的私有本地终端（192.168.0.10：13532）和 B 的公共终端（155.55.80.5：8940）发送企图连接。B 会重复同样

Download at www.Pin5i.Com

的步骤。它也会建立一个 TCP socket，监听与 C 连接的端口。并且从同一个 B 连接到 C 的端口号用 TCP socket 向 A 的公共与私有本地终端发送 TCP 连接请求。发送公共与本地两个连接企图的原因是，因为 A 和 B 可能会在同一个 NAT 内。在这种情况下，NAT 将不能识别外发连接请求实际上是请求回到本地的私有网络中。A 和 B 从 C 接受到的连接请求和记号码是以验证真实性为目的。节点 A 和节点 B 同时试着连接和等着被连接。第一个完成真实性验证阶段（参见真实性验证阶段）的连接标志着连接阶段的结束。这样发送连接的节点就可以停止发送企图，正在等待被连接的节点也可以停止等待。如果在 A 和 B 之间的 NAT 设备是一台运行得很好的 NAT 设备，完成几次 TCP 连接企图之后，客户端 A 和 B 就可以被连接上了。

我们刚刚介绍的技术需要将多重 TCP socket 绑定到同一个本地端口上。不止这些，它还需要将接受 TCP 连接请求和发送 TCP 连接请求绑定在同一个端口上。尽管没有一个关于编写 TCP socket 的常规方法，但是所有这些操作，大多数操作系统的 socket API 都能够支持，因为一个由协议、本地 IP、本地端口、远程 IP、远程端口组成的 5 序列能够确定一个 socket 的唯一性。这就意味着只要目的地的 IP 地址和端口不同，socket 编程就允许在本地端口上绑定多重 TCP socket。

为了使多重 TCP socket 能够分享同一个端口，应用程序必须要建立一个 TCP socket 选项 SO_REUSEADDR 或者 SO_REUSEPORT。应用程序可以通过 setsockopt() API 调用设置 socket 选项。当配置完 socket 选项后，节点 A 和节点 B 通过调用一个 bind()socket API 的函数可以把 socket 绑在共享端口上。注意如果应用程序并没有将 socket 选项设置的很好，御定操作有可能会失败并回到一个“地址已被占用”的错误上。

4. 真实性认证阶段

为了防止客户端接收到一个过时连接请求或者伪造的连接请求，连接请求需要被验证真实性。A 和 B 都会在它们的连接请求中提供一个由 C 分发的唯一的记号。在接收一个连接请求的时候，计算机也会通过比较接收到的记号和连接中继提供的记号来校验这个请求是否来自于一个过时的进程。如果这两个记号相匹配，企图连接就会被接受。

7.5.4　应用方面

这种在多个 NAT 内的家庭 PC 之间建立 P2P 的 TCP 连接的技巧，在游戏领域中会有很广阔的应用。首先，可以直接应用于大量的 P2P 游戏中。很多这样的游戏都是在小部分玩家的 PC 之间建立 P2P TCP 连接的休闲游戏。这些游戏可以通过该技巧来提高连接性。第二，对于其他类型的在线游戏，例如，基于客户端-服务器端架构的游戏，我们介绍的技巧可以帮助在客户端计算机之间建立随选网络连接作为服务器连接的辅助。游戏应用也可以利用这种 P2P 连接的优势来传输一些不需要与服务器同步的即时信息。这类信息可能包括玩家的语音、从某些设备上得来的即时视频（例如：网络摄像头）以及在线聊天文字等。如果没有 P2P 连接的帮助，服务器必须转接每个客户端的每一个信息片断，这会消耗昂贵的服务器资源。

7.5.5　局限性

我们所谈到的技巧只会在锥形 NAT[Rosenberg03]中适用。在一个锥形的 NAT 中，NAT

Download at www.Pin5i.Com

设备会一直将一对内部私有 IP 地址和端口号转换为一对外部 IP 地址和端口号，并且忽略目的地的地址和端口号。不幸的是，并不是每一台 NAT 设备都是锥形 NAT。与锥形 NAT 相反，对称型NAT并不会持续的将内部计算机IP地址和端口号转换成唯一的外部IP地址和端口号。

下面以图 7.5.2 举一个例子，客户端计算机 A，内部 IP 为 10.0.1.5，在全局 IP 地址为 172.168.5.23 的 NAT 设备 NA 内；另外一台计算机 B，内部 IP 为 192.168.0.10，在一个全局 IP 是 155.55.80.5 的 NAT 设备 NB 内。假设 A 想要连接到 B，首先它会连接到连接中继服务器 C，C 在 68.238.128.12 上，端口号是 3478。假设 A 的 socket 绑定到一个本地端口号 23691 上。为了外发请求的接收，NAT NA 将会找到一个闲置端口号 4583，并在它的 NAT 表中加入一个转换入口。对于每一个向外发送的源 IP 地址为 10.0.1.5 和端口号 23691 的数据包，NAT NA 会忽略目的地，并总是将这个源地址及端口对转换成 172.168.5.23 和 4583；这是在 NAT NA 是一个锥形 NAT 的情况下。与此相反，如果 NAT NA 是一个堆成的 NAT，它将会为每一个向不同目标地址的外发的请求分配一个端口号。这样，在连接到 C 后，向客户端 B 的连接请求都会被 NAT NA 自动转换为另外一个端口号，而不是锥形 NAT 中的 4583。

要使我们所介绍的这种技巧能够正常工作，NAT NA 和 NAT NB 必须都是锥形 NAT。不同的研究表明，大部分的商业 NAT 设备都是锥形 NAT 兼容的[Ford05]，并且有一种趋势就是对称型的 NAT 在 ISP 和 NAT 卖家中已经变得越来越少，因为它们总是支持最受欢迎的 P2P 应用程序。

7.5.6 结论

本节揭示了怎样在同为 NAT 内的游戏 PC 之间利用 hole punching 建立 TCP 连接。我们所谈到的技巧可以增强 P2P 游戏的连接性。对于服务器-客户端架构的游戏来说，能够增加额外的连接通道，如果应用得好的话，很有可能增加在线游戏的游戏性。

7.5.7 参考文献

[Biggadike05] Biggadike, A., D. Ferullo, G. Wilson, and A. Perrig. "NATBLASTER: Establishing TCP Connections Between Hosts Behind NATs." ACM SIGCOMM Asia Workshop, Beijing, China, April 2005.

[Eppinger05] J. Eppinger, "TCP Connections for P2P Apps. A Software Approach to Solving the NAT Problem." Technical Report CMU-ISRI-05-104, Carnegie Mellon University, January 2005.

[Ford05] Ford, B., P. Srisuresh, and D. Kegel, "Peer-to-Peer Communication Across Network Address Translator." Usenix Annual Report, February 2005.

[Guha05] Guha, S., and P. Francis, "Simple Traversal of UDP Through NATs and TCP too (STUNT)." 2005. Available online at *http://nutss.gforge.cis.cornell.edu*.

[Rosenberg 03] Rosenberg, J., J. Weinberger, C. Huitema, and R. Mahy, "RFC 3489 - STUN - Simple Traversal of User Datagram Protocol (UDP) Through Network Address Translators (NATs)." 2003. Available online at *http://www.faqs.org rfcs/rfc3489.html*.

Download at www.Pin5i.Com